

vLLM 学习手册

- Validated vLLM: b23bd73f540175f9e117eaae5029cd7d8df63964
- Upstream committed: 2026-07-20T15:32:54+00:00
- Validated: 2026-08-25T07:44:40Z
- Latest candidate: 5e379a361e3ea8bb82b7efd768c36f39a0cf32fd
- Candidate lag: 1463 commits

 Deploy vLLM learning docs passing

 Check vLLM upstream failing

site jwzheng96.github.io/vllm-learning-book

vllm b23bd73 (2026-07-20)

一份写给大模型推理工程入门者的源码教程。 **64 章 · 30K+ 行**，从 PagedAttention 论文到 384 卡 H100 / 昇腾 910B 生产部署、端到端 profiling 与 Mooncake 分布式 KV 存储，覆盖整条链路。每章都用可刷新语义锚点对照锁定 commit 的 vLLM 源码，可以“读笔记 ↔ 跳源码”无缝切换。

☒ 在线阅读：jwzheng96.github.io/vllm-learning-book

这份手册解决什么问题

如果你正在做下面这些事，这是为你写的：

- 系统补课**：想把 vLLM 的核心机制啃透，不再只记"PagedAttention 解决了什么"这种结论。
- 业务接入**：要上线 LLM 推理服务，要选 v0/v1、调度策略、量化方案、部署架构。
- 性能优化**：TTFT/TPOT 不达标，需要从架构层定位到内核层逐级排查。
- 底层贡献**：想给 vLLM 提 PR，先得知道 scheduler / kv manager / attention backend 怎么咬合。

它**不适合**：完全没接触过 LLM 推理 → 先看 01-overview/00-prerequisites.md 把前置概念铺平；纯 prompt engineer 不碰服务侧 → 这本太工程。

怎么用这份资料

两种打开方式：

方式	入口	适合
Markdown 直接读	本 README.md → 按章节文件名跳转	IDE 内阅读、对照源码、想在 GitHub 上读
HTML 在线版	python3 build_html.py 后开 _site/index.html	想要侧栏 + 全文搜索 + Mermaid 渲染 + 暗色主题 + 阅读时间提示

所有跨章链接、内嵌 Mermaid、代码块都在两种模式下都能用。HTML 版额外有 lunr.js 全文搜索和阅读时间估算。

源码版本、语义锚点、影响报告和人工复核的完整流程见 docs/source-sync.md。这里的“已验证”是 fail-closed 语义门禁：锁定 SHA 与子模块一致、64 章 inventory 完整、源码锚点可解析、没有 unmanaged source line，而且 content-review.toml 的每一章都在该 SHA 上完成 source / command / metric / diagram review。PR 与 Pages 使用 validate --profile full --require-committed；每周工作流只创建候选 PR，不会自动合并或发布。

文中任何硬件验证徽章都必须对应可索引、可复现的运行记录（锁定 commit、硬件、命令和结果）；只有静态源码复核时，不得标注为 GPU 已验证。手动 GPU workflow 见 .github/workflows/gpu-validation.yml：它只在预置 vLLM / driver 的 self-hosted NVIDIA runner 上运行 scripts/gpu-validation.sh，并归档脱敏证据；普通 CI 不安装或假装拥有 GPU toolchain。

每章统一的结构：

谁该读这一篇？ ... 前置阅读： ... 耗时： N 分钟 学完能： ...

正文（含 mermaid / 表格 / 代码引用）

小结 ## 自检（3-5 题，自答） ## 下一步（跳转推荐）
--

按这个节奏走，刷完整本约 25-35 小时（不含动手实验）。

学习路径

flowchart TB Start[选择你的目标] --> Q[30 分钟理解] Start --> S[源码主线] Start --> P[工业实战] Start --> I[面试冲刺] Q --> Q1[前置概念 → vLLM 是什么 → 架构 → 首个 API 服务] S --> S1[入口 → 输入 → 调度 / KV → Runner / Attention → Sampling → 输出] P --> P1[环境 → 基准 → 调优 → 部署 / SLO → 安全 / 升级 → Capstone] I --> I1[高频题 → 计算题 → 系统设计 → 排障 → 模拟面试] classDef phase fill:#eff5ff,stroke:#2563eb,color:#1a1f29; classDef topic fill:#f7f8fa,stroke:#5b6573,color:#1a1f29; class Q,S,P,I phase; class Start,Q1,S1,P1,I1 topic;

四条路径可以独立走，也可以从“30 分钟理解”起步后再分流。

30 分钟理解

适合第一次接触 vLLM、需要快速建立全局心智模型的人。

前置知识（按需跳读） → vLLM 是什么 → 整体架构 → 启动 OpenAI-Compatible API

源码主线

适合准备读代码、改代码或定位引擎问题的人。顺序刻意沿一次请求的数据流展开。

入口与主循环 → 输入与 Tokenization → Scheduler → KV Cache → Model Runner → Attention → Sampling → 输出与 Streaming

工业实战

适合要把服务从“能跑”推进到“可量化、可调优、可上线、可回滚”的工程团队。

环境搭建 → Benchmark 方法论 → 调优 Playbook → 部署架构 → 容量规划 → SLO 与可观测性 → H100/910B 大规模部署 → 端到端 Profiling → Mooncake 分布式 KV 存储 → 生产 Capstone

面试冲刺

适合用可计算、可追问、可评分的方式准备推理工程面试。

30 道三层回答 → 容量与故障练习 → 系统设计 → 五轮模拟面试

如果目标是完整掌握，仍建议按 01 → 09 顺序阅读，并在每章用锁定 commit 的语义源码锚点回到真实实现。

章节索引（带钩子）

每章后面一句话告诉你为什么要读。

1. 总览 • 01-overview/ — 6 章

- 00-prerequisites.md — [NEW](#) token / KV cache / TTFT • TPOT / TP • PP • DP 一次铺平，零基础起点。
- 01-what-is-vllm.md — vLLM 是什么，为什么快，靠哪三大武器。
- 02-architecture.md — 三层进程、四个核心数据结构、一步推理的完整数据流。
- 03-v0-vs-v1.md — V1 重构改了什么，为什么改。
- 04-project-structure.md — 1700+ 文件按模块分类导航（这章很长，按需查阅）。
- 05-process-and-ipc-internals.md — 从进程/地址空间讲到 ZMQ socket pattern、共享内存状态机与 384 卡分布式通信边界。

2. 核心概念 • 02-core-concepts/ — 5 章

- 01-paged-attention.md — 用 OS 虚拟内存类比讲 paged KV cache + block table。
- 02-continuous-batching.md — 为什么 vLLM 不用 static batching，迭代级调度的两个前提。
- 03-kv-cache-management.md — Block Pool / 引用计数 / preempt vs swap 策略。
- 04-prefix-caching.md — Merkle 链式 hash + extra_keys，多卡 / LoRA / 多模态怎么算。

- 05-chunked-prefill.md — 长 prompt 怎么切才不卡 decode，max-num-batched-tokens 怎么调。

3. 源码走读 • 03-code-walkthrough/ — 10 章

- 01-entry-points.md — LLM / AsyncLLM / EngineCore 的调用链。
- 02-scheduler.md — Scheduler.schedule() 完整走读，token budget + preemption。
- 02b-scheduling-policies.md — FCFS vs PRIORITY、优先级反演、抢占代价量化。
- 03-kv-cache-manager.md — allocate / free / hash 的代码级细节。
- 04-model-runner.md — execute_model：输入拼装 / forward / sampler。
- 05-attention-backends.md — FlashAttn / FlashInfer / Triton / MLA 怎么选。
- 06-cuda-kernels.md — PagedAttention v1/v2、RoPE、RMSNorm CUDA 实现。
- 07-model-architectures.md — MLA / Mamba / MoE / GQA 在源码层的差异。
- 08-input-processing-and-tokenization.md — 从 OpenAI 请求、chat template、tokenizer 到 EngineCoreRequest。
- 09-output-processing-and-streaming.md — 从 sampler / detokenizer 到 SSE、finish reason 与取消传播。

4. 优化 • 04-optimizations/ — 5 章

- 01-quantization.md — FP8 / INT4 / AWQ / GPTQ / Marlin 的选型矩阵。
- 02-speculative-decoding.md — EAGLE3 / MTP / PARD / DFlash / DSpark、动态 K 与自适应验证的源码导读。
- 03-cudagraph-and-compile.md — CUDA Graph + torch.compile 何时开 / 何时关 / 失败降级路径。
- 04-compilation-internals.md — CompilerManager / VllmBackend / 自定义 pass 深度。
- 05-roofline-and-arithmetic-intensity.md —  存算比定量推导：prefill 为何 compute-bound、decode 为何 memory-bound，Llama-70B 算一遍，batching 摊薄权重却摊不薄 KV。

5. 分布式 • 05-distributed/ — 5 章

- 01-tp-pp-ep.md — TP 切法 / PP 流水气泡 / EP expert 负载均衡。
- 02-disaggregated.md — Prefill/Decode 分离的数据流 + NIXL RDMA + 决策表。
- 03-expert-parallel-deep-dive.md — MoE AllToAll 6 个后端、EPLB、宽 EP 部署模式。
- 04-context-parallel.md — PCP / DCP 双维度长上下文切分；MLA 模型 a2a backend 省 NCCL。
- 05-large-scale-cluster-inference.md —  千卡/万卡实战：通信墙/故障墙/长尾墙、EP 同步长尾、AllToAll 撞网络、blast radius、6 类规模化故障 runbook。

6. 工程问答 • 06-interview/ — 4 章

- 01-common-questions.md — 30 个问题按结论、机制、取舍、验证 / 回滚和追问展开。
- 02-system-design.md — 需求优先的容量、架构、故障域、发布与成本推演。
- 03-capacity-and-troubleshooting-drills.md — 8 道带单位计算题 + 8 类证据优先故障题。
- 04-mock-interview-and-rubric.md — 概念 / 源码 / 计算 / 设计 / 事故五轮评分与项目叙事模板。

7. 实操 • 07-hands-on/ — 8 章

- 01-setup.md — uv 环境 / 预编译 vs 源码 / GPU 检查。
- 02-trace-a-request.md — debugger 跟一个请求从 HTTP 到 token。
- 03-mini-experiments.md — 5 个动手实验 (block_size / prefix hit / batching / 量化等)。
- 04-profiling-and-debugging.md — torch.profiler / NVTX / py-spy / 显存泄漏。

- 05-serve-openai-api.md — 从零启动、健康检查、鉴权、streaming 与失败证据。
- 06-benchmark-methodology.md — 固定变量、quality gate、goodput 与可复现 benchmark。
- 07-tuning-playbook.md — 从症状到单变量、可回滚调优实验。
- 08-production-capstone.md — 交付可运维、可扩容、可回滚的证据包。

8. 生产部署 • 08-production-deployment/ — 16 章

- 01-deployment-architectures.md — vLLM Production Stack / llm-d / AIBrix 三套参考栈对比。
- 02-smart-routing-and-load-balancing.md — prefix-cache aware / session sticky / 负载打分。
- 03-gateway-and-service-mesh.md — Istio + Gateway API Inference Extension + ExtProc。
- 04-autoscaling-and-capacity.md — KEDA / 容量公式 / 冷启动 / 优雅 drain。
- 05-slo-and-observability.md — TTFT/TPOT/p99 SLO + Prometheus + OTel。
- 06-reliability-and-failure-modes.md — 8 个失效模式与防护。
- 07-incident-playbook.md — 8 个真实故障 runbook。
- 08-monitoring-cookbook.md — 可直接抄走的 PromQL / 告警规则 YAML / Grafana dashboard 骨架。
- 09-vllm-doctor-skill.md — 把 06-07-08 章人工流程编成 agent 自动跑：7 阶段工作流 + 决策树 + 三级整改 + 离线 dry-run。
- 10-gpu-utilization-and-tail-latency.md — NEW 全链路性能诊断：GPU-Util 为何是谎言、MBU/MFU、带宽/利用率为何打不满、长尾 8 类根因与处置。
- 11-security-and-multi-tenancy.md — 威胁模型、auth、quota、artifact / LoRA trust、脱敏与 tenant isolation。
- 12-upgrades-rollbacks-and-compatibility.md — 兼容矩阵、golden、shadow / canary、drain 与 rollback。
- 13-384-h100-glm-deepseek-deployment.md — 48 节点/384×H100：GLM-5.2、DeepSeek-V4-Flash/Pro 的副本内并行、H100 验证边界、上线门禁与故障处置。
- 14-384-ascend-910b-glm-deepseek-deployment.md — 48 节点/384×910B：vLLM Ascend 软件栈、GLM-5.2 与 DeepSeek-V4 Flash/Pro 的 8/16/32 卡服务单元、HCCL、rank、Kubernetes 与发布 runbook。
- 15-end-to-end-latency-profiling-and-optimization.md — 客户端到 kernel 的延迟账本：metrics/OTel/Torch profiler、H100 Nsight/NCCL、910B Ascend PT/MS Service Profiler 与模型专项优化闭环。
- 16-mooncake-distributed-inference-storage.md — Mooncake P/D 直传、共享 DRAM/SSD KV 池、MultiConnector、RDMA、Prometheus、Kubernetes 与生产故障 Runbook。

9. 应用特性 • 09-advanced-features/ — 5 章

- 01-sampling-and-logits.md — 当前 sampling 顺序、backend、seed 边界、logprobs 与投机兼容性。
- 02-structured-output.md — auto / 显式后端、grammar compile / bitmask、fallback 与错误边界。
- 03-multimodal.md — 图像/视频/音频编码器、encoder cache、Qwen2-VL。
- 04-lora-serving.md — LoRAModelManager / Punica / 多 LoRA batching。
- 05-embedding-and-pooling.md — BGE / E5 / BGE-M3 / 复用 vLLM 引擎做 embedding。

vLLM 仓库地标速查

源码锚点：[vllm/v1/core/kv_cache_utils.py](#) • [hash_block_tokens](#)

想知道什么	去哪里看
用户怎么调用 vLLM	vllm/entrypoints/llm.py、vllm/entrypoints/openai/api_server.py
引擎主循环	vllm/v1/engine/core.py、vllm/v1/engine/llm_engine.py
调度器（决定本步跑哪些请求）	vllm/v1/core/sched/scheduler.py
KV cache 块管理	vllm/v1/core/kv_cache_manager.py、block_pool.py
Prefix caching hash	vllm/v1/core/kv_cache_utils.py (hash_block_tokens)
Model runner（前向）	vllm/v1/worker/gpu_model_runner.py
Attention 后端选择	vllm/v1/attention/backends/（FlashAttn / FlashInfer / Triton / MLA）
PagedAttention CUDA	csrc/attention/
模型实现	vllm/model_executor/models/（按当前 registry 核准架构支持）
张量并行 / 集合通信	vllm/distributed/parallel_state.py
量化	vllm/model_executor/layers/quantization/
投机解码	vllm/v1/spec_decode/
采样	vllm/v1/sample/、csrc/sampler.cu
KV transfer（disaggregated）	vllm/distributed/kv_transfer/、vllm/v1/kv_offload/

完整地图见 01-overview/04-project-structure.md。

学习方法（5 条铁律）

1. **概念落到代码。** 不看博客的二手解读，源码才是唯一真相。
2. **先看数据契约，再看内部。** 读 Scheduler 之前先看 SchedulerOutput 的字段；读 KV manager 之前先看 KVCacheBlocks 的形状。
3. **打 print 比读注释快。** 用 LLM("facebook/opt-125m").generate(...) 给 Scheduler、KVCacheManager 各加一句 print，跑一次就懂。
4. **三张图刻进脑子。** 请求生命周期、KV 物理↔逻辑映射、Scheduler 一步内的决策流。每张都自己画一遍。
5. **永远做对比。** 讲 vLLM 的优势，必须能讲清"HF Transformers 是怎么做的、为什么慢"。

工程理解自检清单

读完整套笔记后，下面每个问题应该能在 1-2 分钟内讲清，并指出对应源码位置：

- Paged KV cache、continuous batching 与 prefix caching 分别解决什么，代价是什么？
- KV block size 太大太小各有什么问题？如何在目标 backend / workload 上验证？

- Continuous batching 和 static batching 的本质区别？为什么 GPU 利用率提升？
- Prefix caching 的 hash 怎么算？怎么避免冲突？多模态怎么处理？
- Chunked prefill 解决了什么？如何核准目标版本默认值与模型限制？
- Tensor parallel 在 MLP 用 column → row 的原因？AllReduce 落在哪？
- Speculative decoding 的接受率怎么算？拒绝采样的数学推导写一遍。
- FP8 / INT8 / INT4 各自的精度损失主要发生在哪？
- V0 → V1 重构的三个最大改变是什么？为什么这么改？
- KV 不够时怎么处理？V1 默认 recompute 还是 swap，为什么？

每题都有专门展开，见 06-interview/01-common-questions.md。

必读资料

按阅读顺序：

1. **PagedAttention 论文** — Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention*, SOSP 2023。
先读这篇，再读代码。
2. **Continuous Batching 博客** — Anyscale, *How Continuous Batching Enables 23× Throughput*。
3. **vLLM 官方文档** — <https://docs.vllm.ai/>，重点看 design / kernel 章节。
4. **FlashAttention v1/v2/v3** — 理解 SRAM tiling 的关键。
5. **Speculative Decoding 论文** — Leviathan et al., 2023, *Fast Inference from Transformers via Speculative Decoding*。
6. **EAGLE3 / MTP / PARD / Suffix Decoding** — 分别理解 hidden-state drafter、原生多 token 头、并行 draft 与 workload-aware proposal。

构建与部署

把这份手册转成 HTML 网站、PDF、EPUB，或者部署到 GitHub Pages，看 DEPLOY.md。一键命令（脚本会自动用项目相对路径，不再依赖固定位置）：

```
python3 build_html.py    # → ../vllm-learning-html/ (含搜索 + 暗黑切换 + Mermaid + 阅读时间)
python3 build_pdf_epub.py # → ../vllm-learning-html/vllm-learning.pdf + .epub
./deploy_gh_pages.sh <repo-url>
```

如果你想把材料放别处，设环境变量 VLLM_LEARNING_SRC / VLLM_LEARNING_DST 即可。

自动排障 Skill：vllm-doctor

仓库内置了一个 Claude Code skill vllm-doctor，把第 06-07-08 章里散落的 incident playbook 编成 agent 可以自动跑的 7 阶段流程：环境探测 → 拉 Golden 3 指标 → 决策树路由 → 深度诊断 → 生成整改计划 → 显式审批后执行 mutation → 恢复验证与报告。缺少当前 metric、threshold 或证据时 fail closed，不把“没有 active route”误报为恢复。

安装到本地 Claude Code：

```
cp -r .claude/skills/vllm-doctor ~/.claude/skills/
```

触发：在 Claude Code 里输入 `/vllm-doctor`（前置必须 `export VLLM_NAMESPACE / PROM_URL / KUBECONFIG`）。

不连集群也想验证逻辑：

```
export VLLM_DOCTOR_FIXTURE=/path/to/golden3.json #跳过 Prometheus，直接喂 mock 数据
```

覆盖的 8 类事故：KV 抢占级联、NCCL hang、GPU OOM、客户端重试雪崩、prefix cache 命中率塌方、冷启动、输出质量异常、LoRA 适配器抖动。完整说明见 `.claude/skills/vllm-doctor/SKILL.md`。

贡献与扩展

发现 `file_path:line_number` 失效了？vLLM 主分支变化快，欢迎 PR 修正。

想新加一章？沿用每章统一的"章首导读 + 正文 + 小结/自检/下一步"模板（任意章可作范例）。

开始读 `01-overview/01-what-is-vllm.md`。或者从 `00-prerequisites.md` 铺前置。

00. 前置知识：进入 vLLM 之前要懂的概念

谁该读这一篇？ 第一次接触 LLM 推理工程的同学（比如刚学完 Transformer 论文的本科生）；或对 KV cache / batching / TP 等概念心里没底的工程师。后面 40 多章默认这里每个术语你都熟。

前置阅读： 无（起点章节）。

耗时： 约 30 分钟。每节末尾的"在 vLLM 里"指出该概念后续章节会怎么用到。

学完能：

1. 解释 LLM 自回归生成、KV cache 必要性、prefill vs decode 的根本差异。
2. 用 roofline 模型说清 decode 阶段为什么是 memory-bound、batching 为什么能拉吞吐。
3. 区分 DP / TP / PP，并指出哪种通信强依赖 NVLink。
4. 用 TTFT / TPOT / 吞吐三个指标解释推理服务"卡不卡"和"赚不赚"。

当前源码复核 (b23bd73)： 本章把 TTFT（请求到首个可见输出）、TPOT（除首 token 外的平均输出 token 间隔）与逐 token ITL 分开；CacheConfig.DEFAULT_BLOCK_SIZE 是 16，但最终 block size 可由平台或用户覆盖，不能把 16 当成所有后端的固定事实。

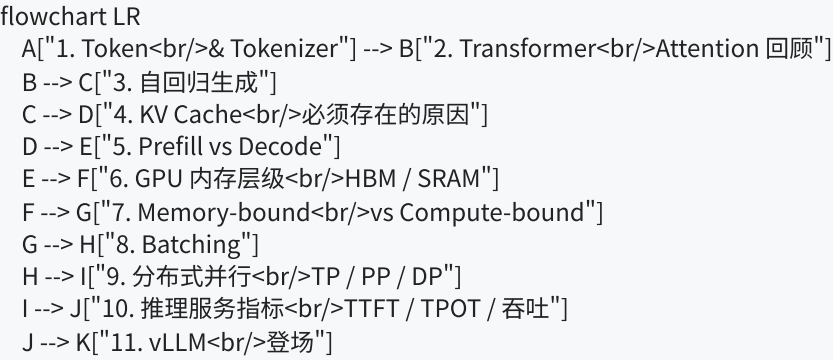
0. 给读者的心智地图

vLLM 解决的是这样一类问题：

一个训练好的大语言模型（如 Llama-3-70B），怎么在一台 / 一组 GPU 上同时服务尽可能多的用户、把每个用户的等待时间压到最低？

要看懂 vLLM 在做什么，先要把"训练好的模型如何生成下一个 token"这条链路里每一步的瓶颈都摸清楚。本章从 token 开始，一路推到为什么需要 PagedAttention。

学习路径图：



```
classDef base fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
classDef perf fill:#fef3c7,stroke:#b45309,color:#1a1f29;
classDef sys fill:#dcfce7,stroke:#15803d,color:#1a1f29;
class A,B,C,D,E base;
class F,G,H perf;
class I,J,K sys;
```

1. Token 与 Tokenizer

什么是 **token**? LLM 不直接处理字符串。它把文本切成离散的"小块"——*token*。一个 token 可能是：

- 一个常见单词: "the"、"machine"
- 一个子词: "learn" + "ing"、"un" + "happy"
- 一个 Unicode 字符（中文 / emoji 常见）

切分由 **tokenizer** 完成，背后通常是 BPE（Byte Pair Encoding）或 SentencePiece 算法。把文本拍成 token 序列后，每个 token 对应一个**整数 id**：

```
"Hello, world"
↓ tokenizer (Llama-3)
[15043, 11, 1917]  # 3 个 token id
```

词表大小（vocab size） 是 tokenizer 认识的全部 token 数。常见值：

模型	vocab size
GPT-2	50,257
Llama-3	128,256
Qwen3	151,936
DeepSeek-V3	129,280

一句话长度从字符数变成 token 数——这是 LLM 推理里所有"长度"指标的实际单位。**1 个英文单词 ≈ 1.3 token，1 个汉字 ≈ 2 token**（粗略经验值）。

在 vLLM 里：

- API 请求里的 prompt 会在进入引擎前被 tokenize 成 prompt_token_ids: list[int]

- 后续所有 batch 处理都在 token id 层面进行
- 用户的 max_tokens 参数限制生成的 token 数

2. Transformer 与 Attention 回顾

LLM 99% 是 **decoder-only Transformer**（GPT 家族）。一层 Transformer 干两件事：

1. **Self-Attention**：让当前 token 看到序列里的其他 token，决定"应该参考谁"
2. **MLP（前馈网络）**：对每个 token 单独做非线性变换

2.1 Attention 公式（先掌握）

$$\text{Attention}(Q, K, V) = \text{softmax}!\left(\frac{Q \cdot K^{\text{top}}}{\sqrt{d_k}}\right) V$$

直观理解：

- **Q (Query)**：当前 token "我想找谁"
- **K (Key)**：每个 token "我能被找到的特征"
- **V (Value)**：每个 token "我的具体内容"
- $Q \cdot K^{\text{top}}$ $\rightarrow$ 每个 token 与所有 token 的相似度矩阵
- softmax 把相似度变成概率（注意力权重）
- 权重乘 V $\rightarrow$ 加权求和的结果

每个 token 算自己的 Q、K、V：

$$Q_i = x_i W_Q, \quad K_i = x_i W_K, \quad V_i = x_i W_V$$

其中 x_i 是第 i 个 token 的输入向量。 W_Q, W_K, W_V 是模型权重，**对所有 token 共享**。

2.2 多头注意力

把 Q、K、V 在维度上切成 n_{heads} 份并行算（每份称为一个 head），结果拼回去。每个 head 关注不同的"语义子空间"。

```
hidden_size = 4096
n_heads    = 32
head_size  = 4096 / 32 = 128
```

2.3 因果 mask

LLM 是 **自回归** 模型——第 i 个 token 只能看到 0.. i 这些 token，不能"看到未来"。实现上就是在 $Q \cdot K^T$ 后加一个上三角负无穷 mask：

```
mask[i, j] = 0    if j ≤ i (允许)
            -inf  if j > i (屏蔽)
```

softmax 后未来位置权重为 0。

在 vLLM 里：

- 每层 Attention 走 PagedAttention（详见 02-core-concepts/01-paged-attention.md）
- 因果 mask 是 attention backend 的默认行为，不用每次手动构造

3. 自回归生成：从输入到输出

LLM 生成是逐 token 的串行过程：

```
输入 prompt: "今天天气"  
  ↓ tokenize → [今天, 天气]  
模型 forward → 预测下一个 token 的概率分布  
  ↓ 采样 (greedy / top-k / top-p)  
得到 token: "真"  
  ↓ 把 "真" 拼到输入末尾  
新输入: "今天天气真"  
  ↓ 再 forward  
预测下一个 token: "好"  
  ↓ ...  
  
直到生成 <EOS> (结束符) 或达到 max_tokens
```

这里有个核心观察：**每生成 1 个新 token，整个模型要 forward 一次**。如果生成 100 个 token，模型要跑 100 次。

```
sequenceDiagram
    participant U as User
    participant L as LLM (一次 forward)
    U->>L: 今天天气
    L-->>U: token = "真"
    U->>L: 今天天气真
    L-->>U: token = "好"
    U->>L: 今天天气真好
    L-->>U: token = ", "
    U->>L: 今天天气真好,
    L-->>U: token = "..."
    Note over U,L: 直到 EOS 或 max_tokens
```

在 vLLM 里：

- 这条循环在 EngineCore.run_busy_loop() 里
- 每生成 1 个 token = 1 个 **step**（也叫 1 个 iteration）

4. KV Cache：必须存在的原因

朴素实现的灾难：每生成一个新 token，前面所有 token 的 K、V 都要**重新计算一遍**。

生成第 5 个 token：
输入 = [tok_0, tok_1, tok_2, tok_3, tok_4]
对每个位置算 Q_i, K_i, V_i ← 重新算了 5 遍
attention(Q_4, K_0..4, V_0..4)
采样 → tok_5

生成第 6 个 token：
输入 = [tok_0, tok_1, tok_2, tok_3, tok_4, tok_5]
对每个位置算 Q_i, K_i, V_i ← 又重新算了 6 遍！前 5 个之前已算过
attention(Q_5, K_0..5, V_0..5)

关键观察：K_i、V_i 只依赖 token i 的输入，**算一次就固定了**。所以——把它们**缓存起来**，下次直接读。这就是 **KV cache**（Key-Value cache）。

```
flowchart LR
    subgraph Step5["生成第 5 个 token"]
        T5["tokens 0..4"] --> KV5["算 K_0..4, V_0..4<br/>(全新)"]
        KV5 --> Cache5["KV Cache"]
        KV5 --> A5["Attention"]
    end
    subgraph Step6["生成第 6 个 token"]
        T6["tokens 0..5"]
        T6 --> K6["只算 K_5, V_5<br/>(新增 1 个)"]
        Cache5 -.读取 K_0..4, V_0..4.-> A6["Attention"]
        K6 --> Cache6["KV Cache (扩 1 行)"]
        K6 --> A6
    end
    classDef step fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
    classDef cache fill:#fef3c7,stroke:#b45309,color:#1a1f29;
    class T5,T6,KV5,K6,A5,A6 step;
    class Cache5,Cache6 cache;
```

4.1 KV cache 多大？

一个 token 在一个 attention 层占的 KV 大小（每 token 都要存 K 和 V 两份）：

$$2 \times \text{hidden_size} \times \text{dtype_bytes}$$

整个模型 N 层累加：

$$N \times 2 \times \text{hidden_size} \times \text{dtype_bytes}$$

Llama-3-70B 实例： $N = 80$ 、hidden = 8192、BF16（2 字节）：

$$80 \times 2 \times 8192 \times 2 = 2,621,440 \text{ 字节} \approx 2.5 \text{ MB}$$

一个 4K token 的请求 → 10 GB KV！一张 80GB H100 也装不下几个并发请求。

这就是 vLLM 要解决的核心矛盾：KV cache 太大、太碎，怎么用得起来。

在 vLLM 里：

- KV cache 由 BlockPool（vllm/v1/core/block_pool.py）管
- 每个请求一张 BlockTable 把逻辑序列映射到物理 block
- 详见 02-core-concepts/01-paged-attention.md

5. Prefill vs Decode：LLM 推理的两个阶段

一个请求的生命周期分两段，性能特性完全不同：

Prefill（填充）

处理用户输入的 prompt：

```
prompt = [t_0, t_1, ..., t_{n-1}]    # n 个 token
      ↓ 一次性 forward
对 prompt 里每个 token 算 Q/K/V → 写 KV cache
对最后一个 token 采样 → 第一个生成 token
```

- 一次 forward 处理整个 prompt（N 个 token 并行）
- 算力密集：Q、K、V 投影 + $N \times N$ attention + MLP，都是大矩阵乘
- 算术强度高 → compute-bound

Decode（解码）

逐个生成新 token：

```
[t_0, ..., t_{n-1}, t_{n_new}] 每步只新增 1 个 token
      ↓ forward
对新 token 算 Q/K/V
attention(Q_new, K_0..n, V_0..n) ← 读全部 KV cache
采样 → 下一个 token
```

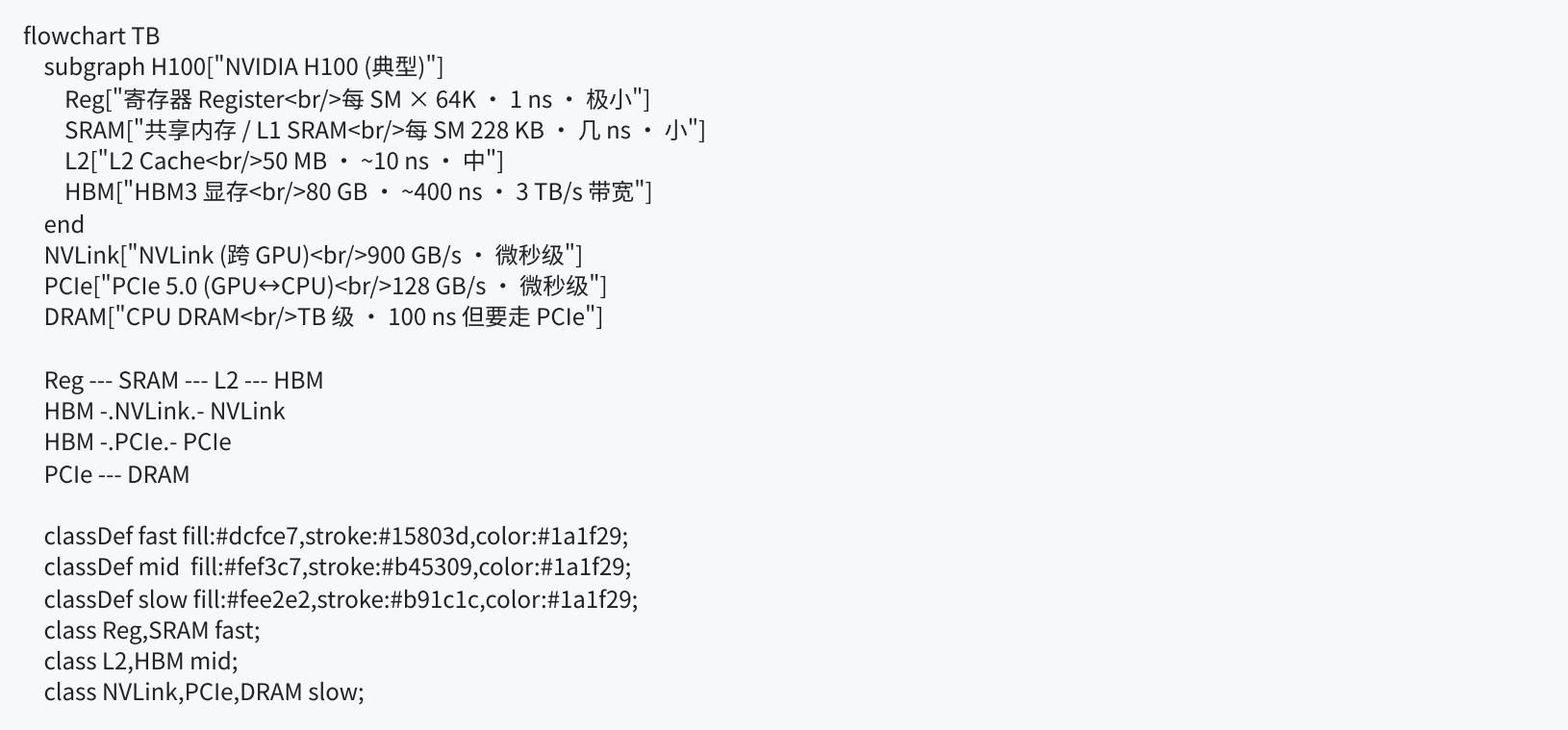
- 每步 forward 只处理 1 个 token
- 但仍要把整个模型的权重读一遍
- 算术强度低 → memory-bound（详见第 7 节）

维度	Prefill	Decode
一次输入 token 数	N（prompt 长度）	1
FLOPs	大（ $O(N)$ 或 $O(N^2)$ for attention）	小（ $O(1)$ per token）
内存读	模型权重 + 写 N 个 KV	模型权重 + 读 N 个 KV
主导瓶颈	算力（compute-bound）	带宽（memory-bound）
一个请求里有几次	1 次	output_tokens 次

在 vLLM 里：

- V1 把这两个阶段混在同一个 **forward** 里跑（continuous batching + chunked prefill），详见 02-core-concepts/02,05
- 一个 step 内可能既有几个请求在 prefill、又有十几个请求在 decode

6. GPU 内存层级速览



关键数字（H100 量级）：

存储	容量	带宽	延迟
寄存器	KB 量级 / SM	极快	~1 ns
L1 SRAM	228 KB / SM	~30 TB/s	~5 ns
L2	50 MB	~5 TB/s	~50 ns
HBM 显存	80 GB	3 TB/s	~400 ns
NVLink	—	900 GB/s（卡间）	μs
PCIe	—	128 GB/s（CPU↔GPU）	μs
CPU DRAM	TB 级	100 GB/s（CPU 内部）	100 ns 但要绕 PCIe

两条规律：

1. 越靠近计算单元（寄存器）越快但越小——这是所有 GPU 优化的物理基础
2. HBM 带宽是 LLM 推理 decode 阶段的核心瓶颈——3 TB/s 看似很大，但读一遍 70B 权重（140 GB FP16）就要 47 ms。所以每生成 1 token 的硬件极限就是 ~47 ms

在 vLLM 里：

- --gpu-memory-utilization 0.9 指 HBM 用 90%
- KV cache 占的就是这 90% 里扣掉模型权重和 activation 的剩余空间
- --kv-cache-dtype fp8 让 KV 用一半 HBM，相当于翻倍并发量

7. Memory-bound vs Compute-bound（roofline 模型）

衡量一个计算的**算术强度**（Arithmetic Intensity）：

AI = FLOPs / Bytes

↑

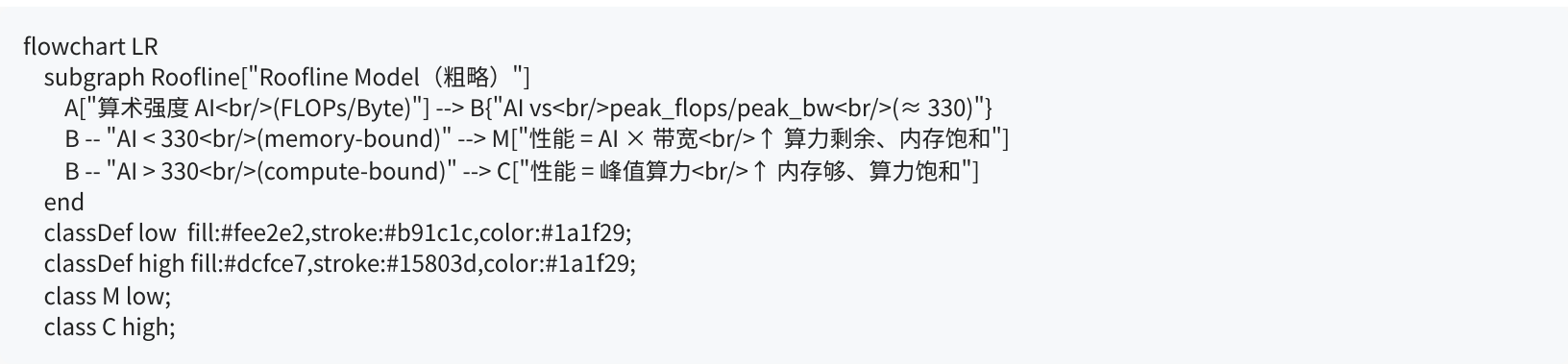
总浮点运算

↑

从 HBM 读 / 写的字节数

GPU 有两条硬上限：

- **峰值算力**：H100 FP16 ~1000 TFLOPS
- **峰值带宽**：H100 HBM3 ~3 TB/s



应用到 LLM：

阶段	算术强度	主瓶颈	加速思路
Prefill（N 个 token 并行）	高（~N）	算力	不容易再加速，已经接近 100% 算力
Decode（1 个 token）	低（~1）	HBM 带宽	减少要读的数据： batching （多个请求共享模型权重读取）、 量化 （FP8 / INT4）、 KV cache 压缩 （MLA）

为什么 batching 能让 decode 大幅提速？ 因为权重只读一次就服务整批请求。batch=1 vs batch=32 时，HBM 读权重的开销是同一份，但 GPU 算力被 32 倍利用。

在 vLLM 里： 这是 continuous batching 之所以能把吞吐拉 24× 的物理基础。

这里是粗略版。想看**定量推导**（算出拐点、证明"Linear 层存算比 ≈ token 数"、用 Llama-70B 算 token/s 上限、以及"batching 摊薄权重却摊不薄 KV"这个生产关键点），见 04-optimizations/05-roofline-and-arithmetic-intensity.md。

8. Batching：把多个请求挤进一次 forward

串行处理：

请求 A: [forward][forward][forward]...	← 单个请求每步只用 1/N 的 GPU
请求 B:	...
请求 C:	...

批处理：

一次 forward 同时处理多个请求的 1 个 token：
batched_input = [A_tok, B_tok, C_tok, D_tok, ...] shape [batch, hidden]

直觉：每生成 1 个 token 都要把整个模型从 HBM 读一遍（~140 GB）。

- 串行 1 个请求：读 140 GB → 算 1 个 token
- batch=32：读 140 GB → 算 32 个 token

吞吐 $32\times$ ，延迟几乎不变（因为是 memory-bound）。这是 LLM 推理服务能做的最重要的事。

但批处理有两个变种：

8.1 Static Batching（朴素）

一批请求**一起进、一起出**。最慢的决定整组——参考 02-core-concepts/02-continuous-batching.md 的 gantt 图。

8.2 Continuous Batching（vLLM 默认）

每生成 1 个 token 都重新组 batch：完成的请求立刻退出，新请求立刻进入。GPU 永远满载。

关键词：

- **Throughput（吞吐）**：单位时间服务的总 token 数（tokens/s）
- **Latency（延迟）**：单个用户感受到的等待时间

Batching 提高吞吐，**但单请求延迟可能略增**（要等 batch 凑够）。这是所有推理服务设计的核心 trade-off。

9. 分布式并行速览

一个模型一张 GPU 装不下，或者算太慢，怎么办？三种切法：

9.1 数据并行（Data Parallel, DP）

复制模型到多卡，每卡服务不同请求。简单粗暴。

- 训练里也叫 DP；推理里就是"多副本"。

9.2 张量并行 (Tensor Parallel, TP)

把一层的权重切到多卡，每步合并：

- MLP 列切 → 行切，每层末尾一次 AllReduce
- Attention QKV 列切 → output 行切

特点：通信极频繁（每层 2 次），强依赖 NVLink。

9.3 流水并行 (Pipeline Parallel, PP)

不同层放不同卡，前一段算完把 hidden 传给下一段。

- 适合跨机器（通信少，每段边界一次）
- 有 bubble（流水启动期的空闲）

flowchart TB

```
subgraph DP["DP · 数据并行 (模型复制)"]
    DP1["卡 0<br/>完整模型"] --- DP2["卡 1<br/>完整模型"]
end
subgraph TP["TP · 张量并行 (切一层)"]
    TP1["卡 0<br/>Layer.left"] --- TP2["卡 1<br/>Layer.right"]
    TP1 -. AllReduce .-> TP2
end
subgraph PP["PP · 流水并行 (切若干层)"]
    PP1["卡 0<br/>Layer 0-15"] --> PP2["卡 1<br/>Layer 16-31"]
end
```

```
classDef a fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
classDef b fill:#fef3c7,stroke:#b45309,color:#1a1f29;
classDef c fill:#dcfce7,stroke:#15803d,color:#1a1f29;
class DP1,DP2 a;
class TP1,TP2 b;
class PP1,PP2 c;
```

实际配置：同机器 8 卡几乎都用 TP=8；跨机器再叠 PP。详见 05-distributed/01-tp-pp-ep.md。

10. 推理服务的核心指标

LLM 推理服务的"性能"不只是一个数字。至少 4 个：

TTFT — Time To First Token

从请求发出到**第一个 token 返回**的时间。用户感受"卡不卡"主要看这个。

- 主要被 prefill 时长决定
- 服务于"我点了发送按钮，多久看到第一个字"

TPOT — Time Per Output Token

首 token 出来后，**每个后续 token** 的平均间隔。决定"打字速度感"。

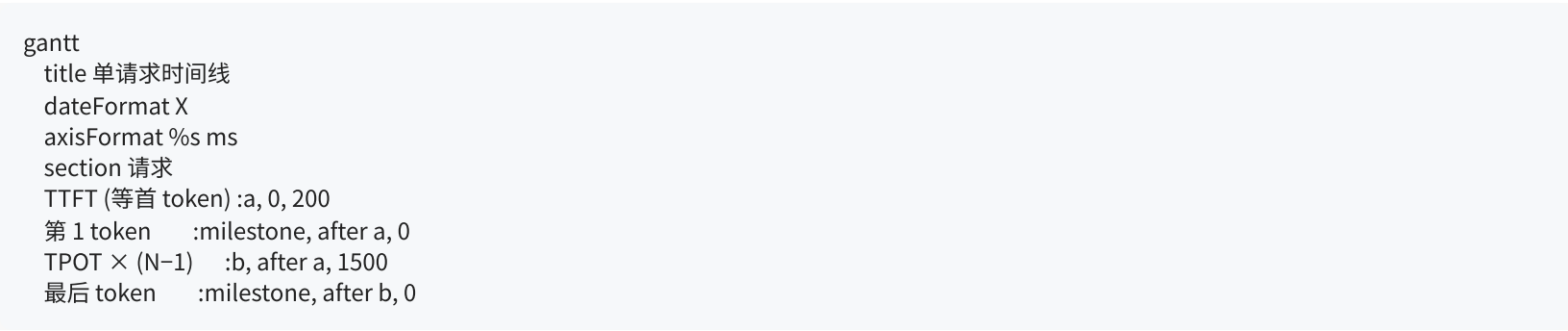
- 主要被 decode 时长决定
- 也叫 ITL（Inter-Token Latency）

TTLT — Time To Last Token

总响应时间 $\approx$ TTFT + (output_tokens - 1) $\times$ TPOT。SLA 的总时长指标。

Throughput — 吞吐

- **每秒生成 token 数**（tokens/s）
- **每秒处理请求数**（req/s）



指标 vs 关注的人：

指标	谁关心
TTFT	用户体验 / 产品
TPOT	用户体验 / 产品
吞吐（tokens/s）	平台容量规划
并发数（in-flight 请求）	平台容量
GPU util	工程师 / 老板（花的钱有没有用满）

更详细：08-production-deployment/05-slo-and-observability.md。

11. vLLM 登场：把上面拼起来

到这里你应该看清楚问题的形状了：

- LLM 生成是**自回归**的（一次 forward = 1 token）
- 每次 forward 要读整个模型权重（**memory-bound**）
- 单请求 GPU 用不满 $\rightarrow$ 必须 **batching**
- 每请求要存 **KV cache** $\rightarrow$ 想 batching 必须把 KV 管好
- 朴素 KV cache 分配（每请求预留 max_len 连续显存） $\rightarrow$ 浪费 60-80%

- 模型大了一张卡装不下 → **TP / PP**

vLLM 的回答：

问题	vLLM 的解
KV cache 浪费	PagedAttention —— 像 OS 虚拟内存一样分页管理
静态 batching 让 GPU 空转	Continuous batching —— iteration-level 调度
长 prefill 卡死短 decode	Chunked prefill —— 把长 prefill 切片、与 decode 混跑
相同前缀重复算	Prefix caching —— 跨请求 / 跨 step 复用 KV block
训练栈不够快	自己写 / 集成 FlashAttention 、 CUDA Graph 、 torch.compile
单卡装不下	模型并行 (TP / PP / EP)，架构上把 Scheduler 与 Worker 解耦

后面的章节就是把这张表里的每个解法**展开到代码层面**。

12. 常见术语速查（A-Z）

读后面章节遇到不熟的词回这里查。

术语	含义
AllReduce	集合通信原语，把多卡的某张量求和后广播回所有卡。TP 每层用 1-2 次。
AllToAll	集合通信，每卡把不同片段发给不同卡。EP（专家并行）下 dispatch / combine 用。
Attention backend	vLLM 中"具体怎么算注意力"的可插拔后端：FlashAttention / FlashInfer / Triton / MLA / ...
BF16 / FP16 / FP8 / INT8 / INT4	浮点 / 整数精度。BF16 $\approx$ FP16 但动态范围更大；FP8 是 H100+ 的新格式，约一半显存与算力。
Block / KV block	PagedAttention 把 KV cache 切成固定大小的小块。通用 fallback 是 16 token / block，最终值由平台、模型和用户配置共同决定。
Block table	每个请求一张表，把"第几个 block"映射到"物理 block id"。类比 OS 页表。
Continuous batching	iteration-level 动态批处理。每生成一个 token 就重新组 batch。
CUDA Graph	把一系列 CUDA kernel 调用录制成静态图，runtime 一次性提交，省 launch 开销。
Decode	自回归生成阶段，每步只算 1 个新 token。memory-bound。
DP / TP / PP / EP / CP	数据 / 张量 / 流水 / 专家 / context parallel。详见 05-distributed/。
EAGLE / MTP / Medusa / Ngram	几种投机解码（speculative decoding）方法。
EngineCore	vLLM V1 里的调度引擎进程。运行 Scheduler + Executor。
Executor	EngineCore 与 Worker 之间的中间层。有 multiproc / Ray / uniproc 三种实现。
EP（Expert Parallel）	MoE 模型里把不同 expert 分到不同卡。
EPP（Endpoint Picker）	Gateway API Inference Extension 的接口，由 ExtProc 调用。Smart router 用。
Flash Attention	Tri Dao 的高效 attention kernel（SRAM tiling + fused softmax）。它是 CUDA 上的候选后端之一；vLLM 会按平台、dtype、head layout 与模型能力选择。
GQA / MQA	Grouped / Multi-Query Attention。多个 Q head 共享一组 KV head，省 KV cache。
HBM	High-Bandwidth Memory。GPU 显存的主体（80 GB on H100），带宽 $\sim$ 3 TB/s。
InputBatch	V1 持久化的 batch 状态。每步只增删 diff，不重建。
KV cache	缓存历史 token 的 K / V 矩阵，避免重算。LLM 推理内存占用主体。
LMCache	把 KV cache 跨进程 / 跨节点持久化的项目。
LoRA / PEFT	Low-Rank Adaptation 微调。只新增两个小矩阵 B、A 而不动 base 权重。
Marlin	INT4 weight $\times$ FP16 activation 的高速 GEMM kernel。
memory-bound	性能上限被内存带宽决定（算力闲着）。decode 阶段是典型。

术语	含义
MLA	Multi-head Latent Attention（DeepSeek-V2/V3）。把 KV 压到低秩 latent，省 $\sim 10\times$ 显存。
MoE	Mixture of Experts。每层多个 FFN（experts），router 选 top-k。
NCCL	NVIDIA Collective Communication Library。跨 GPU 集合通信底层。
NIXL	NVIDIA InfiniBand eXchange Library。跨节点 GPU-Direct RDMA 传 KV。
NVLink	NVIDIA 卡间高速互联（H100 900 GB/s）。
PagedAttention	vLLM 核心创新。把 KV cache 分页管理。详见 02-core-concepts/01。
Prefill	处理 prompt 的阶段，一次 forward 处理整个 prompt。compute-bound。
Prefix caching	跨请求复用相同前缀的 KV block。chatbot 场景命中率极高。
Preemption	KV 不够时把某请求踢回 waiting。当前 V1 释放其 block、把 num_computed_tokens 归零并重新排队，相当于 recompute；KV offload/connector 是另一条能力边界。
RoPE	Rotary Position Embedding。给 Q、K 加旋转编码注入位置信息。
Sampler	把 logits 转成 token id 的模块。含 top-k / top-p / temperature 等。
Scheduler	EngineCore 内的调度核心。每步决定哪些请求跑多少 token。
Speculative decoding	投机解码。draft 小模型提议 N 个，target 大模型一次验证。详见 04-optimizations/02。
TTFT / TPOT / TTLT	推理服务三大延迟指标。详见第 10 节。
Tokenizer	文本 $\leftrightarrow$ token id 双向转换器。
TP（Tensor Parallel）	把一层权重切到多卡。详见 05-distributed/01。
vLLM Production Stack / AIBrix / llm-d	三套主流的 vLLM 生产部署参考栈。详见 08-production-deployment/01。
Worker	每张 GPU 一个 Worker 进程。持有模型分片 + KV 显存，执行 forward。
ZMQ	ZeroMQ。API Server $\leftrightarrow$ EngineCore 之间的轻量消息队列。

小结

- LLM 推理是**自回归**的：每一步 forward 只产出 1 个 token，且要把整个模型权重重新读一遍——这是 decode 阶段 memory-bound 的根因。
- KV cache** 把历史 token 的 K/V 缓存下来，是把"重算 N^2 "降到"重读 N"的关键，但也是显存占用的大头。
- Prefill 是 compute-bound、Decode 是 memory-bound**；要用 batching 把权重读取摊薄到多个并发请求上。
- 工程指标分两层：用户侧看 **TTFT / TPOT**，平台侧看**吞吐 / GPU util / 并发数**——这是后面所有优化要回去对照的"靶子"。

自检

无 GPU 源码定位练习

只读锁定源码，不导入 vLLM：依次在 `vllm/v1/engine/llm_engine.py`、`vllm/v1/engine/core_client.py`、`vllm/v1/engine/core.py`、`vllm/v1/core/sched/scheduler.py`、`vllm/v1/worker/gpu_model_runner.py` 中定位“提交请求、跨进程、调度、执行”边界，并记下每一跳传递的数据类型。最后回答：哪一步定义 TTFT 的服务端起点，哪一步只能解释 GPU 执行时间？

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 画出"用户输入 → token → forward → 采样"这条循环，并标出哪一步会读 / 写 KV cache。

```
[用户输入文本]
  ↓ tokenize
[input_ids: [n] 个 token]
  ↓ embed (W_embed)
[hidden: [n, H]]
  ↓ 每个 attention 层:
    ① Q/K/V 投影 → 写 KV cache (把这步新算的 K/V 存入 cache)
    ② attention:  $Q \cdot K^T$ , K 来自 cache → 读 KV cache
    ③ FFN
  ↓ × num_layers
[hidden: [n, H]]
  ↓ lm_head → logits: [n, V]
  ↓ sample → next_token_id (单个 int)
  ↓ append 到序列
  ↓ 回到 "embed" 这步 (只 embed 这 1 个新 token, n=1)
[循环直到 EOS / max_tokens]
```

关键点：

- 第一次 (prefill) n 个 token 一起算，K/V 写入 cache；之后每次 decode 只算 1 个 token，但 attention 还要读**全部历史的 K/V cache**——这就是 decode memory-bound 的根因。
- 写 cache**：每层 attention 算完 K/V 投影后立刻写。
- 读 cache**：每层 attention 做 $Q \cdot K^T$ 和 $\text{attention} \cdot V$ 时读。
- LayerNorm / FFN / embed / lm_head 都不碰 cache。

2. Llama-3-70B (80 层、hidden=8192、BF16) 一个 token 在 KV cache 里占多少字节，1 个 4K-token 请求需要多少 GB？

注意 Llama-3-70B 用了 GQA：64 query head + 8 KV head + head_dim=128，所以 KV 实际维度是 $8 \times 128 = 1024$ ，**不是** 8192。

单层单 token 的 KV 字节数：
 $2(K+V) \times \text{num_kv_heads} \times \text{head_dim} \times \text{dtype_bytes}$
 $= 2 \times 8 \times 128 \times 2$
 $= 4,096$ 字节 (4 KB)

80 层一个 token：
 $4 \text{ KB} \times 80 = 320 \text{ KB}$

4096 token 请求：
 $320 \text{ KB} \times 4096 = 1,310,720 \text{ KB} \approx 1.28 \text{ GB}$

如果没开 GQA、按 hidden=8192 算（错误算法但常见误区）：

$$2 \times 8192 \times 2 \times 80 \times 4096 = 10.7 \text{ GB} \leftarrow 8\times \text{大!}$$

→ 80GB H100 装一个 4K 请求只占 1.6% 显存（GQA），不开 GQA 就要 13%。这就是为什么现代模型必须做 GQA / MQA / MLA。

3. 为什么 batch=32 的吞吐能接近 batch=1 的 32 倍，但 batch=1 的延迟和 batch=32 的延迟相差不大？

核心：decode 是 memory-bound，每步要把所有模型权重（ $70B \times 2 \text{ byte} = 140 \text{ GB}$ ）从 HBM 读一遍，**算力远远没用完。**

- batch=1：读 140 GB 权重，算 1 个 token 的活。一步时间 $\approx$ 权重读取时间 $\approx 140 \text{ GB} / 3 \text{ TB/s} \approx 47 \text{ ms}$ 。
- batch=32：还是读 140 GB 权重（**只读一次**），算 32 个 token 的活（算力够用，因为之前富余）。一步时间 $\approx 50\text{-}60 \text{ ms}$ （略增因算力开始紧）。

→ 时延几乎没变（ $47 \rightarrow 55 \text{ ms}$ ），但同时间内输出 $32\times$ 个 token，吞吐 $\approx 32\times$ 。这就是 continuous batching 的全部魅力来源。

类比：你去快递点取件，自己一个人取 = 一个人路上时间 = 半小时；32 个人一起去取 = 还是半小时，但取回 32 个件。

4. 给真实场景，TTFT / TPOT / 吞吐 优先级？

场景	TTFT	TPOT	吞吐	理由
Chatbot（在线对话）	关键	关键	中	用户点了发送后等响应：TTFT < 500ms 才不卡顿；TPOT 决定打字速度，> 50ms 就感觉慢
离线 batch 评估	无所谓	无所谓	关键	一次跑几万 prompt，总时长是 KPI；TTFT 大几秒也无所谓
RAG（短上下文）	关键	中	中	用户问问题等答案，TTFT 是核心；输出一般几百 token，TPOT 影响小
RAG（长文档）	关键	关键	中	长 prompt 的 TTFT 容易爆（chunked prefill 救），输出也长 → TPOT 累积影响明显
Code completion	极关键	关键	低	TTFT > 100ms 用户就会觉得 IDE 卡；输出短
Agent / 工具调用	关键	关键	中	多轮交互，每轮 TTFT/TPOT 都累积

→ **优化时只挑 1-2 个指标作为靶子**，不能"全部都要"。不同指标的优化方向往往冲突（如大 batch 提吞吐但损失 TPOT）。

5. 用 OS / 体系结构 / 分布式 课里学过的一个概念，对应 vLLM 后面要讲的某项设计。

课程概念	vLLM 对应
OS 虚拟内存 / 分页 / 页表	PagedAttention: KV 切成定长 block + block table 间接寻址。详见 02-core-concepts/01-paged-attention.md
OS 进程调度 / 时间片	Scheduler 每步决定哪些请求跑、跑几个 token (token budget 类比时间片)。详见 03-code-walkthrough/02-scheduler.md
OS COW (写时复制)	Beam search 共享前缀 block, 分歧时才 COW 新 block
OS LRU 缓存	Prefix cache 用 LRU 淘汰冷 block。详见 02-core-concepts/04-prefix-caching.md
体系结构: 流水线 / bubble	Pipeline Parallel 的流水填充 / 排空。详见 05-distributed/01-tp-pp-ep.md § 3
体系结构: 算术强度 / roofline	Prefill compute-bound vs decode memory-bound 的根本原因
CPU cache 局部性 / cache line	PagedAttention block_size 选 16 而不是 1 或 128: 太小 indirection 多、太大内部碎片
分布式: Merkle tree	Prefix cache 用链式 hash (每个 block 的 hash 依赖前一个 block 的 hash)
分布式: 心跳 / consensus	DP Coordinator 监控 rank 状态、协调 sleep/wake
分布式: AllReduce / AllGather	TP 末尾 AllReduce、SP MoE Pass 把 AllReduce 转 ReduceScatter
OS 共享内存 IPC	vLLM EngineCore ↔ Worker 用共享内存 ring buffer 传 SchedulerOutput

→ vLLM 大量"工程创新"实质上是把成熟系统设计搬到 LLM 推理场景。读源码时如果觉得某个设计奇怪, 先想想"这像 OS / 数据库 / 网络里的哪个老朋友", 通常都能对上号。

下一步

- 下一节: 01-what-is-vllm.md (把这里所有"在 vLLM 里"的伏笔串成 vLLM 的三大武器)。
- 想跳到核心创新: 02-core-concepts/01-paged-attention.md (KV 碎片是怎么解决的)。
- 想动手: 07-hands-on/01-setup.md (先把环境跑起来, 再看后面就有实物对照)。
- 如果只有 1 天, 按 README.md 第 1 节的"最短路径"读。

01. vLLM 是什么？为什么快？

谁该读这一篇？ 想用三句话讲清"vLLM 是干嘛的、解决什么瓶颈、相比 HF/TGI/TRT-LLM 凭什么快"的读者。

前置阅读： 00-prerequisites.md（至少 § 4 KV cache、§ 7 roofline、§ 8 batching 这三节）。

耗时： 约 12 分钟。

学完能：

1. 用一句话说出 vLLM 的定位，并复述 PagedAttention 论文里 KV 利用率 20-38% → 96% 的关键数字。
2. 列出 vLLM 的三大武器（PagedAttention / Continuous Batching / Prefix Caching + Chunked Prefill）并对应它们解决的具体瓶颈。
3. 区分 vLLM 与 TGI / TensorRT-LLM / SGLang / LMDeploy 的定位差异。
4. 指出哪些场景**不适合**用 vLLM，避免"为了 vLLM 而 vLLM"。

当前源码复核 (b23bd73)： PagedAttention、iteration-level scheduling、prefix caching 与 chunked prefill 仍是核心机制；具体吞吐或 TTFT 百分比属于论文/特定 benchmark 结果，不能外推成当前版本对任意模型、硬件和负载的承诺。

1. 一句话定义

vLLM 是为大语言模型推理与服务设计的**高吞吐、低延迟引擎**。由 UC Berkeley Sky Computing Lab 在 2023 年开源，核心论文是 SOSP'23 的 PagedAttention。

它要解决的问题：**让一张 GPU（或一组 GPU）在单位时间内服务尽可能多的并发请求，同时把每个用户感知到的延迟压到最低。**

2. 朴素方案的三个瓶颈

直接用 HuggingFace Transformers 部署 Llama-7B 推理服务，会立刻撞到三堵墙：

2.1 KV cache 内存碎片

LLM 自回归生成的每一步都需要把已生成 token 的 Key/Value 缓存下来，否则下一个 token 的 attention 没法回看历史。这就是 KV cache。

朴素做法：给每个请求预分配一段**连续显存**，长度按 max_seq_len（如 2048）顶格。

- 实际只生成 100 token → 浪费 95% 的预留
- 多请求长度不一 → **内部碎片**（reserved 但未用）+ **外部碎片**（小空闲块凑不出大请求需要的连续段）

PagedAttention 论文实测：KV cache 的有效利用率只有 **20.4% ~ 38.2%**。一张 80 GB 的 H100，60 GB 在浪费。

2.2 静态 batching 让 GPU 空转

朴素批处理（static batching）：一批请求一起进、一起出。最慢的那个决定整组的延迟。

```
gantt
  title Static Batching: 长请求把整组拖死
  dateFormat X
  axisFormat %s

  section Slot 0 (A 短)
  A 生成 10 token : a, 0, 10
  空闲      :crit, after a, 90

  section Slot 1 (B 中)
  B 生成 60 token : b, 0, 60
  空闲      :crit, after b, 40

  section Slot 2 (C 长)
  C 生成 100 token : c, 0, 100

  section Slot 3 (D 中)
  D 生成 80 token : d, 0, 80
  空闲      :crit, after d, 20
```

A 早早做完，但槽位释放不了——下一个 batch 必须等 C 跑完。GPU 平均利用率往往不到 30%。

2.3 decode 阶段的访存瓶颈

decode（自回归生成）是 memory-bound 的：每生成一个 token，要把整个 KV cache 从 HBM 拉到 SRAM 才能做 attention。算术强度（FLOPs/Bytes） ≈ 1 ，远低于 GPU 算力提供的 100+。算力大头闲着。

3. vLLM 的三大武器

```
flowchart LR
  Naive["朴素方案<br/>KV 利用率 20-38%<br/>GPU 利用率 ~30%<br/>访存瓶颈"]
  Naive -->|碎片消除| W1["PagedAttention<br/>KV 利用率 96%+"]
  Naive -->|动态调度| W2["Continuous Batching<br/>每步重新组 batch"]
  Naive -->|跨请求复用| W3["Prefix Caching<br/>+ Chunked Prefill"]
  W1 --> Result["吞吐最高 24x<br/>(vs HF TGI)"]
  W2 --> Result
  W3 --> Result
```

```
classDef bad fill:#fee2e2,stroke:#b91c1c,color:#1a1f29;
classDef good fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
classDef result fill:#dcfce7,stroke:#15803d,color:#1a1f29;
class Naive bad;
class W1,W2,W3 good;
class Result result;
```

3.1 PagedAttention — 解决 KV 碎片

借鉴 OS 虚拟内存：

- KV cache 切成定长 **block** (通用 fallback 为 16 token/block，平台或显式参数可以覆盖)
- 每个请求一张**块表 (block table)**，把逻辑 block 索引映射到物理 block
- 物理 block 池统一管理、按需分配
- 内存利用率 > **96%**

详见 02-core-concepts/01-paged-attention.md。

3.2 Continuous Batching — 解决空转

每一步（每生成一个 token）都重新调度：

- 完成的请求立刻退出，腾出 KV 空间
- 等待队列里的新请求立刻进 batch
- 不同请求的 prefill / decode 可以混在同一个 forward step

实测吞吐 vs HuggingFace：最高 **24×**。详见 02-core-concepts/02-continuous-batching.md。

3.3 Prefix Caching + Chunked Prefill — 再上一台阶

- **Prefix Caching**：相同前缀（system prompt、few-shot 示例）的 KV block 只算一次，跨请求共享
- **Chunked Prefill**：长 prompt 的 prefill 切成多 chunk，与 decode 混跑，避免长 prefill 把 decode 卡住

4. 设计哲学

读代码前先建立这几条直觉，能省很多迷路时间：

原则	体现
内存即一等公民	调度的第一问是"还有 KV block 吗"，不是"队列有请求吗"
请求是异步事件，不是同步循环	EngineCore 异步轮询；Scheduler 每步独立决策，没有"等一个 batch 跑完"的概念
任何长操作都要可调度	长 prefill 可切块；资源不足时请求会被 preempt、释放 KV 并回到 waiting，之后重算或命中 prefix cache
Worker 是纯执行器	Scheduler 决定"做什么"，Worker 只负责"做"。分布式扩展时这个边界很重要
后端是可插拔的	Attention 有 7+ 个后端，量化 10+ 种，平台 CUDA / ROCm / CPU / TPU
正确性 > 性能，但两者都要	Chunked prefill 的 attention mask 必须严格等价于完整 prefill 的结果

5. 性能数据（理解吞吐时可引用）

对比对象	vLLM 提升	备注
HuggingFace TGI（朴素 batching）	24×（吞吐）	OPT-13B 论文数据
FasterTransformer（NVIDIA）	2.2× ~ 3.5×	主要靠 PagedAttention
Triton + 静态 batch	8×+	同模型同卡

- KV cache 利用率：20-38% → **96%+**
- TTFT：与朴素方案接近，但**吞吐高一个量级**

6. 不适合 vLLM 的场景

这里经常容易混淆，建议提前分清：

- **极低延迟（< 10 ms TTFT）的单请求场景**：vLLM 为吞吐设计，单请求会被 batching overhead 拖累。这种场景用 TensorRT-LLM + CUDA Graph 重度优化更合适。
- **超大模型 + 极少并发**：405B 模型只服务 2 个用户，PagedAttention 的内存收益体现不出来。
- **训练**：vLLM 是纯推理引擎，没有反向传播。

7. vLLM 与生态

容易混淆的"近邻"：

系统	定位	对比 vLLM
TensorRT-LLM	NVIDIA 官方推理优化	单请求延迟更低，但灵活性差、新模型支持慢
SGLang	结构化生成 + RadixAttention	RadixAttention 是 PagedAttention 的 trie 变体，前缀复用更强
TGI	HuggingFace 官方	简单易用，性能弱于 vLLM
Llama.cpp	CPU / Edge	量化生态好，吞吐场景非对手
DeepSpeed-Inference	微软	从训练栈衍生，迭代慢
LMDeploy	上海 AI Lab	Turbomind 后端性能强，灵活性不如 vLLM

小结

- 朴素 LLM 服务有三堵墙：**KV cache 碎片**（利用率 20-38%）、**静态 batching 让 GPU 空转**、**decode 阶段算力闲置**——vLLM 的每一项创新都对应消解其中一堵。
- 三大武器：**PagedAttention** 把 KV 切块管理，利用率拉到 96%+；**Continuous Batching** 每步重组 batch；**Prefix Caching + Chunked Prefill** 再榨干跨请求复用与长 prompt 的间隙。
- vLLM 是为**吞吐**优化的工程系统；极低单请求延迟、超大模型 + 极少并发、训练这三类场景不是它的主战场。
- 同生态对比：TGI 简单但慢、TensorRT-LLM 单请求延迟更低但灵活性差、SGLang 用 RadixAttention 在前缀复用上更激进、LMDeploy Turbomind 后端性能强但生态弱。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 不查资料说出 PagedAttention 借鉴的 OS 概念、3 条对应关系（页 / 页表 / COW）。

OS 概念	PagedAttention 对应	共同动机
物理页（Page）	KV block（默认 16 个 token 的 K+V）	把大空间切成定长小块，统一管理，消除外部碎片
页表（Page Table）	Block Table（每个请求一张）	把"逻辑连续"的视图映射到"物理离散"的存储；indirection 一层换灵活性
COW（写时复制）	Beam search / prefix caching 的 block 共享	多请求共享只读 block（ref_cnt++），分歧时才复制

补充细节：还能讲**LRU 淘汰**（OS swap 选 victim ↔ Prefix cache evict 冷 block）、**虚拟地址翻译开销** ↔ block_table 间接寻址带来约 20% attention 慢，但整体吞吐 24× 净赚。

2. 给客户介绍 vLLM 为什么比 HF 快 24×，90 秒三大武器各讲一句话。

"朴素方案有三堵墙——KV 浪费 60%、GPU 空转 70%、decode 卡内存。

武器一：PagedAttention。把 KV cache 像 OS 分页一样切成小块，块表间接映射。利用率从 20-38% 拉到 96%，相当于同一张 H100 多塞 2-3 倍并发。

武器二：Continuous Batching。每生成 1 个 token 就重新组 batch，做完的退出、等待的进来。再没人陪跑，GPU 利用率从 30% 拉到 80%+。

武器三：Prefix Caching + Chunked Prefill。相同 prompt 前缀的 KV 跨请求复用，TTFT 直降 95%；长 prompt 切块跟 decode 混跑，杜绝长请求把别人卡死。

综合起来对比 HF Transformers，吞吐 24×。"

3. 老板说"给单个 VIP 用户做极低延迟 chatbot，QPS 个位数"——用 vLLM 吗？

不建议（或至少不要为它而设计），给三点理由：

1. **vLLM 是为吞吐设计**：PagedAttention / Continuous Batching 在低并发下几乎没有收益（没有"其他请求"可以摊薄权重读取）。
2. **batching overhead 反而拖累延迟**：vLLM 每步有 scheduler decision、block allocation、metadata 同步等固定开销。单请求场景下这些是纯成本。
3. **TensorRT-LLM + CUDA Graph 是更好的选择**：NVIDIA 把单请求路径优化到了 metal——CUDA Graph capture 单 batch、kernel fusion 重度做、无 PagedAttention overhead。

反建议：如果"VIP + 低延迟"实际上是"少量 VIP + 中等并发 + 严苛 SLO"，vLLM 还是值得用，但要：

- 关 `--enforce-eager` 走 CUDA Graph
- 把 `max-num-batched-tokens` 调小以减 step 时长方差
- 用 `--scheduling-policy priority` 给 VIP 优先级
- 多实例 + 路由层做物理隔离更稳

4. scheduler.py VS block_pool.py，哪份实现 PagedAttention、哪份实现 Continuous Batching？

- **vllm/v1/core/block_pool.py** 实现 **PagedAttention 的内存层**：定义 KVCacheBlock 数据结构、free queue（LRU 双链表）、allocate / free / ref_cnt 管理。这是"物理页分配器"。
- **vllm/v1/core/sched/scheduler.py** 实现 **Continuous Batching 的调度层**：每步决定哪些请求跑、跑几个 token、token budget 怎么分。这是"调度器主循环"。

两者通过 kv_cache_manager.py 衔接：scheduler 说"这个请求要 K 个新 block"，kv_cache_manager 调 block_pool 分配。**没有 block_pool，scheduler 无法做内存决策；没有 scheduler，block_pool 只是个空闲块池。**

补充：再讲清 hash_block_tokens 是 **prefix caching** 的关键计算（链式 hash，详见 02-core-concepts/04-prefix-caching.md）。这是第四件"次要武器"，跟 block pool 配合做跨请求 KV 复用。

下一步

- 下一节：02-architecture.md（把上面三大武器落到 API Server / EngineCore / Worker 三层进程结构里）。
- 想看源码：vllm/v1/core/block_pool.py（PagedAttention 的物理 block 池）、vllm/v1/core/sched/scheduler.py（Continuous Batching 的调度循环）。
- 想动手：07-hands-on/02-trace-a-request.md（跑通第一个 vLLM 请求，把吞吐对比"算给自己看"）。
- 想从生产视角理解：08-production-deployment/01-deployment-architectures.md（这三大武器在 K8s 与生产网关里如何被组合）。

02. vLLM 整体架构

谁该读这一篇？ 想在脑子里建立一张"一个请求从 HTTP 到 GPU 再到流式返回"的全景图的同学；后面所有源码走读都靠这张图定位"自己读到了哪一层"。

前置阅读： 01-what-is-vllm.md（知道 vLLM 在解决什么问题）；最好扫一眼 00-prerequisites.md § 8-10 的 batching / 分布式 / 指标。

耗时： 约 20 分钟（含两张 Mermaid 仔细看）。

学完能：

- 1. 不看代码描述一个请求从 HTTP 进来到 token 流式返回的所有进程跳转。
- 2. 列出 SchedulerOutput、Request、KVCacheBlock、ModelRunnerOutput 四个数据契约的关键字段。
- 3. 在脑子里展开单次 step 的执行链：Scheduler → Executor → Worker → forward → Sampler → 输出。
- 4. 把 vLLM 的每一项设计映射回操作系统 / 网络 / 分布式系统课里学过的对应概念。

当前源码复核（b23bd73）： 在线服务的主链是 API server / AsyncLLM → EngineCoreClient（常见为 AsyncMPClient）→ EngineCoreProc → Executor → Worker。离线单进程、multiprocessing、Ray 与 external launcher 的边界不同，三层图是逻辑职责图，不保证每种部署都产生相同 PID 数量或同一种 IPC。

目标是脑子里有一张清晰的图——一个请求从 HTTP 到 GPU 再到流式返回的完整路径。这张图刻进脑子，后面所有源码走读都能定位到它的某一层。

1. 进程视角：三层分工

vLLM 不是单进程序序。生产部署时它由多个进程协作：



源码定位：

组件	文件
API Server	vllm/entrypoints/openai/api_server.py
LLM 类（offline 入口）	vllm/entrypoints/llm.py
EngineCore	vllm/v1/engine/core.py
LLMEngine	vllm/v1/engine/llm_engine.py
Worker	vllm/v1/worker/gpu_worker.py
ModelRunner	vllm/v1/worker/gpu_model_runner.py

1.5 进程模型 • 简版（深入见 05-process-and-ipc-internals.md）

逻辑三层背后的真实进程长这样（每个 Worker 进程绑一张 GPU）：

源码锚点：[vllm/v1/engine/core_client.py](#) • [MPClient](#)

源码锚点：[vllm/v1/engine/utils.py](#) • [launch_core_engines](#)

源码锚点：[vllm/v1/engine/core.py](#) • [EngineCoreProc.run_engine_core](#)

源码锚点：[vllm/v1/executor/multiproc_executor.py](#) • [MultiprocExecutor](#)

源码锚点：[vllm/v1/executor/multiproc_executor.py](#) • [WorkerProc.make_worker_process](#)

源码锚点：[vllm/v1/executor/multiproc_executor.py](#) • [WorkerProc.worker_main](#)

```
graph TD
    subgraph TB
        subgraph API ["API Server 进程<br/>名字: 主进程 (uvloop event-loop)"]
            APICli["vllm/entrypoints/cli/serve.py:main()<br/>(args.api_server_count 控制副本数, 默认 1)"]
            APIfast["vllm/entrypoints/openai/api_server.py<br/>FastAPI + uvloop"]
            AsyncMP["AsyncMPCClient<br/>(MPCClient 子类)<br/>vllm/v1/engine/core_client.py"]
            APICli --> APIfast
            APIfast --> AsyncMP
        end

        subgraph ECP ["EngineCore 进程<br/>名字: EngineCore (DP 时 EngineCore_DP{N})"]
            Launcher["launch_core_engines()<br/>vllm/v1/engine/utils.py<br/>context.Process(target=run_engine_core)"]
            ECPProc["EngineCoreProc.run_engine_core(...)<br/>vllm/v1/engine/core.py<br/>(set_process_title 'EngineCore')"]
            Sched["Scheduler<br/>vllm/v1/core/sched/scheduler.py"]
            Exec["Executor (Multiproc / Ray / Uniproc)<br/>vllm/v1/executor/"]
            Launcher --> ECPProc
            ECPProc --> Sched
            ECPProc --> Exec
        end

        subgraph WP ["Worker 进程 × N<br/>名字: VllmWorker-{rank}"]
            MPExec["MultiprocExecutor._init_executor<br/>vllm/v1/executor/multiproc_executor.py"]
            MakeProc["WorkerProc.make_worker_process<br/>multiproc_executor.py<br/>context.Process(target=worker_main)"]
            WorkerMain["WorkerProc.worker_main<br/>multiproc_executor.py"]
            Runner["GPUWorker / GPUModelRunner<br/>vllm/v1/worker/gpu_worker.py<br/>gpu_model_runner.py"]
            MPExec --> MakeProc
            MakeProc --> WorkerMain
            WorkerMain --> Runner
        end

        AsyncMP -- "ZMQ ROUTER/DEALER (input)<br/>ZMQ PULL/PUSH (output)<br/>msgpack" --> ECPProc
        Exec -- "shared-memory MessageQueue<br/>(rpc_broadcast_mq)" --> WorkerMain

        classDef api fill:#fef3c7,stroke:#b45309,color:#1a1f29;
        classDef engine fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
        classDef worker fill:#dcfce7,stroke:#15803d,color:#1a1f29;
        class APICli,APIfast,AsyncMP api;
        class Launcher,ECPProc,Sched,Exec engine;
        class MPExec,MakeProc,WorkerMain,Runner worker;
```

简版到此为止——以上 Mermaid 已展示三层进程结构。

要看 进程怎么 spawn（fork vs spawn、入口函数、PID 树）、ZMQ 控制平面、shared-memory 数据平面、PEP 574 零拷贝、三种部署形态、真实 ps -ef 进程清单，请进入下一篇 05-process-and-ipc-internals.md。本节不在架构总览里展开，让这一篇保持"全景"定位。

2. 数据契约：四个核心数据结构

读源码之前先认这四个结构。后面所有调用都在围着它们转。

2.1 Request（请求）

- 位置：vllm/v1/request.py
- 字段：request_id、prompt_token_ids、sampling_params、status、output_token_ids、num_computed_tokens
- 状态机：WAITING → RUNNING → FINISHED / PREEMPTED

2.2 SchedulerOutput（调度器一步的决策）

- 位置：vllm/v1/core/sched/output.py
- 关键字段：
 - scheduled_new_reqs — 本步新进入 batch 的请求
 - scheduled_cached_reqs — 本步继续执行的请求
 - num_scheduled_tokens — 每个请求本步要算几个 token（prefill 可能 > 1，decode = 1）
 - total_num_scheduled_tokens — 所有请求加起来的本步总 token 数
 - preempted_req_ids — 本步被踢出去的请求
 - finished_req_ids — 本步完成的请求
 - kv_connector_metadata — 跨 worker 传输 KV 的元数据（多机用）

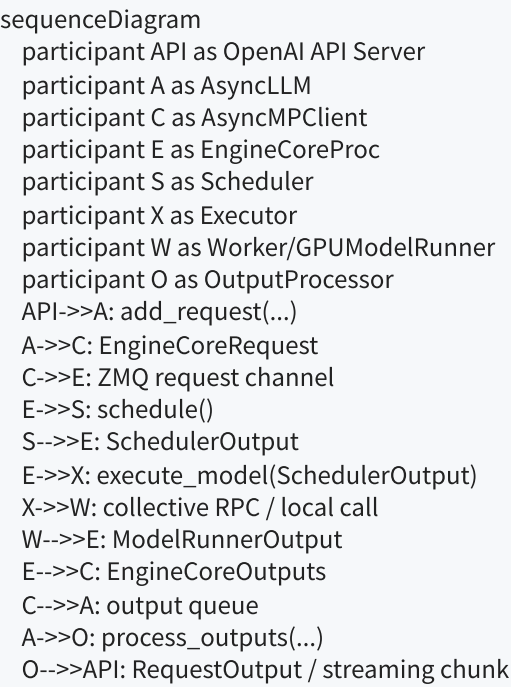
2.3 KVCacheBlock / BlockTable

- KVCacheBlock：一个物理 block（一段固定显存），含 block_id、ref_cnt
- BlockTable：每个请求的逻辑 block 序列（list[block_id]）
- 位置：vllm/v1/core/kv_cache_utils.py、vllm/v1/core/block_pool.py

2.4 ModelRunnerOutput（Worker 跑完的结果）

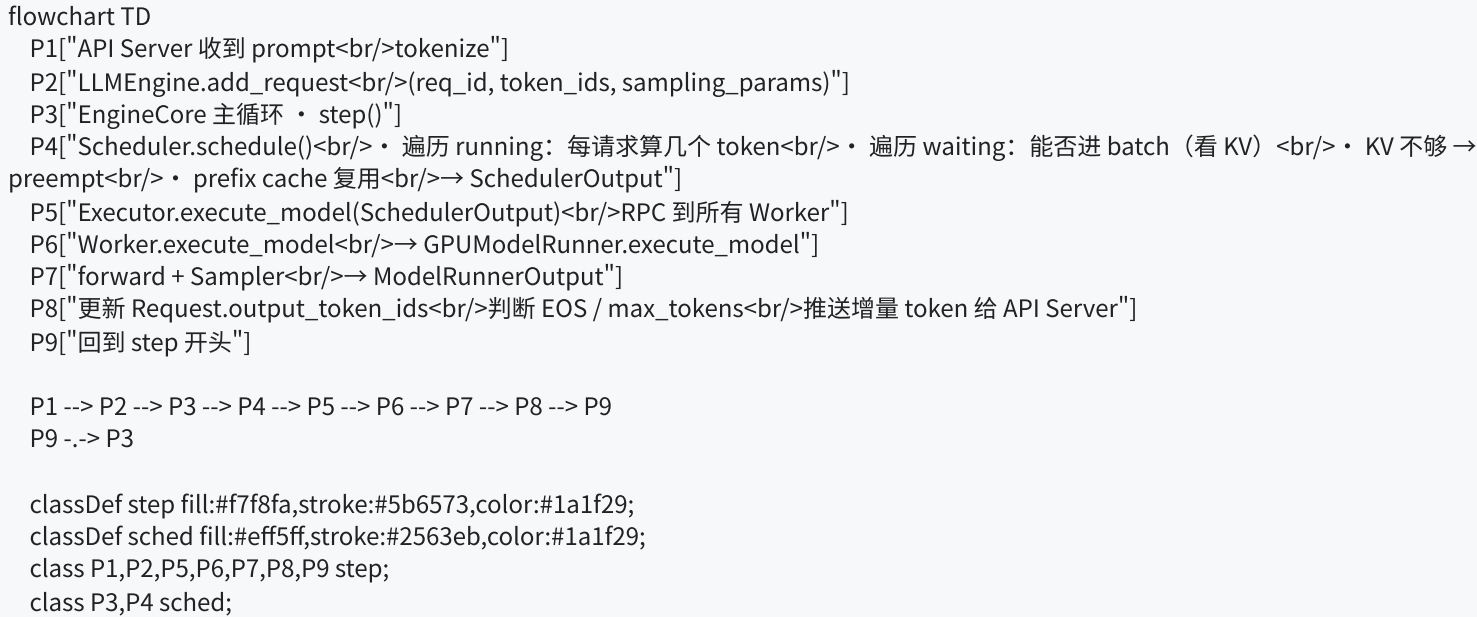
- 字段：sampled_token_ids、logprobs、prompt_logprobs
- Worker 返回给 EngineCore 的"答卷"

3. 单次推理 step 的完整流程



InprocClient、Ray executor 或外部 launcher 会替换其中的传输实现，但不会改变“请求契约 → 调度契约 → runner 输出 → 用户输出”的职责顺序。

整个推理过程的核心 step 循环：



forward 内部进一步展开（一层 Transformer 的执行链）：



这张链路图刻在脑子里之后，绝大多数实现问题都能往这张图上靠——问 attention 在哪一步、问 TP 通信在哪一步、问 KV 写在哪一步，都能直接指出位置。

4. 同步 vs 异步：V1 的 async scheduler

vllm/v1/core/sched/async_scheduler.py 让调度与前向 overlap：

```
gantt
    title 普通模式 vs Async Scheduler
    dateFormat X
    axisFormat %s
```

```
section 普通
step 1 schedule   :s1, 0, 1
step 1 forward   :f1, after s1, 4
step 1 output    :o1, after f1, 1
step 2 schedule   :s2, after o1, 1
step 2 forward   :f2, after s2, 4
step 2 output    :o2, after f2, 1

section Async
step 1 schedule   :as1, 0, 1
step 1 forward   :af1, after as1, 4
step 2 schedule   :as2, after as1, 1
step 1 output    :ao1, after af1, 1
step 2 forward   :af2, after as2, 4
step 2 output    :ao2, after af2, 1
```

Scheduler 是纯 CPU 操作，与 GPU forward 在不同执行单元上跑。让 step N+1 的 schedule 在 step N 的 forward 还在跑时启动，大 batch 下可省 5-10% 端到端时间（Python 调度开销在大 batch 下不可忽略）。

5. 模型加载：从 HuggingFace 到 GPU

```
flowchart TD
    A["ModelConfig<br/>model='meta-llama/Llama-3-8B'"]
    B["ModelLoader<br/>vllm/model_executor/model_loader/<br/>自动选 loader (HF / GGUF / S3 / Tensorizer)"]
    C["模型类<br/>vllm/model_executor/models/llama.py<br/>nn.Linear → ColumnParallel/RowParallel<br/>LayerNorm → RMSNorm CUDA kernel<br/>Attention → vLLM Attention (PagedAttn)"]
    D["权重加载<br/>TP/PP 切分（每卡只加载自己分片）"]
    E["可选阶段<br/>量化转换<br/>torch.compile<br/>CUDA Graph capture"]
    F["模型就绪"]

    A --> B --> C --> D --> E --> F

classDef step fill:#fff,stroke:#2563eb,color:#1a1f29;
class A,B,C,D,E,F step;
```

源码入口：vllm/v1/worker/gpu_worker.py:load_model()。

6. 一个具体例子：单 prompt 生成 10 tokens

设 prompt 32 tokens、block_size=16、max_tokens=10：

Step	阶段	Scheduler	Worker	输出
1	Prefill	分配 $\text{ceil}(32/16)=2$ 个 block	计算 32 token 的 QKV，写入 2 个 block，采样第 33 个 token	1 token
2..11	Decode	检查 $33+i$ 落在第几个 block，跨边界则新分配 1 个 block	算 1 个新 token 的 QKV，读全部历史 KV 做 attention，采样下一个 token	1 token / step
12	结束	达到 max_tokens，标记 FINISHED，free 所有 block (ref_cnt -= 1)；prefix caching 开启时 block 进 free queue 但不删 hash	—	—

注意几个非显然的点：

1. **Prefill 与 decode 是同一段代码**，区别只是 num_scheduled_tokens > 1 vs == 1。源码里没有"prefill 阶段"和"decode 阶段"的分支。
2. **第 33 个 token 写在第几个 block？** 写在 block 1 的位置 0（因为 0-15 在 block 0，16-31 在 block 1，32 才落到 block 2 第一格——但 prompt 只到 31，第 33 个 token 是采样出来的）。
3. **第 32 个 token 后采样出来的第 33 个 token 不算 prefill 输入**——下一步它才会被算 attention。

7. vLLM 与系统课的对应关系（给本科生的桥）

vLLM 几乎每一个设计决定都能在本科系统课里找到原型。把这层映射建立起来，能让你**用已经熟悉的概念**去理解陌生的代码，而不是把每个名词都当新东西死记。

7.1 操作系统课

源码锚点：[vllm/v1/engine/core.py](#) • [EngineCoreProc.run_engine_core](#)

源码锚点：[vllm/v1/executor/multiproc_executor.py](#) • [WorkerProc.make_worker_process](#)

源码锚点：[vllm/distributed/device_communicators/shm_broadcast.py](#) • [SpinCondition](#)

课上学过的	vLLM 里的体现	详见
进程 vs 线程	Worker 用进程不用线程（避开 Python GIL；每进程独立 CUDA context）	05-process-and-ipc-internals.md § 2
虚拟内存 / 页表	PagedAttention —— KV block 是物理页，每请求一张 block table	02-core-concepts/01-paged-attention.md
页错误 → 抢占	KV 不够时的 preempt（当前 V1 释放并重算；offload/connector 另行建模）	02-core-concepts/03-kv-cache-management.md
写时复制（COW）	beam search / parallel sampling 多候选共享前缀 block，分歧时再 fork	同上
引用计数（refcount）	KVCacheBlock.ref_cnt	同上
LRU 缓存替换	BlockPool 的 free_block_queue 是 LRU 双向链表	03-code-walkthrough/03-kv-cache-manager.md
进程间通信（socket / shm / pipe）	API↔Engine 用 ZMQ；Engine↔Worker 用 multiprocessing.shared_memory	05-process-and-ipc-internals.md § 4-5
fork vs spawn	get_mp_context()；Linux fork、macOS spawn	05-process-and-ipc-internals.md § 2.2
信号 / 优雅退出	SIGTERM → EngineShutdownState.REQUESTED → drain → shutdown	vllm/v1/engine/core.py
生产者-消费者	Scheduler.waiting / running 队列	03-code-walkthrough/02-scheduler.md
发布-订阅（pubsub）	MessageQueue 是一写多读的共享内存广播器	05-process-and-ipc-internals.md § 5
自旋锁 / 条件变量	SpinCondition（shm_broadcast.py）等共享内存就绪信号	同上
调度策略（FCFS / 优先级）	Scheduler 默认 FCFS，可切 priority	vllm/v1/core/sched/scheduler.py
资源隔离（namespace / cgroup）	K8s Pod + NUMA binding + nvidia.com/gpu device plugin	08-production-deployment/01-deployment-architectures.md
文件描述符传递	make_worker_process 用 Pipe 把就绪/死亡信号传给子进程	vllm/v1/executor/multiproc_executor.py
Memory-mapped I/O	模型权重加载常用 mmap（safetensors 默认走 mmap 加载）	vllm/model_executor/model_loader/

7.2 计算机网络课

课上学过的	vLLM 里的体现	详见
TCP / Unix domain socket	API↔Engine 默认 ipc:// (UDS)，跨机退化 tcp://	05-process-and-ipc-internals.md § 4.2
HTTP/2 + Server-Sent Events	OpenAI 兼容的流式 token 输出	vllm/entrypoints/openai/api_server.py
RPC / 消息序列化	msgspec.msgpack 替代 pickle 做控制平面消息	05-process-and-ipc-internals.md § 4.3
集合通信 (MPI 风格 AllReduce)	NCCL 在 TP 内每层做 AllReduce、AllGather	05-distributed/01-tp-pp-ep.md
网络拓扑分层	NVLink (卡间, 900 GB/s) > InfiniBand / RoCE (机间, 400 Gbps) > Ethernet	00-prerequisites.md § 6
控制平面 vs 数据平面分离	API ↔ Engine 控制平面 + Engine ↔ Worker 数据平面分离	05-process-and-ipc-internals.md § 7
一致性哈希	API Gateway session sticky 路由 (conversation_id → 固定 Pod)	08-production-deployment/02-smart-routing-and-load-balancing.md
反向代理 / Service Mesh	Envoy AI Gateway + Istio Inference Extension (ExtProc → EPP)	08-production-deployment/03-gateway-and-service-mesh.md
RDMA / DMA 直传	NIXL 跨节点 GPU-Direct, 绕过 CPU 转发 KV	08-production-deployment/02-disaggregated.md
拥塞控制 / 限流	Gateway 的 token-level / RPS-level ratelimit、admission control	08-production-deployment/03-gateway-and-service-mesh.md § 5
重试 / 熔断	Envoy outlier detection + 客户端指数退避 + jitter	08-production-deployment/06-reliability-and-failure-modes.md

7.3 计算机体系结构 / GPU 编程

源码锚点: [vllm/distributed/device_communicators/shm_broadcast.py](#) • [MessageQueue.enqueue](#)

源码锚点: [vllm/distributed/device_communicators/shm_broadcast.py](#) • [MessageQueue.dequeue](#)

课上学过的	vLLM 里的体现	详见
存储层级（寄存器→cache→DRAM）	GPU 是寄存器 → SRAM → L2 → HBM → NVLink → PCIe → DRAM 七级	00-prerequisites.md § 6
算术强度 / Roofline	LLM prefill 是 compute-bound，decode 是 memory-bound	00-prerequisites.md § 7
SIMT / Warp	CUDA kernel 设计基础（FlashAttention tiling 都基于此）	03-code-walkthrough/06-cuda-kernels.md
流水线 ILP / 双 buffer	CUDA Graph + 异步 Scheduler，schedule 与 forward overlap	03-cudagraph-and-compile.md § 1 / 本篇 § 4
多级缓存	KV cache 的 L1（GPU HBM） / L2（CPU DRAM） / L3（远端 LMCache）	08-production-deployment/01-deployment-architectures.md
数据 vs 指令 cache	量化 weight 缓存 + KV cache 是数据；CUDA Graph 是 "instruction cache" 类比	04-optimizations/03-cudagraph-and-compile.md
字节序 / endian	shared-memory ring 内 to_bytes_big 大端打包	shm_broadcast.py
引用 vs 拷贝	PEP 574 PickleBuffer + OOB 实现零拷贝 IPC	05-process-and-ipc-internals.md § 5.3

7.4 分布式系统课

课上学过的	vLLM 里的体现	详见
主从模型 (leader/follower)	rank 0 是 driver worker，广播 SchedulerOutput 给其他 worker	03-code-walkthrough/04-model-runner.md
共识 / 协调	DP 模式下 DPCoordinator 协调多个 EngineCore 的全局状态	vllm/v1/engine/coordinator.py
故障恢复	KV preempt = 释放本地 block、清零进度、重新排队；外部 KV connector 另有状态协议	02-core-concepts/02-continuous-batching.md § 7
心跳 / liveness	EngineCore 监控 Worker 健康，崩了整组重启	08-production-deployment/06-reliability-and-failure-modes.md
一致性视图	AlBrix 的 KV Event Sync 让所有 Pod 看到一致的 cache 状态	08-production-deployment/02-smart-routing-and-load-balancing.md § 4.2
水平扩展 vs 垂直扩展	API Server 多副本（水平）vs Worker 加 TP（垂直）	本篇 § 1
数据本地性	session sticky 路由让 prefix cache 在同一 Pod 里命中	08-production-deployment/02-smart-routing-and-load-balancing.md § 3
异步流水线	Async Scheduler 让本 step 的 schedule() 与上 step 的 forward 并行	本篇 § 4
服务发现 / 负载均衡	Smart Router (llm-d EPP / AlBrix Gateway) 按 cache/load/LoRA 打分	08-production-deployment/02-smart-routing-and-load-balancing.md

7.5 一句话总结

vLLM 不是凭空发明新概念，而是把操作系统的虚拟内存 + 分布式系统的生产者-消费者 + GPU 编程的内存层级感知 + 网络的控制/数据平面分离 这四套成熟思路，针对"自回归 LLM 推理"这个工作负载重新组合。每读到一个看起来高深的设计，先问自己"这在系统课里叫什么？"——80% 的情况你能立刻找到对应原型，剩下 20% 才是 vLLM 真正的新东西（主要是 PagedAttention 论文那一套）。

8. 延伸阅读

按主题展开本篇覆盖的高层概念：

- **进程模型 + IPC 内部机制** → 05-process-and-ipc-internals.md（ZMQ socket 类型、msgpack、shared-memory MessageQueue、PEP 574 OOB 零拷贝、ps -ef 真实进程清单）
- **PagedAttention（虚拟内存类比）** → 02-core-concepts/01-paged-attention.md
- **KV cache 抽象** → 02-core-concepts/03-kv-cache-management.md
- **Scheduler 调度策略** → 03-code-walkthrough/02-scheduler.md
- **TP / PP / EP 通信** → 05-distributed/01-tp-pp-ep.md
- **生产部署形态** → 08-production-deployment/01-deployment-architectures.md

小结

- vLLM 进程拓扑分三层：**API Server**（HTTP / tokenize）→ **EngineCore**（调度 + KV 管理）→ **Worker**（每 GPU 一个进程，跑 forward）。控制平面 ZMQ + msgpack，数据平面 shared-memory MessageQueue。
- 四个核心数据契约：Request（请求状态机）、SchedulerOutput（调度器一步的决策）、KVCacheBlock + BlockTable（KV 的物理 / 逻辑表示）、ModelRunnerOutput（Worker 答卷）——所有调用都围着它们转。
- 单 step 的执行链：schedule() → 广播 SchedulerOutput → Worker forward → Sampler → 回收 token → 进入下一 step；prefill 与 decode 共用同一条链，区别只是 num_scheduled_tokens 是否大于 1。
- V1 的 **AsyncScheduler** 让"本 step 的 CPU 调度"与"上 step 的 GPU forward" overlap，大 batch 下省 5-10% 端到端时间。
- vLLM 80% 的设计在 OS / 网络 / 分布式系统课里都有原型（虚拟内存 / COW / refcount / 控制 vs 数据平面 / 主从模型）——遇到陌生设计先去课本里找原型。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 画出完整请求路径，标 IPC 机制。



三层 IPC 的关键区别：

- **API ↔ EngineCore**：ZMQ + msgpack。低频（每请求 1-2 次），跨机器友好，μs 级序列化。
- **EngineCore ↔ Worker**：共享内存 ring + OOB 零拷贝。高频（每 step 1 次，100+ Hz），KB-MB 级消息，序列化几乎免费。
- **Worker ↔ Worker**：NCCL。每层 forward 1-2 次 AllReduce，必须最高带宽。

详见 05-process-and-ipc-internals.md。

2. 列 SchedulerOutput 至少 5 个字段，说出谁在哪侧用。

源码：vllm/v1/core/sched/output.py

字段	Scheduler 写入	Worker 消费
scheduled_new_reqs : list[NewRequestData]	本步新进入的请求（含 prompt token、采样参数）	Worker 把 prompt 转 tensor、分配 KV slot
scheduled_cached_reqs : CachedRequestData	已 running 请求的 block_ids / num_computed_tokens	Worker 构造 input_batch / attention metadata
num_scheduled_tokens : dict[req_id, int]	每个本步要算的 token 数（prefill chunk / decode = 1）	Worker 据此组装 packed batch
total_num_scheduled_tokens : int	全步 token 总数	Worker 据此 pad / 选 CUDA Graph capture 尺寸
scheduled_spec_decode_tokens : dict[req_id, list[int]]	投机解码的草稿 token	Worker 一起塞进 forward，验证后采纳/拒绝
preempted_req_ids : set[str]	本步被抢占的请求	runner 清理持久 batch 状态；Scheduler 侧已释放 KV 并重置请求进度
finished_req_ids : set[str]	本步完成的请求	Worker 释放 KV、清理 sampling 状态
kv_connector_metadata	P/D 分离 / KV offload 元数据	Worker 据此触发 KV transfer
grammar_bitmask	结构化输出的 token mask	Worker 在 sampler 里 apply

口诀：SchedulerOutput 是"做什么"的契约——Scheduler 决定，Worker 执行。字段越多，Worker 越被动——这是 V1 的核心哲学。

3. 在 EngineCoreProc 里指出 schedule / execute / 回写大约哪几行。

源码：vllm/v1/engine/core.py::EngineCoreProc.run_engine_core（约 line 1087+）。一个 step 大致：

```
# 伪代码，实际行号随版本变动
def step(self):
    # ① schedule (CPU 上算, ~3 ms)
    scheduler_output = self.scheduler.schedule()          # vllm/v1/core/sched/scheduler.py

    # ② execute_model (GPU forward + sample, ~30 ms)
    model_runner_output = self.executor.execute_model(
        scheduler_output
    )                                                    # vllm/v1/executor/multiproc_executor.py
                                                         # 内部调 Worker 的 gpu_model_runner.py

    # ③ 把输出 (token_ids / logprobs / finished) 写回 Scheduler 状态
    self.scheduler.update_from_output(
        scheduler_output, model_runner_output
    )                                                    # vllm/v1/core/sched/scheduler.py::_update_request_with_output

    # ④ finished 请求结果通过 ZMQ 推回 API Server
    for output in self.collect_finished_outputs():
        self.output_socket.send(...)
```

定位方法：在 vllm/v1/engine/core.py 里 grep def step、schedule、execute_model、update_from_output。

4. V1 为什么多进程而不是多线程？至少 3 条理由。

理由	解释
Python GIL	多线程下 schedule / sampler / detokenizer 等 Python 逻辑还是串行；多进程让每个 Worker 有独立 GIL，真正并发
CUDA context 隔离	每个 CUDA context 绑一个进程；多进程让每张 GPU 有独立 context，崩溃互不牵连
故障域隔离	Worker OOM / segfault 不会拖垮 EngineCore；可以重启单个 Worker（NCCL 重建除外）
NCCL 习惯	业界标准是 one-rank-per-process；多线程跑 NCCL 历史上有死锁 bug
CUDA Graph 兼容	每张 GPU 一个 stream + capture context，多进程天然契合
资源会计清晰	ps / top / nvidia-smi 直接看每 Worker 的 CPU/GPU 占用

代价：进程间通信比线程贵——这就是为什么数据平面（EngineCore ↔ Worker）用**共享内存 + 零拷贝**而不是 ZMQ。

5. async scheduler 节省多少 ms？

前提：单 step forward = 30 ms（GPU），schedule = 3 ms（CPU）。

同步（V0）：

```
step n: [sched 3ms][forward 30ms][sched 3ms][forward 30ms]...
单 step = 3 + 30 = 33 ms
```

async（V1）：

```
step n: CPU [sched 3ms]
      ↓ 启动 forward 后立即返回
      CPU [sched next 3ms]      ← 与 GPU forward 并行
      GPU [forward 30ms      ]
单 step = max(30, 3) = 30 ms
```

CPU schedule 完全藏在 GPU forward 内 (3 ms < 30 ms)。每步省 3 ms，相对 33 ms = **9.1%**。

收益与 schedule / forward 比值有关：

- schedule 重 (大 batch + 复杂调度，可能 5-10 ms) → async 收益最大
- schedule 轻 (小 batch + FCFS, 1 ms) → async 几乎看不到收益

下一步

- 下一节：03-v0-vs-v1.md (讲清 V0 → V1 重构的 5 大动机，避免你读错代码)。
- 想深挖进程模型：05-process-and-ipc-internals.md (ZMQ socket 类型、shared-memory MessageQueue、零拷贝 trick、ps -ef 真实进程清单)。
- 想看源码：vllm/v1/engine/core.py (EngineCore 主体)、vllm/v1/core/sched/scheduler.py (调度入口)、vllm/v1/worker/gpu_model_runner.py (forward 主体)。
- 想动手：07-hands-on/04-profiling-and-debugging.md (用 NVTX / py-spy 验证本节讲的执行链是否真的在跑)。
- 想从生产视角理解：08-production-deployment/01-deployment-architectures.md (三层进程在 K8s LeaderWorkerSet 里如何映射成 Pod / 容器)。

03. V0 → V1：vLLM 的架构重构

谁该读这一篇？ 打开 vLLM 源码看到 vllm/engine/ 和 vllm/v1/ 两个目录犹豫"该读哪个"的读者；想系统理解 V0 和 V1 差异的工程师。

前置阅读： 02-architecture.md（先建立三层架构与单 step 执行链的脑图，再来看哪些地方被 V1 重写）。

耗时： 约 10 分钟。

学完能：

- 1. 看到 vllm/v1/... 就知道是当前主路径，看到 vllm/engine/... 就知道是兼容层（少数 CPU 后端还在）。
- 2. 用 5 条变化（统一调度 / 持久 InputBatch / KV 解耦 / AsyncScheduler / 进程结构）讲清 V1 重构动机。
- 3. 对照伪代码说出 V0 "两阶段调度" 与 V1 "token budget" 的本质差异。
- 4. 在源码目录里快速找到 V0 → V1 的对应文件迁移路径。

当前源码复核（b23bd73）： 新的生成式推理主线以 V1 为准，但“路径不含 /v1/ 就是 V0”已经不是可靠判据：配置、入口、模型层、分布式和通用工具仍是共享模块。迁移判断应以调用关系和类契约为准，而不是只看目录名。

1. 现在的 vLLM 是 V1

vllm/v1/ 是当前主路径（2024 年中重构，2024 年底基本完成）。vllm/engine/ 是 V0 兼容层，未来会移除。

学习引擎与调度时优先读 V1。 但入口、配置、模型实现、分布式和算子仍大量位于 vllm/ 的共享目录；不要把“V1 主路径”误读为“只需要读 vllm/v1/”。本笔记未特别说明时，Engine 与 Scheduler 指 V1 实现。

2. V0 的痛点

V0 是 PagedAttention 论文那一版的实现。两年迭代下来积累了几个结构性问题：

V0 痛点	后果
Scheduler 把 prefill 与 decode 分两段调度	长 prefill 阻塞短 decode，TTFT/TPOT 矛盾
KV cache 管理与 Sequence 紧耦合	Prefix caching / 多模态 / MLA 接入困难
GPUModelRunner 嵌大量 "是不是 prefill" 的分支	Chunked prefill 实现极其复杂
单 EngineLoop 同步 schedule + 前向	Python 调度开销难以隐藏
每步重建输入 batch tensor	大 batch 下 CPU overhead > 10 ms
多模态、spec decoding、disaggregated 都靠 hack	代码越来越烂

3. V1 的五大改变

记住这五条，能覆盖 80% 的 "V0 vs V1" 提问。

3.1 统一调度：prefill 与 decode 不再分两阶段

V0：先把所有能 prefill 的请求跑完，再做 decode。结果是长 prompt 卡住所有 decode。V1：每一步混合调度。每个请求带 num_scheduled_tokens，prefill = 多 token、decode = 1 token，全部混在同一个 forward。

Chunked prefill 在 V1 是**默认行为**（V0 是 opt-in）。

3.2 持久化输入 Batch（InputBatch）

V0 每步重建 tensor，伴随大量 CPU→GPU 小张量拷贝。V1 引入 InputBatch（vllm/v1/worker/gpu_input_batch.py），把请求 ID → 行索引的映射稳定下来，每步只更新 diff（新增/删除的行）。

效果：大 batch 下 CPU overhead 降低一个数量级。

3.3 KV cache 抽象与 Sequence 解耦

V0 的 Sequence 同时管 token、KV 索引、metadata，臃肿且难扩展。V1 把 KV 管理彻底独立到 vllm/v1/core/kv_cache_manager.py + block_pool.py：

- Request 只持有 token_ids 和 sampling_params
- KVCacheManager 单独管 block 分配 / 释放 / 复用

副作用：

- Prefix caching 在 manager 内部就能闭环
- MLA（DeepSeek 的注意力变体）只需换一种 manager
- 多种 KV 类型（普通 paged KV + Mamba state）可以共存

3.4 异步调度（AsyncScheduler）

让 schedule()（纯 CPU）与上一步的 forward（GPU）overlap，挤出 5-10% 端到端时间。文件：vllm/v1/core/sched/async_scheduler.py。

3.5 进程结构清晰化

V0 进程边界混乱，API server / engine / worker 经常融合。V1 严格分三层，进程间走 ZMQ + Ray，无共享状态，方便横向扩展：

flowchart LR

```
A["API Server<br/>(HTTP)"] -->|ZMQ| B["EngineCore<br/>(调度)"]
B -->|Ray / multiproc| C["Worker(s)<br/>(前向 + 采样)"]
```

```
classDef proc fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
class A,B,C proc;
```


4. 调度循环对比

V0 的伪代码（示意）：

```
def step():  
    # 阶段 1: 尽可能多地 prefill  
    while waiting_queue and have_kv_space():  
        req = waiting_queue.pop()  
        run_prefill(req)    # 一次跑完整 prompt  
  
    # 阶段 2: 所有 running 做 decode  
    for req in running:  
        run_decode(req)    # 每个跑 1 token
```

V1 的伪代码：

```
def step():  
    # 一次性决定每个请求本步算几个 token  
    scheduled = {}  
    token_budget = max_batched_tokens # 比如 8192  
  
    # 先保 running（已经在跑的优先）  
    for req in running:  
        n = min(req.remaining_prompt + req.decode_steps, token_budget)  
        scheduled[req] = n  
        token_budget -= n  
  
    # 再看 waiting 能不能塞进来  
    for req in waiting:  
        if can_allocate_kv(req) and token_budget > 0:  
            n = min(req.prompt_len, token_budget)  
            scheduled[req] = n  
            token_budget -= n  
            running.append(req)  
  
    # 一次 forward 跑完所有 scheduled[req] 个 token  
    run_forward(scheduled)
```

V1 没有 "prefill 阶段" 与 "decode 阶段" 的概念，只有"这一步算多少 token"。这是设计层面的本质差异。

5. 代码地图对照

功能	V0 路径	V1 路径
Engine	vllm/engine/llm_engine.py	vllm/v1/engine/llm_engine.py
Scheduler	vllm/core/scheduler.py（已删）	vllm/v1/core/sched/scheduler.py
Block Mgr	vllm/core/block_manager.py（已删）	vllm/v1/core/kv_cache_manager.py
Worker	vllm/worker/（部分留作 CPU 后端）	vllm/v1/worker/

V0 的目录已经被渐进式删除得很精简——这是迁移的痕迹。

6. 实践建议

1. **不要读 V0 代码**，除非考古或解决特定历史问题。
2. PagedAttention 论文（SOSP'23）讲的是 V0 思想，但实现细节已变。**思想不变，代码全换**。
3. 解释 "V0/V1 区别" 时，按上面 5 点框架展开，最后落一句话："本质是把 *调度策略与执行机制* 彻底解耦"。

小结

- **现在的 vLLM 是 V1**：vllm/v1/ 是主路径，vllm/engine/ 只是兼容层；学习时只读 V1。
- 五大重构动机：①统一调度（chunked prefill 默认开）②持久 InputBatch（增量更新）③KV 与 Sequence 解耦（KVCacheManager 独立）④AsyncScheduler（CPU 调度与 GPU forward overlap）⑤进程结构清晰化（API / Engine / Worker 严格分层）。
- V1 没有 "prefill 阶段 / decode 阶段" 的概念，只有 "这一步 token budget 内算多少 token"——这是与 V0 最根本的设计差异，也是 chunked prefill 自然落地的前提。
- 一句话概括："V1 把 *调度策略与执行机制* 彻底解耦"。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. vllm/v1/core/sched/scheduler.py VS vllm/core/scheduler.py，哪个是主路径？

vllm/v1/core/sched/scheduler.py 是当前主路径（V1 实现，~2300 行）。vllm/core/scheduler.py **已被删除**——V0 调度器已不存在。判断方法：

- 文件存在与否：ls vllm/core/scheduler.py 大概率 404
- 看 vllm/v1/engine/core.py 里 import 的是 vllm.v1.core.sched.scheduler.Scheduler
- vllm/engine/llm_engine.py 现在只是 V1 LLMEngine 的兼容包装

→ **判断 "V0 vs V1" 的可靠方法**：从公开入口沿调用链确认对象最终落到 vllm.v1.engine / vllm.v1.core.sched，同时承认配置、模型与分布式代码是共享层；目录名只能当线索。

2. V0 vs V1 step() 伪代码对比。

V0（两阶段循环）：

```
def step():
    if waiting and current_phase == "decode":
        # 切换到 prefill phase, 吃掉一批新请求
        for req in waiting[:batch_size]:
            prefill_forward(req)          # 整段 prefill 一次跑完
            running.append(req)
        current_phase = "decode"
    else:
        # 全员 decode 一步
        for req in running:
            decode_forward(req, 1_token)  # decode 时 prefill 阻塞
```

问题：长 prefill 来一个就把所有 running 的 decode 卡几百 ms。

V1 (token budget 单阶段)：

```
def step():
    budget = MAX_NUM_BATCHED_TOKENS      # 比如 8192
    work = {}                             # req_id -> num_tokens_this_step

    # 先给所有 running 各 1 个 decode token
    for req in running:
        if budget > 0:
            work[req.id] = 1
            budget -= 1

    # 剩余 budget 给 prefill (新请求或老请求未完 prefill 段)
    for req in waiting + still_prefilling:
        n = min(req.remaining_prefill_tokens, budget)
        if n > 0:
            work[req.id] = n
            budget -= n

    # 一次 forward 跑完所有 work
    forward(work)                         # prefill + decode 混在同一个 packed batch
```

→ 因为 attention metadata (query_start_loc / seq_start_loc / block_table 等) 能描述每个请求各算几个 token，同一次 forward 自然支持 prefill chunk + decode token 混合。这就是 chunked prefill 默认开的物理基础。

3. KVCacheManager 解耦后，哪些新特性各从哪个点获益？

特性	受益的解耦点
Prefix caching	KV manager 知道"block 内容用 hash 标识", 与"请求归属"解耦。可以多请求共享同一物理 block (ref_cnt++)
MLA (DeepSeek 多头潜在)	KV manager 不假设 KV 形状是 [2, num_blocks, ...]; MLA 的 latent KV 形状是 [num_blocks, latent_dim], 照样能 plugin 进 KVCacheSpec
Mamba state	Mamba 不存"K/V"而是 SSM hidden state (每层一个固定大小张量); KV manager 把它建模为另一种 KVCacheSpec 即可, scheduler 不用改
Hybrid (Jamba 类)	同一个模型既有 attention KV 又有 Mamba state, KV manager 用 KVCacheGroup 同时管多种 spec
P/D 分离 / KV offload	KV manager 内部有 KVConnectorMetadata 接口, 物理 block 可以在远端, 不影响 scheduler 决策

抽象的好处： scheduler 完全不知道 KV 物理形状是什么——它只问"能不能给我 K 个 block"。这让上层调度策略和底层存储格式独立演进。

4. AsyncScheduler 优化谁？省百分之几？

优化的是 **CPU**（调度逻辑）。本质是让本步的 CPU schedule 与上一步的 GPU forward 重叠。

同步：CPU [sched 3ms] GPU [forward 30ms] CPU [sched 3ms] GPU [forward 30ms]...
← 33 ms per step →

异步：CPU [sched 3ms→] GPU [forward 30ms]
 CPU [sched 3ms→ next] GPU [forward 30ms]
← 30 ms per step →

收益 = schedule 时长 / (schedule + forward)。 $3 / 33 \approx 9.1\%$ 。

大 batch（schedule 5-10 ms）下能到 **15-25%**；小 batch（schedule 1 ms）几乎无收益。

详见 02-architecture.md § 6 与本节自检题 5。

5. V0 fork 升 V1，至少要重写哪 4 个文件？

文件	改动方向
vllm/engine/llm_engine.py → vllm/v1/engine/llm_engine.py	重写为 V1 风格的 engine wrapper（调用 EngineCore 进程而非内嵌）
vllm/core/scheduler.py → vllm/v1/core/sched/scheduler.py	token budget 单阶段调度，废弃 phase 概念
vllm/core/block_manager.py → vllm/v1/core/kv_cache_manager.py + block_pool.py	抽象 KVCacheSpec，分离 block 物理池与请求逻辑
vllm/worker/worker.py → vllm/v1/worker/gpu_worker.py + gpu_model_runner.py	进程化 Worker，消费 SchedulerOutput 而非自己组装 batch

顺手检查：**还要改的隐藏文件：**

- vllm/sequence.py / vllm/v1/request.py：Request 状态机重写（V0 的 SequenceGroup 被简化）
- vllm/attention/* → vllm/v1/attention/backends/*：attention metadata 需要支持 packed batch + chunked prefill
- 各 attention backend（FlashAttention / FlashInfer / Triton / MLA）的 metadata 构造逻辑全部要重写

→ 实际上 vLLM 团队走的不是 fork rewrite，是在主仓库做 **V0 → V1 渐进式迁移**，旧代码删除而非保留，这是为什么 vllm/core/ 现在基本是空目录。

下一步

- 下一节：04-project-structure.md（V1 代码在仓库里到底分布在哪——读源码前的最后一张地图）。
- 跳到核心实现：02-core-concepts/01-paged-attention.md（架构铺垫完，正式进入 vLLM 的心脏）。

- 想看源码：`vllm/v1/engine/core.py` (V1 EngineCore)、`vllm/v1/core/sched/scheduler.py` (V1 调度核心)、`vllm/v1/core/sched/async_scheduler.py` (async 优化)。
- 想从生产视角理解：`08-production-deployment/06-reliability-and-failure-modes.md` (V0 → V1 升级带来的故障模式变化，以及上线前需要验证的回归项)。

04. vLLM 项目文件架构地图

谁该读这一篇？ 第一次 clone 完 vLLM 仓库、被持续增长的 Python 与 CUDA/C++ 文件吓退的同学；后续每一篇源码走读前，需要先知"我要找的东西在哪一级目录"的读者。

前置阅读： 03-v0-vs-v1.md（知道为什么本节几乎只讲 vllm/v1/ 而不讲 vllm/engine/）。

耗时： 约 25 分钟（建议第一遍**通读 § 1-3 + § 21**，找东西时再回来查具体子目录）。

学完能：

- 1. 看到任何一个 vllm/... 路径能立刻说出"它属于 V1 引擎 / 模型层 / 多模态 / 量化 / 量化算子 / 配置 / 入口"等大类。
- 2. 拿到"我要改 X"的需求，能直接在脑子里翻到 § 21 的速查表，定位首要修改文件。
- 3. 区分 vllm/v1/、vllm/model_executor/、vllm/csrc/、vllm/distributed/ 四大重量级目录各自的边界与上下游。
- 4. 不会再把 csrc/attention/ 与 vllm/v1/attention/backends/ 搞混（前者是 CUDA kernel，后者是 Python backend 接口）。

当前源码复核 (b23bd73)： 目录职责和关键入口已按锁定树核对。文件/模型数量属于瞬时统计，本章不再把它们当架构事实；需要数量时请在同一 commit 上运行 find vllm -name '*.py'、find csrc -type f 等可复现命令。

vLLM 仓库规模很大且会随主线快速变化。第一次打开仓库容易迷路。本节给你一张"地图"——每个目录、每个关键文件干什么。找东西时回头查这一篇就行。

1. 仓库顶层 (/Users/zjw/Documents/github-project/vllm/)

```
vllm/
├── vllm/          ← Python 主包（重点）
├── csrc/          ← C++/CUDA 内核
├── benchmarks/    ← 性能基准
├── tests/         ← 单元测试 + 集成测试
├── examples/      ← 使用示例（offline / online / 多模态 / tool use）
├── docs/          ← 官方文档源（mkdocs）
├── docker/        ← Dockerfile（CUDA / ROCm / TPU / CPU）
├── cmake/         ← C++ 编译配置
├── requirements/  ← 各平台 pip 依赖
├── scripts/       ← 开发辅助脚本
├── tools/         ← 内部工具
├── third_party/   ← 第三方代码
├── setup.py       ← Python 包安装（含 C++ 编译）
├── pyproject.toml ← Python 项目配置
├── CMakeLists.txt ← 整体 CMake 入口
└── README.md / AGENTS.md / CLAUDE.md / CONTRIBUTING.md ...
```

2. vllm/ 包：顶层文件

打开 vllm/__init__.py 暴露的就是这些公共类。重要的顶层模块：

文件	作用
__init__.py	顶层导出 LLM、SamplingParams、PoolingParams 等
version.py	版本号
envs.py	所有环境变量注册表 （vLLM_*、NCCL、CUDA）
env_override.py	平台特定环境覆盖
sampling_params.py	SamplingParams、StructuredOutputsParams（OpenAI 协议入口）
pooling_params.py	PoolingParams（embedding/reranker 协议入口）
outputs.py	RequestOutput、CompletionOutput 等用户可见数据类
sequence.py	旧版 Sequence（V0 残留，V1 不用）
tasks.py	任务类型枚举（generate / embed / classify / score / rerank）
scalar_type.py	量化用的标量类型枚举（INT4 / FP8 / NF4 ...）
_custom_ops.py	把 C++ kernel 暴露给 Python（torch.ops.vllm.*）
_aiter_ops.py / _xpu_ops.py / _tilelang_ops.py	对应 ROCm AITER / XPU / TileLang 后端
forward_context.py	运行时上下文 （attention metadata 注入用）
logits_process.py	用户自定义 LogitsProcessor 框架
logprobs.py	logprobs 公共数据类型
beam_search.py	Beam search（V1 已弃用）
connections.py	HTTP 会话池工具
logger.py	日志初始化
model_inspection.py	模型架构识别（HF config → vLLM model 类）
exceptions.py	自定义异常
collect_env.py	vllm collect-env 命令的实现
scripts.py	CLI 入口（vllm serve / vllm bench）

3. vllm/v1/：V1 引擎（重点中的重点）

v1/

- ├── __init__.py
- ├── cudagraph_dispatcher.py ← runtime size → captured graph 路由
- ├── kv_cache_interface.py ← KVCacheSpec (多 KV 类型抽象)
- ├── outputs.py ← EngineCoreOutput / ModelRunnerOutput
- ├── request.py ← Request 类 (状态机: WAITING/RUNNING/...)
- ├── serial_utils.py ← ZMQ msgpack 编码
- ├── utils.py
- ├── attention/
- ├── core/
- ├── engine/
- ├── executor/
- ├── kv_offload/
- ├── metrics/
- ├── pool/
- ├── sample/
- ├── simple_kv_offload/
- ├── spec_decode/
- ├── structured_output/
- └── worker/

3.1 v1/engine/：引擎门面与异步入口

文件	作用
llm_engine.py	同步 LLMEngine (offline 入口, 门面)
async_llm.py	AsyncLLM (在线服务用, asyncio queue)
core.py	EngineCore —— 真正的调度引擎进程主体
core_client.py	EngineCoreClient —— ZMQ 客户端
coordinator.py	DP / EP 跨实例协调
detokenizer.py	token_id → 文本 (streaming)
input_processor.py	prompt 预处理 + 多模态 input 准备
output_processor.py	把 EngineCoreOutput → 用户可见 RequestOutput
parallel_sampling.py	n>1 时的并行采样
tensor_ipc.py	跨进程传 tensor
logprobs.py	logprobs 公共逻辑
exceptions.py	
utils.py	

3.2 v1/core/：调度与 KV 管理

路径	作用
core/sched/scheduler.py	核心 Scheduler.schedule() (2300+ 行)
core/sched/async_scheduler.py	AsyncScheduler (schedule 与 forward overlap)
core/sched/interface.py	Scheduler 抽象基类
core/sched/output.py	SchedulerOutput 数据类
core/sched/request_queue.py	优先级队列 / FCFS 队列
core/sched/utils.py	
core/block_pool.py	BlockPool —— 物理 KV block 中央仓库
core/kv_cache_manager.py	KVCacheManager —— Scheduler 看到的接口
core/kv_cache_coordinator.py	多 KV 类型协调器 (普通/MLA/Mamba 共存)
core/kv_cache_metrics.py	KV 指标采集
core/kv_cache_utils.py	hash 计算、KVCacheBlock
core/single_type_kv_cache_manager.py	单种 KV 类型的 manager 基类 (Full/Sliding/Mamba)
core/encoder_cache_manager.py	多模态 encoder 输出缓存

3.3 v1/worker/: Worker 进程与模型执行

文件	作用
worker_base.py	Worker 抽象基类（所有平台共用接口）
gpu_worker.py	GPU Worker （init + load_model + execute_model）
gpu_model_runner.py	GPUModelRunner （3000+ 行，每步 forward 本体）
gpu_input_batch.py	持久化 InputBatch（V1 性能关键）
cpu_worker.py / cpu_model_runner.py	CPU 后端
xpu_worker.py / xpu_model_runner.py	Intel XPU 后端
gpu_ubatch_wrapper.py	micro-batching 包装
block_table.py	GPU 上的 block_table tensor 维护
kv_connector_model_runner_mixin.py	KV connector 集成 mixin
ec_connector_model_runner_mixin.py	encoder cache connector mixin
lora_model_runner_mixin.py	LoRA 集成 mixin
dp_utils.py / cp_utils.py	DP / Context-Parallel 工具
ubatching.py / ubatch_utils.py	micro-batching
encoder_cudagraph.py	encoder 路径的 CUDA Graph
mamba_utils.py	Mamba state 处理
workspace.py	显存 workspace 管理
gpu/	GPU-specific 子模块（mm / model_states / pool / sample / spec_decode）

3.4 v1/executor/：Worker 编排（每种部署一份）

文件	作用
abstract.py	Executor 接口
uniproc_executor.py	单进程单卡
multiproc_executor.py	单机多卡（multiprocessing + shared memory）
ray_executor.py / ray_executor_v2.py	多机多卡 Ray
ray_utils.py / ray_env_utils.py	Ray 辅助

3.5 v1/attention/：Attention 后端

路径	作用
backend.py	后端基类（AttentionBackend / Impl / MetadataBuilder）
backends/registry.py	所有后端枚举与按需 import
backends/flash_attn.py	FlashAttention v2/v3 （默认）
backends/flashinfer.py	FlashInfer（H100 decode 小 batch 优选）
backends/triton_attn.py	Triton fallback
backends/rocm_attn.py / rocm_aiter_*.py	AMD ROCm
backends/cpu_attn.py	CPU 后端
backends/flex_attention.py	torch FlexAttention
backends/turboquant_attn.py	量化 attention
backends/{mamba1,mamba2,mamba}_attn.py	Mamba SSM
backends/gdn_attn.py / linear_attn.py / short_conv_attn.py	线性注意力变体
backends/mla/	MLA 子目录 （DeepSeek）
flashmla.py / flashattn_mla.py / cutlass_mla.py	NVIDIA MLA kernel 套件
flashinfer_mla.py / triton_mla.py	备选
rocm_aiter_mla*.py / xpu_mla_sparse.py	其他硬件
selector.py	自动选择 backend
ops/	Triton attention ops

3.6 v1/sample/：采样

文件	作用
sampler.py	Sampler （forward 入口、温度、greedy/random 合并）
rejection_sampler.py	spec decode 接受/拒绝采样
metadata.py	SamplingMetadata（打包整 batch）
tpu_sampler.py	TPU 版本
ops/topk_topp_sampler.py	top-k / top-p 入口
ops/topk_topp_triton.py	Triton kernel
ops/penalties.py	repetition / presence / frequency penalty
ops/bad_words.py	bad_words mask
ops/logprobs.py	logprobs 工具（fused log_softmax + gather）
logits_processor/	自定义 logits processor 框架

3.7 v1/structured_output/：JSON / Grammar 约束

文件	作用
__init__.py	StructuredOutputManager （engine-level 调度）
backend_types.py	基类（StructuredOutputBackend、StructuredOutputGrammar）
backend_xgrammar.py	xgrammar（默认）
backend_guidance.py	Microsoft llguidance
backend_outlines.py	outlines
backend_lm_format_enforcer.py	lm-format-enforcer
request.py	StructuredOutputRequest（per-request 状态）
utils.py	

3.8 v1/spec_decode/：投机解码

文件	作用
llm_base_proposer.py	LLM-based draft proposer 基类
draft_model.py	独立 draft model；可选跨词表 TLI
ngram_proposer.py / ngram_proposer_gpu.py	CPU / GPU n-gram proposal
suffix_decoding.py	基于 prompt 与历史生成 suffix tree 的动态 proposal
eagle.py	EAGLE / EAGLE3 hidden-state proposer
dflash.py	DFlash 并行 masked-query block proposer
gemma4.py / step3p5.py	模型家族专用 MTP proposer
medusa.py	Medusa 兼容路径
custom_class_proposer.py	实验性自定义 proposer 装载入口
vocab_mapping.py	异构 vocabulary 的 Token-Level Intersection 映射
dynamic/	dynamic speculative decoding 的 batch-size schedule 辅助
metadata.py	共享数据结构
utils.py	rejection sampler 数学辅助

验证与采样还分布在 `v1/sample/rejection_sampler.py`；Model Runner V2 的方法专用 `speculator` 和 `block verification` 位于 `v1/worker/gpu/spec_decode/`。完整调用链见 `04-optimizations/02-speculative-decoding.md`。

3.9 v1/pool/：embedding/pooling 模型用

文件	作用
metadata.py	PoolingMetadata
late_interaction.py	ColBERT-style late interaction

3.10 v1/kv_offload/、v1/simple_kv_offload/：KV 卸载

KV 从 GPU HBM offload 到 CPU 内存 / NVMe 的实现：

- `kv_offload/cpu/` —— CPU memory offload
- `kv_offload/cpu/policies/` —— LRU / LFU 等
- `kv_offload/tiering/` —— 多层（GPU → CPU → 远端）
- `kv_offload/worker/` —— Worker 侧 hooks
- `simple_kv_offload/` —— 简单实现（学习用）

3.11 v1/metrics/：Prometheus / 日志

文件	作用
loggers.py	stat logger（周期性打印 throughput / KV usage）
prometheus.py	vllm:* Prometheus 暴露
stats.py	统计数据类（PrefixCacheStats 等）

4. vllm/model_executor/：模型与算子层

model_executor/	
├── custom_op.py	← @CustomOp 装饰器（注册到 torch.library）
├── parameter.py	← 参数加载工具
├── utils.py	
├── kernels/	← 调度到不同 kernel 后端
├── layers/	← 各种 layer 实现
├── model_loader/	← 模型权重加载
├── models/	← **292 个模型实现**
├── offloader/	← 权重 offload 到 CPU
└── warmup/	← 启动 warmup

4.1 model_executor/layers/：算子层

路径	作用
linear.py	ColumnParallelLinear / RowParallelLinear / ReplicatedLinear
activation.py	SiLU / GeLU / SwiGLU
layernorm.py	RMSNorm / fused versions
rotary_embedding/	RoPE 各种变种（YaRN / NTK / Linear / decoupled）
vocab_parallel_embedding.py	词表并行 embedding
logits_processor.py	LM head + logits processor
attention/	Attention layer 上层包装（不是 backend）
attention_layer_base.py	Attention 基类
mla.py	MLA 包装层（DeepSeek）
mhc.py / kda.py	Multi-head conv / Kernel decomposition
mamba/	Mamba mixer
fla/	Flash Linear Attention
lightning_attn.py / linear_attn.py	线性注意力
sparse_attn_indexer.py	sparse attention
deepseek_compressor.py / deepseek_v4_attention.py	DeepSeek 专属
conv.py	1D / 2D conv（Mamba 等用）
resampler.py	Perceiver-style resampler（多模态）
batch_invariant.py	Batch-invariant 模式
fused_moe/	MoE 实现
pooler/	Pooling（embedding 模型用）
quantization/	量化集成

4.2 model_executor/layers/fused_moe/：MoE 层

文件	作用
layer.py	FusedMoE 主层
fused_moe.py	核心 grouped GEMM 调度
fused_moe_method_base.py	MoE method 基类
fused_moe_modular_method.py	模块化实现
moe_align_block_size.py	token → expert 对齐
experts/	每种量化的 expert 实现（FP8 / AWQ / GPTQ）
all2all_utils.py	EP 用 AllToAll
expert_map_manager.py	EP 下 expert ↔ rank 映射
deep_gemm_utils.py	H100 deep GEMM
cpu_fused_moe.py	CPU MoE
activation.py	
modular_kernel.py	
config.py / configs/	

4.3 model_executor/layers/quantization/： 量化

文件	作用
base_config.py	QuantizationConfig 基类
schema.py	
kv_cache.py	KV cache 量化（FP8 / INT8）
fp8.py	FP8 (E4M3 / E5M2)
fp_quant.py	
bitsandbytes.py	NF4 / FP4（QLoRA 同款）
awq.py / awq_marlin.py / awq_triton.py	AWQ INT4
auto_gptq.py	GPTQ
gguf.py	GGUF（llama.cpp 兼容）
mxfp4.py	Microscaling FP4
fbgemm_fp8.py	Meta FBGEMM
modelopt.py	NVIDIA Model Optimizer
torchao.py	TorchAO
experts_int8.py / moe_wna16.py	MoE 专用
cpu_wna16.py	CPU 量化
inc.py	Intel Neural Compressor
humming.py	
compressed_tensors/	Neural Magic 统一格式
online/	Online quantization
quark/	Quark format
turboquant/	TurboQuant
utils/	

4.4 model_executor/model_loader：权重加载

文件	作用
base_loader.py	Loader 接口
default_loader.py	HF safetensors（默认）
gguf_loader.py	GGUF
bitsandbytes_loader.py	bnb
tensorizer.py / tensorizer_loader.py	Tensorizer（CoreWeave 快速加载）
runai_streamer_loader.py	RunAI Model Streamer（S3 流式）
sharded_state_loader.py	分片格式（训练 checkpoint）
dummy_loader.py	不加载，仅 benchmark
weight_utils.py	权重转换工具
ep_weight_filter.py	EP 下按 expert 过滤权重
utils.py	
reload/	在线重载权重

4.5 model_executor/models/：模型实现（数量以锁定树扫描为准）

按家族分类（这里只列有代表性的）：

家族	文件举例
Llama 系	llama.py / llama4.py / tinyllama.py
Qwen 系	qwen2.py / qwen3.py / qwen2_vl.py / qwen2_audio.py
DeepSeek 系	deepseek_v2.py / deepseek_v3.py / deepseek_v4.py （含 MLA + MoE + MTP）
Mistral 系	mistral.py / mixtral.py / mistral3.py （含 Mixtral MoE）
GLM / ChatGLM	chatglm.py
Phi 系	phi.py / phi3.py / phi3v.py / phi4.py
Gemma	gemma.py / gemma2.py / gemma3.py / gemma4.py
BERT / embedding	bert.py / bge_m3.py / jina_embeddings_v3.py
Mamba 系	mamba.py / mamba2.py / jamba.py / zamba2.py
多模态	qwen2_vl.py / llava.py / phi3v.py / internvl.py / aria.py / chameleon.py
Whisper	whisper.py （encoder-decoder）
工具	adapters.py / __init__.py （家族注册）

每个模型文件结构相似：

```
class XxxAttention(nn.Module): ...    # 用 vLLM Attention + 量化 Linear
class XxxMLP(nn.Module): ...
class XxxDecoderLayer(nn.Module): ...
class XxxModel(nn.Module): ...      # stack of DecoderLayer
class XxxForCausalLM(nn.Module): ... # add LM head + forward 入口
```

5. vllm/multimodal/：多模态输入

文件	作用
inputs.py	MultiModalKwargs / PlaceholderRange / MultiModalFieldElem （1015 行）
registry.py	MultiModalRegistry （模型 ↔ processor 绑定）
image.py	image 输入解析 （PIL / np / bytes / URL）
video.py	video 帧抽样 / 时序重采样 （1055 行）
audio.py	waveform 加载与切片
hasher.py	mm 内容哈希 （影响 prefix caching）
cache.py	输入侧 cache
encoder_budget.py	encoder 算力 budget
evs.py	
parse.py	OpenAI message → mm_data 解析
media/	子模块
processing/	每模型一份 Processor
utils.py	

6. vllm/lora/：LoRA 适配器

路径	作用
model_manager.py	LoRAModelManager / LRUCacheLoRAModelManager（1057 行）
worker_manager.py	Worker 侧 LoRA 管理（add/remove/pin）
lora_model.py	LoRA 权重容器
lora_weights.py	权重 dataclass
peft_helper.py	PEFT 格式适配
request.py	LoRARequest
resolver.py	解析 lora_path（local / HF / S3）
utils.py	
layers/	各种 Linear 的 LoRA 包装（column/row/replicated/MoE）
punica_wrapper/	Punica kernel 入口（base/gpu/cpu/tpu）
ops/	Triton bgmv kernels

7. vllm/distributed/：分布式

路径	作用
parallel_state.py	核心 TP / PP / EP 进程组初始化
communication_op.py	AllReduce / AllGather wrapper
device_communicators/	NCCL / HPU / TPU / CPU 通信后端
kv_transfer/	KV connector（NIXL / LMCache / Mooncake）
kv_events.py	KV cache 事件
weight_transfer/	跨进程 weight 传输（训练 + 推理同卡复用）
ec_transfer/	encoder cache 传输
eplb/	Expert Parallel Load Balancer
elastic_ep/	Elastic EP（动态 expert 扩缩）
nixl_utils.py	NIXL RDMA 工具
stateless_coordinator.py	无状态 DP 协调器
utils.py	

8. vllm/entrypoints/: API 入口

路径	作用
llm.py	LLM 类（offline 入口）
api_server.py	通用 API server
launcher.py	vllm serve 启动器
grpc_server.py	gRPC 入口
cli/	命令行入口（vllm serve / bench / chat / ...）
openai/	OpenAI 兼容 （绝大多数生产路径）
api_server.py	FastAPI app
chat_completion/ / completion/ / responses/	各 endpoint
generative_scoring/	生成式打分
models/	/v1/models endpoint
engine/	与 EngineCore 交互
cli_args.py	启动参数
run_batch.py	批量推理
server_utils.py	工具
parser/	request 解析
orca_metrics.py	ORCA 协议（负载报告给 LB）
anthropic/	Anthropic 协议兼容
mcp/	MCP（Model Context Protocol）兼容
pooling/	Embedding / rerank endpoints
sagemaker/	SageMaker 适配
speech_to_text/	Whisper 风格 ASR
serve/	Service 框架
chat_utils.py	chat template 应用
logger.py	请求日志
ssl.py	TLS 设置
utils.py / constants.py	

9. vllm/compilation/： torch.compile 集成

文件	作用
backends.py	VllmBackend / CompilerManager / split_graph （1331 行）
compiler_interface.py	CompilerInterface / InductorAdaptor （782 行）
cuda_graph.py	CUDA Graph 包装
caching.py	编译缓存
codegen.py	代码生成辅助
decorators.py	@CustomOp 注册自定义 op
partition_rules.py	切图规则
piecewise_backend.py	piecewise 编译
wrapper.py	顶层 wrapper
monitor.py	编译监控
counter.py	
base_static_graph.py	
passes/	自定义 pass
vllm_inductor_pass.py	主 pass 集合
pass_manager.py	pass 串联
fusion/	AllReduce / activation / attention fusion
fx_utils.py / inductor_pass.py	
utility/	

10. vllm/config/： 所有配置类

每种 config 是一个 dataclass。用户通过 EngineArgs 设置后，组合成 VllmConfig：

文件	内容
vllm.py	VllmConfig（聚合所有 sub-config）
model.py	ModelConfig（hf_config、dtype、max_model_len）
model_arch.py	模型架构信息
cache.py	CacheConfig（block_size、gpu_memory_utilization、KV dtype）
parallel.py	ParallelConfig（TP/PP/EP/DP）
scheduler.py	SchedulerConfig（max_num_seqs、max_num_batched_tokens、policy）
device.py	DeviceConfig（cuda/cpu/tpu/xpu）
load.py	LoadConfig（loader 类型、download_dir）
lora.py	LoRAConfig（max_loras、max_lora_rank）
compilation.py	CompilationConfig（level、CUDAGraph 配置）
quantization.py	QuantizationConfig 工厂
attention.py	AttentionConfig
kernel.py	KernelConfig
multimodal.py	MultimodalConfig
speculative.py	SpeculativeConfig
structured_outputs.py	StructuredOutputsConfig
mamba.py	Mamba 专属
pooler.py	PoolerConfig（embedding 模型）
kv_transfer.py / kv_events.py / ec_transfer.py	KV / EC connector 配置
weight_transfer.py	权重传输
observability.py	Prom / OTel 配置
profiler.py	Profile 配置
reasoning.py	Reasoning parser 配置
speech_to_text.py	STT 配置
offload.py	KV offload 配置
utils.py	

11. vllm/profiler/、 vllm/tool_parsers/、 vllm/reasoning/

profiler/

- layerwise_profile.py —— 层级 profiler（每 transformer 层耗时）
- wrapper.py —— profile 包装
- utils.py

tool_parsers/

- abstract_tool_parser.py —— 接口
- 每模型一个：deepseekv3_tool_parser.py / gemma4_tool_parser.py / glm4_moe_tool_parser.py / granite4_tool_parser.py / ...
- 把 LLM 输出的 function call 字符串解析成结构化对象

reasoning/

- DeepSeek-R1 / o1 风格的 <think> 段解析
- 每模型一份 parser

12. vllm/platforms/： 硬件抽象

文件	作用
interface.py	Platform 基类
cuda.py	NVIDIA GPU
rocm.py	AMD ROCm
tpu.py	Google TPU
xpu.py	Intel XPU
cpu.py	CPU
zen_cpu.py	AMD Zen CPU

Platform.get_attn_backend_cls()、get_communicator_cls() 等方法让上层无关硬件。

13. vllm/transformers_utils/： HF 生态适配

路径	作用
tokenizer.py	HF tokenizer 包装
processor.py	HF processor
config.py	从 HF config 推断模型
chat_templates/	各模型 chat template
configs/	各模型 config 类
processors/	各模型 processor
gguf_utils.py	GGUF 元数据
runai_utils.py / s3_utils.py	远端权重加载
dynamic_module.py	trust_remote_code 支持
repo_utils.py	
model_arch_config_convertor.py	模型 config 转换

14. vllm/inputs/：请求输入处理

文件	作用
llm.py	LLM offline 入口的输入处理
engine.py	Engine 层输入打包
preprocess.py	tokenize、chat template apply、mm preprocess

15. vllm/utils/：工具集

最常用的：

文件	作用
cache.py	LRUCache 等
hashing.py	哈希工具
mem_utils.py / mem_constants.py	显存工具
nccl.py	NCCL 辅助
flashinfer.py	FlashInfer 包装
deep_gemm.py	DeepGEMM 包装
numa_utils.py / numa_wrapper.sh	NUMA 亲和
nvtx_pytorch_hooks.py	NVTX 标记
multi_stream_utils.py	CUDA 多 stream
gc_utils.py	GC 控制
import_utils.py / argparse_utils.py / func_utils.py	通用
network_utils.py	端口探测等
cpu_resource_utils.py / cpu_triton_utils.py	CPU 相关
mistral.py	Mistral 专属
counter.py / async_utils.py / collection_utils.py / jsontree.py	杂项
math_utils.py	

16. `csrc/`: C++ / CUDA 内核

```
csrc/
├── torch_bindings.cpp      ← 主 pybind 入口
├── ops.h                  ← 全部 op 声明
├── cuda_compat.h / cuda_utils.h / cuda_vec_utils.cuh / dispatch_utils.h ...
├── cumem_allocator.{cpp,h} ← cumem 内存池
├── async_util.cuh / spinloop.cpp ← 异步工具
├── launch_bounds_utils.h
├──
├── activation_kernels.cu      ← SiLU / GeLU / SwiGLU
├── layernorm_kernels.cu       ← RMSNorm
├── layernorm_quant_kernels.cu ← RMSNorm + 量化融合
├── pos_encoding_kernels.cu   ← RoPE
├── cache_kernels.cu / cache_kernels_fused.cu ← KV reshape / copy
├── nvfp4_kv_cache_kernels.cu ← NVFP4 KV
├── sampler.cu                ← top-k / top-p sampling
├── topk.cu                   ← topk 辅助
├── custom_all_reduce.{cu,cuh} ← 自定义 AllReduce
├── custom_all_reduce_test.cu
├── custom_quickreduce.cu      ← quickreduce
├── cuda_utils_kernels.cu
├── cuda_view.cu
├── fused_qknorm_rope_kernel.cu ← QK-norm + RoPE 融合
├── fused_deepseek_v4_qnorm_rope_kv_insert_kernel.cu ← DSv4 专融合
├── minimax_reduce_rms_kernel.{cu,h}
├──
├── attention/                ← PagedAttention v1/v2 + 辅助
│   ├── paged_attention_v1.cu
│   ├── paged_attention_v2.cu
│   ├── merge_attn_states.cu ← v2 的归并阶段
│   ├── attention_kernels.cuh ← 核心 kernel 模板
│   ├── attention_generic.cuh / attention_utils.cuh
│   ├── attention_dtypes.h
│   ├── dtype_{bfloat16,float16,float32,fp8}.cuh
│   └── vertical_slash_index.cu ← Sparse attention 索引
├──
├── moe/                       ← MoE 路由 + 分组 GEMM
│   ├── moe_align_sum_kernels.cu
│   ├── grouped_topk_kernels.cu
│   ├── moe_permute_unpermute_op.cu
│   ├── moe_wna16.cu
│   ├── dsv3_router_gemm_*.cu ← DeepSeek-V3 路由 GEMM
│   ├── dsv4_norm_router_gemm_*.h,cuh ← V4
│   ├── dynamic_4bit_int_moe_cpu.cpp
│   ├── marlin_moe_wna16/      ← Marlin × MoE
│   └── moeTopKFuncs.cuh / moe_ops.h
├──
├── quantization/              ← 量化 GEMM
│   ├── gptq/                  ← GPTQ
│   ├── awq/ (不在此处) ← AWQ 在上层 layers/quantization/awq*
│   ├── machete/               ← H100+ 高速 mixed precision GEMM
│   ├── marlin/                ← INT4 W × FP16 A 最快 GEMM
│   ├── w8a8/                  ← W8A8 量化
│   ├── fused_kernels/         ← 融合量化 op
│   ├── gguf/                  ← GGUF kernel
│   ├── activation_kernels.cu
│   └── utils.cuh
├──
├── core/                      ← 公共 utility
├── cutlass_extensions/        ← CUTLASS 自定义模板
├── libtorch_stable/           ← 稳定 ABI 封装
├──
├── concat_mla_q.cuh           ← MLA Q 拼接
├── persistent_topk.cuh        ← persistent kernel
└── type_convert.cuh
```



17. benchmarks/：性能基准

文件	用途
benchmark_throughput.py	离线吞吐
benchmark_latency.py	单请求延迟
benchmark_serving.py	在线服务（OpenAI 协议压测）
benchmark_serving_structured_output.py	结构化输出
benchmark_prefix_caching.py	prefix cache 效果
benchmark_prefix_block_hash.py	block hash 性能
benchmark_block_pool.py	block pool 性能
benchmark_hash.py	hash 工具
benchmark_ngram_proposer.py	ngram spec decode
benchmark_topk_topp.py	sampling kernel
benchmark_long_document_qa_throughput.py	长上下文
benchmark_prioritization.py	优先级调度
benchmark_batch_invariance.py	batch-invariant 模式
backend_request_func.py	各 backend client（vLLM / TGI / OpenAI / DeepSpeed）
benchmark_utils.py	
auto_tune/	自动调参
attention_benchmarks/	attention kernel benchmark
kernels/ / fused_kernels/ / cutlass_benchmarks/	kernel 级
disagg_benchmarks/	disaggregated prefill
multi_turn/	多轮对话
overheads/	overhead 测量

18. examples/：使用范例

子目录	内容
basic/	最简单 demo
offline_inference/	离线推理脚本
applications/	完整应用
deployment/	部署示例（K8s YAML）
disaggregated/	Prefill / Decode 分离
features/	各功能示例（LoRA / spec decode / 量化 / structured output）
generate/	生成参数示例
pooling/	embedding / rerank
reasoning/	reasoning 模型
rl/	RL 训练接入
tool_calling/	function calling
speech_to_text/	Whisper
ray_serving/	Ray Serve 集成
observability/	OTel 配置
template_*.jinja / tool_chat_template_*.jinja	chat / tool 模板

19. tests/：测试

按 vllm 子模块对应：

- tests/engine/ / tests/spec_decode/ / tests/samplers/ / tests/lora/ / tests/multimodal/ / tests/models/ / tests/kernels/ / tests/distributed/ ...
- basic_correctness/ —— 端到端正确性
- evals/ —— 模型评估
- compile/ —— torch.compile 测试
- prompts/ —— 测试用 prompt
- standalone_tests/ —— 不需要装 vLLM 的独立测试
- conftest.py —— pytest 共享 fixture
- ci_envs.py —— CI 环境变量

20. docs/：官方文档源

目录	内容
getting_started/	入门
serving/	服务部署
models/	模型支持矩阵
features/	各功能
deployment/	K8s / Docker
design/	架构设计文档（含 V1 设计文档 ）
api/	API 参考
cli/	CLI 参考
usage/	使用模式
benchmarking/	性能 benchmark
configuration/	配置参考
contributing/	贡献指南
community/	
governance/	
training/	
mkdocs/	mkdocs 配置
assets/	图片
examples/	docs 引用的例子

21. "我要找 X，去哪里"速查表

想做什么	第一站
看一个请求生命周期	v1/engine/llm_engine.py → v1/engine/core.py
看调度逻辑	v1/core/sched/scheduler.py
看 KV cache 怎么分	v1/core/{block_pool,kv_cache_manager}.py
看 forward 怎么跑	v1/worker/gpu_model_runner.py
看 attention kernel	v1/attention/backends/flash_attn.py → csrc/attention/
看采样	v1/sample/sampler.py
看一个模型实现	model_executor/models/<model>.py
看量化方法	model_executor/layers/quantization/<method>.py
看 OpenAI API	entrypoints/openai/api_server.py
看 TP / PP / EP 切	model_executor/layers/linear.py + distributed/parallel_state.py
看 multimodal	multimodal/inputs.py + 模型文件
看 LoRA	lora/model_manager.py + lora/punica_wrapper/
看投机解码	v1/spec_decode/ + v1/sample/rejection_sampler.py
看结构化输出	v1/structured_output/__init__.py
看 CUDA Graph + compile	compilation/backends.py
看环境变量	envs.py
看一个配置项	config/<sub>.py
看 benchmark 怎么跑	benchmarks/benchmark_serving.py（或 vllm bench serve）
看官方设计文档	docs/design/（含 V1 架构、KV cache 设计、scheduler）
看模型支持矩阵	docs/models/

22. 这份地图怎么用

- 1. **写代码前先来这查：**要修改/扩展某个特性，先在地图里找到对应文件，再去打开看
- 2. **读源码迷路时回这查：**知道自己在哪一层
- 3. **查"X 在哪实现"时：**直接说出 file path

读完这节，配合 03-code-walkthrough/（重点文件深读）你就能拍着胸脯说"vLLM 我熟"。

小结

- vLLM 仓库的"重量级"目录就 4 个：**vllm/v1/**（V1 引擎、调度、KV 管理、采样、worker）+ **vllm/model_executor/**（模型实现、量化、loader、layers）+ **vllm/csrc/**（CUDA/C++ kernel）+ **vllm/distributed/**（TP/PP/EP/DP 通信）。其余目录围绕这四块展开。
- 入口在 vllm/entrypoints/，配置全集中在 vllm/config/，环境变量全在 vllm/envs.py——这是查任何"用户可见参数 / 默认值"的三大入口。
- Attention 路径**双层**：vllm/v1/attention/backends/<backend>.py 是 Python 后端（选 FlashAttn / FlashInfer / Triton / MLA / Mamba 等），真正的 CUDA kernel 在 csrc/attention/ 与 csrc/quantization/、csrc/moe/。
- 模型实现都遵循 XxxAttention / XxxMLP / XxxDecoderLayer / XxxModel / XxxForCausalLM 五层结构——读懂 Llama 一份模型，其他 292 个模型基本都能照着扫。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 三个任务的入口文件。

任务	第一个打开的文件
Prefix caching 怎么算 hash	vllm/v1/core/kv_cache_utils.py（hash_block_tokens，约 line 541）
Chunked prefill 怎么切	vllm/v1/core/sched/scheduler.py（_schedule_waiting + token budget）
加一个新模型	vllm/model_executor/models/registry.py（注册入口），复制最像的现有模型改

2. Python flash_attn.py vs CUDA paged_attention_v2.cu 怎么连？

- **vllm/v1/attention/backends/flash_attn.py**：构造 attention metadata（block_table / seq_lens / query_start_loc）、决定调哪个 kernel。
- **csrc/attention/paged_attention_v2.cu**：实际 GPU kernel，通过 block_table 间接寻址访问物理 KV block。
- **桥**：vllm/_custom_ops.py 用 torch.library 注册 CUDA op，Python 端调 torch.ops.vllm.paged_attention_v2(...)，PyTorch dispatcher 自动派发。这样 torch.compile 不会 graph break。

3. 量化拆 Python / CUDA 两层的原因。

- vllm/model_executor/layers/quantization/（Python）：算法描述、权重加载、Linear 层 wrap、决定调哪个 kernel
- csrc/quantization/（CUDA）：纯 kernel（marlin / awq / gptq / fp8 / cutlass_w8a8）

拆两层原因：

1. 算法与硬件解耦——同一 AWQ 可适配 SM80/SM90/SM100，Python 不动
2. 多算法共享 kernel——fp8 kernel 同时给 fp8-static / fp8-dynamic / fp8-marlin 用
3. front-end / back-end 分离：Python 编排，CUDA 算

4. 加 NVMe KV offload 后端要改哪？

```
vllm/v1/kv_offload/
├── base.py      ← 已有，抽象接口
├── factory.py   ← 已有，注册要加 "nvme" → 你的 connector
├── nvme/        ← 新目录
│   ├── connector.py ← 实现 base 接口（save/load/evict 必须异步）
│   ├── pool.py    ← NVMe block pool（mmap + io_uring）
│   └── config.py   ← 容量、路径、并发
```

外加 `vllm/config/cache.py` 或 `kv_transfer_config` 暴露 `--kv-offload-backend nvme`，`tests/v1/kv_offload/test_nvme.py` 新增单测。

关键：NVMe 比 DRAM 慢 100×，接口必须**异步**返回 Future，否则把 step 时长拖死。

5. 30 秒速查 3 件事。

任务	入口	主要文件
TP / PP / EP 切	<code>vllm/distributed/</code>	<code>parallel_state.py</code> 、 <code>device_communicators/all2all.py</code> 、 <code>eplb/</code>
OpenAI API	<code>vllm/entrypoints/openai/</code>	<code>api_server.py</code> 、 <code>serving_chat.py</code>
LoRA 加载	<code>vllm/lora/</code>	<code>model_manager.py</code> 、 <code>punica_wrapper/</code> 、 <code>layers/</code>

口诀：分布式 → `distributed/`；HTTP 入口 → `entrypoints/`；特性（LoRA / 多模态 / `spec_decode`）各占顶层目录。

下一步

- 下一节：05-process-and-ipc-internals.md（地图看完，正式深入 `vllm/v1/engine/` + `vllm/v1/executor/` + `vllm/distributed/device_communicators/` 的进程与 IPC 内部）。
- 源码深读起点：03-code-walkthrough/01-entry-points.md（从 LLM(...) 一路 trace 到 `step()`）。
- 想看核心创新：02-core-concepts/01-paged-attention.md（带着地图去读 `vllm/v1/core/block_pool.py` 不会迷路）。
- 想动手：07-hands-on/01-setup.md（把仓库 clone 下来照着这张地图浏览一遍）。

05. 进程模型与进程间通信内部机制

谁该读这一篇？ 第一次接触多进程、共享内存或 ZeroMQ（ZMQ），以及需要定位 vLLM 分布式推理卡死、IPC 超时、/dev/shm 不足问题的读者。

前置阅读： 02-architecture.md § 1-2。本文不假定读者已经懂 socket、序列化、虚拟地址空间或 ZMQ。

耗时： 约 60～80 分钟；如果已有操作系统和网络编程基础，可从 § 5 开始。

源码基线： 本章按仓库锁定的 vLLM b23bd73（2026-07-20）V1 实现复核。引用格式仍为 file_path:line_number；源码升级后行号可能漂移，应优先搜索类名、函数名及下方语义锚点。

学完本章，应能回答五个问题：

- 1. 进程、线程、虚拟地址空间、文件描述符分别是什么？
- 2. fork、spawn、写时复制和 CUDA “poison fork” 有什么关系？
- 3. ZMQ 与普通 TCP socket 有什么区别？ROUTER/DEALER、PUSH/PULL、PUB/SUB 各解决什么问题？
- 4. vLLM 的 API Server、EngineCore、Worker 之间分别传什么，经过几次序列化和内存复制？
- 5. 扩展到 48 台 8×H100（共 384 卡）时，哪些通信仍是节点内 IPC，哪些会跨节点？

先给出全章最重要的纠偏：

共享内存不等于“完全零拷贝”，ZMQ 也不等于“网络通信”。

vLLM 的 MessageQueue 是混合实现：本机常规消息写入共享内存环形缓冲区；ZMQ 负责唤醒通知、超大消息回退和跨节点传输。writer 把序列化结果写入共享内存时仍有一次内存复制。PEP 574 的收益是让大 buffer 不必先被复制进 pickle 主字节流，并让多个本机 reader 直接读取同一共享段，而不是承诺端到端 0 次复制。

1. 先建立最小操作系统模型

1.1 程序、进程、线程不是一回事

名词	可以把它理解成	主要拥有或共享什么
程序	磁盘上的可执行文件和 Python 源码	静态文件，本身没有运行状态
进程	一次正在运行的程序实例	独立虚拟地址空间、PID、文件描述符表、信号处理状态
线程	进程内的一条执行流	独立栈和寄存器；与同进程线程共享堆、模块、文件描述符
协程	用户态调度的可暂停函数	仍运行在线程和进程中，不提供地址空间隔离

两个 Python 进程即使都存在名为 scheduler 的变量，它们也通常位于彼此隔离的地址空间。一个进程执行：

state["running"] = 100

不会自动改变另一个进程里的 state。要让另一个进程看到变化，必须使用 IPC（Inter-Process Communication，进程间通信），或者让两边映射同一段共享内存。

线程不同：两个线程能直接看到同一 Python 对象，因此少了 IPC，却增加了锁、竞态和故障隔离问题。CPython 的 GIL 限制 Python 字节码并行，但这不是“任何线程都不能并行”：C/CUDA 扩展和阻塞 I/O 可以释放 GIL。vLLM 的 EngineCore 正是用 I/O 线程把 ZMQ 收发、部分编解码和 GPU 执行重叠起来，见 `vllm/v1/engine/core.py`。

1.2 一个进程的虚拟地址空间

高地址

内核映射（用户态不能随意访问）	
线程栈 / 主线程栈	向下增长
mmap 区：动态库、文件、共享内存	
heap：Python 对象、allocator	向上增长
data / bss	
text：程序机器码	

低地址

这里的地址是**虚拟地址**。CPU 的页表把它翻译为物理页。两个进程可以同时打印出相同的虚拟地址，但默认映射到不同物理页；也可以打印出不同虚拟地址，却通过共享内存映射到同一物理页。

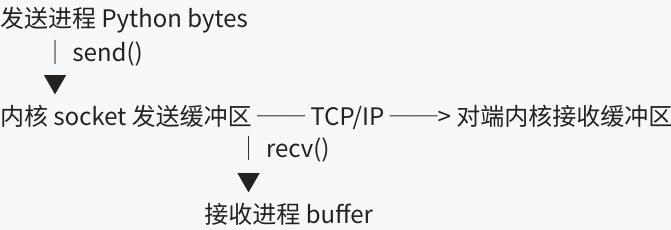
因此判断“是否共享”不能只看 Python 的 `id()` 或指针值，而要问：

- 两个进程的页表是否指向同一物理页？
- 数据是否已被复制到 socket 内核缓冲区或另一块用户态缓冲区？
- 接收端构造对象时是创建 view，还是分配新内存并复制？

1.3 文件描述符、socket 和内核缓冲区

Linux 进程访问文件、pipe 和 socket 时，用户态拿到的是一个小整数，例如 fd 17。fd 是当前进程文件描述符表的索引，不是数据本身。

普通 TCP 发送可以粗略画成：



一次“传消息”的成本至少要分成四类：

1. **编码/序列化**：Python 对象变成字节及附属 buffer；
2. **内存复制**：用户 buffer、共享段、内核 socket buffer 之间搬运；
3. **系统调用和上下文切换**：进出内核、进程被唤醒；
4. **排队和同步**：发送者等队列空间，接收者等数据，慢消费者制造背压。

以后看到“zero-copy”必须追问：省掉的是哪一次复制？只在发送端，还是接收端？只对 NumPy/Tensor payload，还是连元数据也不编码？

1.4 进程退出、daemon 和僵尸进程

父进程创建子进程后，需要监控其退出状态并回收。子进程已经退出、父进程却还没 wait() 时，会短暂成为 zombie；它不再执行，但 PID 和退出信息仍占内核表项。

Python multiprocessing.Process(daemon=True) 的含义更接近“随父进程退出的后台子进程”，不是 systemd daemon。当前 vLLM Worker 是 daemon process，见 vllm/v1/executor/multiproc_executor.py；它还用 death pipe 检测父进程 EOF，并唤醒阻塞中的队列。EngineCore 进程的创建点则在 vllm/v1/engine/utils.py，没有在此处设置 daemon=True。

2. 进程是如何创建的：fork、spawn 与写时复制

2.1 fork 做了什么

fork 创建一个几乎复制父进程状态的子进程。这里“复制”不等于立即 memcpy 整个进程内存：Linux 通常先让父子页表共同指向相同物理页，并把页标成只读；任一方第一次写某页时，内核才复制该页，这就是 COW（Copy-on-Write，写时复制）。



fork 的优点是启动快、未修改页面可共享；风险包括：

- 父进程其他线程不会以正常状态“复制过去”，但锁可能保留在被持有状态；
- socket fd 会被继承，多份进程若没有关闭不需要的 fd，会妨碍 EOF 和资源回收；
- allocator、日志库、通信库的后台线程状态可能不适合 fork 后继续使用；
- 已初始化 CUDA runtime 后再 fork 可能产生不可恢复的 CUDA 状态错误。

vLLM 在使用 fork 时显式跟踪 Worker 继承的 socket fd，随后关闭不需要的副本，见 vllm/v1/executor/multiproc_executor.py。

2.2 spawn 做了什么

spawn 启动一个全新的 Python interpreter，再导入入口模块，并通过 pickle 传递 target、参数等初始化信息。它不会继承父进程的 Python 堆和锁状态，因此 CUDA 场景通常更稳，但代价是启动和重复 import 更慢、传参必须可序列化。

手写多进程程序时，spawn 要求入口保护：

```
import multiprocessing as mp
```

```
def child(rank: int) -> None:
    print(rank)
```

```
if __name__ == "__main__":
    ctx = mp.get_context("spawn")
    p = ctx.Process(target=child, args=(0,))
    p.start()
    p.join()
```

没有 `if __name__ == "__main__"`，子解释器重新导入模块时可能再次创建子进程，形成递归启动。

2.3 vLLM 如何选择

源码锚点：[vllm/utils/system_utils.py](#) · [_maybe_force_spawn](#)

当前实现由 `get_mp_context()` 统一选择，源码是 `vllm/utils/system_utils.py`：

- `VLLM_WORKER_MULTIPROC_METHOD` 的默认值是 `fork`；
- 已经初始化 CUDA/XPU 时强制改为 `spawn`；
- Ray actor、NUMA bind 和 WSL 等条件也会强制 `spawn`；
- ROCm 还会同步可见设备环境变量，再创建 context。

所以不能简单记成“vLLM 永远 fork”或“CUDA 程序永远 spawn”。准确说法是：

vLLM 默认允许 `fork`，但在检测到不安全条件时强制 `spawn`；无论哪种方式，都应避免在错误的进程里过早初始化 CUDA。

Python 本身在不同版本和平台上的默认 `start method` 会变化，排障时不要靠常识猜，直接记录：

```
import multiprocessing as mp
print(mp.get_start_method())
```

vLLM 则应同时检查 `VLLM_WORKER_MULTIPROC_METHOD` 和启动日志中的 `override warning`。

3. IPC 方法全景：为什么不能只会“队列”

机制	是否跨机器	数据如何到对端	常见用途	主要代价/限制
匿名 pipe	否	内核 pipe buffer	父子 readiness、EOF/death 通知	一般是一对端点，传对象仍需序列化
Unix domain socket	否	本机内核 socket	本机结构化消息、ZMQ ipc://	有系统调用和 buffer 复制
TCP socket	是	TCP/IP	跨节点控制消息	网络、拥塞、重连、安全配置
POSIX shared memory	否	多进程映射同一物理页	高频大 payload	必须自行定义布局、同步、生命周期
mmap 文件	通常本机	映射文件页	持久化或文件共享	页缓存和文件生命周期
multiprocessing.Queue	否	pipe + pickle + feeder thread	通用 Python 对象	隐式线程、序列化和复制开销
ZMQ	取决于 transport	消息库封装 inproc/ipc/tcp	异步路由、扇入扇出、发布订阅	非持久消息代理；要理解 socket pattern/HWM
NCCL	多 GPU/可跨节点	NVLink/PCIe/RDMA/TCP	Tensor collective/P2P	只解决设备间数据通信，不传任意 Python 对象

选择 IPC 不能只看单条消息延迟，还要看：

- 是一对一、一对多还是多对一？
- 消息是几十字节的命令，还是几十 MiB 的张量？
- receiver 都在同一主机吗？
- 是否要求可靠持久化、重放或 exactly-once？
- 慢 reader 应阻塞 writer、丢弃旧通知，还是独立隔离？
- 进程异常退出后谁负责清理和通知其他进程？

vLLM 没有用一种机制包打天下，而是把它们组合起来。

4. ZMQ 从零开始

源码锚点：[vllm/utils/network_utils.py](#) • [make_zmq_socket](#)

4.1 ZMQ 是什么，不是什么

ZeroMQ/ØMQ 是一个异步消息库。应用看到的仍叫 socket，但它不是“更快的 socket.socket”这么简单：

- 普通 TCP socket 面向字节流；ZMQ socket 面向**离散消息**；
- ZMQ 在后台 I/O thread 中处理连接、排队、重连和 framing；
- socket type 自带通信模式，如路由、负载分发、扇出；
- bind/connect 决定谁拥有 endpoint，不决定谁是业务上的请求方；
- 它可以走 inproc://、ipc:// 或 tcp://，所以“用了 ZMQ”并不能推出“数据经过网卡”。

它也不是 Kafka/RabbitMQ：没有默认持久化日志、消费 offset、事务或 exactly-once。进程崩溃、队列溢出或错误关闭时，应用必须接受其语义并设计握手、超时和恢复。

官方 libzmq 文档还特别强调：大多数经典 socket type **不是线程安全的**。通常做法是“一条线程拥有一个 socket”，线程之间另用 queue 或 inproc:// 通信。vLLM 的 EngineCore 也让 input thread 和 output thread 分别创建、持有各自的 socket。

4.2 Context、socket、endpoint 三层对象

```
import zmq

ctx = zmq.Context(io_threads=1) # 管理后台 I/O 线程和 socket 生命周期
s = ctx.socket(zmq.PUSH)       # 创建带 PUSH 语义的消息 socket
s.bind("ipc:///tmp/demo.sock") # endpoint = transport + address
s.send(b"hello")               # 发送一条完整消息
```

逐层理解：

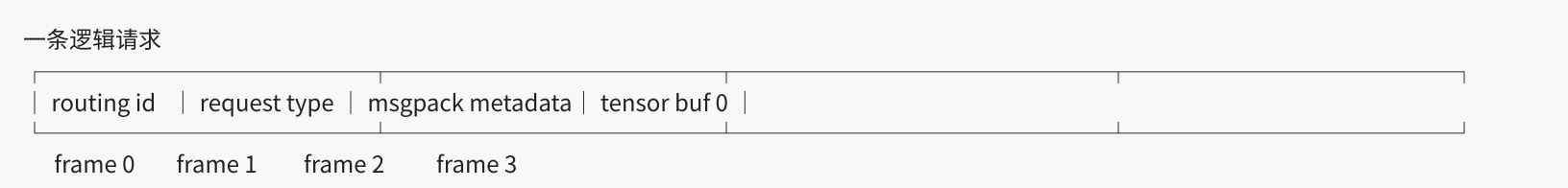
- 1. **Context**：进程内的 ZMQ 运行时，管理 I/O thread；不是每条消息都创建一个。
- 2. **Socket**：带特定 pattern 的逻辑端点；有发送/接收队列、HWM、identity 等状态。
- 3. **Endpoint**：ipc:///path、tcp://host:port 或 inproc://name。

常见 transport：

transport	范围	典型地址	注意点
inproc://	同一 ZMQ context 内的线程	inproc://cancel-123	不跨进程；延迟低
ipc://	同一主机的进程	ipc:///tmp/vllm-xxx	依赖 Unix socket 路径和目录权限
tcp://	可跨主机	tcp://10.0.0.8:5000	要处理路由、防火墙、端口和安全边界

4.3 message、frame 与 multipart

一条 ZMQ message 可以由多个 frame 组成，接收端用 recv_multipart() 一次取回完整边界：



这比 TCP 字节流方便：TCP 的一次 send() 不保证对应接收端一次 recv()，应用必须自己编码长度；ZMQ 保留 message/frame 边界。

send_multipart(..., copy=False) 的含义也要谨慎。它请求 pyzmq 尽量让 libzmq 引用现有 buffer，减少 Python 用户态的一次复制，但：

- 小消息可能仍被库复制；
- TCP/IPC 进入内核和网卡时仍有其自己的数据移动；
- 发送完成前原 buffer 必须保持存活，因此 vLLM 用 MessageTracker 和 pending 保存引用，见 vllm/v1/engine/core_client.py、vllm/v1/engine/core.py。

4.4 vLLM 用到的 socket pattern

pattern	方向/行为	vLLM 中的用途	最容易误解的点
ROUTER	收到消息时附带对端 identity；发送时按 identity 路由	API client 请求入口	它不是 HTTP router；第一帧是路由信封
DEALER	双向异步，能连接 ROUTER	EngineCore 输入和启动握手	不受 REQ/REP 严格一问一答约束
PUSH/PULL	单向 pipeline；多个 peer 间分发/公平接收	EngineCore 输出 → client	不是广播；一条消息只交给一个下游 PULL
PUB/SUB	publisher 扇出，subscriber 按 topic 前缀订阅	SHM 写入通知	慢订阅者或未完成订阅时可丢消息，不适合可靠命令
XPUB/XSUB	可观察订阅事件的高级 pub/sub	MessageQueue readiness、DP coordinator	XPUB 可确认 reader 已订阅，解决启动“慢加入者”窗口
PAIR	单连接、双向	同进程 cancellation ping	不适合通用多对多拓扑

ROUTER/DEALER 的信封

假设两个 EngineCore 的 identity 分别是 b"\x00\x00" 和 b"\x01\x00"：



ROUTER 消费路由帧，DEALER 只看到业务帧。当前 vLLM client 发送元组的代码在 vllm/v1/engine/core_client.py；EngineCore 用 type_frame, *data_frames = recv_multipart(copy=False) 接收，见 vllm/v1/engine/core.py。

PUSH/PULL 不是 PUB/SUB

如果一个 PUSH 连接三个 PULL，消息通常在三个 PULL 之间分配，而不是每个都收到。如果一个 PUB 连接三个 SUB，每个匹配 topic 的 SUB 都可能收到一份。前者适合工作分发/扇入，后者适合状态广播或“有新数据了”的提示。

4.5 bind/connect 不等于 server/client

bind 表示这个 socket 创建并拥有地址，connect 表示连接已有地址。谁先发送由 socket pattern 决定，而不是由 bind/connect 决定。

vLLM 的通用工厂 make_zmq_socket() 在 vllm/utils/network_utils.py：默认让 PUSH/SUB/XSUB connect，其他类型 bind，但调用方可以显式覆盖。API 侧 ROUTER bind，EngineCore 侧 DEALER connect；输出侧 client 的 PULL 默认 bind，EngineCore 的 PUSH 默认 connect。

4.6 HWM、背压和“为什么进程像卡死了”

HWM（High Water Mark）是 socket 队列的高水位。达到 HWM 后，不同 socket type 可能阻塞、返回 EAGAIN，或像 PUB 一样丢给慢 subscriber 的消息。它不是字节数的严格全局上限，通常按 peer/消息队列理解。

vLLM 的 `make_zmq_socket()` 对 PULL/DEALER/ROUTER 设置 `RCVHWM=0`，对 PUSH/DEALER/ROUTER 设置 `SNDBWM=0`，见 `vllm/utils/network_utils.py`。在 ZMQ 语义里 0 表示不施加该 HWM 限制；这能减少突发流量下的阻塞，却意味着应用更需要用上层队列、请求上限和监控控制内存增长。

共享内存通知是另一种取舍：SpinCondition 的 PUB `SNDBWM=1`，SUB 开 `CONFLATE=1`，见 `vllm/distributed/device_communicators/shm_broadcast.py`。通知只表达“去检查 ring”，旧 ping 被合并或丢掉没关系，因为真实状态保存在共享内存 metadata 中。

这是一条很重要的设计原则：

通知可以有损，状态必须可重新读取。 如果把唯一的业务 payload 放在有损 PUB/SUB 上，这个结论就不成立。

4.7 Poller、握手、重连与关闭

- **Poller**：一条线程同时等待多个 socket，不必逐个阻塞 `recv()`；EngineCore 输入线程从 `vllm/v1/engine/core.py` 开始创建并注册多个输入端点。
- **握手**：ZMQ connect 是异步的，“调用 connect 成功”不代表对端已经 ready。vLLM 的 DEALER 先向 ROUTER 发 ready payload，client 等齐所有 engine identity，见 `vllm/v1/engine/core_client.py`。
- **slow joiner**：PUB 在 SUB 完成订阅前发送的消息可能收不到；MessageQueue 使用 XPUB 接收订阅事件，再广播 READY，见 `shm_broadcast.py`。
- **reconnect**：ZMQ 可后台重连，但不会自动恢复应用状态、重放任意丢失请求或判断推理是否已执行。
- **linger**：socket close 时等待待发消息的时长。vLLM 输出线程设置 `linger`，以尽量先发出 `ENGINE_CORE_DEAD`，见 `vllm/v1/engine/core.py`；部分 shutdown socket 明确 `linger=0`，避免退出无限等待。

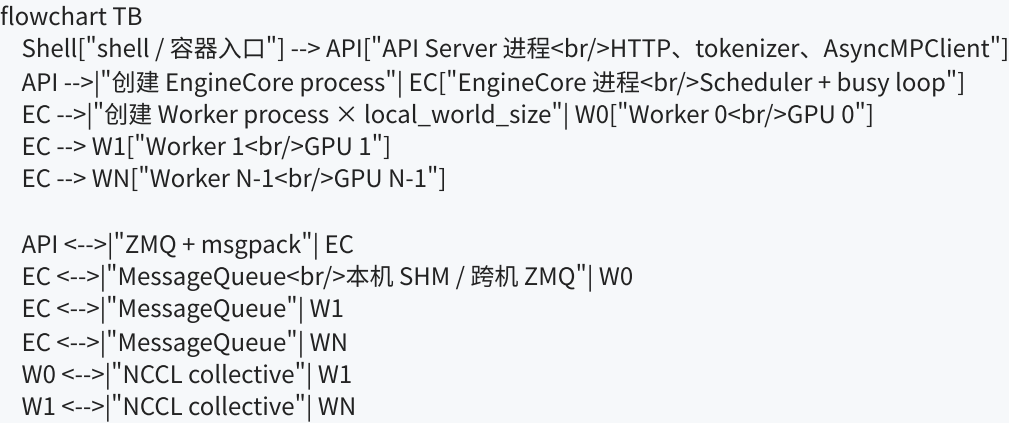
5. vLLM V1 的进程拓扑

源码锚点：[vllm/v1/engine/core_client.py](#) • [MPClient](#)

源码锚点：[vllm/v1/engine/utils.py](#) • [launch_core_engines](#)

源码锚点：[vllm/v1/executor/multiproc_executor.py](#) • [WorkerProc.worker_main](#)

5.1 单个在线实例的主链



主要创建点：

对象	当前源码入口	关键事实
In-process EngineCore	vllm/v1/engine/core_client.py	InprocClient 直接构造 EngineCore，没有 API↔Engine ZMQ
Multi-process EngineCore	vllm/v1/engine/utils.py	CoreEngineProcManager 按本机 DP rank 创建 EngineCore process
Worker	vllm/v1/executor/multiproc_executor.py	每个 local rank 创建 daemon Worker process
Worker readiness/death	multiproc_executor.py	单向 pipe 通知 ready，death pipe 用 EOF 感知父进程退出

“每 GPU 一个 Worker” 是常见 CUDA MultiprocExecutor 形态，不应扩展成所有 backend 的绝对定律：Ray actor、外部 launcher、CPU/XPU backend 或将来执行器实现可能改变进程承载方式。读源码时以选中的 Executor 为准。

5.2 Inproc、MP 与分布式不是同一个维度

模式	Client → EngineCore	EngineCore → Worker	适用情况
InprocClient	Python 直接调用	由所选 Executor 决定	离线同步路径、调试
SyncMPClient / AsyncMPClient	ZMQ	由所选 Executor 决定	在线服务、进程隔离
MultiprocExecutor	不适用	MessageQueue + 本机进程/分布式组	常见单机 TP/PP，也可有远端 reader
RayDistributedExecutor	不适用	Ray actor/RPC + GPU 通信	多节点资源编排

Client 类型描述的是**前端和 EngineCore 的关系**；Executor 类型描述的是**EngineCore 如何管理 Worker**。把两个维度混成 “Inproc/MP/Ray 三选一” 会导致错误推理。

5.3 EngineCore 为什么还有 I/O 线程

EngineCore 进程里不只有 scheduler 主循环：

- input thread：ZMQ socket → msgpack decode → input_queue；
- core busy loop：处理请求、调 scheduler、执行 step；
- output thread:output_queue → msgpack encode → ZMQ socket。

6. API Server ↔ EngineCore：控制请求到底怎么走

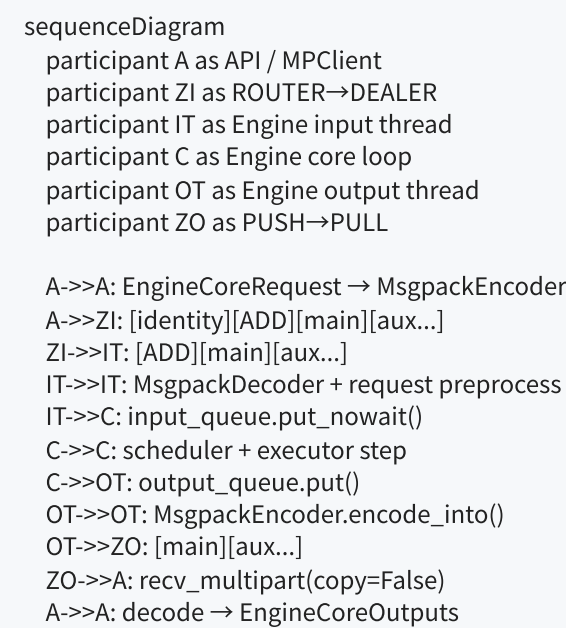
6.1 两条单向逻辑通道

方向	client 侧	EngineCore 侧	payload
请求输入	ROUTER bind	DEALER connect	identity + request type + msgpack 主帧 + 可选附属 buffers
输出回传	PULL bind	PUSH connect	msgpack 主帧 + 可选附属 buffers

client 创建 socket 见 vllm/v1/engine/core_client.py；EngineCore 的输入与输出 socket 线程见 vllm/v1/engine/core.py。

为什么输入不用简单 PUSH/PULL？因为一个 client 可能管理多个 DP EngineCore，需要把指定请求发给指定 identity；ROUTER 正好提供寻址。输出已经知道对应 client index，且是单向回传，用 PUSH/PULL 更简单。

6.2 一次 ADD 请求的时间线



请求端组 frame 的代码在 vllm/v1/engine/core_client.py，接收端解码与输出编码/发送在 vllm/v1/engine/core.py。

6.3 msgpack 主帧与附属 buffer

源码锚点：[vllm/v1/serial_utils.py](#) • [MsgpackEncoder](#)

MsgpackEncoder 位于 vllm/v1/serial_utils.py：

- 普通字段编码进 msgpack 主帧；
- Tensor/NumPy 大于阈值时，将 backing buffer 放到 aux_buffers，主帧只编码索引和 dtype/shape 等元数据；
- 非连续 ndarray 仍需要先变连续，源码在 serial_utils.py；

- 默认不允许随意 fallback 到 pickle；只有显式打开 VLLM_ALLOW_INSECURE_SERIALIZATION 才允许，见 serial_utils.py。

这也是为什么“msgpack 就一定只有一段 bytes”不准确。vLLM 在 ZMQ multipart 中把主帧和大 buffer 分开发，减少把 payload 拼接进单一大字节串的复制。

多模态请求还可以配置 tensor IPC consumer/provider。client 在 vllm/v1/engine/core_client.py 构造 TensorIpcSender，EngineCore 在 vllm/v1/engine/core.py 接收。它与后面的 Worker MessageQueue 是另一条专用优化，不要混为同一套 PEP 574 pickle 协议。

6.4 这条链的复制账本

环节	一定发生什么	可能避免什么
Python 请求 → msgpack	元数据被编码	大连续 Tensor/ndarray 不拼进主帧
pyzmq copy=False	libzmq/transport 仍管理消息	避免一部分 Python→libzmq 用户态复制
ipc://	经过本机 socket 机制	不经过外部网络路由
tcp://	经过 TCP/IP，可能经过 NIC	不能因 copy=False 称端到端零拷贝
Engine decode	构造请求对象/元数据	附属 buffer 可被 view 引用，取决于对象类型和生命周期

所以本章把它称为“控制/请求通道”而不是“零拷贝通道”。

7. EngineCore ↔ Worker：MessageQueue 与共享内存

源码锚点：[vllm/distributed/device_communicators/shm_broadcast.py](#) • [MessageQueue](#)

源码锚点：[vllm/distributed/device_communicators/shm_broadcast.py](#) • [ShmRingBuffer](#)

源码锚点：[vllm/distributed/device_communicators/shm_broadcast.py](#) • [MessageQueue.enqueue](#)

7.1 先纠正“数据面完全不用 ZMQ”

当前 MessageQueue 位于 vllm/distributed/device_communicators/shm_broadcast.py。它同时支持：

1. 本机 reader 的常规 payload：ShmRingBuffer；
2. 本机 reader 的 wake-up：ZMQ PUB/SUB；
3. 本机 payload 超过 ring 单 chunk 上限：ZMQ XPUB/SUB 回退；
4. 远端 reader：ZMQ TCP XPUB/SUB 直接传 multipart payload。

初始化代码明确计算 n_local_reader 和 n_remote_reader，并分别建立本地与远端路径，见 shm_broadcast.py。

7.2 SharedMemory 到底共享了什么

writer 调用：

```
shared_memory.SharedMemory(create=True, size=...)
```

内核创建具名共享段；reader 只拿到名字和布局参数，再调用：

```
shared_memory.SharedMemory(name=name)
```

两边各自得到一个 memoryview，页表最终映射到同一批共享物理页。创建与 attach 都在 shm_broadcast.py 的 ShmRingBuffer 中完成。

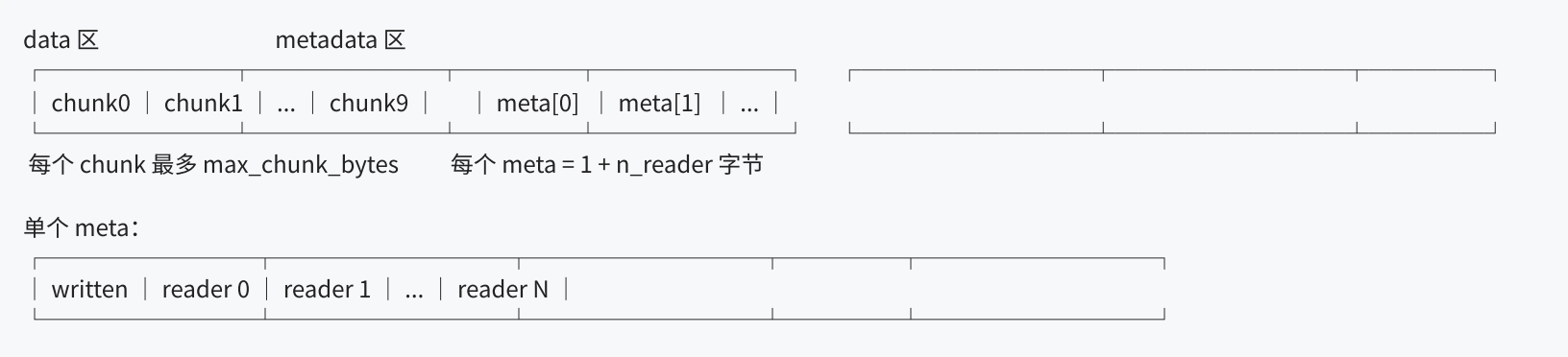
共享内存不自动提供：

- Python 对象结构；
- 消息边界；
- 谁可以写、谁已经读；
- 原子性和 CPU 内存可见性；
- 慢 reader 的背压策略；
- 异常退出后的清理。

这些都由 ShmRingBuffer 的协议补齐。

7.3 Ring buffer 的内存布局

ShmRingBuffer 是 single-writer / multi-reader broadcast queue，布局注释从 shm_broadcast.py 开始：



默认 max_chunk_bytes=24 MiB、max_chunks=10，见 shm_broadcast.py。只算 data 区，一个本地广播队列默认约 240 MiB；metadata 很小。实际 executor 可以按 payload 需求传入不同 chunk 大小，所以容量规划应读取运行配置和日志，不能机械地给每个队列都乘 240 MiB。

7.4 slot 状态机

对于某个 slot：

metadata	含义	writer 能写?	reader i 能读?
0 ???...???	还未发布/正在写	是（由单 writer 保证）	否
1 000...000	刚发布，无人读	否	是
1 010...100	部分 reader 已读	否	未读 reader 可以
1 111...111	所有 reader 已读	是，可复用 slot	否

writer 的步骤见 shm_broadcast.py：

1. 等当前 slot 未写或已被所有 reader 读完；
2. 把 written=0；
3. 写完整 payload；
4. 清零所有 reader flag；
5. memory fence；
6. 把 written=1，再次 fence，推进 ring index。

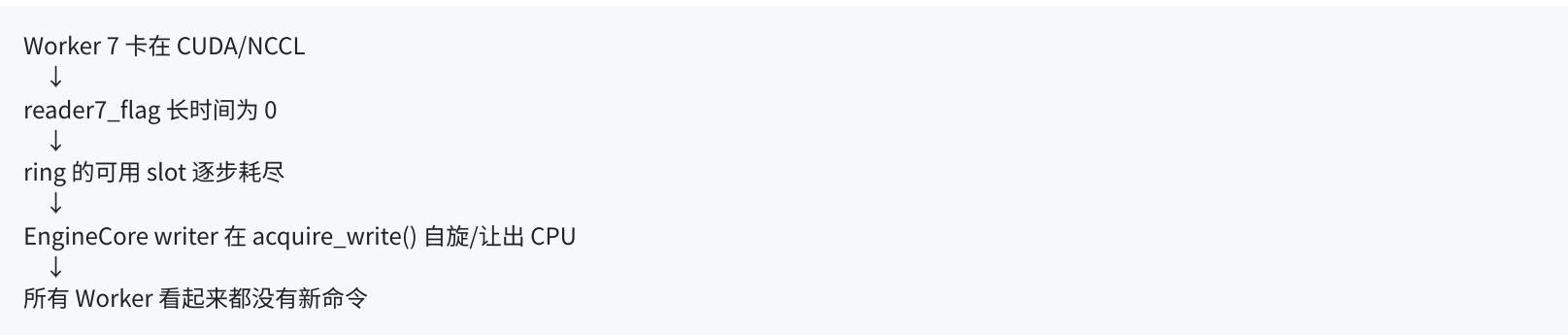
reader 的步骤见 shm_broadcast.py：

1. 等 written=1 且自己的 read flag 为 0；
2. 读取 slot 并反序列化；
3. 把自己的 read flag 置 1；
4. memory fence，推进自己的 ring index。

这里不使用一把大 mutex，是因为只有一个 writer，每个 reader 只写属于自己的 flag；但仍需要 memory fence 保证 “payload 写完” 先于 “written 可见”。Python 层面的赋值顺序不等于所有 CPU 架构上的跨核可见顺序。

7.5 慢 reader 如何形成背压

ring 不是覆盖式日志。只要一个 reader 没把某 slot 标为已读，writer 绕一圈后就不能重用它。于是：



这不是一定说明共享内存坏了；根因可能是某个 Worker、GPU collective 或远端节点先卡住。源码会按 VLLM_RINGBUFFER_WARNING_INTERVAL 周期记录长等待，见 shm_broadcast.py。

7.6 SpinCondition：为什么共享内存旁边还有 PUB/SUB

reader 可以持续轮询 metadata，延迟低但空闲时烧 CPU；也可以每次阻塞等 socket，省 CPU 但高负载时增加系统调用。SpinCondition 做自适应折中，源码说明在 shm_broadcast.py：

- 最近持续有消息：sched_yield() 后快速重查共享 flag；
- 空闲超过阈值：用 Poller 等 ZMQ notification；
- writer 每次发布后发一个 ping；
- shutdown monitor 用同进程 PAIR/inproc 发 cancel ping，唤醒 idle reader。

由于共享 metadata 是真相来源，PUB/SUB ping 即使合并也不会丢失 payload。reader 醒来后重新检查 flag，而不是把 ping 当消息内容。

7.7 PEP 574 OOB buffer 的真实作用

enqueue 从 shm_broadcast.py 开始：

```
all_buffers[0] = pickle.dumps(
    obj,
    protocol=pickle.HIGHEST_PROTOCOL,
    buffer_callback=oob_callback,
)
```

当支持 buffer protocol 的大对象被 pickle 时，callback 可以把 payload 作为 PickleBuffer 留在主 pickle 流之外。当前 vLLM 对小于 1 MiB 的 buffer 仍 inline，大 buffer 加入 all_buffers，见 shm_broadcast.py。

逻辑结果是：

```
all_buffers[0] = pickle 元数据 + NEXT_BUFFER 引用
all_buffers[1] = 大 payload 的 memoryview
all_buffers[2] = 另一个大 payload 的 memoryview
...
```

reader 再调用：

```
pickle.loads(all_buffers[0], buffers=all_buffers[1:])
```

恢复对象，见 shm_broadcast.py。

逐段复制账本

假设 writer 广播一个含 8 MiB NumPy buffer 的对象给同机 8 个 Worker：

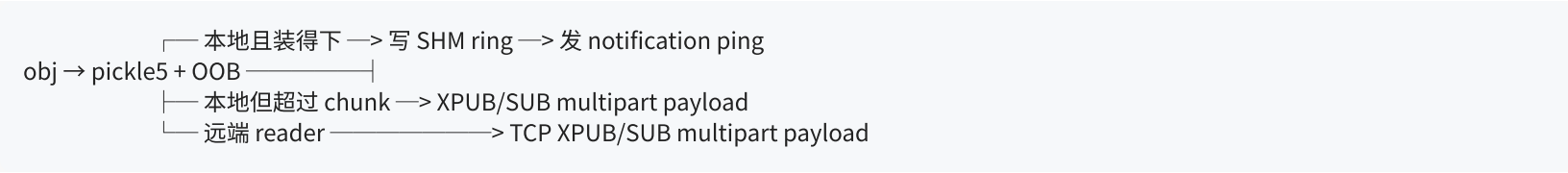
阶段	是否复制 8 MiB payload	说明
pickle 生成主流	通常不把它复制进主流	PEP 574 OOB 的核心收益
writer 写入 SHM slot	复制 1 次	buf[...] = buffer，见 shm_broadcast.py
8 个 reader 读取 SHM	不为每个 reader 再复制一份 IPC payload	都映射同一共享物理页
pickle.loads(..., buffers=...)	可基于 buffer 重建 view	是否最终共享取决于对象 reducer/消费者，不能一概保证
GPU 使用	可能还有 host→device 或设备内复制	属于另一层数据移动

因此比较合理的表述是：

对本机广播，vLLM 把“给 N 个 Worker 各复制一次 payload”降为“writer 写共享 ring 一次，N 个 reader 从同一共享段取 view”；PEP 574 还避免把大 buffer 先拼进 pickle 主流。它不是端到端 0-copy。

7.8 小消息、溢出消息和远端消息走哪条路

enqueue() 的分支位于 shm_broadcast.py：



本机 overflow 会先在 ring 中写一个 overflow=1 标志，再通过 local socket 发真实 multipart；reader 看到标志后转去 socket recv()。这保证 ring 顺序与大消息回退路径对齐。

send_multipart(all_buffers, copy=False) 对本机 overflow 和远端路径减少可避免的用户态拼接复制，但仍服从前述 ZMQ/transport 语义。

7.9 MessageQueue 在 Executor 里传什么

MultiprocExecutor 创建一个一写多读 rpc_broadcast_mq，将调用编码并广播给 Worker，见 vllm/v1/executor/multiproc_executor.py，核心是：

```
(send_method, args, kwargs, output_rank)
```

每个 Worker 从广播队列 dequeue()，执行方法，再通过自己的 response MessageQueue 回结果；相关入口都在 multiproc_executor.py 的 WorkerProc 中。

这意味着不能把 API↔Engine 的 msgpack 协议套到这里：

- API↔EngineCore：MsgpackEncoder/Decoder + ZMQ multipart；
- MultiprocExecutor↔Worker：pickle protocol 5/OOB + hybrid MessageQueue；
- Worker↔Worker Tensor：NCCL 等 device communicator。

8. GPU 通信是第三个世界：NCCL 不等于 IPC queue

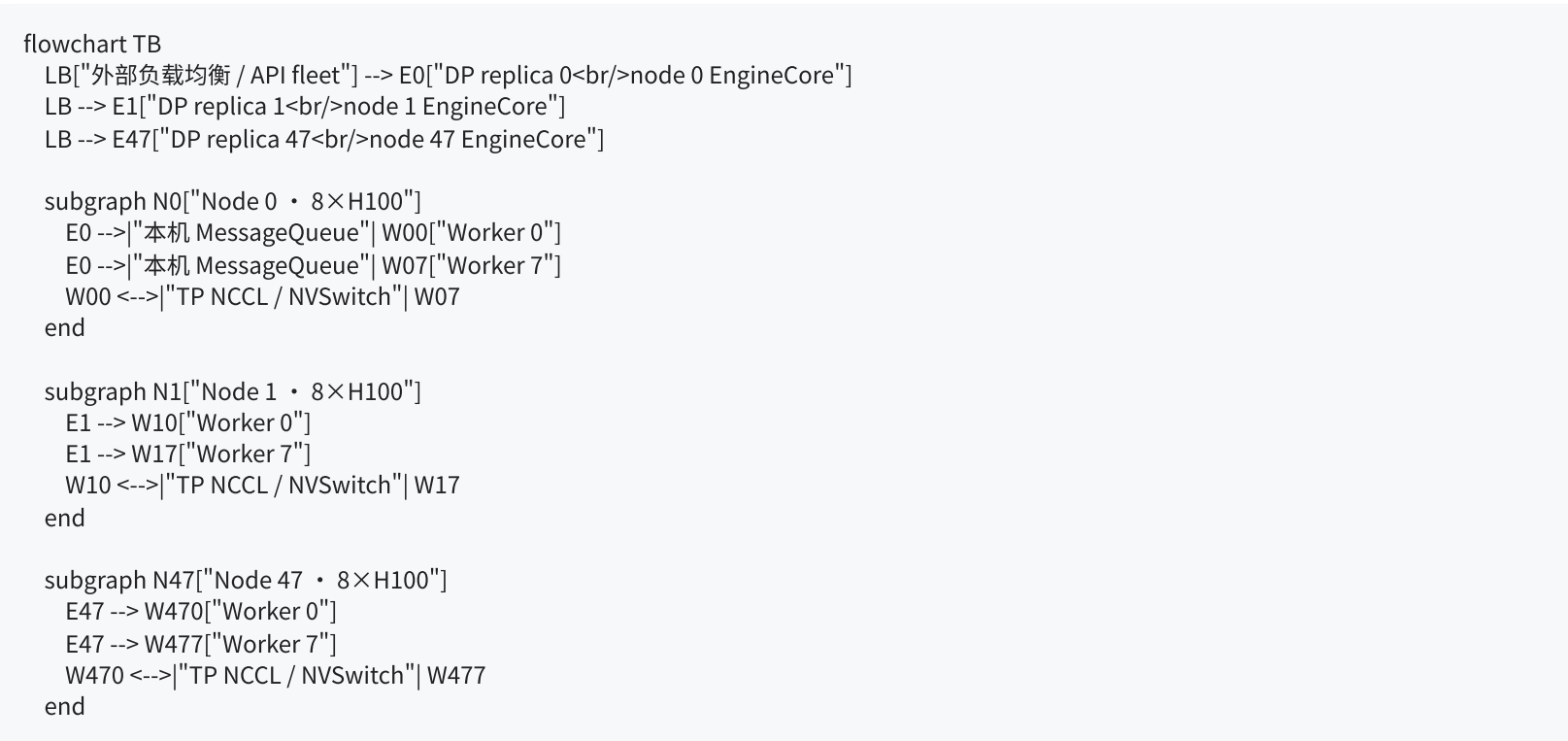
Worker 之间的 tensor parallel、pipeline parallel 或 expert parallel 通信，通常进入 NCCL/Gloo/custom communicator，而不是把 GPU tensor pickle 后塞进 MessageQueue。

层	典型内容	机制
API ↔ EngineCore	request、abort、token output、utility command	ZMQ + msgpack
EngineCore ↔ Worker	“执行哪个方法、参数是什么”、结果状态	MessageQueue：SHM + ZMQ + pickle5
Worker ↔ Worker	activation、partial logits、all-reduce/all-to-all 数据	NCCL / device communicator

节点内 NCCL 可能走 NVLink/NVSwitch 或 PCIe；跨节点可走 InfiniBand/RoCE + GPUDirect RDMA，也可能回退 TCP。这里的数据路径、拓扑和超时与 ZMQ 控制通道不同，但上层故障会互相表现为背压：一个 NCCL rank 卡住，会让对应 Worker 不再消费 MessageQueue，最终让 EngineCore 的 ring 也堵住。

9. 384×H100：不要画成“一条 384-reader 共享内存队列”

假设 48 台机器，每台 8×H100；一种常见映射是模型副本 TP=8、DP=48。这里只用于理解进程和通信边界，并不代表所有模型都应该固定 TP=8。



在这个例子中：

- Worker 数量约 384，每个 DP replica 有自己的 scheduler/EngineCore 和 8 个 TP Worker；
- 不是一个 EngineCore 用共享内存广播给 384 个进程；POSIX SHM 不能天然跨 48 台机器；
- 每台节点内的 executor 命令可走本机 SHM，TP collective 尽量留在 NVSwitch 域；
- API fleet 与多个 DP EngineCore 的控制路径可通过 TCP ZMQ 或外部路由组织；
- DP 副本通常彼此独立处理请求，不会为每一步做 384 卡全局 all-reduce。

如果单个模型副本必须跨节点，例如 TP=16 或 PP=2, TP=8：

- MessageQueue 会区分 local/remote reader，远端 payload 不能走本机 SHM；
- GPU collective/PP activation 会跨 NIC；
- 任何远端慢 rank 都可能反向形成 executor 背压；
- ZMQ TCP、NCCL/RDMA、服务网格/负载均衡是三套不同连接，监控和防火墙要分别处理。

9.1 进程数量不能只用 GPU 数估算

以 48 个 TP=8 副本为例，最低限度就可能包含：

- 384 个 Worker 承载 GPU；

- 约 48 个 EngineCore（每 DP rank 一个）；
- 若干 API Server；
- DP coordinator（具体取决于 LB 模式和 MoE wave coordination）；
- Ray actor/agent、日志和监控 sidecar、NCCL 辅助线程、ZMQ I/O thread。

所以容量与 pid limit、fd limit、/dev/shm 容量、Unix socket 路径、TCP 端口范围都要按**副本和节点**计算，不能只说“384 个进程”。详细部署切分见 ../08-production-deployment/13-384-h100-glm-deepseek-deployment.md。

10. 一条请求的端到端通信账本



看到延迟上升时按层分解：

1. API queue/tokenize；
2. ROUTER 输入排队/编解码；
3. scheduler queue；
4. MessageQueue 等 slot 或远端 ZMQ；
5. GPU compute/NCCL；
6. 输出 thread/PULL；
7. HTTP 客户端回压。

“GPU utilization 低”不能直接推出模型算子慢；也可能是 Worker 正等命令、EngineCore 被慢 reader 堵住、或前端输出回压。

11. 故障与背压排查

11.1 症状 → 可能层次

症状	优先怀疑	先看什么
启动卡在 waiting for engine core	握手、模型加载、EngineCore 早退	EngineCore stderr、ready timeout、进程存活
Address already in use	IPC path/TCP port 冲突	ss -xlpn、ss -ltnp、残留进程
No space left on device 但磁盘有空间	/dev/shm 不足	df -h /dev/shm、容器 --shm-size
ring buffer long wait	慢/死 Worker、NCCL hang、reader 未消费	rank 日志、GPU/NCCL 状态、metadata warning
CPU 空闲时仍很高	reader busy-spin 或大量序列化	线程栈、SpinCondition 活跃度
内存持续增长	ZMQ HWM 无界、上层 queue 堆积、引用等待 tracker	RSS/PSS、请求队列、pending messages
shutdown 卡住	linger、未关闭 fd、子进程/NCCL 未退出	进程树、fd、线程栈
fork 后 CUDA 错误	父进程提前初始化 CUDA	start method、import/初始化顺序
只有跨节点失败	TCP 路由/防火墙、NCCL NIC/RDMA、地址发布错误	分开验证 ZMQ port 与 NCCL interface

11.2 常用只读诊断命令

```
# 进程树、父子关系、状态和线程数
ps -eo pid,ppid,stat,nlwp,rss,cmd --forest | grep -E 'vllm|EngineCore|VllmWorker'

# 某进程打开的 fd; Unix/TCP socket 是否存在
ls -l /proc/<PID>/fd
ss -xlpn
ss -ltnp

# 共享内存容量和残留段
df -h /dev/shm
ls -lh /dev/shm

# 虚拟内存映射与共享/私有页 (PSS 比 RSS 更适合看共享页)
cat /proc/<PID>/smaps_rollup

# GPU 进程、显存和利用率
nvidia-smi
```

不要看到 /dev/shm/psm_* 就直接删除。先确认创建进程已经退出且没有 reader 仍 attach；物理段的生命周期由 close()/unlink() 协作管理，误删活跃段会让其他进程出错。当前 ShmRingBuffer.__del__ 只有 creator 执行 unlink，见 shm_broadcast.py。

11.3 分层二分法

1. **进程层**：API、EngineCore、所有 Worker 是否存活？PPID 是否合理？

2. **握手层**：ready identity 是否收齐？IPC/TCP endpoint 是否可达？
3. **队列层**：卡在 input queue、SHM acquire、response dequeue 还是 output queue？
4. **GPU 层**：所有 NCCL rank 是否进入同一个 collective？
5. **客户端层**：输出是否已生成但 HTTP client 不读？

不要一上来同时修改 NCCL、ZMQ HWM、共享内存大小和 batch 参数。先定位阻塞层，再改一个变量。

12. 三个最小实验

这些实验用于建立直觉，不是 vLLM benchmark。

12.1 验证 “普通变量隔离，共享段可见”

```
from multiprocessing import get_context, shared_memory

normal = bytearray(b"parent")

def child(shm_name: str) -> None:
    normal[:] = b"child!"      # 只改子进程自己的堆
    shm = shared_memory.SharedMemory(name=shm_name)
    shm.buf[:6] = b"shared"     # 改共同映射的物理页
    shm.close()

if __name__ == "__main__":
    ctx = get_context("spawn")
    shm = shared_memory.SharedMemory(create=True, size=6)
    shm.buf[:] = b"parent"
    p = ctx.Process(target=child, args=(shm.name,))
    p.start(); p.join()
    print(normal)              # bytearray(b'parent')
    print(bytes(shm.buf))      # b'shared'
    shm.close(); shm.unlink()
```

12.2 看懂 ROUTER identity 和 multipart

```
import multiprocessing as mp
import zmq
```

```
ADDR = "tcp://127.0.0.1:29876"
```

```
def dealer() -> None:
    ctx = zmq.Context()
    s = ctx.socket(zmq.DEALER)
    s.setsockopt(zmq.IDENTITY, b"engine-7")
    s.connect(ADDR)
    s.send_multipart([b"ADD", b"payload"])
    print(s.recv_multipart())
    s.close(); ctx.term()

if __name__ == "__main__":
    ctx = zmq.Context()
    router = ctx.socket(zmq.ROUTER)
    router.bind(ADDR)
    p = mp.get_context("spawn").Process(target=dealer)
    p.start()
    identity, request_type, payload = router.recv_multipart()
    print(identity, request_type, payload)
    router.send_multipart([identity, b"OK"])
    p.join()
    router.close(); ctx.term()
```

观察：ROUTER 收到三帧，DEALER 只收到业务帧 OK；identity 是路由信封，不会自动进入 DEALER 的业务消息。

12.3 观察 PEP 574 的 OOB

```
import pickle
import numpy as np
```

```
a = np.arange(1_000_000, dtype=np.float32)
oob = []
main = pickle.dumps(a, protocol=5, buffer_callback=oob.append)

print(len(main), len(oob), memoryview(oob[0]).nbytes)
b = pickle.loads(main, buffers=oob)
print(np.shares_memory(a, b))
```

这个实验演示 pickle API 可以让重建对象共享原 buffer；它不代表 vLLM 跨进程 SHM 写入没有复制。vLLM 还要把 OOB buffer 写进 ring，具体复制发生在 shm_broadcast.py。

13. 常见误区逐条纠正

误区	准确说法
“进程之间绝对不能共享内存”	默认地址空间隔离，但可以显式映射同一 shared memory
“fork 会立刻复制全部内存”	通常先 COW 共享页，写入时按页复制
“Python 线程完全没有并行”	GIL 限制 Python bytecode；I/O 和释放 GIL 的 native code 可并行
“bind 的一端就是客户端”	bind/connect 是地址所有权；业务角色由协议和 socket type 决定
“ZMQ 就是 TCP”	ZMQ 是消息层，可用 inproc/ipc/tcp transport
“一次 send 对应一次 TCP recv”	TCP 是字节流；ZMQ 才保存 message/multipart 边界
“PUB/SUB 保证每条消息送达”	慢加入、HWM 和断线都可能导致丢失；它不是持久消息系统
“copy=False 就没有任何复制”	只减少特定用户态复制；内核、网络、SHM 写入、GPU 搬运仍可能复制
“PEP 574 自动让所有对象零拷贝”	只对支持 buffer protocol/reducer 的数据有效，并依赖 consumer 如何重建
“MessageQueue 就是共享内存”	当前实现还用 ZMQ 通知、overflow 回退和远端传输
“NCCL hang 与 IPC 无关”	根因在 GPU 层，但会让 Worker 停止消费 IPC，最终表现为 ring 背压
“384 卡就是一个 384 进程通信组”	通常按 DP replica、节点和并行域分层，不应建立全局 SHM queue

14. 代码索引

想追的问题	源码位置
InprocClient 直接调用 EngineCore	vllm/v1/engine/core_client.py
MPClient 创建 ROUTER/PULL	vllm/v1/engine/core_client.py • MPClient
client 等待所有 EngineCore READY	vllm/v1/engine/core_client.py
client 组装 identity/type/data multipart	vllm/v1/engine/core_client.py
EngineCore process 创建	vllm/v1/engine/utils.py • CoreEngineProcManager
EngineCore 输入/输出 I/O thread	vllm/v1/engine/core.py
EngineCore DEALER 收消息	vllm/v1/engine/core.py
EngineCore PUSH 发输出	vllm/v1/engine/core.py
ZMQ socket 工厂/HWM/identity/linger	vllm/utils/network_utils.py
msgpack 与附属 tensor/ndarray buffer	vllm/v1/serial_utils.py
start method 强制条件	vllm/utils/system_utils.py • _maybe_force_spawn / get_mp_context
MultiprocExecutor 创建广播 MQ	vllm/v1/executor/multiproc_executor.py
Worker process、ready/death pipe	multiproc_executor.py • WorkerProc
SpinCondition 的 ZMQ 通知	vllm/distributed/device_communicators/shm_broadcast.py
SHM ring 布局和 attach	shm_broadcast.py
MessageQueue 本地/远端分支	shm_broadcast.py
writer/reader slot 状态转换	shm_broadcast.py • acquire_write / acquire_read
pickle5/OOB、SHM copy、overflow	shm_broadcast.py

外部基础资料：

- [Python multiprocessing：start methods](#)
- [Python SharedMemory 官方文档](#)
- [PEP 574：pickle protocol 5 与 out-of-band buffer](#)
- [libzmq socket 官方语义](#)
- [ØMQ Guide](#)

15. 面试追问：从定义走到源码

15.1 为什么 vLLM 不把所有东西都放进 multiprocessing.Queue？

答题链：

1. multiprocessing.Queue 适合通用对象，但通常经 pickle、pipe 和 feeder thread；
2. 高频广播给多个 Worker 会重复序列化/复制，慢 reader 和容量也难按 vLLM step 协议精确控制；
3. vLLM 用 single-writer/multi-reader SHM ring，使本机多个 reader 读同一份 payload；
4. 用 PEP 574 拆出大 buffer，避免塞入 pickle 主流；
5. 仍用 ZMQ 解决通知、overflow 和跨节点，因为 SHM 不能跨主机。

15.2 ROUTER/DEALER 为什么适合 DP EngineCore?

答题链：

1. client 需要管理多个 EngineCore connection；
2. EngineCore 用稳定 identity 注册；
3. ROUTER 发消息时用第一帧选择 EngineCore；
4. DEALER 是异步双向，不要求严格 request/reply lockstep；
5. ready handshake 先建立 identity 路由，避免 ROUTER 在未知连接上发消息。

15.3 共享内存为什么还需要同步？

答题链：共享物理页只解决“看到同一批 bytes”，不解决消息边界、写完/未写完、多个 reader 的进度和 CPU 内存顺序。vLLM 用固定 chunk、written flag、每 reader flag、single-writer 约束和 memory fence 建立协议。

15.4 一个 Worker 卡住为何拖住全部 Worker?

它不一定立刻拖住其他 Worker，但广播 ring 的 slot 只有所有 reader 都确认后才可复用。慢 reader 累积到 ring 绕回时，writer 不能覆盖未读 slot，于是停止发布新调用；其他 Worker 随后也拿不到新工作。这是有界 broadcast queue 的背压语义。

15.5 “零拷贝”该怎么严谨回答？

不要回答“用了 mmap，所以 0 copy”。应列路径：pickle OOB 避免大 payload 进入主流；writer 仍将 payload 复制一次进 SHM；N 个本地 reader 直接 view 同一共享段；对象重建和 H2D 可能继续复制。copy=False 也只针对 pyzmq 某段用户态路径。

15.6 384 卡时为什么不做一个中心 EngineCore?

一个中心 scheduler 会形成 CPU、队列、故障域和跨节点广播瓶颈；POSIX SHM 又不能跨节点。常见做法是按 DP replica 分 EngineCore，每个副本只调度自己的一组 TP/PP Worker，由外层 LB 分发请求。只有模型本身跨节点时，才在单副本内引入 remote reader 和跨节点 collective。

15.7 ZMQ 自动重连是否等于请求不会丢？

不等于。重连只恢复 transport connection；它不知道某条推理请求是否已到达、已执行或输出是否被消费。需要应用层 request id、状态机、超时、幂等 abort/重试策略。持久化和 exactly-once 不是 ZMQ 默认能力。

- 进程默认地址空间隔离；共享内存通过页表映射让多个进程访问同一物理页，但同步和生命周期仍由应用负责。
- fork 主要靠 COW，不会立刻复制全部内存；CUDA/线程/锁状态让 spawn 在部分场景更安全。vLLM 会根据环境强制选择。
- ZMQ 是异步消息库，不是单纯 TCP wrapper，也不是持久消息代理。理解 frame、identity、socket pattern、HWM、握手和 linger 才能排障。
- API↔EngineCore 使用 ZMQ + msgpack multipart；EngineCore↔Worker 的 MessageQueue 是 SHM + ZMQ + pickle5/OOB 的混合机制；GPU tensor collective 属于 NCCL 层。
- PEP 574 和 copy=False 都只省特定复制，不能笼统宣称端到端零拷贝。
- 384×H100 应按 DP replica、节点和 TP/PP 通信域分层建模，而不是画成一个全局共享内存队列。

自检

1. 两个进程打印出相同虚拟地址，是否说明共享同一物理内存？为什么？
2. fork 后父子进程的 10 GiB heap 是否立刻占用额外 10 GiB？什么时候会增长？
3. spawn 为什么要求 target 可 pickle，入口为什么要加 __main__ guard？
4. ZMQ bind/connect 与业务上的 client/server 有何区别？
5. ROUTER 收到的 identity frame 到 DEALER 端后为什么消失？
6. PUSH/PULL 与 PUB/SUB 在“一条消息有几个消费者”上有何区别？
7. HWM 达到后，所有 socket type 都会阻塞吗？
8. 为什么 SHM notification 可以 CONFLATE，真实 scheduler command 却不能随便丢？
9. buf[buf_offset:...]=buffer 对“零拷贝”结论意味着什么？
10. 一个远端 Worker 卡在 NCCL，为什么可能最终表现为本地 EngineCore ring buffer long wait？
11. 48 台 8×H100、TP=8、DP=48 时，大致有多少 Worker 和 EngineCore？哪些通信通常不跨节点？
12. API→Engine 使用的 msgpack/OOB 与 Executor→Worker 使用的 pickle5/OOB 是同一个协议吗？

如果第 8、9、10 题能沿源码状态机讲清楚，才算真正理解这一章。

参考答案

1. **不一定。** 虚拟地址只是在每个进程自己的页表中有相同数值；两个页表可能映射到不同物理页。只有使用共享内存、同一 mmap 文件映射或显式 CUDA IPC，并核对映射关系，才能证明共享了同一物理存储。
2. fork 采用写时复制，父子进程最初共享只读物理页，所以不会立刻多占 10 GiB。任一方写入某页时才触发 page fault，内核复制该页；实际增长量取决于被修改的页，而不是整个 heap 大小。
3. spawn 会启动全新的解释器，只能通过 pickle 把 target、参数和必要状态传过去；__main__ guard 防止子进程重新 import 主模块时再次执行启动逻辑，避免递归创建进程。
4. ZMQ 的 bind / connect 是拓扑动作，描述谁拥有监听端点；它不等同于业务上的 server/client。一个业务 client 可以连接另一个业务 client 的 socket，真正的请求方向由 socket 类型和消息协议决定。
5. ROUTER 端收到的 identity 是路由元数据，ROUTER 用它选择返回路径；转发给 DEALER 时通常只传业务帧，因此 DEALER 看不到 identity。若需要在业务层保留请求关联，必须显式放入 request ID。
6. PUSH/PULL 是负载分发：一条消息只会交给一个下游 PULL。PUB/SUB 是广播：每个订阅者都可能收到消息，但订阅建立前的消息通常会丢失，且没有逐消费者确认。
7. 不会。HWM 只表示队列水位；不同 socket 的满队列行为不同，可能阻塞、返回 EAGAIN、丢弃旧消息或丢弃新消息。生产代码必须根据 socket 类型、SNDTIMEO/RCVTIMEO 和错误码验证实际语义。
8. notification 是“状态已变化”的提示，旧通知被新通知覆盖通常仍能由消费者重新读取最新状态，所以可以 CONFLATE。scheduler command 是有顺序、有副作用的增量操作，丢一条可能导致请求、KV block 或 worker 状态永久不一致。
9. 这行代码把 buffer 的内容复制到目标 slice，因此它不是零拷贝；零拷贝只能说明元数据或 buffer 所有权被复用，不能把一次实际内存 copy 误说成零拷贝。应结合 data_ptr、shared-memory buffer 和 profile 证据判断。

10. EngineCore 把 command 写入 worker ring 后等待 completion；远端 worker 若卡在 NCCL，相关 completion 不会产生，本地 ring slot 也不会释放，最终表现为 EngineCore 的 long wait、queue 堆积甚至 API 超时。根因在 collective，表象在 IPC。
11. 若每个 TP=8 的服务单元占一台 8-GPU 节点，则 48 台共有 384 个 GPU worker；若每台节点上一个 EngineCore/服务单元，则约 48 个 EngineCore。TP 内的 NCCL 通常在节点内，DP 副本之间不需要同步；跨节点通信取决于 PP、EP 或具体部署拓扑。
12. 不是同一个协议。API→Engine 通常是 ZMQ 上的 msgpack/OOB；Executor→Worker 是进程内/本机的 pickle5/OOB 与共享内存队列。它们都可借用 out-of-band buffer 降低复制，但 framing、生命周期和故障语义不同，不能互换。

下一步

- 想继续看 scheduler 如何形成每一步执行决策：../03-code-walkthrough/02-scheduler.md
- 想看 TP/PP/EP 与 collective：../05-distributed/01-tp-pp-ep.md
- 想把本章映射到 384×H100 生产部署：../08-production-deployment/13-384-h100-glm-deepseek-deployment.md

01. PagedAttention 详解

谁该读这一篇？ 想用一个概念把 vLLM 的核心创新讲透，并把 PagedAttention 与 OS 虚拟内存类比建立起来的读者。

前置阅读： 01-overview/00-prerequisites.md § 4 KV cache、§ 6 GPU 内存层级；以及 01-overview/01-what-is-vllm.md（三大瓶颈的第一条）。

耗时： 约 18 分钟。

学完能：

1. 在白板画出"OS 虚拟内存 ↔ PagedAttention"对照表（页 / 页表 / COW / 缺页）。
2. 算清 block_size=16 的取舍，并能解释为什么不取 1 或 128。
3. 描述一个请求从 prefill 到 decode 在 BlockPool / BlockTable 上的 KV 分配序列。
4. 讲清"PagedAttention 跟 FlashAttention 是什么关系" / "PagedAttention vs RadixAttention 的区别"。

当前源码复核 (b23bd73)： CacheConfig.DEFAULT_BLOCK_SIZE 为 16，但 block_size=None 表示先走平台默认，混合 KV 模型还会做组间对齐；下文用 16 做算例，不把它当成所有硬件/模型的不可变默认。PagedAttention 是 KV 地址管理接口，FlashAttention/FlashInfer/Triton 等是受能力约束的执行后端。

vLLM 的心脏。如果只能讲清一个概念，必须是它。

1. 类比：从操作系统虚拟内存说起

OS 解决进程内存碎片的办法：

- 物理内存切成 page（页，比如 4 KB 一个）
- 进程看到的是虚拟地址空间，连续
- 通过页表把虚拟页映射到物理页
- 物理页可以不连续
- 进程之间可以共享同一物理页（如 fork 后的 copy-on-write）

PagedAttention 完整复刻这套：

OS 概念	PagedAttention 对应
物理页 (Page)	KV block（默认 16 个 token 的 K+V）
虚拟地址空间	请求看到的"逻辑 KV cache"
页表	Block Table（每个请求一张）
页错误	没有 block 可分时触发抢占（preempt）
共享页 (CoW)	Prefix caching（共享前缀的 block）
页大小	block_size（默认 16，可调）

2. 朴素 KV cache 的问题

设：模型 hidden_size=4096、num_layers=32、FP16。一个 token 的 KV 占用： $2 \text{ (K,V)} \times 4096 \times 2 \text{ (FP16)} \times 32 = 524 \text{ KB}$ 。

服务一个 max_seq_len=4096 的请求 → 预留 **2 GB**。

10 个并发请求、每个实际只用 200 token：

项目	大小
预留	$10 \times 2 \text{ GB} = \mathbf{20 \text{ GB}}$
实际用量	$10 \times 200 \times 524 \text{ KB} \approx \mathbf{1 \text{ GB}}$
浪费	19 GB

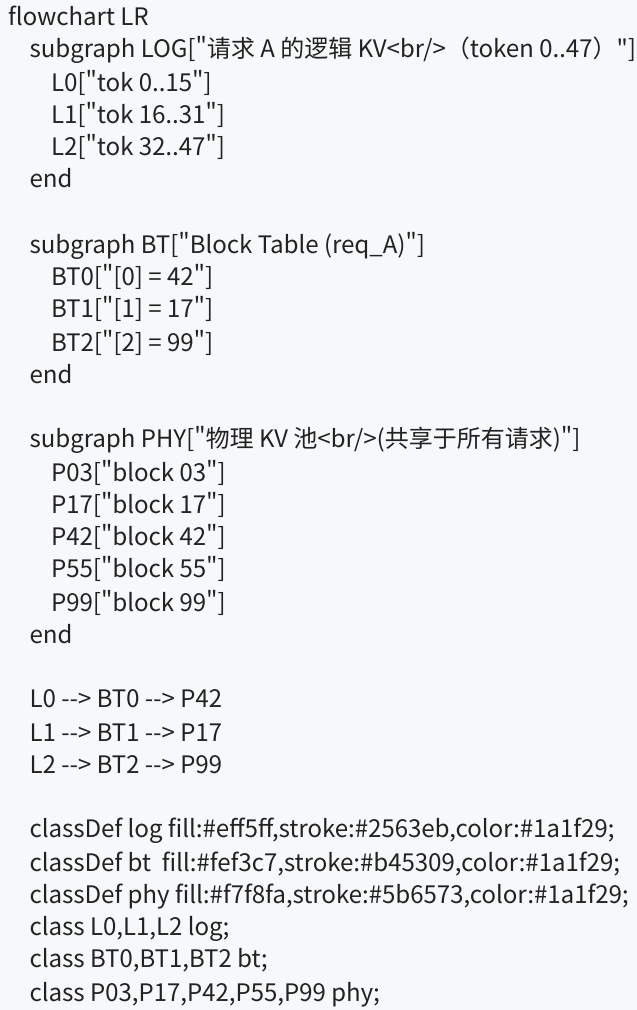
PagedAttention 论文图 4 报告的有效利用率只有 20-38%——剩下的全在内/外部碎片里耗掉。

3. 存储布局：物理 block + Block Table

物理 KV cache 在显存里是一个大张量。每种 attention 后端布局略不同；FlashAttention 后端的形状是：

key_cache: [num_blocks, block_size, num_kv_heads, head_size]
value_cache: [num_blocks, block_size, num_kv_heads, head_size]

num_blocks 是显存能装下的最大物理 block 数。每个请求持有一张 block table 把逻辑位置映射到物理 block：



关键性质：

- 物理 block 无需相邻 → **外部碎片 = 0**
- 同一个物理 block 可以挂在多个请求的 block table 下 → **跨请求共享**
- 增长一个 block 是 O(1) (追加一个 entry)，不需要搬数据

4. 计算流程

数学公式还是经典的：

$$\text{out} = \text{softmax}!\left(\frac{Q K^{\text{top}}}{\sqrt{d}}\right) V$$

K、V 不再是连续张量，要通过 block table 间接访问。

Prefill 阶段（一次喂 N 个 token，N > 1）：

1. 算 N 个 token 的 Q、K、V
2. 按顺序把 K、V 写入 block_table 指向的物理 block
3. 跑标准 attention（FlashAttention 或 paged kernel）

Decode 阶段（每步只 1 个新 token）：

1. 算这个新 token 的 q、k、v

2. 追加写到最后一个 block 的下一格
3. 算 $q \cdot K^T$ 时，按 block_table 逐 block 加载历史 K 做 dot product

decode kernel 的循环骨架（伪代码）：

```
// PagedAttention decode kernel
for block_idx in block_table[req_id]:
    K_block = key_cache[block_idx] // 一次 indirect read
    partial = softmax(q @ K_block.T) @ value_cache[block_idx]
    accumulate(partial)
```

实际代码：csrc/attention/paged_attention_v1.cu、paged_attention_v2.cu（v2 用 split-K 并行长序列）。

5. 为什么常用 block_size=16

- 太小：每个 block 一次 indirection，block_table 变长，访存开销线性放大
- 太大：内部碎片回归（每个请求最多浪费 block_size - 1 个 token 的槽位）

16 是工程经验：

- 配合 FlashAttention 的 tile size 友好
- 配合 RoPE / GQA 的 head_dim 友好
- 内部碎片影响 < 2%

可通过 --block-size 调整；有效范围与限制由当前平台实现校验，不能假设所有后端都支持同一组候选值。

6. 共享：Copy-on-Write 的应用

PagedAttention 的另一个能力是跨请求 / 跨候选共享 block。


```
flowchart LR
    subgraph A["请求 A • block_table"]
        A0["[0]"] --> P42
        A1["[1]"] --> P17
        A2["[2]"] --> P99
    end

    subgraph B["请求 B • block_table (同 system prompt) "]
        B0["[0]"] --> P42
        B1["[1]"] --> P17
        B2["[2]"] --> PX["block 88<br/>(B 独有)"]
    end

    P42["block 42<br/>ref_cnt = 2"]
    P17["block 17<br/>ref_cnt = 2"]
    P99["block 99<br/>ref_cnt = 1"]
    PX

    classDef shared fill:#fef3c7,stroke:#b45309,color:#1a1f29;
    classDef owned fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
    class P42,P17 shared;
    class P99,PX owned;
```

三种用法：

场景	行为
Prefix caching	两个请求同 system prompt，前缀 block ref_cnt = 2，最后一个引用消失才回收
Beam search （beam=4）	4 条候选可能共享前缀，到分歧点 fork：共享时 ref_cnt++，分歧时 copy-on-write
Parallel sampling （n=4）	prompt 段 4 个候选共享 block，generation 段各自独立

7. 代码地标

概念	文件	关键类 / 函数
KV cache 张量分配	vllm/v1/worker/gpu_worker.py	initialize_cache
Block Pool（空闲池）	vllm/v1/core/block_pool.py	BlockPool
Block hash（前缀复用）	vllm/v1/core/kv_cache_utils.py	hash_request_tokens
KVCacheManager	vllm/v1/core/kv_cache_manager.py	allocate_slots()
Block Table（GPU 侧）	vllm/v1/worker/block_table.py	BlockTable
PagedAttn Python API	vllm/v1/attention/backends/flash_attn.py 等	—
PagedAttn CUDA	csrc/attention/	paged_attention_v1/v2_kernel

8. 推荐的源码阅读路径

吃透 PagedAttention 的顺序：

1. `vllm/v1/core/block_pool.py` - `BlockPool.__init__`：物理 block 池怎么初始化 - `get_new_blocks` / `free_blocks`：分配 / 释放
2. `vllm/v1/core/kv_cache_manager.py` - `allocate_slots(request, num_new_tokens)`：核心逻辑（算需要几个 block、查 prefix cache、不够就 preempt）
3. `vllm/v1/worker/block_table.py` - `BlockTable`：在 GPU 上维护 `block_id` 序列；调度器算好的逻辑 → GPU tensor
4. `vllm/v1/attention/backends/flash_attn.py` - 实际 attention 调用；看它怎么把 `block_table` 传给 CUDA kernel
5. `csrc/attention/paged_attention_v1.cu` - 真正的 GPU 内核（想看 CUDA 实现的话）

9. 论文 vs 当前实现

读 PagedAttention 论文容易以为 vLLM 自己写了 attention kernel。实际现状：

- 2023 论文版：vLLM 自己的 `paged_attention_v1/v2.cu`
- 2024 之后：默认走 **FlashAttention v2/v3** 或 **FlashInfer**，它们原生支持 paged KV（接受 `block_table` 参数）
- vLLM 自己的 paged kernel 只在没有 FlashAttn 时 fallback

被问"vLLM 的 attention kernel 你看过吗"，可以这样答：

"vLLM 现在主用 FlashAttention 和 FlashInfer 的 paged 版本，它们都接受 `block_table` 作为参数。vLLM 自己也写了 `paged_attention_v1/v2.cu` 作为 fallback，v2 用 split-K 来并行长序列。"

10. 工程自检问答

Q: PagedAttention 会不会比连续 KV 慢？ A: 会，但很少。每个 block 一次 indirect lookup，`block_size=16` 摊薄了开销。论文实测 attention 慢 ~20%，但整体吞吐提升 24×，总账净赚。

Q: block_table 怎么传到 GPU？ A: Scheduler 在 CPU 维护逻辑 `block_id` 列表；每步开始时 `InputBatch` 把所有 active 请求的 `block_table` 打包成一个 GPU tensor 一次性 copy，避免逐请求小拷贝。

Q: 100k token 的请求 block_table 不会很长吗？ A: $100000 / 16 = 6250 \text{ entries} \times \text{int32} \approx 25 \text{ KB}$ ，完全可控。访问 `block_table` 本身不是瓶颈。

Q: PagedAttention 与 RadixAttention (SGLang) 的区别？ A: PagedAttention 是**线性 block 序列**，每个请求一张表。RadixAttention 把所有请求的前缀组织成 **trie/radix tree**，前缀复用更激进（任意公共前缀都能自动发现，不只是 system prompt）。代价是树维护更复杂、写时拷贝更频繁。

小结

- PagedAttention = **把 KV cache 当 OS 物理页管**：定长 block + 每请求一张 BlockTable + 物理 block 池 + refcount + COW + LRU 复用。每条性质都能在 OS 教材里找到原型。
- 通用 fallback block_size=16 是工程取舍——太小 indirection 多、太大内部碎片回归；最终值仍受平台、模型与显式配置约束。
- "PagedAttention 论文"讲的是思想，**当前代码主要走 FlashAttention v2/v3 / FlashInfer 的 paged 版本**，自家 paged_attention_v1/v2.cu 只是 fallback。
- 共享语义有三种用法：跨请求 prefix caching、beam search、parallel sampling——本质都是 ref_cnt 的加减游戏。
- 它跟 RadixAttention 是"线性 block 序列" vs "trie / radix 树"的区别——前者实现简单稳定，后者前缀复用更激进。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. OS 虚拟内存 vs PagedAttention 6 行对照表。

OS	PagedAttention	vLLM 类 / 字段
物理页 (Page)	KV block （默认 16 token 的 K+V）	KVCacheBlock （block_pool.py）
页表 (Page Table)	Block Table	block_table : list[int] 每请求一份
进程地址空间	请求的逻辑 token 序列	Request.token_ids
物理内存池	GPU HBM 上预分配的 block 池	BlockPool 的 free queue （LRU 双链表）
缺页（请求新页）	get_new_blocks 时 ref_cnt++	KVCacheManager.allocate_slots
释放 / swap	请求完成 / preempt 时 ref_cnt 归零 → 回 free queue	_free_request_blocks / preempt 流程
COW（写时复制）	Beam search / prefix 共享 → 分歧时 copy	block ref_cnt > 1 时 split

补充：再讲清 **LRU 淘汰**（OS swap victim ↔ prefix cache evict 冷 block）。

2. block_size=16, hidden=4096, num_layers=32, FP16, 100k token 请求的 BlockTable / KV 占多少？

BlockTable（每请求一张）：

- 需要的 block 数 = $\lceil 100000 / 16 \rceil = 6250$ 个
- 每条 entry 是个 int32（physical_block_id）= 4 字节
- BlockTable 大小 = $6250 \times 4 = \mathbf{25\ KB}$

物理 KV：

- 单 token 单层 KV = $2\ (K+V) \times \text{hidden} \times \text{dtype_bytes} = 2 \times 4096 \times 2 = 16\ \text{KB}$
- 32 层一个 token = $16\ \text{KB} \times 32 = 512\ \text{KB}$
- 100k token = $512\ \text{KB} \times 100000 = \mathbf{48.8\ GB}$

→ BlockTable 比 KV 小 200 万倍。BlockTable indirection 的内存开销可以忽略，**真正的内存大头是物理 KV**。

敏感性：如果开 GQA（8 KV head 而非 32），KV 缩到 12.2 GB（1/4）；MLA 进一步缩到 ~6 GB（约 1/8）。

3. get_new_blocks 与 free_blocks 各做什么 ref_cnt 修改？

源码：vllm/v1/core/block_pool.py

- **get_new_blocks(num_blocks)：** python for _ in range(num_blocks): block = free_queue.popleft() # 从 free queue 头取（LRU 顺序）
block.ref_cnt = 1 # 新分配的归属新请求，ref_cnt = 1 yield block
- **free_blocks(blocks)**（请求结束 / preempt）： python for block in blocks: block.ref_cnt -= 1 if block.ref_cnt == 0:
free_queue.append(block) # 回到 free queue 尾（LRU 尾，最近释放的最后被复用）

ref_cnt > 1 的情况：prefix caching 命中时 _get_cached_block() 给 hash 命中的 block ref_cnt++（多请求共享）。Beam search / parallel sampling 同理。这就是为什么不是 ref_cnt = 0 就立即归还——共享 block 还在被用。

4. PagedAttention 单步慢 ~20%，端到端为什么还能 24× 吞吐？

慢的来源：每个 KV 访问要先查 block_table（间接寻址），多一层指针 chase，attention kernel 比连续 KV 的版本慢约 20%。

赚回来的更多：

1. **KV 利用率 20% → 96%：**同一张 H100 装下 5× 的并发请求
2. **GPU 利用率 30% → 80%+：**continuous batching 让 GPU 不空转
3. **prefix caching 净赚：**相同前缀直接复用，跳过 prefill，省的不只是 20% 而是 95%+

算术：单步慢 20%（× 0.8 throughput）× 并发 5×（× 5）× GPU 利用 2.5×（× 2.5）= **× 10 起步**，加上 prefix caching 等其他优化，论文给的 24× 来自 OPT-13B benchmark，realistic。

类比：超市收银台从"一个客人一个收银员"变成"虚拟收银号 + 任意收银台"——单笔结账可能慢 1 秒（要看号），但整个超市同时间能处理的人数翻了几倍。

5. 用一段话答"vLLM 自家 paged kernel 现状"。

"PagedAttention **思想**是 vLLM 的核心论文贡献，但**实现**已经走过两代演化。早期（2023-2024）vLLM 自己写了 paged_attention_v1.cu 和 paged_attention_v2.cu，论文里报的 24× 主要来自这套自家 kernel。

但 FlashAttention v2/v3 / FlashInfer 后来都原生支持了 **paged + varlen** 模式——同样的 block_table 间接寻址，他们的 kernel 比 vLLM 自家版本快得多（更激进的 tiling、async memcpy、CTA scheduling）。

现在的 vLLM：**默认走 FlashAttention 或 FlashInfer 的 paged 版本**，自家 paged_attention_v1/v2.cu 只是在某些 GPU 不支持 FlashAttention 时 fallback。MLA 模型走 FlashMLA（DeepSeek 自家 kernel），Mamba 模型走专门的 state-space kernel。

所以讲 PagedAttention 时，**思想引用论文，实现要说 FlashAttention paged 版本**——不然知识会停留在 2023。"

下一步

- 下一节：02-continuous-batching.md (PagedAttention 让 KV 不必连续后，调度才能做"每步重组 batch")。
- 想看 KV 管理细节：03-kv-cache-management.md (启动时 num_blocks 怎么算、BlockPool 的 LRU 怎么工作)。
- 想看源码：vllm/v1/core/block_pool.py (物理 block 中央仓库)、vllm/v1/core/kv_cache_manager.py (Scheduler 看到的接口)、csrc/attention/paged_attention_v1.cu (自家 CUDA kernel)。
- 想动手：07-hands-on/03-mini-experiments.md (亲手观察 BlockTable 在 prefill / decode 时的变化)。
- 想从生产视角理解：08-production-deployment/01-deployment-architectures.md (block_size、kv-cache-dtype 在不同生产形态下怎么调)。

02. Continuous Batching（连续批处理）

谁该读这一篇？ 想搞清"为什么 vLLM 比 HF Transformers 吞吐高一个量级"且不靠 buzzword 讲清的读者；想自己实现一套 iteration-level 调度器的工程师。

前置阅读： 01-paged-attention.md（PagedAttention 让 KV 不必连续，是 continuous batching 能跑起来的物理前提）；01-overview/00-prerequisites.md § 8 batching。

耗时： 约 14 分钟。

学完能：

- 1. 在白板画出 No / Static / Continuous 三种 batching 的时间线，并说出 GPU 利用率差异。
- 2. 用"token budget" 概念解释 max_num_batched_tokens 如何同时影响吞吐和延迟。
- 3. 区分 vLLM 的 token-level 调度与 TGI 的 sequence-level 调度。
- 4. 解释当前 V1 的 recompute 抢占路径，并把它与 KV offload / connector 区分开。

当前源码复核（b23bd73）： max_num_batched_tokens 没有单一生产默认：EngineArgs.get_batch_defaults() 按 usage context、设备显存/型号、CPU/TPU 与并行方式选择（常见 API server 为 2048，≥70 GiB 且非 A100 的设备为 8192）。当前 V1 _preempt_request() 只实现释放 KV、清零计算进度并重新排队，不存在可切换的 --preemption-mode swap。

这是 vLLM 性能上的第二个杀器，与 PagedAttention 平级重要。它解决"GPU 在等慢请求"的问题。

1. 三种 batching 直观对比

1.1 No Batching（朴素逐请求）

GPU: [A.....][B.....][C.....][D.....]
请求一个接一个跑。GPU 长期空转。

1.2 Static Batching（HF Transformers 默认）

T=0 T=1 T=2 ... T=10
[ABCD][ABCD][AB-D] ← C 完了，但槽位还占着 ... [---D]
 ↑
 只剩 D 在跑，浪费 75%

所有请求**同时进、同时出**。最慢的那个决定整组的延迟。

1.3 Continuous / Iteration-level Batching（vLLM）



每一步（每生成一个 token）都检查：

- 谁完成了 → 立刻退出，释放 KV
- 等待队列有谁 → 立刻加入

GPU 几乎从不空转。

2. 为什么"每一步都调度"是可行的？

直觉上你可能觉得：每一步都重新组 batch、重建 tensor，开销不大吗？

vLLM 的回答是：

- PagedAttention 让 KV 不必连续。**请求加入/退出 batch 不需要搬数据。
- InputBatch 持久化**（V1 优化）：每步只更新 diff，不重建。
- Token-level**而非 sequence-level 调度：一次调度的单位是"算多少 token"，不是"跑哪些请求到底"，所以新请求中途插入完全可以。

3. Token Budget 是核心

V1 调度器的关键参数：max_num_batched_tokens。默认由 EngineArgs.get_batch_defaults() 按使用场景与硬件选择；不要脱离启动日志或最终 SchedulerConfig 假定为 8192。

每一步，所有请求**加起来**最多算这么多 token：

running 请求 A (decode): 1 token
running 请求 B (decode): 1 token
running 请求 C (prefill, prompt=2048): 2048 token
waiting 请求 D (prefill, prompt=512): 512 token

2562 token < 8192 ✓

如果超了 budget：

- 长 prefill 会被 **chunk**（拆成多步跑，详见 05-chunked-prefill.md）
- 新请求暂时不让进

这样设计的好处：**每一步的工作量上限可预测**，单 step 延迟稳定。

4. Static vs Continuous 数学直觉

设有 N 个请求，长度服从某分布。Static batching：

```
total_time = N × max(L_i) # 所有请求等最长的那个
```

Continuous batching（忽略调度开销）：

```
total_time = sum(L_i) / batch_capacity # 紧凑利用
```

当长度方差大（实际场景几乎总是），continuous 的加速比可以达到 $5-24\times$ 。

5. 调度优先级与公平性

Continuous batching 不是"先来后到"，vLLM 支持多种调度策略：

策略	行为	适用场景
FCFS（默认）	先到先服务	通用、公平
Priority	高优先级请求优先调度，可抢占低优先级	多租户、SLO 服务

实际看 `vllm/v1/core/sched/scheduler.py` 里的 `_schedule_running` 和 `_schedule_waiting` 函数。

6. 完成检测：怎么知道一个请求该退出？

在每一步的 forward 后，Scheduler 检查每个请求：

1. 采样到了 **EOS token** → FINISHED
2. 达到 **max_tokens** → FINISHED
3. **stop_token / stop_string 匹配** → FINISHED

完成的请求：

- 从 running 队列移除
- 它的所有 KV block `ref_cnt -= 1`
- 通知 API server 流式返回（带 `finish_reason`）

代码：`vllm/v1/engine/llm_engine.py` 的输出处理 + `Scheduler.update_from_output()`。

7. 抢占（Preemption）：当 KV 不够时

continuous batching 的隐藏假设是"KV cache 总是够用"。但请求数多 + 序列长时会爆。

vLLM 的策略：

7.1 Recompute（默认，推荐）

- 直接 free 低优先级请求的所有 block
- 把它的 status 设回 WAITING
- 它的已生成 token 保留在 CPU 端
- 重新调度时**重新 prefill**（KV 重新算）

代价：算力浪费 好处：实现简单、不占额外显存

7.2 KV offload / connector（不是 V1 preemption mode）

- 把 block 从 GPU 拷到 CPU 内存
- 恢复时拷回来

代价：PCIe 拷贝慢、占 CPU 内存 好处：算力不浪费

当前 V1 preemption 采用 recompute 路径，因为：

1. PCIe 反而经常比重算还慢
2. 代码更简单
3. 配合 prefix caching，重算时前面的部分还能命中 cache

代码：Scheduler._preempt()。

8. 代码定位

行为	文件：函数
主调度入口	vllm/v1/core/sched/scheduler.py : Scheduler.schedule()
运行中请求继续	Scheduler._schedule_running()
等待中请求加入	Scheduler._schedule_waiting()
抢占	Scheduler._preempt() 或 _preempt_lowest_priority_request()
输出处理	Scheduler.update_from_output()
输入 batch 增量更新	vllm/v1/worker/gpu_input_batch.py

9. 工程自检问答

Q: continuous batching 一定比 static 快吗？ A: 不一定。如果所有请求长度高度一致（比如 benchmark 的固定长度），两者性能接近。但生产场景长度方差大，continuous 必赢。

Q: 调度本身不是有开销吗？为什么不阻塞 GPU？ A: ①调度是纯 CPU 操作，跟 GPU 前向 overlap（V1 的 AsyncScheduler 显式做这件事）。②InputBatch 增量更新，不重建。③单步调度通常 < 1ms，GPU forward 几十 ms，比例可忽略。

Q: 怎么平衡 throughput 和 latency？ A: 关键旋钮是 max_num_batched_tokens：

- 大 → 大 batch，throughput 高，但 step 时长长，latency 抖动大
- 小 → 反之 生产场景一般 4096-8192，配合 chunked prefill 限制单步 prefill 量。

Q: TGI 也声称支持 continuous batching，差别在哪？ A: TGI 是 sequence-level（请求级），仍然按"prefill 阶段→decode 阶段"。vLLM 是 token-level，prefill 和 decode 混跑，且 chunked prefill 默认开启，更细。

小结

- Continuous batching = **iteration-level 动态批处理**：每生成 1 个 token 就重新组 batch，完成的请求立刻退出、等待的立刻进入，GPU 几乎不空转。
- 能跑起来的三个前提：① PagedAttention 让 KV 加入/退出不搬数据；② InputBatch 持久化只更新 diff；③ token-level 调度让"算多少 token"成为唯一调度单位。
- **max_num_batched_tokens 是核心旋钮**：大值通常提高吞吐上限但可能增加单步延迟，小值通常反之；初值应读取最终配置，再用业务负载测量。
- 当前 V1 抢占是 **recompute**（释放 KV、清零计算进度、重新排队）；CPU/远端 KV offload 与 connector 有独立配置和一致性约束，不等同于旧式 swap 抢占。
- vLLM 的 token-level 与 TGI 的 sequence-level 是本质差异——后者仍按"prefill 阶段 → decode 阶段"分组。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. No / Static / Continuous 三种 batching 的 gantt + GPU 空闲位置。

No batching（每请求独立 forward）：

卡: [A 全程][闲][闲][B 全程][闲]...

空闲：请求间切换 + 单请求 decode 时 GPU 算力闲（每步只算 1 token）

Static batching（一批一起进出）：

卡: [批1: A短B中C长D中一起跑][批2: 等批1全做完才进]

空闲：A 早做完后槽位被 C 锁住，等到 C 结束才能换下一批
(典型："长尾尾"现象，GPU 利用率 < 30%)

Continuous batching（每 step 重组）：

卡: [step1: A B C D][step2: A 退出 + E 加入: B C D E][step3: B 退出 + F 加入]...

空闲：几乎没有；瓶颈变成 KV 容量与算力
(GPU 利用率 80%+)

→ **核心区别**：static 的批边界是固定的，continuous 的边界是 token 级。

2. max_num_batched_tokens=8192，16 个 decode + 1 个 prompt=12000 的 prefill，怎么排？

1. 先给 16 个 decode 各 1 token → 占 budget 16
2. 剩余 budget = $8192 - 16 = 8176$
3. 给那个 prefill 请求切第一个 **chunk = 8176 token** (不是 12000)
4. 一步内 forward 总 token = $16 + 8176 = 8192$, 正好打满

接下来的 step:

- step 2: 16 个 decode (已经又生成 1 token) + prefill 剩余 $12000 - 8176 = 3824$ token
- 16 decode + 3824 prefill chunk = 3840 token, 仍在 8192 之内 → 这一步把 prefill 跑完
- prefill 完了立刻 decode 第一个新 token

两个 step 跑完 12000 prefill + 32 个 decode token, 每步时长接近一致 (约 8000 token forward), TPOT 平稳。如果不切, 那 12000 token 一次 forward 会让其他 16 个用户等几百 ms。

3. token budget 在哪几行被扣?

源码: vllm/v1/core/sched/scheduler.py (行号会随版本变)

```
def schedule(self):
    token_budget = self.scheduler_config.max_num_batched_tokens

    # ① 先调度 running (已在 batch 内的请求, 多数 decode 1 token)
    self._schedule_running(token_budget)
    # 内部: for req in self.running:
    #     num_tokens = min(req.remaining_tokens, token_budget)
    #     token_budget -= num_tokens          ← 扣减 #1
    #     num_scheduled_tokens[req.id] = num_tokens

    # ② 再调度 waiting (新请求 / preempted)
    self._schedule_waiting(token_budget)
    # 内部: while waiting and token_budget > 0:
    #     req = waiting.pop_request()
    #     num_blocks = ...
    #     new_blocks = kv_cache_manager.allocate_slots(req, ...)
    #     if new_blocks is None:
    #         # KV 不够, 触发 preempt (见下面 _preempt_request)
    #     num_tokens = min(req.num_prompt_tokens, token_budget)
    #     token_budget -= num_tokens          ← 扣减 #2

    return SchedulerOutput(num_scheduled_tokens=...)
```

定位方法: grep token_budget -= 或 num_scheduled_tokens。两个扣减点分别对应 decode-first 和 prefill 部分。

4. 当前为何使用 recompute; 设计 KV offload 时还要权衡什么?

Recompute 快的原因:

1. **PCIe 太慢**: H100 PCIe Gen5 × 16 单向 ~64 GB/s, 70B 模型一个长上下文请求 KV 可达 GB 级, swap 一次几十到几百 ms
2. **GPU 算力富余**: 现代 H100/B200 算力 dense, prefill 100 ms 内能干完几千 token
3. **Prefix cache 救场**: 被踢请求重新调度时, 前缀部分大概率命中 cache, 实际重算只是 cache miss 段——往往很短

4. 代码实现简单：recompute 路径就是"释放 block + 重新入 waiting"，无需管 host buffer / 拷贝调度

独立 KV offload 可能有价值的场景（不是当前 preemption mode）：

- **KV 极大 + prefix cache 命中率低**：重算成本高，值得测量 offload 的传输与一致性成本
- **Prefill 算力极度紧张**：外部 KV 可能减少重算，但会引入容量、回压和故障恢复问题
- **CPU/GPU 互连更快的硬件**：传输更有竞争力，但仍需实测，不能把理论带宽当端到端结果

当前 V1 没有 --preemption-mode swap 这个 CLI。需要 CPU/远端 KV 时，应评估 --kv-offload-backend 或 --kv-transfer-config 对应的专用路径，并按所选 backend 文档验证能力。

5. 长度方差极小的 workload（1024 in / 256 out），continuous 赢 static 多少？

预期：continuous 仍赢，但优势缩小到约 1.5-2× 而不是 24×。原因：

- Static batching 的痛点是"长请求拖死短请求" → 当所有请求长度一样时，这个痛点消失
- Static 仍有的劣势：
- 每批一起进出 → 批与批之间有 prefill / setup gap
- decode 1 token 是 memory-bound，static batch 大小固定就是固定，不会动态填充
- Continuous 仍能赢的地方：
- 新请求随到随进，**没有批切换的间隙**
- prefill / decode 混跑，prefill 来时不卡死 decode
- GPU 利用率仍更高（80%+ vs 70%）

估算：1024 in + 256 out workload，continuous ~1.5-1.8× static。论文里 24× 的来源是 ShareGPT 真实 workload，长度方差极大（10 token 到 4096 token），static 时长被最长请求决定。

反直觉：合成长度均匀的 benchmark 其实对 vLLM **不利**——掩盖了它最擅长解决的问题（长度方差）。生产 workload 长度方差越大，vLLM 越能拉开差距。

下一步

- 下一节：[03-kv-cache-management.md（深入 BlockPool / LRU / 启动 profile run / preemption 实现细节）。
- 跟着读：05-chunked-prefill.md（continuous batching 的延伸优化，解决长 prompt 卡死 decode 的问题）。
- 想看源码：vllm/v1/core/sched/scheduler.py（schedule 主循环）、vllm/v1/core/sched/async_scheduler.py（async overlap）、vllm/v1/worker/gpu_input_batch.py（持久 InputBatch）。
- 想动手：07-hands-on/03-mini-experiments.md（调 max_num_batched_tokens 观察 TTFT/吞吐曲线）。
- 想从生产视角理解：08-production-deployment/04-autoscaling-and-capacity.md（token budget 与 HPA、并发上限的关系）。
- 想看背后的性能模型：04-optimizations/05-roofline-and-arithmetic-intensity.md（用存算比定量证明"为什么加 batch 几乎免费地拉吞吐"）。

03. KV Cache 管理深入

谁该读这一篇？ 想从"知道有 BlockPool"升级到"能 trace 出一个 block 的完整生命周期"的读者；需要讲清"vLLM 启动时怎么决定 num_blocks / 怎么 preempt / hash 怎么算"的工程师。

前置阅读： 01-paged-attention.md（理念层）、02-continuous-batching.md（schedule 与 preempt 的上层逻辑）。

耗时： 约 20 分钟。

学完能：

- 1. 复述启动阶段 profile run 算 num_blocks 的 5 步公式，并解释 --gpu-memory-utilization 默认 0.92 的含义与风险。
- 2. 写出 KVCacheBlock / FreeKVCacheBlockQueue / cached_block_hash_to_block 三个数据结构的不变式。
- 3. 画出一个请求"WAITING → RUNNING → FINISHED"过程中 KV block 的分配 / 复用 / 释放序列。
- 4. 讲清 chain hash 为什么必须 chain、prefix caching 的命中是"按顺序"的。

当前源码复核 (b23bd73)： CacheConfig.gpu_memory_utilization 当前默认 0.92；prefix hash 默认 sha256。ref_cnt==0 表示普通 block 在 free queue，但 null block 是明确例外；hash 不再能简化成 Python 内置 hash()，命中正确性也不能靠“二次 token 比对”兜底。

PagedAttention 是"理念"，本节是"工程实现"。看完这节，你才能讲清"vLLM 启动时怎么决定要分多少 block"。

1. 启动阶段：profile run 决定 num_blocks

vLLM 启动时不会用 torch.cuda.memory_allocated() 估算 KV 可用空间，而是做一次**真实的 dummy forward** 测峰值显存。流程：

- 1. 加载模型权重 → 占去 W GB
- 2. 构造一个 max batch、max seq_len 的假输入
- 3. 跑一次 forward（关 grad）→ 记录峰值显存 P GB
- 4. KV 可用空间 = (gpu_memory_utilization × total_mem) - P
- 5. num_blocks = floor(KV_avail / (block_size × per_token_kv_bytes × num_layers))

代码：vllm/v1/worker/gpu_worker.py : determine_available_memory()

参数：

- --gpu-memory-utilization：默认 0.92（这是当前实例的 executor 显存预算比例，不是 nvidia-smi GPU-Util）
- --kv-cache-dtype：默认与模型 dtype 一致；可设 fp8 减半

启动日志会打印类似 # GPU blocks: 12345，这就是 num_blocks。

2. BlockPool：物理 block 的中央仓库

```
class BlockPool:
    # 所有物理 block 的元数据
    blocks: list[KVCacheBlock]    # 长度 = num_blocks

    # 空闲列表（双向链表，便于 LRU 复用）
    free_block_queue: FreeKVCacheBlockQueue

    # Hash → block 的索引（用于 prefix caching 命中）
    cached_block_hash_to_block: dict[BlockHash, list[KVCacheBlock]]
```

核心方法：

- get_new_blocks(num_blocks)：从 free queue 头部取
- free_blocks(blocks)：ref_cnt -= 1，归零则放回 free queue 尾部
- touch(blocks)：把命中的 block 从 free queue 拿出（被复用了）

注意 free queue 不是 LIFO 是 LRU：最近 free 的放尾巴，最早 free 的先被复用。这样**最旧的 block 才会被覆盖**，prefix cache 命中率最大。

3. KVCacheBlock 的内部状态

```
@dataclass
class KVCacheBlock:
    block_id: int        # 物理 block 索引 (0 .. num_blocks-1)
    ref_cnt: int = 0     # 引用数
    block_hash: Optional[...] # 内容哈希（用于 prefix caching）
    prev_free_block: ...   # free queue 前驱
    next_free_block: ...   # free queue 后继
```

关键不变式：

- ref_cnt == 0 ⇔ block 在 free queue 里
- ref_cnt > 0 ⇔ block 被至少一个请求引用
- block_hash != None ⇔ block 已写满且参与 prefix caching

4. 一个请求的 KV 分配生命周期

以一个 prompt=50 token、生成 100 token 的请求为例（block_size=16）：

T=0: 入队 (WAITING)

- block_table = []
- num_computed_tokens = 0

T=1: schedule 时调用 KVCacheManager.allocate_slots(req, 50)

- 需要 $\text{ceil}(50/16) = 4$ 个 block
- 查 prefix cache: 假设没有命中
- 从 BlockPool 取 4 个新 block: [42, 17, 99, 3]
- $\text{ref_cnt}[42,17,99,3] += 1$
- $\text{req.block_table} = [42, 17, 99, 3]$
- status $\rightarrow$ RUNNING

T=2: prefill 完成，生成第 51 个 token

- 第 51 token 落在 block 3 的位置 $50\%16=2$ （还有空间）
- 不分新 block
- block 99 现在"满了"（16 token），计算它的 hash，挂到 hash table（这样未来其他请求的 prefix 包含同样内容时能命中）
- block 3 还没满，不计算 hash

T=3..N: decode 每步

- 每步加 1 token
- 落在当前末尾 block 内时不分配
- 越过 block 边界时，allocate_slots(req, 1) 取 1 个新 block

T=N+1: 生成 EOS

- status $\rightarrow$ FINISHED
- KVCacheManager.free(req)
- 对每个 block， $\text{ref_cnt} -= 1$
- 归零的 block 放回 free queue 尾部
- 如果开启 prefix caching，hash 保留（block 还能被命中复用）

5. Prefix Caching 的 hash 机制

这是理解 KV 管理时最容易漏掉的一点。

5.1 Hash 的输入

对每个 block，hash 包含：

- block 内的所有 token_ids
- 前一个 block 的 hash（chain，保证位置敏感）
- 额外的"上下文"：LoRA 适配器、多模态 placeholder 等

代码：vllm/v1/core/kv_cache_utils.py : hash_request_tokens / BlockHash

5.2 为什么要 chain 前一个 hash?

因为 token "你好" 在不同前缀下产生的 KV 不同（取决于前面）。chain hash 保证：相同前缀且相同当前 block 才命中。

伪代码：

```
def block_hash(token_ids, prev_hash, extra_keys):  
    return hash((prev_hash, tuple(token_ids), extra_keys))
```

5.3 命中流程

```
def allocate_slots(req):  
    # 1. 算出 req 全部 prompt 的每个 block 的 hash  
    hashes = compute_block_hashes(req.prompt_token_ids)  
  
    # 2. 查 BlockPool.cached_block_hash_to_block  
    for h in hashes:  
        if h in pool.cached_hash:  
            existing_block = pool.cached_hash[h]  
            # 命中! 直接挪用  
            existing_block.ref_cnt += 1  
            if existing_block.ref_cnt == 1:  
                pool.touch(existing_block) # 从 free queue 拿出  
                req.block_table.append(existing_block.block_id)  
        else:  
            break # 第一个 miss 之后就不再命中 (前缀性质)  
  
    # 3. 剩下的 block 用 get_new_blocks 分配
```

重点：命中是按顺序的，第一个 miss 后面全部 miss。因为前缀 hash 是链式的。

5.4 多请求并发的复用

```
T=0: req_A 完成 prefill, blocks=[42, 17, 99], hash 都注册了  
T=1: req_A FINISHED, 3 个 block ref_cnt → 0, 进 free queue  
    但 hash 仍在 cached_hash_to_block 中  
T=2: req_B 用同样 system prompt 进入  
    - 计算 hash 命中 42、17、99  
    - 把它们从 free queue 取出, ref_cnt = 1  
    - req_B 跳过这 3 个 block 的 prefill 计算! 直接从第 4 个 block 开始
```

这就是为什么 chatbot 重复用同一个 system prompt 能极大降 TTFT。

6. 怎么决定 num_scheduled_tokens?

KVCacheManager 的核心算法（简化）：


```
def allocate_slots(req, num_new_tokens):
    # 当前已分配的 block 数
    cur_blocks = len(req.block_table)
    # 需要的总 block 数
    new_total_tokens = req.num_computed_tokens + num_new_tokens
    need_blocks = ceil(new_total_tokens / block_size)
    # 需要再分多少
    delta = need_blocks - cur_blocks

    if delta == 0:
        return True # 现有 block 够用

    # 减去 prefix cache 命中节省的
    delta -= hits_for_uncomputed_part(req)

    if pool.free_count() < delta:
        return False # 不够, 需要 preempt 或拒绝

    new_blocks = pool.get_new_blocks(delta)
    req.block_table.extend(b.block_id for b in new_blocks)
    return True
```

7. 抢占（preemption）的实现细节

```
def _preempt(self, req):
    # 1. 把 req 的所有 block ref_cnt -= 1
    self.kv_cache_manager.free(req)
    # 2. 改状态
    req.status = RequestStatus.WAITING_PREEMPTED
    req.num_computed_tokens = 0 # recompute 模式
    # 3. 从 running 队列移到 waiting 队列头部（保证它优先重启）
    self.running.remove(req)
    self.waiting.appendleft(req)
```

选谁来 preempt? 默认 FCFS 倒序（最晚到的先牺牲）；priority 模式下选优先级最低的。

当前 V1 preemption 直接使用 recompute；CPU/远端 KV offload 由独立 backend/connector 负责，不是这里的分支。

8. KV Cache 的 dtype

- kv-cache-dtype auto：跟模型 dtype（FP16/BF16）
- kv-cache-dtype fp8（或 fp8_e4m3 / fp8_e5m2）：KV 存 FP8，每 token 占用减半，等价于 **num_blocks × 2**

FP8 量化 KV 在 attention 算之前 dequantize，引入精度损失。实测 perplexity 损失 < 1%。

9. 多 KV 类型：Mamba / Linear Attention

vLLM 的 KVCacheManager 已经被扩展为多类型并存：

- 普通 Transformer 层：上面讲的 paged KV
- Mamba SSM 层：固定大小的 state（不是 KV）
- Linear / sparse / sliding window：不同 block 结构

代码：vllm/v1/core/single_type_kv_cache_manager.py 和 kv_cache_coordinator.py 协调多种 manager。

初读可以先略过，但你应该知道 vLLM 已经不只是"Transformer paged KV"了。

10. 工程自检问答

Q: vLLM 启动慢，为什么？ A: profile run 阶段会跑 dummy forward + 可能 capture CUDA Graph 多个 batch_size + torch.compile，加起来几十秒到几分钟。可以用 --enforce-eager 关 CUDA Graph 加速启动（但跑慢）。

Q: gpu_memory_utilization 设 0.95 行不行？ A: 风险大。profile run 测的是单一 batch 的峰值，runtime 实际可能有 activation 临时高峰。0.9 给 10% 缓冲是经验值。

Q: 一个 block 内 token 数量比 block_size 小时，attention 怎么处理？ A: 通过 seq_lens / block_table 中的有效长度告诉 kernel "这个请求总共 N 个 token，超过 N 的位置 mask 掉"。kernel 内部按 block 加载，但 softmax 时排除无效位置。

Q: prefix caching 会不会出错？比如 hash 冲突？ A: 当前默认用 sha256 序列化后哈希；也支持 sha256_cbor、xxhash、xxhash_cbor。缓存索引按 hash 查找，不能宣称总有 token 序列二次比对；非加密 hash 的碰撞风险在多租户环境还可能成为隔离问题，所以必须按风险选择算法。

小结

- 启动时通过 **profile run** 实测峰值显存，再按 $(\text{gpu_mem_util} \times \text{total}) - \text{peak}$ 算 KV 可用空间，得到 num_blocks——这是 --gpu-memory-utilization 留 0.9 给 activation 余量的原因。
- **BlockPool** 是物理 block 的中央仓库：blocks 元数据数组 + LRU **FreeKVCacheBlockQueue** + **cached_block_hash_to_block** 索引。对普通 block，ref_cnt==0 表示在 free queue、ref_cnt>0 表示被引用；null block 的引用计数不按该规则维护。缓存 hash 与物理写满/可复用状态相关，但 hybrid KV 与组 ID 使真实键比三元伪代码更丰富。
- 一个请求的 KV 一生：WAITING → 调度时 allocate_slots → RUNNING → 每越 block 边界再 allocate 1 → FINISHED 时 free() 把 ref_cnt-=1，归零的 block 进 free queue 尾（**但 hash 保留供未来命中**）。
- Prefix caching 用**链式 hash**（hash((prev_hash, tokens, extra_keys))）保证位置敏感；命中按顺序，第一个 miss 之后全部 miss。
- Preemption 默认走 **recompute**：释放所有 block + 状态回到 WAITING，下次 schedule 时重新 prefill（重算时还能命中 prefix cache）。FP8 KV 是免费翻倍 num_blocks 的常用手段。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. prompt=50, 生成 100 token, block_size=16, 画 KV block 一生。

- prompt=50 token → 需要 $\lceil 50/16 \rceil = 4$ 个 **block**（最后一个只用 2 个 slot）

- 生成 100 token → 累计 150 token，需要 $\lceil 150/16 \rceil = 10$ 个 **block**
- 期间需要 alloc 6 个新 block（block 4..9）

T=0 (prefill 进 batch): alloc block 0,1,2,3 → ref_cnt 全 = 1
T=1..N (decode 累积 token):
 当 token 数到达 64 时（block 3 已经满）→ alloc block 4 → ref_cnt[4]=1
 当 token 数到达 80 → alloc block 5
 ...
 当 token 数到达 144 → alloc block 9
T=END (请求 finish): block 0..9 ref_cnt -= 1 → 全归 0
 → 全部回 free_queue 尾（如果有 prefix cache，被 hash 命中的 block 仍可被未来请求 ref_cnt++ 复用）

如果有共享 **prefix**（比如 system prompt 占 block 0-2，跟另一个请求一样）：

- 启动时：block 0-2 是 hash 命中 → ref_cnt++ 共享，**只新 alloc block 3 + 后续**
- 完成时：block 0-2 ref_cnt-- 回到原值（可能 > 0，仍在共享）；block 3+ ref_cnt → 0 回 free queue

2. hash_block_tokens(tokens, prev_hash, extra_keys) 伪代码 + 不 chain 的 bug。

```
NONE_HASH = bytes(32) # 哨兵

def hash_block_tokens(hash_function, prev_hash, tokens, extra_keys=None):
    if prev_hash is None:
        prev_hash = NONE_HASH
    # 关键: 把 prev_hash 也进 hash → Merkle 链
    return hash_function((prev_hash, tuple(tokens), extra_keys))
```

不 chain prev_hash 的 bug：

假设两个请求：

- A: tokens [1,2,3,4,...,16, 99,100,...] (block 0 = [1..16], block 1 = [99,100,...])
- B: tokens [5,6,7,8,...,20, 99,100,...] (block 0 = [5..20], block 1 = [99,100,...])

不 chain 时：A 的 block 1 hash = hash([99,100,...])；B 的 block 1 hash = 同样 = hash([99,100,...])。

结果：B 错误命中 A 的 block 1 KV，但 attention 历史完全不同——B 的 attention 应该 attend 到 [5..20]，实际拿到的是 A 的 [1..16] 的 K/V。**输出乱码。**

chain prev_hash 后：A 的 block 1 hash = hash(hash([1..16]), [99,100,...])，B 的 block 1 hash = hash(hash([5..20]), [99,100,...])，**完全不同**，不会误命中。

→ Merkle chain 让"前缀完全相同"才命中，是 prefix caching 正确性的根基。

3. FreeKVCacheBlockQueue 为什么 LRU 而不是 LIFO？

源码：vllm/v1/core/block_pool.py 的 FreeKVCacheBlockQueue（双向链表）。

- 入队（block ref_cnt 归零时）：append 到链表**尾部**
- 出队（alloc 新 block 时）：从链表**头部**取

为什么不 LIFO（从尾取）？

- LIFO：最近释放的 block 先被覆盖 → **prefix cache 立刻丢失**。比如刚结束的 chatbot 对话，10 秒后用户接着问 → 同 prefix 但 KV 已经被覆盖
- LRU：最早释放的先被覆盖 → 给 prefix cache **更长存活时间**

实测影响：LRU 比 LIFO 在多轮对话 / RAG 等场景下 prefix cache 命中率高 2-5×，对应 TTFT 收益巨大。

代价：LRU 实现复杂一些（双向链表 + dict 维护"block → list_node"映射，O(1) 删除）。但相比命中率收益完全划得来。

4. 当前 V1 为何用 recompute；什么时候值得单独评估 KV offload？

Recompute 偏好的 3 条理由：

1. **PCIe 太慢：**H100 PCIe Gen5 ×16 单向 ~64 GB/s。Llama-70B 长 2500 token KV ≈ 2.5 GB → swap out 40 ms + swap in 40 ms = 80 ms 通信。而 recompute 一个 2500-token 的 prefill 在 H100 上也就 100-150 ms，相差不大
2. **Prefix cache 救场：**被踢请求重新进 waiting 后，多数情况能命中之前 cached 的 block（同 hash），实际只重算 cache miss 段——通常远小于全部 prefill。综合 recompute 比 swap 快
3. **实现简单：**recompute = "free 所有 block + 回到 WAITING 状态"，无需管理 host buffer 池、PCIe DMA 调度、双端同步

独立 KV offload 值得评估的场景（不是 preemption mode）：

- **Grace Hopper / GH200 等快互连：**理论传输上限更高，但仍须测序列化、排队和双向迁移
- **超长上下文 + 极低 prefix cache 命中：**每次 prompt 都不一样，recompute 等于全部重 prefill 没意义
- **prefill 算力严重 contended：**大并发同时争 GPU，外部 KV 可能减少重算，但会引入回压和一致性状态

当前 V1 抢占没有 swap CLI：Scheduler._preempt_request() 释放 request blocks、清零 num_computed_tokens 并放回 waiting。KV offload/transfer 是独立子系统，不能混写成抢占模式。

5. H100 80GB / Llama-3-8B / BF16，切到 fp8 KV num_blocks 多多少少？

BF16 基线：

- Llama-3-8B：32 层、num_kv_heads=8（GQA）、head_dim=128
- 单 token KV = $2 \times 8 \times 128 \times 32 \times 2 = 131072$ 字节 = **128 KB**
- 单 block = 16 token × 128 KB = **2 MB**
- 模型权重 8B × 2 = 16 GB
- 可用 KV 显存 ≈ 80 - 16 - 6（CUDA / 激活预算）= **58 GB**
- num_blocks = 58 GB / 2 MB = **29,696 个 block** ≈ 475,136 token

FP8 KV：

- 单 token KV 字节减半 = 64 KB
- 单 block = 1 MB
- 可用 KV 显存还是 58 GB（fp8 不影响模型权重）
- num_blocks = 58 GB / 1 MB = **59,392 个 block** ≈ 950,272 token

→ **block 数翻倍**，理论并发 / 并发上下文长度也接近翻倍。

注意：

- fp8 KV 是有损精度（per-tensor 或 per-channel scale + fp8 encoding）。多数模型 perplexity 影响 < 1%，但极敏感场景（math reasoning）要 ablation 验证
- 启用：--kv-cache-dtype fp8（前提硬件支持 fp8，H100/H200/B200 OK，A100 不行）
- 实际吞吐增益往往低于理论 2×，因为算力 / 调度也会成为新瓶颈

下一步

- 下一节：04-prefix-caching.md（把本节 § 5 hash 机制展开到完整的 prefix caching 专题：extra_keys / LoRA / 多模态 / 跨进程共享）。
- 跟着读：05-chunked-prefill.md（与 KV 管理强相关的另一项 V1 默认行为，剩余 prefill 是怎么切的）。
- 想看源码：vllm/v1/core/block_pool.py（BlockPool）、vllm/v1/core/kv_cache_manager.py（allocate_slots / free）、vllm/v1/core/kv_cache_utils.py（hash 计算）、vllm/v1/worker/gpu_worker.py（determine_available_memory）。
- 想动手：07-hands-on/03-mini-experiments.md（用 --gpu-memory-utilization / --kv-cache-dtype fp8 对比 num_blocks 与并发上限）。
- 想从生产视角理解：08-production-deployment/05-slo-and-observability.md（vllm:kv_cache_usage_perc，以及由 prefix cache hit/query counters 计算命中率）。

04. Prefix Caching 专题

谁该读这一篇？ 想搞清"为什么同样的 system prompt 第二次发请求会快很多"、"怎么调命中率"、"分布式 / 多模态 / LoRA 场景下 hash 怎么算"的读者。

前置阅读： 03-kv-cache-management.md (block pool + ref_cnt)；最好先扫一眼 01-paged-attention.md。

耗时： 约 15 分钟。

学完能：

- 1. 说清 vLLM block hash 是怎么算出来的 (hash 函数、parent chain、extra_keys)。
- 2. 解释为什么 LoRA / 多模态 / prompt_embeds 不能直接复用纯文本的 hash。
- 3. 调出一个低命中率场景的 root cause (block_size 不对齐 / cache 容量不够 / hash 函数选错)。
- 4. 知道分布式 KV cache (CPU offload、远端 LMCache、P/D KV transfer) 在哪里接入。

当前源码复核 (b23bd73)： CacheConfig.prefix_caching_hash_algo 当前默认 sha256，可选 sha256_cbor、xxhash、xxhash_cbor。普通受支持模型默认开启 prefix caching，但 hybrid model 目前保持 opt-in；“V1 一律默认开启”不准确。

1. 解决什么问题

LLM 应用中存在大量"前缀重复"：

场景	重复的部分	占比
Chatbot	system prompt + 历史对话	60-90% 输入
Few-shot	few-shot 示例	50-95%
RAG	多个查询用相同的 retrieval 结果模板	30-70%
Tool use	工具描述 (function calling)	40-80%
Batch eval	同 prompt 不同输入	几乎全部

如果每个请求都重新算这些前缀的 KV，是巨大的浪费。

2. 直观对比：开与不开

假设 system prompt 长 1000 tokens，用户每次问 50 tokens，生成 100 tokens。

关闭 prefix caching：

- 每次 prefill = 1050 token
- 总 token = 1050 (prefill) + 100 (decode) = 1150
- 实际算力 $\approx$ prefill dominates

开启 **prefix caching**（第二次开始）：

- 每次 prefill = 50 token（前 1000 命中）
- TTFT 直接降 95%
- 吞吐提升对应倍数

3. 启用方式

V1 对支持 prefix caching 的非 hybrid 模型默认开启；hybrid model 当前保持 opt-in，RISC-V CPU 等不支持路径还会关闭。命令：

```
vllm serve <model> [--no-enable-prefix-caching] # 关闭
```

参数：--block-size（影响命中粒度）、--num-gpu-blocks-override。

4. Hash 算法详解（V1）

源码锚点：[vllm/v1/core/kv_cache_utils.py](#) • [hash_block_tokens](#)

源码：vllm/v1/core/kv_cache_utils.py 的 hash_block_tokens()。

4.1 Merkle 链式哈希

vLLM 的 block hash **不是孤立算一个 block**，而是把"上一个 block 的 hash"作为输入，形成一条链：



代码对应：

源码锚点：[vllm/v1/core/kv_cache_utils.py](#) • [hash_block_tokens](#)

```
# vllm/v1/core/kv_cache_utils.py
def hash_block_tokens(hash_function, parent_block_hash,
                      curr_block_token_ids, extra_keys=None):
    if not parent_block_hash:
        parent_block_hash = NONE_HASH
    curr_block_token_ids_tuple = tuple(curr_block_token_ids)
    return BlockHash(
        hash_function((parent_block_hash, curr_block_token_ids_tuple, extra_keys))
    )
```

为什么要链式？ 防止"前缀不同、但碰巧某个中间 block 内容相同"的错误命中。H₀ 蕴含了 tokens[0..47] 的全部信息——只有当**整段历史都相同**时 hash 才会撞上。本质上是 Merkle tree 的退化形式（一条链）。

4.2 Hash 函数选择

V1 提供四个选项，序列化格式和 hash 强度是两个独立维度：

函数	序列化	安全属性	建议
sha256	Pickle	加密 hash	当前默认
sha256_cbor	canonical CBOR	加密 hash、跨语言可复现	跨语言共享键
xxhash	Pickle	非加密 hash	明确接受碰撞/多租户风险时才用
xxhash_cbor	canonical CBOR	非加密 hash、跨语言可复现	同上，且需要可复现序列化

通过 `cache_config.prefix_caching_hash_algo` 切换。多租户或跨进程共享 cache 时应优先加密 hash，并同时设计租户命名空间；单靠 hash 算法不能完成租户隔离。

序列化注意： `init_none_hash()` 初始化 parent sentinel。是否跨语言可复现取决于 Pickle/CBOR 选择；是否抗对抗碰撞取决于 SHA-256/xxHash 选择，不要把两者混为一谈。

4.3 extra_keys：把"上下文"塞进 hash

光有 token id 不够。两个不同 LoRA 适配器跑同一段 prompt，KV 输出**完全不同**，但 token 一样——直接命中会污染。extra_keys 把这些"决定 KV 内容的额外信息"也拌进 hash：

来源	函数	包含什么
LoRA	_gen_lora_extra_hash_keys	LoRA 适配器名/路径
多模态	_gen_mm_extra_hash_keys	每个 multi-modal feature 的 identifier + token 偏移
Prompt embeds	_gen_prompt_embeds_extra_hash_keys	embed tensor 的 sha256(tensor_data)
Cache salt	request 层	用户传入的隔离标记（防跨用户命中）

入口：generate_block_hash_extra_keys() 按 block 区间合并这些 key。

实战含义：

- 多模态：图像内容变了 hash 就变（不会错误命中）。但**完全相同的图像**仍能命中——这是优势。
- LoRA：同 token + 同 LoRA 才命中。多 LoRA 服务下命中率自然分摊。
- 端到端加密场景：传 cache_salt，每用户独立 cache。

5. 命中的几个边界

5.1 block 边界对齐

hash 是按 block 算的。如果 system prompt 是 1003 token（block_size=16），那么：

- block 0..61：完全相同 → 命中
- block 62：第 62 个 block 有 11 个 prompt token + 5 个 user 输入 → **不命中**（user 输入不同）

所以**前缀越能 block 对齐越好**。一种实现技巧是在 system prompt 末尾加 padding 或 separator 使长度成为 block_size 的倍数。
block_size 调大命中粒度变粗（节省 hash 算量但容易丢命中），调小相反。常见值 16/32。

5.2 sampling_params 不影响命中

hash 只包含 token_ids 和上下文，不包含 temperature / top_p。这是对的：sampling 只影响输出，不影响 KV 内容。

5.3 hybrid KV cache 的两套 block_size

V1 引入了 Mamba / 滑窗等混合 KV cache，scheduler 和 hash 用的 block_size 可能不同（resolve_kv_cache_block_sizes(), line 571）：

- scheduler_block_size：调度对齐的粒度（受 DCP/PCP 影响）
- hash_block_size：prefix caching 用的粒度（多组时取各组 GCD）

普通 Llama 类模型两者相等；混合架构（如 Jamba）下 hash_block_size 通常更小。

6. 与 chunkedprefill 的配合

prefix cache 命中后，剩余未命中的 token 还需要 prefill。如果剩余很长，会被 chunked prefill 进一步切。

逻辑顺序（在 KVCacheManager 里）：

1. 算请求 prompt 每个 block 的 hash（链式，O(num_blocks)）
2. 顺序匹配 cached_hash，找到第一个 miss 的位置 K
3. block_table[0..K-1] 用现有 block，ref_cnt++
4. 剩余 (prompt_len - K*block_size) 个 token 走 chunked prefill

详见 05-chunked-prefill.md。

7. 多卡 / 多机 prefix caching

7.1 TP 内部

每张 TP rank 各自维护自己的 KV cache 和 BlockPool，但：

- **hash 计算在 driver rank（rank 0）完成：**因为 hash 只依赖 token + extra_keys，不依赖 KV 数值
- **结果广播给所有 rank：**scheduler 把"哪些 block 命中、用哪些物理 block id"塞进 SchedulerOutput
- 所有 rank 拿到相同的 block table，独立从自己的 KV cache 取数据

这保证了"同一 hash 在所有 rank 同步命中或同步 miss"的不变量。

7.2 PP / EP

PP rank 各自管自己那几层的 KV，因此 prefix caching 也是各 PP rank 独立维护。但**调度入口仍是单一 scheduler**，所以 block table 仍统一。

EP（专家并行）只影响 MoE 层的 expert 选择，对 KV 不产生新维度——prefix caching 行为同 TP。

7.3 跨实例共享（高级）

vLLM 通过 KV connector 接口（vllm/distributed/kv_transfer/、vllm/v1/kv_offload/）支持把 cache 卸载/转移：

层	介质	时延	容量
L1	GPU HBM	微秒	数十 GB
L2	CPU DRAM	数十 μ s	TB
L3	NVMe / RDMA / Object Store	ms 级	无限（按成本）

实际项目：

- **LMCache：**开源 L2/L3 prefix cache 库，与 vLLM 通过 connector 集成
- **NIXL：**NVIDIA 的高性能 KV 传输库，P/D 分离常用（见 05-distributed/02-disaggregated.md）

跨实例共享时**必须用 sha256_cbor**（参见 4.2）。

8. 容量与逐出策略

prefix cache 不是无限的。当 BlockPool 用尽时：

- 现有的某个 ref_cnt=0 的 block 会被覆盖
- 从 free queue **头部**取（LRU，最久未用）
- 一旦被覆盖，这个 hash 在 cached_block_hash_to_block 表中删除

这意味着：**冷的 cached block 会被淘汰**，新请求要重算。

当前 Prometheus 暴露 vllm:prefix_cache_hits_total 与 vllm:prefix_cache_queries_total counter；命中率应在同一窗口用二者的 rate() 相除，注意 query 的单位是 token，不要引用不存在的预计算 hit-rate gauge。

9. 命中率低？troubleshooting checklist

症状	可能原因	排查命令
同 prompt 多次发，第二次还慢	第一次还没填完 cache，或 block_size 边界导致末尾 block 不命中	用 hit/query counters 计算窗口命中率；核对最终 block_size
改了 LoRA 后命中率清零	extra_keys 变了，hash 全不一样（ 这是对的 ）	这是 by design
TP 多卡命中不一致	hash 函数 race（用了 builtin hash()）	--prefix-caching-hash-algo sha256
多模态请求几乎不命中	图像内容每次不同 → mm extra_key 变 → hash 变	多模态本就难命中；只有"同图反复问"才命中
容量频繁逐出	num_gpu_blocks 太小、并发太大	调小 max_num_seqs 或加 GPU memory
跨重启后命中清零	KV 在 HBM，进程死了就没了	需要 L2/L3 cache（LMCache 等）

10. 工程自检问答

Q: 如果两个请求 prompt 完全相同，但 sampling 不同，会发生什么？ A: prefill KV 全部命中 → 跳过 prefill。decode 阶段各自独立采样（因为 sampling 有 RNG 状态）。两个请求生成可能完全不同的输出。

Q: prefix cache 在多卡 TP 下怎么工作？ A: 每张卡独立维护各自的 KV cache 和 hash 表，但 hash 算法一致，所以同一个 block hash 在所有 rank 上同步命中或同步 miss。Scheduler 在 driver rank 上算 hash，广播给 worker（见 § 7.1）。

Q: 为什么当前默认用 SHA-256，什么时候才考虑 xxHash？ A: hash 碰撞会导致错误复用，甚至形成多租户数据风险，因此当前默认 sha256。只有在可信、隔离的环境里量化出 hash CPU 成本，且接受非加密碰撞风险时，才考虑 xxhash；需要跨语言稳定键时再选择对应的 CBOR 变体。

Q: hash 链长了后还能 O(1) 查吗？ A: 能。cached_block_hash_to_block 是个普通 dict——拿到当前 block 的 hash 直接查。链式只影响"算 hash"的过程（顺序计算），不影响查询。

Q: 怎么测 prefix caching 的效果? A:

1. 跑 benchmarks/benchmark_prefix_caching.py
2. 自定义 workload: 同 system prompt + 不同 user query, 对比开关 prefix caching 的 TTFT / throughput
3. 看 Prometheus 的 vllm:prefix_cache_hits_total 和 vllm:prefix_cache_queries_total, 用相同窗口的 rate() 计算比值

Q: prefix cache 会不会占用太多 KV cache 让新请求挤不进去? A: 不会。cached 但 ref_cnt=0 的 block 仍在 free queue 里, 可以被新请求透明地复用 (覆写)。代价是失去 cache 内容。

小结

- vLLM prefix caching 的核心是**链式 block hash + extra_keys**: 前者保证前缀完整匹配才命中, 后者把 LoRA / 多模态 / embed 等"决定 KV 内容的非 token 因素"也纳入 hash。
- 默认用 xxhash_cbor (快)。跨进程/远端共享场景一定要切 sha256_cbor。
- 命中粒度由 block_size 决定。LRU 逐出在 ref_cnt=0 时透明进行。
- 分布式扩展通过 KV connector 接 L2/L3 (CPU/远端), 常见于 P/D 分离与 LMCache 集成。

自检

1. 写出 $H_i = ?$ 的表达式 (参考 § 4.1 图)。如果 tokens[16..31] 改一个 byte, H_i 会变吗?
2. 同 prompt 同 token、A 用 LoRA- α 、B 用 LoRA- β , 能 hit 同一个 block 吗? 为什么?
3. 想跨 vLLM 实例共享 prefix cache, 至少要改哪两项配置?
4. 由 hit/query counters 算出的 5 分钟命中率从 70% 掉到 5%。给出 3 个可能的 root cause 并说怎么验证。

参考答案

1. 链式 hash 通常写成 $H_i = \text{hash}(H_{i-1} \parallel \text{tokens}[32..47] \parallel \text{extra_keys})$, 其中 H_{i-1} 是前一个 block 的 hash, extra_keys 包含影响 KV 语义的模型、LoRA 等上下文。如果 tokens[16..31] 改动, H_{i-1} 已改变, 后续 H_i 也会改变, 即使第三个 block 的 token 本身没变。
2. 默认不能直接命中。LoRA 会改变线性层输出和后续 KV, cache key 必须把 adapter 身份纳入 extra_keys; A/B 使用不同 LoRA 时应隔离。只有明确证明 adapter 不影响该段隐状态, 才可以设计共享, 但不能凭 prompt 相同推断共享安全。
3. 至少要让所有实例使用相同的模型/tokenizer、block/hash 语义和固定 PYTHONHASHSEED, 并配置同一个外部 KV connector/store 与一致的 cache_prefix。还要确保路由把请求送到真正共享的 Store, 而不是每个 Pod 的本地 L1。
4. 可能原因包括: hash seed 或 namespace 改变; tokenizer/template/model revision 漂移; 路由从热实例切到冷实例或缓存被大量淘汰。验证时分别比较进程环境、cache_prefix/版本指纹、token IDs、命中长度分布和 eviction/store bytes, 不能只看一个 hit counter。

下一步

- 下一节 05-chunked-prefill.md: 理解 prefix cache 命中后"剩余 prefill"是怎么切的。

源码锚点: [vllm/v1/core/kv_cache_utils.py](#) • [hash_block_tokens](#)

- 想看源码: vllm/v1/core/kv_cache_utils.py (hash_block_tokens) 和 vllm/v1/core/kv_cache_manager.py (allocate / free)。
- 想做实验: 07-hands-on/03-mini-experiments.md 第 3 个实验"prefix hit"。
- 想看 L2/L3 cache: vllm/v1/kv_offload/ + LMCache 项目。

05. Chunked Prefill

谁该读这一篇？ 想理解"为什么一个长 prompt 进来不会让其他人卡死"、"prefill 和 decode 怎么混在同一步算"、"max-num-batched-tokens 调多大合适"的读者。

前置阅读： 02-continuous-batching.md（先理解 continuous batching 才能讲 chunk）；最好读过 04-prefix-caching.md（prefix cache 命中后剩余 token 会走 chunked prefill）。

耗时： 约 12 分钟。

学完能：

- 1. 画出"开 / 不开 chunked prefill"两种 step 时长对比图。
- 2. 解释 chunk 内 attention 的 mask 怎么保证 causal 正确。
- 3. 按业务（低 TTFT vs 低 TPOT）选 max-num-batched-tokens。
- 4. 在 metrics 上识别"该开 chunked 但没开"的症状（TPOT p99 长尾、iteration token 方差大）。

当前源码复核（b23bd73）： chunked prefill 默认值取决于 model_config.is_chunked_prefill_supported，不是对所有 runner/模型/平台无条件为 True；max_num_batched_tokens 也按 usage context 与硬件生成。下文的 8192 是算例或 ≥ 70 GiB 非 A100 API server 的当前默认之一，不是全局默认。

1. 问题：长 prefill 会"占据"整个 step

朴素 continuous batching 的 step：

running: [A decode 1tok][B decode 1tok][C prefill 8192tok] ← 一步算 8194 token

C 的 prefill 极大（8192 tokens 一次 forward），导致这一步的 GPU 时间可能 200ms+。其他用户感受到的 **TPOT（每 token 延迟）** 突然飙高 → 卡顿。

```
gantt
  title 不开 chunked prefill 时的 step 时长
  dateFormat X
  axisFormat %s

  section 不开
  step n-1 (decode 全员)    :a1, 0, 30
  step n (含 8192-tok prefill) :a2, 30, 230
  step n+1 (decode 全员)    :a3, 230, 260

  section 开 chunked
  step n-1                  :b1, 0, 30
  step n (decode + 2048 chunk) :b2, 30, 80
  step n+1 (decode + 2048 chunk) :b3, 80, 130
  step n+2 (decode + 2048 chunk) :b4, 130, 180
  step n+3 (decode + 2048 chunk) :b5, 180, 230
```

不开 chunked 时单个长 step **200 ms 全员卡**；开了之后均摊到 4 个 step，每步 50 ms，其他请求每步都能拿到 decode token——TPOT 平稳。

2. 解决：把 prefill 切块

chunked prefill：把 C 的 8192 token 切成 4 个 2048-token 的 chunk，每个 step 只跑一个 chunk：

```
step 1: [A decode][B decode][C prefill chunk 0..2048]
step 2: [A decode][B decode][C prefill chunk 2048..4096]
step 3: [A decode][B decode][C prefill chunk 4096..6144]
step 4: [A decode][B decode][C prefill chunk 6144..8192][C decode 第1个 token]
```

好处：

1. 单 step 时长平稳 → TPOT 抖动小
2. decode 和 prefill **同框** → GPU 利用率高
3. 长 prompt 不再阻塞所有人

代价：

1. C 的 TTFT 略增（多了几次 step 的 schedule overhead）
2. attention 计算要支持 chunk（注意 mask）

3. 关键技术挑战：chunked attention 的正确性

prefill 的 attention 是 **causal mask**（位置 i 只能看 0..i）。

切 chunk 后，第 N 个 chunk 的 attention 必须同时：

- 看到所有**前面 chunk 的 K/V**（已经在 KV cache 里）
- 看到**当前 chunk 内** token 的 causal mask

实现上：

- prefill chunk 调用的 attention 把 query 限定为当前 chunk 的 token
- 把 key/value 视野扩展到 KV cache 中已有的所有 token（包括之前 chunk 写入的）
- mask：当前 chunk 内 causal + 跨 chunk 全可见

FlashAttention v2/v3、FlashInfer 都原生支持这种 "varlen + paged + chunked" 模式。

4. 与 decode 混跑的实现

V1 调度器把 prefill chunk 和 decode 都视作"算几个 token"，统一处理：

```
SchedulerOutput.num_scheduled_tokens = {
  "A": 1,    # decode
  "B": 1,    # decode
  "C": 2048, # prefill chunk
}
total = 2050
```

Model runner 把所有请求的 input token 拼成一个 1D 序列长度 2050 的张量，attention 通过 seq_lens / query_start_loc / seq_start_loc 等元数据区分。

输入打平（packed batch）是 vLLM 的核心技巧之一——避免 padding。

5. 调度策略：先 decode 还是先 prefill?

每个 step 的 token budget B（以下用 8192 举例）需要在 prefill 和 decode 之间分配。两种直觉：

5.1 Prefill-first

优先把 budget 给新进的 prefill 请求。

- 优点：TTFT 低
- 缺点：决定到达的请求被服务前，老请求 TPOT 增大

5.2 Decode-first（V1 默认）

优先保证 running 请求 decode 1 个 token，剩余 budget 给 prefill。

- 优点：TPOT 稳定
- 缺点：TTFT 可能变长

实际 V1 是 **混合策略**：

1. 先给所有 running 请求各 1 个 decode token（占 |running| 个 budget）
2. 剩余 budget = B - |running|，用于 prefill（新请求或 chunked 续接）
3. 单步的总 prefill 不能超过 --max-num-batched-tokens

6. 参数旋钮

参数	作用	默认
--enable-chunked-prefill	开关；默认跟随模型是否支持	受模型/runner/平台约束
--max-num-batched-tokens	单步总 token 上限（决定 chunk 大小）	按 usage context 与硬件生成
--long-prefill-token-threshold	超过此长度强制 chunk	与上面相关

调优经验：

- 追求低 TTFT → 大 max-num-batched-tokens（弱化 chunk）
- 追求低 TPOT → 小 max-num-batched-tokens（强 chunk）
- 不存在跨硬件的“90 分默认值”；以启动后的最终配置为基线做负载测试

6.1 按业务场景的推荐值

场景	max-num-batched-tokens	理由
代码补全（短输入、ITL 敏感）	2048	TPOT 抖动直接影响打字节奏
聊天 chatbot	4096-8192	默认通用值
RAG 长上下文（输入 16K+）	8192-16384	prefill 太多次会拖 TTFT；适度大 chunk
离线 batch 摘要	16384+	TTFT/TPOT 都不敏感，最大化 GPU 利用
Agent 工具调用（多轮短交互）	4096	平衡新轮次 TTFT 与老轮次 TPOT

调参流程：

1. 先按上表选初值，跑 benchmarks/benchmark_serving.py 测当前 SLO（TTFT/TPOT p99）。
2. TPOT p99 高于目标 → 把值减半。
3. TTFT p99 高于目标且 GPU 利用率没到 90% → 把值加 2×，直到 GPU 利用率打满或 TPOT 退化。
4. 上线后看 vllm:iteration_tokens_total 的分布——理想是窄 + 集中在目标值附近。

7. 代码定位

行为	文件：函数
决定一个请求本步算几个 token	Scheduler._schedule_running / _schedule_waiting
算 budget	Scheduler.schedule() 内的 token_budget 计算
Attention 处理 chunk	vllm/v1/attention/backends/flash_attn.py 等的 FlashAttentionMetadata
把 batch 打平	vllm/v1/worker/gpu_input_batch.py

8. 工程自检问答

Q: chunked prefill 为什么要默认开？老的实现不开也能跑啊？ A: 不开的话长 prompt 会阻塞 decode，生产场景必然遇到长 prompt（document QA、长上下文 chat），TPOT 抖动用户体验不可接受。V0 时代 opt-in 是因为正确性还在验证、attention kernel 还不全支持。V1 已经全栈支持，自然 default on。

Q: chunk 大小有没有最优解？ A: 没有 universal 最优。经验法则：让 prefill 和 decode 的算力大致平衡。比如 8 个 decode 请求 + 一个 8192-token chunk 大致占 GPU 的同等时间。监控 `vllm:iteration_tokens_total` 调整。

Q: chunk 内的 attention 怎么算？ A: query 是当前 chunk 的 N 个 token，K/V 是 KV cache 里 cache 的所有历史（包括之前 chunk 写入的 + 当前 chunk 新写的）。FlashAttention 通过 varlen + paged 模式原生支持。

Q: 没开 prefix caching 时 chunked prefill 还有用吗？ A: 有。chunked prefill 的核心收益不依赖 cache，它是 schedule 层面的优化（避免 step 时长方差）。prefix caching 是另一维度的优化（减少计算）。两者正交。

小结

- chunked prefill 把长 prefill 切成多个 step，与 decode 同框运行，**把 step 时长方差压下来**——TPOT p99 直接受益。
- 正确性靠 "varlen + paged" attention：query 限定当前 chunk，K/V 可见全部历史。FlashAttention/FlashInfer 原生支持。
- V1 默认混合策略：先保证所有 running 各 1 个 decode token，剩余 budget 给 prefill。
- 调参主旋钮是 `max-num-batched-tokens`：大值偏 TTFT、小值偏 TPOT；按业务场景查上面的表。

自检

1. 画出"开 / 不开 chunked prefill"两种 step 时长对比图（参考 § 1 mermaid）。
2. C 的 prefill 在第 4 个 chunk 完成后立刻 decode 第一个 token——为什么调度器会这样安排？
3. 业务 SLO 是 TPOT p99 $\leq 30\text{ms}$ ，你把 `max-num-batched-tokens` 从 8192 改成多少？
4. chunked prefill 没开会让 `vllm:iteration_tokens_total` 直方图变什么样？

参考答案

1. 开启时，一个长 prompt 被拆成多个较窄的 prefill step，step 时长分布更集中，并能在中间插入 decode；关闭时，长 prompt 以大块甚至整段进入，直方图会出现更长的右尾，decode 请求的 ITL/TPOT 也更容易被尖峰拖慢。
2. 第四个 chunk 完成后，请求的 prompt KV 已齐，scheduler 会把它从“仍在 prefill”转成可 decode 状态。若本步 token budget 和 KV 资源允许，它可以在后续 step 立即生成第一个 token，不必额外等待一个完整 batch 周期。
3. 不能凭 SLO 直接写死一个数。应以 8192 为基线，按 4096、2048、1024 做单变量阶梯实验，固定到达率、长度分布和 warmup，选择仍满足 TPOT p99 $\leq 30\text{ ms}$ 且 goodput 最高的最大值；若 8192 已满足，就不应为了“看起来更小”盲目下调。
4. 未开启时，`iteration_tokens_total` 更容易集中在大 token 数和长尾区间，长 prefill 形成明显尖峰；开启后会出现更多中小 token step，分布更平滑。应同时看 `request_prefill_time`、TPOT p99 和 waiting queue，不能只凭直方图形状下结论。

下一步

- 至此 02-core-concepts/ 全部读完。下一步进入 03-code-walkthrough/01-entry-points.md，从源码角度走一遍调用链。
- 想从 metrics 角度看 chunked prefill 的健康度：08-production-deployment/05-slo-and-observability.md。
- 想看 attention 怎么处理 chunk：03-code-walkthrough/05-attention-backends.md。
- 想动手验证 chunk size 对 TPOT 的影响：07-hands-on/03-mini-experiments.md。

01. 入口与引擎主循环

谁该读这一篇？ 第一次打开 vLLM 源码、想知道"该从哪个文件开始读、调用链怎么串"的工程师。

前置阅读： 02-architecture.md、03-v0-vs-v1.md、04-project-structure.md

耗时： 约 15 分钟

学完能：

1. 复述一条请求从用户代码到 GPU forward 的完整文件级路径
2. 区分 LLMEngine (门面) 与 EngineCore (心脏) 的职责
3. 在源码里准确定位 9 个核心文件并知道每个文件的关键方法
4. 画出 step() 一次调用的 mermaid 调用链

当前源码复核 (b23bd73)： 在线主链以 OpenAIServingChat/Completion → AsyncLLM → EngineCoreClient → EngineCoreProc 为准；离线 LLM 可选择 in-process client。成功路径之外，request validation、EngineDead、client disconnect/abort 也必须沿同一 request ID 追踪。

一个请求是怎么从用户代码（或 HTTP）走到 GPU 的？本节回答"我打开 vllm 仓库，从哪里开始读"。

1. 两种使用方式 → 两个入口

方式 A：离线批推理 (offline)

```
from vllm import LLM, SamplingParams
llm = LLM(model="meta-llama/Llama-3-8B")
outputs = llm.generate(prompts, SamplingParams(...))
```

入口：vllm/entrypoints/llm.py : LLM

方式 B：在线服务 (online, OpenAI 兼容)

```
$ vllm serve meta-llama/Llama-3-8B --port 8000
$ curl -X POST localhost:8000/v1/completions ...
```

入口：vllm/entrypoints/openai/api_server.py

脚本：vllm/scripts.py 里 `vllm serve` 子命令

两条路最终都汇聚到同一个 **LLMEngine** (V1)。

2. 从 LLM.__init__ 看一遍

vllm/entrypoints/llm.py 里的 LLM 类是离线入口。简化的初始化逻辑：

```
class LLM:
    def __init__(self, model, **kwargs):
        # 1. 把所有参数打包成 EngineArgs
        engine_args = EngineArgs(model=model, **kwargs)
        # 2. 解析成多个具体 config
        vllm_config = engine_args.create_engine_config()
        # 包括: ModelConfig / ParallelConfig / SchedulerConfig /
        #       CacheConfig / DeviceConfig / LoRAConfig / ...
        # 3. 创建引擎
        self.llm_engine = LLMEngine.from_vllm_config(vllm_config)
```

generate() 流程：

```
def generate(self, prompts, sampling_params):
    for prompt in prompts:
        # tokenize (在 main 进程里)
        token_ids = self.tokenizer.encode(prompt)
        # 入队
        self.llm_engine.add_request(req_id, token_ids, sampling_params)

    outputs = []
    while self.llm_engine.has_unfinished_requests():
        step_outputs = self.llm_engine.step()
        outputs.extend(step_outputs)
    return outputs
```

3. OpenAI API Server

vllm/entrypoints/openai/api_server.py 用 FastAPI 实现：

- /v1/completions： legacy completion
- /v1/chat/completions： chat completion（支持 tool_use、structured output）
- /v1/embeddings： 嵌入向量
- /v1/models： 列出已加载模型
- /metrics： Prometheus 监控

每条请求路径：

```
HTTP → FastAPI handler → AsyncLLMEngine.add_request()
    → 异步 yield 流式 token
    → SSE 输出给客户端
```

AsyncLLMEngine（vllm/v1/engine/async_llm.py）是 LLMEngine 的 async 包装，每个请求维护一个 asyncio queue，每步从 engine 拿增量 token 推到 queue。

4. LLMEngine 在 V1 的真实角色

vllm/v1/engine/llm_engine.py（约 400 行）

注意：V1 的 LLMEngine 比 V0 轻得多——它是门面（**Facade**），真正干活的是另一个进程里的 EngineCore。

```
class LLMEngine:
    def __init__(...):
        self.engine_core: EngineCoreClient = ... # ZMQ 客户端
        self.tokenizer = ...
        self.output_processor = OutputProcessor(...)

    def add_request(self, req_id, prompt, sampling_params):
        # tokenize, 多模态预处理
        engine_request = self._make_engine_core_request(...)
        # 通过 ZMQ 把请求扔给 EngineCore 进程
        self.engine_core.add_request_async(engine_request)

    def step(self):
        # 从 ZMQ 拉本步 output
        outputs = self.engine_core.get_output()
        # 解 token id → text
        return self.output_processor.process(outputs)
```

5. EngineCore：调度引擎本体

vllm/v1/engine/core.py

这是 vLLM 的"心脏"进程。每个 instance 一个 EngineCore 进程。

```
class EngineCore:
    def __init__(self, vllm_config):
        self.scheduler = Scheduler(...)
        self.model_executor = Executor(...) # Worker 的代理
        self.request_map: dict[str, Request] = {}

    def run_busy_loop(self):
        while True:
            # 从 ZMQ 收新请求
            for req in self.input_queue.drain():
                self._add_request(req)

            # 一步调度 + 执行
            scheduler_output = self.scheduler.schedule()
            if scheduler_output.total_num_scheduled_tokens > 0:
                model_output = self.model_executor.execute_model(scheduler_output)
                engine_core_output = self.scheduler.update_from_output(scheduler_output, model_output)
                # 把结果发回 LLMEngine (也是通过 ZMQ)
                self.output_queue.put(engine_core_output)
            else:
                # 没活干, sleep 一会
                time.sleep(...)
```

`scheduler.schedule()` → `execute_model()` → `update_from_output()` 这三步是一次 step 的全部，看懂这三步你就懂了主流程。

6. Executor：把 SchedulerOutput 扔给 Worker

vllm/v1/executor/ 下有多种实现：

实现	用途
uniproc_executor.py	单卡，直接调用 Worker（同一进程）
multiproc_executor.py	单机多卡，多进程 + shared memory IPC
ray_distributed_executor.py	多机多卡，Ray actor
abstract.py	接口定义

接口大致是：

```
class Executor:
    def execute_model(self, scheduler_output) -> ModelRunnerOutput:
        # 把 scheduler_output 分发给所有 worker (TP/PP rank)
        # 收集 driver worker 的输出
        ...
```

7. Worker：每张 GPU 一个

vllm/v1/worker/gpu_worker.py

```
class Worker:
    def __init__(self, vllm_config, rank, local_rank):
        # 设置当前 GPU
        torch.cuda.set_device(local_rank)
        # 初始化分布式 (NCCL)
        init_distributed(...)
        # 加载模型分片
        self.model_runner = GPUModelRunner(vllm_config, ...)

    def load_model(self):
        self.model_runner.load_model()

    def initialize_cache(self, num_gpu_blocks):
        # 真正分配 KV cache 显存
        ...

    def execute_model(self, scheduler_output):
        return self.model_runner.execute_model(scheduler_output)
```

8. GPUModelRunner：每步前向的本体

vllm/v1/worker/gpu_model_runner.py（这个文件很大，3000+ 行）

```
class GPUModelRunner:
def execute_model(self, scheduler_output):
    # 1. 更新 InputBatch (增删请求行)
    self._update_states(scheduler_output)
    # 2. 准备输入 tensor (token_ids, positions, block_tables, ...)
    model_input = self._prepare_model_input()
    # 3. 跑前向 (可能 CUDAGraph)
    hidden_states = self.model(**model_input)
    # 4. LM head + sample
    logits = self.model.compute_logits(hidden_states, ...)
    sampled = self.sampler(logits, sampling_metadata)
    # 5. 返回
    return ModelRunnerOutput(sampled_token_ids=sampled, ...)
```

9. 一次 step 的完整调用链

API/对象	内部对象	边界	状态变化	可观测证据
OpenAI protocol model	rendered EngineInput	API/renderer	validated	4xx 与 request ID
AsyncLLM.add_request	EngineCoreRequest	API → core client	collector registered	input/queue trace span
EngineCore.add_request	internal Request	IPC → EngineCore	WAITING	scheduler waiting / queue time
Scheduler.schedule	SchedulerOutput	core → executor	RUNNING / PREEMPTED	scheduled tokens / preemption counter
ModelRunnerOutput	EngineCoreOutputs	worker → core client	output/finished	first-token、 iteration、 finish reason

错误与取消不是旁路：validation 可在第一行提前返回；EngineCore 失败由 client 映射为 engine-dead；disconnect 触发 abort，Scheduler 从 waiting/running 清理，output collector 最终关闭。生产 trace 必须覆盖这些状态，而不只覆盖成功生成。

flowchart TD
 A["LLMEngine.step()
_{vllm/v1/engine/llm_engine.py}"]
 B["engine_core.get_output()
_{ZMQ recv}"]
 C["EngineCore.run_busy_loop iteration
_{vllm/v1/engine/core.py}"]
 D["scheduler.schedule()
_{vllm/v1/core/sched/scheduler.py}"]
 E["model_executor.execute_model(scheduler_output)
_{vllm/v1/executor/*}"]
 F["worker.execute_model(...)
_{vllm/v1/worker/gpu_worker.py}"]
 G["gpu_model_runner.execute_model(...)
_{vllm/v1/worker/gpu_model_runner.py}"]
 H["_update_states • _prepare_inputs"]
 I["self.model(**inputs)
_{vllm/model_executor/models/<arch>.py}"]
 J["每层
QKV → Attention(paged) → MLP → AllReduce"]
 K["sampler(logits, metadata)
_{vllm/v1/sample/}"]
 L["return ModelRunnerOutput"]
 M["scheduler.update_from_output(model_output)
_{更新 Request 状态
产出 EngineCoreOutput}"]

A --> B
A --> C
C --> D --> E --> F --> G
G --> H
G --> I --> J
G --> K --> L
C --> M

classDef facade fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
classDef sched fill:#fef3c7,stroke:#b45309,color:#1a1f29;
classDef exec fill:#f7f8fa,stroke:#5b6573,color:#1a1f29;
classDef gpu fill:#dcfce7,stroke:#15803d,color:#1a1f29;
class A,B facade;
class C,D,M sched;
class E,F,G,H exec;
class I,J,K,L gpu;

把这条链路走通之后，"vLLM 怎么处理一个请求"就能按 9 步讲清，并随时指出对应文件路径。

10. 推荐阅读顺序（源码入门）

按顺序读，每个文件挑核心方法看 + 加 print 跑一遍：

- 1. vllm/entrypoints/llm.py → 看 generate
- 2. vllm/v1/engine/llm_engine.py → 看 add_request、step
- 3. vllm/v1/engine/core.py → 看 run_busy_loop
- 4. vllm/v1/core/sched/scheduler.py → 重点，看 schedule、_schedule_running、_schedule_waiting、update_from_output
- 5. vllm/v1/core/kv_cache_manager.py → 看 allocate_slots
- 6. vllm/v1/worker/gpu_worker.py → 看 execute_model、load_model、initialize_cache
- 7. vllm/v1/worker/gpu_model_runner.py → 看 execute_model、_prepare_inputs
- 8. vllm/model_executor/models/llama.py → 看 LlamaForCausalLM.forward
- 9. vllm/v1/attention/backends/flash_attn.py → 看 FlashAttentionImpl.forward

通读这 9 个文件，对 vLLM 的运行流就有了完整心智模型，源码层面再深入哪一层都不会迷路。

小结

- vLLM 的两个入口（LLM 离线类、api_server 在线服务）最终都汇聚到同一个 LLMEngine，再通过 ZMQ 把请求转给独立进程里的 EngineCore。
- 一次 step 的核心三步是 scheduler.schedule() → executor.execute_model() → scheduler.update_from_output()，记住这三步就抓住了主流程。
- Worker 进程里 GPUModelRunner.execute_model 是 forward + 采样的实际本体，所有 GPU 上的活都在这里。
- V1 的 LLMEngine 是个轻量门面（约 400 行），真活在 EngineCore 进程；这点跟 V0 不同，是 V1 性能与可维护性的关键设计。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. add_request 和 step 各对应哪条 ZMQ 通道？发还是收？

- **add_request**：API Server / LLMEngine 端 → **发到** EngineCore 的 **ROUTER 入口 socket**（ROUTER↔DEALER 模式）。EngineCore 端从同一 socket **收**。
- **step**：EngineCore 端把生成的 token / finished 状态 → **发到** API Server 的 **PUSH 出口 socket**（PUSH↔PULL 模式）。API Server 从 PULL socket **收**。

两条通道**方向相反**：请求路径用 ROUTER/DEALER（请求-响应耦合），输出路径用 PUSH/PULL（单向流）。

源码：vllm/v1/engine/core_client.py 里 AsyncMPClient 维护这两个 socket。

2. 进程树里有几个进程？谁是 EngineCore / Worker？

```
$ ps -ef --forest | grep vllm
1234 vllm serve ...          ← API Server (主进程)
├─ 1235 EngineCore          ← EngineCore (子进程)
│   ├─ 1236 VllmWorker-0 (CUDA_VISIBLE_DEVICES=0)
│   ├─ 1237 VllmWorker-1 (CUDA_VISIBLE_DEVICES=1)
│   ├─ 1238 VllmWorker-2 (CUDA_VISIBLE_DEVICES=2)
│   └─ 1239 VllmWorker-3 (CUDA_VISIBLE_DEVICES=3)
```

TP=4 时共 **6 个进程**：1 API + 1 EngineCore + 4 Worker。

辨识方法：

- API Server：vllm serve 命令名，跑 FastAPI/uvloop
- EngineCore：进程名通常含 EngineCore 或 _run_engine_core，CPU 占用高
- Worker：进程名含 VllmWorker，绑定某张 GPU（nvidia-smi 能看到 PID）

3. 9 步调用链 LLM.generate(["hi"]) → FlashAttentionImpl.forward。

1.	LLM.generate(["hi"])	vllm/entrypoints/llm.py
2.	→ LLMEngine.add_request(req)	vllm/v1/engine/llm_engine.py
3.	→ core_client.add_request_async()	vllm/v1/engine/core_client.py
4.	→ ZMQ ROUTER socket 发请求	(跨进程)
5.	→ LLMEngine.step()	同上
6.	→ EngineCoreProc.run() 主循环	vllm/v1/engine/core.py
7.	→ Scheduler.schedule()	vllm/v1/core/sched/scheduler.py
8.	→ Executor.execute_model(scheduler_output)	vllm/v1/executor/multiproc_executor.py
9.	→ Worker.execute_model()	vllm/v1/worker/gpu_worker.py
10.	→ GPUModelRunner.execute_model()	vllm/v1/worker/gpu_model_runner.py
11.	→ model.forward()	vllm/model_executor/models/llama.py
12.	→ Attention.forward()	vllm/model_executor/layers/attention.py
13.	→ FlashAttentionImpl.forward()	vllm/v1/attention/backends/flash_attn.py

实际是 13 步而非 9（早期教学时合并了几步）。**关键三跳**：① 用户 → LLMEngine（同进程）② LLMEngine → EngineCore（ZMQ）③ EngineCore → Worker（共享内存 + RPC）。

4. step() 加日志统计本步 token 数：改 LLMEngine 还是 EngineCore？

改 EngineCore（vllm/v1/engine/core.py 的 EngineCoreProc.step() 内）。

理由：

- LLMEngine.step() 只是个**客户端 wrapper**——把请求扔给 EngineCore，再从 ZMQ 收回结果。它根本不知道 EngineCore 内部 schedule 了多少 token
- EngineCoreProc.step() 是真正的"step"——它持有 scheduler_output.total_num_scheduled_tokens 字段，直接 log 即可
- 加在 LLMEngine 端只能拿到"完成的请求数"，拿不到"本步 token 数"——粒度不对

实际代码（伪）：

```
# vllm/v1/engine/core.py
def step(self):
    so = self.scheduler.schedule()
    logger.info(f"step token count: {so.total_num_scheduled_tokens}") # ← 加在这里
    output = self.executor.execute_model(so)
    self.scheduler.update_from_output(so, output)
    return output
```

补充细节：vLLM 已经在 vllm/v1/metrics/loggers.py 把这个 metric 暴露成 Prometheus vllm:iteration_tokens_total（histogram），不需要手 log；要看就 curl /metrics。

下一步

- 下一节：02-scheduler.md（深入 2300 行的 Scheduler，看 token budget、preempt、chunked prefill 怎么实现）
- 想看源码：vllm/v1/engine/core.py、vllm/v1/worker/gpu_model_runner.py（最推荐用 IDE 跟一遍 step 调用栈）
- 想动手：07-hands-on/02-trace-a-request.md（加 print 实际追一条请求）
- 想从生产视角理解：08-production-deployment/01-deployment-architectures.md（看进程拓扑怎么映射到真实部署）

02. Scheduler 深读

谁该读这一篇？ 已理解 V1 请求生命周期，想真正读懂 token budget、KV 分配、抢占、spec decode、structured output 与 KV connector 如何在同一步互相约束的工程师。

前置阅读： 01-entry-points.md、02-core-concepts/03-kv-cache-management.md。

耗时： 约 25 分钟。

学完能：

- 1. 沿当前 Scheduler.schedule() 的真实顺序解释 running / waiting / skipped waiting。
- 2. 手算一个 step 的 token、encoder 与 KV block 预算。
- 3. 准确描述 FCFS/PRIORITY 的 victim 选择和当前 recompute 抢占。
- 4. 从 SchedulerOutput 判断 runner 本步要新增、更新、释放哪些状态。

静态复核： 锁定 b23bd73f540175f9e117eae5029cd7d8df63964。当前实现不是 _schedule_running() / _schedule_waiting() 两个方法，也没有 --preemption-mode swap；这些旧版伪代码不再作为事实引用。

1. 先抓住统一模型

源码锚点：[Scheduler.schedule](#)

当前注释先否定“prefill phase / decode phase”二分法。每个请求只有两组关键量：

- num_computed_tokens：已经由模型计算完成的位置；
- num_tokens_with_spec + num_output_placeholders：本轮希望追上的位置。

差值就是候选 num_new_tokens。长 prompt 得到多个 token，普通 decode 通常得到一个或少量 speculative token；Scheduler 对它们使用同一个预算循环。这一抽象自然覆盖 chunked prefill、prefix cache、spec decode 和 jump decoding。

2. 三个队列/集合，不是两个阶段

状态	含义	本步可能发生什么
running	已被 runner 持久化、持有或关联执行状态	先尝试追赶 token；KV 不足时可能成为 victim
waiting	尚未进入或被抢占后重新排队	在 running 之后准入；先查 prefix/external KV，再分配槽位
skipped_waiting	本次遍历因约束暂时跳过	后续 step 重新尝试，避免一个阻塞请求卡住全部队列

此外还有 paused streaming session、encoder cache、KV connector pending 等状态。面试时说“只有 waiting/running”可以做入门简化，但读源码时必须看到这些阻塞状态。

3. schedule() 的真实顺序

```
flowchart TD
    A[初始化 token / encoder budget] --> B[KV manager: new_step_starts]
    B --> C[遍历 running]
    C --> D{计算 num_new_tokens}
    D -->|0| C
    D -->|>0| E[处理 encoder / hybrid 对齐]
    E --> F[allocate_slots]
    F -->|成功| G[登记 scheduled running]
    F -->|失败| H[选择 victim 并 preempt]
    H --> F
    G --> I{本步发生 preempt?}
    I -->|否| J[遍历 waiting / skipped]
    I -->|是| K[不准入新 waiting]
    J --> L[生成 SchedulerOutput]
    K --> L
```

关键细节：

- 1. token_budget 取 max_num_scheduled_tokens，通常与 max_num_batched_tokens 相等，但 speculative decode 等场景可更小。
- 2. running 优先处理，不表示永远严格 FCFS；encoder budget、PP cadence、hybrid block alignment 等原因会让某请求 continue。
- 3. allocate_slots() 返回 None 才触发 KV 压力抢占。
- 4. 本步一旦发生抢占，就不再准入 waiting 请求，避免刚释放的资源马上引入更多状态变化。
- 5. waiting 准入还要满足最大 running 数、LoRA 容量、structured output、远端 KV/encoder connector 等约束。

4. 手算一个 step

设最终配置：

- max_num_scheduled_tokens = 16；
- running A 还差 1 个 decode token；
- running B 还差 5 个 speculative/normal token；
- waiting C 的 prompt cache 命中 8 token，仍需计算 20 token；
- encoder budget 充足，KV 也足够。

计算：

顺序	请求	申请	实际调度	剩余 budget
1	A	1	1	15
2	B	5	5	10
3	C	20	10	0

C 的 8 个 cache-hit token 影响 num_computed_tokens 起点，但不会消耗本步 forward token budget；剩余 20 个 miss token 被 chunk 为 10。下一步是否继续 C，取决于 A/B 新增输出、KV 和所有预算的最新状态。

5. KV 不够时发生什么

源码锚点：[Scheduler._preempt_request](#)

当前 `_preempt_request()` 做五件事：

1. 释放 request KV blocks 和 encoder cache；
2. 清除 inflight prefill；
3. 状态改为 PREEMPTED；
4. `num_computed_tokens = 0`，清掉 spec tokens；
5. prepend 回 waiting，并记录 preemption event/ID。

这就是 recompute。它没有 swap 分支。KV offload 或 P/D connector 会在独立接口中保存/加载外部状态，不能写成 `--preemption-mode swap`。

victim 选择：

- FCFS：`running.pop()`，牺牲队尾；
- PRIORITY：选择 (priority, arrival_time) 最大者。当前约定数值越小优先级越高，因此最大者是最低优先级、同优先级中较晚到达者。

如果 victim 已在本步被登记，Scheduler 还会归还它占用的 token/encoder budget，并移除相应输出字段。这是不能只看 `_preempt_request()` 的原因。

6. SchedulerOutput 是 runner 的差量协议

源码锚点：[SchedulerOutput](#)

字段族	作用
scheduled_new_reqs	首次发送完整请求数据，runner 建立持久状态
scheduled_cached_reqs	已知请求只发送变化，减少每 step IPC/CPU 开销
num_scheduled_tokens / total	本步每请求和总 forward token 数
scheduled_spec_decode_tokens	本步真正采用的 draft token 子集
scheduled_encoder_inputs	需要处理的图像/音频等 encoder item
finished_req_ids / preempted_req_ids	通知 runner 清理或重置持久状态
common prefix / connector metadata	cascade attention、外部 KV/encoder cache 的执行契约
structured-output flags	async scheduling 下 grammar bitmask 是否就绪

“Scheduler 决定，Worker 执行” 不是说 Worker 没有状态，而是 Worker 的状态变化由这个差量协议驱动。

7. 约束如何相互作用

Structured output

grammar 状态未就绪时，请求可能停在 waiting；async scheduling 还要保证用于 bitmask 的输出 token 已到达。不要把 grammar 开销只归因于 sampler。

Speculative decoding

候选 draft tokens 进入 num_tokens_with_spec，但受 model length、token budget 与本步允许数量裁剪。被拒 token 会影响下一步的 computed-token 校正。

Multimodal / encoder

除了 token budget，还有 encoder compute budget、encoder cache 容量和“一个 item 是否允许切分”的约束。num_new_tokens == 0 不一定是 token budget 用尽。

KV / EC connector

waiting 请求可能处于异步加载外部 KV 或 encoder cache 的状态；Scheduler 必须先确认完成，才能把请求变成可执行输入。超时与 connector failure 是生产排障的一等分支。

8. 生产诊断

当前 Prometheus counter vllm:num_preemptions_total 上升时，联看：

- vllm:kv_cache_usage_perc 是否贴近 1；
- waiting 数和 queue time 是否同步上升；
- prompt token 分布、max_num_seqs 与 max_num_batched_tokens；
- priority workload 是否让低优先级长请求反复重算；
- 外部 KV connector 是否卡在 load/save/ack。

不要只把 --gpu-memory-utilization 调高：0.92 已是当前默认，继续提高可能把 NCCL、CUDA Graph、临时 tensor 和模型动态峰值挤出安全余量。

9. 30 秒面试回答

V1 Scheduler 不区分 prefill/decode 阶段，而是让每个请求的 num_computed_tokens 追上已有 token，统一受 token、KV、encoder 和请求数预算约束。它先服务 running，再准入 waiting；allocate_slots 失败时按 FCFS 队尾或 priority 最低者抢占。当前抢占会释放 KV、清零进度并重新排队，没有 swap mode。结果通过 SchedulerOutput 以 new/cached request 差量交给 runner。

小结

- num_computed_tokens 与目标 token 的差值统一描述 prefill、decode 和 spec decode。
- KV、token、encoder、LoRA、grammar、connector 都能阻塞调度，不能只画一个 token budget。

- 抢占必须同时回滚本步预算和 runner 状态；当前 V1 使用 recompute。
- SchedulerOutput 是跨 Scheduler/runner 的增量状态协议。

自检

1. running 请求 `num_new_tokens == 0` 有哪四类原因？
2. 为什么发生一次 preempt 后，本步不再准入 waiting？
3. priority 数值 0 和 10 谁更高？victim 如何打破同优先级平局？
4. cache hit token 为什么不消耗 forward token budget？
5. preempted_req_ids 对持久 InputBatch 有什么意义？

参考答案

1. `num_new_tokens == 0` 可能是：请求已经完成本轮所需 token；KV/connector/grammar/encoder 等前置条件尚未就绪；本步 token budget 已耗尽；或者请求被取消、抢占、等待外部 load。要结合 `num_computed_tokens`、`waiting reason`、`connector output` 和 `request status` 区分，不能直接当成 scheduler bug。
2. preempt 表示本步资源已经不足，继续准入 waiting 会让 KV 压力更大并造成抖动。停止准入是一次 fail-safe：先释放 victim、回滚本步计划并让系统恢复可调度状态，再在后续 step 重新评估。
3. 具体比较规则要读当前 scheduler 的 priority key；通常 priority 数值越小优先级越高，victim 从低优先级/更适合释放的请求中选，再用到达时间、运行状态等字段打破平局。不要把“数值大”直觉当成优先级高，必须以配置和排序 key 为准。
4. cache hit token 已经有 KV，forward 不需要重新计算这些 token；它只减少新 token 的计算需求。因此 hit token 不消耗本步 forward token budget，但仍可能消耗 lookup、block allocation 和外部 load 的时间与资源。
5. 持久 InputBatch 不会因为一次抢占就丢失请求行。preempted_req_ids 告诉 runner 清理或重置这些请求的持久状态，避免旧 block table、采样状态或 slot 映射在重算时污染新 batch。

下一步

- 02b-scheduling-policies.md：公平性、优先级与抢占代价。
- 03-kv-cache-manager.md：allocate_slots() 为什么返回 None。

02b. 调度策略：FCFS vs PRIORITY、优先级反演与抢占代价

谁该读这一篇？ 想理解"vLLM 当前其实只有 2 种调度策略"、"PRIORITY 模式真的会优先级反演吗"、"应该把 SLA-敏感的请求标 priority 还是分独立 LLM 实例"的工程师；做平台调度路由的同学。

前置阅读： 02-scheduler.md（必须先读完 schedule() 主流程，本节专攻其中策略 / 队列层）；02-core-concepts/03-kv-cache-management.md（抢占代价来自 KV 重算或换出）。

耗时： 约 14 分钟。

学完能：

1. 说清 vLLM V1 仅有的 2 种官方策略：FCFS（deque） / PRIORITY（min-heap），数据结构、复杂度、各自的 add/pop 行为。
2. 推导 PRIORITY 模式下"抢占—释放—重排"的发生顺序，并解释为什么会出现"实质上的优先级反演"。
3. 量化当前 recompute 抢占代价，并与独立 KV offload 的传输代价比较。
4. 给定 SLA 描述（多租户 / 优先级请求 / 长尾），选择"用 PRIORITY"、"上多实例"、"加路由层"中的哪个方案。

当前源码复核（b23bd73）： 官方 policy 仍为 fcfs / priority；数值越小优先级越高。当前 _preempt_request() 只有释放 KV、清零进度并回 waiting 的 recompute 路径，--preemption-mode swap 已不是当前 V1 CLI。

Scheduler 文章讲了"一步内怎么决定跑谁"，本节是其中**队列层**的专题：vLLM 真正提供的调度策略只有两种，但里面藏了几个坑。

1. 仅有的 2 种官方策略

源码锚点：[vllm/v1/core/sched/request_queue.py](#) · [SchedulingPolicy](#)

源码：[vllm/v1/core/sched/request_queue.py](#)

```
class SchedulingPolicy(Enum):  
    FCFS = "fcfs"  
    PRIORITY = "priority"
```

没有 SJF（最短作业优先）、没有 EDF（最早 deadline）、没有 weighted fair queueing。想要这些都得自己在 routing 层做，或者通过 --scheduling-policy=priority + 自定义 request.priority 字段间接表达。

源码锚点：[vllm/v1/core/sched/scheduler.py](#) · [Scheduler.init](#)

策略在启动时定，**全局生效**（scheduler.py），运行中不能切。

2. 数据结构对比

维度	FCFS	PRIORITY
底层	<code>collections.deque[Request]</code>	<code>heapq min-heap list[Request]</code>
顺序键	入队顺序（append）	<code>(request.priority, request.arrival_time)</code> — 小者先
<code>add_request</code>	$O(1)$	$O(\log n)$
<code>pop_request</code>	$O(1)$ <code>popleft</code>	$O(\log n)$ <code>heappop</code>
<code>remove_request</code> （中途踢一个）	$O(n)$	$O(n) + O(n)$ <code>heapify</code>
<code>prepend_request</code> （preempt 回插）	$O(1)$ <code>appendleft</code>	无意义 — 仍按 priority 重排（见 § 170）
适用	公平、低 overhead	多租户/优先级分级、可控 TTFT 倾斜

`PriorityRequestQueue.prepend_request()` 注释直接说："In a priority queue, there is no concept of prepending"——就是 add 的别名。这点是 PRIORITY 模式坑的源头（见 § 5）。

3. 多个队列： `waiting` 与 `skipped_waiting`

源码锚点：[vllm/v1/core/sched/scheduler.py](#) • [Scheduler.init](#)

源码锚点：[vllm/v1/core/sched/scheduler.py](#) • [Scheduler.init](#)

`scheduler.py` 创建两个：

```
self.waiting = create_request_queue(self.policy)
self.skipped_waiting = create_request_queue(self.policy)
```

源码锚点：[vllm/v1/core/sched/scheduler.py](#) • [Scheduler.schedule](#)

源码锚点：[vllm/v1/core/sched/scheduler.py](#) • [Scheduler.schedule](#)

`skipped_waiting` 是个"被本步跳过"的旁置队列。常见跳过原因（`scheduler.py` 区段）：

- 请求超 `max_model_len`
- `encoder budget` 耗尽
- `encoder cache` 耗尽
- 混合模型（含 Mamba）`block` 不对齐

注释里有句关键话：

```
"Here, by doing continue instead of break, we do not strictly follow the FCFS scheduling policy and allow the lower-priority requests to be scheduled."
```

即：**FCFS 模式下，如果队头请求暂时不能跑（如等 `encoder cache`），后面的小请求会被先跑——FCFS 不严格守"先来先服务"。**这通常是好事（throughput +），但生产中调试"为什么我的请求被晚到的请求挤了"时要记住。

到下一步 `_select_waiting_queue_for_scheduling()` (`scheduler.py`) 会决定先消费哪个:

- FCFS: 先消费 `skipped_waiting`, 腾完再 `waiting`
- PRIORITY: 比较两队头, 选更小的 (`priority`, `arrival_time`)

4. 抢占 (preempt): 什么时候触发, 谁被踢

源码锚点: [vllm/v1/core/sched/scheduler.py](#) · [Scheduler.schedule](#)

源码锚点: [vllm/v1/core/sched/scheduler.py](#) · [Scheduler.schedule](#)

抢占只在**给 running 请求 allocate KV 失败**时触发 (`scheduler.py`):



关键点:

- **FCFS** 踢 running 末尾 (最近被加入 batch 的) —— 简单粗暴, 靠"近来的最易被踢"近似公平。
- **PRIORITY** 用 `max(self.running, key=lambda r: (r.priority, r.arrival_time))` 选最低优先级 (最大 priority 值)。
- 被踢请求重新入 waiting, 本步剩余 budget / 已分配 block 全部退还。
- 整个过程**单步内可发生多次** (while 循环), 直到 allocate 成功或 running 空。

4.1 当前 V1 使用 RECOMPUTE

当前实现选择 RECOMPUTE。工程原因包括:

- SWAP 需要 PCIe 拷贝 KV: H100 ↔ host ~50 GB/s, 70B 模型一个长上下文请求 KV 可达数 GB —— 拷贝几十到几百 ms。
- RECOMPUTE 只需在重新调度时跑一次 prefill; 现代 GPU FLOPS 充足, 且 chunked prefill 可分摊。
- prefix caching 命中时 RECOMPUTE 大部分免费 —— 前面 cached 的块复用, 只重算 miss 段。
- 综合 RECOMPUTE 通常**更快且实现更简单**。

需要 CPU/远端 KV 时, 应评估 `--kv-offload-backend` 或 `--kv-transfer-config` 的独立能力、状态协议和故障语义, 不能用不存在的 `preemption flag` 切换。

5. PRIORITY 模式的 3 个坑

5.1 "事实上的优先级反演"

场景：低优先级请求 A 已经跑了 80% 的 decode，KV 占 100 个 block。高优先级请求 B 到达，KV 不够，preempt 触发。

vLLM 会踢 A（因为它优先级最低）→ A 的 100 个 block 全释放 → B 进入。**A 之前消耗的 forward 算力 + 已生成的 token 全部白费**（RECOMPUTE 下要重新 prefill）。

这不是 vLLM 的 bug，是 priority preemption 的本质代价。**生产应用要意识到："标 priority=high" 是有外部性的——它会让 priority=low 的工作被丢弃。**

5.2 priority 是 add-only，不能动态改

request.priority 在 add_request 时就定下。运行中**没有 API 重新排队**。想给某个长尾请求"加速"必须重建请求。

5.3 prepend_request 在 PRIORITY 下退化

preempt 后请求要"放回 waiting 头"——FCFS 下是 O(1) appendleft，**保证它下次第一个被试**。PRIORITY 下退化为 add → 重新按 (priority, arrival_time) 排——它若 priority 高自然还在前，若 priority 低就**可能再次排到后面**，造成抖动。

6. 抢占代价的实际计算

6.1 RECOMPUTE 模式

被踢请求损失：

- 已生成 N 个 decode token 全丢
- prefill 算力 = $\text{prompt_len} \times \text{hidden_size}^2 \times \text{layers} \times 2$ （粗算）
- 加上重新调度的 schedule overhead

示例：Llama-70B + prompt=2000 + 已 decode 500 → 损失约 2500 token 的 forward。在 H100 上约 100-150 ms 算力 + 4 GB 显存短暂占用。

但若 prefix cache 命中率高（如 system prompt 长 + cache 未 evict），实际重算只是 miss 段——可能仅几十 ms。

6.2 独立 KV offload 的对比模型

被踢请求 KV 拷出 CPU + 恢复时拷回。Llama-70B 长 2500 token KV $\approx$ 2.5 GB（BF16 GQA）。

- H100 PCIe Gen5 $\times$ 16 单向理论 $\sim$ 64 GB/s → 一次 $\sim$ 40 ms
- 恢复时再 40 ms
- 占 host 内存

与 RECOMPUTE 对比：

- KV 极大、prefix cache 命中率低 → SWAP 偏好
- KV 中等、prefix cache 命中高 → RECOMPUTE 偏好

6.3 监控

抢占次数：Prometheus 暴露 `vllm:num_preemptions_total` (Counter)。生产突然 spike 通常意味着：

1. 大批高优先级请求涌入；
2. KV cache 容量配置过小 (`--num-gpu-blocks-override` 不足)；
3. 长上下文请求集中到达。

7. 选型决策表

场景	建议	理由
公平单租户（聊天 chatbot 各用户平等）	FCFS	简单、低 overhead、避免优先级反演损耗
多租户但优先级层级少（最多 3-5 档）	PRIORITY + 设计良好的 priority 映射	业务侧定义清楚 high/medium/low
多租户且 SLA 严苛	多实例 + 路由层 weighted	priority 抢占代价不可控，物理隔离更安全
想做 SJF（短请求先）	自己路由：output_tokens<N 的去专门实例	vLLM 本身不支持
Realtime（agent 中工具回调）	PRIORITY + 高 priority 给 callback request	用 priority 表达"必须插队"语义
长 batch 离线推理	FCFS	不需要打断，吞吐最大化

总原则：在 vLLM 内调度只能调到"队列"粒度。需要更细的策略（公平份额、严格 SLA、token-rate quota）必须**前置到路由层**——参考 08-production-deployment/02-smart-routing-and-load-balancing.md。

8. 自定义策略：能不能加？

理论上能：实现一个 RequestQueue 子类 + 把 SchedulingPolicy 加一项 + 修 scheduler.py 中 `if self.policy == ...` 的分支。

但实际**不建议**：

- vLLM 主线只有 FCFS / PRIORITY，每次主线升级你要重新 rebase。
- 复杂策略放路由层更合适——你能拿到全局视野（多实例、跨用户配额）。
- 调度复杂度 vs. 多卡 batching 的吞吐增益经常不成正比。

如果一定要做，从 `request_queue.py` 抽接口开始——它的 ABC 设计就是为扩展留的。

小结

- vLLM V1 官方只有 **FCFS (deque)** 和 **PRIORITY (heap)** 两种策略，启动时固定。
- FCFS 并不严格"先来先服务"——skipped_waiting 队列允许小请求绕过卡住的大请求 (throughput +)。
- PRIORITY 模式下抢占踢 $\max((\text{priority}, \text{arrival_time}))$ ，会**等效优先级反演**：低优请求做的工作可能被高优请求到达完全废弃。
- 当前 V1 抢占使用 RECOMPUTE；KV offload/transfer 是独立系统，必须另外核算传输、容量和一致性。
- 复杂策略 (SJF / EDF / weighted fair) 请放路由层做，不要在 scheduler 里加。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. PRIORITY 模式 num_preemptions 持续走高，3 个 root cause + 验证方法。

假设 root cause	验证方法
高优先级请求暴涨	vllm:request_success 按 priority 标签分桶 (如果有)，看 high 优先级的 QPS 是否突然涨
KV cache 容量不足	vllm:kv_cache_usage_perc 是否长时间 > 0.9；如是，扩 --gpu-memory-utilization 或减 max_num_seqs
优先级反演 (低优 victim 反复被踢、又重进 waiting 又被踢)	看 num_preemptions 与 request_success{finished_reason="length"} 的比值；高优请求 prompt 很短的话会反复挤掉长任务的 KV
PRIORITY queue + 长尾请求 (已运行的低优请求被新进的高优请求踢飞)	request_queue_time_seconds p99 异常高；说明请求在 waiting 长时间排队

排查命令：

curl :8000/metrics | grep -E "preempt|kv_cache_usage|queue_time"

2. FCFS 与 PRIORITY 各操作复杂度。

操作	FCFS (deque)	PRIORITY (heap)
add_request	$O(1)$ append	$O(\log n)$ heappush
pop_request	$O(1)$ popleft	$O(\log n)$ heappop
peek_request	$O(1)$	$O(1)$ (heap[0])
prepend_request (preempt 回插)	$O(1)$ appendleft, 立即排回头	退化为 add_request — $O(\log n)$ + 按 priority 排序, 可能再次落到中间或末尾
remove_request (中途踢一个)	$O(n)$ 线性搜索 + 移除	$O(n)$ 找位置 + $O(n)$ heapify 修复

关键差异：

prepend_request 在 FCFS 下"保证下次最先尝试", 在 PRIORITY 下**无保证**——这是 PRIORITY 模式抖动的源头 (参见正文 § 5.3)。

3. 7B 模型, prompt=4000, 已 decode 1000, preempt 后 RECOMPUTE 浪费多少 token?

被踢请求损失:

- 已 decode 的 1000 个 token 全部丢弃 (重新进 waiting 后从 prompt 重新 prefill)
- 加上**重新 prefill 4000 token**

总浪费的算力 $\approx 4000 + 1000 = 5000$ token 的 forward。

如果 prefix cache 命中 0% (题目假设), 这 4000 prompt prefill 全部重算。Llama-7B forward 在 H100 上单 token ~ 0.05 ms, 5000 token 重算约 **250 ms 算力**。

如果 prefix cache 命中 (典型场景命中 50-80%), 只需重算 cache miss 段 + 那 1000 decode $\rightarrow$ 实际浪费约 60-100 ms。

额外代价: 被踢请求要重新进 waiting $\rightarrow$ 等到再次被调度可能又过几个 step $\rightarrow$ 用户感知到的 latency 在原本 TTLT 上至少多 200-500 ms。

4. 多租户 A 付费 $10\times$ B 付费 $1\times$, 能不能用 PRIORITY 实现"A 拿 $10\times$ 吞吐"?

不能, 至少不能精确控制到 $10\times$ 。理由:

1. **PRIORITY 只是排序不是配额:** A 的请求总是先被调度。如果 A 请求数 $\leq B$, A 实际只占用 $\leq 50\%$ 资源; 如果 A 远多于 B, A 会**完全饿死** B, 比例 $\infty:0$ 而不是 $10:1$ 。
2. **vLLM 不支持加权 fair queueing:** FCFS 和 PRIORITY 都不能实现"按比例分配 token / step"。SchedulingPolicy 只有这两选。
3. **预算粒度太粗:** token budget 按步分配, 无法做 inter-request weighted 切分。

正确做法 (放路由层):

- **Weighted round-robin:** 路由器按 $10:1$ 比例 dispatch 请求到 vLLM 实例
- **多实例物理隔离:** A 用 10 个 vLLM pod, B 用 1 个, K8s HPA 各自扩缩容
- **Token rate limiting:** 在 API gateway 用 leaky bucket, A 限 10000 token/s, B 限 1000 token/s

$\rightarrow$ vLLM 内的 PRIORITY 只解决"严格优先级"问题 (如插入系统调用 / 紧急 callback), 不解决配额公平。详见 08-production-deployment/02-smart-routing-and-load-balancing.md。

下一步

- 下一节: 03-kv-cache-manager.md (理解 preempt 时 KV 是怎么 allocate / free 的)。

源码锚点: [vllm/v1/core/sched/scheduler.py](#) · [Scheduler.schedule](#)

源码锚点: [vllm/v1/core/sched/scheduler.py](#) · [Scheduler.schedule](#)

- 想看源码: [vllm/v1/core/sched/request_queue.py](#)、[vllm/v1/core/sched/scheduler.py](#) 的 preempt 段。
- 想做实验: 07-hands-on/03-mini-experiments.md 第 4 个实验"batching 极限"中可手工触发 preempt 观察 num_preemptions 增长。
- 想从生产视角理解: 08-production-deployment/02-smart-routing-and-load-balancing.md (路由层是 vLLM 调度策略的真正延伸)。

03. KVCacheManager 与 BlockPool 源码深读

谁该读这一篇？ 已经看过 Scheduler 的 allocate_slots 调用，想搞懂"一个 block 在 vLLM 内部经历什么生命周期、prefix cache 命中是怎么实现"的工程师。

前置阅读： 01-paged-attention.md、03-kv-cache-management.md、04-prefix-caching.md、02-scheduler.md

耗时： 约 22 分钟

学完能：

- 1. 画出 Scheduler → KVCacheManager → Coordinator → SingleType → BlockPool 的三层抽象关系
- 2. 在白板上画出 BlockPool 的内部数据结构（blocks 数组、free queue 双向链表、hash 表）
- 3. 解释 KVCacheBlock.ref_cnt 的 5 个变化时机，以及"free 时为何要 reverse"
- 4. 读懂 allocate_slots 的 9 个参数和那张五段 layout 图

当前源码复核（b23bd73）： KVCacheManager 通过 coordinator 支持单类型与 hybrid group；allocate_slots() 返回 None 仍是 Scheduler 的容量失败信号。普通 block 的 ref_cnt==0 对应 free queue，null block 是例外；hash key 还包含 group/extra context，不能只按 token tuple 理解。

核心文件是 vllm/v1/core/kv_cache_manager.py 与 vllm/v1/core/block_pool.py；行数随主线变化，不作为接口事实。这是 PagedAttention 的工程心脏，本节按数据契约与调用顺序拆解。

1. 类的关系：三层抽象



设计意图： 让一个模型可以同时含多种"KV 类似物"（Transformer KV + Mamba state + sparse attention），每种走自己的 manager，但对 Scheduler 暴露统一的 allocate_slots 接口。

学习时可以先只看 Full Attention 那一条线，忽略 coordinator 的多类型分支。

2. KVCacheBlock：一个物理 block 的元数据

vllm/v1/core/kv_cache_utils.py:

```
@dataclass
class KVCacheBlock:
    block_id: int          # 物理 block 在 KV 张量里的索引
    ref_cnt: int = 0       # 引用计数
    _block_hash: BlockHashWithGroupId | None = None
    prev_free_block: KVCacheBlock | None = None # free queue 前驱
    next_free_block: KVCacheBlock | None = None # free queue 后继
    is_null: bool = False  # null block 占位
```

不变式：

- `ref_cnt == 0` ☑ 在 free queue 里
- `ref_cnt > 0` ☑ 至少一个 Request 的 `block_table` 引用了它
- `_block_hash != None` ☑ block 已写满且参与 prefix caching

3. BlockPool：物理 block 的中央仓库

源码锚点：[vllm/v1/core/block_pool.py](#) • [BlockPool](#)

vllm/v1/core/block_pool.py — class BlockPool

3.1 初始化与 block pool 不变式

```
def __init__(self, num_gpu_blocks, enable_caching, hash_block_size, ...):
    # 创建所有物理 block 的元数据
    self.blocks: list[KVCacheBlock] = [
        KVCacheBlock(idx) for idx in range(num_gpu_blocks)
    ]
    # 双向链表的 free queue (LRU 顺序)
    self.free_block_queue = FreeKVCacheBlockQueue(self.blocks)
    # hash → block 索引 (prefix caching 的核心数据结构)
    self.cached_block_hash_to_block: BlockHashToBlockMap = BlockHashToBlockMap()
    # block_id=0 永远留作占位 (null block)，不参与分配
    self.null_block = self.free_block_queue.popleft()
    self.null_block.is_null = True
```

关键设计：

1. `blocks` 是预分配的**元数据数组**，长度固定为 `num_gpu_blocks`。物理 KV 显存张量在 Worker 那里持有；BlockPool 只管"哪个 id 现在被谁占"。
2. `free_block_queue` 是双向链表而非栈：`append_n` 放尾部，`popleft` 取头部 → **LRU 复用**。
3. `null_block` 是个占位 trick，避免 `block_id=0` 被误用。

3.2 分配新 block

```
def get_new_blocks(self, num_blocks: int) -> list[KVCacheBlock]:
    if num_blocks > self.get_num_free_blocks():
        raise ValueError(f"Cannot get {num_blocks} free blocks from the pool")

    ret: list[KVCacheBlock] = self.free_block_queue.popleft_n(num_blocks)

    if self.enable_caching:
        for block in ret:
            self._maybe_evict_cached_block(block) # 如果是 cached block, 先 evict
            assert block.ref_cnt == 0
            block.ref_cnt += 1 # ref_cnt 1
        else:
            for block in ret:
                block.ref_cnt += 1
    return ret
```

注意两件事：

- 从 free queue **头部**取（LRU 中最早 free 的）→ 配合 prefix caching，最旧的 cached block 才会被覆盖。
- 若取出的 block 仍带 hash（曾被 cache 过），先把它从 cached_block_hash_to_block 里移除（_maybe_evict_cached_block），才能给新请求用。

3.3 命中已 cache 的 block

```
def touch(self, blocks: Sequence[KVCacheBlock]) -> None:
    """Touch a block increases its reference count by 1, and may remove
    the block from the free queue."""
    for block in blocks:
        if block.ref_cnt == 0 and not block.is_null:
            self.free_block_queue.remove(block) # 从 free queue 拿出
            block.ref_cnt += 1
```

这是 prefix cache 命中时调用的：block 之前可能 ref_cnt=0（在 free queue 里待回收），命中后 ref_cnt=1、并从 free queue 里抽出。

3.4 释放 block

```
def free_blocks(self, ordered_blocks: Iterable[KVCacheBlock]) -> None:
    """Free a list of blocks. The blocks should be ordered by their
    eviction priority, where the first block will be evicted first."""
    blocks_list = list(ordered_blocks)
    for block in blocks_list:
        block.ref_cnt -= 1
        self.free_block_queue.append_n(
            [block for block in blocks_list if block.ref_cnt == 0 and not block.is_null]
        )
```

两个细节：

- 调用方负责把 blocks 按"优先回收顺序"传入（通常是 **逆序**：tail 先 free）。KVCacheManager.free 就是 reversed(...)。这样回收时 free queue 里 tail 排前面，会被先 evict（保留更可能复用的 head block）。
- 只有 ref_cnt → 0 的才真正放回 free queue。

3.5 把可复用 block 挂到 hash 表


```
def cache_full_blocks(self, request, blocks, num_cached_blocks, num_full_blocks, block_size, kv_cache_group_id):
    ...
    new_full_blocks = blocks[num_cached_blocks:num_full_blocks]
    new_block_hashes = block_hashes[num_cached_blocks:]
    for i, blk in enumerate(new_full_blocks):
        if blk.is_null:
            continue
        assert blk.block_hash is None
        block_hash = new_block_hashes[i]
        block_hash_with_group_id = make_block_hash_with_group_id(block_hash, kv_cache_group_id)
        blk.block_hash = block_hash_with_group_id
        self.cached_block_hash_to_block.insert(block_hash_with_group_id, blk)
```

这是 **prefix caching 写入端**：一个 block 写满了之后（比如 16 token 都填好），把它的 hash 注册到 `cached_block_hash_to_block`，之后其他请求 hash 查到同样的内容就能复用。

注意 hash 必须含 `kv_cache_group_id`——一个模型如果有多种 KV（如 sliding window + full），不同 group 的 hash 表分开。

4. KVCacheManager: Scheduler 看到的接口

源码锚点：[vllm/v1/core/kv_cache_manager.py](#) • [KVCacheManager.init](#)

`vllm/v1/core/kv_cache_manager.py` — class `KVCacheManager`

4.1 数据契约: KVCacheBlocks

Scheduler 拿到的不是 `list[KVCacheBlock]` 而是 `KVCacheBlocks`：

```
@dataclass
class KVCacheBlocks:
    blocks: tuple[Sequence[KVCacheBlock], ...]
    # 外层 tuple = kv_cache_groups
    # 内层 list = 该 group 的 block 序列
```

意图：Scheduler **完全不知道**内部有几种 KV group。它只管"给这个请求的所有 KV 加了 N 个 block"。

4.2 prefix 命中

```
def get_computed_blocks(self, request: Request) -> tuple[KVCacheBlocks, int]:
    if not self.enable_caching or request.skip_reading_prefix_cache:
        return self.empty_kv_cache_blocks, 0

    # NOTE: When all tokens hit the cache, we must recompute the last token
    # to obtain logits. Thus, set max_cache_hit_length to prompt_length - 1.
    max_cache_hit_length = request.num_tokens - 1

    computed_blocks, num_new_computed_tokens = (
        self.coordinator.find_longest_cache_hit(
            request.block_hashes, max_cache_hit_length
        )
    )

    if self.log_stats:
        self.prefix_cache_stats.record(
            num_tokens=request.num_tokens,
            num_hits=num_new_computed_tokens,
            preempted=request.num_preemptions > 0,
        )
    return self.create_kv_cache_blocks(computed_blocks), num_new_computed_tokens
```

关键点：

- **最后一个 token 必须重算**：因为 vLLM 需要它的 logits 来采样下一个 token；如果命中了它，反而没有 logits 可用。
- find_longest_cache_hit 按 block_hashes 链式查找——**第一个 miss 后停下**（前缀性质，文档里讲过）。

4.3 allocate_slots：容量与复用的核心

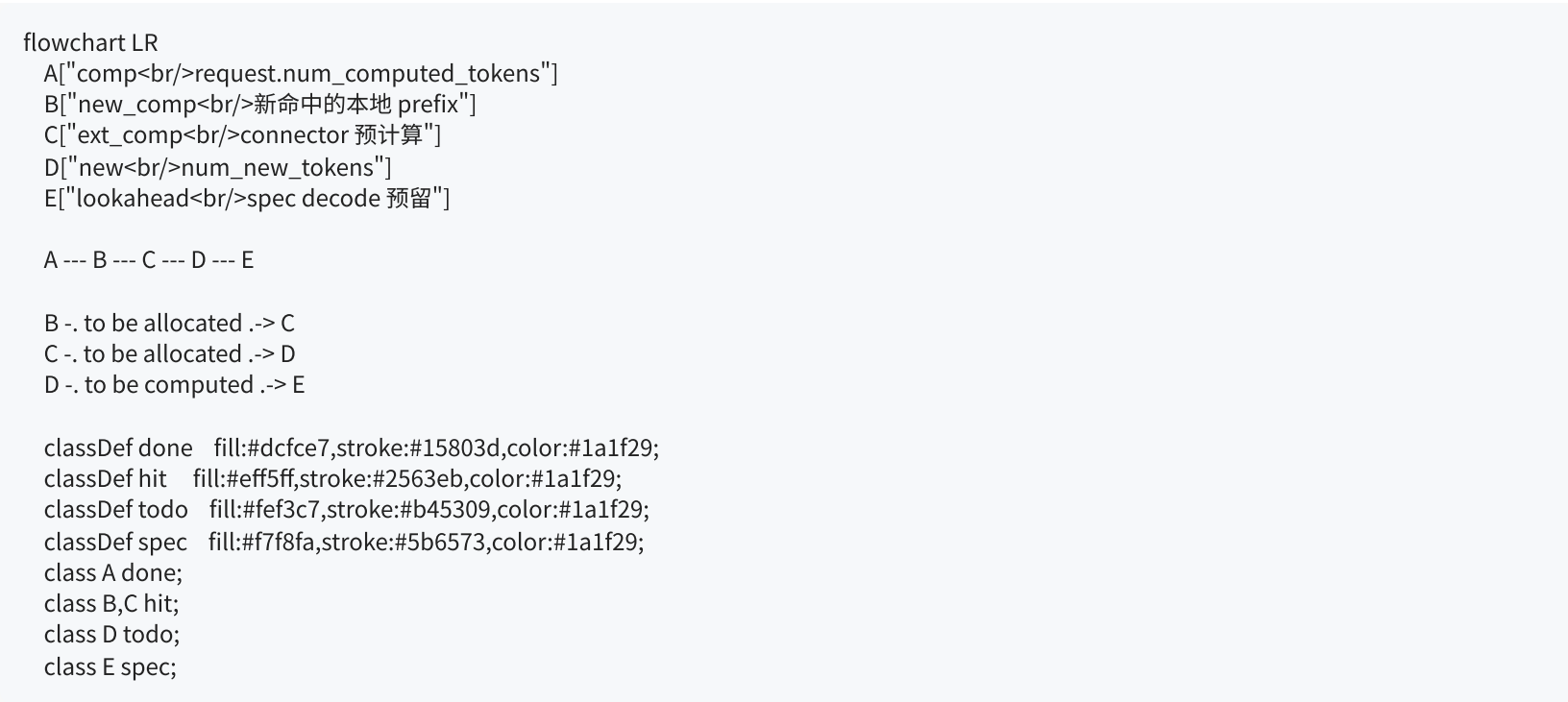
完整签名：

```
def allocate_slots(
    self,
    request: Request,
    num_new_tokens: int,
    num_new_computed_tokens: int = 0,
    new_computed_blocks: KVCacheBlocks | None = None,
    num_lookahead_tokens: int = 0,
    num_external_computed_tokens: int = 0,
    delay_cache_blocks: bool = False,
    num_encoder_tokens: int = 0,
    full_sequence_must_fit: bool = False,
) -> KVCacheBlocks | None:
```

参数语义（容易混淆，建议看代码时对齐）：

参数	含义
num_new_tokens	本步要算的 token 数（含未验证的 draft 投机 token）
num_new_computed_tokens	本次 prefix caching 新命中的 token 数（vLLM 本地 cache）
new_computed_blocks	上面对应的 block 列表（已经按 group 分好）
num_lookahead_tokens	spec decoding 的 lookahead 数（要为这些 未来 token 预留 slot）
num_external_computed_tokens	KV connector（如 LMCache）传来的预计算 token 数
delay_cache_blocks	P/D disaggregated 时，传输还没完成不立刻 cache
num_encoder_tokens	encoder-decoder 模型（如 Whisper）跨注意力的 encoder token
full_sequence_must_fit	入场准入控制：必须能装下整个请求才接收（避免长请求陷入循环 chunk）

文档头部画了 layout 示意（line 273-294），核心是这张图：



简化逻辑（去掉 corner case）：

```

# 1. 算这个请求一共需要多少个 block 槽位
num_tokens_main_model = total_computed_tokens + num_new_tokens
num_tokens_need_slot = min(num_tokens_main_model + num_lookahead_tokens, max_model_len)

# 2. 算实际要新分多少 block
num_blocks_to_allocate = self.coordinator.get_num_blocks_to_allocate(...)

# 3. 不够就返回 None (让 Scheduler preempt)
if num_blocks_to_allocate > self.block_pool.get_num_free_blocks():
    return None

# 4. 把 prefix cache 命中的 block 挂到请求上 (ref_cnt++)
if new_computed_block_list is not empty:
    self.coordinator.allocate_new_computed_blocks(...)

# 5. 真正分配新 block
new_blocks = self.coordinator.allocate_new_blocks(...)

# 6. 把已经写满的 block 注册到 hash 表 (除非 disaggregated 还在传输)
if not delay_cache_blocks:
    self.coordinator.cache_blocks(request, num_tokens_to_cache)

return self.create_kv_cache_blocks(new_blocks)

```

返回值约定：None = "无法分配，请 preempt"；非 None = "已分配的新 block 列表"。

4.4 free

```

def free(self, request: Request) -> None:
    """Free the blocks allocated for the request.
    We free the blocks in reverse order so that the tail blocks are evicted
    first when caching is enabled."""
    self.coordinator.free(request.request_id)

```

为什么 reverse? 因为 free queue 是 FIFO 出 (LRU)，先放进去的先被覆盖。把 tail block 先 free，head block 后 free → 后续 head block 排在 free queue 后面 → 不容易被立刻 evict → 保留前缀。

4.5 reset_prefix_cache

```

def reset_prefix_cache(self) -> bool:
    """清空 prefix cache，但保留正在 use 的 block。"""

```

用途：测试、workload 切换、内存压力大时。可以通过 API endpoint 触发。

5. 一次 allocate 的完整调用栈（追代码用）



```
classDef sched fill:#fef3c7,stroke:#b45309,color:#1a1f29;
classDef mgr fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
classDef pool fill:#dcfce7,stroke:#15803d,color:#1a1f29;
class SW sched;
class GCB,AS,FLC,SLF,GN,ANCB,ANB,CB mgr;
class BPC,BHM,Touch,GNB,Evict,CFB,Insert pool;
```

6. 推荐阅读顺序

精读建议（带 2-3 小时计划）：

- 1. vllm/v1/core/kv_cache_utils.py：先认清 KVCacheBlock、BlockHash*、hash_request_tokens
- 2. vllm/v1/core/block_pool.py：从 __init__ 一路读到 free_blocks
- 3. vllm/v1/core/kv_cache_manager.py：重点 get_computed_blocks + allocate_slots + free
- 4. vllm/v1/core/single_type_kv_cache_manager.py：看一种具体的 manager 怎么实现
- 5. （可选）vllm/v1/core/kv_cache_coordinator.py：多 group 协调

读完你应该能：

- 在白板上画 BlockPool 的内部数据结构
- 解释为什么 free 时要 reverse
- 描述一个请求 prefix 命中后 KV manager 内部发生的 5 个动作

小结

- KVCacheManager 之下分三层：Coordinator 协调多种 KV 类型、SingleTypeKVCacheManager 处理一种类型、BlockPool 真正持有物理 block 的元数据。
- BlockPool 的两个关键数据结构：blocks 数组按 id 索引、free_block_queue 是 LRU 双向链表，配合 cached_block_hash_to_block 支撑 prefix cache 复用。
- allocate_slots 返回 None 是约定"请 preempt"的信号；它会在一次调用里完成 hash 命中接管、新分配、写入 hash 表三件事。
- free 时按 reverse 顺序入队，让 head block 排在队尾，更不易被覆盖，从而最大化前缀复用。
- 最后一个 token 永远不命中 prefix cache（max_cache_hit_length = num_tokens - 1），否则没有 logits 可采样。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. block_pool.py 里所有改 ref_cnt 的函数 + ++ 还是 --。

至少 5 处（具体行号会变动，按函数名定位）：

函数	操作	触发时机
get_new_blocks	++（从 free queue 取出，0→1）	新请求 / 增加 token 需要 new block
_get_cached_block	++（hash 命中已 cached block）	Prefix caching 命中时复用
_free_block	--（请求 finish 或 preempt）	ref_cnt 到 0 时回 free queue
free_blocks 批量	--	同上，遍历多个 block
touch / _increment_ref_count	++	beam search / parallel sampling 共享时

补充细节：ref_cnt > 1 意味着多请求共享，这 block 不能写（COW 保护）；某些代码路径会 _split_block（分裂出新 block 给 writer），原 block ref_cnt--。

2. cached_block_hash_to_block 里的 block 是 ref_cnt == 0 还是 > 0？为什么两种都允许？

两种都有。

- ref_cnt > 0 的 cached block：正在被至少一个 active 请求使用（hash 表 + 持有引用）
- ref_cnt == 0 的 cached block：曾被使用、现在没人用了，但**还没被覆盖**——hash 表保留它的 entry，是 prefix cache 的"冷库存"

为什么两种都允许？

- 只允许 > 0 → 请求结束就立刻丢失 cache，多轮对话第二轮命中率 = 0
- 只允许 == 0 → cache 内容随时被人改写，不可信

→ "**ref_cnt == 0 但仍在 free queue + hash 表**"是 vLLM 的关键设计：等待被覆盖前可以被任何新请求 ref_cnt++ "捡回来"。一旦 free queue 取了这个 block 给新请求做不同内容，hash 表 entry 才被删除。

3. max_cache_hit_length = num_tokens - 1，设成 num_tokens 会怎样？

num_tokens - 1 的原因：必须留至少 1 个 token 让模型实际算 forward 才能产生 logits 采样下一个 token。

如果设成 num_tokens（全部命中）：

- prefill 完全跳过，进入 decode 直接...采样什么？
- decode 需要 logits = lm_head(hidden_state)，而 hidden_state 来自最后一个 forward
- 没 forward 就**没 logits**，sampler 直接 crash 或返回垃圾

实际语义：即使 cache 全命中，**最后一个 token 必须重算 forward**——但只算 1 token，prefill 时长几乎归零（这也是 prefix cache 命中后 TTFT 直降 95% 的根本机制）。

类比：你预订了一桌菜全部端上来（cache hit），但你还得自己拿起筷子吃下第一口（forward）才能知道下一口想吃什么（sample）。

4. prefix 命中 10 block + 新算 3 token, allocate_slots 内部调哪些函数？

按顺序：

```
allocate_slots(request, num_new_tokens=3)
|
|—— ① 先看 prefix 命中（如果还没做过）：
|   get_computed_blocks(request)
|   → 算 block_hashes (chain hash)
|   → 查 cached_block_hash_to_block，找到前 10 个 hit
|   → 给这 10 个 block ref_cnt++（_touch / _get_cached_block）
|   → request.block_ids[0..9] 填入这些 cached block id
|
|—— ② 算还需要几个新 block：
|   num_existing_blocks = ⌈(10 × 16 已 cached) / 16⌉ = 10
|   num_total_blocks = ⌈(10 × 16 + 3) / 16⌉ = ⌈163/16⌉ = 11
|   num_new_blocks = 11 - 10 = 1
|
|—— ③ 分配 1 个 new block：
|   block_pool.get_new_blocks(1)
|   → free_queue.popleft() → 拿到一个 ref_cnt=0 的 block
|   → 如果它之前 cached，从 cached_block_hash_to_block 移除 entry
|   → 设 ref_cnt = 1
|
|—— ④ 添加到 request.block_ids：
|   request.block_ids[10] = new_block.id
|
|—— ⑤ 返回新分配的 KVCacheBlocks 列表给 scheduler
```

整个过程中 KV manager **只更新 metadata（block_table / ref_cnt / hash 表）**，没有任何 KV 数据拷贝——KV 内容由 attention kernel 在 forward 时写入物理 block。这是 PagedAttention 设计上让 schedule 与 KV 写入完全解耦的关键。

下一步

- 下一节：04-model-runner.md（KV manager 分到的 block 在 Worker 那侧怎么被 attention kernel 读写）
- 想看源码：vllm/v1/core/kv_cache_manager.py、vllm/v1/core/block_pool.py、vllm/v1/core/single_type_kv_cache_manager.py
- 想动手：07-hands-on/03-mini-experiments.md（自己构造 prefix 命中、观察 free queue 长度变化）

- 想从生产视角理解：08-production-deployment/04-autoscaling-and-capacity.md（gpu-memory-utilization 与 num_gpu_blocks 的容量规划）

04. GPUModelRunner 源码深读

谁该读这一篇？ 已经搞清调度和 KV 管理，准备啃 vLLM 最长文件（3000+ 行）的 forward 工程实现的工程师；想答清"packed batch、slot_mapping、forward_context"等性能 trick 的同学。

前置阅读： 01-entry-points.md、03-kv-cache-manager.md、02-continuous-batching.md

耗时： 约 22 分钟

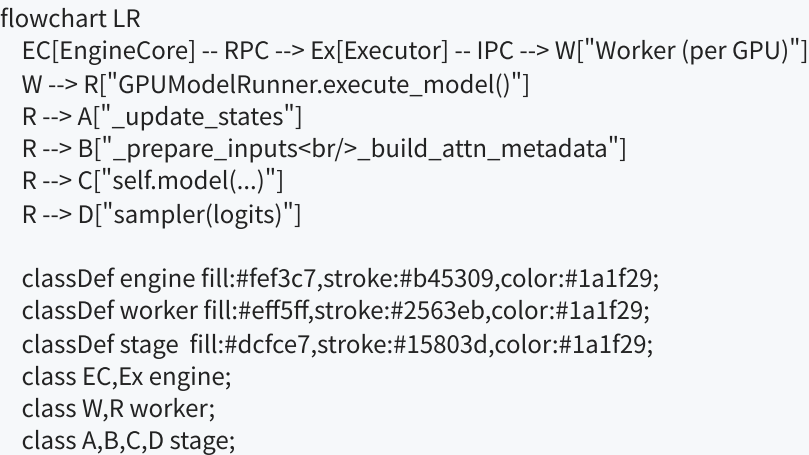
学完能：

- 1. 复述 execute_model 的 5 大阶段（update_states → prepare_inputs → mm encoder → forward → sample），每阶段干了什么
- 2. 解释 packed batch + query_start_loc 的设计为什么不需要 padding
- 3. 默写 slot_mapping 的计算公式与 PagedAttention 写入端的关系
- 4. 说明 InputBatch 持久化为何是 CUDA Graph 的前提条件
- 5. 看懂 forward_context 如何把 attention metadata 注入到无侵入式 nn.Module

当前源码复核（b23bd73）： GPUModelRunner.execute_model() 仍以持久 InputBatch、block table、attention/sampling metadata 为中心，但 compile/CUDA Graph dispatch、spec decode 与 async output 会改变一次调用的输入输出时序。prefill/decode 是工作负载标签，Scheduler 实际按统一 computed-token 模型发出差量。

文件：vllm/v1/worker/gpu_model_runner.py。它持续增长，行数和“最长文件”都不是接口事实；请从 GPUModelRunner.execute_model 与 InputBatch 的语义入口开始。这是 Worker 进程里“每步前向 + 采样”的本体。

1. 角色定位



GPUModelRunner 不直接接触 HTTP / 调度，它只接受 SchedulerOutput，跑一遍 forward + sample，吐 ModelRunnerOutput。

2. 类的结构（以语义成员为导航）

```
class GPUModelRunner(KVConnectorModelRunnerMixin, LoraModelRunnerMixin):
    def __init__(self, vllm_config, device):
        # 持久化输入 batch: 行索引稳定的请求 → buffer 映射
        self.input_batch = ... # vllm/v1/worker/gpu_input_batch.py
        # 各种持久 GPU buffer (避免每步分配)
        self.input_ids_cpu = ... # CPU 端 staging
        self.input_ids_gpu = ... # GPU 端
        self.positions_gpu = ...
        self.slot_mapping_gpu = ...
        # 模型本体
        self.model: nn.Module # Llama / Qwen / ...
        # 采样器
        self.sampler = ... # vllm/v1/sample/sampler.py
        # 注意力 metadata builder (每种 backend 一个)
        self.attn_metadata_builders = ...
```

2.1 持久化 buffer：避免每步分配

vLLM 性能秘诀之一：所有"形状最大可能"的 tensor 在 init 时预分配，每步只填一部分。例如：

```
self.input_ids_cpu = torch.zeros(max_num_tokens, dtype=torch.int32)
self.input_ids_gpu = torch.zeros(max_num_tokens, dtype=torch.int32, device="cuda")
```

每步：填前 total_num_scheduled_tokens 个槽位，剩余的不动。

为什么这样？因为 CUDA Graph 要求 tensor 地址固定。每步重新分配的话 graph 拒绝。

3. execute_model 的主循环（伪代码版）

GPUModelRunner.execute_model(scheduler_output)（实际函数比这复杂得多，先看骨架）：

```

def execute_model(self, scheduler_output):
    # ===== Stage A: 更新内部状态 =====
    # 新请求加入 input_batch、旧请求移除、状态行重排序
    self._update_states(scheduler_output)

    # ===== Stage B: 准备 forward 输入 =====
    # 拷 token_ids 到 GPU、算 positions、组 block_table、算 slot_mapping
    model_input = self._prepare_inputs(scheduler_output)
    attn_metadata = self._build_attention_metadata(...)

    # ===== Stage C: 多模态 embedding（如果有图像/音频） =====
    multimodal_embeds = self._execute_mm_encoder(...)

    # ===== Stage D: 模型 forward =====
    # 走 CUDA Graph 或 eager; attn_metadata 通过 forward_context 传给每层
    with set_forward_context(attn_metadata, ...):
        hidden_states = self.model(
            input_ids=model_input.input_ids,
            positions=model_input.positions,
            intermediate_tensors=..., # PP 用
            inputs_embeds=multimodal_embeds,
        )

    # ===== Stage E: 采样 =====
    # 只算 "需要 logits 的位置"（每个请求的最后一个 token）
    logits = self.model.compute_logits(hidden_states, sampling_metadata)
    sampler_output = self.sampler(logits, sampling_metadata)

    # ===== Stage F: 投机解码验证（如开启） =====
    if self.spec_decode:
        sampled = self._verify_and_accept(...)
    else:
        sampled = sampler_output.sampled_token_ids

    return ModelRunnerOutput(sampled_token_ids=sampled, ...)

```

4. _update_states：维护 InputBatch

这一段是 V1 的招牌优化——**不重建 batch，只更新 diff。**

```

def _update_states(self, scheduler_output):
    # 1. 移除完成的请求（释放对应行）
    for req_id in scheduler_output.finished_req_ids:
        self.input_batch.remove_request(req_id)

    # 2. 应用 preempt（行先标 inactive）
    for req_id in scheduler_output.preempted_req_ids:
        ...

    # 3. 新请求加入：找一个空行索引塞进去
    for new_req in scheduler_output.scheduled_new_reqs:
        row = self.input_batch.add_request(new_req)
        # 持久 buffer 的对应行写入 prompt_token_ids、sampling_params 等

    # 4. 现有请求增量更新：block_table 加 block_id、num_computed_tokens++
    for cached_req in scheduler_output.scheduled_cached_reqs:
        self.input_batch.update_request(cached_req)

```

效果：100 个并发请求每步只更新新增/删除/增量行，而不是重建全部状态。具体 CPU 收益依赖请求形状、runner 版本与硬件，应通过 profiler 量化，不能把 ms $\rightarrow$ μ s 当通用承诺。

参考：vllm/v1/worker/gpu_input_batch.py 里 InputBatch 的实现。

5. _prepare_inputs：组 forward 输入

这一步把 InputBatch 里的逻辑数据搬到 GPU forward 需要的 tensor 形态。核心要产出：

```
input_ids: [total_num_scheduled_tokens]  int32
positions: [total_num_scheduled_tokens]  int64 (RoPE 位置)
slot_mapping: [total_num_scheduled_tokens]  int64 (写入 KV cache 的位置)
block_table: [num_reqs, max_blocks_per_req]  int32 (读 KV cache)
query_start_loc: [num_reqs + 1]  int32
seq_lens: [num_reqs]  int32
```

5.1 slot_mapping：把新 token 的 KV 写哪？

对每个本步要计算的 token，slot_mapping 告诉 attention kernel："把这个 token 的 K、V 写到 KV cache 的哪个位置"。

```
slot = block_table[req][token_idx // block_size] * block_size + (token_idx % block_size)
```

这是 PagedAttention "写入端" 的 indirection。Attention kernel 拿到 slot_mapping 后，直接按物理位置存。

5.2 batch 打平 (packed batch)

所有请求的 token 拼成 **1D** 长 tensor：

```
请求 A (decode): [a]
请求 B (decode): [b]
请求 C (prefill 2048 token): [c0, c1, ..., c2047]

input_ids = [a, b, c0, c1, ..., c2047]      # 长度 2050
query_start_loc = [0, 1, 2, 2050]          # A 在 [0,1), B 在 [1,2), C 在 [2,2050)
seq_lens = [N_A, N_B, N_C]                 # 含 KV cache 里的历史
```

FlashAttention 的 varlen 模式就吃这种打平输入。

6. _build_attention_metadata

每种 attention backend 有自己的 metadata。GPU runner 在 init 时给每个 attention 层创建一个 builder，每步调一次：

```
def _build_attention_metadata(self, ...):
    metadata_per_layer = {}
    for layer_name, builder in self.attn_metadata_builders.items():
        metadata_per_layer[layer_name] = builder.build(
            num_actual_tokens=total_tokens,
            max_query_len=...,
            query_start_loc=...,
            seq_lens=...,
            block_table=...,
            slot_mapping=...,
        )
    return metadata_per_layer
```

FlashAttentionMetadataBuilder 等可以做 backend 特定的优化（如 FA3 的 scheduler_metadata 提前算好分片）。

7. forward 的"上下文注入"

forward 本身不能直接吃 metadata，因为模型代码（vllm/model_executor/models/llama.py 等）是普通 PyTorch nn.Module，签名固定。

vLLM 用 **forward_context** 这个 contextvars 把 attention metadata 注入：

```
with set_forward_context(attn_metadata, vllm_config):
    hidden_states = self.model(input_ids, positions)
```

每个 Attention layer.forward() 内部从 forward_context 拿对应 layer 的 metadata，再调 backend。

代码：vllm/forward_context.py

8. 多模态：在 forward 前算 embedding

vllm/v1/worker/gpu_model_runner.py 的 _execute_mm_encoder（line 2813）和 _gather_mm_embeddings（line 3024）：

```
# 1. 把图像/视频/音频喂给 vision encoder（如 CLIP / SigLIP）
mm_embeds = self.model.get_multimodal_embeddings(images, ...)
# 2. 把 token_ids 中的 placeholder（如 <image>）替换成对应 embed
input_embeds = self.model.get_input_embeddings(input_ids)
input_embeds = scatter_mm_embeds_into(input_embeds, mm_embeds, mm_positions)
# 3. forward 接受 inputs_embeds 而非 input_ids
hidden_states = self.model(inputs_embeds=input_embeds, ...)
```

vision encoder 也用 encoder_cache_manager 管 embed 复用（同一图片不重算）。

9. 采样：只算需要的 logits

vllm/v1/sample/

注意：不是每个 token 都需要 logits!

- prefill chunk 中间的 token：只是为了写 KV cache，不需要 logits
- 一个请求的最后一个 scheduled token：需要 logits 来采样下一个 token

ModelRunner 算 logits 时只对这些位置算：

```
# compute_logits 的 indices_to_sample 参数告诉它哪些行需要
logits = self.model.compute_logits(hidden_states, sampling_metadata)
# logits.shape = [num_reqs_to_sample, vocab_size]
```

这是巨大节省：每个 prefill 请求只算 1 行 logits，不是 N 行。

10. CUDA Graph 集成

GPUModelRunner 在启动时 capture_model() 录制多个 batch_size 的 graph：

```
def capture_model(self):
    for size in self.cudagraph_capture_sizes: # [1, 2, 4, 8, ..., max]
        # 用 dummy 输入跑一遍 forward, CUDA 录制到 graph
        ...
```

runtime 时：

```
if total_num_scheduled_tokens in self.cudagraph_sizes:
    # 用对应 graph replay
    graph = self.graphs[total_num_scheduled_tokens]
    graph.replay()
else:
    # 找最近的 capture size, pad 然后 replay
    ...
```

代码：vllm/v1/cudagraph_dispatcher.py

11. 推荐阅读顺序

长文件需要按语义入口阅读，行号会持续漂移：

1. **先读类成员**：知道有哪些 buffer 和 sub-component
2. **跳过初始化细节**（init 各种 buffer），直奔 execute_model
3. **拆五大阶段**：每个阶段单独读一遍 - _update_states - _prepare_inputs - _build_attention_metadata - 然后是 mm encoder、forward、采样
4. **配合**：打开 vllm/forward_context.py、vllm/v1/sample/sampler.py 一起看

12. 工程自检问答

Q: vLLM 一次 forward 为什么不用 padding? A: 用 packed batch (打平) + query_start_loc 描述每个请求的范围。FlashAttention varlen 模式直接吃这种输入。比 padding 节省算力 (不算无效位置)。

Q: slot_mapping 是干嘛的? A: 每个本步要算的 token 的 KV 要写到 KV cache 张量的哪个位置 (物理地址)。是 PagedAttention 写入端的 indirection 表。

Q: 为什么不是每个 token 都算 logits? A: 只有"要采样下一个 token 的位置"需要 logits。一个 N token 的 prefill 只对最后一个 token 算 logits, 节省 99% 的 LM head 计算 (vocab_size 通常 32k-256k, 是大头)。

Q: InputBatch 持久化的核心好处? A: ①避免每步重新分配 GPU tensor; ②满足 CUDA Graph 对固定地址的要求; ③Python overhead 从 ms 降到 μ s (不必重排列 100 行)。

小结

- execute_model 五阶段: _update_states 增量改 InputBatch、_prepare_inputs 组 GPU tensor、_build_attention_metadata 按 backend 生成元数据、forward 跑模型、sample 算 logits 并采样。
- packed batch + query_start_loc 让 vLLM 无需 padding, 所有请求拼成一条 1D 序列喂给 FlashAttention varlen。
- slot_mapping 是 PagedAttention 写入端的 indirection: $\text{block_table}[\text{req}][i // \text{bs}] * \text{bs} + i \% \text{bs}$, 决定本步每个新 token 的 KV 写到哪里。
- 所有"形状最大"的 GPU buffer 在 init 时预分配, 每步只覆盖前 N 行——既是性能优化, 又是 CUDA Graph 要求 tensor 地址固定的前提。
- forward_context 用 contextvars 把 attention metadata 透传给 nn.Module, 让模型代码保持普通 PyTorch 风格。

自检

不用照着原文复述, 重点是把现象、机制、源码入口和取舍讲顺。

1. 新请求加入 InputBatch 的 row 分配规则? 是否复用 free row?

规则: 维护一个 free_rows 集合 (位图或 list)。新请求来时优先从 free_rows.pop() 取**最小空闲 row index**; 没有空闲时往 next_row 扩。请求结束 / preempt 时把 row 加回 free_rows。

关键: 复用 free row 而不重新分配。 原因:

- InputBatch 是预分配的固定大 buffer ([max_num_seqs, max_model_len])
- 复用 row 意味着 GPU tensor 地址不变 → CUDA Graph 可以跨 step 复用
- 紧致填充 (取 min index) → 实际跑的 batch 是 input_batch[:num_active_rows], 省去对未用 row 的计算

源码定位: gpu_model_runner.py 找 _update_states 或 _add_request_to_input_batch, 关键变量是 self.free_rows。

2. prefill 1024 token, 最终算多少 logits? 为什么省 99%?

只算 1 条 logits (最后一个 token 的 hidden_state → lm_head)。

为什么?

- Prefill 的目的是把 prompt 的 KV 写入 cache，**不需要中间每个 token 的 logits**——这些 token 是用户给的，已知
- 只有最后一个 token 的 logits 用来采样**第一个新 token**

节省：lm_head 是 [H,V] 矩阵乘（hidden × vocab_size），V 通常 128K，**单次 lm_head 计算 $\approx 8192 \times 128000 \times 2 \text{ bytes} \approx 2 \text{ GB}$ 显存读取**。1024 token 全算 = 2 TB 读取；只算 1 个 = 2 GB。**省了 99.9%**。

实现：gpu_model_runner.py 里 _select_last_token_indices / _compute_logits_for_last_token——只把每个序列的最后一个 token 的 hidden state 喂 lm_head。

补充细节：spec decode 例外——草稿 token 要 logits 验证，所以 prefill+decode 算多条 logits。

3. query_start_loc / seq_lens / max_query_len 语义 + 为什么 varlen 需要 query_start_loc?

假设 3 个请求 packed batch：req A 跑 100 token (prefill chunk), req B 跑 1 token (decode), req C 跑 50 token (prefill chunk)：

字段	形状	内容	含义
query_start_loc	[num_seqs+1] = [4]	[0, 100, 101, 151]	每个请求在 packed batch 中的起始位置（前缀和）
seq_lens	[num_seqs] = [3]	[5000, 200, 1050]	每个请求 完整的 K/V 序列长度 （含已 cached 的 prefix）
max_query_len	scalar	100	本步最长 query（max over A/B/C 的 query 长度）

为什么 varlen 需要 query_start_loc?

FlashAttention varlen kernel 把所有 query 拼成 1D [total_query_tokens, num_heads, head_dim] 张量。kernel 需要知道**第 i 个 token 属于哪个序列**才能：

- 1. 用正确的 block_table[seq_id] 找它的历史 K/V
- 2. 用正确的 seq_lens[seq_id] 算 attention mask

query_start_loc 让 kernel $O(\log n)$ 二分找到 seq_id（或者通过 CTA 调度时静态指派）。**没有它，varlen 退化为 padded batch**——每个序列 pad 到 max_query_len，浪费 75%+ 算力。

4. 加一个全新 attention backend, gpu_model_runner.py 之外要改哪些文件?

文件	改什么
vllm/v1/attention/backends/<new_backend>.py	实现 AttentionBackend 接口：get_metadata_class()、get_impl_class()、get_kv_cache_shape()
同一文件里	实现 AttentionMetadataBuilder：从 SchedulerOutput 构造 metadata；以及 AttentionImpl.forward 调底层 kernel
vllm/v1/attention/selector.py（或 backend selector）	在 _Backend enum 加新条目；加 supports_* 检查逻辑；在 get_attn_backend() 中添加选择路径
vllm/platforms/<platform>.py	在 platform 默认 backend 决策树中加新分支
vllm/v1/attention/backends/utils.py	如果新 backend 用通用 metadata 字段，加 helper
tests/v1/attention/test_<new_backend>.py	单测 + 与 FlashAttention 数值一致性测试

可选：

- csrc/<new_backend>/：如果需要新 CUDA kernel
- vllm/_custom_ops.py：注册 PyTorch op

最重要的事：保证新 backend 与 PagedAttention 接口兼容，get_kv_cache_shape() 返回的 shape 要让 KV manager 正确分配。

5. CUDA Graph 为什么需要 InputBatch 持久化？每步重分配会怎样？

CUDA Graph 录制时："把 kernel A(input_ids_ptr_0x1234) 这条指令录下来"。input_ids 的**GPU 内存地址** 0x1234 被编码到 graph 里。

如果每步 input_ids = torch.zeros(...) 重新分配：

- 新 tensor 地址是 0x5678，与 graph 录制的 0x1234 不符
- replay graph 会读 0x1234 那块内存——里面是上一步的旧值（如果还没释放）或随机数据
- **结果错误，不报错**——CUDA Graph 不验证地址内容，沉默崩坏最难查

InputBatch 持久化：self.input_ids_buf = torch.zeros(max_size, ...) 在 init 时分配一次，每步往 buf[:n] copy 新内容。地址 0x1234 永远是这个 buf 的起始 → graph 录制有效。

补充细节：实际还要把 batch_size pad 到 capture 过的尺寸之一（如 [1,2,4,8,16,...,max_num_seqs]）。这就是为什么 vLLM 启动时要 capture 多个 graph（cudagraph_capture_sizes）。

下一步

- 下一节：05-attention-backends.md（ModelRunner 调用的 attention backend 是怎样的接口，FlashAttention 实现长什么样）
- 想看源码：vllm/v1/worker/gpu_model_runner.py、vllm/v1/worker/gpu_input_batch.py、vllm/forward_context.py
- 想动手：07-hands-on/04-profiling-and-debugging.md（用 nsys 看 _prepare_inputs、forward、sample 各自占比）
- 想从生产视角理解：08-production-deployment/04-autoscaling-and-capacity.md（CUDA Graph 的 capture size 列表对吞吐/延迟的影响）

05. Attention Backends 源码深读

谁该读这一篇？ 想搞懂 vLLM 怎么把 attention 抽象成可插拔后端、不同硬件/模型对应哪个 backend 的工程师。

前置阅读： 01-paged-attention.md、04-model-runner.md、03-kv-cache-manager.md

耗时： 约 18 分钟

学完能：

- 1. 列出 vLLM 主流 attention backend 和它们的适用硬件 / 模型
- 2. 解释 Backend / Impl / MetadataBuilder 三件套接口的职责划分
- 3. 默写 KV cache 张量的 5 维 shape，每一维含义和 FlashAttention 期望布局
- 4. 在白板上画出 query_len / context_len / seq_len 的关系
- 5. 描述 cascade attention 何时启用、解决什么问题

当前源码复核 (b23bd73)： backend 选择先经过 selector 与 platform capability 检查；模型类型 (MHA/GQA/MLA)、KV layout、dtype、head size、compute capability、compile/CUDA Graph 与显式 override 都能改变结果。下文不再把 FlashAttention、FlashInfer 或 Triton 写成跨环境静态排名。

目录： vllm/v1/attention/backends/。 vLLM 把"怎么算 attention"完全抽象成可插拔后端。本节看 backend 接口 + FlashAttention 实现。

1. 后端总表

vllm/v1/attention/backends/registry.py 定义了所有后端：

```
class AttentionBackendEnum(Enum, metaclass=_AttentionBackendEnumMeta):
    FLASH_ATTN      = "vllm.v1.attention.backends.flash_attn.FlashAttentionBackend"
    FLASH_ATTN_DIFFKV = "...flash_attn_diffkv.FlashAttentionDiffKVBackend"
    TRITON_ATTN     = "...triton_attn.TritonAttentionBackend"
    ROCM_ATTN      = "...rocm_attn.RocmAttentionBackend"
    ROCM_AITER_MLA  = "...mla.rocm_aiter_mla.AiterMLABackend"
    ROCM_AITER_FA   = "...rocm_aiter_fa.AiterFlashAttentionBackend"
    FLASHINFER     = "...flashinfer.FlashInferBackend"
    FLASHINFER_MLA  = "...mla.flashinfer_mla.FlashInferMLABackend"
    TRITON_MLA      = "...mla.triton_mla.TritonMLABackend"
    CUTLASS_MLA     = "...mla.cutlass_mla.CutlassMLABackend"
    FLASHMLA       = "...mla.flashmla.FlashMLABackend"
    FLASH_ATTN_MLA  = "...mla.flashattn_mla.FlashAttnMLABackend"
    FLEX_ATTENTION  = "...flex_attention.FlexAttentionBackend"
    CPU_ATTN       = "...cpu_attn.CPUAttentionBackend"
    TURBOQUANT     = "...turboquant_attn.TurboQuantAttentionBackend"
    # ...
```

直觉记法：

- 普通 Transformer attention： FLASH_ATTN / FLASHINFER / TRITON_ATTN

- DeepSeek MLA：所有名字含 MLA 的（FLASHMLA、CUTLASS_MLA、TRITON_MLA…）
- AMD ROCm 平台：ROCM_* 系列
- 实验 / 长尾：FLEX_ATTENTION（动态 mask）、TURBOQUANT、CPU_ATTN

2. 抽象接口

vllm/v1/attention/backend.py（注意是单数 backend，定义基类）：

```
class AttentionBackend(abc.ABC):
    @staticmethod
    def get_name() -> str: ...
    @staticmethod
    def get_impl_cls() -> type["AttentionImpl"]: ...
    @staticmethod
    def get_builder_cls() -> type["AttentionMetadataBuilder"]: ...
    @staticmethod
    def get_kv_cache_shape(...): ...
    # 各种 supports_xxx() 用于自动选择
```

三件套：

- **Backend**：静态信息（名字、KV shape、能力声明）
- **Impl**：实际跑 attention 的 forward
- **MetadataBuilder**：每步 build attention 需要的元数据

3. FlashAttentionBackend：一个重要 CUDA 候选

源码锚点：[vllm/v1/attention/backends/flash_attn.py](#) • [FlashAttentionBackend](#)

vllm/v1/attention/backends/flash_attn.py

```
class FlashAttentionBackend(AttentionBackend):
    @staticmethod
    def get_kv_cache_shape(num_blocks, block_size, num_kv_heads, head_size):
        # FlashAttention 期望 layout:
        # [2, num_blocks, block_size, num_kv_heads, head_size]
        #   ↑           ↑
        #   K, V 各一份   block 维度
        return (2, num_blocks, block_size, num_kv_heads, head_size)

    @staticmethod
    def supports_head_size(head_size: int) -> bool: ...
    @staticmethod
    def supports_kv_cache_dtype(kv_cache_dtype) -> bool: ...
    @staticmethod
    def supports_compute_capability(capability) -> bool:
        # SM80+ (A100)、SM89 (L40)、SM90 (H100)
        ...
```

4. FlashAttentionMetadata: 每步的"地图"

源码锚点: [vllm/v1/attention/backends/flash_attn.py](#) • [FlashAttentionMetadata](#)

flash_attn.py, 注释画了精彩的一图:

```
@dataclass
class FlashAttentionMetadata:
    # |----- N-1 iteration -----|
    # |----- N iteration -----|
    # |- tokenA -|.....|--- newTokens ---|
    # |----- context_len -----|
    # |----- seq_len -----|
    #                               |-- query_len ---|

    num_actual_tokens: int      # batch 内有效 token 数 (不含 padding)
    max_query_len: int          # 最大 query 长度 (决定 grid)
    query_start_loc: torch.Tensor # cumsum, 定位每个请求在 packed 输入里
    max_seq_len: int            # 最大 seq 长度 (含 KV cache 历史)
    seq_lens: torch.Tensor      # 每个请求当前 seq 长度
    block_table: torch.Tensor    # 读 KV 的间接索引
    slot_mapping: torch.Tensor   # 新 token 写到哪里

    # cascade attention: 共享前缀的 attention 优化
    use_cascade: bool
    common_prefix_len: int
    cu_prefix_query_lens: torch.Tensor | None
    prefix_kv_lens: torch.Tensor | None
    suffix_kv_lens: torch.Tensor | None

    # 长上下文: context parallel decode
    max_dcp_context_kv_len: int | None = None
    dcp_context_kv_lens: torch.Tensor | None = None
```

看懂这张图, 你才能解释"query_len、context_len、seq_len 的区别":

- context_len: N-1 步结束时已有的 token (KV 已写好的部分)
- query_len: 本步要新算的 token
- seq_len: context_len + query_len, 总长度

5. FlashAttentionImpl.forward (line 677)

简化版 (去掉 encoder/cascade/dcp 分支):

```
def forward(self, layer, query, key, value, kv_cache, attn_metadata, output):
    """
    query: [num_tokens, num_heads, head_size]
    key: [num_tokens, num_kv_heads, head_size] ← GQA 时 num_kv_heads < num_heads
    value: [num_tokens, num_kv_heads, head_size]
    kv_cache: [2, num_blocks, block_size, num_kv_heads, head_size]
    """

    if attn_metadata is None:
        return output.fill_(0) # profile run

    num_actual_tokens = attn_metadata.num_actual_tokens

    # 1. 拆 KV cache 为 K cache 和 V cache
    key_cache, value_cache = kv_cache.unbind(0)

    # 2. 必要时 view 成 FP8 dtype
    if is_quantized_kv_cache(self.kv_cache_dtype):
        dtype = FlashAttentionBackend.get_fp8_dtype_for_flashattn(self.kv_cache_dtype)
        key_cache = key_cache.view(dtype)
        value_cache = value_cache.view(dtype)

    # 3. 调用 FlashAttention varlen + paged
    flash_attn_varlen_func(
        q=query[:num_actual_tokens],
        k_cache=key_cache,
        v_cache=value_cache,
        out=output[:num_actual_tokens],
        cu_seqlens_q=attn_metadata.query_start_loc,
        seqused_k=attn_metadata.seq_lens,
        max_seqlen_q=attn_metadata.max_query_len,
        max_seqlen_k=attn_metadata.max_seq_len,
        block_table=attn_metadata.block_table,
        causal=attn_metadata.causal,
        # 还要传 new K/V, 让 kernel 顺手写入 KV cache
        k=key[:num_actual_tokens],
        v=value[:num_actual_tokens],
        slot_mapping=attn_metadata.slot_mapping,
        ...
    )
    return output
```

注意几个点：

1. K/V 写入 cache 是 attention kernel 顺手做的——不是单独一次 op。slot_mapping 决定写哪。
2. block_table 是 paged 寻址的核心：kernel 内部按 block_table[req][i] 找到第 i 个 block 的物理位置。
3. FlashAttention 的 varlen 模式让我们传打平的 1D 输入 + cu_seqlens_q，无需 padding。

6. 后端怎么自动选择？

vllm/v1/attention/selector.py 根据：

- 当前硬件（compute capability）
- 模型 head_size / KV dtype / sliding window
- 用户参数 --attention-backend
- 各 backend 的 supports_*() 静态方法

选出最优后端。优先级大致：

```
NVIDIA H100+ : FLASH_ATTN (v3) > FLASHINFER
NVIDIA A100  : FLASH_ATTN (v2) > FLASHINFER
NVIDIA L40   : FLASH_ATTN (v2) > FLASHINFER
DeepSeek MLA : FLASHMLA > FLASH_ATTN_MLA > CUTLASS_MLA > TRITON_MLA
AMD MI300    : ROCM_AITER_FA > ROCM_ATTN
CPU          : CPU_ATTN
```

用户强制：--attention-backend FLASHINFER。

7. 几个值得了解的对比

7.1 FlashAttention vs FlashInfer

- 都支持 paged KV、varlen、causal、GQA
- FlashAttention 与 FlashInfer 都有各自支持矩阵和快速路径；同一 H100 上也会随 prefill/decode 比例、GQA、batch、dtype 与版本改变结果。
- 先让 selector 给出兼容候选，再用真实请求分布 benchmark；不要从模型名或 GPU 名直接断言谁更快。

7.2 普通 Attention vs MLA

- 普通：每层每头 K、V 单独存
- MLA：把 KV 压成 low-rank latent，存 latent + 升维投影矩阵。**KV 占用降到 ~1/10**
- DeepSeek-V2/V3 用 MLA。代码：[vllm/v1/attention/backends/mla/](#)

7.3 FlashAttention vs Triton attn

- FlashAttention：预编译/专用 kernel，能力与 wheel/设备版本相关
- Triton attn：Python DSL，便于扩展，也可能在特定 shape 走优化路径
- fallback 只能在满足数值、布局和功能约束时发生，不保证任意后端互换

8. Cascade Attention（高级优化）

FlashAttentionMetadata.use_cascade、common_prefix_len 等字段实现一个加速：

- 一组请求共享前缀时（prefix caching 多请求场景）
- 不重复对前缀做 attention，而是**先算 prefix 一次**，再各请求算 suffix
- 数学等价，但访存减少（共享前缀的 K/V 只读一次）

实际生效条件：Scheduler.get_num_common_prefix_blocks(...) 报告有公共前缀且足够长。

9. 推荐阅读顺序

1. vllm/v1/attention/backend.py：基类
2. vllm/v1/attention/backends/registry.py：后端注册表
3. vllm/v1/attention/backends/flash_attn.py（重要候选后端）：
 - class FlashAttentionBackend：静态方法 - class FlashAttentionMetadata：那张精彩注释 - class FlashAttentionMetadataBuilder.build：每步怎么造 metadata - class FlashAttentionImpl.forward：怎么调 FA kernel
4. vllm/v1/attention/selector.py：自动选择
5. （MLA 想了解就读）vllm/v1/attention/backends/mla/flashmla.py

10. 工程自检问答

Q: vLLM 的 attention 是自己写的吗？ A: 早期是（csrc/attention/paged_attention_v1/v2.cu），现在主用 FlashAttention v2/v3 和 FlashInfer 的 paged 版本——它们原生支持 block_table。vLLM 自己的 kernel 作为 fallback。

Q: KV cache 在 GPU 上具体是什么布局？ A: [2, num_blocks, block_size, num_kv_heads, head_size]。第 0 维是 K vs V，第 1 维是物理 block 索引。block_table 通过 indirection 找物理位置。

Q: forward 时 K、V 是怎么写入 cache 的？ A: 由 attention kernel 顺带完成（拿到 slot_mapping 后直接索引写入），不是单独一个 cache write op。这避免了一次 kernel launch。

Q: query_start_loc 怎么用？ A: 它是 cumulative sum，长度 num_reqs + 1。请求 i 的 query 在打平输入里的范围是 [query_start_loc[i], query_start_loc[i+1]]。FlashAttention varlen 直接吃这个。

Q: cascade attention 解决什么？ A: 多个请求共享同一前缀时，避免重复对前缀算 attention，节省 K/V cache 访存。

小结

- attention 后端三件套：Backend（静态信息、能力声明）、Impl（forward 真正算）、MetadataBuilder（每步造元数据）。
- FlashAttention 的 KV cache 布局是 [2, num_blocks, block_size, num_kv_heads, head_size]，K/V 共享同一个 5D 张量。
- 关键三段长度：context_len（已缓存的）+ query_len（本步要算的）= seq_len；attention kernel 用 query_start_loc 在打平输入里定位每个请求。
- K/V 写入 KV cache 是 attention kernel 顺手做的（用 slot_mapping），不是单独 op，省一次 launch。
- selector 按硬件/模型自动选最优后端；MLA 走独立子目录，DeepSeek 这类模型必须用 MLA backend。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. N-1 与 N iteration 的关系（FlashAttentionMetadata 注释图复述）。

vLLM 的 attention 设计把"上一步 forward 完成的状态"作为"本步的输入历史"：

iteration N-1:

- 跑 forward，写新 K/V 到 cache 的 block_table[req] 末尾对应物理位置
- sample 出新 token，append 到 request.token_ids
- 更新 request.num_tokens 加 1

iteration N（本步）:

- 本步 query = 上步 sample 出的 1 个新 token (decode) 或 prefill chunk
- K/V cache 已含 N-1 步及之前全部 token 的 K/V
- attention: Q @ K[0..N-1] @ V[0..N-1]
- context_len = N-1（cached 历史）
- query_len = 1 (decode) 或 chunk_size (prefill)
- seq_len = context_len + query_len
- 写新 K/V 到 cache 末尾
- sample 出 next token

关键不变量：cache 里的 K/V 永远是"截至上一步末尾"的完整历史；本步 query 与之前 cache 拼起来做 attention，新 K/V 同步写入 cache 末尾。

2. K/V 写到 cache 哪里？slot_mapping[i] 与 block_table 换算公式。

KV cache 形状：[2, num_blocks, block_size, num_kv_heads, head_dim]（第 0 维：K vs V）。

每个本步 query token i 的 K/V 写入位置：

```
slot_id = slot_mapping[i]    # 由 metadata builder 提前算好
block_idx = slot_id // block_size
intra_block_offset = slot_id % block_size

K_cache[block_idx, intra_block_offset, :, :] = key[i, :, :]
V_cache[block_idx, intra_block_offset, :, :] = value[i, :, :]
```

slot_mapping[i] 怎么算：

```
req_id = sequence_id_of(i)
new_token_idx = num_cached_tokens_for(req_id) + (i - query_start_loc[req_id])
block_table_idx = new_token_idx // block_size
physical_block_id = block_table[req_id][block_table_idx]
slot_id = physical_block_id * block_size + (new_token_idx % block_size)
```

核心 indirection：物理 block id 来自 block_table（请求自己的逻辑→物理映射），slot id 是"扁平化的物理坐标"，让 kernel 一次 store 完事。

3. H100 + Llama 默认后端选择路径需要哪些 supports_* 通过？

源码：vllm/v1/attention/selector.py 的 get_attn_backend()。决策大致：

1. 检查 platform: CUDA / ROCm / TPU / ... → 进入 CUDA 分支
2. 检查模型类型: MLA? GQA/MHA? → 普通 Llama 是 GQA, 走标准 path
3. 检查 GPU 支持 FlashAttention v2/v3:
 - SM 8.0+ (A100/H100/Hopper) ✓
 - dtype = bfloat16 / float16 ✓
 - head_dim ∈ {32,64,128,256} ✓
 - 不需要 sliding window 特殊版本 ✓
4. 检查 use_v3:
 - SM 90 (H100) + FlashAttn v3 wheel 装了 → use v3
 - 否则降级 v2
5. 返回 FlashAttentionBackend

如果某一步不支持, selector 会继续尝试平台允许的候选; 不存在跨版本固定的“FlashInfer → Triton → 自家 kernel”总顺序, 必须以当前 selector 与启动日志为准。

H100 + 普通 Llama-3 也不能只凭名称断言结果: 必须读取启动日志/最终 backend, 并核对 dtype、head size、wheel 能力、环境变量与 compile 配置。

补充细节: FlashInfer 在某些场景 (特别是 GQA + 极大 batch) 比 FlashAttention v3 还快; 可通过 `VLLM_ATTENTION_BACKEND=FLASHINFER` 强制覆盖默认选择。

4. cascade attention 触发条件 + get_num_common_prefix_blocks 调用时机?

cascade attention 的核心 idea: 当 batch 内多个请求共享一段长 prefix (如同一 system prompt), 把 attention 拆成两段:

1. 共享段: 所有 query 一起跟同一份 KV 做 attention (cache miss 一次, 所有 query 重复利用)
2. 独立段: 每个请求各自跟自己后段的 KV

触发条件 (典型):

- batch 内请求数 ≥ 阈值 (一般 ≥ 2)
- 共享 prefix 长度 ≥ 阈值 (一般 ≥ 256 token)
- attention backend 支持 cascade (FlashInfer 已支持, FlashAttn 在加)

get_num_common_prefix_blocks 调用时机: scheduler 每步 schedule() 决策完 batch 后、构造 SchedulerOutput 前。它扫描本步 running 的所有请求的 block_table 前缀, 找最长公共前缀的 block 数, 写入 SchedulerOutput.num_common_prefix_blocks。Attention metadata builder 据此决定要不要走 cascade 模式。

源码: `vllm/v1/core/sched/scheduler.py::Scheduler.schedule()` 末尾、构造 SchedulerOutput 时。

5. DeepSeek-V3 KV cache shape 还是 [2, ...] 吗?

不是。 DeepSeek-V3 用 **MLA (Multi-head Latent Attention)**, KV 是通过低秩投影从 latent 维度算出来的, **只存一个 latent tensor 而不是 K/V 两个。**

MLA KV cache shape: `[num_blocks, block_size, kv_lora_rank + qk_rope_head_dim]` (注意没有 `[2, ...]` 的二元维度)

具体数字 (DeepSeek-V3):

- `kv_lora_rank = 512` (latent 维度)
- `qk_rope_head_dim = 64` (位置编码部分单独存)

- 单 token KV $\approx (512 + 64) \times 2 \text{ byte} = \mathbf{1152 \text{ 字节}}$

对比标准 GQA (Llama-3-70B 单 token 4 KB)：MLA 大约 **省 4×**。论文报 8×

来自 MLA 形状 + 配套的 W_dkv 拆 rope/nope 优化。

→ vLLM 用专门的 MLA 系列 attention backend (vllm/v1/attention/backends/mla.py 等)，KVCacheSpec 也是单独类 (MLAKVCacheSpec)。KV manager 通过 KVCacheSpec 抽象屏蔽形状差异——这就是 V1 解耦 KVCacheManager 的回报。

下一步

- 下一节：06-cuda-kernels.md (attention backend 之下，vLLM 自带的 paged attention CUDA kernel 是什么样的)
- 想看源码：vllm/v1/attention/backends/flash_attn.py、vllm/v1/attention/backends/mla/flashmla.py、vllm/v1/attention/selector.py
- 想动手：07-hands-on/04-profiling-and-debugging.md (切换 --attention-backend 看 forward 时间变化)
- 想从生产视角理解：08-production-deployment/04-autoscaling-and-capacity.md (不同 backend 在不同 batch size 下吞吐差异，对容量规划的影响)

06. CUDA Kernels: 从 paged_attention 到自定义算子

谁该读这一篇？ 想看一眼 csrc/ 真实 kernel、理解 CUDA 细节的工程师；以及未来打算改 / 写 vLLM 自定义算子的人。

前置阅读： 05-attention-backends.md、01-paged-attention.md、00-prerequisites.md

耗时： 约 18 分钟

学完能：

1. 在 csrc/ 里准确定位 5 个以上重要 kernel 文件并说出用途
2. 解释 paged_attention v1 的 grid/block 设计（per head $\times$ per seq 一个 CTA）
3. 描述 v2 的 split-K 解决的问题（decode 小 batch 填不满 SM）
4. 说明 attention kernel 怎么用 block_tables 间接读 KV cache
5. 说出 vLLM 现在主用 FlashAttention/FlashInfer、自家 kernel 作为 fallback 的工程取舍

当前源码复核 (b23bd73)： 自定义 PagedAttention、FlashAttention、FlashInfer、Triton/平台专用实现并非固定主备顺序；selector 只能在各自支持矩阵内选择。修改 kernel 前要先证明请求实际命中该 backend/dispatch path，并为 eager、compile、CUDA Graph 和数值回退分别留证据。

目录：csrc/。这是 vLLM 的 C++ / CUDA 部分。本节带你穿过最重要的几个文件，看 PagedAttention v1/v2 的真实 kernel 实现。CUDA 细节不是入门必需，但读过以后，很多性能问题会更容易定位。

1. csrc/ 总览

```
csrc/
├── attention/           ← PagedAttention 内核（核心！）
│   ├── attention_kernels.cuh    - 核心 kernel 模板
│   ├── attention_utils.cuh      - dot product / softmax 辅助
│   ├── paged_attention_v1.cu     - launcher v1（单 split）
│   ├── paged_attention_v2.cu     - launcher v2（split-K，长序列加速）
│   ├── merge_attn_states.cu      - v2 的归并阶段
│   └── dtype_*.cuh              - FP16 / BF16 / FP8 类型支持
├── cache_kernels.cu        ← KV reshape / copy 等辅助
├── activation_kernels.cu   ← SiLU / GeLU 等
├── layernorm_kernels.cu    ← RMSNorm / fused RMSNorm + add
├── pos_encoding_kernels.cu ← RoPE
├── moe/                    ← MoE 路由 + 分组 GEMM
├── quantization/           ← FP8 / GPTQ / AWQ / Marlin / Machete
├── custom_all_reduce.cu    ← 自定义 AllReduce（小消息 NVLink 优化）
├── cumem_allocator.cpp     ← CUDA 内存池
├── sampler.cu              ← top-k/top-p sampling
└── torch_bindings.cpp      ← pybind11/torch.ops 注册入口
```

torch_bindings.cpp 是 Python 端 torch.ops.vllm_C* 的入口；其他 .cu 文件实现具体 kernel。

2. PagedAttention v1: 单 block 多线程组

csrc/attention/paged_attention_v1.cu

2.1 launcher (line 46-)

```
template <typename T, typename CACHE_T, int BLOCK_SIZE,
          vllm::Fp8KVCacheDataType KV_DTYPE, bool IS_BLOCK_SPARSE,
          int NUM_THREADS = 128>
void paged_attention_v1_launcher(
    torch::Tensor& out, torch::Tensor& query, torch::Tensor& key_cache,
    torch::Tensor& value_cache, int num_kv_heads, float scale,
    torch::Tensor& block_tables, torch::Tensor& seq_lens, int max_seq_len,
    ...) {
    int num_seqs = query.size(0);
    int num_heads = query.size(1);
    int head_size = query.size(2);
    int max_num_blocks_per_seq = block_tables.size(1);
    int q_stride = query.stride(0);
    int kv_block_stride = key_cache.stride(0);
    int kv_head_stride = key_cache.stride(1);
    ...

    dim3 grid(num_heads, num_seqs, 1); // 每个 head × 每个 sequence 一个 CTA
    dim3 block(NUM_THREADS);          // 128 threads / CTA

    vllm::paged_attention_v1_kernel<T, CACHE_T, HEAD_SIZE, BLOCK_SIZE, ...>
        <<<grid, block, shared_mem_size, stream>>>(...);
}
```

线程组织：

- grid: (num_heads, num_seqs) —— 每个 (head, seq) 一个 CTA (thread block)
- block: 128 threads 协作处理一个 (head, seq) 的 attention
- shared memory: 存 partial logits 和归约结果

这是 decode 阶段的 layout (query 长度 = 1)。

2.2 kernel 主体 (concept)

实际 kernel 在 attention_kernels.cuh, 简化思路如下：

```

__global__ void paged_attention_v1_kernel(
    out, query, key_cache, value_cache,
    block_tables, seq_lens, ...
){
    int seq_idx = blockIdx.y;
    int head_idx = blockIdx.x;
    int seq_len = seq_lens[seq_idx];
    int num_blocks = (seq_len + BLOCK_SIZE - 1) / BLOCK_SIZE;

    // 1. 把这个 head 的 query 加载到寄存器
    T q[head_size / threads];

    // 2. 遍历所有 KV block, 算  $QK^T / \sqrt{d}$ 
    for (int b = 0; b < num_blocks; b++) {
        int phys_block = block_tables[seq_idx * max_blocks + b]; // ← 间接寻址!
        T k_block[BLOCK_SIZE][head_size_per_thread];
        // 从 key_cache 加载 phys_block 那段
        load_K(k_block, key_cache + phys_block * kv_block_stride);

        // 算这一 block 的 logits (写到 shared memory)
        for (int t = 0; t < BLOCK_SIZE; t++)
            logits[b * BLOCK_SIZE + t] = dot(q, k_block[t]) * scale;
    }

    // 3. softmax (在 shared memory 上 reduce)
    softmax(logits, seq_len);

    // 4. 第二遍遍历 block, 算 logits @ V
    T acc = 0;
    for (int b = 0; b < num_blocks; b++) {
        int phys_block = block_tables[seq_idx * max_blocks + b];
        T v_block[BLOCK_SIZE][head_size_per_thread];
        load_V(v_block, value_cache + phys_block * kv_block_stride);
        for (int t = 0; t < BLOCK_SIZE; t++)
            acc += logits[b * BLOCK_SIZE + t] * v_block[t];
    }

    // 5. 写回 out
    out[seq_idx, head_idx] = acc;
}

```

核心 trick 是 `block_tables[seq_idx * max_blocks + b]` 这一行——把 paged 寻址完全在 kernel 内部用 indirection 完成，对调用者透明。

3. PagedAttention v2: split-K 并行

长序列（几千 token）时，v1 一个 CTA 串行算所有 block，性能不好。v2 用 **split-K**：

csrc/attention/paged_attention_v2.cu

```
flowchart TB
    subgraph V1["v1 · 1 CTA 串行"]
        V1B["block 0 → block 1 → ... → block N-1"]
        V1O["out[head, seq]"]
        V1B --> V1O
    end
    subgraph V2["v2 · split-K · P 个 CTA 并行"]
        V2C0["CTA 0<br/>blocks 0..N/P<br/>partial softmax(QK) · V<br/>+ partial sum / max"]
        V2C1["CTA 1<br/>blocks N/P..2N/P<br/>partial ..."]
        V2Cn["..."]
        Merge["merge kernel<br/>合并 P 段 partial<br/>(LogSumExp 修正)"]
        V2O["out[head, seq]"]
        V2C0 --> Merge
        V2C1 --> Merge
        V2Cn --> Merge
        Merge --> V2O
    end

classDef serial fill:#fef3c7,stroke:#b45309,color:#1a1f29;
classDef par fill:#dcfce7,stroke:#15803d,color:#1a1f29;
classDef merge fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
class V1B,V1O serial;
class V2C0,V2C1,V2Cn par;
class Merge,V2O merge;
```

merge_attn_states.cu 负责"合并 partial softmax"——用 LogSumExp trick 把多段 partial softmax 拼成正确的全段 softmax。

为什么 split-K 有用？

- decode 一个请求只有 1 个 query token，一个 head 一个 CTA → CTA 数 = head 数 × seq 数
- 假设 num_heads = 32, batch = 1，只有 32 个 CTA，跑不满 H100 的 132 个 SM
- split-K 把每个 (head, seq) 拆 P 份，CTA 数 = 32 × P，能填满 SM

4. 现代实现：多 backend 并存

如本课程前面提到的，vLLM 现在主用：

- vllm-flash-attn (vllm 维护的 fork，FA2/FA3)
- flashinfer

它们常通过独立包/扩展提供，**不在同一个 csrc/attention 实现里**。自定义 PagedAttention 是否被调用取决于平台、模型、dtype、layout、功能与显式 backend；不能仅凭安装了 FlashAttention 就认定它“接管”。

但学习目的上看 v1/v2 仍很值：你能搞懂 paged 寻址、shared memory 用法、split-K reduction，这些是 attention kernel 的通用思想。

5. 其他重要 kernel

5.1 layernorm_kernels.cu

实现：

- rms_norm —— 标准 RMSNorm
- fused_add_rms_norm —— $x + \text{residual} \rightarrow \text{RMSNorm}$ ，省一次 launch
- fused_rms_norm_quant —— RMSNorm + 输出 FP8 量化

为什么 fused？因为 RMSNorm 是 memory-bound，融合后只读一次 input。

5.2 pos_encoding_kernels.cu

RoPE（旋转位置编码）的高效实现。对 Q、K 同时做 cos/sin 旋转。

5.3 activation_kernels.cu

SiLU、GeLU、SwiGLU（Llama MLP 用）。

```
__global__ void silu_and_mul_kernel(out, input) {  
    // input = [gate, up] (concat), out = silu(gate) * up  
    // Llama MLP 的核心非线性  
}
```

注意 silu_and_mul 是融合的 op：把 SwiGLU 的两个分量同时算 + 相乘 + 写出。

5.4 cache_kernels.cu

KV cache 辅助：reshape、copy（用于 swap in/out、KV connector）、scaled store。

5.5 sampler.cu

top-k / top-p sampling 的 GPU 实现。decode 的最后一步。

5.6 quantization/

各种量化 GEMM kernel：

- fp8/ —— FP8 matmul
- gptq_marlin/ —— GPTQ INT4 $W \times \text{FP16 } A$ 的 Marlin kernel（INT4 推理性能秘诀）
- awq/ —— AWQ INT4
- machete/ —— H100+ 上更快的 mixed-precision GEMM

6. 怎么把 C++ kernel 暴露给 Python？

csrc/torch_bindings.cpp 用 TORCH_LIBRARY_FRAGMENT 注册：

```
TORCH_LIBRARY_FRAGMENT(_C, ops) {
  ops.def(
    "paged_attention_v1("
    "  Tensor! out, Tensor query, Tensor key_cache, Tensor value_cache,"
    "  int num_kv_heads, float scale, Tensor block_tables, Tensor seq_lens, ..."
    ") -> ()"
  );
  ops.impl("paged_attention_v1", torch::kCUDA, &paged_attention_v1);
}
```

Python 调用：torch.ops._C.paged_attention_v1(out, query, ...)

这种方式让 op 能参与 torch.compile 的图融合。

7. 自己改 CUDA kernel 的工作流

如果要优化某个 kernel，我一般按这个顺序看：

1. **profile**：用 Nsight Compute 看当前 kernel 的瓶颈 - memory-bound? compute-bound? launch overhead?
2. **shared memory layout**：避免 bank conflict
3. **vectorize**：用 float4 / __half2 把多个元素一次读写
4. **register tile**：把热数据放寄存器，减少 shared memory 访问
5. **CUDA Graph**：在支持的 capture mode/shape 下减少 launch overhead；最终模式由配置与兼容性决定
6. **tensor core**：让 GEMM 走 WMMA / MMA 指令
7. **fusion**：减少 kernel 数（RMSNorm + 后续 op）

vLLM 的 kernel 都是按这套优化的。

8. 推荐阅读顺序

如果你想真正读 CUDA：

1. csrc/torch_bindings.cpp —— 入口
2. csrc/attention/paged_attention_v1.cu —— launcher，理解 grid/block 怎么组
3. csrc/attention/attention_kernels.cuh —— 真正的 kernel 模板（高难度）
4. csrc/layernorm_kernels.cu —— 简单点的，看 reduce 怎么写
5. csrc/activation_kernels.cu —— 最简单，热身用
6. （进阶）csrc/quantization/gptq_marlin/ —— Marlin 是 SOTA 的 INT4 GEMM

9. 工程自检问答

Q: PagedAttention 的 CUDA kernel 跟普通 attention 有什么区别？ A: 核心区别是 KV 读取走 block_tables[seq_idx * max_blocks + b] 间接寻址，物理 block 不连续。其他（softmax、reduction）一样。代价是多一层指针 chase，但 block_size=16 摊薄了开销。

Q: split-K 在 attention 里是什么? A: 把同一个 query 的 KV 序列拆成 P 段, P 个 CTA 各算部分 $\text{softmax}(QK) \cdot V$, 最后一个 merge kernel 用 LogSumExp 合并。在长 seq、小 batch 时让 CTA 数 $\times P$ 填满 SM。

Q: 你看过 FlashAttention 的 kernel 吗? A: 更实际的说法是: "我读过 paged_attention_v1, 比较通俗; FlashAttention 我读过论文但 kernel 没完整读, 因为它用了 TMA 和 WGMMMA 指令, 门槛很高。"

Q: vLLM 为什么放弃自己的 kernel 用 FlashAttention? A: 性能。FlashAttention v2/v3 由 Tri Dao 团队持续优化, 针对最新硬件 (TMA、WGMMMA、async copy) 做了极致优化; vLLM 自己写跟不上。把精力放在调度和内存管理这些高 ROI 的层。

小结

- `csrc/attention/paged_attention_v1/v2.cu` 是自定义 PagedAttention 路径; 当前还并存 FlashAttention、FlashInfer、Triton 与平台专用路径, 选择必须以最终 backend/dispatch 证据为准。
- v1 的 grid 是 (num_heads, num_seqs), 每个 (head, seq) 一个 CTA, 128 threads; 适合普通 batch。
- v2 用 split-K 把每个 (head, seq) 拆 P 段并行算, 再用 `merge_attn_states.cu` 按 LogSumExp 合并; 解决 "decode 小 batch 时 CTA 数填不满 SM"。
- 间接寻址 `block_tables[seq_idx * max_blocks + b]` 是 paged 寻址的核心, 所有 paged attention kernel 共享这个思想。
- 其他重要 kernel: layernorm (fused RMSNorm)、pos_encoding (RoPE)、activation (silu_and_mul)、quantization (Marlin/Machete)、moe (topk_softmax / align_block_size)。

自检

不用照着原文复述, 重点是把现象、机制、源码入口和取舍讲顺。

1. v1 的 grid 维度 (num_heads, num_seqs) 为什么不反过来?

`dim3 grid(num_heads, num_seqs)` $\rightarrow$ 总共 $\text{num_heads} \times \text{num_seqs}$ 个 CTA。每个 CTA 算 (head_i, seq_j) 这一对的完整 attention。

为什么 num_heads 在前?

- CUDA grid 第一维 `gridDim.x` 是 32-bit 上限大 (约 2^{31}), 第二维 `gridDim.y` 上限 65535
- num_heads 可能很大 (多头 attention, 64 头 + 多层即可达上千) $\rightarrow$ 放第一维更安全
- num_seqs 通常 < 256 (batch size 受 KV 容量限制) $\rightarrow$ 65535 足够

反过来会怎样? `dim3 grid(num_seqs, num_heads)` 行为上一致, 但如果 $\text{num_heads} > 65535$ (不太可能但理论上) 就 launch 失败。属于工程防御性设计。

补充细节: grid 形状不影响 GPU 调度 (GPU 实际是按 block id 线性派发, 二维只是程序员视角)。重要的是 **总 CTA 数 $\geq$ SM 数 $\times$ 每 SM 容量** 才能填满 GPU。

2. v2 split-K 划分粒度由哪个参数控制? P 大 vs 小权衡?

参数: `partition_size` (一般 256 或 512 token)。

$\text{num_partitions} = \lceil \text{seq_len} / \text{partition_size} \rceil = P$
 $\text{grid} = (\text{num_heads}, \text{num_seqs}, P)$

P 大（细粒度切）的优势 / 代价：

- ✓ CTA 数翻 P 倍，小 batch 下也能填满 SM
- ✗ merge kernel 工作量翻 P 倍
- ✗ 每个 partition 太短的话，attention 计算量摊到 partition 启动开销上

P 小：相反——长 seq + 大 batch 时 $P=1$ 即可（每 CTA 算完全 seq）。

自适应选择：vLLM 启发式：if $\text{num_seqs} \times \text{num_heads} < \text{SM_count}$: 增大 P 填满 GPU。代码在 `paged_attention_v2_launcher` 入口处。

→ 典型用例：单请求 decode ($\text{num_seqs}=1, \text{num_heads}=64$) + $\text{seq_len}=10\text{K}$, $P=20$ 让 64×20 个 CTA 跑满 132 SM。

3. silu_and_mul_kernel 为什么 fusion？分开写损失在哪？

公式： $y = \text{silu}(a) * b$ ，其中 a 和 b 是同形 tensor。

Fused kernel：

```
y[i] = silu(a[i]) * b[i]  // 一次 load a + load b, 一次 store y
```

分开两个 kernel：

```
// kernel 1: tmp = silu(a)
tmp[i] = silu(a[i])    // load a, store tmp
// kernel 2: y = tmp * b
y[i] = tmp[i] * b[i]   // load tmp, load b, store y
```

性能损失：

- 多一次 store + load tmp：tmp 是 $[B, L_{\text{local}}]$ 大小，FP16 下数 MB 级，进出 HBM
- 多一次 kernel launch：~5 μs overhead
- 这两个 op 都是 element-wise，**纯 memory-bound**——多走一次 HBM 就慢一倍

实测：Llama SwiGLU MLP 每层有 silu_and_mul，80 层一次 forward → fusion 省 $80 \times (\text{一次 HBM 多余读写}) \approx$ 几个 ms。decode 时占总时长 1-3%。

→ 所有 element-wise + element-wise 的相邻 op 都该 fusion。这是 PyTorch `torch.compile` 自动 fusion 的核心 use case；手写 CUDA 是它的下限。

4. torch_bindings.cpp 注册的 op 怎么被 Python 调用？

```
// csrc/torch_bindings.cpp (简化)
TORCH_LIBRARY_EXPAND(vllm_C, m) {
  m.def(
    "paged_attention_v1(Tensor! out, Tensor query, Tensor key_cache, "
    "Tensor value_cache, Tensor block_tables, ...) -> ()",
    paged_attention_v1
  );
}
```

Python 端调用：

```
import torch

out = torch.empty_like(query)
torch.ops.vllm_C.paged_attention_v1(
  out,          # 输出 (in-place 写)
  query,        # [num_tokens, num_heads, head_size]
  key_cache,    # [num_blocks, ...]
  value_cache,  # [num_blocks, ...]
  block_tables, # [num_seqs, max_num_blocks_per_seq]
  seq_lens,
  block_size,
  max_seq_len,
  alibi_slopes, # 可选
  kv_cache_dtype,
  k_scale,
  v_scale,
)
```

vLLM 通常通过 vllm/_custom_ops.py 的 wrapper 调用：

```
# vllm/_custom_ops.py
def paged_attention_v1(out, query, ...):
  torch.ops.vllm_C.paged_attention_v1(out, query, ...)
```

好处： torch.ops.* 是 PyTorch 已知 op，torch.compile 不会 graph break（fusion 跨越它）。如果直接 ctypes 调 CUDA，torch.compile 会被打断。

5. 给 paged_attention 加 sliding window 支持，从哪个 .cuh 开始？

入口：**csrc/attention/attention_generic.cuh** 或者 **csrc/attention/attention_kernels.cuh**——这里定义了 attention 的内层循环（遍历 token、算 softmax）。

修改思路：

1. 在 inner loop 加 mask：if (token_idx < seq_len - window_size) continue;——跳过滑窗外的 K/V
2. 修 reduction：sliding window 模式下不需要全 seq softmax，只需要窗口内
3. attention metadata 加 window_size 字段从 Python 传进来
4. paged_attention_v1_launcher 接收新参数并传给 kernel template

注意点：

- sliding window 与 prefix caching 不太兼容（窗口滑过的部分被覆盖）——要么单独存窗口外的 KV，要么 Mistral 那种纯 sliding window 模型直接丢掉历史
- 想看现成实现：FlashAttention 已有 window_size 参数，vLLM 的 `vllm/v1/attention/backends/flash_attn.py` 把它传下去就行——**别自己手写 CUDA**，直接复用 FlashAttention

→ 这里容易被误导：现代实现通常不动自家 CUDA kernel，而是统一用 FlashAttention paged 版本。"先复用现有"通常是正确选择。

下一步

- 下一节：07-model-architectures.md（MLA / Mamba / MoE 三类非标准架构在 vLLM 里的源码挂载点）
- 想看源码：`csrc/attention/`、`csrc/torch_bindings.cpp`、`csrc/quantization/gptq_marlin/`
- 想动手：07-hands-on/04-profiling-and-debugging.md（用 Nsight Compute 看一个 kernel 的 SM 利用率）
- 想从生产视角理解：08-production-deployment/06-reliability-and-failure-modes.md（kernel 升级 / fallback 引入的稳定性风险）

07. 模型架构特异：MLA / Mamba / MoE 在 vLLM 源码层

谁该读这一篇？ 已经掌握标准 Llama 路径，想了解 DeepSeek MLA / Mamba SSM / Mixtral MoE 这三类"非标准"模型如何挂载到 vLLM 通用机制上的工程师。

前置阅读：04-model-runner.md、05-attention-backends.md、03-kv-cache-management.md

耗时：约 22 分钟

学完能：

- 1. 解释 MLA 怎么让 KV 占用降到约 1/8，以及代价是什么
- 2. 描述 Mamba state 与 KV block 的本质差异，以及 vLLM 怎么通过 SingleType 抽象统一管理
- 3. 画出 MoE 一层 forward 的 Router → Align → Dispatch → Compute → Combine → Weighted sum 流程
- 4. 说出 EPLB 解决什么问题、控制平面如何动作
- 5. 在 vLLM 源码里定位 MLA / Mamba / MoE 各自的 backend / layer / kernel 文件

当前源码复核 (b23bd73)：模型加载先由 _ModelRegistry.inspect_model_cls/resolve_model_cls 结合 architectures、model_impl、runner/convert type 解析，必要时 fallback 到 Transformers 实现。dense/GQA/MLA/MoE/Mamba 的差异最终都必须满足 runner、KV/cache 与 parallel contracts；文件数量不是兼容性证据。

Llama / Qwen / GPT 都是"标准 transformer + GQA"。但生产里你会遇到 DeepSeek-V2/V3 的 MLA (latent KV)、Mamba / Mamba2 / Jamba 的 SSM 状态 (非 KV)、Mixtral / DeepSeek MoE 的 Top-k expert routing。这三类把 vLLM 通用机制"拉扯"得最厉害，本节把每种的源码挂载点讲清。

1. 标准注意力 vs MLA vs SSM 一图对比



2. MLA (DeepSeek-V2 / V3)

2.1 算法核心

标准 GQA: 每层每头存 K, V (high-rank、大)。MLA: 把 K、V 压到 latent_kv (low-rank、~1/10)。Attention 时临时升维:

```
# 训练 / 推理时:
c_kv = x @ W_dkv    # [N, latent_d]
W_uk, W_uv 是 layer-fixed
K = c_kv @ W_uk     # 临时升维
V = c_kv @ W_uv

# Cache 只存 c_kv (小)
```

KV 占用从 $2 \times n_{kv_h} \times head_d$ 降到 latent_d。Llama-70B 标准 ~256 byte/token, DeepSeek-V3 MLA ~30 byte/token (约 1/8)。

2.2 vLLM 源码挂载点

vllm/v1/attention/backends/mla/:

```
__init__.py
cutlass_mla.py    ← H100 CUTLASS 高性能实现
flashattn_mla.py  ← FlashAttention 的 MLA 变体
flashmla.py       ← FlashMLA (DeepSeek 开源 kernel)
flashinfer_mla.py ← FlashInfer 实现
triton_mla.py     ← Triton fallback
rocm_aiter_mla.py ← AMD ROCm
flashmla_sparse.py ← MLA + sparse attention (DSv4)
indexer.py        ← MLA 特定的 indexer
```

源码锚点: [vllm/model_executor/layers/mla.py](#) • [MultiHeadLatentAttentionWrapper](#)

主入口 (包装层): vllm/model_executor/layers/mla.py:

```
class MultiHeadLatentAttentionWrapper(PluggableLayer): # line 34
    """包装一个 MLA 层, 给上层 (attention layer base) 统一接口。"""

    def __init__(self, config, ...):
        self.mla_modules = MLAModules(...)
        # 内部: down_proj (W_dkv)、up_proj (W_uk, W_uv)、RoPE 处理

    def forward(self, hidden_states, positions, kv_cache, attn_metadata, ...):
        # 1. down_proj → c_kv
        # 2. RoPE 加位置 (注意: MLA 用 decoupled RoPE)
        # 3. 写 c_kv 到 KV cache (按 slot_mapping)
        # 4. attention: 内部完成 c_kv → K/V 升维 + softmax(QK)V
        return output
```

2.3 与 KV manager 的接缝

vllm/v1/core/kv_cache_coordinator.py 协调多 KV 类型。MLA 模型的 KV cache shape 不同:

```
# 普通 GQA: (2, n_blocks, block, n_kv_h, head_d)
# MLA:      (n_blocks, block, 1, latent_d + rope_d)
```

KVCacheManager 通过 KVCacheSpec 区分类型, 单 manager 不混合, 多类型用 coordinator 统一。

2.4 Decoupled RoPE

MLA 的 RoPE 不能简单作用于 latent，因为升维后位置信息失真。DeepSeek 的解决：把 head 拆成"含 RoPE"和"不含 RoPE"两部分，各自处理。代码体现：MLAModules 内部 W_dkv 也拆 rope / nope 两段。

3. Mamba SSM

3.1 算法核心

Mamba 是 State Space Model，**不算 attention**。每个 token：

```
y_t, state_t = SSM(x_t, state_{t-1})
```

state 大小固定（与 seq_len 无关），是个 [d_state, d_inner] 的 tensor。比 transformer attention 在长序列上**算力线性**而非二次。

3.2 vLLM 源码挂载点

```
vllm/model_executor/layers/mamba/
├── abstract.py
├── mamba_mixer.py      ← Mamba v1
├── mamba_mixer2.py     ← Mamba v2（更高效）
├── gdn_linear_attn.py  ← GDN（Gated Delta Net，Qwen3 系列用）
├── linear_attn.py      ← Linear Attention 系列
├── short_conv.py       ← Mamba 前置 1D conv
├── lamport_workspace.py ← Mamba 跨 step state 持久化
└── ops/               ← Triton kernel（selective_scan）
```

3.3 KV 之外的"状态管理"

Mamba state 不是 KV，不能用 BlockPool 管。但 vLLM V1 把它统一抽象到：

```
vllm/v1/core/single_type_kv_cache_manager.py
├── FullAttentionManager
├── SlidingWindowManager
├── MambaStateManager    ← 这里
└── ...
```

每种 manager 实现 KVCacheManager 的接口（allocate_slots / free / cache_blocks），但内部物理布局完全不同：

- Full attention：paged blocks
- Mamba：连续的 [n_running, n_layers, ssm_state] tensor，每个 slot 一行

3.4 Hybrid 模型（Jamba / Granite-MoE / Zamba）

混合 Mamba 层 + Attention 层。每层用对应的 manager。KVCacheCoordinator 协调两种 group：

```
flowchart LR
    Jamba["Jamba<br/>32 层 = 28 Mamba + 4 Attention"]
    Mamba["28 × Mamba layers"]
    Attn["4 × Attention layers"]
    MSM["MambaStateManager"]
    FAM["FullAttentionManager"]
    Coord["KVCacheCoordinator<br/>同时 allocate / free 两组 slot"]

    Jamba --> Mamba
    Jamba --> Attn
    Mamba --> MSM
    Attn --> FAM
    MSM --> Coord
    FAM --> Coord

    classDef model fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
    classDef mgr fill:#fef3c7,stroke:#b45309,color:#1a1f29;
    class Jamba,Mamba,Attn model;
    class MSM,FAM,Coord mgr;
```

4. MoE (Mixtral / DeepSeek / Qwen3-MoE)

4.1 算法回顾

每层有 N 个 FFN (experts)。router 算 [batch, N] 的概率，选 top-k expert per token，加权求和它们的输出。

4.2 vLLM 源码挂载点

```
vllm/model_executor/layers/fused_moe/
├── layer.py      ← FusedMoE 主层 (用户 model 调用入口)
├── fused_moe.py  ← 核心 grouped GEMM kernel 调度
├── fused_moe_method_base.py
├── fused_moe_modular_method.py
├── moe_align_block_size.py ← token → expert 的对齐
├── activation.py  ← MoE 内 SiLU / SwiGLU
├── experts/      ← 每种 expert 实现 (FP8, AWQ, GPTQ MoE)
├── all2all_utils.py ← EP 用的 AllToAll
├── expert_map_manager.py ← EP 下 expert ↔ rank 映射
└── deep_gemm_utils.py ← H100+ 上的 deep GEMM (小 batch 高吞吐)
```

CUDA 层: csrc/moe/, 含:

- topk_softmax: router 输出 top-k 选择
- moe_align_block_size: 把 token 按 expert 分组 (核心 perf 关键)
- fused_moe_kernel: 批量 grouped matmul

4.3 Routing → Dispatch → Compute → Combine



4.4 EPLB (Expert Parallel Load Balancer)

vllm/distributed/eplb/: 处理 expert 流量不均：

- 监控每 expert 的活跃 token 数
- 把热门 expert 复制到多 rank（rebalance）
- 配合 AllToAll 路由表更新

代码：vllm/distributed/eplb/eplb_state.py 主入口，状态在 EngineCore 维护。

4.5 与 LoRA + MoE 的组合

vllm/lora/layers/fused_moe.py 是 LoRA × MoE 的特殊版本：

- 每个 expert 可以有自己的 LoRA delta
- punica wrapper 在 expert 维度也分组

DeepSeek-V3 等大 MoE + LoRA 微调是这条路。

5. 模型实现位置

每个具体模型的实现在 vllm/model_executor/models/：

llama.py	← 经典 GQA
qwen2.py / qwen3.py	
deepseek_v2.py	← MLA + MoE
deepseek_v3.py	← MLA + MoE + MTP
mixtral.py	← 标准 attention + MoE
jamba.py	← Mamba + Attention hybrid
granite_moe.py	← Granite MoE
mamba.py / mamba2.py	← Pure Mamba
zamba2.py	← Zamba hybrid
qwen2_vl.py	← Vision + LLM

每个文件按 HuggingFace transformers 同名类组织，里面把 nn.Linear 替换为 vLLM 的 ColumnParallelLinear、attention 层用 vLLM 的 Attention、MoE 用 FusedMoE。

6. 看一个例子：DeepSeek-V3 forward 怎么编排

vllm/model_executor/models/deepseek_v3.py（简化）：

```
class DeepseekV3DecoderLayer(nn.Module):
    def __init__(self, config, layer_idx):
        self.input_layernorm = RMSNorm(...)
        self.self_attn = DeepseekV3Attention(...) # MLA
        self.post_attention_layernorm = RMSNorm(...)
        if layer_idx < config.first_k_dense_replace:
            self.mlp = DeepseekV3MLP(...) # 前几层用普通 MLP
        else:
            self.mlp = DeepseekV3MoE(...) # 后面层用 MoE

    def forward(self, hidden_states, positions, ...):
        residual = hidden_states
        hidden_states = self.input_layernorm(hidden_states)
        hidden_states = self.self_attn(positions, hidden_states, ...)
        hidden_states = residual + hidden_states

        residual = hidden_states
        hidden_states = self.post_attention_layernorm(hidden_states)
        hidden_states = self.mlp(hidden_states) # MoE 走 FusedMoE
        return residual + hidden_states
```

注意：

- self_attn 内部走 MLA backend，KV 写 latent
- mlp 在浅层 dense、深层 MoE（DeepSeek 特定配置）
- 整体 forward 与 Llama 一致，只是子层不同

V3 还有 MTP head（multi-token predict），是投机解码的 in-built 实现，见 04-optimizations/02-speculative-decoding.md。

7. 工程自检问答

Q: MLA 为什么能比 GQA 省那么多 KV？ A: 把 K、V 投影到一个 latent_dim 远小于 $n_{kv_heads} \times head_dim$ 的空间存储；attention 时按需升维。Llama-70B 每 token 320 byte → DeepSeek-V3 ~40 byte，约 1/8。代价是 attention 计算多两次 matmul（升维）。

Q: Mamba 没有 KV，怎么做长 context？ A: 它本来就是为长 context 设计的：state 大小固定，与 seq_len 无关。算量线性而非二次。代价：state 容量有限，长 context 上召回能力弱于 attention（实测略弱）。

Q: vLLM 怎么同时支持 GQA + MLA + Mamba？ A: KVCacheManager 内部有 KVCacheCoordinator，每种 kv-type 一个 SingleTypeKVCacheManager。allocate_slots 时按 layer kv_cache_group 分别 allocate。Scheduler 看到的还是统一接口（一个请求要几个 block），但底层物理布局不同。

Q: MoE 的 token dispatch 为啥需要 AllToAll？ A: token 在原 rank 上算出 router 概率，但选中的 expert 可能在另一个 rank 上（EP 模式）。AllToAll 把 token 物理搬运到 expert 所在 rank，算完再 AllToAll 回来。这两次 AllToAll 是 EP 的主要通信开销。

Q: EPLB 怎么动态做负载均衡? A: ①每 N step 统计 expert 命中频率; ②热门 expert 复制到多 rank; ③更新 expert→rank 路由表; ④Worker 重新打开 AllToAll plan。整套是 control plane 操作, 不阻塞推理 forward。

小结

- MLA 把 K/V 压到 latent, cache 只存 latent + decoupled RoPE 分量; KV/token 占用降到约 1/8, 代价是 attention 多两次升维 matmul。
- Mamba state 大小固定 (与 seq_len 无关), 不用 BlockPool, 但通过 SingleTypeKVCacheManager 与 KVCacheCoordinator 统一到 Scheduler 接口。
- MoE 一层 = router + align + (EP 时 AllToAll) + grouped GEMM + (EP 时 AllToAll) + 加权求和; moe_align_block_size 是性能关键。
- Jamba 这类 hybrid 模型同时持有两种 KV 类型, Coordinator 是协调器; EPLB 通过 expert 复制和路由表更新做控制平面级负载均衡, 不阻塞 forward。

自检

不用照着原文复述, 重点是把现象、机制、源码入口和取舍讲顺。

1. W_dkv 为什么拆 rope / nope 两段?

MLA 的 K/V 实际是从 latent 维度 c_kv 投影出来的:

```
c_kv = x · W_dkv          # [B, kv_lora_rank], 低秩 latent
K = (c_kv) · W_uk_nope    # nope 部分
K_rope = x · W_kr         # rope 部分 (直接从 x 投影, 不经 latent)
K_full = concat(K_nope, K_rope) # 完整 K
```

为什么拆?

- RoPE (旋转位置编码) 需要对 K 的某些维度做位置变换; 如果先压到 latent 再展开, **位置信息丢失** (latent 维度太低, 丢失高频)
- 解法: 把 K 拆成两段——K_nope 走 latent 压缩节省 KV cache; K_rope 维持原维度独立存, 专门承载位置信息
- 形状: K_nope 维度 = qk_nope_head_dim (128), K_rope 维度 = qk_rope_head_dim (64)

KV cache 存什么: 只存 c_kv (512 维) + K_rope (64 维), 不存完整 K/V。运行时按需展开 (W_uk / W_uv 投影回 head_dim)。这就是 MLA 比 GQA 省 4-8× KV 的原因。

源码: vllm/model_executor/layers/mla.py 的 MLAModules, 注意 W_dkv 实际是 W_dkv_nope + W_uk + W_uv + W_kr 几个矩阵的组合。

2. Jamba 一层 Mamba + 一层 Attention, KVCacheCoordinator.allocate_new_blocks 调几个 SingleType manager?

Jamba 是 hybrid 架构: 部分层是 Mamba (SSM state), 部分层是 Attention (K/V)。

KVCacheCoordinator 持有多多个 SingleTypeKVCacheManager, 每个管理一种 KV 格式:

- group_id=0: AttentionKVCacheManager → 管 attention 层的 K/V block
- group_id=1: MambaKVCacheManager → 管 Mamba 层的 SSM state

一次 allocate_new_blocks(request, num_new_tokens) 调用：

1. 遍历 self.managers (group_id 0 → 1)
2. 每个 manager 单独算它管的那种 KV 需要多少 block
3. 都成功才整体成功；任一失败回滚

group_id 与 block_table 关系：

- request 的 block_ids 是 dict[group_id, list[int]]
- block_ids[0] 是 attention block 序列，block_ids[1] 是 Mamba state block 序列
- 两组长度不同：attention 按 token 累积，Mamba 通常每层一个固定大小的 state（不按 token 增长）

源码：vllm/v1/core/kv_cache_coordinator.py、vllm/v1/core/single_type_kv_cache_manager.py。

3. moe_align_block_size 输出什么？排序/对齐为什么加速 grouped GEMM？

源码：csrc/moe/moe_align_block_size_kernels.cu

输入：每个 token 的 expert id（top-k 选出来的） $[B \times k]$ ，比如 $[3, 1, 7, 3, 1, 5, \dots]$

输出：

- sorted_token_ids：按 expert id 排序后的 token id 列表
- expert_ids：每个 group 对应哪个 expert
- num_tokens_post_padded：padding 后总 token 数（pad 到 block_size 倍数）

为什么加速：

1. **按 expert 分组**：原本 token 散落（用户角度），排序后**同一 expert 的 token 连续**
2. **Grouped GEMM 友好**：CUDA grouped gemm 接口要求"每个 group 是一段连续 batch"，排序后正好满足
3. **CTA 分配清晰**：每个 expert 一个 CTA 块，处理它那段连续 token，wave 数减少
4. **避免 atomic / 索引混乱**：未排序时每个 token 要 atomicAdd 到 expert 的输出区，排序后直接连续 write

性能影响：排序 / padding 本身只占整 MoE 时长 < 5%，但能让后续 grouped GEMM 快 2-3×。是经典的"花点 setup 时间，省更大计算时间"。

4. EPLB 把 expert 从 rank A 复制到 rank B, 路由表何时生效？正在跑的 step 会被打断吗？

不会打断正在跑的 step。EPLB 走 rolling update 模式：

T=0: 当前 router 表：expert_5 → rank A (单 owner)
T=1..N: forward 正常跑，统计 expert_5 是 hot expert
T=N+1: EPLB 决策：复制 expert_5 到 rank B
T=N+2..M: 后台异步把 W_gate_5 / W_up_5 / W_down_5 weights 从 rank A → rank B（NCCL broadcast）
T=M: 等所有 rank 同步到新 weights → 全局原子切换 router 表：expert_5 → {rank A, rank B}
T=M+1: 下一 step forward 开始用新表，dispatch 时 token 在 A/B 间负载均衡

关键点：

- weights 复制期间 router 表**未变**，forward 仍 dispatch 到 rank A——正确性不破坏
- 切换是**整步边界**进行（同步点）——不在 forward 中途切
- 切换后旧 weights 不立刻 free（要等所有还在 fly 的 token 处理完）

控制 plane vs 数据 plane：EPLB 改控制 plane（router 表），forward 仍跑数据 plane——两者解耦，所以不阻塞推理。

源码：vllm/distributed/eplb/eplb_state.py、vllm/model_executor/layers/fused_moe/expert_map_manager.py。

5. DeepSeek-V3 first_k_dense_replace 之前 dense / 之后 MoE 的分裂有副作用吗？

DeepSeek-V3 前 first_k_dense_replace = 3 层是普通 DeepseekV3MLP（dense FFN），第 4 层开始才是 DeepseekV3MoE。

对 vLLM 调度 / KV 的副作用：

维度	影响
KV cache	无影响——MLA attention 跟 dense/MoE 无关，前 3 层和后续都用同一种 KV 格式
forward 时长	前 3 层无 AllToAll，后续层有 → forward 内"延迟前段、密集后段"。可观察 NVTX profiling 时前 3 层与后续节奏不同
TP 通信	dense 层走 standard TP（AllReduce），MoE 层走 EP（AllToAll）。混在同一 forward 里没冲突，但 通信组拓扑必须前期就配好 （既支持 TP AllReduce 又支持 EP AllToAll）
EPLB	只统计 MoE 层的 expert 负载，前 3 dense 层不参与
量化	dense 层和 MoE 层可以用不同量化（如 dense BF16 + MoE FP8）——first_k_dense_replace 把它们分开就是为这个
forward 总参数读取	第一次 forward 时前 3 层权重一次读完（小，~GB 级），后续 MoE 层每步 dispatch 后只读 top-k expert 的权重（大模型但稀疏访问）

没坑：vLLM 的 forward 主要看每层的 forward() 调用，不论是 Dense 还是 MoE 都是 nn.Module——继承自同一 DecoderLayer，对调度透明。

补充细节：DeepSeek 这个设计的动机是**前几层学的是低层 lexical 特征，不需要 expert routing**（学习率不稳定）；MoE 从中层开始介入后效率更高。这是模型架构选择，不是 vLLM 限制。

下一步

- 下一章：04-optimizations/01-quantization.md（量化对 MLA / MoE 的特殊处理：FP8 KV、experts 量化）
- 想看源码：vllm/v1/attention/backends/mla/、vllm/model_executor/layers/mamba/、vllm/model_executor/layers/fused_moe/
- 想动手：07-hands-on/03-mini-experiments.md（同硬件跑 Llama vs DeepSeek-V2 看 KV 占用差异）
- 想从生产视角理解：05-distributed/01-tp-pp-ep.md（EP / AllToAll 的分布式细节）

Sources

- [vllm/v1/attention/backends/mla/*](#) ([flashmla.py](#)、[cutlass_mla.py](#)、[flashattn_mla.py](#)、[indexer.py](#) 等)

源码锚点：[vllm/model_executor/layers/mla.py](#) • [MLAModules](#)

源码锚点：[vllm/model_executor/layers/mla.py](#) • [MultiHeadLatentAttentionWrapper](#)

源码锚点：[vllm/model_executor/layers/mla.py](#) • [MultiHeadLatentAttentionWrapper](#).**init**

源码锚点：[vllm/model_executor/layers/mla.py](#) • [MultiHeadLatentAttentionWrapper](#).**forward**

- [vllm/model_executor/layers/mla.py](#) ([MLAModules](#) / [MultiHeadLatentAttentionWrapper](#))
- [vllm/model_executor/layers/mamba/{mamba_mixer,mamba_mixer2,gdn_linear_attn,short_conv}.py](#)
- [vllm/v1/core/single_type_kv_cache_manager.py](#) (多 KV 类型分支)
- [vllm/v1/core/kv_cache_coordinator.py](#)
- [vllm/model_executor/layers/fused_moe/{layer,fused_moe,modular_kernel,moe_align_block_size}.py](#)
- [vllm/distributed/eplb/eplb_state.py](#)
- [csrc/moe/](#)、[csrc/attention/](#) (MLA / MoE CUDA kernel)
- 范例模型：[vllm/model_executor/models/{deepseek_v3,mixtral,jamba,mamba2,llama}.py](#)

See also

- [02-core-concepts/03-kv-cache-management.md](#) —— 多 KV 类型协调
- [05-distributed/01-tp-pp-ep.md](#) —— EP / AllToAll
- [04-optimizations/02-speculative-decoding.md](#) —— DeepSeek-V3 MTP

08. 输入处理与 Tokenization：从 OpenAI 请求到 EngineCoreRequest

谁该读这一篇？ 能启动 API server，但不清楚 chat template、tokenizer、多模态处理和 AsyncLLM 的 CPU 工作到底落在哪里的人；正在排查高 TTFT、tokenizer mismatch 或请求取消问题的工程师。

前置阅读： 01-entry-points.md、01-overview/02-architecture.md。

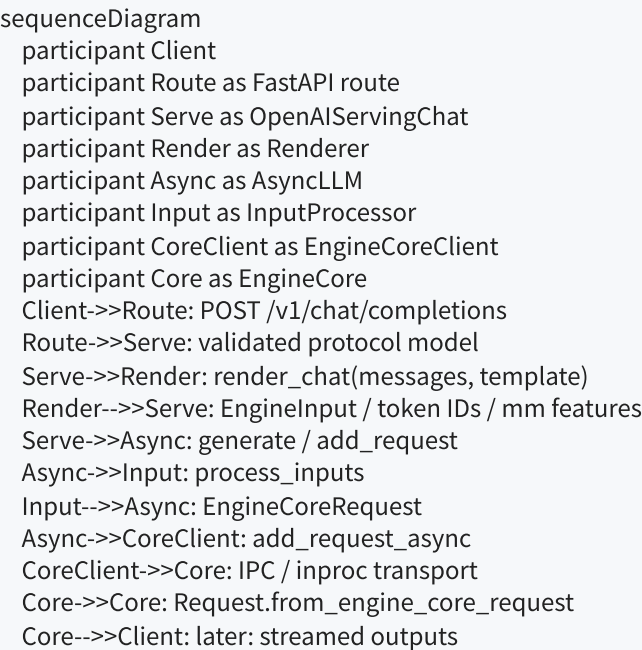
耗时： 约 25 分钟。

学完能：

- 1. 从 /v1/chat/completions 追到 EngineCoreRequest，说清每个进程边界。
- 2. 区分 chat rendering、tokenization、input preprocessing 与 EngineCore request 初始化。
- 3. 解释 prompt embeddings、LoRA、prompt adapter、多模态 feature 在哪一层改变输入或缓存键。
- 4. 用错误类型和指标判断 CPU 前处理、队列、GPU 或网络谁在拖慢 TTFT。

静态复核： 锁定 b23bd73f540175f9e117eae5029cd7d8df63964；未声称 GPU 实测。

1. 一张图先看全链路



四个容易混淆的边界：

- 1. Pydantic/protocol validation 判断 HTTP 字段是否合法；
- 2. Renderer 把 messages 与 chat template 变成模型输入；
- 3. Input preprocessing/tokenization 把 text、tokens、embeds、多模态统一成 engine input；

4. InputProcessor 加上 sampling/pooling、LoRA、priority、trace 与 DP 元数据，产出跨进程契约。

2. OpenAI 协议层：不要从 tokenizer 开始追

源码锚点：[OpenAIServingChat.create_chat_completion](#)

协议层先处理：

- 模型名、served model 与 LoRA model 选择；
- request ID、trace headers、priority 与 disconnect 检查；
- chat messages、tools、reasoning、structured output 等协议字段；
- streaming / non-streaming 两条响应路径；
- max_tokens、stop、logprobs 等到 SamplingParams 的转换。

因此 HTTP 400 不一定进入 tokenizer；模型不存在、字段冲突、prompt/prompt_embeds 互斥等错误可能在 protocol model 或 serving 层直接返回。

3. Chat template 与 tokenization

Chat completion 不是把 messages[*].content 简单拼接。Renderer 负责：

1. 选择显式 chat_template、tokenizer 自带模板或模型约定；
2. 把 role/content/tool call 转成模型实际看到的文本或结构；
3. 决定是否添加 generation prompt；
4. 对多模态 content part 建立 placeholder 与 feature 的位置关系；
5. 调 tokenizer，或保留已经是 token/embedding 的输入。

最常见的“服务能跑但答案变差”问题，是离线训练/评估与在线服务使用不同 chat template、BOS/EOS 或 tokenizer revision。排查时保存“渲染后文本 + token IDs + tokenizer/model revision”，不要只保存原始 messages。

4. InputPreprocessor：统一文本、token、embeds 和多模态

源码锚点：[InputPreprocessor.preprocess](#)

输入形态最终归一到 EngineInput 家族：

输入	关键验证	进入模型前的形态
text prompt	tokenizer、special tokens、长度	prompt_token_ids
token IDs	token 范围、长度、模型约束	保留 IDs，不重复 tokenize
prompt embeds	shape/dtype/长度与互斥字段	prompt_embeds，forward 时替换对应位置
multimodal	modality 数量、processor、placeholder 对齐	token IDs + MultiModalFeatureSpec
encoder-decoder	encoder/decoder 两侧完整性	拆成 encoder 与 decoder engine input

prompt_embeds 会绕过普通文本 tokenization，但仍需要长度、模型能力和位置约束。它不是“任意 tensor 直通 GPU”的后门。

5. InputProcessor.process_inputs(): 跨 EngineCore 的契约

源码锚点: [InputProcessor.process_inputs](#)

这里做当前 V1 的最后一轮输入侧组装：

- 校验 SamplingParams / PoolingParams 与 supported task；
- 校验 LoRA request 和 data-parallel rank；
- 拆 encoder/decoder input，并调用平台 request validation；
- 补齐 max_tokens，合并 generation config、tokenizer EOS 等规则；
- 生成 multimodal feature spec 与 cache metadata；
- 附上 arrival time、priority、trace headers、resumable/streaming 状态；
- 产出可序列化的 EngineCoreRequest。

源码已经提示：把 raw prompt 直接交给 InputProcessor 的路径在弃用，新的调用者应先使用 Renderer 的 render_cmpl() / render_chat()。写扩展时不要复制将被移除的兼容路径。

6. AsyncLLM.add_request(): 异步安全与取消

源码锚点: [AsyncLLM.add_request](#)

AsyncLLM 不是一个简单队列：

- engine 已失败时立即抛 EngineDeadError；
- 检查某些 KV sharing 与 prompt logprobs 的不兼容组合；
- streaming input 使用独立添加/更新流程；
- output collector 在请求提交前建立，避免首个输出与消费者注册竞态；
- 调用失败或客户端断开时，abort 必须同时通知 output processor 与 EngineCore。

客户端断开 ≠ GPU 立即停止。取消信号要跨 event loop、output collector、core client 到 Scheduler；已提交的 kernel 也不能在任意指令点被强制中断。生产容量模型必须允许这段取消传播延迟。

7. EngineCore 侧的最后一步

源码锚点: [EngineCore.preprocess_add_request](#)

EngineCore 将传输对象转为内部 Request，并处理只有 core 才掌握的状态：

- request ID 隔离/映射；
- multimodal receiver cache；
- block hash 生成所需状态；
- structured-output grammar 的异步初始化；
- request wave / resumable 等调度信息。

到这里才进入 Scheduler 的 waiting 队列。把“HTTP 已返回 200/stream headers”当成“请求已在 GPU running”是错误的可观测性假设。

8. CPU 成本与 backpressure

输入侧 CPU 热点通常来自：

- 大 prompt 的 tokenizer；
- chat template / tool schema / JSON 处理；
- 图像解码、resize、vision processor；
- prompt embeds 反序列化与校验；
- grammar 编译；
- 大量短请求导致 event loop、IPC 与对象分配开销占比上升。

诊断要把时间拆成：HTTP 排队 → validation/render/tokenize → EngineCore queue → scheduled → first model output → detokenize/network。只有第一段上升时，调 GPU 参数通常无效。

backpressure 原则：

1. 在进入昂贵 tokenizer/mm processor 前做 body size、并发和租户 quota；
2. 为 CPU 前处理设置有界队列，不无限创建 async task；
3. 把取消传到正在等待的前处理任务；
4. 对多模态下载设置域名 allowlist、大小、超时和解码资源限制；
5. 不在 event loop 上运行不可让出的长 CPU 工作。

9. 五类失败怎么定位

失败	常见层	证据	处置
invalid input	protocol/serving	4xx、字段级错误	修请求；不要重试放大流量
tokenizer mismatch	renderer/tokenizer	相同 messages 得到不同 IDs	固定 model/tokenizer revision 与 template
oversized prompt	processor/config	prompt + max tokens 超 model limit	前置 token budget、截断或拒绝
multimodal limit	protocol/mm processor	item 数、像素、feature token 超限	网关限额与模型配置一致
cancellation leak	async/core boundary	client 断开后 running/waiting 不下降	trace abort IDs，核对 core ack 与 collector 清理

错误分类还决定重试：输入错误不重试；EngineCore 暂时过载可带 jitter/backoff；客户端取消不应转换成服务端自动重试。

10. 无 GPU 源码 trace

```
cd /path/to/vllm
grep -n "async def create_chat_completion" vllm/entrypoints/openai/chat_completion/serving.py
grep -n "def process_inputs" vllm/v1/engine/input_processor.py
grep -n "async def add_request" vllm/v1/engine/async_llm.py
grep -n "def preprocess_add_request" vllm/v1/engine/core.py
```

练习输出一张五列表：API object、internal object、process boundary、状态变化、可观测证据。额外为 invalid model、oversized prompt 和 disconnect 各画一条提前退出路径。

11. 远端 endpoint 实验

不要求本机安装 vLLM，只要有授权的测试 endpoint：

```
export VLLM_TEST_ENDPOINT=http://127.0.0.1:8000
curl -N -sS "$VLLM_TEST_ENDPOINT/v1/chat/completions" \
-H 'Content-Type: application/json' \
-d '{
  "model": "your-served-model",
  "messages": [{"role": "user", "content": "用一句话解释 KV cache"}],
  "max_tokens": 32,
  "stream": true
}'
```

记录 DNS/connect、response headers、首个 data chunk、末 chunk 四个时间点。再分别发送非法 model、超过限制的 prompt，并在流式请求中途 Ctrl-C；用服务日志确认三条路径是否进入 EngineCore、是否触发 abort。

12. 生产检查表

- [] model、tokenizer、chat template revision 可追溯；
- [] body/token/multimodal/tenant limits 在昂贵处理前生效；
- [] CPU 前处理有界并发与 queue time 指标；
- [] request ID 贯穿网关、API、EngineCore 和输出日志；
- [] 4xx、429、5xx、disconnect 使用不同重试策略；
- [] prompt 文本/embedding 的日志与 trace 满足脱敏要求；
- [] LoRA、prompt adapter、multimodal 标识进入正确 cache 隔离键；
- [] 取消与 shutdown 有 drain 超时和泄漏监控。

13. 面试回答

30 秒版：

Chat 请求先在 OpenAI serving 层做协议校验和 chat rendering，再由 InputPreprocessor 统一 text/tokens/embeds/multimodal，InputProcessor.process_inputs 合并采样、LoRA、priority 和 trace，生成 EngineCoreRequest。AsyncLLM 通过 core client 提交；EngineCore 再建立内部 Request 和 grammar/hash 状态，最后进入 Scheduler。CPU 前处理、EngineCore queue、GPU 和输出网络必须分段观测。

3 分钟追问框架：按“协议 → Renderer → EngineInput → EngineCoreRequest → internal Request”五层展开；每层各讲数据契约、失败、取消和可观测证据，再补 tokenizer/template 一致性与多租户资源限制。

小结

- HTTP model 不是 EngineCore Request；中间有 rendering、preprocessing 与 input processing 三个语义边界。
- tokenizer/template 漂移会改变 token IDs、长度、cache hit 和最终答案。
- 多模态、embeds、LoRA 不只是 API 参数，还影响 feature、forward 输入与缓存隔离。
- TTFT 要分段；CPU preprocessing 慢时调 Scheduler/GPU 没用。

自检

1. protocol validation 与 platform request validation 分别能拦什么？
2. 为什么 raw prompt 直传 InputProcessor 是不应继续依赖的兼容路径？
3. prompt embeds 绕过了什么，又没有绕过什么？
4. client disconnect 后为何不能假设 GPU 立刻停止？
5. 你会为 tokenizer mismatch 留下哪三份证据？

参考答案

1. protocol validation 负责通用 API 形状、字段类型、互斥参数和安全边界；platform request validation 负责当前模型、平台和 backend 能否执行，例如 multimodal、pooling、LoRA 或 attention 限制。前者是“请求是否合法”，后者是“当前服务能否接”。
2. raw prompt 直传绕过了统一的 rendering/tokenizer/template 语义，容易让不同入口得到不同 token IDs、prompt length 和 cache key。它可能保留兼容性，但只能作为明确版本化的 fallback，不能成为生产默认路径。
3. prompt embeds 绕过 tokenizer 和文本 template，因为调用方已经提供 embedding；但它没有绕过长度、dtype、shape、模型 hidden size、KV cache、scheduler、batching 和安全/配额校验。embeds 也不能自动证明与目标 tokenizer 语义等价。
4. disconnect 通常只让 HTTP/streaming 层停止发送；EngineCore 可能已经把请求排入 GPU、NCCL 或 connector transfer。必须通过 abort/cancel 信号、request status、finished/aborted metrics 和 KV usage 下降确认最终释放，而不是看到 TCP 断开就认为 GPU 停了。
5. 保存同一请求的 tokenizer/model revision/template 指纹、输入 token IDs/长度 hash、以及服务端与客户端的 rendered prompt/response usage 对照。不要把原始敏感 prompt 写进日志；可保存脱敏 hash、长度和必要的 token-ID 证据。

下一步

- 02-scheduler.md：EngineCore Request 如何获得本步 token/KV 预算。
- 09-output-processing-and-streaming.md：反向追输出、detokenization、streaming 与取消。

09. 输出处理与流式返回：从 Sampler 到 SSE

谁该读这一篇？ 想解释“GPU 已经出 token，为什么客户端还没看到”、排查流式卡顿、乱码、stop 多吐字、usage 不一致或取消不生效的工程师。

前置阅读： 04-model-runner.md、08-input-processing-and-tokenization.md。

耗时： 约 25 分钟。

学完能：

1. 从 ModelRunnerOutput 追到 RequestOutput 与 OpenAI SSE chunk。
2. 区分采样、stop、incremental detokenization、usage 与协议序列化职责。
3. 分离 TTFT、ITL、scheduler/forward、detokenize 和网络 backpressure。
4. 设计取消、慢客户端与中途错误的资源回收路径。

静态复核： 锁定 b23bd73f540175f9e117eaae5029cd7d8df63964；本章没有当前 SHA 的网络/GPU 实测徽章。

1. 输出链路总览

```
sequenceDiagram
    participant R as GPUModelRunner/Sampler
    participant E as EngineCore
    participant C as EngineCoreClient
    participant P as OutputProcessor
    participant D as IncrementalDetokenizer
    participant S as OpenAI Serving
    participant N as Client/Network
    R-->>E: ModelRunnerOutput(token IDs, logprobs...)
    E-->>C: EngineCoreOutputs
    C-->>P: output batch
    P-->>D: update(new_token_ids, stop_terminated)
    D-->>P: text delta
    P-->>S: RequestOutput / finish state / usage basis
    S-->>N: SSE data chunk
    N-->>S: ACK/flow control or disconnect
    S-->>E: abort on cancellation
```

GPU 输出的是 token IDs 与附加张量，不是最终 JSON。首 token 要经过 EngineCore、IPC、output processor、detokenizer、协议序列化和网络，任一段都能扩大客户端 TTFT。

2. Sampler 输出不等于用户输出

Model runner 的 sampler 负责从 logits 选 token，并产出需要的 logprob/grammar/spec-decode 信息。随后还要决定：

- token 是接受的正式输出、被拒的 speculative token，还是 placeholder；

- EOS、stop token、max_tokens / model length 是否结束；
- stop string 是否需要等待更多字符才能确认；
- $n > 1$ 时属于哪个 choice；
- 请求是否要返回 prompt/output logprobs。

所以“每个 forward 一定返回一个可见字符”不成立：token 可能解码为空、是多字节/子词的一部分、被 stop buffer 暂存，或仅用于内部状态。

3. OutputProcessor.process_outputs()

源码锚点：[OutputProcessor.process_outputs](#)

OutputProcessor 是 EngineCore 输出到用户请求状态的主汇合点：

1. 按 request ID 找到 RequestState 和 collector；
2. 合并 token IDs、logprobs、events 与 finish 状态；
3. 让 detokenizer 增量生成 text delta；
4. 更新 prompt/output token 计数和 timing；
5. 生成 RequestOutput，推入 async collector；
6. 对 finished/aborted 请求清理状态，并把需要的 abort 反馈给 EngineCore。

RequestOutputCollector 必须处理生产者快于消费者的情况。slow client 若让 collector/HTTP send 无限堆积，会把网络问题转换成 API 进程内存问题。

4. Incremental detokenization 为什么有状态

源码锚点：[IncrementalDetokenizer.update](#)

Tokenizer 的 token $\rightarrow$ text 并非逐 token 独立函数：

- BPE/SentencePiece token 可能需要相邻上下文；
- UTF-8 字符可能跨 token 才完整；
- special token 是否跳过受请求参数影响；
- stop string 可能跨多个 delta；
- byte fallback 会让中间状态暂时不可显示。

因此 detokenizer 保存已读 IDs、prefix/offset 与 stop buffer。遇到乱码时，先确认客户端是否错误地把每个 chunk 当独立字符串解码；服务端也不能为低延迟直接跳过增量状态。

5. Stop 与 finish reason

结束条件分层：

条件	发现位置	用户可见结果
EOS / stop token ID	engine/output state	通常不输出终止 token
stop string	detokenized text	可能缓存尾部字符，确认后裁剪
max_tokens	request token count	finish reason length
max model length	scheduler/engine	length/限制相关结束
abort / disconnect	serving/output/core	无正常 finish chunk 或中止错误
engine error	core client/serving	5xx 或流中断，取决于 headers 是否已发送

ignore_eos、include_stop_str_in_output 等参数会改变细节。协议层的 finish reason 是对内部状态的映射，不应该靠客户端猜最后一个 token。

6. Logprobs 与 usage

logprobs 成本不只是 JSON 变大：runner 要保留/传输额外 top-k 信息，output processor 要对齐 token，协议层还要序列化。大 logprobs + 高并发可让 CPU 和网络成为瓶颈。

Usage 至少区分：

- prompt tokens；
- cached prompt tokens / prompt token details；
- completion tokens；
- choices n 对总输出的影响；
- speculative tokens 不应按未接受 draft 计费；
- streaming usage 是否仅在尾 chunk 返回或连续返回。

客户端必须显式请求/支持流式 usage；如果连接在尾 chunk 前中断，客户端观察值可能不完整，服务端计费应基于内部最终记录而不是“客户端收到多少 chunk”。

7. OpenAI SSE 流

源码锚点：[OpenAIServingChat.chat_completion_stream_generator](#)

流式 generator 负责把内部 delta 转成 OpenAI-compatible chunk：

- 首 chunk 的 role/choice 初始化；
- content、reasoning、tool call 等不同 delta；
- 每个 choice 的 index、logprobs 与 finish reason；
- 可选 continuous/final usage；
- 最终 [DONE]；
- exception 到 SSE error/连接中断的映射。

代理层必须关闭不合适的 response buffering，并配置足够的 idle timeout。否则 server 正常逐 token yield，客户端仍可能几秒一批收到。

8. 一条时间线拆开 TTFT/ITL

```
gantt
    title 一次流式请求的延迟分解（示意）
    dateFormat X
    axisFormat %L
    section 输入
    网关与前处理      :a, 0, 20
    EngineCore 排队   :b, 20, 55
    section 执行
    首次调度与 prefill :c, 55, 115
    sampler/output IPC :d, 115, 123
    section 输出
    detokenize/JSON   :e, 123, 128
    网络到客户端      :f, 128, 140
    后续 iteration    :g, 140, 175
```

- 客户端 TTFT：发送完成到收到首个可见 SSE delta；
- 服务端 time_to_first_token_seconds：服务端定义的请求生命周期到首 token，通常不含完整外部网络；
- inter-token latency：相邻输出事件间隔分布；
- TPOT：请求级、排除首 token 的平均值；
- scheduler/forward time：只解释引擎内部一部分；
- network gap：server yield 到 client receive 的差值，需要两端时间戳或 trace。

不要用平均 TPOT 替代 ITL p99：平均值正常时，周期性长 step、GC、代理 flush 仍会造成卡顿。

9. 取消与 backpressure

源码锚点：[AsyncLLM.abort](#)

取消的安全顺序：

1. HTTP 层发现 disconnect 或应用取消；
2. 停止继续向该 socket 写；
3. AsyncLLM.abort 清 output processor/collector，并向 EngineCore 发 abort；
4. Scheduler 从 waiting/running 移除并释放 KV/encoder 状态；
5. runner 在差量状态中清理持久 request row。

正在执行的 GPU kernel 通常运行到完成；取消收益主要从后续 step 开始。必须让 abort 幂等，因为 disconnect、timeout、shutdown 与内部错误可能同时触发。

慢客户端策略要显式：

- 有界 per-request buffer；
- send timeout 与最大积压字节；
- 超限 abort，而不是无限占 KV；
- 网关/mesh 禁止聚合 SSE；
- 记录 server-yield 和 socket-send 延迟。

10. 错误传播

在首个 response headers 前，服务可以返回结构化 4xx/5xx；headers/SSE 已开始后，HTTP status 不能重写，只能发协议允许的 error event 或关闭连接。客户端库必须把 “正常 [DONE]” “带 error event” “EOF without DONE” 区分开。

重试安全性：已经收到部分文本后自动重试会产生重复内容和重复计费。若业务需要续传，必须用 request ID、已确认 token/offset 与服务端 resumable 协议，而不是盲目重发完整请求。

11. curl 与 Python 实验

```
export VLLM_TEST_ENDPOINT=http://127.0.0.1:8000
curl -N -sS -w '\nconnect=%{time_connect} start=%{time_start}transfer} total=%{time_total}\n' \
"$VLLM_TEST_ENDPOINT/v1/chat/completions" \
-H 'Content-Type: application/json' \
-d '{"model": "your-served-model", "messages": [{"role": "user", "content": "数到 20"}], "stream": true, "stream_options": {"include_usage": true, "max_tokens": 64}'
```

Python 客户端应记录每一行到达的 monotonic timestamp，而不是只打印文本：

```
import json, time, requests

started = time.monotonic()
with requests.post(
    "http://127.0.0.1:8000/v1/chat/completions",
    json={
        "model": "your-served-model",
        "messages": [{"role": "user", "content": "数到 20"}],
        "stream": True,
        "max_tokens": 64,
    },
    stream=True,
    timeout=(5, 60),
) as response:
    response.raise_for_status()
    for line in response.iter_lines(decode_unicode=True):
        if line:
            print(f"{time.monotonic() - started:.6f}s {line}")
```

实验三次：正常消费、每个 chunk 后 sleep 1 秒模拟慢客户端、中途关闭连接。对照 running/waiting、KV usage、abort 日志和服务端/客户端 chunk 时间。

12. 生产检查表

- [] SSE proxy buffering 关闭，idle/read timeout 大于允许的最长 token gap；
- [] per-request output buffer 有界，慢消费者触发 abort；
- [] TTFT/ITL/TPOT 使用清晰定义和直方图，不混用平均值；
- [] server yield、socket send、client receive 可用 request ID 对齐；
- [] usage/finish reason 在 streaming 与 non-streaming 一致；
- [] logprobs、n、tool/reasoning delta 有容量和兼容测试；
- [] disconnect、timeout、shutdown、engine error 的 abort 幂等；
- [] 客户端识别 [DONE]、error event 与异常 EOF。

13. 面试题回答

30 秒版：

Sampler 产出 token IDs/logprobs，EngineCore 经 client 把它交给 OutputProcessor。OutputProcessor 维护请求状态、stop、usage 与增量 detokenizer，生成 RequestOutput；OpenAI serving 再映射成 SSE delta。客户端 TTFT 还包含 IPC、detokenize、JSON 和网络，ITL 也可能被 proxy buffering 或慢客户端放大。取消要从 HTTP 传播到 AsyncLLM、Scheduler 和 runner，并释放 KV。

3 分钟版：按 sampler → EngineCoreOutputs → OutputProcessor → detokenizer → protocol → network 六层展开；每层讲数据契约、延迟、错误和资源回收，最后用双端时间戳定位 backpressure。

小结

- token ID 到可见 SSE 还有完整 CPU/IPC/网络链路。
- 增量 detokenization 与 stop string 都有跨 token 状态。
- usage、logprobs、finish reason 是协议契约，不应从文本反推。
- 慢客户端必须有界并最终 abort，否则长期占用 collector、内存与 KV。

自检

1. 为什么一个 sampled token 可能不产生可见 delta？
2. TTFT 指标正常但客户端首字慢，优先查哪三段？
3. streaming usage 尾 chunk 丢失时，服务端计费应以什么为准？
4. headers 已发送后发生 EngineCore error，HTTP 层还能做什么？
5. 慢消费者为什么会反向影响 KV 容量？

参考答案

1. token 可能只是 UTF-8 多字节字符的一部分、BPE 合并尚未形成稳定文本、仍处于 stop string 的 look-behind 缓冲，或被 reasoning/tool parser 暂存。因此 sampled token 数会增加，但可见 delta 可能为空。
2. 先查客户端连接池/事件循环和 Gateway buffering，再查响应 header 到首个 SSE event 的网络路径，最后核对服务端 engine TTFT 与 first serialized byte。若 engine TTFT 正常而 client TTFT 高，优先查引擎外链路，不要先调 attention。
3. 计费应以服务端最终确认的 token usage、request finish state 和已提交输出为准；stream 尾 chunk 只是传输通知，不能成为唯一账本。若客户端断流，记录已确认 token 数并按产品契约处理未发送部分。
4. headers 已发送后不能再把 HTTP 状态码改成 5xx；只能发送协议允许的 error event、关闭 stream，并记录 request/trace ID。客户端必须把“收到部分 token 后连接关闭”视为不完整响应，而不是成功答案。
5. 慢消费者会让 response queue、collector 和请求对象长期存活，进而延迟完成信号、KV free 和 connector cleanup。若没有 bounded buffer、write timeout 和 abort 策略，少数慢连接就能把 GPU KV 容量转化为长期内存占用。

下一步

- 07-hands-on/05-serve-openai-api.md：启动服务并验证 streaming/usage/error。
- 08-production-deployment/05-slo-and-observability.md：把分段延迟做成 SLO 与仪表盘。

01. 量化 (Quantization)

谁该读这一篇？ 准备给大模型做部署调优、需要在不同硬件上选 FP8 / AWQ / GPTQ / KV-cache 量化的工程师。

前置阅读： 00-prerequisites.md、04-model-runner.md、05-attention-backends.md

耗时： 约 10 分钟

学完能：

1. 解释 FP8 / INT4 weight-only / KV 量化各自的收益场景与代价
2. 说出 AWQ 与 GPTQ 的核心算法差异
3. 解释 Marlin 为什么能让 INT4 W $\times$ FP16 A 比反量化路径快 3-4 $\times$
4. 根据硬件 (H100 / A100 / 4090 / MI300) 选出合适的量化组合并写出 vLLM 启动命令

当前源码复核 (b23bd73)： quantization 名称只是 checkpoint/config 与 kernel 路径的入口；是否支持还取决于 GPU 架构、模型层、group size、dtype、TP 切分、attention/compile 组合。硬件表是候选矩阵，不是绝对推荐；质量、吞吐、TTFT/TPOT、显存和回滚必须在同一 workload 下测量。

把权重 / 激活 / KV 从 FP16/BF16 压到 FP8/INT8/INT4，换显存、换带宽、换吞吐。代价是少量精度损失。

1. 为什么量化？

LLM 推理瓶颈：

- **Decode 阶段访存密集** (FLOPs/Bytes ~ 1)，weight 从 HBM 读到 SRAM 是主要时间
- **显存有限** (H100 80GB 只能装 40B FP16 模型)
- **KV cache 占用** 与 dtype 线性相关

量化的核心收益：

- 权重压缩：FP16 $\rightarrow$ INT4 = **4 $\times$ 显存节省 + 4 $\times$ 带宽节省**
- decode 阶段几乎线性提速
- prefill 阶段提速有限 (compute-bound)

2. 量化的三个维度

量化对象	收益	损失	典型方法
Weight only	显存 + 带宽	小	AWQ / GPTQ / Marlin
Weight + Activation	+ 计算 (INT8 matmul)	中	SmoothQuant / FP8
KV cache	KV 显存 (间接提升并发数)	小	FP8 / INT8 KV

3. 常见量化格式速览

3.1 FP8 (H100+)

- 两种格式：E4M3 (精度高)、E5M2 (动态范围大)
- 权重通常用 E4M3，梯度/激活用 E5M2 (训练)
- 推理时：weight + activation 都 FP8，matmul 直接 FP8 Tensor Core 跑
- 精度损失：约 1% (perplexity)
- **vLLM 支持**：--quantization fp8

3.2 INT8 / INT4 Weight-Only

- 只压权重，激活保 FP16
- 计算时反量化权重再 matmul
- 损失很小 (INT8 < 0.5%，INT4 1-2%)
- 关键 kernel: **Marlin** (INT4 W × FP16 A 的极速 GEMM)

3.3 AWQ (Activation-aware Weight Quantization)

- 量化前观察激活，识别"重要" channel
- 给重要 channel 加 scale 保留精度
- INT4 W 下 perplexity 比 GPTQ 略好
- 文件：vllm/model_executor/layers/quantization/awq.py、awq_marlin.py

3.4 GPTQ (Generative Pre-trained Transformer Quantization)

- 逐层做 Hessian 二阶校正量化
- INT4 W 标准方法
- 文件：vllm/model_executor/layers/quantization/gptq.py、gptq_marlin.py

3.5 BitsAndBytes (NF4 / FP4)

- QLoRA 同款，主要服务训练但 vLLM 也支持
- 不如 AWQ/GPTQ 快，主要图易用

3.6 KV Cache 量化

- --kv-cache-dtype fp8 / fp8_e4m3 / fp8_e5m2
- KV 写入时量化、读出时反量化
- 每 token KV 占用减半 → **num_blocks × 2**

4. vLLM 的量化目录

```
vllm/model_executor/layers/quantization/
├── __init__.py      - 注册表
├── fp8.py           - FP8 implementation
├── awq.py / awq_marlin.py
├── gptq.py / gptq_marlin.py
├── compressed_tensors/ - Neural Magic 的统一格式
├── kernels/         - Marlin / Machete / 自定义 GEMM
├── bitsandbytes.py
├── auto_round/
├── modelopt.py      - NVIDIA Model Optimizer
├── deepseek_v4_*    - DeepSeek 专用低比特
└── ...
```

vLLM 推荐路径：**通过 LinearMethodBase 接口让每种量化方法实现自己的 Linear 层替换。**

5. 量化的关键 trick

5.1 异常值（Outlier）处理

LLM activation 有少量极大值（outlier），naive 量化会让它们挤压剩余值。

- AWQ：激活感知，重要 channel 不量化或加大 scale
- SmoothQuant：把 activation 的难度迁移到 weight（scale + 反 scale）

5.2 Per-channel / Per-tensor / Per-group

- per-tensor：整个矩阵一个 scale（最简单，最不准）
- per-channel：每列/行一个 scale（中等）
- per-group：每 128 个元素一个 scale（INT4 标配）

5.3 Marlin Kernel 的魔法

INT4 W × FP16 A 直接做 GEMM 比"反量化再 FP16 GEMM"快 3-4×。原理：

- 用 tensor core 的 mixed precision 模式
- 反量化和乘法在寄存器层面融合
- 极致的 shared memory layout

6. 推荐配置（按硬件）

硬件	推荐量化	备注
H100/H200	FP8 (weight + activation)	原生支持，损失最小
A100	AWQ INT4 或 GPTQ INT4 (Marlin)	A100 没 FP8 硬件
L40 / RTX 4090	AWQ INT4	消费卡显存小，量化收益最大
MI300	FP8（rocm 支持）	AMD 也有 FP8 了
TPU / CPU	INT8 / BF16	走 OneDNN / XLA

7. 实操示例

在 vLLM 中使用预量化好的 AWQ 模型

```
vllm serve TheBloke/Llama-2-70B-AWQ \
  --quantization awq \
  --tensor-parallel-size 2
```

使用 FP8 模型（H100）

```
vllm serve neuralmagic/Meta-Llama-3-70B-FP8 \
  --tensor-parallel-size 2
```

在线 FP8 量化（不推荐，先用 llmcompressor 离线量化）

```
vllm serve meta-llama/Llama-3-70B \
  --quantization fp8 \
  --tensor-parallel-size 2
```

KV cache FP8（与权重量化正交）

```
vllm serve meta-llama/Llama-3-70B \
  --kv-cache-dtype fp8 \
  --tensor-parallel-size 2
```

8. 工程自检问答

Q: 量化哪里损失最大？ A: outlier（异常激活值）和 LM head（生成 logits 的最后一层，对精度敏感）。所以 vLLM 默认 LM head 不量化。

Q: FP8 比 INT8 好在哪？ A: FP8 是浮点，保留小数和指数，处理大动态范围比 INT8 强；硬件支持后计算几乎免费（H100 FP8 TFLOPS = FP16 × 2）。INT8 需要 zero-point + scale，反量化更繁琐。

Q: 为什么 KV 量化在 vLLM 里很简单？ A: KV 写入时量化、读出时反量化（在 attention kernel 内部完成），不需要全局 scaler 调整。FlashAttention 和 vLLM 自己的 paged kernel 都支持 FP8 KV。

Q: 量化后模型为什么有时反而变慢？ A: 反量化开销 > 算力节省时（小 batch、prefill 阶段 compute-bound、没用 Marlin）。要看具体 workload。

单变量实验：量化决策记录

固定 model/revision、数据集、输入/输出长度分布、并发、seed、服务参数与硬件，只改变 quantization/KV dtype。记录：质量基线（perplexity 或任务准确率）、TTFT/TPOT/吞吐、峰值显存、启动时间、backend 日志和错误率。预先写回滚线，例如“准确率下降 >1 个百分点或 p99 TPOT 退化 >10% 即回退”。未过质量门禁，不因显存收益上线。

小结

- 量化三层正交：weight-only (AWQ/GPTQ/Marlin)、weight+activation (FP8/SmoothQuant)、KV cache (FP8)。weight-only 主要省显存与带宽，KV 量化几乎翻倍 num_blocks。
- AWQ 是激活感知（保护重要 channel），GPTQ 是 Hessian 二阶校正；INT4 下 AWQ 略好，两者都用 Marlin 反量化加速。
- Marlin 把反量化和 GEMM 在寄存器层融合，配合 tensor core mixed precision，跑 INT4 W × FP16 A 比"反量化再 FP16 GEMM"快 3-4×。
- 硬件选型：H100/H200 优先 FP8；A100/L40/4090 用 AWQ-INT4 + Marlin；MI300 走 ROCm FP8。LM head 通常不量化以保精度。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. AWQ 与 GPTQ 入口 + 是否都调 Marlin?

- **AWQ**：vllm/model_executor/layers/quantization/awq.py（基础 awq dequant + gemm）+ awq_marlin.py（Marlin 加速版）
- **GPTQ**：vllm/model_executor/layers/quantization/gptq.py + gptq_marlin.py

apply 是否调 Marlin：

- 默认场景（H100/H200 + INT4）：**两者都走 Marlin**——AWQ-Marlin 和 GPTQ-Marlin 共享底层 csrc/quantization/marlin/ kernel
- 不能走 Marlin 的场景：硬件不支持（A100 之前）、checkpoint 格式不兼容、group_size 不匹配 → fallback 到原始 dequant + cublas gemm

判断方法：启动时 vLLM log 会打"Using Marlin kernel"或"Using AWQ kernel (fallback)"。生产部署要确认走了 Marlin，不然性能差 3×。

2. Llama-70B FP16/FP8/AWQ-INT4 在 80GB H100 装多少？KV 留多少 block?

配置	模型权重	可用 KV	num_blocks	理论并发
FP16	$70 \times 2 = 140$ GB	☒ 装不下单 80GB H100	—	必须 TP-2+
FP8	$70 \times 1 = 70$ GB	80-70-5(activation) = ~5 GB	~5GB / 4KB-per-token-per-layer / 80 layer $\times$ 80 GB $\approx$ 1k block	极少（10 个 short 请求左右）
AWQ-INT4	$70 \times 0.5 = 35$ GB	80-35-5 = 40 GB	~40GB / 4KB / 80 = 130k token = ~8000 block	大（>50 个并发）

实战推荐：单机单 H100 跑 70B 必须 INT4（AWQ 或 GPTQ）。FP8 也能装但 KV 太小，几乎无并发能力。多机 / 多卡 TP=2 才能上 FP16/FP8 跑 70B。

补充细节：实际还有 CUDA Graph buffer / Marlin workspace 等额外占用，大小依赖 shape 和配置。生产部署从当前默认 --gpu-memory-utilization 0.92 起测，并按 NCCL、workspace 与动态峰值留余量。

3. --kv-cache-dtype fp8 启用后 attention kernel 怎么改？

KV cache 物理存储改为 FP8（1 byte/element 而非 2）。Attention kernel 改动：

原 FP16 路径：

```

K = load_fp16(K_cache[block, offset, ...])    # 直接 load
attention_score = Q · K^T

```

FP8 路径：

```

K_fp8 = load_fp8(K_cache[block, offset, ...])  # FP8 load
K = K_fp8.to(fp16) * k_scale                  # 反量化回 FP16
attention_score = Q · K^T

```

读出来是 FP16（或 BF16），attention 内部 compute 仍是 FP16 精度——只是 HBM 存储省了。

实现细节：

- k_scale、v_scale：每个 layer 一个（可选 per-tensor），从 calibration 来
- attention kernel 模板特化：FlashAttention v3 支持 fp8 KV
- 写入端 slot_mapping：写之前先 quantize（FP16 → FP8 + scale）

精度损失：典型 < 1% perplexity 退化。Math reasoning / code 等对小数值敏感的任务可能更明显，要 ablation。

4. 量化反而比 FP16 慢的场景。

场景	慢的原因
极小 batch (< 4) + 短 seq: 算力本不富余	反量化的 dequant kernel 启动开销 ≥ 节省的 HBM 带宽收益
prefill compute-bound 阶段 (长 prompt + 大 batch)	FP16 直接 GEMM 算力是瓶颈; 反量化加了 dequant 步骤, 反而拖慢
AWQ/GPTQ 走非 Marlin fallback (如 A100 之前的卡)	dequant + cublas gemm 比 Marlin fused 慢 2-3×
激活仍 FP16: W4A16 量化	matmul 时实际算 dequant(W) · A, 仍是 FP16 算力
量化粒度极细 (per-channel 甚至 per-token)	反量化时 scale 表很大, 影响 cache 效率
多 LoRA 切换频繁: 每个 LoRA 单独反量化	LoRA 的 A/B 是 FP16, 与量化主权重交互时频繁切 dtype

→ **量化是 decode (memory-bound) 的朋友、prefill (compute-bound) 的敌人**。生产中如果 prefill 占比很高 (短 prompt + 短输出), 量化收益小甚至为负。

5. 为什么默认不量化 LM head? 换成 FP8 会怎样?

LM head: hidden_state [B, H] · W_lm_head [H, V] → logits [B, V]。V 通常 128K (vocab size), 矩阵很大但只跑**一次/请求** (decode 时每步 1×)。

不量化原因:

- 输出敏感性**: LM head 直接产 logits, logits 的微小偏差经 softmax 放大后**采样分布显著偏移**——不同于 attention/MLP 输出还会被后续层"消化"
- 精度需求**: top-k / top-p 采样依赖排序, FP8 的 1 位 mantissa 精度可能让相邻概率的 token 排序错位
- 总收益小**: LM head 只占模型权重的 ~5% (70B 模型 LM head 才 ~4 GB), 量化收益不大但风险大

量化 LM head 会怎样:

- Perplexity 通常退化 2-5% (attention/MLP 量化 < 1%)
- 容易出"小幅高频错误"——比如把 "the" 输出成 "The"、"is" → "ls"、token 边界附近偏移
- code generation / 结构化输出 (JSON / function call) 尤其敏感

例外: 极致追求显存的场景 (如 8B 模型上 LM head 占 25% 时), 可以量化 LM head 但建议用 INT8 而非 FP8/INT4, 平衡精度与显存。

参数: --quantization-lm-head 显式开启 (默认 false)。

下一步

- 下一节: 02-speculative-decoding.md (另一类正交优化: 用小模型提议、大模型验证)
- 想看源码: vllm/model_executor/layers/quantization/awq_marlin.py、vllm/model_executor/layers/quantization/fp8.py、csrc/quantization/gptq_marlin/
- 想动手: 07-hands-on/03-mini-experiments.md (同模型跑 FP16 vs AWQ vs FP8, 对比 throughput / latency / 显存占用)
- 想从生产视角理解: 08-production-deployment/04-autoscaling-and-capacity.md (量化对单卡容量与单位 token 成本的影响)

02. Speculative Decoding（投机推理）：从 EAGLE3、并行 Draft 到自适应验证

谁该读这一篇？ 想降低单请求 / 小 batch TPOT 的推理工程师；要为代码、Agent、RL rollout 选择 proposer 的平台工程师；希望沿 vLLM V1 源码追踪 proposal、verification、rejection sampling 与 KV 状态提交的贡献者。

前置阅读： 02-scheduler.md、03-kv-cache-manager.md、04-model-runner.md、01-sampling-and-logits.md

耗时： 约 40 分钟

学完能：

1. 推导标准 speculative sampling 的接受与 residual resampling 公式，并说清“无损”的边界
2. 按 proposal 成本、串并行方式和 workload 选择 n-gram、Suffix、Draft Model、EAGLE3、MTP、PARD、DFlash 或 DSpark
3. 沿 SpeculativeConfig -> Scheduler -> GPUModelRunner -> RejectionSampler -> metrics 追踪一次完整迭代
4. 解释 dynamic K、confidence-aware adaptive verification、跨词表 TLI 为什么代表当前演进方向
5. 用接受长度、位置接受率、TPOT、吞吐与显存做可复现的开关实验

版本边界（2026-08-25）： 本书的可执行源码契约锁定在 b23bd73f540175f9e117eaae5029cd7d8df63964。本文主体均可在该版本定位；最后的“上游前沿观察”另外核验到 vLLM main@5e379a361e3ea8bb82b7efd768c36f39a0cf32fd，并用独立的不可变链接标记，不能反推旧版本已支持同一能力。

投机推理不是“用小模型替换大模型”，而是把 **proposal 与 verification 解耦**：便宜的 proposer 猜一段候选，target 模型并行验证，再只提交数学上合法的前缀。它买到的是“更少的串行 decode step”，支付的是 draft 计算、额外 target query、采样与状态管理开销。

这也决定了它最适合的区域：**中低 QPS、decode 仍偏 memory-bound、接受长度足够高**。高并发下 batch_size x K 把验证推向 compute-bound，固定的 K 很容易从收益变成负担。

1. 先看结果：一次验证到底提交几个 token

设 proposer 给出 K=4 个候选：

draft: [A, B, C, D]
target: [A, B, X, ...]
commit: [A, B, R]

- A、B 通过验证；
- C 在第 3 个位置被拒绝，后面的 D 随之失效；
- R 从 target 与 draft 的 residual distribution 采样，保证当前位置仍服从 target；
- 本步提交 3 个 token，下一步从 R 继续。

如果 4 个候选全部接受，target 已经算出了下一位置的分布，因此还能提交 1 个 bonus token：

```
draft: [A, B, C, D]
target: [A, B, C, D, E]
commit: [A, B, C, D, E] # K + 1
```

所以工程上更有用的指标不是孤立的“命中率”，而是：

```
mean_acceptance_length
= 1 + accepted_draft_tokens / verification_steps
```

这里的 1 是每个 verification step 至少会提交的 target-origin token：要么是首个拒绝位置的 recovered token，要么是全接受后的 bonus token。

2. 为什么标准 speculative sampling 无损

在位置 i ，target 分布为 $p_i(x)$ ，draft 分布为 $q_i(x)$ ，draft 提议 $x_i \sim q_i$ 。标准算法以

$$a_i(x_i) = \min(1, p_i(x_i) / q_i(x_i))$$

的概率接受；一旦拒绝，就从

$$r_i(x) = \max(p_i(x) - q_i(x), 0) / Z$$

重新采样并停止验证后续候选。接受分支贡献 $\min(p_i, q_i)$ ，拒绝分支补上 $p_i - \min(p_i, q_i)$ ，两者相加仍是 p_i 。

```
flowchart LR
    P["Proposer 生成 K 个 token<br/>以及可选的 draft 概率"]
    V["Target 一次验证<br/>K 个候选 + bonus 位置"]
    A["候选是否按顺序接受?"]
    N["接受并检查下一位置"]
    R["从 residual 分布重采样<br/>截断后续候选"]
    B["全部接受<br/>提交 bonus token"]
    C["只提交合法 token 前缀"]

    P --> V --> A
    A -->|是| N --> A
    A -->|否| R --> C
    A -->|全部通过| B --> C
```

vLLM 的经典 V1 runner 在 RejectionSampler.forward 中分别取 bonus logits 与 target logits，应用 sampling constraints 后进入 rejection_sample：

[RejectionSampler.forward](#)

Model Runner V2 则把 standard / synthetic / block 三种 verification 路径收进独立 sampler。block 并不是“整块要么全收”，它改变联合验证和 residual 的计算方式：

[Model Runner V2 RejectionSampler](#)

2.1 “无损”有三个边界

- 1. 只对具有正确 **acceptance / residual** 规则的路径成立。 rejection_sample_method="synthetic" 是压测和建模工具，按人为接受率丢弃候选，不应宣称与 target 分布等价。
- 2. 数学等价不等于 **bitwise** 相同。 batch shape、kernel、浮点精度、随机数消费顺序都可能改变单次输出或 logprob；应验证分布与质量门禁，而不是只比较一条字符串。
- 3. **logits processor** 必须逐候选位置保持语义。 penalties、bad words、structured output、thinking budget 等状态不能把被拒绝 token 当成已提交历史。当前 sampler 为 draft 位置展开这些约束，而 structured output 还需要验证和回滚 grammar 状态。

3. 当前方法地图：不要只记算法名

选择 proposer 时，先问三件事：需不需要额外权重、**proposal** 是否串行、机会来自模型知识还是 **workload** 重复性。

方法	Proposal 来源	Draft 阶段	额外权重	更适合的机会	主要风险
ngram	当前请求 token 历史	CPU / Numba 查找	无	prompt 内重复、代码补全	长匹配少时接受长度短
ngram_gpu	当前请求 token 历史	GPU 查找	无	高并发、希望避免 CPU proposal	GPU 状态与同步开销也要测
suffix	prompt + 过去生成的 suffix tree	自适应树查找	无，需 Arctic Inference	Agent 循环、代码编辑、RL rollout	非重复 workload 收益有限，缓存有容量成本
draft_model	独立小 LM	通常自回归 K 步	有	通用 target、有合适小模型	draft 串行时延、显存、词表兼容
eagle / eagle3	target hidden states + 轻量 head	特征 / token 级自回归	有	已有配套 EAGLE head	checkpoint 必须匹配 target；aux hidden state 增加耦合
mtp	target 原生未来 token 模块	模型家族决定	通常随 target 自带	DeepSeek / Qwen / GLM 等原生 MTP 模型	不是任意模型都能“打开 MTP”
PARD	适配后的 parallel draft model	单次并行预测 K 个位置	有	降低串行 draft 开销	必须使用按 parallel drafting 训练的权重
dflash	target 多层 hidden state + masked query block	单次并行 block forward	有	配套 DFlash checkpoint	attention backend、query/KV 布局更复杂
dspark	并行 block backbone + 顺序/Markov 修正	混合并行与轻量顺序阶段	有或随模型提供	新一代 block drafter、可输出 confidence	checkpoint / target 强耦合，能力仍快速演进
medusa / mlp_speculator	多 head 或 MLP	依模型实现	有	兼容的历史 checkpoint	runner / 模型支持面需逐版本核验

配置层公开的方法集合、MTP 家族枚举和高级开关都集中在 SpeculativeConfig；表格只说明设计类别，**真正的支持矩阵以配置校验、model registry、runner 分支和 E2E tests 四者交集为准。**

SpeculativeConfig

3.1 EAGLE3：从顶层特征预测走向多层特征融合

EAGLE 的关键不是“再放一个小 LM”，而是复用 target hidden states。EAGLE-3 更进一步：论文把目标改为直接 token prediction，并融合 target 多层特征，避免只依赖顶层特征。vLLM 侧对应两个明确集成点：

- EagleProposer 继承统一的 model-based proposer，并声明需要 target hidden states；
- GPUModelRunner._setup_eagle3_aux_hidden_state_outputs 依据 draft config 选择 target 的辅助层输出。

EagleProposer

EAGLE3 aux hidden-state wiring

这也解释了部署约束：EAGLE3 head 不是只看 vocabulary 是否相同，还要匹配 target 架构、hidden size、被抽取层和训练约定。随便把一个 EAGLE3 checkpoint 接到“同规模”模型上，通常不是低接受率问题，而是契约错误。

3.2 MTP：训练时能力，推理时仍要验证

原生 Multi-Token Prediction 让 target checkpoint 自带未来 token 模块。vLLM 会根据 target HF config 映射出对应 MTP architecture，并把它接入与 EAGLE 共用的 proposer 框架。它省掉独立 draft model 的选择，但不意味着：

- proposal 是免费的；MTP layer 仍有 forward、KV 与显存成本；
- 所有未来 token 可以直接提交；它们仍需 target verification；
- num_speculative_tokens 可以无限加大；模型训练的 n_predict、runner 和 KV layout 都会限制深度。

当前配置代码已经枚举多个 MTP model type，并把旧的 family-specific method 归一为 method="mtp"。因此生产配置应优先写通用方法名，让 vLLM 从 target config 推导具体 architecture。

3.3 PARD、DFlash、DSpark：前沿重点是消除 draft 的 K 步串行

传统小 draft model 为了提议 K 个 token，自己也要自回归 K 次。PARD 训练模型在一次 forward 中并行预测多个未来位置，在 vLLM 中表现为：

```
{
  "method": "draft_model",
  "model": "amd/PARD-Qwen3-0.6B",
  "num_speculative_tokens": 12,
  "parallel_drafting": true
}
```

SpecDecodeBaseProposer.propose 对 parallel_drafting 走一次 forward，否则循环生成后续候选：

SpecDecodeBaseProposer.propose

DFlash 也把 masked query block 放进一次 non-causal / infill 风格的 draft forward，并从 target hidden states 预计算 context K/V。它不是简单的 draft_model + parallel_drafting=true 别名：scheduler 甚至要为其多预留一个 query slot。

DSpark 在此基础上把 block proposal 与轻量的顺序依赖修正组合起来。锁定版本已包含 method="dspark"、专用 lookahead 规则与模型实现；它的 confidence-aware verification 则属于后续上游演进，见第 9 节。

3.4 Suffix Decoding: Agent workload 让“数据本身”成为 drafter

普通 n-gram 只在当前 token 序列内找匹配。Suffix Decoding 同时维护 request-local prompt tree 和跨请求 global suffix tree，根据频次估计 continuation 概率，并逐请求动态决定 proposal 长度。它把机会从“模型更聪明”转成“工作负载高度重复”：

- 代码编辑会反复复制未改动区域；
- Agent self-reflection / tool loop 会重复模板与上下文；
- RL rollout 会从相似 prompt 生成结构近似的轨迹。

SuffixDecodingProposer.propose 会先把本轮真正提交的 token 写回 suffix cache，再查询新的候选，因此被拒绝 token 不会污染后续树状态。

[SuffixDecodingProposer.propose](#)

3.5 TLI: draft 与 target 不再必须共享 tokenizer

传统 draft model 要求 vocabulary 对齐，否则同一个 token id 在两个模型里可能代表不同字符串。use_heterogeneous_vocab=true 启用 Token-Level Intersection (TLI)：

1. 初始化时规范化两个 tokenizer 的 token 字符串；
2. 构造 draft_to_target、target_to_draft 映射与交集 mask；
3. 把 draft logits 限制在交集 token；
4. proposal 后把 draft id 翻译成 target id，再做 verification。

[VocabMapping](#)

它扩大了可选 draft 模型池，但交集小会限制 proposal 分布，也会降低接受率。锁定版本只允许 method="draft_model" + greedy draft sampling 组合；不能把 TLI 理解为任意 tokenizer、任意 sampling 都已无缝兼容。

4. 各类投机推理的机制：proposal 是怎么生成的

方法名相似，执行形状却可能完全不同。读源码时，先沿着同一组问题看每个 proposer：

1. 候选 token 来自历史匹配、独立模型，还是 target 的 hidden state？
2. K 个位置是串行生成，还是一次 block forward？
3. target 验证需要哪些 logits、KV 和额外状态？
4. 某个位置拒绝后，哪些 token 会提交，哪些状态会回退？

下面的剖面图把这四件事放在一张图里。所有方法最后都汇入同一个“按顺序验收 + residual / bonus”出口；差异主要发生在 proposal 阶段。

4.1 N-gram: 把当前序列当作检索库

ngram 不加载 draft 模型，也不计算 draft logits。它取序列末尾的 suffix，在历史 token 中寻找最长匹配，把匹配位置后面的 K 个 token 直接复制出来。CPU 实现使用 NumPy/Numba；ngram_gpu 则在 GPU 上用 unfold 展开窗口、并行比较、argmax 找到匹配位置，再用 mask 提取 continuation。



它的优势是启动快、显存增量小，尤其适合代码模板、重复 prompt、结构化字段和多轮 Agent 上下文。限制也很直接：历史里没有足够长的匹配时，proposal 会退化为空或很短；GPU 版本还要把查找和同步成本计入端到端延迟。

CPU N-gram proposer

GPU N-gram kernel

4.2 Suffix Decoding：跨请求复用 continuation

Suffix Decoding 把“历史匹配”从单个序列扩大为两棵树：request-local prompt tree 保存当前请求的 prompt 结构，global suffix tree 保存历史请求真正提交过的 suffix。每轮先写入已提交 token，再用最近 max_tree_depth 个 token 查询 continuation；节点频次估计下一 token 的概率，max_spec_factor 与 min_token_prob 共同决定本请求本轮的 draft 长度。



因此它适合代码编辑、tool loop 和 RL rollout 等重复 workload，而不是所有自然语言请求。缓存容量、跨请求污染和低重复率会直接吞掉收益；被拒绝的 token 不应写回树，否则下一轮会把错误候选当成经验。

Suffix proposer

4.3 Draft Model：小模型自回归猜，target 一次验收

独立 draft model 的第一步消费 target 的上下文 hidden state，后续步骤消费 draft 自己的 KV 和刚采样的 token。普通路径会循环 K-1 次，因此 proposal 本身是串行的；draft logits 既可用于 greedy / probabilistic draft sampling，也可参与 p/q acceptance ratio。target 与 draft 必须共享词表，除非启用 TLI。

```
flowchart TB
    classDef target fill:#ecf3eb,stroke:#2f5d3a,color:#1a1814
    classDef draft fill:#e8eff5,stroke:#2c5282,color:#1a1814
    classDef state fill:#eee8f5,stroke:#6b4488,color:#2a2723
    classDef verify fill:#f7efe2,stroke:#b85c00,color:#2a2723
    T["Target context / hidden state"]:::target --> D0["Draft step 0"]:::draft
    D0 --> X1["sample token 1"]:::draft
    X1 --> D1["Draft step 1 + draft KV"]:::state
    D1 --> X2["sample token 2"]:::draft
    X2 --> D2["继续自回归直到 K"]:::state
    D2 --> V["Target 并行验证 K+1 个位置"]:::verify
```

它适合 target 没有原生 speculative head、但能找到同 tokenizer 小模型的场景。主要成本是 draft 的 K 次 launch、额外权重和 KV；当 K 较大或 batch 较高时，串行 proposal 可能抵消验证收益。

DraftModelProposer

4.4 EAGLE / EAGLE3：复用 target 特征而不是完整小模型

EAGLE 在 feature level 做自回归；EAGLE3 直接预测 token，并融合 target 的多层 auxiliary hidden states。proposal 仍可能逐步生成，但每一步使用轻量 head，而不是再跑一遍完整 decoder。runner 必须按 checkpoint 配置抽取正确层，随后 proposer 才能把这些特征与 draft KV 组合。

```
flowchart LR
    classDef target fill:#ecf3eb,stroke:#2f5d3a,color:#1a1814
    classDef hidden fill:#eee8f5,stroke:#6b4488,color:#2a2723
    classDef draft fill:#e8eff5,stroke:#2c5282,color:#1a1814
    classDef verify fill:#f7efe2,stroke:#b85c00,color:#2a2723
    T["Target backbone"]:::target --> H1["aux layer h8"]:::hidden
    T --> H2["aux layer h16"]:::hidden
    T --> H3["aux layer h24"]:::hidden
    H1 --> F["EAGLE3 多层特征融合"]:::draft
    H2 --> F
    H3 --> F
    F --> P["draft token block"]:::draft
    P --> V["Target verification"]:::verify
```

EAGLE3 的限制不是“模型规模相同就能接”：target 架构、hidden size、抽取层、词表和训练约定都必须匹配。prefix cache 命中边界也可能要求回退一个位置，以重建 proposer 所需的输入特征。

EagleProposer

4.5 MTP：target checkpoint 自带未来 token 模块

Multi-Token Prediction（MTP）把未来位置预测头或层一起训练进 target checkpoint。vLLM 依据 SpeculativeConfig.hf_config_override 把 Hugging Face config 映射为对应 MTP architecture；proposal 能力由模型家族、MTP layer 数和 checkpoint 的 n_predict 决定，而不是由一个通用开关凭空产生。


```
flowchart LR
    classDef target fill:#ecf3eb,stroke:#2f5d3a,color:#1a1814
    classDef draft fill:#e8eff5,stroke:#2c5282,color:#1a1814
    classDef verify fill:#f7efe2,stroke:#b85c00,color:#2a2723
    T["Target backbone"]:::target --> M1["MTP layer 1<br/>预测 t+1"]:::draft
    T --> M2["MTP layer 2<br/>预测 t+2"]:::draft
    T --> M3["MTP layer n<br/>预测 t+n"]:::draft
    M1 --> V["统一 target verification"]:::verify
    M2 --> V
    M3 --> V
```

MTP 省去了独立 draft checkpoint 的选择和部分数据搬运，但 MTP layer、proposal KV、验证 query 仍有计算与显存成本。只能使用模型家族已训练并暴露的深度；生产配置优先使用 method="mtp"，并以启动时解析出的 config 为准。

SpeculativeConfig and MTP mapping

4.6 Medusa 与 MLP speculator：共享上下文的轻量 head

Medusa 在同一个 target hidden state 上挂多个 head，每个 head 直接预测一个未来位置；vLLM 的 MedusaProposer.propose 对各 head 取候选并 stack，不需要像普通 draft model 那样为每个位置运行完整 decoder。MLP speculator 也属于轻量路径：用 context vector 和 sampled token 条件化一个小 MLP，输出未来位置 logits。

```
flowchart TB
    classDef target fill:#ecf3eb,stroke:#2f5d3a,color:#1a1814
    classDef draft fill:#e8eff5,stroke:#2c5282,color:#1a1814
    classDef verify fill:#f7efe2,stroke:#b85c00,color:#2a2723
    T["共享 target hidden state"]:::target --> H1["Medusa head 1"]:::draft
    T --> H2["Medusa head 2"]:::draft
    T --> H3["MLP / Medusa head 3"]:::draft
    H1 --> V["Target verification"]:::verify
    H2 --> V
    H3 --> V
```

这类方法的关键是 checkpoint compatibility，而不是 head 数量越多越好：head 之间通常缺少完整 decoder 的自回归纠错，长 K 的后部接受率可能快速下降。它们适合已有兼容权重的模型族，不能据此承诺一定优于 EAGLE3。

MedusaProposer

4.7 PARD / parallel drafting：把 K 次小模型调用折成一次

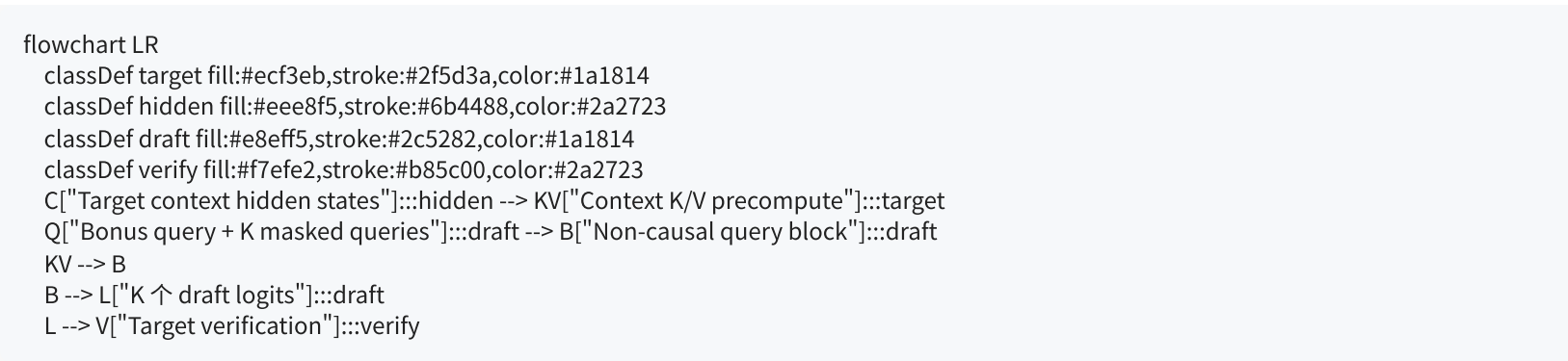
PARD checkpoint 经过多位置并行预测训练。vLLM 在 parallel_drafting=true 分支把一次 forward 的 logits reshape 成 [batch, K, vocab]，并对 sampling metadata 做 request 级 repeat-interleave；普通 autoregressive draft checkpoint 不能只靠这个开关获得相同语义。

```
flowchart LR
    classDef serial fill:#f7efe2,stroke:#b85c00,color:#2a2723
    classDef parallel fill:#e8eff5,stroke:#2c5282,color:#1a1814
    classDef verify fill:#ecf3eb,stroke:#2f5d3a,color:#1a1814
    S["普通 AR draft"]:::serial --> S1["step 1"]:::serial
    S1 --> S2["step 2"]:::serial
    S2 --> S3["..."]:::serial
    P["PARD 一次 forward"]:::parallel --> Q["K 行 logits<br/>t+1 ... t+K"]:::parallel
    S3 --> V["Target verification"]:::verify
    Q --> V
```

并行 proposal 降低了 launch 和串行等待，但一次 forward 的激活和显存峰值更高，且需要专门训练的 position semantics。应同时比较 draft latency、每位置接受率和目标 batch 下的 TPOT。

4.8 DFlash: context K/V 与 masked query block 分离

DFlash 先用 target hidden states 预计算 context K/V，再构造一个 bonus query 加 K 个 masked query。query block 内允许 non-causal attention，一次 forward 产出多个 draft positions；scheduler 为这组 infill query 额外预留 lookahead slot。

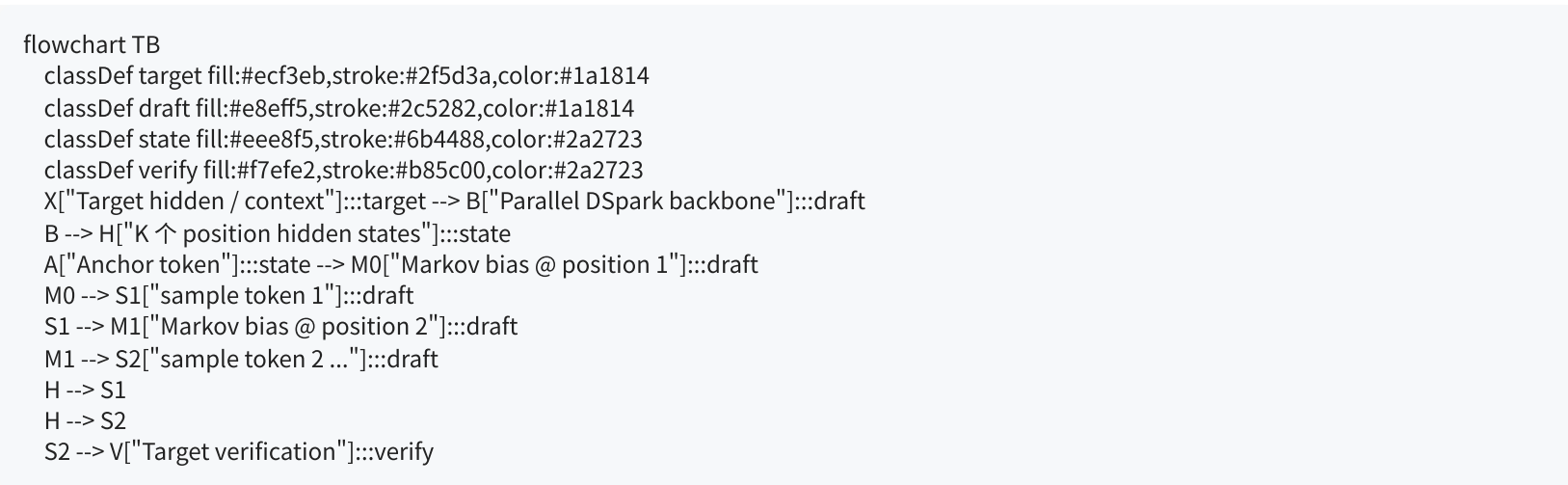


DFlash 的性能和正确性更依赖 attention backend、slot mapping、KV layout 与 CUDA Graph shape；它不能视为普通 parallel_drafting 的别名。源码中 num_query_per_req = 1 + num_speculative_tokens，正是额外 slot 的来源。

DFlash first-pass inputs

4.9 DSpark: 并行 backbone 加轻量 Markov 修正

DSpark 沿用 block proposer 的并行 backbone，但在采样阶段加入 sequential Markov head：第 j 个位置的 logits 会根据前一个已采样 token 加上 markov_embed / markov_bias，让 block 内候选保留必要的 token dependency。锁定版本已有该 proposal 路径；confidence_head 的 adaptive verification 属于后续上游版本，见第 9 节。

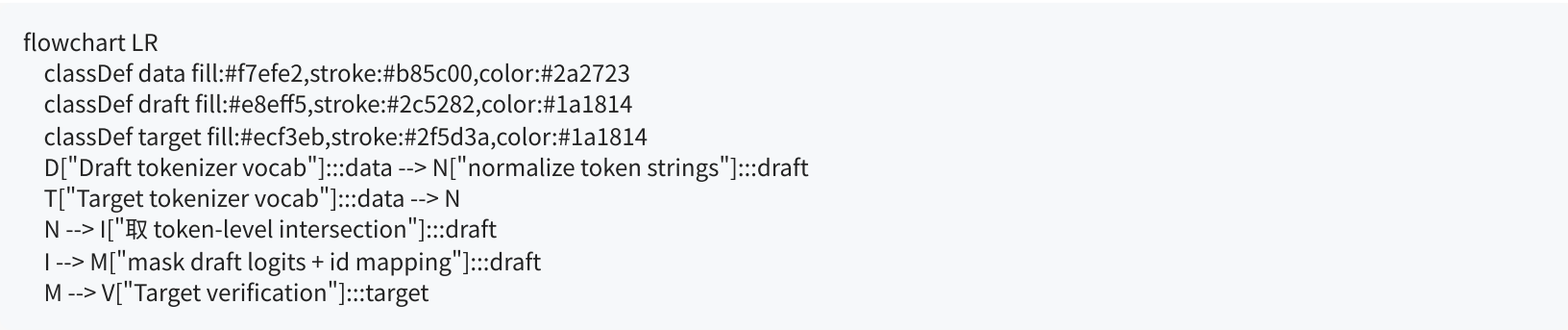


这种混合结构在 proposal 端保留并行吞吐，又用少量顺序计算修正候选依赖；代价是模型实现、target 配置和 lookahead 规则高度耦合。若使用 reduced draft vocab，还必须确认 draft-to-target mapping 覆盖目标 workload 的常见 token。

DSpark sequential sampling

4.10 TLI: 用 token 字符串交集连接异构词表

Token-Level Intersection 不是新的 proposer，而是 draft model 的词表适配层。初始化时规范化两边 tokenizer 的 token string，构造交集和双向 id 映射；采样前 mask 掉 draft 不在交集集中的 logits，采样后把 draft id 翻译成 target id 再进入 verification。



交集越小，draft 的 proposal coverage 越差，接受率可能反而下降；锁定版本还限制 TLI 为 draft_model + greedy draft 组合。它扩大了可用 draft 池，但不等于任意 tokenizer 和任意 sampling 已经无缝兼容。

VocabMapping

4.11 一张表看清机制差异

机制	候选来源	proposal 形状	额外状态	拒绝后的关键动作
N-gram / Suffix	历史 token / suffix tree	查找或动态长度	cache / tree	只写回真实提交 token
Draft Model	独立小 LM	通常 K 次串行	draft weights + KV	回退 draft KV，target residual 继续
EAGLE / EAGLE3	target hidden states	轻量 head，自回归或小 block	auxiliary layers + head KV	重建正确 feature 起点
MTP	target 原生 MTP layers	多 head / 多层	MTP layer KV	仍走统一 target verification
Medusa / MLP	共享 hidden + heads	多 head 并行	兼容 checkpoint	长尾 head 候选常被截断
PARD	并行训练的 draft LM	一次 K 位置 forward	draft weights	reshape 后逐位置验收
DFlash	target context + masked query	一次 non-causal block	context K/V + query slots	处理额外 infill slot
DSpark	block backbone + Markov head	并行 hidden + 轻量顺序采样	model-specific head	依前一 sampled token 更新 bias
TLI	draft model + vocab intersection	不改变 draft 主形状	双向词表映射	draft id 翻译为 target id

5. vLLM 一步调用链： proposal 实际上为下一轮服务

最容易误读的地方是时序。稳定状态下，runner 在本轮 target sampling 后生成下一轮 draft token，scheduler 再把这些 token 放进下一轮 verification。

```
sequenceDiagram
    participant S as Scheduler
    participant R as GPUModelRunner
    participant T as Target Model
    participant A as RejectionSampler
    participant D as Proposer

    S->>R: scheduled request + 上轮 spec_token_ids
    R->>T: real token + draft token block
    T-->>R: target logits / hidden states
    R->>A: target logits + draft tokens/probs
    A-->>R: accepted prefix + recovered/bonus token
    R->>D: 已提交 token + target hidden states
    D-->>R: 下一轮 draft_token_ids
    R-->>S: sampled tokens + next drafts
    S->>S: 回退 rejected 逻辑进度, 记录 stats
    S->>S: 保存下一轮 spec_token_ids
```

5.1 Config: 先决定方法、K 与 runner 契约

SpeculativeConfig 不只是 CLI 参数容器。__post_init__ 会：

- 推断或规范化 method；
- 为 MTP / EAGLE / DFlash / DSpark 构造 draft model config；
- 检查 draft TP、vocabulary、model family 与 speculative depth；
- 为 suffix、parallel drafting、dynamic K 等组合做 fail-fast 校验。

正确的生产习惯是查看启动日志中的最终 SpeculativeConfig(...)，不能只相信传入 JSON。

5.2 Scheduler: 预算和 KV lookahead 是 method-specific

Scheduler 初始化时按 method 计算 num_lookahead_tokens。普通 EAGLE / draft model / DSpark 使用 K，DFlash 因 infill query 额外需要一个 slot。随后 schedule() 只把本轮预算允许的 draft 前缀放入 scheduled_spec_decode_tokens。

[不同方法的 lookahead 预算](#)

[Scheduler 消费上轮 draft token](#)

5.3 Runner: target verify 与下一轮 proposal 在同一执行路径汇合

GPUModelRunner._prepare_inputs 根据每个请求实际 draft 长度展开 input、position、slot mapping 和 logits index；_sample 在有 spec metadata 时调用 rejection sampler；propose_draft_token_ids 再分派到 n-gram、suffix、Medusa、EAGLE/MTP、DFlash 或 draft model proposer。

[构造 verification metadata](#)

[proposal method dispatch](#)

5.4 Sampler: 先对每个候选位置应用约束，再做接受/拒绝

target logits 不能直接拿来验收。temperature、top-k/top-p、min tokens、penalties、bad words、allowed token mask 等会改变真正的 target distribution。vLLM 先按每个请求的 draft 长度展开 sampling metadata，再逐位置应用约束，然后才比较 draft 与

target。

这也是“投机解码 + structured output / reasoning parser”最容易出错的边界：输出一次可能返回多个 token，任何下游 parser 都不能假设每个 delta 只有一个 token。

5.5 Scheduler commit: 逻辑回滚优先于物理清零

runner 返回后，Scheduler.update_from_output 计算：

```
num_rejected = proposed_draft_tokens - accepted_draft_tokens
num_computed_tokens -= num_rejected
```

[拒绝后的逻辑进度回退](#)

这不等于“立刻擦掉 GPU 上的 KV”或“每次都释放末尾 block”。权威状态是 num_computed_tokens 与下轮 slot mapping：被拒绝位置不再属于已提交前缀，后续计算会覆盖或复用；只有满足 block 生命周期条件时才回收到 free queue。正确性来自**不再引用错误 KV**，而不是显存清零。

6. Dynamic speculative decoding: 固定 K 为什么不够

假设 batch size 为 B，每个请求验证 K 个 draft token。粗略地，target 本轮需要处理的 query 数从 B 扩到 $B \times (K+1)$ 。当 GPU 仍 memory-bound，多几个 query 可能只小幅增加时延；一旦越过 compute knee，多算的 rejected token 就是实打实的成本。

锁定版本支持按并发区间选择 K：

```
{
  "method": "eagle3",
  "model": "<matching-eagle3-checkpoint>",
  "num_speculative_tokens": 5,
  "num_speculative_tokens_per_batch_size": [
    [1, 16, 5],
    [17, 64, 3],
    [65, 128, 1],
    [129, 512, 0]
  ]
}
```

Scheduler 会把区间表编译成 lookup，并在每步按 running batch size 选择 num_spec_tokens_to_schedule：

[按 batch size 选择 K](#)

这仍是**离线调好的规则表**，不是在线自适应算法。它只观察 batch size，不知道当前请求第 4 个 token 的接受概率是否远高于另一个请求第 1 个 token。

7. 可观测性：接受率只是第一层

vLLM 的 Prometheus counter 可以计算三类指标：

```
# draft token 接受率
rate(vllm:spec_decode_num_accepted_tokens_total[5m])
/
rate(vllm:spec_decode_num_draft_tokens_total[5m])

# 平均接受长度，包含每步 1 个 recovered / bonus token
1 + (
  rate(vllm:spec_decode_num_accepted_tokens_total[5m])
  /
  rate(vllm:spec_decode_num_drafts_total[5m])
)
```

位置 j 的接受率更能指导 K：如果前 2 个位置稳定、后 3 个位置快速衰减，继续验证长尾只是在消耗 compute。

SpecDecodingProm 指标定义与 PromQL

注意 Prometheus Python client 会给 counter 暴露 _total 后缀；源码中的构造名可能没有 _total，查询服务的 /metrics 输出才是最终证据。

7.1 必须一起看的端到端指标

维度	至少记录	只看单项会错在哪里
Proposal	draft latency、draft tokens/s、额外显存	接受率高但 drafter 太慢仍会亏
Verification	mean acceptance length、每位置接受率	总接受率掩盖长 K 尾部浪费
用户时延	TTFT、TPOT / ITL、P50/P95/P99	平均吞吐可能掩盖长尾回归
容量	output tokens/s、request goodput、最大稳定并发	bs=1 的漂亮结果不能外推到线上
正确性	greedy 对照、分布 / 质量 eval、structured-output 成功率	“服务没报错”不等于无损路径正确
资源	GPU memory、KV 使用、CPU、H2D/D2H sync	n-gram / GPU proposer 也并非绝对零成本

8. 一套可执行的选择与调优流程

8.1 先选 proposer 家族

1. **模型原生带 MTP**：先试 method="mtp"，从 K=1 开始，不要超过 checkpoint 暴露的能力。
2. **有严格匹配的 EAGLE3 / DFlash / DSpark 权重**：优先测 model-aware proposer；checkpoint 契约比算法名更重要。
3. **代码编辑、Agent loop、RL rollout 高重复**：先测 suffix，再以 n-gram 做最低复杂度基线。
4. **只有通用小模型**：用 draft_model；词表不同且版本支持时再试 TLI。
5. **draft 自回归时延占比高**：寻找匹配的 PARD / parallel drafting 权重；不能仅靠开关把普通 AR checkpoint 变成并行 drafter。

8.2 单变量矩阵

固定 target checkpoint、dtype / quantization、TP、attention backend、KV cache、sampling params、prompt/output 分布和请求到达轨迹，只改变 speculative config：

实验	K	并发	目的
baseline	0	1 / 8 / 32 / 128	找 target 的 memory/compute knee
shallow	1-2	同上	验证 proposer 是否有净收益
medium	3-5	同上	找平均接受长度与 verify 成本交点
long	8+	仅合适方法	检查 suffix / parallel drafter 的长 proposal 机会
dynamic	schedule	全轨迹	验证跨负载区间是否比固定 K 稳定

每组至少包含 warmup、稳态窗口和多次重复。论文速度数字只能做假设，不能作为你的验收阈值。

8.3 回滚门禁

满足任意一项就回退到 baseline 或更小 K：

- P95/P99 TPOT 超过 SLO；
- request goodput 或 output tokens/s 在目标到达率下降；
- OOM、KV pressure、CPU sync 或 scheduler overhead 明显上升；
- greedy 对照、质量 eval、structured output / tool calling 出现正确性差异；
- 位置接受率显示后半段长期接近零；
- 新模型 / 新 vLLM 版本没有重新跑完整矩阵。

9. 上游前沿观察：从 dynamic K 到 confidence-aware verification

截至 2026-08-25，上游 main@5e379a3 在锁定版本之后新增了两项值得单独追踪的能力。它们是**候选版本事实**，不是本书锁定版本的运行承诺。

9.1 DSpark adaptive verification

固定 K 和 batch-size schedule 都给同一 batch 的请求相同上限。上游 adaptive verification 改为：

1. DSpark confidence head 给每个请求、每个位置估计 acceptance confidence；
2. 将截至位置 j 的 confidence 连乘，得到 draft slot 的 survival probability；
3. 所有请求的 slot 在全局 verification budget 下竞争；
4. 启动时 profile 不同 shape 的 step cost，选择预期 accepted tokens/s 更高的预算。

于是“高置信请求的第 5 个位置”可以排在“低置信请求的第 1 个位置”之前。这比按 batch size 统一改 K 更细，但当前只支持带 confidence head 的 DSpark，并要求 full CUDA Graph；LoRA、PP、output logprobs 等组合仍有限制。

- Adaptive Verification 官方说明
- [DSpark confidence-scheduled verification 提交](#)

9.2 Per-request acceptance metrics

锁定版本提供服务级 Prometheus 聚合指标；上游又给 OpenAI API response 增加实验性的 metrics.speculative_decoding，可返回单请求的：

- mean_acceptance_length；
- draft_acceptance_rate；
- acceptance_histogram；
- num_spec_steps / num_draft_tokens / num_accepted_draft_tokens；
- detailed 模式下每步 drafted / accepted 数组。

这让“哪类请求适合哪种 proposer / K”从离线猜测变成按 route、tenant、prompt 类型分析的可能。该 schema 明确是 experimental，依赖它的客户端必须 pin vLLM 版本。

- Per-request Acceptance Metrics 官方说明
- [OpenAI response acceptance stats 提交](#)

这两项合在一起指向同一趋势：投机推理正在从“部署时选一个 K”演进为 **按请求估计价值、按实时成本分配 verification compute、再用细粒度 telemetry 闭环**。

10. 常见误区与源码判断题

误区 1：接受率 80% 就一定加速

错误。还需要知道 draft latency、K、target verify shape、batch size、每位置 survival rate 和系统同步开销。应直接比较目标负载下的 TPOT 与 goodput。

误区 2：MTP 是“零开销”

错误。它省掉独立 draft checkpoint 的选择与部分数据搬运，但 MTP layer、proposal KV、verification query 都有成本。

误区 3：prefix caching 与 speculative decoding 完全正交

概念上分别优化 prefill 与 decode，但实现上共享 KV block、slot mapping、prefix hit 边界和 lookahead reservation。EAGLE / MTP 还可能要求 prefix hit 回退一个位置来重建 drafter 输入，组合正确性必须测试。

误区 4：拒绝后必须把 KV 显存清零

错误。必须回退的是逻辑已计算长度和后续引用；显存内容可以留着，之后覆盖即可。清零不是正确性的必要条件。

误区 5：parallel_drafting=true 能加速任意 draft model

错误。checkpoint 必须按多位置并行预测训练；否则张量形状即使能跑，proposal 语义也不成立。

误区 6：synthetic rejection 也无损

错误。synthetic mode 使用人为 acceptance profile，适合隔离 verification 性能与做容量研究，不应作为等价采样路径上线。

11. 自检答案

1. 为什么 mean acceptance length 比 draft acceptance rate 更接近 decode 加速机会？

因为它直接表示每次昂贵 verification 平均提交多少 token，并包含 recovered / bonus token。draft acceptance rate 只统计候选命中比例，不包含每步固定提交的 1 个 token，也不能单独表达不同 K 的收益。

2. 为什么大 batch 下 K 应下降？

验证 token 数近似随 $B \times (K+1)$ 增长。小 batch memory-bound 时额外 query 能利用闲置 compute；大 batch 跨过 compute knee 后，rejected query 会线性吃掉算力，降低 TPOT 和 goodput。

3. EAGLE3 与普通 draft model 在源码输入上最关键的差异是什么？

普通 draft model 主要消费 token / position / 自己的 KV；EAGLE3 proposer 还消费 target 的多层 auxiliary hidden states。runner 必须按 checkpoint config 抽取正确 target layers。

4. rejected draft 的 KV 如何处理？

Scheduler 按拒绝数回退 num_computed_tokens，下轮 slot mapping 不再把 rejected 位置视为有效前缀。物理 block 可以继续保留并被覆盖，也可能在 block 生命周期允许时回收；不要求清零。

5. Dynamic SD 与 adaptive verification 的本质区别是什么？

Dynamic SD 用离线配置的 batch size $\rightarrow$ K 区间表，batch 内请求共享 K。Adaptive verification 使用每请求、每位置 confidence 和 profile 得到的 cost model，在全局预算内选择更有价值的 draft slots。

12. 一手资料

1. [Leviathan et al., Fast Inference from Transformers via Speculative Decoding](#) —— 标准 speculative decoding 与等价分布证明。
2. [Li et al., EAGLE](#) —— target feature-level autoregression。
3. [Li et al., EAGLE-2](#) —— context-aware dynamic draft tree。
4. [Li et al., EAGLE-3](#) —— 直接 token prediction 与多层 feature fusion。
5. [Oliaro et al., SuffixDecoding](#) —— 面向 Agent / 重复 workload 的 suffix-tree proposal。
6. [An et al., PARD](#) —— target-family 可复用、单 forward 并行 draft。
7. [vLLM Speculative Decoding 官方文档（锁定版本）](#) —— 配置示例与支持边界。
8. [vLLM speculators](#) —— draft model 数据生成、训练与 checkpoint 格式。

小结

- 标准 speculative sampling 通过 $\min(1, p/q)$ 接受和 $(p-q)_+ + \text{residual resampling}$ 保持 target 分布；synthetic mode 不属于这一保证。

- 当前方法的主线不是单纯追求更大 K，而是降低 proposal 串行成本、利用 target hidden states、利用 workload 重复性，并让 verification budget 随负载与置信度变化。
- vLLM 的关键时序是“本轮 verify + sample 后生成下一轮 proposal”；Scheduler 管 token / KV budget，Runner 展开 query，Sampler 决定合法提交前缀。
- 生产决策必须同时看 mean acceptance length、位置接受率、draft / verify 时延、TPOT、goodput、显存和正确性。
- 上游 adaptive verification 与 per-request telemetry 表明下一阶段会从静态 K 转向按请求、按成本闭环调度。

下一步

- 03-cudagraph-and-compile.md —— 理解 dynamic K、不同 query shape 与 CUDA Graph 的组合成本。
- 03-mini-experiments.md —— 对同一 workload 做开 / 关 speculative decoding 的单变量实验。
- 05-slo-and-observability.md —— 把接受率、TPOT 与 goodput 接进生产门禁。

03. CUDA Graph 与 torch.compile

谁该读这一篇？ 启动慢得离谱、改了模型出现奇怪错误、想自己改 attention kernel、或者想搞清"默认开 / 什么时候要关 / 怎么诊断"的读者。

前置阅读： 02-architecture.md（明白 model runner 在哪一层）。如果想看更深的编译器细节，本节读完接 04-compilation-internals.md。

耗时： 约 12 分钟。

学完能：

1. 解释 CUDA Graph 和 torch.compile 各自消除的是哪类 overhead，能不能叠加。
2. 判断"我这个改动要不要 --enforce-eager"——给出 3 个该关的典型场景。
3. 在启动失败 / 运行结果异常时，按降级路径定位到根因。
4. 配 compile cache 让重启避免重复编译。

当前源码复核 (b23bd73)： 当前 CompilationConfig 使用 cudagraph_mode、capture sizes、splitting ops 与 capability resolution，最终可能是 NONE、PIECEWISE、FULL、FULL_DECODE_ONLY 或组合模式；不能只用“默认开/关”描述。--enforce-eager 是诊断基线，不代表修复根因。

1. CUDA Graph：消除 kernel launch overhead

1.1 问题

一个 Transformer 层有几十个 CUDA kernel（matmul、softmax、layernorm、activation、attention...）。每个 kernel launch overhead 约 5-10 μ s。一个 80 层 Llama-70B 一次 forward 可能 launch 1000+ 个 kernel，总开销 5-10 ms。

decode 阶段 forward 本身可能只 20 ms，launch overhead 占比 25-50%。

1.2 解决

CUDA Graph：把一系列 kernel 调用录制成静态图（DAG），runtime 一次性提交给 GPU 调度器，省去 launch overhead。

```
# 录制阶段（一次性）
g = cuda.CUDAGraph()
with cuda.graph(g):
    model(dummy_input)

# 运行阶段（每步）
dummy_input.copy_(real_input) # 输入复制到固定 tensor
g.replay()                    # 一次性提交
```

1.3 限制

- 输入 tensor 地址、shape 必须固定
- kernel 序列必须确定（不能有依赖运行时数据的 if）
- 显存分配不能在 graph 内

1.4 vLLM 的做法

- 启动时 capture 多个 batch_size（如 1, 2, 4, 8, ..., max）的 graph
- runtime 把当前 batch pad 到最近的 captured size（如 batch=5 用 size=8 的 graph，多出来的 3 行不算）
- InputBatch 用持久化 buffer + 索引切片实现"固定地址"

参数：

- --enforce-eager：禁用 CUDA Graph（启动快但跑慢）
- --cudagraph-capture-sizes：自定义要 capture 的尺寸
- 启动日志可见 "Capturing CUDA Graphs..."

代码：vllm/v1/cudagraph_dispatcher.py、vllm/v1/worker/gpu_model_runner.py 内 capture_model()。

2. torch.compile：图编译

2.1 问题

PyTorch eager mode 每个 op 是独立 Python 调用。即使 GPU 计算被 stream 化，Python 端的调度还是有可观开销，且 op 之间无法融合（一个 RMSNorm + 一个 Linear 本可融合，但 eager 不会）。

2.2 解决

torch.compile 用 Inductor 后端：

1. 用 Dynamo 把 PyTorch 代码 trace 成 FX graph
2. Inductor 把 graph 转成 Triton kernel（融合多个 op 到一个 GPU kernel）
3. 缓存编译结果

效果：

- 减少 Python overhead
- op fusion（RMSNorm + linear、SiLU + multiply 等）
- 比 eager 快 10-30%

2.3 vLLM 的集成

vLLM 编译范围可配：

- 完整模型 compile（最激进）
- 只 compile 每个 layer（默认）
- 只 compile 选定 op（保守）

自定义 op（如 paged attention）通过 torch.library 注册，让 compile 能识别。

代码：vllm/compilation/

参数：

- --compilation-config：JSON 字符串配置
- --enforce-eager：完全关闭 compile + CUDA Graph

2.4 与 CUDA Graph 的关系

两者可以叠加：

- torch.compile 先把 op 融合成更少更大的 kernel
- 再 CUDA Graph capture 把这些 kernel 录制成静态图
- 双重收益

3. 启动慢的代价

启动时 vLLM 要做：

1. 加载权重（几秒到几分钟）
2. Profile run（测显存，~10 秒）
3. torch.compile（每个 size 一次，几十秒到几分钟）
4. CUDA Graph capture（几秒）

总共可能 1-5 分钟。Production 服务部署时这是个问题。

加速方法：

- 使用 --enforce-eager 跳过 CG + compile（debug 用）
- 使用 --compilation-config={"level": 0} 关 compile（保留 CG）
- 缓存 compile 结果到磁盘（VLLM_TORCH_COMPILE_CACHE_DIR）
- 热 swap 模型时复用 compile cache

4. 调试 CUDA Graph 问题

CUDA Graph 出错通常表现为：

- 启动时 "RuntimeError during graph capture"
- 运行结果错误但不报错（最难查）
- OOM during capture

排查：

1. --enforce-eager 先确认是不是 CG 问题

- 2. 看 vllm 的 capture_model 日志
- 3. 检查是否 add 了不支持 CG 的 op（dynamic shape、host-device sync 等）
- 4. 减小 capture batch size 范围

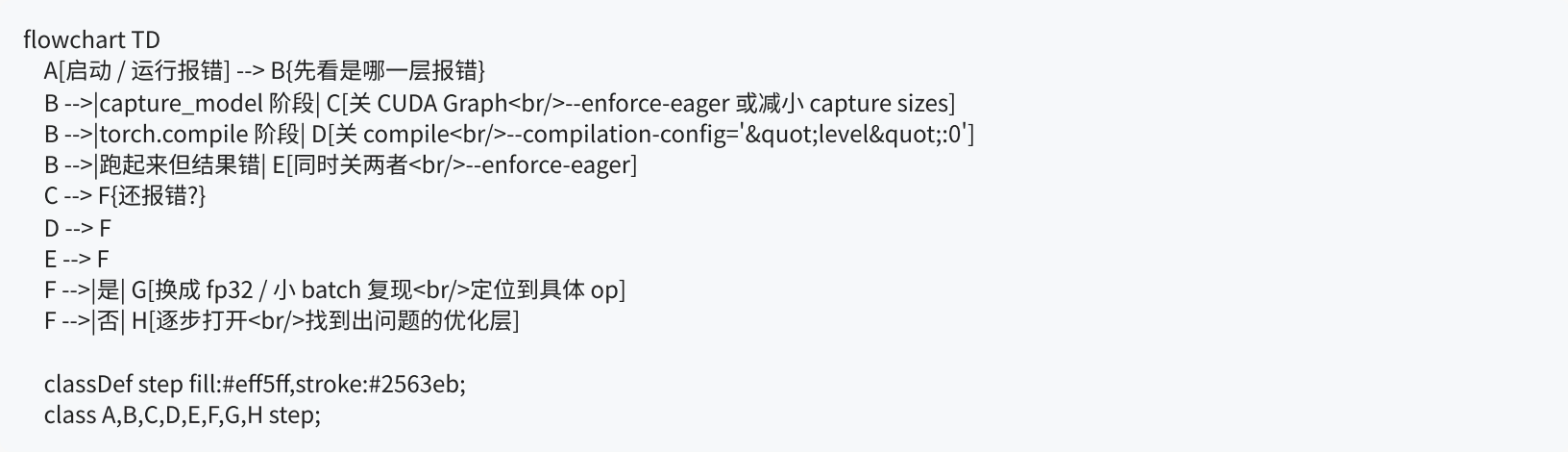
4.5 什么时候不要开（要 --enforce-eager）

CUDA Graph + compile 不是免费午餐，下面这些场景应该考虑关闭：

场景	为什么关	替代方案
调试模型正确性	CG 录制的 op 序列固定，不能加 print/breakpoint；compile 后 trace 难读	--enforce-eager，调好再开
新模型架构刚上线 vLLM	torch.compile 对新 op 容易 trace 失败 / 数值不一致	先 --enforce-eager 跑通正确性
严重 OOM 边缘	capture 多档 batch + compile 缓存都占显存	--enforce-eager 释放几百 MB
启动延迟敏感（serverless 冷启动）	compile 几十秒到几分钟	--enforce-eager + --compilation-config={'level':0}
动态 shape 非常稀疏	每来一个新 shape 重 trace 一次，compile 比省的多	关 compile，保留 CG
自定义 op 未注册	compile 看到不认识的 op 会 fallback 到 eager 区段，可能没收益还更慢	注册 op 或 --enforce-eager

4.6 失败降级路径

按这个顺序逐级关，直到能跑：



判断技巧：

- 错误信息里有 cudaGraphExec* / graphExec* → CG 问题
- 错误里有 inductor / dynamo / FX trace 栈 → compile 问题
- 数值无声错位（perplexity 异常但不报错）→ 优先怀疑 compile 数值精度，先 --enforce-eager 复现

5. 工程自检问答

Q: CUDA Graph 为什么不能完全替代 compile? A: CG 只优化 launch overhead, 不优化 op 本身。compile 做 op fusion 是另一维度。两者叠加最佳。

Q: 为什么 capture 多个 batch_size 而不是只 capture 一个? A: 实际请求的 batch 在变 (continuous batching)。pad 到固定 size 浪费算力。所以 capture 多档 (如 1,2,4,8,16,32,...,max), runtime 选最近的。

Q: prefill 阶段也用 CUDA Graph 吗? A: prefill 的输入长度 variable 且大, 不适合 CG。vLLM 主要在 decode (input 长度都是 1, 可固定) 用 CG。chunked prefill 的 chunk size 也固定, 可以 capture。

Q: torch.compile 第一次很慢, 怎么办? A: warmup (启动后跑几个 dummy 请求触发编译)。开 VLLM_TORCH_COMPILE_CACHE_DIR 让下次直接读缓存。

单变量实验: eager → compile → graph

用同一 workload 依次跑: --enforce-eager 正确性基线、compile-only 候选、解析后的 graph mode 候选。每组区分 cold start、首次请求 warmup、steady state, 记录 compile/cache hit 日志、启动时长、显存、TTFT/TPOT/吞吐和输出一致性。缓存实验必须换新 cache dir 测 cold, 再复用同一目录测 warm; 出现 silent accuracy drift 立即回 eager 定位。

小结

- CUDA Graph 消 **kernel launch overhead** (decode 主要受益), torch.compile 消 **Python overhead + op fusion**, 两者叠加最佳。
- 最终模式受平台、模型、compile config 和 capability resolution 共同决定。调试、新模型、严重 OOM、冷启动敏感、动态 shape 多时, 可先用 --enforce-eager 建立正确性基线, 再逐层恢复能力。
- 失败按 § 4.6 流程从 CG → compile → 两者都关 逐级降级。
- 启动慢的解药: VLLM_TORCH_COMPILE_CACHE_DIR 缓存到磁盘。

自检

1. 一个 decode 步 forward 20ms 里假定 launch overhead 5ms, 开 CUDA Graph 能优化到多少?
2. 你给项目加了一个自定义 attention 变体, 没注册到 torch.library。compile 默认会怎么处理? 性能会更好还是更差?
3. 启动时报错 cudaGraphInstantiate failed, 先动哪个 flag?
4. 同样模型 batch 改变时 vLLM 为什么不重 capture 而是 pad?

参考答案

1. 若 20 ms 中 launch overhead 是 5 ms, 理想化 CUDA Graph 只消除这部分, 约为 15 ms, 理论加速约 1.33×。真实收益还受 graph replay、同步、padding、memory bandwidth 和 kernel 本身影响, 不能直接承诺 25% 固定收益。
2. 未注册的自定义 op 通常成为 compile 的边界或 graph break, 编译器无法安全融合它及其前后算子, 性能往往更差或退回 eager; 不会因为“没注册”自动变快。应提供 torch.library schema、meta/fake 实现和正确的 dispatch, 再用 graph-on/off A/B 证明收益。

3. 先用 `--enforce-eager` 建立能运行的正确性基线，确认不是模型、显存或输入 shape 问题；随后再逐步恢复 `compile/graph`，并检查 `capture size`、显存余量和 `driver/runtime`。不要在现场同时改多个参数。
4. vLLM 通常选择已捕获的较大 shape，用 padding 和静态 buffer 复用 graph，避免每个新 batch 都重新 capture。代价是少量 padding 计算；收益是稳定 replay、较低启动开销和更可预测的 decode 延迟。

下一步

- 深入编译器实现：04-compilation-internals.md（CompilerManager / VllmBackend / 自定义 pass）。
- 想看 attention 在 CG 模式下怎么跑：03-code-walkthrough/05-attention-backends.md。
- 想配 compile cache 加速重启：vllm/envs.py 搜 `VLLM_TORCH_COMPILE_CACHE_DIR`。
- 想看 CG capture 源码：vllm/v1/cudagraph_dispatcher.py、vllm/v1/worker/gpu_model_runner.py::capture_model。

04. Compilation Internals: CompilerManager / VllmBackend / 自定义 pass

谁该读这一篇？ 想深入理解 vLLM 启动慢在哪、torch.compile 如何与 paged attention 共存的进阶读者。

前置阅读： 04-optimizations/03-cudagraph-and-compile.md（CUDA Graph 与 torch.compile 入门），03-code-walkthrough/04-model-runner.md（capture_model 入口）。

耗时： 约 35 分钟。

学完能：

1. 解释 vLLM 为何自写 backend 而不直接 torch.compile(model)
2. 画出 VllmBackend → CompilerManager → CompilerInterface 的 4 层抽象
3. 描述 piecewise split 的切点策略与每段的编译/CUDA Graph 处理
4. 知道 CompilationConfig 哪些旋钮在生产里要调

当前源码复核（b23bd73）： CompilerManager / VllmBackend 仍是核心边界，但 CompilationConfig 已围绕 cudagraph_mode、Inductor graph partition、attention/KV splitting ops 与 capability resolution 演进。文中的 level/piecewise 术语用于理解历史与抽象，最终行为以解析后的 config 和日志为准。

03-cudagraph-and-compile.md 讲了 "torch.compile + CUDA Graph 是什么"。本节深入 vllm/compilation/，看 vLLM 怎么 **自己包装一层 backend**，把 paged attention 这种自定义 op 也纳入图编译。

1. 全景：4 层抽象

源码锚点：[vllm/compilation/backends.py](#) • [VllmBackend](#)

源码锚点：[vllm/compilation/backends.py](#) • [CompilerManager](#)

flowchart TD

```
Eager["PyTorch model.forward (eager)<br>torch.compile(model, backend=VllmBackend)"]
Backend["VllmBackend<br><sub>vllm/compilation/backends.py</sub><br>1. Dynamo trace → FX Graph<br>2. 自定义 pass（fusion / fix custom ops）<br>3. piecewise compile（按 attention 切段）<br>4. 每段交给 CompilerManager 编译"]
Mgr["CompilerManager<br><sub>backends.py</sub><br>• 调用底层 CompilerInterface（如 Inductor）<br>• hash-based 编译缓存<br>• 包装 CUDA Graph capture"]
Iface["CompilerInterface<br><sub>compiler_interface.py</sub><br>• 抽象接口（默认实现 InductorAdaptor）<br>• hash_compiled_artifacts"]

Eager --> Backend --> Mgr --> Iface
```

```
classDef layer fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
class Eager,Backend,Mgr,Iface layer;
```

2. 为什么需要 vLLM 自己的 backend?

直接 `torch.compile(model)` 在 vLLM 下有几个问题：

1. **PagedAttention 是自定义 op**，Dynamo 不认得 → 整图 fallback eager
2. **Attention 内有 dynamic shape**（`block_table` 长度可变）→ 编译图爆炸
3. **CUDA Graph 要求固定 shape**，但不同 step 的 `num_tokens` 变化
4. **整图编译太大**，编译时间 minutes 级

vLLM 的 `VllmBackend` 解决：

- 把 attention 当 "图断点"：图被切成多个 `piecewise` 子图
- 每个子图小、shape 稳定，可单独编译 + 单独 CUDA Graph
- 自定义 op 通过 `torch.library` 注册成可见 op，参与图融合

3. VllmBackend：编排者

源码锚点：[vllm/compilation/backends.py](#) · `VllmBackend`

`vllm/compilation/backends.py` 起：

```
class VllmBackend:
    """vLLM 的 torch.compile backend。"""

    def __init__(self, vllm_config, prefix=""):
        self.compilation_config: CompilationConfig = vllm_config.compilation_config
        self.compiler_manager = CompilerManager(self.compilation_config)
        # 编译 caches
        self.compiled_codes: list[Callable] = []
        # 自定义 pass
        self.post_grad_passes = ...

    def __call__(self, graph: fx.GraphModule, example_inputs):
        """Dynamo 拿到 trace 后的 FX graph 调用这里。"""

        # 1. 跑自定义 pass: fusion / shape simplification / 修复 custom op
        graph = run_passes(graph, self.post_grad_passes)

        # 2. 按 attention layer 切 piecewise
        items: list[SplitItem] = split_graph(graph)

        # 3. 每个 piece 单独编译 + 包 CUDA Graph
        for item in items:
            compiled = self.compiler_manager.compile(item.subgraph, item.example_inputs)
            wrapped = wrap_with_cudagraph_if_needed(compiled, ...)
            self.compiled_codes.append(wrapped)

        # 4. 返回一个 dispatcher: runtime 时按 piece 顺序串起来调用
        return self._make_dispatcher()
```

4. Piecewise split：图怎么被切

源码锚点: [vllm/compilation/backends.py](#) • [split_graph](#)

vllm/compilation/backends.py (split_graph):

策略: 遇到 **attention op** 就切。

- attention 输入有 block_table、slot_mapping (dynamic shape) → 不能 CUDA Graph
- 其余部分 (QKV proj、MLP、norm、residual) shape 固定 → 适合 CUDA Graph

切出来的 piece 大致这样:

flowchart LR

```
P0["piece 0<br/>embed_tokens • rotary_embedding • qkv_proj"]
P1["piece 1<br/>paged_attention<br/>(eager, 不编译)"]
P2["piece 2<br/>o_proj • residual • post_norm<br/>mlp_gate • mlp_up • silu • mlp_down • residual"]
P3["piece 3<br/>paged_attention<br/>(eager, 不编译)"]
Pn["..."]
```

P0 --> P1 --> P2 --> P3 --> Pn

```
classDef compiled fill:#dcfce7,stroke:#15803d,color:#1a1f29;
classDef eager fill:#fef3c7,stroke:#b45309,color:#1a1f29;
class P0,P2 compiled;
class P1,P3 eager;
```

奇数 piece (attention) 走 eager, 偶数 piece 编译 + CUDA Graph。

PiecewiseCompileInterpreter (line 682) 是个 torch.fx.Interpreter 子类, 运行时按 piece 顺序 dispatch。

5. CompilerManager: 编译缓存

源码锚点: [vllm/compilation/backends.py](#) • [CompilerManager](#)

backends.py:

```
class CompilerManager:
    def __init__(self, compilation_config):
        self.compiler = make_compiler(compilation_config) # 返回 InductorAdaptor
        self.cache: dict[Hash, CompiledArtifact] = {}

    def compile(self, subgraph, example_inputs):
        key = self.compiler.hash_artifact(subgraph, example_inputs, compile_config)

        # 看磁盘缓存
        if key in self.cache:
            return self.cache[key]

        # 编译 (昂贵)
        compiled = self.compiler.compile(subgraph, example_inputs)
        self.cache[key] = compiled

        # 持久化到磁盘 (VLLM_TORCH_COMPILE_CACHE_DIR)
        save_to_disk(key, compiled)
        return compiled
```

hash key: 基于 graph 结构 + 输入 shape + dtype + compile config 的 stable hash。同样的子图同样的输入 → 直接读缓存。

设 VLLM_TORCH_COMPILE_CACHE_DIR=/data/vllm_cache，重启不需要重编译。

6. Custom op: 把 paged attention 注册成"知名 op"

vllm/compilation/decorators.py + torch.library:

```
@torch.library.custom_op("vllm::paged_attention_v1", mutates_args=("out",))
def paged_attention_v1(out, query, key_cache, value_cache, ...):
    ...

@paged_attention_v1.register_fake
def _(out, query, ...):
    """fake (meta) impl: 描述输出 shape, 不实际跑。Dynamo trace 用这个。"""
    return None
```

注册后 Dynamo 能识别这个 op:

- 不会 fallback eager
- 可以参与图变换 (move / fuse)
- 可以放进 CUDA Graph (如果它的 shape 是 graph-friendly 的)

7. 自定义 pass: 让图融合更狠

vllm/compilation/passes/:

```
passes/
├── fusion/
│   ├── allreduce_fusion.py    ← AllReduce + add 融合
│   ├── attention_fusion.py    ← attention 前后 op 融合
│   ├── activation_quant_fusion.py ← SiLU + quantize 融合
│   └── ...
├── fx_utils.py
├── inductor_pass.py           ← Inductor pass 接口
├── pass_manager.py            ← pass 串联
├── vllm_inductor_pass.py      ← vLLM 主 pass 集合
└── utility/
```

例如 allreduce_fusion: 把 TP 的 AllReduce 与之后的 add / norm 融合, 减少一次内存往返。

pass_manager.py 决定 pass 顺序, 在 Inductor 自带 pass 之后跑。

8. CompilationConfig: 调优旋钮

vllm/config/compilation.py:

@dataclass

class CompilationConfig:

```
level: CompilationLevel      # NO_COMPILATION / DYNAMO_AS_IS / PIECEWISE / FULL_COMPILE
backend: str = "inductor"
custom_ops: list[str]        # 启用哪些自定义 op
splitting_ops: list[str]     # 切图的 op 名 (默认 attention)
use_cudagraph: bool = True
cudagraph_capture_sizes: list[int] | None
cudagraph_num_of_warmups: int
cudagraph_copy_inputs: bool
use_inductor: bool = True
compile_sizes: list[int] | None
inductor_compile_config: dict
pass_config: PassConfig      # 哪些自定义 pass 开
debug_dump_path: str | None
```

启动命令传 JSON:

```
vllm serve <model> \
--compilation-config '{"level": 3, "use_cudagraph": true, "cudagraph_capture_sizes": [1,2,4,8,16,32,64]}'
```

CompilationLevel

- **0 NO_COMPILATION**: 纯 eager (debug / 兼容性兜底)
- **1 DYNAMO_AS_IS**: Dynamo trace 但不优化
- **2 PIECEWISE**: 一种常见/历史 level 语义; 当前最终 split 与 graph mode 还要经过模型和 capability resolution
- **3 FULL_COMPILE**: (实验) 整图 compile

9. CUDA Graph 包装

源码锚点: [vllm/compilation/backends.py](#) • [wrap_with_cudagraph_if_needed](#)

backends.py (wrap_with_cudagraph_if_needed):

```
def wrap_with_cudagraph_if_needed(compiled_fn, ...):
    if not should_use_cudagraph(...):
        return compiled_fn

    # 给每个 capture size 录一个 graph
    captured_graphs = {}
    for size in cudagraph_capture_sizes:
        with cuda.graph(...) as g:
            compiled_fn(dummy_input(size))
        captured_graphs[size] = g

    def dispatcher(*args):
        size = current_batch_size(args)
        # 找最近的 captured size, pad 输入
        graph_size = next_capture_size(size)
        copy_inputs_into_buffer(args, graph_size)
        captured_graphs[graph_size].replay()

    return dispatcher
```

10. CompilerInterface 与多后端

源码锚点：[vllm/compilation/compiler_interface.py](#) • [EagerAdaptor.compile](#)

vllm/compilation/compiler_interface.py 起：

```
class CompilerInterface(ABC):
    @abstractmethod
    def compile(self, graph, example_inputs, ...): ...

    @abstractmethod
    def hash_artifact(self, graph, inputs, config) -> str:
        """决定缓存 key。"""

    def save_to_file(self, artifact, path): ...
    def load_from_file(self, path) -> CompiledArtifact: ...
```

实现：

- InductorAdaptor（默认）→ torch.compile inductor backend
- 其他 backend（试验性）：eager、aot_eager 等

11. 编译时间在哪？

启动慢主要是这部分：

阶段	耗时
Profile run + KV alloc	~ 10-30 s
torch.compile（每 capture size 一次，第一次冷编）	30-180 s / size
CUDA Graph capture	~ 5-15 s
合计	30 s ~ 8 min

加速：

1. VLLM_TORCH_COMPILE_CACHE_DIR 命中缓存
2. 减少 cudagraph_capture_sizes（如只留 1,4,16,64,256）
3. compile_sizes：只编译特定 batch size 的优化版本
4. --enforce-eager：完全跳过编译

12. 真实坑

12.1 自定义 op 的 fake impl 必须准确

fake impl 返回 shape 错了 → 编译图错 → runtime crash 或结果错。新增 op 一定要写好 register_fake。

12.2 CUDA Graph + dynamic shape 冲突

某个 op 输出 shape 依赖 runtime tensor 值 → CUDA Graph 拒绝 capture。需要在 splitting_ops 加进它，强制切到 eager piece。

12.3 pass 冲突

某些 fusion pass 重排了 op 顺序，可能破坏 vLLM 假设（如 KV write 顺序）。生产前用 debug_dump_path 看融合后图。

12.4 Cache key 不稳定

graph hash 偶尔不稳定（Python 对象 id、内存地址混入）→ 缓存命中率低。如果发现每次重启都重新编译，开 debug log 看 hash 漂移。

13. 工程自检问答

Q: vLLM 为什么不直接 torch.compile(model)? A: ①PagedAttention 是 custom op，Dynamo fallback eager；②dynamic shape 让整图编译爆炸；③CUDA Graph 要 static shape，整图不可能。vLLM 自己写 backend，按 attention 切 piecewise，每段单独优化。

Q: piecewise 切图的依据是什么？ A: splitting_ops 列表里的 op（默认 attention）作为切点。attention 前后是 static shape 段（QKV proj + MLP），可以编译 + CUDA Graph；attention 自己 eager 跑。

Q: 怎么验证 vLLM compile 真在工作？ A: ①看启动日志的 "Compiling..."；②运行时 GPU util 比 enforce-eager 高；③开 debug_dump_path 看融合后的 graph；④比 throughput benchmark。

Q: 升级 vLLM 后编译 cache 还能用吗？ A: 不能。cache key 含 vLLM 版本、torch 版本、CUDA 版本。任一变化都失效。生产升级必须预留重编译时间窗口。

Q: 自定义 pass 能让 throughput 提升多少？ A: 视情况。AllReduce fusion + activation+quantize fusion 在 H100 上累积 5-15% 吞吐。单 pass 单卡的收益不大，但叠加多 pass + 高并发后明显。

单变量实验：证明是哪一层带来收益

保存解析后的 CompilationConfig、cache key 输入、splitting ops、graph mode/capture sizes 与编译日志。先固定 workload 只改一个 pass 或 partition 设置；分别报告 Dynamo/Inductor 编译时间、cache cold/warm、graph replay 覆盖率、kernel time、端到端指标和数值误差。若 fallback 增多、cache key 不稳定或 warmup 超预算，回退到上一层配置。

小结

- vLLM 的 VllmBackend 在 torch.compile 之上包了一层：按 attention 切 piecewise，每段独立编译 + CUDA Graph，避开 dynamic shape 与 custom op 的两大坑。
- 4 层抽象：VllmBackend（编排）→ CompilerManager（缓存）→ CompilerInterface（后端抽象）→ InductorAdaptor（默认实现）。
- 自定义 op 通过 torch.library.custom_op + register_fake 让 Dynamo 认识，参与图融合而不 fallback eager。
- 编译耗时主要来自每个 capture size 的冷编译，靠 VLLM_TORCH_COMPILE_CACHE_DIR 与精简 cudagraph_capture_sizes 缓解。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 直接 torch.compile(model) 在 vLLM 下的 3 个问题 + 规避方法。

问题	表现	vLLM 规避方法
自定义 CUDA op 不可见	torch.ops.vllm.paged_attention_v1 是手写注册的 op，Dynamo trace 会 graph break（不识别 → 标 unsupported → 切段）	vllm/_custom_ops.py 通过 torch.library.define 给每个 custom op 注册抽象 schema 和 fake tensor 实现，让 Dynamo 把它当成已知 op 跨过
dynamic shape 反复重编译	num_tokens 是 SymInt（symbolic），shape 一变 torch.compile 触发 recompile → 每次 5-30s	vllm/compilation/backends.py 内的 VllmBackend 给关键 axis 标记 dynamic，让编译产物对 shape 维度 generic；同时 capture 少数代表 size 做 CUDA Graph
整图编译太大无法分段优化	80 层 Llama 整图编译 + capture 占 5-10 分钟，且编译失败时整图 fallback	split_graph() 把 forward 切成 piecewise（按 attention op 边界），每段独立编译；某段失败也不影响其他段

补充：还有第 4 个隐性问题——CUDA Graph 录制要求输入地址固定，Dynamo 默认会跨步重分配，vLLM 通过 wrap_with_cudagraph 在持久 buffer 上 capture。

2. split_graph 切点依据 + 默认 op 列表。

源码锚点：[vllm/compilation/backends.py](#) • [split_graph](#)

源码：vllm/compilation/backends.py 的 split_graph。切点由 CompilationConfig.splitting_ops 控制。

默认 splitting_ops 通常包含：

- vllm.unified_attention（或具体 attention backend op）
- vllm.silu_and_mul（如果作为分段标记）
- 部分通信 op（vllm.all_reduce、vllm.reduce_scatter）—— 因为通信 op torch.compile 无法 fuse

切点意义：把整 forward 切成 N+1 段（N 个 attention op → N+1 piecewise），每段是纯 PyTorch op（没有自定义 CUDA），可以放心交给 Inductor 编译 + fusion。

配置例：


```
compilation_config = {
  "mode": 3, # VLLM_COMPILE
  "splitting_ops": ["vllm.unified_attention"],
  "use_inductor_graph_partition": False,
}
```

若想完全不切（整图 compile），传 `splitting_ops=[]`——但通常 CUDA Graph 会爆 / op 不识别。

3. CompilerManager 的 cache key 组件 + 为什么升级 vLLM 必失效。

cache key 由以下组件 hash 而成（源码 `vllm/compilation/compiler_interface.py`）：

```
hash_components = {
  "model_class": model.__class__.__qualname__, # 模型类名
  "model_config_hash": ..., # hidden_size / num_layers / ... 等结构参数
  "graph_hash": hash(fx_graph.code), # FX graph 的代码字符串
  "compilation_config": compilation_config_hash, # mode / splitting_ops / etc
  "torch_version": torch.__version__, # PyTorch 版本
  "cuda_version": torch.version.cuda, # CUDA 版本
  "vllm_version": vllm.__version__, # vLLM 版本（关键）
  "gpu_arch": "sm_90", # GPU 计算能力
  "dtype": "bfloat16",
}
cache_key = sha256(json.dumps(hash_components, sort_keys=True))
```

升级 vLLM 后必然失效的原因：

1. `vllm_version` 直接进 hash → 版本号变 hash 变
2. 升级带来代码变动 → FX graph 序列化字符串变 → `graph_hash` 变
3. 可能伴随 PyTorch 升级 → `torch_version` / `cuda_version` 变

→ 这是**故意**的：vLLM 不保证 Pickle 化的 Inductor 产物跨版本兼容；若复用旧产物可能产生静默错误。每次升级冷启动一次的代价 < 静默错误的成本。

缓解：生产部署在每个版本固化 compile cache 到 PV / S3，pod 启动时拉一次，省去重编译。

4. 注册 `vllm::my_custom_op` 让 Dynamo trace 不破坏 piecewise 编译的最小例子。

```

import torch
from torch.library import Library

# 1. 创建 library 命名空间
my_lib = Library("vllm", "FRAGMENT")

# 2. 定义 op schema (关键: Tensor! 表示 in-place 输出)
my_lib.define(
    "my_custom_op(Tensor input, float scale) -> Tensor"
)

# 3. 实现 CUDA 版本 (实际的 kernel 调用)
@torch.library.impl("vllm::my_custom_op", "CUDA")
def my_custom_op_cuda(input: torch.Tensor, scale: float) -> torch.Tensor:
    output = torch.empty_like(input)
    # 调你自己的 CUDA kernel
    torch.ops.my_extension.actual_kernel(output, input, scale)
    return output

# 4. ⚠ 关键: 注册 fake tensor 实现给 Dynamo
@torch.library.register_fake("vllm::my_custom_op")
def my_custom_op_fake(input: torch.Tensor, scale: float) -> torch.Tensor:
    # 不算真实结果, 只返回形状/dtype 一致的 empty tensor
    # 让 Dynamo 知道 "这个 op 输入什么形状、输出什么形状"
    return torch.empty_like(input)

# 5. 使用
def model_forward(x):
    y = torch.relu(x)
    z = torch.ops.vllm.my_custom_op(y, 0.5) # ← Dynamo 现在能 trace
    return z + 1

```

为什么 fake 注册是关键:

- Dynamo 在 trace 时遇到 unknown op 会**整段 graph break** (切回 eager 跑这一段)
- 注册 fake → Dynamo 知道这个 op 的 abstract (fake) 输出形状 → 能继续 trace 后续 op
- piecewise 编译时, 这个 op 仍是 splitting_ops 之一 (如果你想它当切点) 或被透明跨过 (如果不切)

进一步: 如果 op 是 splitting op (attention 这种), 还要在 vllm/compilation/backends.py 的 splitting_ops 列表里加上 vllm.my_custom_op, 让 split_graph 在它附近切。

参考 vLLM 自己的注册: vllm/_custom_ops.py 给 paged_attention_v1 等做了上述完整链路。

下一步

- 下一节: 05-distributed/01-tp-pp-ep.md (从单卡优化跨入分布式并行策略)
- 想看源码: vllm/compilation/backends.py、vllm/compilation/compiler_interface.py、vllm/compilation/passes/
- 想动手: 07-hands-on/04-profiling-and-debugging.md (用 nsys / torch.profiler 验证 piecewise 是否真在工作)
- 想从生产视角理解: 08-production-deployment/04-autoscaling-and-capacity.md (编译缓存命中率对扩容速度的影响)

Sources

源码锚点: [vllm/compilation/backends.py](#) • [CompilerManager](#)

- [vllm/compilation/backends.py](#) (CompilerManager), :548 (split_graph), :628 (wrap_with_cudagraph), :682 (PiecewiseCompileInterpreter), :800 (VllmBackend)
 - [vllm/compilation/compiler_interface.py](#) (CompilerInterface / InductorAdaptor / hash)
 - [vllm/compilation/passes/{vllm_inductor_pass,pass_manager,fusion/*}.py](#)
 - [vllm/compilation/decorators.py](#) (custom op 注册装饰器)
 - [vllm/compilation/cuda_graph.py](#)
 - [vllm/v1/cudagraph_dispatcher.py](#) (runtime size dispatch)
 - [vllm/config/compilation.py](#) (CompilationConfig)
-

See also

- [04-optimizations/03-cudagraph-and-compile.md](#) — 入门概念
- [03-code-walkthrough/04-model-runner.md](#) — capture_model 入口
- [06-interview/01-common-questions.md](#) — Q25 CUDA Graph / Q26 torch.compile

05. 性能模型：存算比、Roofline，以及 Prefill / Decode 的本质差异

谁该读这一篇？ 需要从"第一性原理"判断一个推理优化值不值得做的人——拿到一台卡，先算清楚理论吞吐上限是多少、现在差多远、瓶颈在算力还是带宽，再决定开 batching、量化、还是投机解码。

前置阅读： 00-prerequisites.md § 7（roofline 的粗略版，本章是它的定量展开）、02-continuous-batching.md、01-quantization.md

耗时： 约 18 分钟

学完能：

- 1. 写出算术强度（存算比）的定义，并算出 H100 / A100 的 roofline 拐点。
- 2. 用一行算式证明 prefill 的存算比 $\approx$ batch 内 token 数、decode 的存算比 $\approx$ 并发请求数，从而推出谁 compute-bound、谁 memory-bound。
- 3. 用 Llama-3-70B 的真实显存和带宽，算出 batch=1 decode 的理论 token/s 上限，并解释为什么加 batch 能几乎免费地拉吞吐。
- 4. 讲清一个反直觉但生产上极关键的点：**batching 能摊薄权重读取，但摊不薄 KV cache 读取**——这决定了长上下文为什么依然慢，也决定了 MLA / GQA / KV 量化为什么是刚需。

当前源码复核 (b23bd73)： 本章 FLOPs/bytes 是分析模型，不是 vLLM runtime 硬件 counter。任何 estimated_flops、read/write bytes 都必须注明是否含权重、KV、激活、collective、padding 和 kernel fusion，并用 Nsight/平台 profiler 的实际时间、带宽与算力交叉验证。

这一章不讲某个具体优化，而是给你一把尺子。后面所有优化（量化、投机、MLA、chunked prefill、disaggregation）本质上都是在 roofline 图上往某个方向挪一格。先有尺子，才知道往哪挪。

1. 存算比（算术强度）到底在量什么

GPU 干一件事要花两样东西：**算**（浮点运算，FLOPs）和**搬**（从 HBM 读写数据，Bytes）。两者并行发生——计算单元在算的同时，访存单元在搬。一个 kernel 的实际耗时是这两者的较大值：

$$t = \max(\text{FLOPs} / \text{峰值算力}, \text{Bytes} / \text{峰值带宽})$$

算术强度（Arithmetic Intensity, AI），中文也叫存算比，就是这两个量的比值：

AI = FLOPs / Bytes 单位：FLOP/Byte

↑ ↑

这次计算的 这次计算要从 HBM

总浮点运算 读 + 写的总字节数

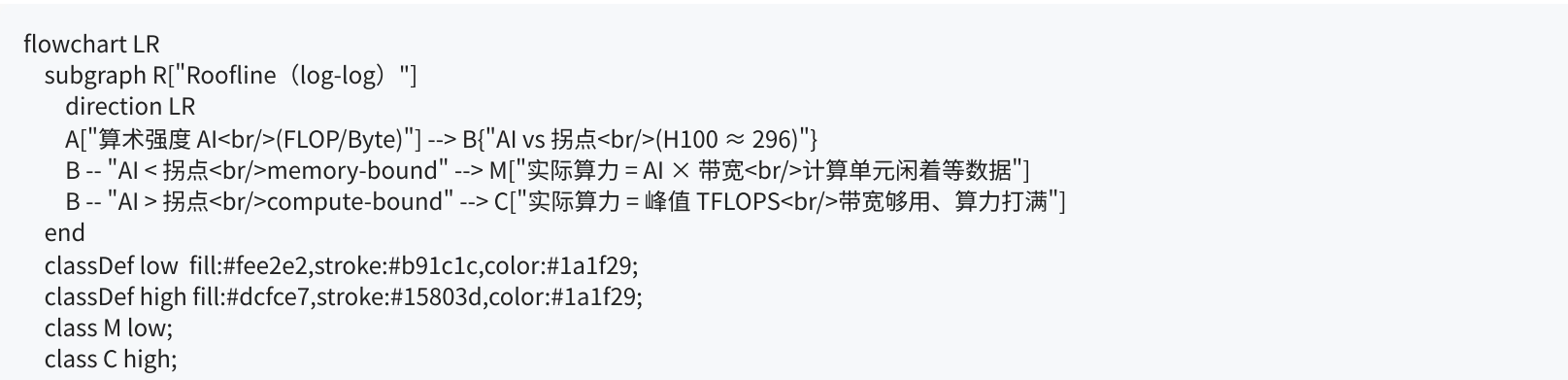
它衡量"每从显存搬 1 个字节，能喂给计算单元多少次运算"。AI 高 = 搬一次数据能算很久 = 算力是瓶颈（**compute-bound**）；AI 低 = 算一下就得再搬 = 带宽是瓶颈（**memory-bound**）。

2. Roofline 拐点：硬件给定的那条分界线

每张卡有两条硬上限。把它们一除，就得到这张卡的**拐点（ridge point）**——AI 超过它就 compute-bound，低于它就 memory-bound：

拐点 = 峰值算力 / 峰值带宽 单位：FLOP/Byte

GPU	峰值算力（dense）	HBM 带宽	拐点 $\approx$
A100 80GB SXM	BF16 312 TFLOPS	2.0 TB/s	156 FLOP/Byte
H100 SXM	BF16 990 TFLOPS	3.35 TB/s	$\approx$ 296 FLOP/Byte
H100 SXM（FP8）	FP8 1979 TFLOPS	3.35 TB/s	$\approx$ 590 FLOP/Byte
H200 SXM	BF16 990 TFLOPS	4.8 TB/s	$\approx$ 206 FLOP/Byte



记两个数就够日常估算：**A100 拐点 ~150，H100 拐点 ~300**。注意拐点会随精度变——FP8 把算力翻倍但带宽不变，拐点反而更高（590），意思是 **FP8 让"打满算力"变得更难**，这点在 § 6 还会回来。

3. 一个 Linear 层的存算比 $\approx$ 参与的 token 数

LLM 的绝大部分 FLOPs 和绝大部分权重读取都在 Linear 层（QKV proj、O proj、MLP 的 gate/up/down）。把一个 Linear 层拆开算，结论会异常干净。

设某个权重矩阵形状 $[K, N]$ （输入维 K 、输出维 N ），一次 forward 喂进 M 个 token（形状 $[M, K]$ ）：

- FLOPs**：矩阵乘 $[M, K] \times [K, N]$ ，约 $2 \cdot M \cdot K \cdot N$ （乘加算两次浮点）。
- Bytes**：权重要从 HBM 读一遍，BF16 是 $2 \cdot K \cdot N$ 字节。（输入 / 输出激活 $2 \cdot M \cdot K$ 、 $2 \cdot M \cdot N$ 量级小得多，先忽略。）

于是：

$$AI \approx (2 \cdot M \cdot K \cdot N) / (2 \cdot K \cdot N) = M$$

一个 **Linear 层的存算比**，约等于这一步喂进去的 **token 数 M**。权重只读一次，但被 M 个 token 反复使用——M 越大，每个搬进来的权重字节被"复用"得越多，AI 越高。这就是整个推理性能模型的核心等式。

把它套到两个阶段：

3.1 Prefill：M = prompt 的 token 数 → 天然 compute-bound

Prefill 一次把整个 prompt（成百上千 token）喂进 forward，所有 token 共享同一份权重读取：

M = prompt 长度（如 2048） → AI $\approx$ 2048 $\gg$ 拐点 300 → compute-bound

只要 prompt 不是特别短，prefill 的 AI 就远超拐点，**算力被打满，带宽有富余**。这就是"prefill 是 compute-bound"的定量来源——不是定性感觉，是 AI $\approx$ 2048 > 296 这个不等式。推论：prefill 阶段很难再靠"加 batch"提速（算力已经满了），但**能靠量化提速**（INT4 把 compute-bound 的 GEMM 换成更快的低精度 GEMM）。

3.2 Decode：M = 并发请求数 → batch 小就是 memory-bound

Decode 每个请求每步只产 1 个新 token。如果 batch 里有 B 个请求一起 decode，那么喂进 Linear 层的就是 B 个 token：

M = 并发请求数 B
B = 1 → AI $\approx$ 1 $\ll$ 300 → 严重 memory-bound（算力 99% 闲置）
B = 32 → AI $\approx$ 32 < 300 → 仍 memory-bound，但好多了
B $\approx$ 300 → AI $\approx$ 300 $\approx$ 拐点 → 刚好把 H100 算力喂满

"**decode 是 memory-bound**"的本质：**batch 不够大时 M 太小，权重搬进来没算几下就得搬下一块**。而把 B 拉到拐点附近（H100 上 ~300），decode 才进入 compute-bound 区。这正是 continuous batching 的物理动机——不是"为了多服务几个用户"那么简单，而是**把 decode 从 roofline 的左半边推到右半边**。

4. 用真实数字算一遍：Llama-3-70B 的 decode 上限

光看 AI 不够直观，直接算理论 token/s。

Llama-3-70B，BF16，单卡装得下（先假设不切 TP）：

- 权重 $\approx 70.6\text{B} \times 2\text{ Byte} \approx$ **141 GB**
- decode 一步，无论 batch 多大，**每个权重字节都要从 HBM 读恰好一次**

在 H100（3.35 TB/s）上，读完 141 GB 权重的时间：

$t_{\text{weight}} = 141\text{ GB} / 3.35\text{ TB/s} \approx 42\text{ ms}$

这 42 ms 是 decode 一步的**带宽下限**（还没算 KV 和 attention）。于是：

batch B	每步耗时	产出 token	吞吐	GPU 算力利用率
1	~42 ms	1	≈ 24 tok/s	~0.3% (AI=1 / 300)
32	~42 ms	32	≈ 760 tok/s	~11%
256	~43 ms	256	≈ 6000 tok/s	~85%

关键洞察：从 B=1 到 B=32，每步耗时几乎不变（都被 42 ms 的权重读取卡住），但吞吐涨了 32 倍。这就是"加 batch 几乎免费"的来源——你在用本来就要付的带宽成本，多服务 31 个用户。也是为什么单请求延迟（TPOT）在中等负载下对并发数不敏感：decode 步长由权重读取决定，不由你一个人决定。

实践里 70B 一般用 **TP=8**，每张卡只存 1/8 权重（~17.6 GB），8 张卡并行读，单步权重读取降到 ~5.3 ms，单流可到 ~190 tok/s——这又是另一个故事（TP 同时引入 AllReduce 通信，见 05-distributed/01-tp-pp-ep.md）。

5. 最关键的反直觉：batching 摊薄权重，但摊不薄 KV cache

到这里有个陷阱。上面说"加 batch 让 AI 涨到 B"，那是不是 B 拉够大，decode 就彻底 compute-bound、长上下文也快了？**不是**。因为 decode 一步其实做两类计算，它们的存算比行为完全不同：

计算	读什么	batch 后能否摊薄？
Linear 层 (QKV/O/MLP)	模型权重（所有请求共享同一份）	☑ 能。B 个请求复用同一份权重 → AI ≈ B
Attention (Q · K [⊗] 、softmax · V)	每个请求 自己的 KV cache （互不共享）	☒ 不能。请求 i 的 query 只读请求 i 的 KV

Attention 这部分的存算比，对单个 decode token 来说**恒等于 ~1**：读进来 context_len 个 KV 向量，每个只被这 1 个新 query 用一次。加 batch 把更多请求并排，但每个请求的 KV 还是只服务自己——KV 读取量随 batch **线性增长**，AI 不增长。

后果（这是生产上长上下文慢的根因）：

- 短上下文：KV 小，Linear 层主导，加 batch 有效，吞吐高。
- 长上下文（如 32K、128K）：每个请求的 KV cache 巨大，decode 的耗时逐渐被"读 KV"主导，而读 KV **无法靠 batch 摊薄**。于是 batch 拉得再大，每 token 延迟也压不下去，吞吐天花板被 KV 带宽锁死。

一笔 KV 账（Llama-3-70B，GQA 8 个 KV head，head_dim 128，80 层）：单请求每 token 的 KV ≈ 2(K和V) × 80层 × 8 × 128 × 2 Byte ≈ 320 KB/token。32K 上下文 → 单请求 KV ≈ **10 GB**。batch=32 个这样的请求 → 每步光读 KV 就要 320 GB，在 H100 上 ~95 ms，**比读权重还贵**。

这把后面几个优化的动机一次性串起来——它们都是冲着"减少 KV 字节"或"提高 KV 的复用"去的：

- GQA / MQA**：减少 KV head 数 → 直接砍 KV 字节（Llama-3 已用 GQA，否则上面数字 × 4）。
- MLA (DeepSeek)**：把 KV 压成一个低秩 latent，KV 字节再降一个量级 → 07-model-architectures.md。
- KV cache FP8**：KV 字节直接减半 → 01-quantization.md。
- Prefix caching**：命中的前缀 KV 不重算也少读 → 04-prefix-caching.md。

6. 把所有优化挂到 roofline 上

有了尺子，每个优化做的事都能用一句话说清——它在 roofline 图上把工作点往哪挪：

优化	在 roofline 上做什么	对谁有效
Continuous batching	提高 decode 的 M (=B) → AI 右移，逼近拐点	decode（核心手段）
Chunked prefill	把 compute-bound 的 prefill chunk 塞进 memory-bound 的 decode 步，用 prefill 的算力填满 decode 闲置的计算单元	混合负载（见下）
权重量化（INT4/FP8）	减少权重 Bytes → 同样 M 下 AI 翻倍；且 compute-bound 的 prefill 用更快的低精度算	decode 提带宽、prefill 提算力
KV 量化 / MLA / GQA	减少 attention 的 Bytes → 缓解 § 5 的 KV 墙	长上下文 decode
投机解码	一次 target forward 验证多个 token → 把"1 token / 次权重读取"变成"k token / 次"，等效提高 decode 的 AI	小 batch、低并发场景

chunked prefill 值得单独点一句，它是 roofline 思维最漂亮的应用：decode 步算力大量闲置（ $AI \approx B \ll$ 拐点），prefill 步算力打满但只在偶发。V1 默认把两者**混在同一步**跑——prefill 的高 AI 工作正好填上 decode 的低 AI 工作留下的算力空洞，让这一步的综合 AI 抬到拐点附近。本质是"拿 compute-bound 的活去补 memory-bound 的算力窟窿"。细节见 05-chunked-prefill.md。

FP8 的小陷阱（呼应 § 2）：FP8 把算力翻倍 → 拐点从 300 抬到 590。这意味着用了 FP8 后，要把 decode 喂到 compute-bound 需要**更大的 batch**。FP8 主要收益其实是"权重 Bytes 减半 → AI 翻倍 → 带宽侧提速"，算力翻倍这件事在 memory-bound 的 decode 里基本吃不到。判断一个量化方案对 decode 的收益，永远先看它省了多少 Bytes，而不是多了多少 TFLOPS。

单变量实验：单位与 sanity check

表格至少包含：每请求/每 step token 数、FLOPs、权重/KV/激活/collective bytes、理论时间 $\max(\text{FLOPs/peak}, \text{bytes/bandwidth})$ 、实测 kernel/step 时间、实测带宽/算力利用率。统一使用 FLOP、byte、second，注明 GB/GiB。一次只改 batch、context 或 dtype 之一；估算与 profiler 相差数量级时先修模型，不据此调参。

小结

- 存算比 (AI) = FLOPs / Bytes**，和硬件**拐点（峰值算力/带宽）**比大小，决定 compute-bound 还是 memory-bound。A100 拐点 ~150，H100 ~300。
- 一个 **Linear 层的 AI $\approx$ 参与的 token 数 M**。Prefill 的 M = prompt 长度（上千）→ AI $\gg$ 拐点 → compute-bound；decode 的 M = 并发请求数 B → B 小则 AI $\ll$ 拐点 → memory-bound。这就是两阶段差异的定量根。
- Llama-70B decode 每步要读 141 GB 权重（H100 ~42 ms），**加 batch 不增加这个成本却线性增吞吐**——这是 batching 的物理本质。

- **batching 摊薄权重读取，但摊不薄 KV cache 读取。**长上下文下 decode 被 KV 带宽锁死，催生 GQA / MLA / KV 量化 / prefix caching。
- 所有推理优化都能翻译成"在 roofline 上把工作点往拐点挪"的一句话。

自检（先自答，再看要点）

1. H100 上 Llama-3-8B（BF16，16 GB 权重）做 batch=1 decode，理论 token/s 上限大概多少？

要点：16 GB / 3.35 TB/s $\approx$ 4.8 ms/步 $\rightarrow$ ~210 tok/s。比 70B 快约 9 倍，因为权重小 9 倍、带宽成本小 9 倍。注意这是无视 KV 和 kernel launch 的理论上限，实测会更低（见 10-gpu-utilization-and-tail-latency.md）。

2. 为什么 prefill 几乎不能靠加 batch 提速，decode 却可以？

要点：prefill 的 M（=prompt token 数）本来就上千，AI 已远超拐点、算力已满，再加 batch 只是排队等算力；decode 的 M（=并发数）很小、AI 远低于拐点、算力大量闲置，加 batch 直接把闲置算力用起来，且不增加权重读取成本。

3. 一个客户上了 128K 长上下文，发现 batch 拉到 64 吞吐也上不去，为什么？该往哪个方向优化？

要点：长上下文下 decode 耗时被**读 KV cache**主导，而 KV 读取**无法靠 batch 摊薄**（每个请求读自己的 KV）。加 batch 只增加总 KV 读取量。方向：砍 KV 字节——KV FP8、换用 MLA 架构的模型、确保 GQA 生效、靠 prefix caching 复用公共前缀；架构上考虑 P/D 分离让 decode 节点专注吞吐。

4. 有人说"上了 FP8 算力翻倍，decode 该快一倍"，对吗？

要点：不全对。FP8 在 decode 的真实收益来自**权重 Bytes 减半 $\rightarrow$ AI 翻倍 $\rightarrow$ 带宽侧提速**；算力翻倍对 memory-bound 的 decode 基本吃不到（拐点反而被抬高到 590，更难 compute-bound）。看量化对 decode 的收益要看省了多少 Bytes，不是多了多少 FLOPS。

下一步

- 上一节回看：01-quantization.md（量化为什么对 decode 是"带宽优化"、对 prefill 是"算力优化"，这章给了它定量解释）。
- 配套生产视角：08-production-deployment/10-gpu-utilization-and-tail-latency.md（理论上限知道了，为什么实测打不满？全链路诊断）。
- 想动手验证：07-hands-on/03-mini-experiments.md（扫 max_num_seqs 观察吞吐随 batch 的曲线，亲眼看到拐点）。
- 回到核心概念：02-core-concepts/02-continuous-batching.md、05-chunked-prefill.md。

01. 分布式推理：TP / PP / EP / DP

谁该读这一篇？ 即将部署 70B+ 大模型、MoE 模型，或需要清晰回答"8 卡怎么切 Llama-70B"的工程师。

前置阅读： 01-overview/02-architecture.md（vLLM 整体架构），02-core-concepts/03-kv-cache-management.md（理解 KV 在 TP 下怎么切）。

耗时： 约 25 分钟。

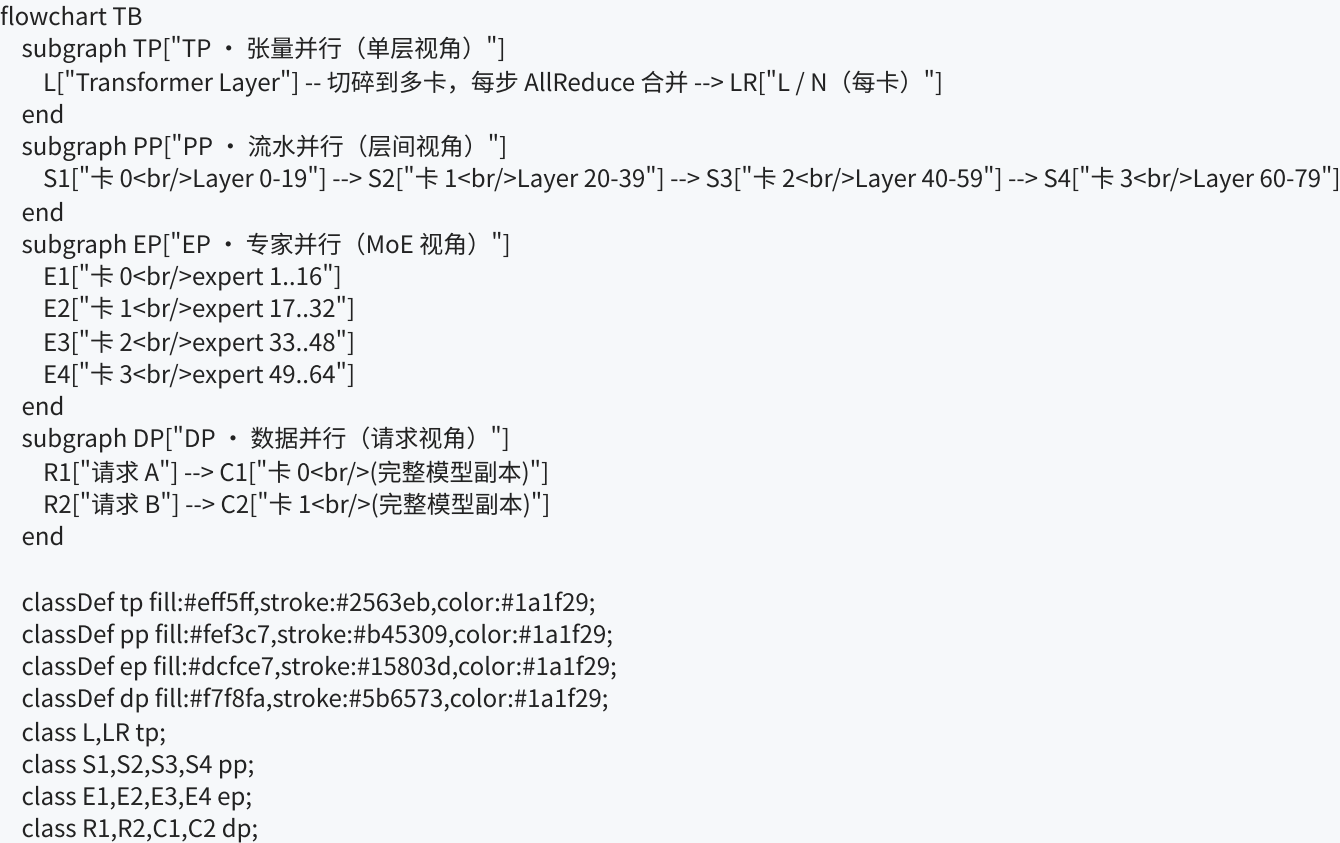
学完能：

1. 在白板画出 MLP 的 column $\rightarrow$ row parallel 切分，说出每层 2 次 AllReduce 的来源
2. 解释 PP 的 bubble 是怎么产生的、推理为何不需要训练里的 1F1B 调度
3. 描述 MoE 的 EP 通信（两次 AllToAll）和 EPLB 解决的负载问题
4. 对一台 $8 \times H100$ ，能给出 Llama-70B / Mixtral / DeepSeek-V3 的推荐并行组合

当前源码复核 (b23bd73)： ParallelConfig.world_size 在普通 executor 内先表示 $TP \times PP \times PCP$ ；跨 DP 总规模看 world_size_across_dp，external launcher 又会把 DP 纳入 worker world。DCP 复用 TP ranks；EP group 跨 $TP \times PCP \times DP$ 。PCP 当前不支持与 $DP > 1$ 组合，Elastic EP 还要求 EPLB、禁用 $PP > 1$ ，并限制 external/hybrid LB。

一个模型一张卡装不下，怎么切？这一节讲三种主流并行 + Data Parallel 共 4 种，以及它们在 vLLM 里的进程模型与代码入口。

1. 三种切法直觉



实际生产组合：TP × PP × EP × DP × (PCP/DCP)（5 维并行）。

1.1 四个维度一句话区分

维度	切什么	主要通信	适用	典型 size
TP	单层权重（hidden/intermediate 维）	AllReduce（每层 2 次）	模型单卡塞不下；单 query 延迟敏感	2 / 4 / 8
PP	按层切（layer 序列）	P2P send/recv（边界）	模型超大跨机；高并发吞吐优先	2 / 4 / 8
EP	MoE expert（个体粒度）	AllToAll（每 MoE 层 2 次）	MoE 模型必加（Mixtral / DeepSeek / Qwen-MoE）	4 / 8 / 16 / 32
DP	请求 batch（副本级）	无（纯副本） / AllToAll（DP+EP）	高 QPS 横向扩展	任意

配套维度（不增 world size 或仅在某阶段使用）：

维度	切什么	何时用
DCP	decode 阶段 KV cache 的 seq 维（复用 TP GPU）	长上下文 decode；MLA 模型用 a2a backend 省 NCCL
PCP	prefill 阶段的 seq 维（增 GPU）	超长上下文 prefill 塞不下激活
SP	单层内"激活的 sequence 维"（附属于 TP）	默认搭配 TP 使用，减 LayerNorm/residual 的冗余

详见 04-context-parallel.md（DCP/PCP）和本节 § 6（SP）。

1.2 决策树：我该用哪种并行？

flowchart TD

```
Start[新部署：选并行策略]
Start --> Q1{模型单卡塞得下?}
Q1 -->|是| Q2{需要更高吞吐?}
Q2 -->|是| OnlyDP[纯 DP<br/>多实例横向扩]
Q2 -->|否| Single[单卡单实例]
Q1 -->|不| Q3{是 MoE 模型?}
Q3 -->|是| Q4{有 RDMA 跨机网络?}
Q4 -->|有| WideEP[Wide-EP<br/>TP×DP 多机]
Q4 -->|没| SingleEP[单机 EP<br/>TP=8 + EP=8]
Q3 -->|不是| Q5{单机塞得下?}
Q5 -->|塞得下| Q6{单 query 延迟敏感?}
Q6 -->|是| OnlyTP[纯 TP<br/>例：TP=8]
Q6 -->|否| Q7{高并发吞吐优先?}
Q7 -->|是| TPDP[TP=4 × DP=2<br/>实例并行]
Q7 -->|否| OnlyTP
Q5 -->|塞不下| Q8{以太网还是 IB?}
Q8 -->|IB/RoCE| TPPP[TP=8 + PP=2/4<br/>跨机 with RDMA]
Q8 -->|以太网| PurePP[PP 跨机<br/>TP 单机 + PP 多机]
Q5 -->|长上下文 128K+| AddCP[基础组合 + DCP/PCP]
```

```
classDef root fill:#eff5ff,stroke:#2563eb;
classDef decision fill:#fff7ed,stroke:#f59e0b;
classDef action fill:#dcfce7,stroke:#15803d;
class Start root;
class Q1,Q2,Q3,Q4,Q5,Q6,Q7,Q8 decision;
class OnlyDP,Single,WideEP,SingleEP,OnlyTP,TPDP,TPPP,PurePP,AddCP action;
```

5 句话口诀：

- 单卡塞不下 → **TP**（同机） / **PP**（跨机）
- MoE 模型 → **评估 EP**；小规模可走 TP 路径，大规模再按权重、AllToAll 与负载决定
- 高 QPS → **DP** 横向
- 长上下文 → **DCP/PCP** 加维度
- 凡跨机 → 看网络：**RDMA 上 TP+PP，以太网上纯 PP**

1.3 当前维度契约速查

维度	切什么	典型 collective	显存/延迟影响	关键条件
TP	hidden/head/intermediate	AllReduce/ReduceScatter/AllGather，依 layer 实现	分权重；每层同步增加 latency	heads/intermediate 与 quant/kernel 的可分性
PP	layers	stage P2P	分权重；bubble 与 stage imbalance	layer partition、stage 输入输出兼容
DP	requests/model replicas	普通 dense forward 可独立；wide EP 时参与 group	复制 dense 权重与 KV；扩吞吐	内/外 LB 与 coordinator 模式
EP	routed experts	逻辑 dispatch + combine（backend 可融合/改写）	分 expert 权重；引入 all-to-all 与 straggler	enable_expert_parallel、expert mapping/backend
PCP	prefill sequence	attention backend 特定通信	分长 prefill 激活/KV；进入 worker world	当前与 DP>1 不兼容
DCP	decode context/KV sequence	ag_rs 或条件式 a2a	复用 TP ranks，不增 worker world	TP % DCP == 0、backend 支持

每个估算都写单位：worker 数、bytes/rank、collective 次数/layer、seconds/step。不要把“每层 2 次 AllReduce/AllToAll”外推到所有 fused、quantized、MLA 或 backend 路径；以实际 module 与 profiler trace 为准。

2. Tensor Parallel（TP）

2.1 怎么切

符号约定

符号	含义
B	一步内 token 总数（batch × seq_len 打平后）
H	hidden size（模型维度）
I	intermediate size（MLP 中间维度，约 2.6× ~ 5× H，见 § 2.1 末尾）
h	num attention heads（注意力头数）
k	num KV heads（GQA 下 k ≤ h）
d	head dim（每头维度，H = h × d）
N	TP world size（参与切分的 GPU 数）

下文所有矩阵都按 PyTorch 习惯写 [输入维度, 输出维度]，张量按 [B, ...] 写。

MLP 层：完整张量流（以 Llama SwiGLU 为例）

公式： $y = (\text{silu}(x \cdot W_{\text{gate}}) * (x \cdot W_{\text{up}})) \cdot W_{\text{down}}$

未切分（单卡）的形状流：

```
x      : [B, H]
W_gate : [H, I]      ← 投影 1 (gate)
W_up   : [H, I]      ← 投影 2 (up)
x * W_gate : [B, I]
x * W_up   : [B, I]
silu(...) * (...) → mid : [B, I] ← 逐元素，仍是 I 维
W_down    : [I, H]    ← 下投影
mid * W_down → y : [B, H]
```

TP=N 切分后：

———— Column Parallel: $W_{\text{gate}} / W_{\text{up}}$ 沿 I 轴切 ————

每卡持有：

```
W_gate_local : [H, I/N]    ← 列切：保留所有输入维 H，输出维分给 N 卡
W_up_local   : [H, I/N]
```

输入 $x : [B, H]$ （每卡都有完整的一份）

每卡算：

```
out_gate_local : [B, I/N]   ← 仍是 partial 输出（沿 I 轴的一段）
out_up_local   : [B, I/N]
mid_local      : [B, I/N]   ← silu * 仍是逐元素，I/N 不变
```

△ 此时不需要任何通信：因为 mid 在下一步要进的 W_{down} 是按行切的，它要的就是 $[B, I/N]$ 这个分片的输入。

———— Row Parallel: W_{down} 沿 I 轴切 ————

每卡持有：

```
W_down_local : [I/N, H]    ← 行切：输入维分给 N 卡，每卡保留完整输出维 H
```

每卡输入：

```
mid_local : [B, I/N]      ← 上一步的输出，刚好对得上
```

每卡算：

```
partial_y_local : [B, H]
= mid_local * W_down_local
```

△ 但 partial_y_local 只是真实结果的「一部分」，不是完整的 y ！

原因见下面 § 「AllReduce 到底在干嘛」。

通信：

```
AllReduce(partial_y_local) → y : [B, H] ← 一次 AllReduce
```

AllReduce 到底在干嘛

数学上，row-parallel 算出来的 partial_y_local 是真实 y 的**沿 I 轴的部分和**。回到矩阵乘的定义：

$$y_{b,h} = \sum_{i=0}^{I-1} \text{mid}_{b,i} \cdot W_{i,h}$$

把求和 $\sum_{i=0}^{I-1}$ 拆成 N 段：

$$y_{b,h} = \underbrace{\sum_{i=0}^{I/N-1} \dots}_{\text{rank 0 的 partial}} + \underbrace{\sum_{i=2I/N-1}^{I-1} \dots}_{\text{rank 1 的 partial}} + \dots + \underbrace{\sum_{i=(N-1)I/N}^{I-1} \dots}_{\text{rank N-1 的 partial}}$$

每张卡只算其中一段，得到 $\text{partial_y_local} : [B, H]$ 。**N 张卡的 partial_y_local 形状相同，值不同；要把它们逐元素相加才是真正的 y 。**这件事就叫 **AllReduce-SUM**：

rank 0: partial_y_local = [B, H] ↙
rank 1: partial_y_local = [B, H] |
... | 逐元素 sum 后广播给所有 rank
rank N-1: partial_y_local = [B, H] ↘

→ 每张 rank 拿到相同的 y : [B, H]

NCCL 在 NVLink / IB 上有高效的 ring-allreduce / tree-allreduce 实现，**每张卡发送+接收的数据量约 $2 \times (N-1)/N \times |[B,H]|$ 字节**—— $B \times H \times 2 \times (N-1)/N \times \text{dtype_bytes}$ 。

关键观察：column-parallel 完后**不通信**就把分片直接喂给 row-parallel，row-parallel 完后**才通信**。这就是 Megatron 论文的核心 trick：**两次切分配合，把 4 次潜在通信压成 1 次 AllReduce**。如果把顺序反过来（row 在前 column 在后），每个 Linear 后都要通信，开销翻倍。

Attention 层：完整张量流

公式（以 GQA / MHA 为例）：

Q = x · W_Q # column parallel
K = x · W_K # column parallel
V = x · W_V # column parallel
attn_out = Attention(Q, K, V)
y = attn_out · W_O # row parallel

未切分形状：

x : [B, H]
W_Q : [H, h × d] h 个 query head，每个 d 维 → 等价 [H, H]
W_K : [H, k × d] k 个 KV head（GQA: k ≤ h）
W_V : [H, k × d]
Q : [B, h, d] 重整 reshape
K : [B, k, d]
V : [B, k, d]
attn_out : [B, h × d] = [B, H]
W_O : [H, H]
y : [B, H]

TP=N 切分后（要求 $h \% N == 0$ 且 $k \% N == 0$ ，否则报错）：

———— Column Parallel: $W_Q / W_K / W_V$ 按头数切 ————

每卡持有：

$W_Q_{local} : [H, (h/N) \times d]$ $\leftarrow$ 每卡负责 h/N 个 query head

$W_K_{local} : [H, (k/N) \times d]$ $\leftarrow$ 每卡负责 k/N 个 KV head

$W_V_{local} : [H, (k/N) \times d]$

输入 $x : [B, H]$ （每卡都有完整一份）

每卡算：

$Q_{local} : [B, h/N, d]$

$K_{local} : [B, k/N, d]$

$V_{local} : [B, k/N, d]$

△ 注意力计算是 head-wise 独立，所以每张卡只用本卡的 K_{local} / V_{local}

 算自己负责的 h/N 个 query head 的 attention：

$attn_out_local : [B, h/N \times d] = [B, H/N]$

△ 此时不通信：attn_out_local 直接喂给 row-parallel 的 W_O 。

———— Row Parallel: W_O 按输入维切 ————

每卡持有：

$W_O_{local} : [H/N, H]$

每卡输入：

$attn_out_local : [B, H/N]$ $\leftarrow$ 跟 W_O_{local} 的 H/N 对得上

每卡算：

$partial_y_local : [B, H]$ $\leftarrow$ 又是「部分和」

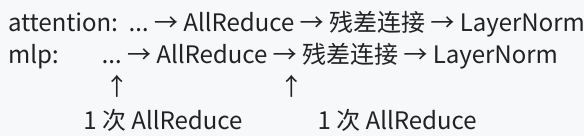
通信：

$AllReduce(partial_y_local) \rightarrow y : [B, H]$ $\leftarrow$ 一次 AllReduce

数学上和 MLP 完全是同一招：column-parallel 切「输出维」、row-parallel 切「输入维」、partial 自动衔接、最后一次 sum-AllReduce 收尾。

KV cache 的副作用：因为 K / V 是 column-parallel 切的，每张卡也**只存 k/N 个 KV head 的 cache**。所以 PagedAttention 的 KV block 形状 $[2, num_blocks, block_size, k/N, d]$ （这里 k 就是源码里的 num_kv_heads，已经是 per-rank 的值）。

每层共 2 次 AllReduce



Llama-3-70B 有 80 层，所以一次 forward 一共 **160 次 AllReduce**。这正是 § 2.2 通信代价分析的起点。

关于 intermediate 这个数：原始 Transformer / GPT-3 用 $intermediate = 4 \times hidden$ 是行业惯例；但 Llama 之后用 SwiGLU（多一个 W_{gate} 投影），为控制参数量改成 $\approx 8/3 \times hidden$ ，所以各家模型 ratio 差异大：Llama-2-7B 是 $2.69\times$ 、Llama-3-70B 是 $3.5\times$ 、Qwen2.5-7B 是 $5.3\times$ 、DeepSeek-V3 dense 层 $2.57\times$ 。TP 切法本身与该数值无关，唯一硬性约束是 $intermediate \% N == 0$ 和 $num_heads \% N == 0$ 、 $num_kv_heads \% N == 0$ 。

——一张表带走：Llama-3-70B TP=4 的所有形状

张量	全局形状	每卡形状（TP=4）
W_Q	[8192, 8192]	[8192, 2048]
W_K	[8192, 1024] (8 KV heads $\times$ 128 dim)	[8192, 256] (2 KV heads/卡)
W_V	[8192, 1024]	[8192, 256]
W_O	[8192, 8192]	[2048, 8192]
W_gate	[8192, 28672]	[8192, 7168]
W_up	[8192, 28672]	[8192, 7168]
W_down	[28672, 8192]	[7168, 8192]
Attention AllReduce 通信量 / token	—	$8192 \times 2 \text{ bytes} = 16 \text{ KB}$
MLP AllReduce 通信量 / token	—	$8192 \times 2 \text{ bytes} = 16 \text{ KB}$
整个 forward 通信量 / token	—	$16 \text{ KB} \times 80 \text{ 层} \times 2 = 2.56 \text{ MB}$

2.2 通信代价

每个 token 每层 AllReduce 数据量 = hidden_size * dtype_bytes。 Llama-70B： hidden=8192， BF16 $\rightarrow$ 16KB / token / 层 / AllReduce $\times 80 \text{ 层} \times 2 \rightarrow 2.5\text{MB}$ / token。

8 卡 NVLink (600GB/s) 通信时间约 ms 级； PCIe 慢 $10\times$ 。

$\rightarrow$ **TP 强依赖高带宽互联**。跨机器 TP 通常不可行。

2.3 代码

```
vllm/distributed/parallel_state.py - 进程组初始化
vllm/model_executor/layers/linear.py - ColumnParallelLinear / RowParallelLinear
vllm/distributed/communication_op.py - AllReduce / AllGather wrapper
```

启用：--tensor-parallel-size 8。

3. Pipeline Parallel（PP）

延用 § 2.1 符号，再加 L = 总层数、P = PP world size、M = micro-batch 个数。

3.1 怎么切

按层切。L 层均匀分到 P 张卡，每张卡持有 L/P 层（要求 $L \% P == 0$ ，或自定义 partition）。

```
flowchart LR
    C0["卡 0<br/>embed +<br/>Layer 0..L/P-1"] -->|hidden| C1["卡 1<br/>Layer L/P..2L/P-1"]
    C1 -->|hidden| C2["卡 2<br/>..."]
    C2 -->|hidden| C3["卡 P-1<br/>Layer (P-1)L/P..L-1<br/>+ lm_head"]

    classDef stage fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
    class C0,C1,C2,C3 stage;
```

注意 embedding 与 LM head 的归属：

- **rank 0** 额外持有 token embedding $([V, H])$ ， V 是词表大小，BF16 下 Llama-3 的 $128K \times 8192 \times 2 \approx 2 \text{ GB}$
- **rank P-1** 额外持有 LM head（同样 $[H, V]$ 或与 embedding tied）
- 中间 rank 只有 transformer 层 + 其 KV cache

3.2 单 micro-batch 的张量流（推理，无反向）

```
rank 0:
input_ids : [B]      ← B = token 数
embed    : [B, H]    ← 走 W_embed [V, H]
layer 0..L/P-1 forward → hidden : [B, H]
                        ↓ NCCL send

rank 1:
hidden : [B, H] ← recv
layer L/P..2L/P-1 → hidden : [B, H]
                  ↓ send

...

rank P-1:
hidden : [B, H] ← recv
layer (P-1)L/P..L-1 → hidden : [B, H]
hidden · W_LM_head → logits : [B, V]
sample → next_token : [B]
```

通信原语：**NCCL P2P send/recv**（不是 collective）。每个段边界传一个 $[B, H]$ 张量给下一段。

形状细节： B 在 vLLM V1 里其实是"打平后的 token 数"（packed batch，详见 02-core-concepts/02-continuous-batching.md）。Prefill 阶段 B 很大（几百到几千），decode 阶段 $B = \text{batch 内 running 请求数}$ 。

3.3 流水填充：bubble 怎么来的、怎么算

直觉：把 PP 想成汽车装配线

想象一条汽车装配线，4 个工位：① 装底盘 → ② 装引擎 → ③ 装内饰 → ④ 喷漆。每辆车必须按 ①→②→③→④ 顺序过完 4 个工位才算造完。

- **刚开机时：**第一辆车在工位 ① 装底盘，工位 ②③④ 还没车可干，**空着**。
- **过一段时间：**第一辆车被推到工位 ②，第二辆车进工位 ①，工位 ③④ 还在空。
- **再过一段：**四个工位全部有车，**满负荷**。
- **最后一辆车进了工位 ①：**之后再没新车进来，工位 ① 先空下来，然后 ②③，最后只有工位 ④ 在喷最后一辆车。

工位"空着"的时间就是 **bubble**。PP 完全一样：每张 GPU 是一个工位，每个 micro-batch 是一辆车。

时间步表格（最直观的画法）

设 $P=4$ （4 张卡）、 $M=4$ （4 个 micro-batch），每个 micro-batch 在每张卡跑 1 个时间步。整条流水共需要 $M + P - 1 = 7$ 步：

时间 →	T=0	T=1	T=2	T=3	T=4	T=5	T=6
卡 0 (layer 0-19)	mb1	mb2	mb3	mb4	—	—	—
卡 1 (layer 20-39)	—	mb1	mb2	mb3	mb4	—	—
卡 2 (layer 40-59)	—	—	mb1	mb2	mb3	mb4	—
卡 3 (layer 60-79)	—	—	—	mb1	mb2	mb3	mb4
阶段	Fill	Fill	Fill	Steady	Drain	Drain	Drain

格子里的 mb1...mb4 表示"那个时刻那张卡正在跑哪个 micro-batch"；— 是 bubble（空闲）。

怎么读这张表：

- **横着读一行** = 这张卡的时间线。卡 0 从 T=0 一直忙到 T=3，之后闲了 3 步。卡 3 反之，前 3 步全闲。
- **竖着读一列** = 这个时刻全集群在干嘛。T=3 是唯一全员满载的时刻。
- **斜着读对角线** = 同一个 micro-batch 的旅程。mb1 从 T=0 卡 0 → T=1 卡 1 → T=2 卡 2 → T=3 卡 3，逐段往下传 hidden state。

三阶段的步数 + bubble 数

阶段	步数	这几步里有几张卡在 bubble	累计 bubble 格
Fill （流水填充）	$P - 1 = 3$	T=0:3, T=1:2, T=2:1	6
Steady （满载）	$M - (P - 1) = 1$	T=3:0	0
Drain （流水排空）	$P - 1 = 3$	T=4:1, T=5:2, T=6:3	6
总计	$M + P - 1 = 7$	—	12

总格子数 = 7 步 × 4 卡 = 28。其中 bubble 格 = 12。

Bubble 占比公式（直接从表格读出来）

$$\text{bubble ratio} = \frac{\text{bubble 格数}}{\text{总格数}} = \frac{2 \times P \times (P-1) / 2}{P \times (M + P - 1)} = \frac{P - 1}{M + P - 1}$$

（分子是上下两个三角形 bubble 区，各 $P \times (P-1)/2$ 格，合起来 $P(P-1)$ 格。）

代不同的 M 验证一下：

P	M	bubble 格	总格	bubble 比
4	4	12	28	43%
4	16	12	76	16%
4	64	12	268	4.5%
8	8	56	120	47%
8	64	56	568	9.9%

两个观察：

- bubble 格数只跟 P 有关，跟 M 无关**（永远是 $P \times (P-1)$ ，因为左上和右下两个三角形）。所以**减小 bubble 唯一的办法是把 M 做大**——分子摊薄到大分母。
- PP=4 + M=4 时 43% 算力被浪费**。这就是为什么生产里 PP 必须配合 $\text{max_num_seqs} \geq 32-64$ ：让 M 远大于 P，bubble 才能压到 10% 以下。

→ **PP 强烈受 batch 大小制约**：低并发场景 bubble 巨大，PP 反而比纯 TP 慢。Agent / code completion 这种"低并发 + 延迟敏感"工作负载请走纯 TP，不要 PP。

3.4 通信代价

每段边界一次 P2P，传一个 [B, H] tensor：

```
per-boundary bytes = B × H × dtype_bytes
total bytes / forward = (P - 1) × B × H × dtype_bytes
```

vs TP 的通信量： $2L \times B \times H \times \text{dtype_bytes}$ （每层 2 次 AllReduce）。

Llama-3-70B PP=4 vs TP=4 通信对比（每个 forward）：

维度	TP=4	PP=4
通信原语	AllReduce（每层 2 次）	P2P send/recv（每边界 1 次）
总次数	$80 \times 2 = 160$	3
单次数据 / token	$8192 \times 2 = 16 \text{ KB}$	$8192 \times 2 = 16 \text{ KB}$
总数据 / token	2.56 MB	48 KB
带宽要求	必须 NVLink/IB（高频小包）	普通以太网够（低频中包）

→ **PP 的核心优势：通信稀疏，能跨机**。这是它在多机推理里不可替代的原因。

3.5 vLLM 里 PP 的实现

- 每个 PP rank 一个 Worker 进程**（同 TP rank 一样的多进程模型）

- **KV cache 按层归属**：rank i 只为它负责的 L/P 层分配 BlockPool。Block 总数随每个 rank 持有的层数缩放
- **Scheduler 还是单实例**（在 PP rank 0 上跑），通过广播 SchedulerOutput 给所有 rank
- **get_pp_group() / get_world_group()** 提供进程组抽象（vllm/distributed/parallel_state.py）
- **send/recv 的实现** 在 vllm/v1/worker/gpu_model_runner.py 的 execute_model 段边界

启用：--pipeline-parallel-size 4，可与 --tensor-parallel-size 组合（如 TP=4, PP=2 用 8 卡跨 2 机）。

3.6 适用场景

场景	推荐
单机 8 卡塞得下 + 单 query 延迟敏感（agent / code completion）	纯 TP （PP 会引入 bubble + send/recv 串行延迟）
单机塞不下，但有 IB / RoCE 多机	TP=单机卡数 × PP=机器数 （典型 TP=8, PP=2/4）
单机塞不下，机器间只有以太网	PP 跨机 （TP 跨以太网会被通信打死，PP 还能跑）
离线高吞吐 batch 推理	大 M（高并发请求）+ PP — bubble 摊薄

4. Expert Parallel（EP）：MoE 专用

本节是 EP 概览。**完整的 6 个 AllToAll backend 对比、EPLB 在线自适应、wide-EP 部署模式**详见 03-expert-parallel-deep-dive.md。

4.1 MoE 是什么

Mixture of Experts 把 dense FFN（一个大的 [H, I] 投影）换成 **E 个独立的 expert FFN + router**。每个 token 经 router 选 top-k 个 expert，加权求和。

模型	num_experts (E)	top-k	形态
Mixtral 8x7B	8	2	每 token 激活 2/8
Qwen2-MoE	64	8	64 expert + 4 shared
DeepSeek-V3	256 routed + 1 shared	8	wide expert pool
GPT-OSS	32	4	NVIDIA 风格

参数量随 E 线性增长，但激活算力只按 k 收费（"参数大 + 算力省"）。

4.2 张量形状与单层公式

未切分（单卡）：

$x_{\text{pack}} : [B, H] \leftarrow B \text{ 是 packed token 数}$
 $W_{\text{router}} : [H, E]$
 $\text{router_logits} : [B, E] = x_{\text{pack}} \cdot W_{\text{router}}$
 $\text{top_k_indices} : [B, k] \leftarrow \text{argmax_k}(\text{softmax}(\text{router_logits}))$
 $\text{top_k_weights} : [B, k] \leftarrow \text{对应权重, 已归一化}$

对每个 token t , 它选中的 k 个 expert 各自跑一遍 SwiGLU MLP:
 $\text{expert_e}(x_t) = (\text{silu}(x_t \cdot W_{\text{gate_e}}) * (x_t \cdot W_{\text{up_e}})) \cdot W_{\text{down_e}}$
其中 $W_{\text{gate_e}}, W_{\text{up_e}} : [H, I_{\text{expert}}], W_{\text{down_e}} : [I_{\text{expert}}, H]$

最终:
 $y[t] = \sum_{e \in \text{topk}(t)} \text{weight_e} \cdot \text{expert_e}(x_t)$ 形状 $[B, H]$

注意: 每个 expert 有自己一套 $W_{\text{gate}}/W_{\text{up}}/W_{\text{down}}$, 所以**全部 expert 的权重 = E × 单 expert 权重**。DeepSeek-V3 256 个 expert × 28 MB ≈ 7 GB 仅 MoE 层（每层！）。

4.3 EP=N 切分: 每卡持有 E/N 个 expert

每张卡只放它负责的那部分 expert 的完整权重, **不切单个 expert 内部**。

Dispatch (AllToAll #1)

每卡持有:
 本卡的 $x_{\text{local}} : [B/N, H] \leftarrow$ 假定 token 在 rank 间均匀分布
 本卡的 expert 子集 (E/N 个) 的所有 weight
router 算 top-k 后, 每个 token 标记 "我要去 expert_e0, expert_e1, ..., expert_ek-1"
对应的 owner_rank = e_id // (E/N)
AllToAll 把每个 token 飞到它的 top-k expert 所在的 rank:

rank r 收到的 tokens : $[n_r, H] \leftarrow n_r$ 由 router 决定, 不均匀!
 理想 $n_r \approx B \cdot k / N$, 实际偏离
 (hot expert 让某些 rank 拥堵)

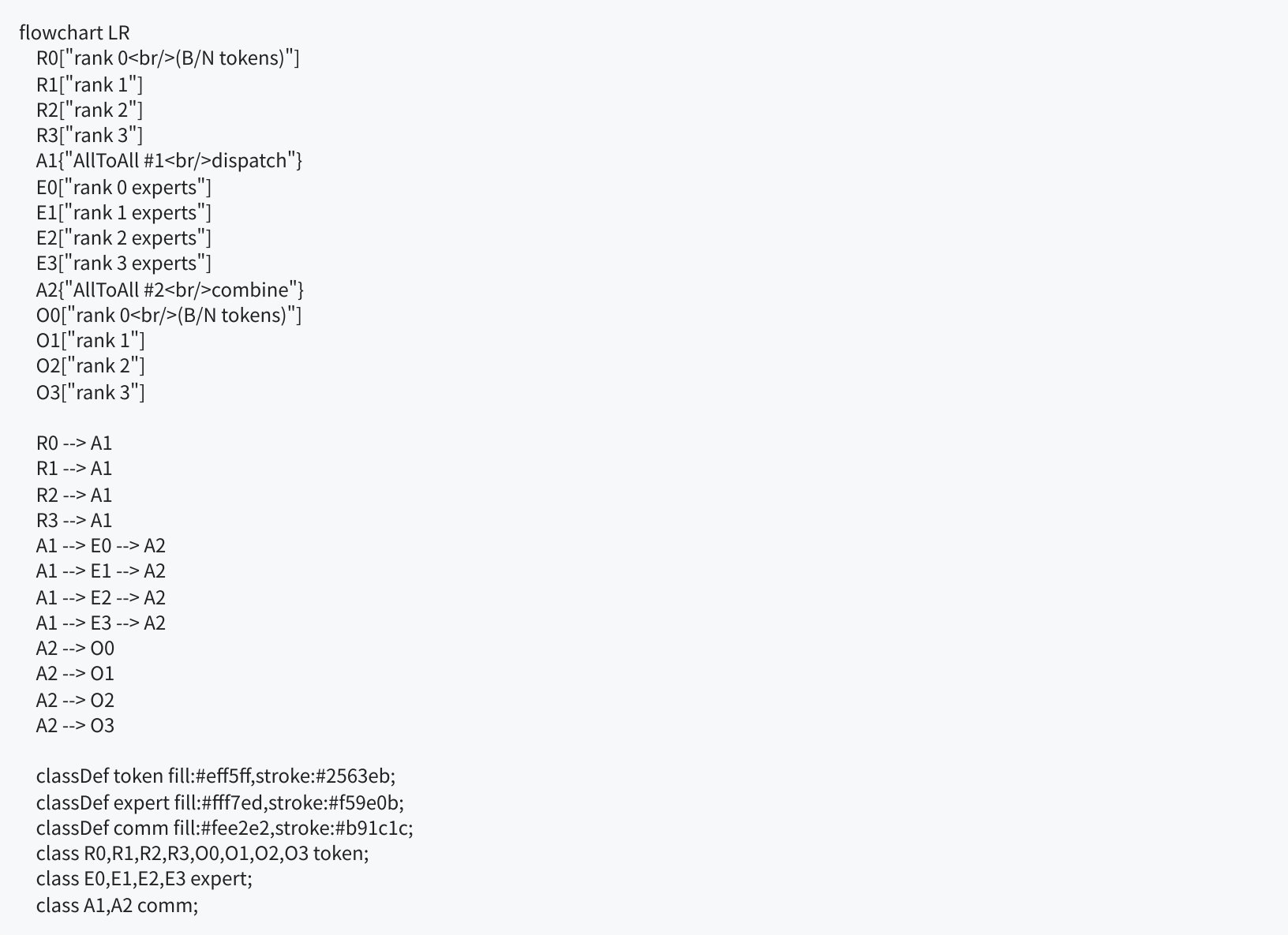
Compute (每卡独立跑自己的 expert)

rank r 上:
 对收到的每个 token、每个 owned expert e, 跑 expert_e(x) → 输出 $[n_r, H]$

Combine (AllToAll #2)

每张卡上 expert 算完的结果按"这个 token 来自哪个 rank"飞回去:
 rank r 收回 : $[B/N, k, H] \leftarrow$ 同一个 token 的 k 个 expert 结果
本地加权求和:
 $y_{\text{local}} : [B/N, H] = \sum_{i=0}^{k-1} \text{weight_i} \cdot \text{expert_output_i}$

关键: AllToAll vs AllReduce. AllReduce 把所有 rank 的同形 tensor 逐元素 sum; AllToAll 是**所有 rank 之间互相交换分片**——rank r 把它的第 r' 片发给 rank r' , 同时从所有 r' 拿回它们的第 r 片。两次 AllToAll 围住 expert compute 是 EP 的核心 pattern。



4.4 通信代价

每层每次 forward：2 次 AllToAll。

AllToAll 数据量比 AllReduce 大得多（每对 rank 都要传数据）。Llama-3 dense 层每层 2 次 AllReduce = $2 \times B \times H \times \text{dtype_bytes}$ ；MoE 层每层 2 次 AllToAll 单次传输近似 $B \times k \times H \times \text{dtype_bytes}$ （带 hot expert 失衡时更高）。

80 层 MoE 模型一次 forward = **160 次 AllToAll**。这就是 DeepSeek 必须写自家 DeepEP kernel 的原因——通用 NCCL AllToAll 在大 MoE 上是真实瓶颈。

详细 backend 选型（DeepEP HT/LL、FlashInfer、Mori、NIXL EP 等 6 种）见 03-expert-parallel-deep-dive.md § 4。

4.5 Load imbalance：hot expert 与 EPLB

理论上 router 均匀分配，实际**5-10 个 hot expert 吃掉 30-50% token**。表现：

- nvidia-smi：某些 GPU 100%，另一些 30-50%
- vllm:iteration_tokens_total 方差大

EPLB (Expert Parallel Load Balancer)：把 hot expert **复制到多 rank**（一 expert 多 owner），让 router 在多个副本里选。代价是显存（多副本）+ 路由复杂度。

DeepSeek-V3 用 256 logical → 288 physical（加 32 副本）。配 `--enable-eplb`。源码：`vllm/distributed/eplb/`。

4.6 关键约束

- $E \% N == 0$ （expert 数得能被 EP rank 数整除；否则 EPLB padding 凑齐）
- EP 需配合 expert parallel 启用：`--enable-expert-parallel`
- EP 与 TP 可共存： $ep_size = TP \times DP \times PCP$ （详见 deep-dive）

5. Data Parallel（DP）

DP 概念最简单：模型权重复制 N 份，每份独立处理一批请求。但 vLLM 里 DP 有**两种截然不同的语义**，搞混会出问题。

5.1 两种 DP 语义

模式 A • 纯副本 DP（Pure Replica）

DP rank 0: 完整模型权重一份 + 独立 KV cache + 独立 Scheduler
DP rank 1: 完整模型权重一份 + 独立 KV cache + 独立 Scheduler
...
DP rank N-1: 同上

每个 rank（含它内部的 TP 组）持有**完整模型副本**。请求维度（batch）在 rank 之间分配。**rank 之间不通信**（forward 阶段）。吞吐近似线性扩展。

适合 dense 模型（Llama / Qwen-dense）做高并发服务。

模式 B • DP for MoE（DP 维度参与 EP 切分）

EP 启用时， $ep_size = TP \times DP \times PCP$ 。这种"DP+EP"组合让 expert 分布更细粒度——典型的"wide-EP"部署。

TP=8, DP=4 → $ep_size=32$
DeepSeek-V3 (256 expert) → 每张卡 8 expert

这种模式下 DP rank 之间**必须通信**（MoE 的 AllToAll 跨 DP），因为 expert 分布跨 DP 维度。

→ **怎么区分自己在哪种模式**：看是否加了 `--enable-expert-parallel`。加了就是 B，没加就是 A。

5.2 纯副本 DP 的张量形状

完全没有跨 rank 通信，每个 rank 等价于一个独立 vLLM 实例：

rank 0:
batch_0 = [request_0, request_1, ...] ← 分给 rank 0 的请求
完整 forward 在 rank 0 内部完成（含其 TP 组）

rank 1:
batch_1 = [request_N, ...]
完整 forward 独立完成

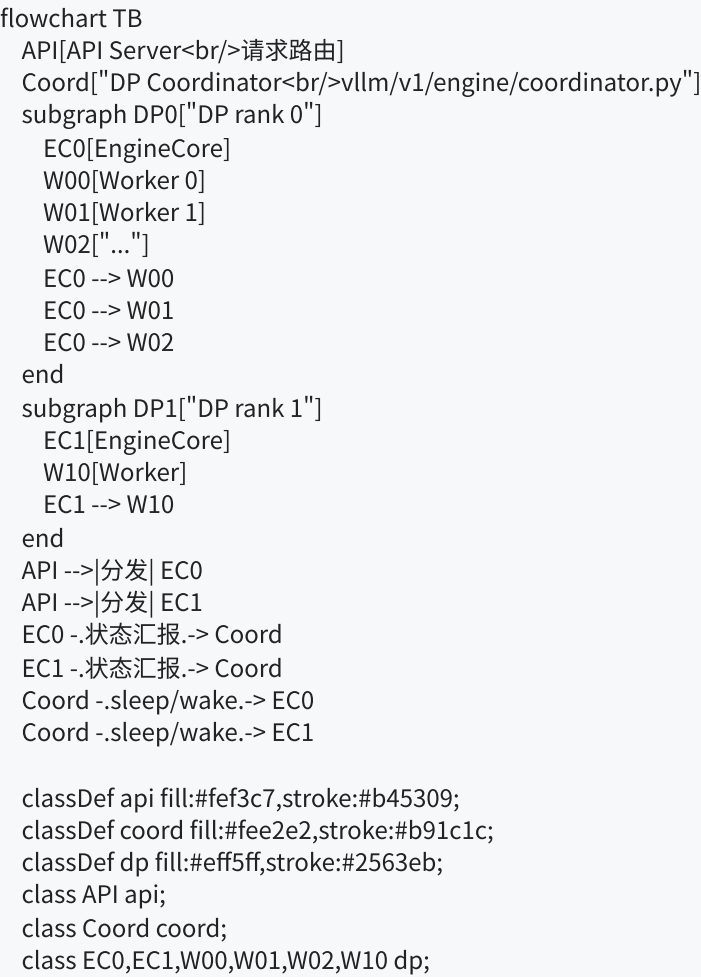
...

跨 rank 通信：0 ← 真的零
跨 rank 共享：除了模型权重（各 rank 都有一份），其他全独立

关键副作用：prefix cache 不跨 rank 共享。同 prompt 打到 rank 0 和 rank 1 会各自全量 prefill。需要在**外部 router 层做 cache-aware 路由**让相同 prefix 的请求打到同一 rank（详见 08-production-deployment/02-smart-routing-and-load-balancing.md）。

5.3 vLLM 里 DP 的实现

每个 DP rank 一个 **独立的 EngineCore 进程**，外加一个 **Coordinator 进程**协调全局状态：



Coordinator 的职责（不做 forward 协同）：

- 监控每个 DP rank 的负载（running / waiting 请求数）
- 协调全局 sleep/wake（elastic EP 场景下空闲 rank 可挂起）
- 处理 EP 启用时跨 DP rank 的 group 初始化

源码：vllm/v1/engine/coordinator.py。

5.4 External LB 模式（DP+K8s 的典型部署）

--data-parallel-external-lb：每个 DP rank 一个独立 K8s pod，前面靠外部 LB（Istio/Envoy）分发请求，不依赖 vLLM 内置 Coordinator。

适合 wide-EP 场景：

- 一 pod 一 rank → 扩缩容粒度细
- pod 间通过 IB/RoCE 跑 EP AllToAll
- 外部 LB 做 cache-aware / lora-aware routing

仅 **MoE 模型推荐**。dense 模型用纯多实例（每个 vLLM 进程独立）就够了，不必拉 external LB 那一套。

5.5 world_size 与组合公式

普通 executor 内的 worker world `= TP × PP × PCP`；跨 DP 总规模 `world\_size\_across\_dp = worker world × DP`。external launcher 的进程 world 定义不同，必须读取解析后的配置。

EP 启用时额外约束：
ep_size = TP × DP × PCP
（PP 不计入 EP，因为 PP 各段持有独立 expert 集合）

DCP（decode context parallel）复用 TP GPU，不增 worker world；还要求 TP 能被 DCP 整除。

5.6 8 卡 H100 的真实组合矩阵

以 DeepSeek-V3 为例：

配置	world_size	ep_size	单 query 延迟	吞吐	备注
TP=8, DP=1	8	8	中	中	经典单实例
TP=1, DP=8	8	8	高（无 TP 加速）	高	每张卡一副本，但 DeepSeek-V3 模型大放不下
TP=4, DP=2	8	8	中	中-高	折中：2 副本 × 每副本 4 卡 TP
TP=8, PP=1, DP=1	8	8	低（TP 加速）	低	单 query 最快
跨机 TP=8, PP=2	16	8	中	高	模型横跨 2 机
跨机 TP=8, DP=2	16	16	中	极高	wide-EP，2 机各一副本

启用参数对应：

```
--tensor-parallel-size 8
--pipeline-parallel-size 2
--data-parallel-size 2
--enable-expert-parallel
--data-parallel-external-lb
```

6. Sequence Parallelism (SP): TP 的"半免费"伴侣

SP 是个常被忽略但默认就开的优化。它**附属于 TP**，不增 world size，但能省下一部分冗余计算。

6.1 SP 在哪里发挥作用

TP 切 Linear 层，但没切 **LayerNorm / Dropout / Residual** 这些 element-wise 的 op。这些 op 在每张 TP rank 上都跑了**完整的 [B, H]**——但每张 rank 都算一样的，浪费 TP-1 份算力。

SP 把这些 element-wise op 也按 seq 维切到不同 rank：

```
TP-only:
rank 0: x [B, H] → LN [B, H] → Linear(切了的)
rank 1: x [B, H] → LN [B, H] → Linear(切了的) ← 重复算 LN
...
rank N: x [B, H] → LN [B, H] → Linear(切了的) ← 重复算 LN

TP + SP:
rank 0: x [B/N, H] → LN [B/N, H] → AllGather → Linear(切了的)
rank 1: x [B/N, H] → LN [B/N, H] → AllGather → Linear(切了的)
...
AllGather + ReduceScatter 总通信量与原 AllReduce 等价（NCCL 性质）
```

收益：

- LayerNorm / Dropout / residual add 各省 (N-1)/N 份算力
- LN 在大 seq 时占 5-15%（不算大但白省）
- 中间激活 [B, H] 在 rank 上变成 [B/N, H] → **显存省 ~33%**

代价：

- 引入 AllGather + ReduceScatter 替代 AllReduce（总通信量近似相等）
- 实现复杂度 ↑

6.2 vLLM 里的 SP

- 默认**自动开启** when TP > 1（V1 大部分模型已经做了 SP 改造）
- 实现入口：vllm/compilation/passes/fusion/sequence_parallelism.py（torch.compile pass）+ 模型代码里的 SequenceParallel* 层
- 关掉：compilation_config={"pass_config":{"enable_sp": false}}

详细 PR：vllm-project/vllm#24134（DeepSeek SP）和 #24982（推广到其他模型）。

6.3 与 AsyncTP / SP-MoE 的关系

- **AsyncTP**（async tensor parallel）：让 AllGather 与下一个 GEMM overlap，能再省 5-10%
- **SP-MoE**（sequence parallel + MoE）：当 MoE 与 SP 共存时，需要专门的 compile pass 处理 RMSNorm + AllReduce + chunk 的 fusion；vllm-project/vllm#29139 在做（详见 02b-scheduling-policies.md 引用）

7. 集合通信原语速查

vLLM 在不同并行场景调用 5 种 NCCL 原语。一张表对完：

原语	语义	数据量 / rank	vLLM 用在哪	通信成本
AllReduce	所有 rank 同形 tensor 逐元素聚合（默认 SUM），结果广播给所有	$2(N-1)/N \times \backslash \text{tensor} \backslash $	TP 的 Row Parallel 末尾（每层 2 次）；DP 训练时梯度同步（推理无）	中
AllGather	所有 rank 各持有 1 段 tensor，拼成完整后广播给所有	$(N-1)/N \times \backslash \text{拼后 tensor} \backslash $	SP 的 LN→Linear 边界；EP dispatch 的一种实现（AgRs）	中
ReduceScatter	AllReduce 的"前半"：聚合后只把第 r 段给 rank r	同 AllGather	SP 的 Linear→LN 边界；与 AllGather 配对替代 AllReduce	中
AllToAll	所有 rank 互相交换分片：rank r 把第 r' 段发给 rank r'，同时收	$(N-1)/N \times \backslash \text{tensor} \backslash $ ，但拓扑全连接	EP 的 dispatch 和 combine（每 MoE 层 2 次）	重（全连接）
Broadcast	rank 0 的 tensor 广播给所有 rank	$\backslash \text{tensor} \backslash $	scheduler 把 SchedulerOutput 元数据广播给所有 worker；KV cache hash 命中结果广播	轻
P2P Send/Recv	一对一传输	$\backslash \text{tensor} \backslash $	PP 的段边界传 hidden state	极轻

通信量关系：

- AllReduce $\equiv$ ReduceScatter + AllGather（NCCL ring 算法的实际实现就是这样）
- AllToAll 数据量与 AllReduce 同，但**实现复杂度更高**（每对 rank 都要建独立通道），所以 DeepEP / FlashInfer 才有用武之地

vLLM 调用入口：

- AllReduce / AllGather / ReduceScatter：vllm/distributed/communication_op.py
- AllToAll：vllm/distributed/device_communicators/all2all.py（6 个后端）
- Broadcast：vllm/distributed/parallel_state.py::broadcast_tensor_dict
- P2P：vllm/distributed/p2p_communicator.py

8. 组合策略：按模型/场景的真实配置

8.1 单维度配置矩阵（8 卡 H100）

模型	模型大小	推荐配置	启动命令	备注
Llama-7B / 8B	16 GB	DP=8（独立 8 实例）	8 个 vllm serve ... --port N	模型小，并发优先
Llama-13B	26 GB	DP=4 (TP=2 × 4 副本)	--tp 2（×4）	看 KV 大小决定切几份
Llama-70B (BF16)	140 GB	TP=8	vllm serve ... --tp 8	单机内 TP 极致
Llama-70B (AWQ INT4)	35 GB	TP=4 + DP=2	--tp 4 × 2 实例	量化省显存 → 多副本提吞吐
Llama-405B	800 GB	TP=8 + PP=2（跨机）	2 机 × --tp 8 --pp 2	单机塞不下，跨机 PP
Mixtral 8×7B	90 GB	TP=2 + EP=4 + DP=2	--tp 2 --enable-ep	TP 减 MLP 通信，EP 切 expert
DeepSeek-V3 (671B)	1.3 TB	TP=8 + EP=32 + DP=4（多机）	4 机 × --tp 8 --enable-ep --dp 4	wide-EP，必须 IB/RoCE
Qwen2-VL 72B	145 GB	TP=4 + DP=2 + 加 encoder_cache	--tp 4 --max-mm-embeds 100	多模态 KV ≈ 文本 KV × 2
Llama-3-70B 长上下文 128K	140 GB	TP=8 + DCP=2	--tp 8 --dcp 2	DCP 减 KV，复用 TP GPU

8.2 启动命令对照（单维度 → 5 维）

单实例单卡 (最简)

```
vllm serve meta-llama/Llama-3-8B
```

纯 TP (同机 8 卡)

```
vllm serve meta-llama/Llama-3-70B \
  --tensor-parallel-size 8
```

TP + PP (跨机)

机 1 (rank 0-7)

```
vllm serve meta-llama/Llama-3-405B \
  --tensor-parallel-size 8 \
  --pipeline-parallel-size 2 \
  --distributed-executor-backend ray \
  --node-rank 0
```

机 2 (rank 8-15)

```
vllm serve ... --node-rank 1
```

TP + EP (MoE)

```
vllm serve deepseek-ai/DeepSeek-V3 \
  --tensor-parallel-size 8 \
  --enable-expert-parallel \
  --all2all-backend deepcp_high_throughput
```

TP + DP + EP (wide-EP, K8s 外部 LB)

```
vllm serve deepseek-ai/DeepSeek-V3 \
  --tensor-parallel-size 8 \
  --data-parallel-size 4 \
  --enable-expert-parallel \
  --data-parallel-external-lb \
  --all2all-backend deepcp_low_latency
```

长上下文 + DCP (复用 TP)

```
vllm serve meta-llama/Llama-3-70B \
  --tensor-parallel-size 8 \
  --decode-context-parallel-size 2 \
  --dcp-comm-backend a2a # MLA 模型用 a2a, 普通模型用 ag_rs
```

5 维全开 (理论最极端)

```
vllm serve big-moe-model \
  --tensor-parallel-size 8 \
  --pipeline-parallel-size 2 \
  --data-parallel-size 2 \
  --enable-expert-parallel \
  --decode-context-parallel-size 2 \
  --prefill-context-parallel-size 2
```

world_size = 8 × 2 × 2 × 2 = 64 卡

ep_size = 8 × 2 × 2 = 32

8.3 配置取舍的 5 条经验法则

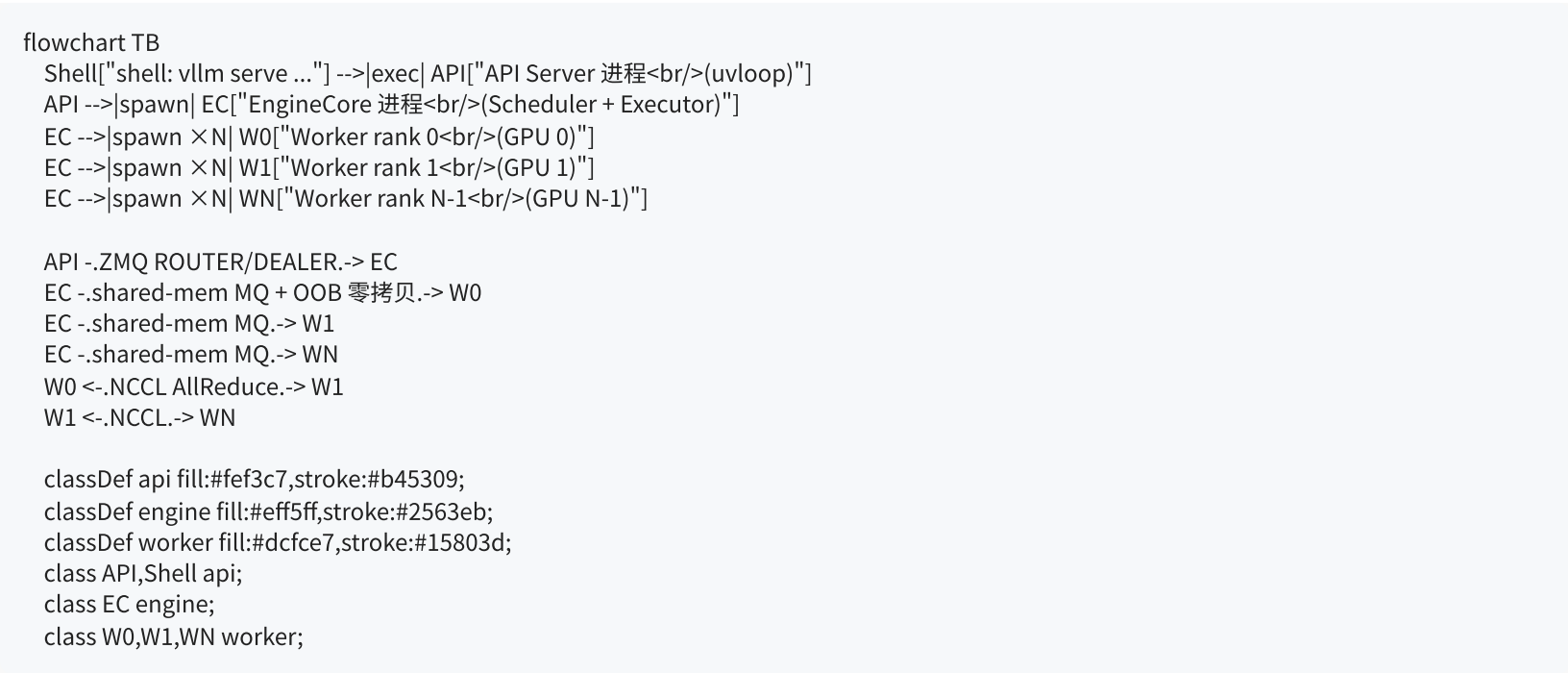
1. **同机优先 TP**: NVLink 内 AllReduce ~10 μs, 跨机 IB 100+ μs, 差 10×
2. **跨机优先 PP**: 通信稀疏 (每边界 1 次 P2P) + 数据小 (B × H 字节)
3. **MoE 必加 EP**: 纯 TP 跑 MoE 会让 router 成瓶颈 + 算力浪费在未被激活的 expert 切片
4. **吞吐优先 DP**: DP 几乎线性扩展, DP=N 翻 N 倍吞吐 (除了 prefix cache 命中率分散)
5. **长上下文加 DCP**: DCP 不增 GPU 但减 KV 占用, 能装下更长 prompt

8.4 反模式 (避免)

❌ 别这样	为什么	改成
跨机 TP（如 TP=16 跨 2 机）	IB 100+ μ s $\times$ 160 次 AllReduce = forward 加几十 ms	TP=8 单机 + PP=2 跨机
纯 TP 跑 MoE	router 算力浪费 + 每张卡都加载所有 expert 切片	TP + EP 组合
极大 PP（PP=16）+ 低并发	bubble = $15/(M+15)$ ，M<10 时几乎全 bubble	减小 PP，加 TP 或 DP
DCP 跑 Mamba 模型	Mamba 的 SSM state 不能按 seq 切	Mamba 不支持 CP，硬要长上下文得换模型
--data-parallel-size N 不配 --enable-expert-parallel 的 dense 模型	不如直接起 N 个独立 vLLM 进程（更简单、故障域更小）	起 N 实例 + 路由层

9. 进程模型与启动流程

9.1 单机多卡进程结构



进程数关系：

- 单机 TP=N： **2 + N** 进程（API + EngineCore + N Worker）
- DP=D： 每个 DP rank 一组上述结构，总 **D $\times$ (2 + N) + 1** 进程（多一个 Coordinator）
- PP=P： 每 PP rank 一个 Worker（同机），跨机时每机一组上述结构

9.2 NCCL 进程组初始化

源码：vllm/distributed/parallel_state.py::initialize_model_parallel

```
# 总 world: world_size = TP × PP × DP × PCP
torch.distributed.init_process_group(backend="nccl", ...)
```

```
# 给每个 rank 算 (tp_rank, pp_rank, dp_rank, pcp_rank) 4 维坐标
# 然后建多个 ProcessGroup:
tp_group = dist.new_group(ranks=[]) # 同 pp/dp/pcp 的所有 TP rank
pp_group = dist.new_group(ranks=[]) # 同 tp/dp/pcp 的所有 PP rank
dp_group = dist.new_group(ranks=[]) # 同 tp/pp/pcp 的所有 DP rank
ep_group = dist.new_group(ranks=[]) # ep_size = tp × dp × pcp 的所有 rank
world_group = ... # 所有 rank
```

每个 ProcessGroup 内独立调 AllReduce / AllGather / AllToAll 等 collective，不互相干扰。

启动顺序的关键（多机部署常踩坑）：

- 1. 所有机器上的 EngineCore + Worker 必须**同时启动**（NCCL handshake 要求）
- 2. rank 0（driver）先发"我是 master，监听 port X"
- 3. 其他 rank 用 master 的 IP:port 加入
- 4. 任一 rank 启动失败 → 全组 hang（必须 K8s LeaderWorkerSet）

9.3 Driver Worker vs 普通 Worker 职责

职责	Driver (rank 0)	普通 Worker
接收 SchedulerOutput	✓（独占）	☒（被 broadcast 给）
算 prefix cache hash	✓（不依赖 KV 数值）	☒
采样 next_token	✓（避免不同 rank RNG 差异）	☒（参与 forward 但不 sample）
forward 执行	✓	✓
AllReduce 参与	✓	✓
输出回传 EngineCore	✓	☒

判断代码：grep is_driver_worker 或 tp_rank == 0。

9.4 跨机器：Ray Distributed Executor

vllm/v1/executor/ray_executor_v2.py 用 Ray 替代 multiprocessing。差异：

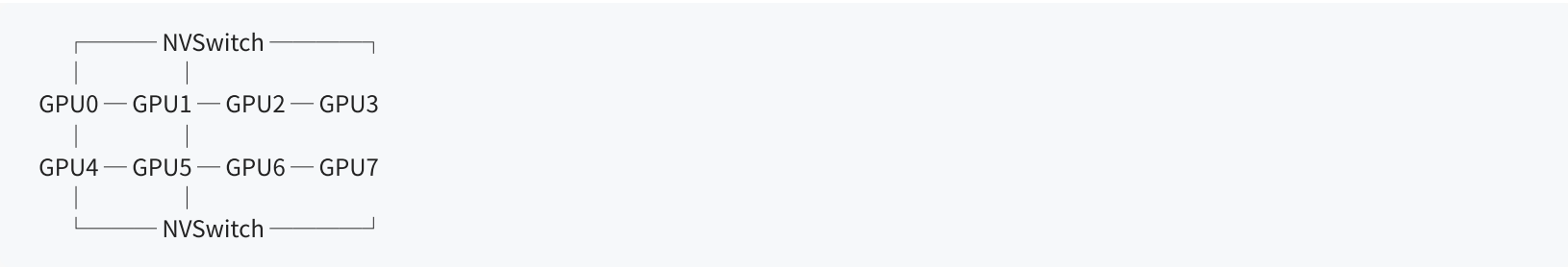
- Worker 是 Ray actor（跨机器调度由 Ray 管）
- 跨机器 IPC 走 Ray object store（gRPC），不再是 shared memory
- Ray 自带健康监控，但 vLLM 还是要 LeaderWorkerSet 整组重启的逻辑

启动：--distributed-executor-backend ray。

10. NCCL 拓扑感知与硬件要求

10.1 单机内 NVLink 拓扑

H100 8 卡机典型拓扑（NVLink 4 + NVSwitch）：



NVLink 4.0 速度：**单方向 900 GB/s** per GPU 对（H100），所有 GPU 互连。

关键：用 `nvidia-smi topo -m` 看实际拓扑：

```
GPU0 GPU1 GPU2 GPU3 GPU4 GPU5 GPU6 GPU7
GPU0  X  NV18 NV18 NV18 NV18 NV18 NV18 NV18
GPU1  NV18  X  NV18 NV18 NV18 NV18 NV18 NV18
...
```

NV18 表示有 NVLink 直连，理想情况所有 GPU 对都是 NV18。如果出现 sys（走 CPU/PCIe）→ NVLink 没正确连，TP 性能会糟糕。

10.2 跨机器：IB / RoCE / 以太网

网络	单向带宽	时延	适合
InfiniBand HDR (400G)	50 GB/s	< 2 μs	TP+PP 跨机首选；NCCL 原生支持
RoCE v2 (400G)	50 GB/s	3-5 μs	RDMA over Ethernet，比 IB 便宜，需要 lossless 网络
100Gbps 以太网	12.5 GB/s	50-100 μs	只能跑 PP，TP 跨机会被打死
10Gbps 以太网	1.25 GB/s	100+ μs	不可用（即使 PP 也卡）

NCCL 关键环境变量：

```
# IB 配置
export NCCL_IB_HCA=mlx5_0:1,mlx5_1:1 # 指定使用的 IB 卡
export NCCL_IB_GID_INDEX=3          # RoCE v2 需要
export NCCL_IB_TC=106               # IB 流量类别

# 调试 / 性能
export NCCL_DEBUG=INFO              # 打印 NCCL 详细 log
export NCCL_DEBUG_SUBSYS=ALL        # 包括拓扑、连接细节
export NCCL_TOPO_DUMP_FILE=/tmp/topo.xml # dump 拓扑
export NCCL_ASYNC_ERROR_HANDLING=1  # async 模式下错误处理

# 安全 (K8s mTLS 场景)
export NCCL_SOCKET_IFNAME=^lo,docker0 # 排除 loopback / docker
```

10.3 多 NUMA 节点

CPU 双路服务器（多 NUMA）下，每张 GPU 通过 PCIe 挂到某一颗 CPU。Worker 进程跑在哪颗 CPU 上影响 H2D / D2H 性能：

```
# 启动 Worker 时绑核
numactl --cpunodebind=0 --membind=0 vllm_worker_for_gpu_0
```

vLLM 已经支持 --numa-bind 参数自动绑核（V1+）。

11. 故障模式与恢复

11.1 各并行维度的故障传染

维度	一张卡挂了影响范围	恢复方式
TP	整个 TP 组全挂（NCCL collective 死锁）	整组重启（LeaderWorkerSet）
PP	整个 PP 链全挂（中间段死 → 前后传送 hang）	整组重启
EP	整个 EP 组全挂（AllToAll 死锁）	整组重启
DP（纯副本）	只挂这一个 DP rank，其他 rank 正常	单独重启该 rank pod
DP+EP	当作 EP 处理 → 整 EP 组挂	整组重启
DCP	复用 TP GPU，与 TP 共命运	同 TP

→ DP 是唯一故障域不传染的维度（仅纯副本场景）。高可用要求高的部署偏好 DP 多实例。

11.2 K8s 配套

部署模式	K8s 抽象
单机 TP=8	LeaderWorkerSet（1 组）
跨机 TP=8 + PP=2	LeaderWorkerSet（1 组，跨 2 机）
DP=4 × TP=8	4 个 LeaderWorkerSet（独立故障域）
DP+EP wide-EP	全部当 EP 组：1 个超大 LeaderWorkerSet

11.3 常见故障 + 对应 metric

故障	症状	metric / 命令
NCCL hang	GPU util 0, num_requests_running > 0	py-spy dump --pid，看是否卡在 c10d.work.wait()
单卡 ECC error	该 rank forward 突然慢 / 出错	nvidia-smi -q -d ECC
NVLink 错误	TP 突然慢 5-10×	nvidia-smi nvlink -e
IB 链路抖	跨机 PP/TP 性能波动	ibstat、ibv_devinfo
OOM during forward	Worker 进程 OOMKilled	dmesg grep oom
Preempt 风暴	TPOT 抖到 1s+	rate(vllm:num_preemptions_total[5m])

详见 08-production-deployment/07-incident-playbook.md。

12. 工程自检问答

Q: TP 通信比 PP 重，为什么不全用 PP？ A: PP 有 bubble，推理 batch 通常不够大（M < 32），bubble 占 30%+。TP 在 NVLink 内 AllReduce 仅 10 μs，单机 8 卡就是 TP 主场。跨机才是 PP 主场。

Q: 跨机器 TP 行不行？ A: 极不推荐。AllReduce 频繁（每层 2 次 × 80 层 = 160 次）且每次 KB 级数据，IB 也比 NVLink 慢 5-10×，单 token forward 多 10-15 ms，延迟敏感场景不可接受。

Q: KV cache 在 TP 下怎么分？ A: KV head 按 TP rank 切（每卡负责 num_kv_heads / TP 个 head）。每卡的 KV cache 是 total_kv / TP。Block table 形状一致，但内容是各卡自己的物理 block id。

Q: 怎么算一个模型的最优 TP/PP/EP？ A: 启发式 5 步：

- 1. 显存够不够：模型 / 8 + KV 估算
- 2. 同机优先 TP（NVLink 带宽好）
- 3. 跨机用 PP（通信少）
- 4. MoE 必加 EP（不然 router 是瓶颈）
- 5. benchmark 验证

Q: DP=8（独立 8 实例）vs TP=8（单实例）哪个吞吐高？ A: 多数 dense 场景 DP=8 高（线性扩展，无通信开销）。但有两个例外：

- 1. **prefix cache 命中率**高的 workload：TP=8 集中 cache，命中率显著高于 DP=8；可能反超
- 2. **单 query 延迟敏感**：DP 单 query 在一卡上跑 = 慢；TP 单 query 8 卡并行 = 快

Q: EP=8 + AllToAll 比 TP=8 + AllReduce 通信多吗？ A: 数据量类似（理论 $2(N-1)/N \times |\text{tensor}|$ ），但 **AllToAll 拓扑全连接**（每对 rank 都要建通道，setup 开销高），实际比 AllReduce 慢 1.5-2×。这就是为什么 DeepSeek 写 DeepEP——通用 NCCL AllToAll 在大 MoE 上是真瓶颈。

Q: PCP 和 chunked prefill 是不是互替关系？ A: 互补，不互替。

- chunked prefill：**时间维度**切（一次 forward 跑一段，多次完成）
- PCP：**空间维度**切（一次 forward 多卡并行，每卡一段）
- 超长上下文 prefill 塞不下时上 PCP；正常场景 chunked prefill 即可

Q: 为什么 vLLM 推理不需要 1F1B 这种 PP 调度? A: 1F1B 是为了让训练 backward 与下一个 micro-batch forward overlap, 缓解 bubble。推理只有 forward, 自然 "just forward all" —— 朴素流水线就够。简单 forward fill schedule 已足够。

Q: vLLM 在 DP+EP 下怎么协调多 DP rank 的 schedule? A: 每个 DP rank 一个独立 EngineCore + 独立 Scheduler, **不互相同步 schedule**。但 EP 的 AllToAll 跨 DP rank → 要求所有 DP rank 在每 step 的 forward 时长接近一致。Coordinator 监控偏差, 必要时让快的 rank 等慢的 (barrier)。源码: `vllm/v1/engine/coordinator.py`。

Q: 跨机部署如果一台机器突然失联, vLLM 怎么处理? A: NCCL collective 阻塞 → forward hang → liveness probe 失败 → K8s LeaderWorkerSet 整组重启 → 新 pod 加入 NCCL group → 服务恢复。MTTR 通常 1-3 分钟。期间客户端请求 5xx, retry policy 重试到健康集群。详见 [§ 11.2](#)。

13. 小结

- **四个核心维度 + 三个附属维度**: TP / PP / EP / DP (主), SP / DCP / PCP (辅, 不增 world size 或仅在某阶段用)
- **TP 切单层** (column→row), 每层 2 次 AllReduce, 通信重 → **强依赖 NVLink, 单机内主战场**
- **PP 切层**, 只在段边界传 hidden state, 通信轻 → **适合跨机**, 代价是 bubble (M < 10 时 30%+ 浪费)
- **EP 把 MoE 的 experts 分卡**, 靠两次 AllToAll 路由 token, EPLB 解决热门 expert 负载不均
- **DP 是唯一故障域不传染的维度** (纯副本场景), 高可用部署偏好
- **SP 默认开**: 附属于 TP, 省 LayerNorm / Dropout / residual 的冗余算力, 几乎免费
- **5 种集合通信原语**: AllReduce、AllGather、ReduceScatter、AllToAll、Broadcast + P2P; vLLM 全部都用
- **NCCL 拓扑敏感**: `nvidia-smi topo -m` 看 NVLink 拓扑、`NCCL_DEBUG=INFO` 排查问题
- **生产部署口诀**: 单机优先 TP, 跨机用 PP, MoE 必加 EP, DP 做实例级扩展, 长上下文加 DCP, 超长上下文加 PCP
- **故障恢复**: TP/PP/EP 一卡挂全组挂; 必须 K8s LeaderWorkerSet 整组重启; DP 多副本是唯一容错友好方案

自检

不用照着原文复述, 重点是把现象、机制、源码入口和取舍讲顺。

1. Llama-70B, TP=8, 一个 token 一次 forward 通信多少字节? PCIe vs NVLink 多慢?

每层 2 次 AllReduce, 80 层 = 160 次。每次传 [1, H] tensor:

单次 AllReduce 数据 / token = $H \times \text{dtype_bytes} = 8192 \times 2 = 16,384$ 字节 = 16 KB
总通信 / token / forward = $160 \times 16 \text{ KB} = 2.56 \text{ MB}$

NVLink 4.0 (H100 内连) = 900 GB/s, 传 2.56 MB 需要 $\sim 2.8 \mu\text{s}$ (基础延迟) + NCCL ring 算法常数开销。一次 decode 总通信时长约 ms 级。

PCIe Gen5 $\times 16$ = 单向 64 GB/s (双向 128 GB/s)。同样数据需 $\sim 20 \mu\text{s}$ /次, 慢 $\sim 10\times$ 。

还有更糟的: NCCL ring-allreduce 实际有 $2(N-1)/N$ 倍数据流过链路, 并且每次 AllReduce 有约 $10 \mu\text{s}$ 的 launch + barrier overhead。160 次 forward overhead 累积 $\sim 1.6 \text{ ms}$ (NVLink) vs 16 ms (PCIe)。

→ **TP 跨 PCIe 单 token 多 10-15 ms**, 决策 / agent / code completion 等延迟敏感场景不可接受。结论: **TP 强依赖 NVLink, 跨机不可行**。

2. PP=4, M=4 时 bubble 占比？降到 5% 以下 M 至少多大？

bubble ratio = $(P-1) / (M + P-1) = 3 / 7 \approx 42.86\%$ 。

要 < 5%：

```
3 / (M + 3) < 0.05
M + 3 > 60
M > 57
```

→ M 至少 **58**。

实际 vLLM 推理里 M = "本批 batch 中的不同请求 / chunk 个数"。要 $M \geq 58$ 意味着 $\text{max_num_seqs} \geq 58$ ，且实际 running batch 经常满。生产 chatbot QPS 高时容易达到，agent 类低 QPS 场景达不到——所以 agent 不适合 PP。

补充细节：可以通过把 prefill chunk 切更细人为增大 M，但增加 schedule overhead。Megatron 训练时还有 1F1B / Interleaved schedule 进一步降 bubble，但推理只 forward 用不上。

3. parallel_state.py 里 TP/PP/DP ProcessGroup 怎么初始化？driver vs 普通 worker 职责？

源码：vllm/distributed/parallel_state.py 入口 init_distributed_environment + initialize_model_parallel。

```
# 总体思路：每个 rank 是 (tp_rank, pp_rank, dp_rank) 的三维坐标
# world_size = TP × PP × DP

# ① 全局 ProcessGroup (NCCL world)
torch.distributed.init_process_group(backend="nccl", ...)

# ② TP group: 同 pp_rank, dp_rank 的 ranks 一组
tp_group = dist.new_group(ranks=[r for r in range(world_size)
                                if r // tp_size == my_pp_dp_id])

# ③ PP group: 同 tp_rank, dp_rank 的 ranks 一组
pp_group = dist.new_group(...)

# ④ DP group: 同 tp_rank, pp_rank 的 ranks 一组（通常 DP 不通信，仅记录）
dp_group = dist.new_group(...)
```

driver vs 普通 worker 的差异：

- **Driver worker**：TP group 内 rank 0 (local_rank=0)。承担额外职责：
- 从 EngineCore 收 SchedulerOutput (broadcast 给其他 TP rank)
- 收集所有 TP rank 的输出，聚合后发回 EngineCore (结果就 1 份)
- prefix cache hash 计算 (不依赖 KV 数值，只算 token，driver 一份就够)
- 采样后的 next_token 由 driver 决定 (避免不同 rank 的 RNG 差异)
- **普通 worker**：被动收 SchedulerOutput、执行自己分片的 forward、AllReduce 时参与通信

判断：grep is_driver_worker 或 local_rank == 0 的代码路径都是 driver 专属逻辑。

4. Mixtral 8×7B 在 4×H100, TP=2+EP=4 vs TP=4+EP=2，哪个通信代价低？

Mixtral: 8 expert, top-2, dense hidden ≈ 4096 。

TP=2 + EP=4:

- $ep_size = 2 \times 4 / dp = 8 \rightarrow$ 不对, 需要 dp 调整。实际配置可能是 TP=2, DP=2, EP=4。
- 简化: TP=2 (每层 2 次 AllReduce, dense 部分通信小, hidden 4096 $\rightarrow$ 8 KB/token/AR)
- EP=4: 每张卡 2 expert, 每 MoE 层 2 次 AllToAll
- AllToAll 数据 = $token \times top_k \times hidden = \dots \times 2 \times 4096 \times 2 = 16 \text{ KB/token/AT}$

TP=4 + EP=2:

- TP=4: 每 dense 层 2 次 AllReduce, hidden 4096, 每次 AR 数据 8 KB/token, 但走 4 卡 ring 比 2 卡 ring 慢约 $1.5 \times$
- EP=2: 每张卡 4 expert, 每 MoE 层 2 次 AllToAll (在 EP=2 时也就是 2 个 rank 互发, 本质是 AllToAll over P2P)
- AllToAll 数据相对小 (仅 2 卡间)

取舍:

- TP 数据小但频繁 (每层 AR), TP=4 比 TP=2 通信总量大约 $1.5 \times$ **per layer**
- EP 数据大但稀疏 (仅 MoE 层), EP=4 vs EP=2 通信量类似 (同样的 token 数量飞)

实测: Mixtral 在 $4 \times H100$ 上 **TP=2 + EP=4 (带 DP=2 平衡) 通常更优**——因为 dense 部分 TP=2 通信少, MoE 部分 EP=4 让 expert 切得更细, 单 expert 跑得更快。

但具体取决于:

- NVLink 拓扑 (4-card hyper cube vs ring)
- 是否启用 DeepEP / FlashInfer 加速 backend
- workload 是 prefill-heavy 还是 decode-heavy

实战建议: 两种配置都跑 benchmark_serving.py 测一遍, 看哪个 throughput 高。理论分析做参考, benchmark 是 ground truth。

下一步

- 下一节: 05-distributed/02-disaggregated.md (把 prefill / decode 拆到不同集群, 针对长上下文与高并发)
- 想看源码: vllm/distributed/parallel_state.py、vllm/model_executor/layers/linear.py、vllm/v1/executor/multiproc_executor.py、vllm/distributed/eplb/
- 想动手: 07-hands-on/03-mini-experiments.md (实验 5 提到的 TP scaling 进阶实验)
- 想从生产视角理解: 08-production-deployment/01-deployment-architectures.md (多实例 vs 大实例的取舍)
- 规模化实战: 05-large-scale-cluster-inference.md (§ 11 的故障传染放到千卡万卡: blast radius、NCCL fail-stop、慢卡驱逐、弹性 EP)

02. Disaggregated Prefill / Decode

谁该读这一篇？ 想理解"为什么生产团队愿意花两倍机器把 prefill / decode 拆开"、"vLLM 通过哪个接口接 NIXL / LMCache / Mooncake"、"什么规模才划算"的读者。

前置阅读： 01-tp-pp-ep.md（先掌握同机的多卡并行才能讲跨集群）；最好读过 02-core-concepts/05-chunked-prefill.md（理解同卡上 prefill/decode 互相干扰的根源）。

耗时： 约 12 分钟。

学完能：

- 1. 解释为什么 prefill 和 decode 适合不同硬件配置（算术强度差）。
- 2. 画出 disaggregated 的数据流（含 KV 转移触发点）。
- 3. 列出 KV connector 接口的 3 个核心方法，知道 NIXL / LMCache / Mooncake 在该接口下怎么接入。
- 4. 判断"我这场景值不值得搞 disaggregated"（并发、SLO、网络成本）。

当前源码复核（b23bd73）： KV connector 是 scheduler/worker 两侧协议，不等于“自动完成 P/D 集群”。生产还需外部路由、request/KV ownership、block/模型兼容、传输完成通知、容量 backpressure、timeout、取消与失败清理；NIXL/LMCache/Mooncake 的能力和配置不能互换。

1. 为什么要拆？

prefill 和 decode 的硬件特性完全不同：

特性	Prefill	Decode
算法复杂度	$O(\text{seq_len}^2)$	$O(\text{seq_len})$
算术强度（FLOP/Byte）	高（compute-bound）	极低（memory-bound）
单请求 token 量	几百到几万	1
适合的 batch	小 batch 也能打满 GPU	大 batch 才能打满带宽
适合的硬件	算力强的卡（H100、B200）	带宽强的卡（同上 + 大 KV）

把它们混在同一卡：

- 长 prefill 阻塞 decode（被 chunked prefill 缓解但没根治）
- decode 的 sampling/调度开销在 prefill 时浪费
- 难以同时优化 TTFT 和 TPOT

2. 解决方案

flowchart LR

P["Prefill 节点群
• 高并发短任务
• 重算力 • 少 KV
• 优化 TTFT"]
 D["Decode 节点群
• 大 batch 长上下文
• 重 KV • 省算力
• 优化 TPOT"]
 P -->|"KV via RDMA (NIXL)"| D

```
classDef prefill fill:#fef3c7,stroke:#b45309,color:#1a1f29;
classDef decode fill:#eff5ff,stroke:#2563eb,color:#1a1f29;
class P prefill;
class D decode;
```

流程：

1. 请求到达 Router
2. Router 派给某个 Prefill 节点跑 prefill
3. Prefill 完成，把 KV 写到该请求的 block，通过 RDMA 传到 Decode 节点
4. Decode 节点接 KV，把请求放入自己的 batch，开始 decode
5. Decode 节点生成 token 流式返回

3. vLLM 的实现：KV Connector

vllm/distributed/kv_transfer/ 提供 KV connector 抽象，每种传输后端实现接口：

Backend	协议	适用
NIXL	NVIDIA 的 GPU-Direct RDMA	InfiniBand / RoCE 集群
LMCache	缓存 + 转发	已有 LMCache 部署
Mooncake	月之暗面的方案	大规模分布式 KV
Custom	自己写	实验/特殊场景

接口要点：

- send_kv(req_id, blocks)：prefill 节点调用
- recv_kv(req_id) -> blocks：decode 节点调用
- 异步：传输与计算 overlap

4. 数据流详解


```
sequenceDiagram
    participant PS as Prefill Scheduler
    participant PW as Prefill Worker
    participant NIXL as NIXL / RDMA
    participant DW as Decode Worker
    participant DS as Decode Scheduler

    PS->>PW: schedule prefill
    PW->>PW: 完成 prefill, KV 写入本地 paged cache
    PW->>NIXL: send_kv(req_id, block_ids)
    NIXL->>DW: 异步 GPU-Direct DMA 到对端显存
    PW->>PS: 完成 prefill, 可释放 block
    DW->>DS: recv_kv 完成, req 加入 running
    DS->>DW: schedule decode step
    DW->>DW: block_table[req] = local blocks, 开始 decode
```

关键点：

- 传输的是 **KV cache 内容**（block 数据），不是模型权重
- 不是 SwapOut/SwapIn 那种"换出再换入"，而是 source 节点的 KV 永久转移
- 用 RDMA 才有意义（PCIe 慢得多）

5. 为什么 NIXL 重要

NIXL 是 NVIDIA 开发的库，封装 GPUDirect RDMA：

- 从一卡的显存直接 DMA 到另一台机器上的卡显存
- 不经 CPU、不经主机内存（不像 vanilla MPI）
- 延迟 < 10 μ s，带宽接近 NIC 上限

vLLM 通过 NIXL 让 disaggregated 在 ms 级 KV 转移上可行。

6. Scheduler 协调

两端都有自己的 Scheduler。挑战：

- Prefill 端要知道"KV 传过去了"才能 free 自己的 block
- Decode 端要知道"KV 收到了"才能开始 decode

vLLM 通过 KV connector 的回调 + Scheduler 的 metadata 字段（kv_connector_metadata）协调。这部分是 V1 时代的新设计。

7. 实际部署案例

- **Moonshot (Kimi)**：声称用 disaggregated 后 TTFT/TPOT 分别比同卡部署优 30%/40%
- **DeepSeek**：开源了 EPLB + disaggregated 工具链
- **NVIDIA Dynamo**：基于 vLLM/TRT-LLM 的 Disaggregated Serving Framework

8. 局限与挑战

- 1. **小并发不划算**：KV 传输有最小开销，请求少时不如同卡
- 2. **运维复杂**：两个集群、路由、故障切换
- 3. **prefix caching 难做**：缓存散布在多个 prefill 节点
- 4. **长上下文**：100k+ token 的 KV 转移本身可能 100ms+

8.1 Connector 生产契约

契约	必须回答的问题	失败时动作
identity/ownership	request、model、KV group/block 如何唯一对应	拒绝错配，不允许静默复用
compatibility	model revision、KV dtype/layout、block size、parallel rank 是否一致	admission 前 fail fast
capacity/backpressure	destination 预留多少 block/bytes，队列是否有界	429/重路由/超时取消，而非无限等待
completion	load/save 哪个 ack 表示可调度/可释放	deadline 后幂等 abort 与回收
partial failure	一部分 layer/rank 到达时如何清理	connector-specific rollback；不得标 cache hit
observability	bytes、queue、transfer、retry、timeout 如何关联 request ID	保留两端 trace 与错误原因

9. 工程自检问答

- Q: 为什么 prefill 和 decode 适合的卡不同？** A: prefill 是 compute-bound，要 FLOPs；decode 是 memory-bound，要 HBM 带宽。理论上 prefill 可用算力强的小显存卡，decode 用大显存高带宽卡。实际多用同型号卡但跑不同配置。
- Q: KV 转移是不是会被 prefill 节点的 free 阻塞？** A: 是异步的，prefill 节点提交完转移就可以 free 本地 block，转移由 NIC + 接收端 buffer 接住。
- Q: 一个请求生成中途想接更多 token 怎么办？还是只在 prefill 节点？** A: 不会回到 prefill。请求一旦进 decode 节点就一直留在那。如果生成中插入新 token（如 tool calling 返回），是新一轮的 prefill 续接，可以在原节点或新 prefill 节点跑。
- Q: vs chunked prefill？** A: 互补。chunked prefill 是同卡上把长 prefill 切片，缓解阻塞但不消除。disaggregated 是物理隔离，根治。

10. 什么场景值得搞 disaggregated？

不是所有场景都值得搞。下面这张决策表给你判据：

场景	建议	理由
QPS < 50 / 单机即可承载	不搞	KV 传输固定开销摊不掉，运维成本 >> 收益
TTFT 与 TPOT 都已达 SLO	不搞	没需求
TTFT 严重不达标但 TPOT OK	可搞 (多 prefill 节点)	但先试 chunked prefill / prefix caching 是否能解决
同机一开 chunked 就 TPOT 抖	强烈考虑	这就是 disaggregated 解决的核心矛盾
长上下文 (≥ 100K token)	看情况	KV 转移本身可能 100ms+，得算清楚
RDMA 网络不具备 (只有以太网)	先测再决定	TCP/存储路径可能可用，但必须把 KV bytes、排队与尾延迟计入收益
多模型 / LoRA 切换频繁	不搞	prefill 节点重启/换模型很贵
高 prefix cache 命中率 (70%+)	谨慎	cache 散布到多 prefill 节点，命中率会下降，需要 cache-aware routing 抵消

业界报告的收益（参考量级，不要当真实数字）：

- Moonshot Kimi：TTFT -30% / TPOT -40%（混合长短 prompt workload）
- llm-d 论文：高 QPS 长上下文下吞吐 +38.9%
- 大部分公开数据来自高并发 + 异构 workload。低并发 / 同构 workload 收益不明显或为负。

小结

- Disaggregated 通过物理隔离 prefill / decode，根治"长 prefill 阻塞 decode"和"硬件特性不匹配"两个矛盾。
- vLLM 用 **KV connector 抽象**屏蔽传输细节，NIXL / LMCache / Mooncake 都通过它接入。
- 数据流核心是异步 KV 转移 + 双端 Scheduler 协调；prefill 完成可立即 free，decode 端按 kv_connector_metadata 知道 KV 何时就绪。
- 不是银弹：小并发、无 RDMA、模型频繁切换的场景反而吃亏。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. KV connector 的 3 个核心方法 + 接入新 transport 要实现什么？

源码：vllm/distributed/kv_transfer/ 的 KVConnectorBase 抽象接口。

```
class KVConnectorBase(ABC):
    @abstractmethod
    def send_kv_caches_and_hidden_states(
        self,
        request_id: str,
        kv_blocks: list[KVCacheBlock],
        hidden_states: torch.Tensor,
    ) -> None:
        """prefill 节点：把这个请求的 KV + 最后 hidden state 发出去"""

    @abstractmethod
    def recv_kv_caches_and_hidden_states(
        self,
        request_id: str,
    ) -> tuple[list[KVCacheBlock], torch.Tensor]:
        """decode 节点：等收到对应请求的 KV + hidden state"""

    @abstractmethod
    def close(self) -> None:
        """清理资源（关闭 RDMA queue pair、释放 buffer 等）"""
```

接入新 transport 要做的事：

1. 继承 KVConnectorBase，实现上面 3 个方法
2. 把传输细节封装在内部（建立连接、序列化 KV、流控）
3. 通常需要异步（send 返回 Future）—— scheduler 不能阻塞等
4. 在 vllm/distributed/kv_transfer/factory.py 或类似注册表里加 "my_transport": MyConnector
5. 配套测试：单元测试（mock 一对发收）+ 集成测（两个 vLLM 实例对发）

关键挑战：metadata 一致性。两端要约定 KV 块的 token 边界、layer 顺序、dtype 等——通常通过元数据 header（msgpack）+ 内容 buffer 分开传，header 走 ZMQ，内容走 RDMA。

2. prefill 节点跑到 80% 时崩了，请求会怎样？

默认 vLLM 行为：

1. **检测：**decode 节点等待 KV 超过所选 connector/路由层的显式 deadline → 标记失败；不要假设跨 backend 通用的 30-60 秒默认值
2. **响应：**API Server 收到 abort，返回 5xx 错误给客户端
3. **重试：**取决于客户端策略，vLLM 自己不重试

进阶（部分平台如 llm-d 实现）：

1. **重新调度：**把请求送回 prefill 节点池（新选一个健康节点），从头开始 prefill
2. **代价：**80% 的 prefill 算力全部白费
3. **复杂度：**需要 router 层记得请求原始 prompt（API Server 端的状态），prefill 输出尚未到 decode 时算丢失

生产建议：

- prefill 集群冗余度高于 decode（prefill 重做代价更高）
- 设置合理的 prefill 超时（不能太短让正常长 prompt 误杀，也不能太长拖累恢复）
- 监控 prefill_node_failure_total metric，超阈值告警

3. 为什么用 InfiniBand / RoCE 而不是以太网？100K token KV 转多少 MB？

Llama-70B KV 计算（GQA：8 KV head × 128 head_dim × 80 layer）：

- 单 token KV = 2 (K+V) × 8 × 128 × 80 × 2 (BF16) = **327,680 字节 ≈ 320 KB**
- 100K token = 320 KB × 100,000 = **32 GB**（确实大）

网络选择：

网络	带宽	转 32GB 时长
1Gbps 以太网	125 MB/s	256 秒 —— 不可能
10Gbps 以太网	1.25 GB/s	25 秒 —— 不可接受
100Gbps 以太网（RoCE）	12.5 GB/s	2.5 秒 —— 还是太长
400G InfiniBand HDR	50 GB/s	640 ms —— 接近可用
NVLink 跨机（NVL72 等）	900 GB/s	35 ms —— 理想

→ 对 100K token 这种极端长上下文，连 IB 都可能吃力。layer/chunk streaming 与 prefix-aware routing 是架构候选，但必须核对所选 connector 是否真的实现对应粒度、完成通知和失败恢复，不能从接口抽象推断已支持。

常见上下文长度（4-16K token）：1-5 GB，10Gbps 以太网 800 ms 可接受，IB 100 ms 流畅。这是 disaggregated 的主战场。

4. prefix cache 在 disaggregated 下命中率怎么变？怎么补救？

变差。原因：

- prefill 集群分片**：原本同 prompt 命中同一 vLLM 实例的 cache，disaggregated 后分散到多个 prefill 节点，每个节点的 cache 独立 → 命中率 ÷ prefill_node_count
- decode 节点 cache 失效**：decode 节点收到的 KV 是"传过来的"，不算 cache 命中（每次都重传等价于重算）

补救：

方案	怎么做	代价
Cache-aware routing	router 维护 "请求 prompt hash → 之前打到的 prefill 节点" 映射，让同 prefix 请求落同一节点	router 层复杂度 + 节点负载可能不均
L2 共享 cache（CPU DRAM）	prefill 节点把 cache spill 到共享 CPU 内存池（LMCache 等），所有 prefill 节点可查	L2 命中比 L1 慢 10×，需 RDMA
L3 共享 cache（远端）	把 cache 放到 NVMe / 对象存储，跨集群 cluster 共享	L3 慢 100×，仅适合超长冷 prompt
prefix-pinned 部署	已知热门 system prompt 在所有 prefill 节点 pre-warm	需要业务知识，pin 列表手工维护
不做 disaggregated	如果 prefix cache 命中率是核心 KPI，重新考虑架构	见 § 10 决策表

架构建议：disaggregated 通常需要 cache/KV-aware routing；具体采用率没有仓库内证据，不写“80%”。否则 latency/TPOT 收益可能被 cache 命中下降和传输排队抵消。详见 08-production-deployment/02-smart-routing-and-load-balancing.md。

下一步

- 想看 KV connector 源码：vllm/distributed/kv_transfer/ 与 vllm/v1/kv_offload/。
- 想落地 Mooncake P/D 直传、共享 DRAM / SSD KV 池与 MultiConnector：08-production-deployment/16-mooncake-distributed-inference-storage.md。
- 想看智能路由怎么帮 disaggregated 降低 KV 传输：08-production-deployment/02-smart-routing-and-load-balancing.md。
- 想理解部署平台怎么管两个集群（llm-d / Dynamo / Albrix）：08-production-deployment/01-deployment-architectures.md。
- 想做生产化故障预案：08-production-deployment/06-reliability-and-failure-modes.md。
- 规模化实战：05-large-scale-cluster-inference.md（P/D 分离在千卡万卡上的 KV 路由、比例动态调整、故障墙）。

03. Expert Parallel 深度：AllToAll、6 个后端、负载均衡

谁该读这一篇？ 部署 DeepSeek-V3 / Qwen3-MoE / Mixtral / GPT-OSS 这类 MoE 模型的工程师；想理解"为什么大 MoE 模型常配 DP + EP 不只是 TP"、"vLLM 提供哪些 all2all 后端、怎么选"的同学；做容量规划要算 expert 负载不均的人。

前置阅读： 01-tp-pp-ep.md（先了解 EP 的基本概念再来看实现细节）；最好读过 03-code-walkthrough/07-model-architectures.md（MoE 层的整体结构）。

耗时： 约 18 分钟。

学完能：

1. 画出 MoE 一层的 dispatch → expert → combine 数据流，标出 AllToAll 的两个发生点。
2. 区分 vLLM 的 6 个 all2all 后端的适用场景：AgRs / DeepEP-HT / DeepEP-LL / FlashInfer NVLink (2-sided / 1-sided) / Mori / NIXL-EP。
3. 解释 "EP + DP" 组合为什么比纯 EP 更常见，以及 $tp_size * dp_size * pcg_size$ 怎么决定 expert 切片数。
4. 判断什么时候需要 EPLB (Expert Parallel Load Balancing)；说出 vLLM 监控负载不均要看哪个 metric。

当前源码复核 (b23bd73)： all2all backend 当前默认 allgather_reducescatter，其他后端需要依赖、平台与拓扑支持。EPLB 需要 CUDA-like 平台、`--enable-expert-parallel` 和有效 EP group；Elastic EP 还要求 EPLB 且有 PP/LB 等组合限制。在线弹性经过显式 scaling API 与 drain/weight-transfer/reshuffle，不是无条件自愈。

1. 为什么大规模 MoE 常用 EP？

DeepSeek-V3 这类大 MoE 每层只激活部分 expert。小模型/小规模可以不用 EP；当 expert 权重、TP 切片浪费或吞吐成为瓶颈时，EP 才把 expert 映射到更大的 rank group。以下用 256 expert/top-k=8 作为算例。

- 每张卡都加载全部 256 expert 的 1/8（按权重切）
- 每个 token 路由到 8 个 expert，**每张卡都被涉及**
- TP-style AllReduce 频繁，且**算力浪费在**不被激活的 expert 切片上

EP 的思路：**每个 expert 整块**放在一张卡上（不切权重），token 按 router 决定**飞到对应卡**。代价是引入 all-to-all 通信。

```
flowchart LR
    subgraph "纯 TP （不适合 MoE） "
        T0["GPU 0<br/>expert 0..255 各 1/8 权重"]
        T1["GPU 1<br/>同上"]
        T2["GPU 2..7<br/>同上"]
        T0 -.AllReduce.-> T1
        T1 -.AllReduce.-> T2
    end

    subgraph "EP （MoE 友好） "
        E0["GPU 0<br/>expert 0..31 完整权重"]
        E1["GPU 1<br/>expert 32..63"]
        E2["GPU 2..7<br/>...各 32 expert"]
        E0 <-.AllToAll.-> E1
        E1 <-.AllToAll.-> E2
    end

    classDef tp fill:#fef3c7,stroke:#b45309;
    classDef ep fill:#eff5ff,stroke:#2563eb;
    class T0,T1,T2 tp;
    class E0,E1,E2 ep;
```

EP 的核心通信是 **AllToAll**，不是 AllReduce。

2. MoE 一层的数据流（含通信点）

```
sequenceDiagram
    participant T as Token (在本 rank)
    participant R as Router
    participant A1 as AllToAll #1<br/>(dispatch)
    participant E as Expert (在 owner rank)
    participant A2 as AllToAll #2<br/>(combine)
    participant O as Output (回本 rank)

    T->>R: hidden_states
    R->>R: top-k 选 expert id
    R->>A1: 每个 token 标 expert id
    A1->>E: token 飞到 expert owner rank
    E->>E: expert forward (GEMM)
    E->>A2: expert 输出
    A2->>O: 飞回 token 原 rank
    O->>O: 加权求和 (router weights)
```

两次 AllToAll：dispatch 和 combine 各一次。每层每次 forward 都来一遍——MoE 通信开销主要在这里。

源码层关键类（vllm/model_executor/layers/fused_moe/modular_kernel.py）：

- FusedMoEPrepareAndFinalizeModular — 抽象 dispatch + combine 流程
- FusedMoEExpertsModular — 抽象 expert 计算
- FusedMoEModularKernel — 组合二者

3. EP 配置：tp_size / dp_size / ep_size / pcp_size 怎么咬合

源码锚点: [vllm/model_executor/layers/fused_moe/config.py](#) • [FusedMoEParallelConfig](#)

源码: `vllm/model_executor/layers/fused_moe/config.py` 的 `FusedMoEParallelConfig`:

```
@dataclass
class FusedMoEParallelConfig:
    tp_size: int
    pcp_size: int    # prefill context parallel
    dp_size: int
    ep_size: int
    use_ep: bool     # 是否启用 EP
    all2all_backend: str
    enable_eplb: bool
```

关键关系 (参考 `flatten_tp_across_dp_and_pcp` 方法):

- 纯 TP 模型: $ep_size = 1$, $dp_size = 1$ 。MoE 权重像普通权重那样按 TP 切。
- EP 启用时: $ep_size = tp_size \times dp_size \times pcp_size$ 。Expert 数被均分到这么多个 rank。

为什么常常 EP + DP 一起用?

DeepSeek-V3 有 256 expert。如果只 EP-8: 每张卡放 32 expert。但 `hidden_size=7168`、`moe_intermediate=2048`, 一个 expert 权重约 28 MB (BF16), 32 个 = 896 MB——加上 base model 还能塞下 H100。若要 EP-32 (每卡 8 expert, 更细粒度负载均衡): 需要 32 张卡, 但 dense 部分用不了 32-way TP (`hidden_size` 不够分), 所以做 TP-8 + DP-4 $\rightarrow ep_size=32$ 。

这就是**"宽 EP"**部署模式 (wide-EP), DeepSeek / Qwen3-MoE 常用。

4. AllToAll 的 6 个后端

vLLM 在 `vllm/distributed/device_communicators/all2all.py` 提供多种实现, 通过 `--all2all-backend` 选:

后端 (all2all_backend)	实现类	协议	适用	限制
allgather_reducescatter	AgRsAll2AllManager	NCCL AllGather + ReduceScatter	默认通用，无第三方依赖	通信量 = world_size × token_per_rank，大集群偏贵
deepep_high_throughput	DeepEP HT	DeepSeek 出的 NVLink/IB 优化 AllToAll	大 batch、prefill 重负载	需装 deep_ep 库；NVLink 集群最优
deepep_low_latency	DeepEP LL	DeepEP 的低延迟变体	decode 阶段、小 batch	同上；LL 牺牲吞吐换 latency
flashinfer_nvlink_two_sided 或 flashinfer_all2allv	FlashInfer 2-sided	NVLink 多节点 (MNNVL)	节点内大集群 NVLink fabric	需 flashinfer + NVLink 拓扑
flashinfer_nvlink_one_sided	FlashInfer 1-sided	NVLink 单向 RDMA	同上，部分场景延迟更优	同上
mori	Mori	阿里出的优化方案	阿里集群 / 特定环境	需装 mori
nixl_ep	NIXL EP	NVIDIA NIXL EP 模式	Disaggregated + EP 混合	需 NIXL 配套

选型决策：

NVLink + DeepSeek 风格大 MoE → 优先 DeepEP（HT for prefill, LL for decode） NVLink-only 单机 8 卡 → AgRs 够用，编译简单 跨节点 RDMA → FlashInfer 系列 不确定 → AgRs（默认即可），生产化再换

源码中通过 FusedMoEParallelConfig.use_deepep_ht_kernels 等 properties 判断当前用哪个。

5. AllToAll 的延迟成本

每层两次 AllToAll，80 层模型 = 160 次/forward。

示意量级（NVLink 集群，H100，EP-8）：

backend	单次 AllToAll latency (ms)	全模型 160 次 (ms)	占 forward 比
AgRs	0.5 - 2.0	80 - 320	30-60%（瓶颈！）
DeepEP HT	0.2 - 0.8	32 - 128	15-30%
DeepEP LL	0.1 - 0.4	16 - 64	5-15%

这就是为什么 DeepSeek 必须自己写 DeepEP——通用 NCCL AllToAll 在大 MoE 上是真瓶颈。

跨节点（多机）成本：再 ×3-5（RoCE）或 ×2（IB）。所以 wide-EP（多机）务必上 RDMA + 优化 backend。

6. Expert 负载不均：原因与监控

理论上 router 均匀分配，但实际：

- 训练数据有偏，某些 expert 被偏好 ("hot experts")
- 长尾分布：5-10 个 expert 吃掉 30-50% 的 token
- 不均会让"忙 expert 所在 rank"成为瓶颈，闲 rank 空转

vLLM 的应对：

6.1 ExpertMapManager

源码锚点：[vllm/model_executor/layers/fused_moe/expert_map_manager.py](#) • [ExpertMapManager](#)

`vllm/model_executor/layers/fused_moe/expert_map_manager.py` 管理 `logical_expert_id` → `physical_rank` 的映射。

6.2 RoutedExpertsCapturer

源码锚点：[vllm/model_executor/layers/fused_moe/routed_experts_capturer.py](#) • [RoutedExpertsCapturer](#)

`vllm/model_executor/layers/fused_moe/routed_experts_capturer.py` 在 forward 时统计每个 expert 收到多少 token，把数据回吐给 scheduler。

6.3 EPLB (Expert Parallel Load Balancing)

启用：`enable_eplb=True` (`FusedMoEParallelConfig.enable_eplb`)。

EPLB 的思路：

- 热 expert **复制到多 rank** (一个 expert 多个 owner)
- 冷 expert 多个**合并到同 rank**
- 物理 expert 数 > 逻辑 expert 数

DeepSeek-V3 论文给的方案：256 logical → 288 physical (多 32 个副本)。

代价：

- 显存 (多副本占 GPU)
- 路由复杂度上升
- 副本同步问题 (推理时一般 stateless，影响小)

6.4 何时该开 EPLB?

信号	是否开 EPLB
同一模型某些 GPU nvidia-smi 持续 100%，另一些 30-50%	开
vllm:iteration_tokens_total 方差大	可能
离线 batch 跑稳定的某个 workload	开（按 workload 调副本）
在线服务请求模式多变	谨慎（profile 后再决定）
单机 EP-8 小模型	不开（不均也不严重）

7. EP + Prefix Caching、EP + Disaggregated

7.1 EP 对 prefix caching 的影响

prefix caching 在 attention 层做（不在 MoE 层）。EP 只影响 MoE 层，**对 prefix caching 行为无直接影响**。但 EP 通常配合 DP 用，DP 各 rank 独立维护 cache——总 cache 容量 $\times$ dp_size 但**互不共享**。

可以通过 cache-aware routing 让相同 prefix 的请求打到同一 DP rank，缓解。

7.2 EP + Disaggregated

理论可叠加：prefill 集群和 decode 集群各自做 EP 部署。nixl_ep backend 就是为此设计。

实践细节：

- prefill 集群 expert 分布 vs decode 集群可以不同（不同 workload 偏好不同 expert）
- KV transfer 量与 EP 无关（KV 仍按 token + layer 维度切）

8. 故障与降级

症状 → 原因 → 处理：

症状	可能原因	处理
启动 expected DeepEP installed	选了 deepep_* 但没装	pip install deep_ep 或换 backend
AllToAll hang	NCCL 死锁 / expert 分布不一致	NCCL_DEBUG=INFO、检查 expert_map 同步
某 rank OOM 但其他 rank 空闲	严重 expert 负载不均	开 EPLB 或调整 expert 分布
forward 慢 + 网络利用率高	AllToAll 是瓶颈	换 DeepEP / FlashInfer，或减少 ep_size
forward 慢 + 网络利用率低	expert GEMM 是瓶颈	减 dp_size、减小 batch、看是否量化能加速

小结

- MoE 模型必须用 EP 而非 TP——expert 整块放一张卡，token 飞到对应卡，通信是 **AllToAll** 而不是 AllReduce。
- 一层 MoE 有 2 次 AllToAll（dispatch + combine），80 层模型一次 forward 共 160 次——这是 MoE 推理的核心瓶颈，因此有 6+ 个后端可选。
- TP × DP × PCP = EP 切片数**。大 MoE 模型常用"宽 EP"：TP-8 × DP-N → ep_size=8N，专家分布更细粒度。
- DeepEP HT/LL 是 DeepSeek 自家优化，NVLink 集群下比 AgRs 快 2-5×；跨节点用 FlashInfer。
- expert 负载不均常见问题；vLLM 通过 RoutedExpertsCapturer 统计 + ExpertMapManager 重映射 + EPLB（多副本）应对。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 256 expert, top-k=8, TP-8 + DP-4, ep_size 和每卡几个 expert?

公式：ep_size = TP × DP × PCP（PCP 默认 1）

```
ep_size = 8 × 4 × 1 = 32
每卡 expert 数 = 256 / 32 = 8
```

总 GPU 数 = TP × DP × PP = 8 × 4 × 1 = 32 卡。

实际部署：DeepSeek-V3 256 expert + TP-8 + DP-4 = 32 卡集群，每卡 8 个 expert，每层 forward 跑 2 次 32-rank AllToAll。常配 IB/NVLink Switch System（GH200 NVL32 / GB200 NVL72）。

补充细节：如果 expert 数不整除 ep_size（如 250 个 expert + ep_size=32），需要 EPLB 加 padding（凑到 256 → 每卡 8）或者整除约束失败启动报错。

2. AgRs 跑 DeepSeek-V3 慢，先改什么？观察什么 metric?

改什么：把 --all2all-backend 从默认的 allgather_reducescatter 改成 **deepep_high_throughput**（NVLink 集群）或 deepep_low_latency（追求 decode latency）。

```
vllm serve deepseek-ai/DeepSeek-V3 \
--enable-expert-parallel \
--all2all-backend deepep_high_throughput
```

前提：装 deep_ep 库（pip install deep_ep）+ NVLink 集群。

观察 metric：

- vllm:time_to_first_token_seconds p99 —— prefill TTFT 应明显下降（AllToAll 占 prefill 不小）
- vllm:request_time_per_output_token_seconds —— decode TPOT 看是否抖动减小
- nvidia-smi dmon 或 NCCL profile —— 每层 AllToAll 时长（AgRs 1-2 ms，DeepEP HT 0.2-0.8 ms，5× 提速）
- vllm:iteration_tokens_total 分布 —— step 时长更平稳

若 DeepEP HT 仍不够（跨多节点）→ 尝试 flashinfer_nvlink_two_sided（FlashInfer + NVLink fabric）。

3. EPLB 256→288 physical, top-k=8, 每 token 发到几个 rank?

仍然是 **top-8 个 expert**——EPLB 不改 top-k 值，只改 logical→physical 映射。

但每个 logical expert 可能对应多个 physical expert（副本），router 选 top-8 logical 后，**还要决定具体打哪个副本**：

logical_expert_id	→	[physical_replica_0, physical_replica_1, ...]
住在 rank A		住在 rank B (副本)

具体 dispatch：

- 副本数为 1 的 expert：必定打到唯一 rank
- 副本数 > 1 的 expert：按负载均衡策略选副本（round-robin / 按本步 owner rank 当前 load / 随机）

每 token 实际涉及多少 rank：最多 8 个（top-k），最少更少（如果 8 个选中的 expert 都恰好在同 rank）。期望 $\approx \min(8, \text{ep_size})$ ——即 8（因为 ep_size=32 远大于 8）。

→ 复制副本主要让 hot expert 不挤——同一个 hot logical expert，原来所有打它的 token 都飞到 rank A，现在分散到 A 和 B。

4. use_deepep_ll_kernels 的具体条件 + 为什么 dp_size=1 不启用 deepep?

源码：vllm/model_executor/layers/fused_moe/config.py

```
@property
def use_all2all_kernels(self):
    return self.dp_size > 1 and self.use_ep

@property
def use_deepep_ll_kernels(self):
    return self.use_all2all_kernels and self.all2all_backend == "deepep_low_latency"
```

两个条件：

1. use_all2all_kernels：需要 dp_size > 1 且 use_ep == True
2. all2all_backend == "deepep_low_latency"

为什么 dp_size = 1 不启用？

当 dp_size = 1 时：

- 整个 EP 退化为"单 DP 实例内部的 EP"——即 ep_size = TP（仅靠 TP 维度切 expert）
- 这种场景下 expert 切分粒度有限（TP 通常 ≤ 8 ），AllToAll 也只在 TP 组内进行
- **完全可以用 TP 组的常规 NCCL collective 处理**——AllReduce / AllGather 等
- 走 DeepEP 这种"跨 DP rank 通信"的专用 kernel 是 overkill，反而引入 setup 开销

只有当 dp_size > 1（典型 wide-EP 配置：TP=8, DP=4, EP=32）时：

- expert 跨 DP 维度分布，需要真正的"非 TP 组的 AllToAll"
- DeepEP / FlashInfer NVLink kernel 才有用武之地

→ 设计上把"小集群单 DP"和"大集群多 DP"两种场景分流，避免 wide-EP kernel 在小场景下成为负担。

下一步

- 下一节：04-context-parallel.md（PCP / DCP——长上下文场景下另一种切分维度）。
- 规模化实战：05-large-scale-cluster-inference.md（EPLB / DP padding / 微批重叠在千卡万卡上怎么治 EP 同步长尾、AllToAll 怎么撞网络墙）。

源码锚点：[vllm/model_executor/layers/fused_moe/config.py](#) • [FusedMoEParallelConfig](#)

- 想看源码：[vllm/model_executor/layers/fused_moe/config.py](#)（ParallelConfig）、[vllm/distributed/device_communicators/all2all.py](#)（6 个后端）、[vllm/model_executor/layers/fused_moe/expert_map_manager.py](#)（EPLB 核心）。
- 想从生产视角理解：08-production-deployment/04-autoscaling-and-capacity.md（MoE 的容量规划要把 EP 维度算进去）。
- 想看 DeepEP 论文与实现：DeepSeek GitHub deep-ep 仓库。

04. Context Parallel: PCP / DCP 与长上下文切分

谁该读这一篇？ 部署 128K / 1M token 长上下文场景的工程师；想理解"为什么 TP 切不动长 prompt、CP 怎么补位"、"PCP 和 DCP 是两个独立维度"的同学；做 MLA 模型（DeepSeek 类）分布式部署的人。

前置阅读： 01-tp-pp-ep.md（先掌握 TP/PP/EP）；最好读过 02-core-concepts/05-chunked-prefill.md（理解 prefill 的切分思路）。

耗时： 约 14 分钟。

学完能：

- 1. 区分 PCP（Prefill CP）和 DCP（Decode CP）——它们是**两个独立的**并行维度。
- 2. 解释 DCP 为什么"不增加 world_size"——它复用 TP 组的 GPU，本质是 attention 的另一种切法。
- 3. 选 DCP 的通信后端：ag_rs vs a2a，以及为什么 MLA 模型用 a2a 能省 1/3 NCCL 调用。
- 4. 计算 cp_kv_cache_interleave_size 对 KV 物理分布的影响。

当前源码复核（b23bd73）： PCP 进入 worker world，DCP 复用 TP ranks；当前校验要求 $TP \% DCP == 0$ ，PCP 暂不支持 $DP>1$ 。dcp_comm_backend 默认 ag_rs，a2a 还要求 $DCP>1$ 并受 attention/model backend 能力约束；选择表只能作为实验起点。

1. 为什么需要 Context Parallel

TP 把权重切到多卡，每张卡 hidden_dim 减少；但**每张卡仍要处理完整的 seq_len**。Llama-70B prompt=128K tokens：

- attention 中间矩阵 $Q @ K^T$ 形状 [batch, num_heads, seq_len, seq_len]
- $seq_len^2 = 128K^2 = 16\text{ G}$ 元素，半精度 32 GB——远超单卡

FlashAttention 把这个常驻矩阵 fuse 掉，但**激活仍是 $O(seq_len)$** 。长上下文下 activation memory + KV cache 才是真瓶颈。

CP 的思路：**把 seq 维度切到多卡**，每卡只持有 seq_len/cp_size 个 token 的 K/V/激活。代价是 attention 跨卡时要通信。

2. vLLM 的两个 CP：PCP 和 DCP

源码锚点：[vllm/config/parallel.py](#) · [ParallelConfig](#)

vLLM 把 CP 分成了两个独立的并行维度，源码：[vllm/config/parallel.py](#)。

维度	字段	作用	是否增加 GPU
PCP	prefill_context_parallel_size	prefill 阶段 切 seq	是 — 与 TP/PP/DP 正交
DCP	decode_context_parallel_size	decode 阶段 切 KV cache 的 seq 维	否 — 复用 TP 组 GPU

为什么要分开？

- Prefill 是 compute-bound，长 seq 算力压力大，PCP 真的需要更多 GPU 参与同一次 forward。
- Decode 每步只产 1 个新 token，算力不大，但 **KV cache 沿 seq 维堆得越来越长**——DCP 把 KV 切到多个 attention head 组（实质 reuse TP），不需要额外 GPU。

source: decode_context_parallel_size 注释明确写：

```
"the world size does not change by dcp, it simply reuse the GPUs of TP group, and tp_size needs to be divisible by dcp_size"
```

3. DCP 的工作原理（最常用的 CP）

flowchart LR

```
subgraph "TP-8（不开 DCP）"
```

```
  H0["GPU 0<br/>头 0..N/8<br/>全 KV"]
```

```
  H1["GPU 1<br/>头 N/8..2N/8<br/>全 KV"]
```

```
  H7["GPU 7<br/>..."]
```

```
end
```

```
subgraph "TP-8 + DCP-2（开 DCP）"
```

```
  D0["GPU 0<br/>头 0..N/8<br/>seq 偶数块"]
```

```
  D1["GPU 1<br/>头 0..N/8<br/>seq 奇数块"]
```

```
  D2["GPU 2..7<br/>其他 head 组<br/>同样按 seq 切"]
```

```
end
```

```
H0 -.-> D0
```

```
H0 -.-> D1
```

```
classDef tp fill:#fef3c7,stroke:#b45309;
```

```
classDef dcp fill:#eff5ff,stroke:#2563eb;
```

```
class H0,H1,H7 tp;
```

```
class D0,D1,D2 dcp;
```

DCP-2 时：原 TP-8 的 head 0 那张卡，**变成 2 张卡**，每张持有一半 seq。Attention 完成后通过通信合并。

源码锚点：[vllm/config/parallel.py](#) • [ParallelConfig._validate_parallel_config](#)

约束： $tp_size \% dcp_size == 0$ （parallel.py）。常用：TP-8 + DCP-2 或 TP-8 + DCP-4。

4. DCP 的通信后端：ag_rs VS a2a

源码锚点：[vllm/config/parallel.py](#) • [ParallelConfig](#)

源码锚点：[vllm/config/parallel.py](#) • [ParallelConfig](#)

源码：parallel.py 的 dcp_comm_backend 配置：

Backend	通信模式	每层 NCCL 调用数	适用
ag_rs（默认）	AllGather + ReduceScatter	3 (AG of K, AG of V, RS of out)	通用
a2a	AllToAll 交换 partial output + LSE，Triton kernel 合并	2	MLA 模型专用

为什么 a2a 对 MLA 友好？MLA（DeepSeek 用的 Multi-head Latent Attention）的 K/V 实际是从 latent 投影出来的小张量——AllGather 拉全 K/V 浪费带宽。AllToAll 各卡只交换"partial output（已 attended）+ LogSumExp 标量"，比 K/V 张量小得多。然后 Triton kernel 拿 partial + LSE 合并出全局正确的 output。

注意约束： a2a 需 dcp_size > 1（line 480-482），否则报错。

源码锚点：[vllm/config/parallel.py](#) · [ParallelConfig._validate_parallel_config](#)

源码锚点：[vllm/config/parallel.py](#) · [ParallelConfig._validate_parallel_config](#)

```
# parallel.py
if self.dcp_comm_backend == "a2a" and self.decode_context_parallel_size <= 1:
    raise ValueError(
        "dcp_comm_backend='a2a' requires decode_context_parallel_size > 1."
    )
```

5. KV cache 的 interleave 切分

DCP / PCP 把 KV 切到多卡后，要决定**怎么按 seq 切**：

源码锚点：[vllm/config/parallel.py](#) · [ParallelConfig](#)

源码：cp_kv_cache_interleave_size（parallel.py）：

```
total_cp_rank = pcp_rank * dcp_world_size + dcp_rank
total_cp_world_size = pcp_world_size * dcp_world_size

store interleave_size tokens on total_cp_rank i,
then store next interleave_size tokens on total_cp_rank i+1.
```

例：interleave_size=1、total_cp_world_size=4：

- token 0 → rank 0, token 1 → rank 1, token 2 → rank 2, token 3 → rank 3
- token 4 → rank 0, token 5 → rank 1, ...（轮询）

interleave_size=16、total_cp_world_size=4：

- token 0..15 → rank 0, token 16..31 → rank 1, ...

为什么不只用 1 或 chunk-of-block_size？

- =1 最均匀但每个 rank 都要存"散乱"的 token，cache 局部性差。
- =block_size（如 16）按块切，物理连续，与 PagedAttention block 友好。
- 默认 1（最均匀），生产可调。

注意： dcp_kv_cache_interleave_size 已被 cp_kv_cache_interleave_size 替代（向后兼容，将逐步 deprecate）。

6. PCP（Prefill CP）：更复杂、更新

PCP 是较新的功能（注释提到"will be deprecated when PCP is fully supported"，说明仍在路上）。

思路： prefill 长 seq 切到多卡，每卡算一段，attention 时跨卡组合。

典型用法： 长上下文（128K+）prefill。例如 prompt 长 200K，TP-8 + PCP-4：

- 每张卡 $\text{hidden_dim} \div 8$
- 每张卡 $\text{seq_len} \div 4 = 50\text{K}$
- attention 跨 4 卡组合（环形 KV 传递，类似 Ring Attention）

与 chunked prefill 的关系： 互补。

- chunked prefill：**时间维度**切（一次 forward 跑一段，多次 forward 完成全 prefill）
- PCP：**空间维度**切（一次 forward 多卡并行，每卡一段）

适用判据：

- chunked prefill 已能用 → 单机够用，简单可靠
- 单机塞不下 prefill 中间激活 → 上 PCP（需要 RDMA 网络）

7. 与其他并行组合

flowchart TD

```
Cfg["世界拓扑<br>world_size = TP × PP × DP × PCP"]
Cfg --> TP["TP<br>切 hidden_dim<br>同步 AllReduce"]
Cfg --> PP["PP<br>切 layer<br>send/recv"]
Cfg --> DP["DP<br>独立副本<br>批维"]
Cfg --> PCP["PCP<br>切 prefill seq<br>新增维度"]
TP --> DCP["DCP<br>复用 TP GPU<br>切 decode KV seq"]
DP --> EP["EP<br>MoE expert 分布<br>ep_size = tp × dp × pcp"]
```

```
classDef cfg fill:#eff5ff,stroke:#2563eb;
classDef extra fill:#fff7ed,stroke:#f59e0b;
class Cfg cfg;
class DCP,EP extra;
```

关键约束总结：

源码锚点： [vllm/v1/core/kv_cache_utils.py](#) • [resolve_kv_cache_block_sizes](#)

源码锚点： [vllm/v1/core/kv_cache_utils.py](#) • [resolve_kv_cache_block_sizes](#)

关系	约束
world_size	$tp \times pp \times dp \times pcp$
ep_size	$tp \times dp \times pcp$ （EP 启用时）
DCP	$tp \% dcp == 0$ ，不影响 world_size
混合 KV cache（Mamba + attention）	当前 不支持 CP （kv_cache_utils.py 报错）

8. 实际选择决策表

场景	建议
短上下文（< 8K）	不开 CP，TP/PP/DP 即可
中长上下文（8K-32K）+ MLA 模型	候选 DCP-2 + a2a；先核对 backend 支持并基准
中长上下文 + 非 MLA	DCP-2 with ag_rs（默认）或不开
长上下文（32K-128K）	DCP-2 或 DCP-4，按 KV 显存压力定
超长上下文（128K+）prefill 阶段塞不下	PCP-2 或更多 + chunked prefill
Mamba 类模型	不能用 CP （vLLM 限制）

9. 监控与调试

CP 不像 TP/PP 有专门 metric，主要观察：

- vllm:kv_cache_usage_perc：DCP 开启后整体 cache 利用率应更均匀
- vllm:time_to_first_token_seconds p99：PCP 开启长 prefill 应更平稳
- nvidia-smi：DCP 不增 GPU 数，但每张卡负载应平衡（之前 head 0 那卡的活分给 dcp_size 张）

常见错误：

- ValueError: dcp_comm_backend='a2a' requires decode_context_parallel_size > 1 — 没开 DCP 却选了 a2a
- RuntimeError: Hybrid KV cache groups ... do not support context parallelism — Mamba 模型不能用 CP

小结

- vLLM 的 CP 是**两个独立维度**：PCP 给 prefill 切 seq + 加 GPU；DCP 给 decode 切 KV seq + **复用** TP GPU。
- DCP 是最常用的，约束 $tp \% dcp == 0$ ；backend 有 ag_rs（默认 3 NCCL/层）和 a2a（2 NCCL/层，**MLA 专用**）。
- cp_kv_cache_interleave_size 控制 KV 在 CP rank 间的分配粒度——大值物理连续，小值更均匀。

- PCP 与 chunked prefill 互补：chunk 是时间维，PCP 是空间维；超长上下文塞不下时上 PCP。
- 当前限制：Mamba / 混合 KV cache 不支持 CP。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. TP-8 + DCP-2 用几张 GPU？为什么 DCP 不增 GPU？

总 GPU = **8 张**（仅 TP 的 8 张）。

为什么 DCP 不增 GPU：DCP 是"复用 TP 组的 GPU 切分 KV seq 维"，**约束 $tp_size \% dcp_size == 0$** ：

原 TP-8 配置：
rank 0..7 各持有 head 集合 H_0..H_7，每张卡有完整 seq 的 K/V

TP-8 + DCP-2：
把 TP 组拆成"head 组 × DCP 组"：
rank 0: head_0 + seq 偶数块 (DCP rank 0)
rank 1: head_0 + seq 奇数块 (DCP rank 1)
rank 2: head_1 + seq 偶数块 (DCP rank 0)
rank 3: head_1 + seq 奇数块 (DCP rank 1)
...

等价于：原 8 个 head（每卡 1 个）变成 4 个 head 组（每组 2 卡共享，按 seq 切）。world_size 不变，但 **KV cache 占用 / 卡 $\approx$ 减半**——长上下文场景可装下更多 token。

补充细节：DCP 实质是"用 head 维度换 seq 维度"的资源重分配；前提是 head 数足够多（tp_size 不被 head 数撑死）。

2. MLA 模型 DCP-4, a2a 比 ag_rs 省多少 NCCL 调用（80 层）？

每层 attention 的 DCP 通信：

- **ag_rs**：3 次 NCCL —— AllGather K, AllGather V, ReduceScatter output
- **a2a**：2 次 NCCL —— AllToAll (partial output + LSE), Triton kernel 合并 (本地)

每层节省：3 - 2 = **1 次 NCCL**

80 层一次 forward 节省：80 × 1 = **80 次 NCCL**

如果每次 NCCL 调用 overhead 约 20-50 μ s（含 GPU sync），单 forward 节省 1.6-4 ms。对 decode（forward 总 20-30 ms）来说 5-15% 的速度提升。

为什么 a2a 对 MLA 友好：MLA 的 K/V 实际是从 latent 投影来的小张量（kv_lora_rank=512 + qk_rope_head_dim=64，远小于普通 GQA 的 8 × 128 = 1024）。AllGather K/V 拉全张量浪费带宽；AllToAll 各卡只交换"partial output（已 attended）+ LogSumExp 标量"——数据量小很多。

代价：需要专门的 Triton kernel 合并 partial outputs。对 GQA 模型这个 kernel 复杂度高于直接 AG，所以 GQA 不选 a2a。

3. cp_kv_cache_interleave_size=1 vs =16 对 prefix caching 影响?

有影响，主要在"哪些 token 在同一 rank 上"。

interleave_size=1 (最均匀):

- token 0 → rank 0, token 1 → rank 1, ..., 轮询
- 一个 16-token block 的 K/V **散布在所有 CP rank**
- prefix cache hash 是按 block 算的，**block 内部 token 跨 rank 无所谓**——hash 仍能命中（因为 hash 用的是 token id，不是物理位置）

interleave_size=16 (= block_size):

- token 0..15 → rank 0, token 16..31 → rank 1, ...
- 一个 16-token block 的 K/V **全在同一 rank**
- prefix cache 命中行为**相同**——hash 一样命中

所以 **prefix cache 命中率本身不变**。但有间接影响:

- interleave_size=1 时，命中 block 后实际 K/V 数据要跨 rank 收集 → DCP 通信路径变长
- interleave_size=16 时，命中 block 后 K/V 都在本地 → 更高效

→ **prefix cache 命中率与物理执行代价是不同问题**。interleave_size = block_size 可作为候选，但默认和最佳值受配置/模型影响，必须用长上下文负载验证。

4. 200K prompt + 单机 8 卡塞不下激活，两种配置 + 利弊。

配置 A (不带 PCP): 纯 TP + chunked prefill

```
--tensor-parallel-size 8
--max-num-batched-tokens 4096 # 切小 chunk
```

- 200K prompt 被 chunked prefill 切成 50 个 4K chunk
- 每 chunk 一步 forward，激活只 [4096, H] 大小——单机 8 卡能装
- TTFT ≈ 50 步 × 单步时长 ≈ 几秒
- **利:** 实现简单，无需特殊硬件
- **弊:** TTFT 慢; 50 次 schedule 开销累积; 每步要 reload 已 cached 部分的 KV (重算 attention)

配置 B (带 PCP): TP-8 + PCP-2, 跨 2 机

```
--tensor-parallel-size 8
--prefill-context-parallel-size 2
```

- 200K prompt 按 seq 维度切 2 段，每段 100K，分别在 2 机 8 卡上
- 一次 forward 完成 prefill (PCP 间 ring attention 跨机交换 K/V)
- TTFT 接近"单卡跑 100K"的时长
- **利:** TTFT 显著快
- **弊:** 需要 2 机 + 高速互联 (IB / RoCE)，ring attention 跨机有通信开销; vLLM 的 PCP 还在路上，部分 attention backend 不完善

何时选哪个：

- 偶发长 prompt + 没有 RDMA 集群 → 配置 A
- 长上下文是核心 KPI + 有 RDMA → 配置 B
- 多数请求 $\leq 32K$ ，只有 1% 是 200K → 配置 A 简单

补充细节：还可以混合用——长 prompt 走 PCP 集群，短 prompt 走 TP 集群，前端 router 按 prompt 长度路由。

下一步

源码锚点：[vllm/config/parallel.py](#) · [ParallelConfig](#)

源码锚点：[vllm/config/parallel.py](#) · [ParallelConfig](#)

- 想看源码：[vllm/config/parallel.py](#)（CP 配置）、[vllm/v1/worker/cp_utils.py](#)（CP 工具函数）、[vllm/v1/attention/backends/](#)（attention 后端怎么消费 CP metadata）。

源码锚点：[vllm/v1/core/kv_cache_utils.py](#) · [resolve_kv_cache_block_sizes](#)

- 想理解 KV cache 在 CP 下怎么寻址：[vllm/v1/core/kv_cache_utils.py](#) ([resolve_kv_cache_block_sizes](#))。
- 想从生产视角理解：[08-production-deployment/04-autoscaling-and-capacity.md](#)（长上下文容量算法）。
- 想看 Ring Attention 论文（PCP 的理论基础）：Liu et al., 2023, *Ring Attention with Blockwise Transformers*。

05. 千卡 / 万卡集群推理：规模化才暴露的工程问题

谁该读这一篇？ 要把一个大模型（尤其大 MoE，如 DeepSeek 量级）部署到几百、上千、上万张卡上的人。你已经懂 TP/PP/EP/DP 怎么切（前四章），现在要回答的是另一类问题：**这些机制在规模上去之后会以什么方式坏掉、怎么定位、怎么救。**

前置阅读： 01-tp-pp-ep.md（并行机制 + § 11 故障传染）、02-disaggregated.md（P/D 分离）、03-expert-parallel-deep-dive.md（AllToAll / EPLB）。本章是这三章在"千卡/万卡"尺度上的实战延伸。

耗时： 约 30 分钟

学完能：

- 1. 说清"规模上去"为什么是质变而不是量变——通信墙、故障墙、长尾墙三条。
- 2. 画出一个大 MoE 服务单元的真实形态（DP-attention + 大 EP），并解释它为什么动辄上百卡。
- 3. 对 6 类规模化典型故障，给出"现象 → 怀疑 → 验证 → 处置"的工业排查路径。
- 4. 解释 EPLB / DP padding 对齐 / 微批重叠 / 弹性 EP / blast radius 控制各自治的是哪个规模化病。

当前源码复核（b23bd73）： 本章同时包含 vLLM 已实现机制与平台架构指导。Elastic EP 已有显式 scale API、standby group、weight transfer、EPLB reshuffle 与 rewarm 流程，但受 EPLB、PP、LB 组合限制；K8s 自愈、全局调度、拓扑放置、watchdog 与多集群容灾仍由平台负责，不是 vLLM 自动保证。

前四章讲"怎么把模型切到多卡"。这一章讲一件不一样的事：当卡数从 8 涨到 8000，**很多在小集群上无所谓的设计，会变成压垮服务的主因**。规模不是把同样的问题放大，而是引入新问题。

1. 规模上去，质变在哪：三堵墙

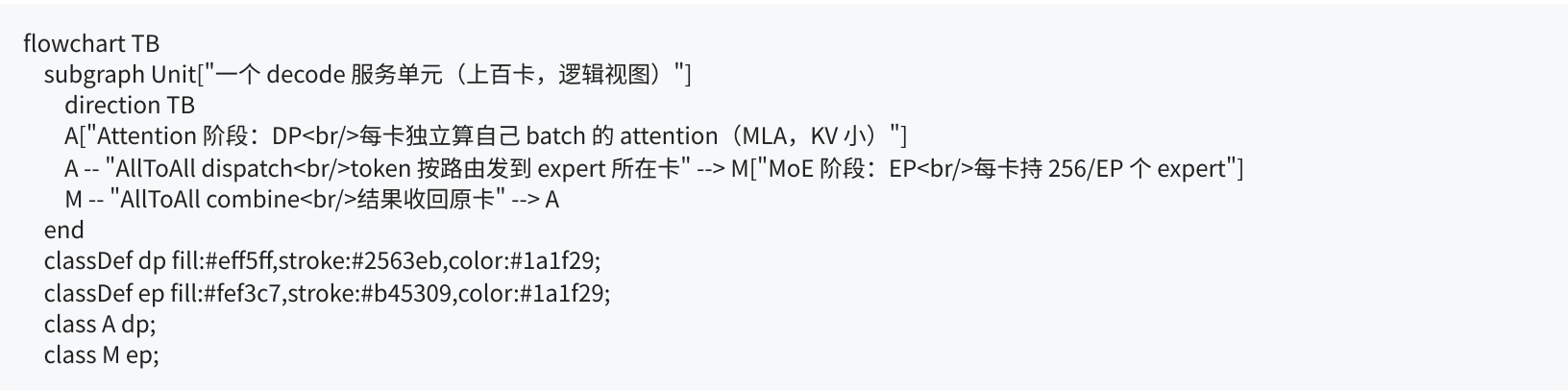
墙	小集群（≤16 卡）	千卡 / 万卡
通信墙	NVLink 机内全连，通信几乎免费	跨机、跨 rail、跨 spine；网络成为吞吐的硬限制，AllToAll/AllReduce 打满网卡
故障墙	几周挂一次，手动重启	单卡年故障率 × 万卡 → 每天挂若干次 ，必须自动化；NCCL 是 fail-stop，一个 rank 死/慢拖垮整组
长尾墙	batch 内长尾，单实例可控	集合通信是同步屏障 ——最慢的那张卡决定整组的步长，一张慢卡污染上百卡

这三堵墙是后面所有具体问题的根。记住一个贯穿全章的判断：**在大规模同步并行里，性能不由平均值决定，由最慢的那个 rank 决定 (tail-bound)；可用性不由单卡可靠性决定，由 blast radius（一次故障波及多少卡）决定。**

2. 一个大 MoE 服务单元长什么样

先建立画面，否则后面问题没有落点。以 DeepSeek-V3/R1 这类 "大 MoE + MLA" 模型为例，**单个服务单元（serving unit）** 通常是这
样组织的（具体数字随版本/硬件变，这里取量级）：

- **Attention 部分走 DP**：每张卡持有完整的注意力权重（MLA 把 KV 压成低秩 latent，KV 很小），各卡处理**不同请求**的 attention。
- **MoE 部分走 EP**：256 个 logical expert 摊到几十张卡上，每张卡只持有几个 expert 的权重；每个 token 经路由发到它命中的 expert 所在的卡（AllToAll），算完再收回（AllToAll）。
- 于是**同一批卡，在 attention 阶段是 DP、在 MoE 阶段是 EP**——这就是 "DP-attention + EP-MoE" 形态。
- 规模上：DeepSeek 公布过 **prefill 阶段 EP32、decode 阶段 EP320**（含冗余专家）这一量级。也就是说，**一个模型实例的 decode 就横跨上百张卡**，再把这样的实例复制 N 份填满集群。



关键认知：**这个单元里每一步 forward 都有两次全员参与的 AllToAll（dispatch + combine），它们是同步点。** 第 3、4 节的长尾和通信问题全从这两次 AllToAll 长出来。

为什么 attention 用 DP 而不是 TP？因为 MLA 的 KV 已经很小，attention 不是显存瓶颈；DP 避免了 attention 里的 AllReduce，把宝贵的网络带宽全留给 MoE 的 AllToAll。这是大 MoE 部署的标志性取舍，参见 07-model-architectures.md（MLA）与 03-expert-parallel-deep-dive.md。

3. 问题一：EP 同步长尾 —— 一张慢卡拖垮整组（最核心）

这是大规模 MoE 推理**最普遍、最隐蔽**的性能问题。

机理。 decode 每步有两次 AllToAll，它们是同步屏障：所有 EP rank 必须都到达才能继续。于是这一步的耗时 = max(各 rank 的计算时间)，而不是平均。任何让某个 rank 变慢的因素，都会让其余几百张卡空等：

- **Expert 负载不均（hot expert）**：路由是数据相关的，某些 expert 天然更热门，承载它的卡 token 多、算得慢，其余卡等它。
- **DP-attention 各 rank batch 不等**：DP 下不同 rank 分到的请求数/序列长度不同 → attention 阶段 token 数不等 → 到 AllToAll 处对齐时，token 少的 rank 早早算完干等。
- **落后的慢卡（straggler）**：ECC 错误重试、thermal throttle 降频、邻居噪声（同机其他实例抢 PCIe/内存带宽）。

分析思路。 1. 看**每个 rank 每步的计算时长直方图**——是个别 rank 长期偏慢（straggler）还是随 batch 抖动（负载不均）。2. 看**AllToAll 等待时间占 step 的比例**——高占比说明在同步点空等。3. 看**expert 命中分布**（vLLM 的 RoutedExpertsCapturer 统计）——是否高度集中在少数 expert。4. 看**各 DP rank 的 batch/token 数方差**——方差大说明 DP 负载没均衡。

处置（按问题对症）。

病因	解法	vLLM 里
Hot expert	EPLB ：给热门 logical expert 加冗余副本（redundant expert），把热度摊到多张卡；周期性 rearrange physical→device 映射	vllm/distributed/eplb/eplb_state.py；DeepSeek-R1 例：256 logical + 32 redundant = 288 physical，32 EP rank → 每卡 9 个
DP rank 负载不均	DP token 对齐（padding） ：每步用一次轻量 all-reduce 同步各 rank 的 token 数，padding 到一致，避免在 AllToAll 处长短不齐	vllm/v1/worker/dp_utils.py（DP padding 同步）
同步点空等本身	微批重叠（DBO / ubatch） ：把一个 batch 拆两个 micro-batch 交错跑，micro-batch A 在 AllToAll 通信时，micro-batch B 在算 attention——用计算把通信时间藏起来	vllm/v1/worker/ubatching.py、gpu_ubatch_wrapper.py
个别慢卡	慢节点检测 + 主动驱逐（见 § 5）	运维层

EPLB 的代价：rearrange 要搬 expert 权重（几百 MB～GB 级），不能太频繁；而且加冗余 expert 吃显存。所以**不是默认开**——只在监控确认负载不均显著、且持续时才开（判据见 03-expert-parallel-deep-dive.md § 6.4）。

4. 问题二：AllToAll 撞上网络墙

EP 越大，AllToAll 越是 all-pairs 通信。万卡尺度上，网络（IB/RoCE）才是真正的天花板，不是算力。

机理。 - **All-pairs 流量**：EP=320 的 dispatch 理论上是 320×320 的点对点，跨机流量极大。 - **拓扑不友好**：跨 rail（不同网卡平面）、跨 spine 的流量要走更高层交换机，带宽更窄、跳数更多。 - **ECMP hash 碰撞**：多条 RDMA 流被哈希到同一条物理链路 → 单链路热点，明明总带宽够却卡在一条链路上。 - **Incast**：combine 阶段多卡同时往一卡回灌 → 交换机队列爆、丢包、触发拥塞控制（DCQCN）退避，延迟飙升。

分析思路。 1. 实测 AllToAll 时延 vs 理论值（03-expert-parallel-deep-dive.md § 5 给了各 backend 的量级）。慢一个数量级 → 网络问题。 2. 看 NIC 收发带宽是否打满、交换机有没有丢包/PFC pause、队列深度。 3. 看是不是个别链路热（ECMP 碰撞）而非整体带宽不足。

处置。 - **分层 AllToAll**：机内走 NVLink、机间走 IB，两级聚合，把跨机流量压到最小——这正是 **DeepEP** 干的事（NVSHMEM + IBGDA，GPU 直发 RDMA 不经 CPU）。vLLM 通过 `--all2all-backend deepep_high_throughput`（prefill）/ `deepep_low_latency`（decode）启用。 - **拓扑感知放置**：让同一个 EP 组尽量落在同 rail/同 leaf 下，减少跨 spine；K8s 上用 topology-aware 调度 + NCCL_IB_HCA 绑定网卡平面。 - **combine 端 reduce 下沉**：在交换机做 in-network reduction（SHARP）或分层规约，缓解 incast。 - **拥塞控制调参**：RoCE 下调 DCQCN/PFC，避免 pause 风暴。

一句话：**大 MoE 的通用 collective 很可能先撞网络/同步墙，但是否需要 DeepEP 要由 topology、backend 支持和实测决定**（参见 03-expert-parallel-deep-dive.md § 4）。

5. 问题三：故障墙 —— 万卡每天都在挂

机理（先算一笔账）。假设单卡 + 其网卡/光模块的年故障率合计 2%，万卡集群期望年故障 ≈ 200 次，**平均一两天就挂一次**；叠加链路抖动、ECC、thermal，实际"非完全健康"事件远多于此。在这个尺度，故障不是异常，是常态，必须设计成自愈。

最致命的一点：NCCL 是 fail-stop 的。一个 rank crash 或 hang，同一个 communicator 里的其他 rank 会在下一次集合通信处一起 **hang**，默认要等到 watchdog timeout（几分钟）才报错退出。也就是说：

- 一个 8 卡 TP 实例里挂 1 张卡 = **整个实例死**。
- 一个 EP320 的单元里有 1 张卡 hang = **这上百卡全卡住**直到超时。

这把"blast radius"变成第一设计原则。

分析思路：区分三种"坏"。 | 类型 | 现象 | 怎么认 | | --- | --- | --- | | **Crash**（卡/进程死） | 某 rank 进程退出、NCCL 报 unhandled cuda error / remote process exited | 进程监控 + NCCL 错误日志 | | **Hang**（卡住不动） | 整组步频归零、watchdog timeout、GPU-Util 100% 但无产出 | NCCL watchdog、心跳超时 | | **Straggler**（变慢不死） | 整组步频下降但没报错，某 rank 每步偏慢 | per-rank step timing 直方图（§ 3） |

最难的是 straggler——不报错，只是慢，监控不盯 per-rank timing 根本发现不了。

处置。 - **控制 blast radius：**实例（TP×PP 的 communicator）尽量小。**不要为省显存无脑拉大 TP**——TP 越大，单卡故障炸掉的范围越大。能用 DP 多副本就别用大 TP。 - **利用"推理无状态"：**推理实例除了 KV cache 没有需要持久化的状态，KV 可重算。所以恢复策略简单粗暴：**从 LB 摘除 → 杀掉整个实例 → 拉新实例 → warmup 后重新入流量**。配合冗余副本 + 全局 LB，单实例死对用户近乎无感（in-flight 的请求重试到别的副本）。 - **快速失败而非长等：**调短 NCCL watchdog/心跳超时，让 hang 尽快暴露并触发重建，而不是干等几分钟。 - **慢卡主动驱逐：**监控 ECC 计数、SM 时钟、per-rank timing，命中阈值就把该节点 cordon 掉，别让它当 straggler 拖整组。 - **弹性 EP（进阶）：**当前实现通过显式 scaling API、drain、standby group、权重传输、expert 映射切换与 rewarm 增减 DP/EP 资源；它不是“发现坏卡即自动无损替换”，仍需平台触发、健康判定和失败回滚。 - **K8s 配套：**liveness/readiness 探针 + 反亲和（同实例的 rank 别全堆一台机的同一故障域）+ PodDisruptionBudget（01-tp-pp-ep.md § 11.2、06-reliability-and-failure-modes.md）。

6. 问题四：冷启动 / 权重加载 / rollout 被规模放大

小集群忽略的环节，万卡上会变成事故。

机理。 - **权重加载风暴：**几百 GB 权重 × 数千张卡**同时**从共享存储（NFS/对象存储）拉 → 存储 IOPS 和出口带宽被打爆，启动时间从分钟变成几十分钟。 - **CUDA Graph capture × 数千实例：**每个实例都要 capture 多个 batch size 的图，单实例几十秒，乘以规模 + 抢占资源 → 漫长。 - **滚动重启慢：**要换版本/改配置时，逐批重启一个万卡 fleet 可能要数小时，期间容量打折。

处置。 - **权重不要让每张卡各自去存储拉：**一处读入，再用 RDMA / tree broadcast 在集群内扩散（"读一次、广播 N 份"）；或用节点本地只读缓存 + 预分发。 - **分批 rollout + warmup gate：**就绪探针必须**等 CUDA Graph capture + warmup 完成**才放流量，否则新副本一上线就长尾爆炸（10-gpu-utilization-and-tail-latency.md § 6.7）。 - **sleep / wake 复用实例：**vLLM 支持把实例置于 sleep（卸 KV、留权重）再快速唤醒（metric vllm:engine_sleep_state），切模型/缩容时比冷启动快得多。 - **容量规划留 rollout 余量：**滚动重启期间的容量缺口要提前用 buffer 副本补上（04-autoscaling-and-capacity.md）。

7. 问题五：跨数千副本的全局调度与路由

单副本内的调度（Scheduler）前面讲过；万卡上多了一层**跨副本的全局路由**问题。

机理。 - **Prefix-cache 亲和失效**：同一会话/同一系统 prompt 的请求若被随机打散到不同副本，本可命中的 KV 前缀全 miss，TTFT 抬升、算力浪费。规模越大、副本越多，随机打散的代价越大。 - **P/D 分离下的 KV 路由**：prefill 池算完要把 KV 传给某个 decode 实例（NIXL/RDMA），选哪个 decode 实例、怎么不打爆它，是全局决策。 - **热点副本**：负载/KV 占用不均，部分副本过载抢占、部分空闲。

处置。 - **分层路由**：上层按区域/容量做粗负载均衡，下层按 **prefix 亲和 + KV 占用 + 队列深度** 打分选副本（02-smart-routing-and-load-balancing.md）。 - **P/D 比例动态调整**：prefill 重（长 prompt 多）就加 prefill 池，decode 重就加 decode 池；workload 漂移时在线调比例（02-disaggregated.md）。 - **全局调度器形态**：llm-d / AIBrix / NVIDIA Dynamo 这类"集群级 inference gateway + 全局调度"就是冲这个问题去的（01-deployment-architectures.md）。

8. 问题六：万卡上的可观测性 —— 找到那一张慢卡

规模化让监控本身变难。

机理与难点。 - **基数爆炸**：per-rank $\times$ per-layer $\times$ 万卡的指标维度，Prometheus 直接被打爆。 - **均值骗人**：集群平均吞吐很好看，但 S3 说了性能是 tail-bound——你要找的是**那一张拖后腿的卡**，它淹没在均值里。

处置。 - **盯分布而非均值**：per-rank step time 的 p99/max、各 DP rank 的 token 数方差、AllToAll 等待时长——这些才暴露 straggler 和负载不均。 - **慢节点雷达**：一个专门的"哪张卡最慢"视图（按 per-rank timing 排序），比一堆均值面板有用。 - **关键先行指标**：vllm:num_preemptions_total（KV 压力）、AllToAll wait 占比、NCCL timeout 计数、DP imbalance、vllm:engine_sleep_state。 - **采样式分布式 trace**：全量 trace 万卡不现实，按比例采样 + 异常请求强制采样。 - 基础打法见 05-slo-and-observability.md 与 08-monitoring-cookbook.md，本节是它们在万卡尺度的补充。

9. 规模化排查总表（现象 → 怀疑 → 验证 → 处置）

把全章压成一张工业 runbook：

现象	第一怀疑	验证	处置
整组吞吐低、各 rank 计算时长方差大	DP/expert 负载不均	per-rank token 数与 step 时长直方图、expert 命中分布	DP padding 对齐；EPLB；微批重叠
整组吞吐低、某一两 rank 长期偏慢	straggler（ECC/throttle/邻居噪声）	per-rank timing 排序、nvidia-smi 时钟/ECC	cordon 驱逐慢卡 → 重建实例
forward 慢、网络带宽高/丢包	AllToAll 撞网络墙	AllToAll 实测 vs 理论、NIC/交换机队列、ECMP 热点	DeepEP/分层 AllToAll；拓扑感知放置；调拥塞控制
整组步频归零、GPU-Util 100% 无产出	NCCL hang（某 rank 死/卡）	NCCL watchdog 日志、心跳	调短 timeout 快速失败 → 摘流量 → 重建整实例
启动/扩容极慢	权重加载风暴 / capture 慢	存储出口带宽、就绪耗时	广播式加载；分批 rollout；warmup gate；sleep/wake
TTFT 抬升、prefix 命中率掉	跨副本路由没做亲和	vllm:prefix_cache_hit_rate、路由日志	prefix-aware/session-sticky 路由
部分副本过载抢占、部分空闲	全局负载不均 / P/D 比例失配	各副本 num_requests_running/KV 占用	负载打分路由；动态调 P/D 比例；扩容

小结

- 规模上去是**质变**：通信墙（网络成瓶颈）、故障墙（挂卡成常态）、长尾墙（最慢的 rank 决定整组）。
- 大 MoE 服务单元 = **DP-attention + 大 EP**，单实例横跨上百卡，每步两次 AllToAll 是同步点。
- EP/DP 同步长尾**是最核心的性能病：hot expert → EPLB，DP rank 不均 → token padding 对齐，同步空等 → 微批重叠（DBO）。
- AllToAll 撞网络墙**靠分层通信（DeepEP/NVSHMEM）+ 拓扑感知放置 + 拥塞控制。
- 故障墙**的第一原则是控制 blast radius（别无脑拉大 TP）+ 利用推理无状态做快速重建 + 慢卡驱逐 + 弹性 EP。
- 冷启动/路由/可观测性在万卡上都被放大，分别用广播加载/warmup gate、分层亲和路由、盯分布而非均值来治。

自检（先自答，再看要点）

1. 为什么说大规模同步并行里"性能由最慢的 rank 决定"？这对监控有什么直接含义？

要点：每步的 AllReduce/AllToAll 是同步屏障，step 耗时 = max(各 rank 时长)，所以一张慢卡拖整组。含义：监控必须盯 **per-rank 时长分布的 p99/max 和方差**，不能只看集群均值——均值好看的同时可能有一张卡在拖后腿（straggler）。

2. 一个 EP320 的 decode 单元里有一张卡 hang 了，会发生什么？为什么这影响"该不该把 TP 拉很大"的决策？

要点：NCCL fail-stop，这上百卡会在下一次 AllToAll 一起 hang 到 watchdog timeout。blast radius = 整个 communicator。所以并行维度（尤其 TP/EP communicator）越大，单卡故障炸掉的范围越大；能用 DP 多副本分散就别为省显存无脑拉大 TP。

3. DP-attention 下发现各 rank 吞吐不齐、整体被拖慢，根因和解法？

要点：DP 各 rank 分到的请求数/序列长度不同 → attention 阶段 token 数不等 → 到 MoE 的 AllToAll 同步点时长短不齐、快的等慢的。解法：每步用一次轻量 all-reduce 同步各 rank token 数并 padding 对齐（vllm/v1/worker/dp_utils.py），再叠加微批重叠把通信藏到计算后。

4. EPLB、DP token padding、微批重叠，三者治的分别是哪个问题？

要点：EPLB 治 **expert 维度**的负载不均（hot expert，加冗余副本+重排）；DP token padding 治 **DP rank 维度**的 token 数不齐；微批重叠（DBO/ubatch）治 **同步点空等**本身（用计算掩盖 AllToAll 通信）。三者正交，可叠加。

5. 万卡集群权重加载为什么会成为瓶颈？怎么解？

要点：几百 GB × 数千卡同时从共享存储拉 → 存储出口带宽/IOPS 被打爆，启动几十分钟。解法：一处读入再 RDMA/tree broadcast 扩散，或节点本地只读缓存预分发；配合分批 rollout 和 warmup gate（capture 完才放流量）。

下一步

- 机制回看：03-expert-parallel-deep-dive.md（AllToAll 6 后端、EPLB 细节）、02-disaggregated.md（P/D 分离与 KV 传输）、01-tp-pp-ep.md § 11（故障传染）。
- 生产配套：06-reliability-and-failure-modes.md、07-incident-playbook.md、10-gpu-utilization-and-tail-latency.md（长尾治理）。
- 部署形态：01-deployment-architectures.md（llm-d / AIBrix 等集群级栈）、02-smart-routing-and-load-balancing.md（全局路由）。
- 384 卡实战：13-384-h100-glm-deepseek-deployment.md（H100）与 14-384-ascend-910b-glm-deepseek-deployment.md（昇腾 910B），覆盖 GLM-5.2 与 DeepSeek-V4 Flash/Pro 的具体并行、容量、门禁和故障演练。
- 想看源码：vllm/distributed/eplb/（EPLB）、vllm/v1/worker/dp_utils.py（DP padding）、vllm/v1/worker/ubatching.py（微批重叠）、vllm/distributed/device_communicators/all2all.py（AllToAll 后端）。

vLLM 工程面试：30 道三层回答

谁该读这一篇？ 已读过核心概念、源码走读与生产章节，准备把“知道名词”升级为“能解释机制、边界和验证方法”的工程师。

前置阅读： PagedAttention、入口与主循环、分布式并行、生产部署架构

耗时： 约 70 分钟；建议每题先口述，再看参考答案。

难度： 中高级

当前性说明： 本章按 vLLM b23bd73f540175f9e117eae5029cd7d8df63964 静态复核；没有在当前 SHA 上执行 GPU benchmark。性能与默认值问题必须回答“目标版本 / 模型 / 硬件 / workload 下测得”，不能背固定倍数。

学完能：

1. 每题先给 30 秒结论，再用 3 分钟讲机制与源码入口
2. 主动补充生产取舍、验证信号、失败边界与回滚条件
3. 面对追问时区分稳定契约、当前实现与待测假设

回答协议

每题都按五格回答：

- **30 秒结论：** 先回答“是什么 / 为什么”。
- **3 分钟机制与源码：** 讲数据流、状态与源码入口，不背行号。
- **生产取舍：** 说明收益、成本和适用条件。
- **验证、失败与回滚：** 给证据；什么现象会否决方案；怎么安全退回。
- **追问：** 至少准备两个能继续展开的问题。

面试官真正区分的是：候选人能否把结论限定在证据范围内，而不是能否报出最多的 feature 名。

A. 引擎与请求生命周期

Q1. vLLM 解决什么问题？

30 秒结论： vLLM 是面向 LLM inference / serving 的引擎。它把请求调度、KV cache、模型执行、采样 / pooling、并行与 OpenAI-compatible serving 组合起来，目标是在给定质量与延迟 SLO 下提高 goodput；Paged KV 与迭代级调度是核心机制，但不是性能承诺。

3 分钟机制与源码： 用户入口在 vllm/entrypoints/，V1 engine loop 在 vllm/v1/engine/，调度在 vllm/v1/core/sched/scheduler.py，worker / runner 在 vllm/v1/worker/。一次请求跨 frontend、EngineCore、worker；每 step 重新形成 batch，执行模型并把结果回传。生成请求走 sampler，pooling runner 走 pooler。

生产取舍： vLLM 提供通用能力，具体模型、量化、attention backend 和平台组合仍有支持矩阵。选择引擎要比较质量、SLO、运维与升级成本，不能引用脱离环境的吞吐倍数。

验证、失败与回滚： 固定模型 revision、tokenizer、dtype、prompt / output 分布和并发，比较 p50 / p99 TTFT、TPOT、goodput、显存与错误率。若质量或尾延迟不达门禁，回到已验证 image / SHA / config。

追问： V1 的进程边界是什么？为什么 tokens/s 不是唯一选型指标？

Q2. 一个生成请求如何走完全链路？

30 秒结论： frontend 校验、渲染并 tokenize，请求进入 EngineCore waiting；Scheduler 分配 token budget 与 KV block；worker forward + sample；EngineCore 更新状态，直到 stop / EOS / length / abort，再释放资源并流式返回。

3 分钟机制与源码： 入口请求转换位于 vllm/entrypoints/openai/；vllm/v1/request.py 保存请求状态；Scheduler.schedule() 产出 SchedulerOutput；GPU runner 维护 persistent batch，执行 model 与 sampler；engine core 根据输出更新 token、finish reason 和队列。取消、超时与客户端断开也必须传播到资源释放路径。

生产取舍： frontend CPU、engine queue、GPU step、网络流式任一段都可能决定尾延迟；只看 GPU utilization 会误诊。

验证、失败与回滚： 用 request ID 关联 gateway、frontend、engine 与 worker 证据，分解 queue、tokenization、prefill、decode、serialization。失败注入客户端取消与 worker 异常，确认请求不泄漏 KV block。

追问： streaming 首字节与 TTFT 有何区别？abort 如何传到 Scheduler？

Q3. Continuous batching 与 static batching 有什么差别？

30 秒结论： static batch 等一组序列共同完成；continuous batching 在迭代边界重新调度，完成请求退出、等待请求进入，因此能处理长度异质性并提高资源利用率。

3 分钟机制与源码： vllm/v1/core/sched/scheduler.py 每 step 在 token budget、KV capacity、优先级与模型约束下选择请求；runner 的 persistent batch 复用相邻 step 中仍运行的请求。chunked prefill 还允许长 prompt 分段，与 decode 请求共享 step。

生产取舍： 更大 batch 可能提高吞吐，却放大单 step 时间与 TPOT；混合长 prefill 还会影响公平性。连续调度不是“GPU 永远 100%”。

验证、失败与回滚： 扫描并发和长度混合，记录 batch tokens、queue time、TTFT / TPOT p99 与 preemption。若 p99 越线，回退 token budget 或拆分 workload pool。

追问： 为什么 iteration-level 不等于任意 token 可随时插入 kernel？persistent batch 优化依赖什么流量特征？

Q4. V1 的 Scheduler 每步决定什么？

30 秒结论： 它决定哪些请求本步计算多少 token、需要哪些 KV block、哪些 encoder / structured-output / speculative 工作可调度，以及谁需要被推迟或抢占。

3 分钟机制与源码： 主入口是 vllm/v1/core/sched/scheduler.py。约束来自 max_num_batched_tokens、max_num_seqs、KV cache manager、encoder budget、请求状态与调度策略；输出包含新 / cached 请求、本步 token 数、block IDs、LoRA / structured-output 等元数据。

生产取舍： 调度策略在 throughput、fairness、priority 与尾延迟之间取舍。配置值是容量上限，不是 SLO 保证。

验证、失败与回滚：构造短 decode、长 prefill、priority 与 KV 压力混合流量，检查是否饥饿、是否频繁 preempt。策略改变先 canary；发现低优先级饿死或高优先级无界抢占时回滚。

追问：token budget 与 KV capacity 为什么是两种约束？waiting 队首不可调度时是否应该跳过？

B. KV cache、分页与前缀复用

Q5. Paged KV cache 解决什么？

30 秒结论：它把请求的逻辑 token 位置映射到固定大小的物理 KV block，让物理块无需连续，并支持按需分配、释放和共享前缀；主要解决连续大块预留造成的碎片与刚性容量问题。

3 分钟机制与源码：vllm/v1/core/block_pool.py 管物理 block，kv_cache_manager.py 管请求映射，attention backend 读取 block table 间接寻址。最后一个 block 仍可能有内部碎片；block metadata、table 和 kernel 也有成本。

生产取舍：block 小降低尾块浪费，却增加 metadata / table 长度；block 大相反。默认值与可用集合受平台 / backend 约束，不能把某个数字当普适最优。

验证、失败与回滚：扫描真实长度分布与 block size，比较可容纳 tokens、尾块浪费、kernel latency 与 p99；启动校验失败或性能退化就恢复已验证值。

追问：逻辑 block 与物理 block 如何解耦？为什么分页仍有内部碎片？

Q6. KV cache 每 token 多少字节？

30 秒结论：对常见 decoder attention，一个 token 的近似 KV 字节为 $2 \times L \times H_{kv} \times D_{head} \times \text{bytes(dtype)}$ ；TP 后每 rank 通常只持有本 rank 的 KV heads，但 MLA、跨层共享、混合 cache spec 会改变公式。

3 分钟机制与源码：2 是 K 与 V；L 为缓存层数； H_{kv} 是 KV head 数而非 query head 数。实际 cache spec 位于 vllm/v1/kv_cache_interface.py，启动 profile 与 cache config 决定物理页面数。GQA / MQA 通过减少 KV heads 降低容量。

生产取舍：FP8 KV 可能省显存，但需要 backend / scale / 精度验证；增加 context 或并发是同一 KV 预算的竞争。

验证、失败与回滚：用模型 config 算理论 bytes/token，再用启动日志的 page size / page count 与 GPU 增量校验；误差大时检查 TP 分片、hybrid layer、padding 与 allocator reserve。

追问：为什么 hidden size 不能直接代替 KV heads 计算？PP 会怎样改变每 rank KV？

Q7. vLLM 如何决定 KV cache 容量？

30 秒结论：启动时先加载模型并 profile 非 KV 峰值，再在配置允许的 GPU memory budget 中给 KV cache 分配页面；显式 KV 字节配置、offload 与不同平台会改变路径。

3 分钟机制与源码：配置在 vllm/config/cache.py，worker 的 memory profiling 与 vllm/v1/kv_cache_interface.py 共同生成 cache config。不能简单写成“总显存乘 utilization 减权重”，因为 graph、activation、非 Torch 内存、并行 rank 与实现 reserve 都参与。

生产取舍：把 utilization 推高可增加 KV，却缩小瞬时峰值余量；跳过 profile / 固定 cache 便于复现，但操作者承担 OOM 风险。

验证、失败与回滚： 保存每 rank 启动 profile、page count、空闲余量，跑最大允许输入与混合流量 soak。出现启动失败或运行 OOM，先恢复旧预算并降低 admission，而不是盲目重试。

追问： 为什么 profile 通过仍可能运行 OOM？CUDA Graph capture 对预算有什么影响？

Q8. Prefix caching 的 hash 为什么要链式？

30 秒结论： 当前 block hash 包含前一 block hash、当前完整 block token 与 extra keys；链式结构让相同 token block 在不同前缀下产生不同 key，避免把语义不同的 KV 错误复用。

3 分钟机制与源码： 入口在 vllm/v1/core/kv_cache_utils.py。extra keys 覆盖多模态 identifier / offset、LoRA identity 与 cache salt 等影响 KV 的上下文。只缓存完整 block，匹配的是最长可复用前缀，不是任意子串。

生产取舍： 更强隔离减少错误命中，但 salt / 路由不一致会降低 hit rate；命中率高也不等于节省了足够 GPU 时间。

验证、失败与回滚： 用同前缀、单 token 差异、不同 LoRA / 图片 / salt 做正反例；同时看 hit tokens、TTFT 与输出一致性。疑似错误命中时立即禁用 prefix caching 并隔离版本。

追问： 为什么只 hash token IDs 不够？tokenizer / chat template 升级如何影响 cache？

Q9. ref count、free queue 与驱逐是什么关系？

30 秒结论： 被请求引用的 block 不能作为新分配复用；引用归零后进入可回收集合，但若仍保留 hash metadata，未来可在被真正复用前命中 prefix cache。

3 分钟机制与源码： vllm/v1/core/block_pool.py 维护 block、free list 与 hash map；cache manager 在请求完成、abort 或 preemption 时释放引用。具体共享 / copy-on-write 能力必须以当前 V1 请求路径为准，不能把旧 beam-search 实现直接套来。

生产取舍： 延迟驱逐提高命中机会，却占用可观察的 cache working set；高 churn 会让 hash 命中与实际保留时间下降。

验证、失败与回滚： 反复请求共享前缀并注入 cache pressure，观察命中、驱逐与 block 使用是否守恒；abort 后 block 不归还属于资源泄漏故障。

追问： ref count 为零为何不等于立刻清空数据？并发命中如何避免驱逐竞态？

C. 调度、抢占与输出

Q10. KV 不够时为什么会 preempt？

30 秒结论： Scheduler 若无法为本步请求分配足够 KV，就必须释放部分运行请求的 block，让系统继续前进；被抢占请求未来重算或按实现支持的方式恢复。

3 分钟机制与源码： 抢占逻辑在 vllm/v1/core/sched/scheduler.py 与 cache manager 交界。当前策略、优先级与重算细节要读锁定 SHA，不能假设所有平台都有 host swap。preemption 会增加重复 prefill、TTFT 与 goodput 损失。

生产取舍： 提高 max_num_seqs 可增加表面并发，却可能造成 thrash；降低并发、缩短上下文、增加 KV 或扩副本更稳。

验证、失败与回滚： 监测当前版本注册的 preemption counter、KV usage、waiting / running 与 TTFT。持续抢占时先 admission / 并发回滚，不能只扩 retry。

追问： 为什么 preemption counter 增长可能导致 retry storm? priority preemption 如何避免低优先级饿死?

Q11. Chunked prefill 解决什么?

30 秒结论： 它把长 prompt 的 prefill 拆成多 step，在 token budget 内与其他请求交错，降低单个长 prompt 独占一步造成的 decode 抖动。

3 分钟机制与源码： Scheduler 给请求安排本步 num_scheduled_tokens；attention 读取已有 KV 与当前 chunk。相关配置在 vllm/config/scheduler.py，多模态还有 disable_chunked_mm_input 等边界。

生产取舍： chunk 小改善公平性，却增加 step / launch / 调度次数；chunk 大提高效率但可能抬高 TPOT p99。

验证、失败与回滚： 用双峰 prompt 长度同时测 TTFT、TPOT 与吞吐，逐步扫描 token budget；出现质量差异或特定模型 / 多模态不支持时回到默认并隔离长请求池。

追问： chunked prefill 与 prefix cache 如何相互作用？为什么长上下文 attention 仍然昂贵？

Q12. seed 能保证跨部署 bitwise deterministic 吗?

30 秒结论： 不能。seed 控制 per-request generator 的随机序列，但 backend、batch 组成、并行规约、浮点精度、driver / kernel 与模型 revision 都可能改变 logits 或采样路径。

3 分钟机制与源码： vllm/sampling_params.py 区分 greedy、random 与 random-seeded；vllm/v1/sample/ 选择 sampling backend。带 generator 还可能让 top-k/top-p 走不同 fallback。greedy 也可能在近似相等 logits 上受数值差异影响。

生产取舍： 强复现通常牺牲 batching、backend 自由度或性能。生产更应定义输出 / quality 容差，而非承诺跨硬件逐 token 相同。

验证、失败与回滚： 固定完整环境与流量顺序重复运行，分别测同进程、重启、跨卡；把确定性等级写进 API 契约。达不到时撤销 bitwise 承诺。

追问： temperature=0 是否足够？为什么 per-request generator 可能改变 backend？

Q13. Sampling 的处理顺序为什么重要?

30 秒结论： raw logprobs、allowed / bad words、自定义 processors、penalties、temperature、min-p、top-k/top-p 与 gather 的先后会改变分布和 API 语义，不能随意交换。

3 分钟机制与源码： 当前顺序在 vllm/v1/sample/sampler.py：先按 mode 保存 raw 值并转 FP32，再应用约束、非 argmax-invariant processor 与 penalties；随机路径应用 temperature、argmax-invariant processor、min-p / top-k / top-p，最后 gather。具体 backend 见 ops/topk_topp_sampler.py。

生产取舍： logprobs 与异质 sampling 参数增加 full-vocab 计算、返回体与 batch 分支；custom processor 还有限制。

验证、失败与回滚： 用可手算小 vocab 单测顺序，检查 sampled token、rank 和 raw / processed logprobs；升级后契约变化则版本化响应或回滚。

追问： raw 与 processed logprobs 有何区别？为什么 top-N 返回仍可能需要 full-vocab softmax？

D. Attention、runner 与编译

Q14. Attention backend 如何选择？

30 秒结论： 由平台、attention type、dtype、head size、cache layout、模型功能与显式配置共同决定；不是“所有 NVIDIA 都固定某一个 backend”。

3 分钟机制与源码： registry 位于 vllm/v1/attention/backends/registry.py，backend 实现位于同目录及 MLA 子树。启动日志与 config 是实际选择证据；某 backend 支持 decode 不代表支持 encoder-only、sliding window、quantized KV 或所有 graph 模式。

生产取舍： 最快 backend 可能限制模型功能或硬件；显式强制提高可复现性，却可能在升级后启动失败。

验证、失败与回滚： 保存 backend 名、版本与启动理由，跑 correctness + shape / length matrix 与 profile。出现 unsupported 或数值回归时回到 auto / 已验证 backend。

追问： backend 与 attention layer 有什么区别？为什么同模型换 KV dtype 可能换路径？

Q15. Paged attention kernel 做了什么？

30 秒结论： 它通过 block table 找到请求逻辑 KV 位置对应的物理页面，在不要求物理连续的前提下完成 QK、softmax 与 V 聚合。

3 分钟机制与源码： vLLM 自有 kernel 在 csrc/attention/，平台 backend 也可提供 paged 实现。kernel 要处理不同 sequence length、head mapping、partition / reduce、cache dtype 与 block table。现代部署实际走哪条路径由 backend selection 决定。

生产取舍： 间接寻址换取内存弹性；长 context、page table locality 与 split 策略决定效率。不能用“分页更省显存”推导“总是更快”。

验证、失败与回滚： 用 contiguous reference 做数值对比，扫描 context / batch / head / block size；性能或正确性异常时切已验证 backend，而非现场改 kernel。

追问： 为什么 decode 常是 memory-bound？block table 何时会成为负担？

Q16. GQA / MQA 为什么能省 KV？

30 秒结论： 多个 query heads 共享更少的 KV heads，KV cache 按 num_kv_heads 存，因此容量按共享比例下降；计算时 backend 把 query head 映射到对应 KV head。

3 分钟机制与源码： 模型 config 给出 query 与 KV head 数，attention layer 与 backend 处理映射。TP 后 KV heads 的分片还受总 head 数与复制规则影响，容量应按每 rank 实际配置算。

生产取舍： 这是模型架构属性，不是 serving flag；换 GQA 模型涉及质量与 checkpoint，而非无损开关。

验证、失败与回滚： 对照 config、每 rank cache spec 与 bytes/token；发现 KV head 复制时修正容量模型。

追问： MQA 是 GQA 的什么特例？TP size 大于 KV heads 时怎么办？

Q17. CUDA Graph 与 torch.compile 分别优化什么？

30 秒结论： CUDA Graph 主要减少重复 kernel launch / CPU 提交开销；torch.compile 捕获并优化可编译图、融合或重排算子。两者可组合，但有动态 shape、自定义 op、内存地址与 warmup 成本。

3 分钟机制与源码： 配置在 vllm/config/compilation.py，V1 graph dispatch 与 capture 位于 worker / compilation 相关模块。graph mode、capture sizes 与 fallback 都是版本相关实现。

生产取舍：更多 capture size 可能改善命中，却增加启动时间和显存；强动态图功能可能回 eager。compile cache 还影响冷启动与镜像可移植性。

验证、失败与回滚：分冷 / 热启动记录 compile、capture、graph replay 命中、显存和 p99；失败时用已验证 compilation config 或 eager 做诊断，不把 eager 当默认性能结论。

追问：为什么输入地址稳定很重要？动态 LoRA / custom op 如何影响 graph？

E. 分布式与模型架构

Q18. Tensor Parallel 如何切分与通信？

30 秒结论：TP 把层内权重 / heads / hidden 维分到多个 rank，局部 matmul 后用 collective 合并。它让单模型跨卡放置，也引入每层通信与同步。

3 分钟机制与源码：核心状态与 process groups 在 vllm/distributed/parallel_state.py；column-parallel 与 row-parallel layers 位于 vllm/model_executor/layers/。具体 collective 次数受 fused layer、sequence parallel、MoE 与模型结构影响，不能固定说“每层恰好两次”。

生产取舍：TP 降低每 rank 权重 / KV，可能降低单请求 latency；跨慢链路扩 TP 会让通信吞噬收益。优先在高速互联域内 TP。

验证、失败与回滚：比较 TP=1/2/4 的 tokens/s、TTFT、collective trace、link bandwidth 与每 rank 显存；跨节点性能倒退则改为较小 TP + replica DP。

追问：row / column parallel 为什么配对？TP 与 DP 的扩容目标有什么不同？

Q19. Pipeline Parallel 的收益和 bubble 是什么？

30 秒结论：PP 把层段放到不同 stage，边界传 activation，主要用于模型放置或降低层内 collective 范围；stage 不均衡与 microbatch 不足会产生 bubble。

3 分钟机制与源码：PP rank / group 仍由 parallel_state.py 管理，模型实现按 layer range 装载。推理只有 forward，但请求 / microbatch 调度、首尾 stage 输出与不均衡仍需处理。

生产取舍：PP 边界通信通常小于大 TP collective，却增加串行 stage latency 与调度复杂度。TP×PP 必须满足 world size。

验证、失败与回滚：测每 stage 时间、bubble、边界通信和 OOM margin；发现一个 stage 长期成为瓶颈时重切层或退回 TP。

追问：为什么层数均分不一定负载均衡？PP 对 TTFT 与 throughput 的影响为何不同？

Q20. Expert Parallel 与 EPLB 解决什么？

30 秒结论：EP 把 MoE experts 分布到 ranks，token 根据 router 结果跨 rank dispatch；EPLB 通过统计负载并重排 / 复制 experts，减少热点导致的 straggler。

3 分钟机制与源码：EP / EPLB 位于 vllm/distributed/ 和 vllm/distributed/eplb/；模型 fused MoE layer 与 collective backend 共同决定数据流。世界大小、expert 数、冗余与 top-k 必须一致。

生产取舍：EP 降低每 rank expert 权重，却引入 all-to-all 与负载不均；EPLB 有观测窗口和重平衡成本。

验证、失败与回滚： 记录每 expert token、rank busy time、all-to-all 和 p99；重平衡导致抖动或收益不足时关闭 EPLB / 恢复固定映射。

追问： 为什么平均 token 数相同仍可能有 straggler？TP 与 EP 可以怎样组合？

Q21. TP / PP / DP / EP world size 怎么算？

30 秒结论： 先画 process-group 维度，常见总进程数近似 $TP \times PP \times DP$ ；EP 通常在既有 ranks 上定义 expert group，是否形成额外乘数取决于部署模式，不能盲目把四者相乘。

3 分钟机制与源码： 看 parallel_state.py 创建的 group 与启动参数，再核对每 rank coordinates。DP 是模型副本 / engine 并行，TP / PP 是单副本内部；EP 复用或重组 ranks。

生产取舍： world size 正确只说明拓扑可构造，不说明链路、NUMA、容错域或性能合理。

验证、失败与回滚： 启动时导出 rank-to-host/GPU/group 表，检查每张卡唯一归属与 group size；NCCL hang 前先验证拓扑和环境一致性。

追问： 两节点各 8 卡、TP=4、PP=2 时能有几个 DP replica？EP=8 应如何解释？

Q22. Disaggregated prefill / decode 何时值得？

30 秒结论： 它把偏计算的 prefill 与偏 KV 读取的 decode 放到独立 worker pool，并通过 connector 转移 KV；只有资源独立优化收益超过 KV 传输、排队和故障复杂度时才值得。

3 分钟机制与源码： connector 接口与传输状态在 V1 KV connector 相关模块；production stack / 外部系统还负责路由与拓扑。必须把 transfer bytes、带宽、首包等待、backpressure 与版本一致性纳入状态机。

生产取舍： 可独立扩缩 P/D、隔离干扰；代价是网络、额外副本、跨池排队和更大故障域。

验证、失败与回滚： 比较聚合与拆分的端到端 TTFT / TPOT / goodput，注入 connector timeout / partial failure。无法安全 fallback 时不应直接上线。

追问： KV transfer 时间怎样估算？prefill / decode replica 比例如何动态调整？

F. 高级能力与质量

Q23. 量化怎么选？

30 秒结论： 先确认目标模型、checkpoint format、GPU architecture 与 vLLM backend 支持，再在质量门禁下比较权重、activation 和 KV dtype；不存在跨模型通用的“最佳 INT4 / FP8”。

3 分钟机制与源码： quantization registry 与实现位于 vllm/model_executor/layers/quantization/；load format、linear / MoE layer 和 kernel 共同决定是否支持。量化节省的理论字节还要扣除 scale、zero point、padding 与未量化层。

生产取舍： 更低位宽省显存 / 带宽，却可能降低质量、增加 dequant 或限制 backend；KV 量化还影响长上下文。

验证、失败与回滚： 用 golden set、perplexity / task metric 与服务 SLO 对照 BF16 / FP16 baseline；不兼容应启动失败，质量越线则回滚不可变 model artifact。

追问： weight-only 与 W8A8 的瓶颈不同在哪里？为什么“模型能加载”不足以验收？

Q24. Speculative decoding 为什么可能加速？

30 秒结论： draft 一次提出多个 token，target 在一次或较少 forward 中验证；接受率高且 draft 成本低时，每次 target step 产出更多被接受 token，同时 rejection sampling 保持 target 分布。

3 分钟机制与源码： V1 实现在 vllm/v1/spec_decode/ 与 sampler rejection path。收益由 proposal length、接受率、target verification、draft overhead、batch 与 sampling 参数决定；某些参数组合会被拒绝。

生产取舍： 低接受率会增加工作反而变慢；额外模型 / weights / state 增加显存与运维。

验证、失败与回滚： 记录 accepted tokens、draft / target time、端到端 output tokens/s 与质量等价性；接受率或 goodput 低于门禁即关闭 speculative config。

追问： 为什么接受率不是唯一指标？greedy 与 stochastic rejection 有何区别？

Q25. Structured output 能保证什么，不能保证什么？

30 秒结论： 它在采样时 mask 当前 grammar 状态不允许的 token，强化语法约束；不保证字段语义、业务授权、工具安全或所有 backend 对完整 JSON Schema 等价支持。

3 分钟机制与源码： vllm/v1/structured_output/ 编译 request grammar、生成 bitmask、推进 / rollback 状态。默认 backend 是 auto，当前有版本相关 fallback；显式 backend 验证失败不会静默换后端。

生产取舍： auto 易用但升级可能换 backend；显式配置可复现但要维护兼容矩阵。复杂 schema 增加 CPU compile 与 TTFT。

验证、失败与回滚： 做 backend × schema × tokenizer 正反例，保留原始 token 与 compile error；语义仍由服务端校验。升级异常可固定旧 backend / 旧 image。

追问： reasoning parser 何时应用 grammar？为什么结构合法仍不能直接执行工具？

Q26. Multi-LoRA serving 的 CPU cache 与 GPU slot 是什么？

30 秒结论： adapter 先注册在容量受限的 CPU cache，活跃 adapter 写入 max_loras GPU slots；Punica mapping 让 batch token 指向不同 slot。max_cpu_loras 与 max_loras 是不同容量。

3 分钟机制与源码： vllm/lora/model_manager.py、worker_manager.py 与 punica_wrapper/ 负责加载、激活、LRU 与增量 matmul。静态 lora-modules 与运行时加载控制面要分开；后者源码只定位为本地开发能力。

生产取舍： 更多 slot 减少换入，却挤压 KV；更多 CPU cache 降低存储读取，却占 host memory。路径 / resolver 是安全边界。

验证、失败与回滚： 扫描热度与 rank，记录注册 / 激活 / 淘汰、TTFT、显存和版本 hash；不兼容或 thrash 时路由到静态版本化 pool。

追问： 单 batch 超过 max_loras 与时间上的 LRU thrash 有何不同？为什么 prefix hash 要包含 LoRA identity？

G. 生产诊断与变更

Q27. 启动慢如何分解？

30 秒结论： 把下载 / 权重读取、反序列化、模型初始化、memory profile、compile、CUDA Graph capture、distributed rendezvous 与 readiness 分开计时；“启动慢”不是一个根因。

3 分钟机制与源码： loader、worker 初始化、compilation config 和 graph capture 分属不同模块。多 rank 还要看最慢 rank 与 shared filesystem。模型 cache 命中、compile cache 与 driver 都影响冷启动。

生产取舍： eager / 减少 capture 可缩短冷启动，却可能损失稳态性能；预烘焙 artifact 提高速度但增加供应链与兼容管理。

验证、失败与回滚： 保存阶段日志、每 rank 时间与 readiness；在同节点区分 cold / warm。启动优化不得跳过 correctness / health gate，失败回旧 image。

追问： 为什么 readiness 不能只看进程存在？compile cache 能否跨 GPU architecture 复用？

Q28. 如何做公平 benchmark？

30 秒结论： 先定义 workload 和 SLO，再固定模型 / tokenizer / quality / sampling，分别报告请求与 token 长度分布、并发、p50 / p99、goodput、错误率和资源；不只报峰值 tokens/s。

3 分钟机制与源码： vLLM CLI benchmark 工具和 07-hands-on/ 给出流程。服务 benchmark 要区分 input / output tokens、TTFT、TPOT / ITL、open / closed loop、warmup 与超时；比较不同引擎还要确保同一渲染和 stop 行为。

生产取舍： 峰值吞吐可能靠违反 p99 SLO 获得；synthetic 数据可复现但不能代表缓存与长度相关性。

验证、失败与回滚： 保存 exact command、dataset hash、environment 与 raw results；结果不可复现或 quality 不等价就作废，不进入容量承诺。

追问： open-loop 与 closed-loop 有何偏差？goodput 怎样把 SLO 纳入吞吐？

Q29. TPOT p99 突升怎么排查？

30 秒结论： 先确认指标与时间窗，再按 workload 变化、batch / step、KV pressure / preemption、backend / compile fallback、collective / hardware、CPU / 网络分层定位；不要先调参数。

3 分钟机制与源码： 对照当前版本 / metrics inventory，关联 waiting / running、KV usage、preemption、input / output 长度、request latency buckets、GPU / link 与 engine logs。TTFT 与 TPOT 要分开；平均 GPU util 不能证明 kernel 健康。

生产取舍： 降 token budget / 并发通常能缓解尾延迟却损失吞吐；扩副本可能因冷启动与路由失衡短期恶化。

验证、失败与回滚： 先做只读取证和单变量 canary；缓解后要用同流量重放验证。若证据不足，不执行集群级 mutation；使用 vllm-doctor 的 fail-closed 流程。

追问： preemption 为零还能是什么？为什么 retry 会放大排队？

Q30. 如何升级 vLLM 而不把“能启动”当成功？

30 秒结论： 固定 vLLM SHA、模型 / tokenizer revision、quantization、GPU / driver / CUDA / PyTorch、backend 与 API config，先做 source impact、golden correctness、性能与 failure gate，再 shadow / canary / drain；回滚单位是不可变兼容矩阵。

3 分钟机制与源码： 文档侧用 tools.source_sync 验证源码锚点；服务侧按 12-upgrades-rollbacks-and-compatibility.md 建 change record。缓存、LoRA、chat template、metrics 与 client behavior 都可能是兼容面。

生产取舍： 快速跟 main 获得功能，也扩大回归面；长期冻结降低变更风险，却积累安全与兼容债务。

验证、失败与回滚： 预先定义 rejection criteria、drain 与 rollback，不在事故中临时决定。任何 quality、API、p99、错误率或恢复演练越线立即停止 rollout。

追问： 为什么旧 Pod 的 cache 不应直接给新版本复用？哪些 change 可只重跑局部矩阵？

口述实验与失败证据

随机抽 5 题，每题录音 4 分钟：前 30 秒不准讲实现细节，后 3 分钟必须包含一个源码入口、一个 production tradeoff、一个验证信号和一个 rollback 条件。复盘时记录：

字段	合格证据
结论	直接回答题目，无绝对性能承诺
机制	状态 / 数据流正确，源码路径存在
取舍	同时说收益与代价，并给适用条件
验证	指标来自当前 inventory，或说明需先核准
失败 / 回滚	有明确否决条件，不用“再观察”代替

失败证据包括：说出不存在的 flag / metric / file、把估算称作实测、被追问假设时无法给单位、只讲 mitigation 不讲验证与回滚。把失败题映射回相应技术章节，而不是继续背答案。

硬件验证状态： 当前 SHA 未执行 GPU 口述题配套实验；涉及性能的问题只给测量方法，不给硬编码结果。

小结

- 30 秒回答负责结论，3 分钟回答负责机制与证据。
- 稳定契约、当前实现、环境默认值与待测假设必须分开。
- 每个优化答案都要带 quality / SLO gate，每个生产答案都要带 failure / rollback。
- 最强的回答不是数字最多，而是能说明数字来自哪里、在什么条件下失效。

自检

1. 随机抽 10 题，是否每题都能在 30 秒内直接回答？
2. 是否至少有 8 题能画出跨组件数据流，而非只报文件名？
3. 是否能把任意性能追问改写为“固定哪些变量、测哪些指标、什么条件否决”？
4. 是否能指出本章至少五个不能跨版本承诺的实现细节？

参考答案

1. 不能只凭记忆判断。30 秒回答应先给结论，再说一个机制和一个证据入口；若涉及默认值或具体行号，还要说明目标 vLLM SHA。完整达标标准是能把答案落到源码、metrics 或可复现实验，而不是背术语。
2. 至少应能画出 HTTP→API server→EngineCore→Scheduler→Worker→GPU→streaming 的主链路，以及 P/D、KV connector、NCCL 等跨组件路径。图上要标出数据类型、进程边界和同步点，否则只是文件名清单。
3. 把性能追问改成四步：固定模型/硬件/workload；定义 TTFT、TPOT、goodput 等指标；只改一个变量并保存证据；预先写明否决条件和 rollback。这样才能区分真实优化与流量、cache 或 warmup 波动。
4. 典型不可跨版本承诺包括默认 scheduler budget、backend 选择、metrics 名称、V0/V1 入口、compile/cudagraph 行为、KV connector 配置、quantization 支持和异常语义。回答时应给“当前锁定版本 + 验证方法”，而不是把历史经验说成稳定 API。

下一步

- 02-system-design.md：把单点知识组织成需求优先的完整架构
- 03-capacity-and-troubleshooting-drills.md：练公式、单位与证据树
- 04-mock-interview-and-rubric.md：按五轮 rubric 做完整模拟

Source trail

- vllm/v1/engine/core.py、vllm/v1/core/sched/scheduler.py
- vllm/v1/core/kv_cache_manager.py、block_pool.py、kv_cache_utils.py
- vllm/v1/sample/、vllm/v1/structured_output/、vllm/v1/spec_decode/
- vllm/v1/attention/backends/、csrc/attention/
- vllm/distributed/parallel_state.py、vllm/distributed/eplb/
- vllm/model_executor/layers/quantization/、vllm/lora/

LLM 推理系统设计：先问需求，再画架构

谁该读这一篇？ 需要设计或面试讲解在线 LLM inference platform 的工程师、tech lead 与容量评审者。

前置阅读： 01-common-questions.md、分布式并行、生产部署架构

耗时： 约 45 分钟

难度： 高级

当前性说明： 本章按 vLLM b23bd73f540175f9e117eae5029cd7d8df63964 静态复核；示例公式用于展示方法，不代表当前 SHA 的 GPU 实测容量。

学完能：

1. 在给架构前问清模型、workload、SLO、硬件、质量、安全与变更窗口
2. 用单位完整的权重 / KV / service demand / replica 公式给出第一版容量边界
3. 比较单节点、replica DP、TP / PP / EP 与 P/D disaggregation，而非只报一个方案
4. 把 routing、cache、failure domain、observability、deployment、cost 与 rejection criteria 放进同一设计

1. 面试开场：先拒绝无前提架构

题目如果只有“设计一个能承载 1 万用户的 LLM 服务”，不要立刻画 GPU 集群。“用户数”不是请求到达率，也不是同时运行数。先把未知量变成问题。

1.1 模型与输出契约

- 模型 / tokenizer / chat template revision 是否固定？dense 还是 MoE，GQA / MLA，生成还是 pooling / multimodal？
- dtype / quantization 是否可改？允许的质量损失和 golden set 是什么？
- context、输入 / 输出长度的 p50 / p95 / p99 是多少？是否有 reasoning、tool call、structured output、LoRA？
- streaming、logprobs、n、seed / determinism、stop 与 finish reason 的 API 契约是什么？

1.2 Workload 与到达过程

- 峰值 arrival rate 是 requests/s 还是 sessions/s？open-loop 还是 closed-loop？burst 持续多久？
- input / output token 分布与相关性是什么？短 chat、长 RAG、batch、embedding 是否混在一个池？
- system prompt、documents、images、LoRA 的复用比例是什么？prefix/cache locality 能否由路由保持？
- 取消率、超时、retry 与优先级分布是什么？

1.3 SLO 与 goodput

- TTFT、TPOT / ITL、端到端 latency 分别看 p50、p95 还是 p99？
- 什么算成功：HTTP 2xx、按时首 token、全部 token 在 deadline 内、还是 quality gate 也通过？
- admission 允许排队多久？超载时是 429、降 max tokens、切小模型还是延迟执行？

1.4 硬件、拓扑与平台

- GPU 型号、显存、单机卡数、NVLink / PCIe、跨节点网络、CPU / RAM / storage 是什么？
- driver、CUDA、PyTorch、vLLM SHA 与 model artifact 是否已有兼容矩阵？
- 启动多久、镜像 / 权重从哪里拉、region / AZ / failure domain 如何划分？

1.5 安全、成本与变更

- 谁认证和限额？tenant 数据、prompt / output / trace 如何隔离和脱敏？LoRA / media URL / chat template 谁能提供？
- 成本目标按 GPU-hour、每百万 output tokens，还是满足 SLO 的 goodput 计？
- 维护窗口多长？能否 shadow / canary？允许多少 warm spare？回滚 RTO / RPO 是什么？

面试信号： 强候选人会把缺失输入列成假设表，并说明哪一个假设最可能让架构翻转；弱候选人把所有问题默认为某张 GPU、某个 70B 模型和一个平均长度。

2. 容量估算：四张账，单位不能丢

2.1 权重与常驻内存

理论权重下界：

$$M_{\text{weights}} = N_{\text{params}} \times b_{\text{weight}}$$

其中 N_{params} 是实际加载参数数， b_{weight} 是每参数字节。量化还要加 scale / zero point / padding，某些 layer 仍保留高精度。每 rank 的近似权重：

$$M_{\text{weights,rank}} \approx \frac{M_{\text{sharded}}}{TP} \times PP + M_{\text{replicated,rank}}$$

这不是总显存：还要加 CUDA context、NCCL、activation、workspace、graph capture、LoRA slot、encoder cache 与安全余量。gpu_memory_utilization 是分配政策，不是这些项的替代公式。

2.2 KV cache

常见 decoder attention 的逻辑 KV：

$$B_{\text{KV/token}} = 2 \times L \times H_{\text{kv}} \times D_{\text{head}} \times b_{\text{kv}}$$

请求 KV：

$$M_{\text{KV,request}} = B_{\text{KV/token}} \times (T_{\text{prompt}} + T_{\text{generated}})$$

再按 TP / PP 的实际 cache spec 修正。MLA、hybrid attention、cross-attention、encoder-only runner 与 KV offload 不能套同一公式。最后用 vllm/v1/kv_cache_interface.py 的实际 page size / page count 和启动日志校验。

2.3 算力与显存带宽服务时间

prefill 与 decode 分开：

- prefill 处理大量 input tokens，attention 随 context 增长，通常更偏 compute / 大矩阵；
- decode 每 step 产生少量 token，却反复读取 weights 与历史 KV，常更偏 memory bandwidth / communication。

不要用理论 FLOPS 直接算 QPS。先从目标硬件 benchmark 得到按长度桶的 service demand：

$$D_{req} = T_{prefill}(T_{in}) + \sum_{i=1}^{T_{out}} T_{decode}(T_{in} + i, batch_i)$$

2.4 Replica 与故障余量

若单 replica 在目标 SLO 下的 sustainable arrival rate 是 μ_{good} ，峰值输入是 λ_{peak} ，目标利用率上限 $\rho_{target} < 1$ ，预留 $R_{failure}$ 个失效副本：

$$R_{steady} = \left\lceil \frac{\lambda_{peak}}{\mu_{good} \rho_{target}} \right\rceil$$

$$R_{total} = R_{steady} + R_{failure} + R_{rollout}$$

μ_{good} 必须来自**满足 TTFT / TPOT / quality / error SLO 的请求**，不是峰值 tokens/s。若单副本 fail 后剩余副本的 ρ 超过门禁，N+1 设计不成立。

2.5 一个带单位的示例

假设仅用于演算：模型实际加载 32×10^9 参数、BF16 权重 2 byte/param；目标 GPU 每张 80 GiB，但平台实测允许模型、non-KV 与 KV 的预算另算。

1. 权重下界： $32e9 \text{ param} \times 2 \text{ byte/param} = 64e9 \text{ byte} \approx 59.6 \text{ GiB}$ 。
2. 不能据此断言单卡可服务：还没有给 activation、workspace、graph 与 KV 留空间。
3. 若模型 config 为 L=64, H_kv=8, D_head=128, BF16 KV： $2 \times 64 \times 8 \times 128 \times 2 = 262,144 \text{ byte/token} = 256 \text{ KiB/token}$ 。
4. 一个 2,000 input + 500 output 请求的逻辑 KV 约： $2,500 \text{ token} \times 256 \text{ KiB/token} = 640,000 \text{ KiB} \approx 625 \text{ MiB}$ 。
5. 这仍不是“每卡并发 = 空闲 GiB / 625 MiB”：TP 分片、block 尾部、prefix sharing、运行时余量与调度上限都要实测。

sanity check：单请求 context 翻倍，逻辑 KV 应近似翻倍；若估算没有这个趋势，公式或单位错了。

3. 从最小架构开始，再给扩展路径

flowchart LR
C["Clients"] --> G["Gateway
auth • quota • request limits"]
G --> R["Router / Admission
model • prefix • LoRA • load"]
R --> P1["vLLM replica A"]
R --> P2["vLLM replica B"]
R --> PN["vLLM replica N"]
P1 --> M["Metrics / logs / traces"]
P2 --> M
PN --> M
CP["Control plane
artifact • rollout • inventory"] -.-> R
CP -.-> P1
CP -.-> P2
CP -.-> PN

```
classDef edge fill:#fef3c7,stroke:#b45309,color:#1a1f29;  
classDef data fill:#eff5ff,stroke:#2563eb,color:#1a1f29;  
classDef ctl fill:#dcfce7,stroke:#15803d,color:#1a1f29;  
class C,G,R edge;  
class P1,P2,PN data;  
class M,CP ctl;
```

3.1 单 GPU / 单节点

先问模型是否能在一个 failure domain 内放下并满足 SLO：

- 单 GPU：最少通信和组件，适合小模型 / pooling / 开发验证；容量和故障域最小。
- 单节点 TP：用高速互联跨卡放模型；collective 增加，但运维边界清晰。
- 单节点 TP×PP：当纯 TP 不合适或层放置需要 PP；要量 stage balance。
- 单节点多 replica：模型单卡 / 小 TP 可放下时，replica DP 常比扩大 TP 更利于吞吐和故障隔离。

拒绝条件： 权重能放下但最大请求 / graph capture 无余量；或 TP 扩大后 collective 让 p99 反而越线。

3.2 多节点 replica DP

每个 replica 是独立 engine / cache，用路由横向扩展。优点是 failure domain 与 rollout 粒度小；代价是 prefix / LoRA / encoder cache 分散。

路由输入应区分：

- hard constraints：model / tokenizer / LoRA version、modality、tenant、region、健康状态；
- soft scores：queue / estimated work、prefix locality、adapter active、cache warmth、failure-zone balance；
- backpressure：无 replica 能在 deadline 内接单时，排队 / 降级 / 429，而不是无界 retry。

拒绝条件： 路由只看连接数，不看 token work；或 sticky routing 无上限，热点 session 能压垮单 Pod。

3.3 跨节点 TP / PP / EP

只有单节点放不下或目标 latency / MoE 布局要求时才跨节点。设计必须给：rank-to-GPU map、process groups、链路、collective、failure handling 与 topology-aware placement。

- TP 跨慢网络：层内同步频繁，先用 trace 证明收益。
- PP：降低层内 group 范围，但有 stage bubble / imbalance。
- EP：expert dispatch / all-to-all，必须量 router skew 与 straggler。

拒绝条件： 只写 TP=16，不写两节点间带宽、collective time 和任一 rank 失败后的恢复路径。

3.4 Prefill / Decode disaggregation

P/D 拆分允许分别扩容和选择硬件，但增加 KV transfer 与跨池 queue：

$$T_{\text{transfer}} \geq \frac{M_{\text{KV, to transfer}}}{BW_{\text{effective}}} + T_{\text{setup}} + T_{\text{queue}}$$

设计要给 connector timeout、backpressure、version compatibility、partial transfer cleanup 和聚合模式 fallback。

拒绝条件： KV transfer p99 接近或超过拆分节省的 compute time；或 connector 故障会让请求悬挂且不能回到聚合池。

4. 四类 workload 如何让设计翻转

4.1 短对话 / 高并发 chat

优先变量：output tokens、TPOT、取消率与 system-prefix locality。

- 用 admission 控制 active sequences，不让 retry 代替容量。
- prefix-aware / session-aware routing 只能作为 soft score，并有负载上限。
- 小 TP + 多 replica 可能比大 TP 更高 aggregate goodput；延迟门禁决定。
- speculative decoding 只有在 target workload 接受率与端到端 goodput 通过时启用。

失败域：热点租户、长输出、客户端断开未 abort、rollout 冷 cache。

4.2 长上下文 RAG

优先变量：input p99、retrieval docs、KV bytes/token、prefill time 与 prefix 重用。

- 在 gateway 限制 tokens / documents，不只限制字符。
- chunked prefill 控制 step 抖动，但不会消除总 attention 成本。
- 先比较 GQA / MLA 模型、KV dtype、context truncation 与检索质量；不能只“加 KV cache”。
- 超长请求可独立 pool，避免把短 chat 的 TPOT p99 拖垮。

失败域：retriever 返回暴涨、template 变化导致 cache hit collapse、上下文截断破坏质量。

4.3 Agent / Structured output / LoRA

优先变量：多轮 turn、tool schema、grammar compile、adapter working set 与安全授权。

- structured output 只保证语法边界；工具执行仍需服务端授权 / schema semantic validation。
- auto backend 可能随版本改变，upgrade gate 要覆盖真实 schema。
- LoRA-aware route 可减少 active-slot miss，但必须限制任意 path / runtime update。
- tool description 的 cache locality 要与 tenant isolation / salt 一起设计。

失败域：schema complexity DoS、tool prompt 泄露、adapter thrash、同名版本漂移。

4.4 Multimodal / Pooling / Batch

这三类不要默认与生成流量混池：

- multimodal 还有 media IO、processor cache、encoder budget / cache 与 URL 安全；
- pooling 没有 autoregressive decode，sequence / token output 的网络成本不同；
- offline batch 更关心截止时间与成本，可接受不同 admission / priority。

失败域：损坏媒体、token-level 巨大响应、encoder OOM、batch job 饿死在线请求。

5. SLO、可观测与证据链

5.1 指标不是名字清单

先从当前服务 /metrics 和锁定 SHA 的 registration code 建 inventory，再定义：

- traffic：requests/s、input / output tokens/s、长度分布、tenant / model；
- queue：waiting / running、admission reject、deadline miss；
- latency：E2E、TTFT、TPOT / ITL，均用 histogram 分位数；
- engine：KV usage、preemption、prefix hit tokens、batch / iteration work；
- platform：GPU memory / bandwidth / power、CPU、network、NCCL / link error；
- quality：golden success、schema / tool validation、retrieval / ranking metric。

不要在设计里发明 metric 名；可以写“从当前 inventory 选择对应 counter / gauge”。

5.2 一个排障关联图

```
flowchart TD
    A["SLO breach"] --> B["TTFT or TPOT?"]
    B -->|TTFT| C["queue • tokenization • prefill • cold state"]
    B -->|TPOT| D["step work • KV/preemption • kernel/collective"]
    C --> E["correlate workload buckets"]
    D --> E
    E --> F["single-variable canary"]
    F --> G["SLO + quality recovered?"]
    G -->|yes| H["document evidence"]
    G -->|no| I["rollback / isolate"]
```

5.3 Alert 与 action 的边界

Alert 应指向 runbook 和 owner，不应直接执行有风险的集群 mutation。自动动作只做经过批准、幂等、有上限、可回滚的操作；证据不足 fail closed。生产诊断见 09-vllm-doctor-skill.md。

6. Reliability、部署与成本

6.1 Failure domain 表

故障	隔离	检测	缓解	恢复证据
单 worker / GPU	replica / node	health + request failures	stop route, drain	golden + SLO
NCCL / link	topology domain	rank logs + link telemetry	drain whole group	multi-rank soak
model artifact	immutable version	load / hash / quality gate	rollback artifact	signature + golden
cache / connector	cache namespace	hit / transfer / timeout	bypass or aggregate pool	no stale reuse
gateway / auth	redundant edge	4xx/5xx + auth audit	fail closed	policy tests
retry storm	admission boundary	attempts / queue growth	retry budget + shedding	queue drains

6.2 Rollout 状态机

artifact verified → shadow → small canary → expand → drain old → remove old。 每一步都有 duration、sample size、quality / SLO gate 和 rollback owner。 缩容先停止新流量，再等 active 请求 / connector state / stream 完成；不能只杀 Pod。

6.3 成本模型

至少报告：

$$Cost_{\text{per 1M good output tokens}} = \frac{\text{GPU hours} \times \text{price} + \text{platform cost}}{\text{good output tokens}} \times 10^6$$

把 rejected、deadline miss、重试重复计算、idle warm spare、rollout overlap 和 P/D 网络算进去。 低每 token 成本若靠更差质量或更高 p99 得到，不是可比方案。

7. 一份完整答题骨架

1. **复述目标**：用户体验、质量、availability、security、cost。
2. **列假设表**：模型、长度、arrival、SLO、硬件、change window；标出最敏感三项。
3. **算四张账**：weights、KV、service demand、replicas + N+1 / rollout reserve。
4. **给最小方案**：单节点 / 小 TP + replicas；说明为什么此时够用。
5. **给翻转条件**：何时需要 PP / EP / disaggregation / workload split。
6. **画 data plane + control plane**：auth、admission、routing、engine、telemetry、artifact / rollout。
7. **讲三个 tradeoff**：latency vs throughput、cache locality vs balance、efficiency vs failure domain。
8. **列 failure table**：detect、mitigate、recover、evidence。
9. **定义 reject / rollback**：哪些指标越线立即停止。
10. **说明未知**：需要 benchmark / profile 的量，不编数字。

8. 设计练习与失败证据

为下面三个场景各写两页，不允许复用同一架构：

1. 短 chat：output-heavy，TPOT p99 严格，有共享 system prompt。
2. 长 RAG：input-heavy，长度长尾明显，retrieval quality 不可降。
3. 多租户 LoRA + tools：adapter working set 大，需要强隔离与审计。

每份必须附：assumption table、单位完整的容量公式、一个 alternative、三个 rejection criteria、一次故障演练和 rollback。失败证据包括：用“并发用户”代替 arrival / active sequences；把 theoretical memory 当可分配显存；没有 N+1；metric 不存在；只画 data plane 不画 artifact / rollout；没有 quality / security gate。

生产取舍： 架构复杂度必须由可量化瓶颈购买。未证明单节点 / replica 方案失败前，不要用跨节点 TP 或 P/D disaggregation 装饰答案。

硬件验证状态： 当前 SHA 未做 GPU system-design 容量实测；所有 μ_{good} 、service demand 与 connector bandwidth 必须由目标环境补齐。

小结

- Requirements 不是开场礼貌，而是决定模型、并行、cache、routing 与 SLO 是否成立的输入。
- 容量必须同时算权重、KV、服务时间和 failure / rollout reserve。
- 从最小架构开始，用明确翻转条件引出复杂方案。
- routing、observability、security、deployment、cost 与 rollback 都是 inference system 的一部分。

自检

1. 哪三个未知量最可能让“单节点 TP”翻转为“多 replica / disaggregation”？
2. 为什么 total users / per-GPU concurrency 不是可靠 replica 公式？
3. prefix-aware routing 如何同时避免热点和跨 tenant 错误复用？
4. 若 canary 的 tokens/s 提升但 TPOT p99 和 quality 变差，应如何裁决？

参考答案

1. 最容易翻转架构的通常是：输入/输出长度和到达率分布、TTFT/TPOT p99 SLO、以及模型权重/KV 是否能放进单节点并满足 failure domain。高并发短请求可能偏 replica DP；超长上下文或 prefill/decode 资源明显不对称时才考虑 disaggregation。
2. 用户数不等于同时运行请求，也不包含 token 长度、到达过程、SLO、失败余量和 goodput。replica 数应从 peak arrival rate / goodput_per_replica_at_SLO 推导，再加故障和 rollout headroom，而不是用一个“每卡并发”除法。
3. Router 维护 prefix/cache locality 与负载分数的联合决策：优先把相同安全域和兼容指纹的 prefix 送到有命中的副本，同时设置负载上限、租户 namespace 和 fallback，防止一个热门 prefix 把单点打热或跨租户误复用。
4. 先暂停扩大 canary，检查质量 golden、错误率、TTFT/TPOT p99、goodput 和 workload 可比性。只要 p99 或质量触发预先批准的 gate，就回滚；tokens/s 的提升不能抵消用户可见质量和 SLO 退化。

下一步

- 03-capacity-and-troubleshooting-drills.md：把公式和故障树做成有解练习

- 04-mock-interview-and-rubric.md：按五轮评分模拟完整面试
- 08-production-capstone.md：把设计落到可复现证据包

Source trail

- vllm/config/{model,cache,scheduler,parallel}.py
- vllm/v1/core/sched/scheduler.py、vllm/v1/core/kv_cache_manager.py
- vllm/v1/kv_cache_interface.py、vllm/v1/attention/backends/
- vllm/distributed/parallel_state.py、vllm/distributed/eplb/
- vllm/v1/engine/、vllm/entrypoints/openai/

容量计算与故障诊断练习：公式、单位、证据

谁该读这一篇？ 会讲概念但容易在白板计算、单位换算或线上排障追问中失分的候选人和在岗工程师。

前置阅读： 01-common-questions.md、02-system-design.md、04-autoscaling-and-capacity.md

耗时： 约 60 分钟

难度： 高级

当前性说明： 本章按 vLLM b23bd73f540175f9e117eae5029cd7d8df63964 静态复核；所有数值题都是明确假设下的教学演算，不是当前 SHA 的硬件 benchmark。

学完能：

1. 计算 weight、KV、request、batch、world size、goodput 与 replica reserve
2. 每一步保留单位、假设、中间值和 sanity check
3. 对八类常见故障先列证据，再做单变量缓解与回滚

0. 白板计算协议

先写四行，再按计算器：

1. **Asked：** 题目要的是理论下界、每 rank、每 replica，还是集群总量？
2. **Assumptions：** 模型 config、dtype、并行、长度分布、SLO、reserve。
3. **Formula + units：** 每个乘数都写单位，确认能约掉。
4. **Sanity check：** 输入翻倍时结果应怎样变化？与物理上限是否同量级？

常用换算：1 KiB=1024 byte，1 MiB=1024 KiB，1 GiB=1024 MiB；厂商常用十进制 GB，GPU / OS 工具显示口径可能不同，答题时先声明。

1. 权重内存：理论下界不等于可部署

题目

一个实际加载 13×10^9 params 的 dense 模型以 BF16 (2 byte/param) 加载；假设 checkpoint / padding / scale 之外的**权重相关**额外开销为理论权重的 3%。TP=2，所有权重均匀分片。求每 rank 权重相关内存。它能否证明两张 16 GiB GPU 可部署？

解答

假设： 这里只算 weights；不包含 CUDA context、activation、workspace、KV、graph 与 allocator reserve。

理论权重：

$13 \times 10^9 \text{ param} \times 2 \text{ byte/param} = 26 \times 10^9 \text{ byte}$

换成 GiB：

$26 \times 10^9 \text{ byte} \div 2^{30} \approx 24.21 \text{ GiB}$

加 3%：

$24.21 \text{ GiB} \times 1.03 \approx 24.94 \text{ GiB}$

TP=2 每 rank：

$24.94 \text{ GiB} \div 2 \text{ rank} \approx 12.47 \text{ GiB/rank}$

答案：每 rank 约 12.47 GiB 权重相关内存。不能据此证明 16 GiB 可部署，因为只剩约 3.53 GiB 给所有 non-weight 与 KV，且“均匀分片”本身也可能不成立。

sanity check：TP 从 2 改 1，均匀分片假设下每 rank 应翻倍到约 24.94 GiB。

面试追问：哪些参数可能每 rank 复制？INT4 为什么不能简单按 0.5 byte/param 当物理占用？

2. KV bytes/token：用 KV heads，不用 query heads

题目

一个 decoder 有 L=32 layer、H_kv=8 KV heads、D_head=128 element/head，KV dtype 为 BF16 (2 byte/element)。求逻辑 KV bytes/token。

解答

公式：

$B_{\text{KV/token}} = 2(K,V) \times L \times H_{\text{kv}} \times D_{\text{head}} \times b_{\text{dtype}}$

代入：

$2 \times 32 \text{ layer} \times 8 \text{ head/layer} \times 128 \text{ element/head} \times 2 \text{ byte/element}$

$= 131,072 \text{ byte/token} = 128 \text{ KiB/token}$

答案：128 KiB/token。

sanity check：KV dtype 改为 1 byte 时理论值减半；KV heads 从 8 到 4 时也减半。若使用 query heads 32，结果会错误放大 4 倍。

边界：TP / PP 后每 rank 容量、MLA、hybrid cache 与 padding 要看实际 KVCacheSpec，不能只除 world size。

3. 单请求 KV：prompt 与已生成 token 都占容量

题目

沿用 128 KiB/token。请求有 1,500 input token，最多生成 500 output token。求最大逻辑 KV；若平均只生成 120 token，平均逻辑 KV 是多少？

解答

最大 tokens：

$$1500 + 500 = 2000 \text{ token}$$

最大 KV：

$$2000 \text{ token} \times 128 \text{ KiB/token} = 256000 \text{ KiB} = 250 \text{ MiB}$$

平均 tokens：

$$1500 + 120 = 1620 \text{ token}$$

平均 KV：

$$1620 \text{ token} \times 128 \text{ KiB} = 207360 \text{ KiB} = 202.5 \text{ MiB}$$

答案： 上限 250 MiB/request，给定输出均值下约 202.5 MiB/request。

sanity check： 输出上限增加 500 token，应增加 $500 \times 128 \text{ KiB} = 62.5 \text{ MiB}$ ，而不是重算权重。

面试追问： admission 应用 max 还是分位数？prefix sharing 会改变物理占用但为什么不能作为安全上限？

4. Batch / 并发容量：先做上界，再交给 profile

题目

一个 replica 启动后报告可供 KV 使用的实际容量为 30 GiB。保留 10% KV 容量给长度长尾 / block 尾部 / 运行波动；按上一题最大 250 MiB/request，求只受 KV 限制的保守并发上界。max_num_seqs 应直接设为这个值吗？

解答

可用 KV：

$$30 \text{ GiB} \times 1024 \text{ MiB/GiB} \times (1 - 0.10) = 27648 \text{ MiB}$$

请求数：

$$\left\lfloor \frac{27648 \text{ MiB}}{250 \text{ MiB/request}} \right\rfloor = 110 \text{ request}$$

答案： KV-only 保守上界约 110 active requests。

不能直接把 max_num_seqs=110 当生产结论，因为：

- 实际长度是分布，page allocation / block size 会离散化；
- scheduler token budget、activation / batch workspace 与 model runner 也有限制；
- SLO 可能在 KV 满之前就因 batch 太大失败；
- multimodal encoder cache、LoRA slots 或 speculative state 还占资源。

sanity check： 若 bytes/token 或每请求 tokens 翻倍，上界应近似减半。

验证： 从较低并发阶梯压测，只有在 p99、错误率、preemption 和显存余量同时通过时提高。

5. TP / PP / DP / EP world size：先画坐标

题目

两节点，每节点 8 GPU，共 16 ranks。单模型副本配置 TP=4、PP=2。可以放多少个 DP replicas？若 MoE 配置 EP=4，总 rank 是否变成 $4 \times 2 \times 2 \times 4 = 64$ ？

解答

单副本 ranks：

$$\$R_{\text{model}} = TP \times PP = 4 \times 2 = 8 \text{ rank/replica} \$$$

可放 DP：

$$\$DP = 16 \text{ rank} \div 8 \text{ rank/replica} = 2 \text{ replica} \$$$

常见坐标可写 (dp, pp, tp)，共 $2 \times 2 \times 4 = 16$ 。

EP=4 通常不是额外乘数。 它在既有模型 ranks 上创建 expert-parallel group / 映射；要读启动模式与 parallel_state.py，确认 EP group 是否复用 TP / DP 维度。盲乘得到 64 与物理 16 ranks 矛盾。

sanity check： 每个 rank 必须恰好映射到一张 GPU；把所有 replica 坐标列出应正好 16 行且无重复。

面试追问： 如果要求每个 TP group 不跨节点，如何放置？PP stage 不均衡怎样从 trace 看出来？

6. Throughput 与 goodput：不要乘独立边际概率

题目

压测发送 120 request/s。观测到 100 request/s 返回成功，其中 96 request/s 同时满足 TTFT 与 TPOT deadline；golden sampling 显示这 96 中有 94 request/s 通过质量校验。求 transport throughput、SLO goodput、quality-adjusted goodput。

解答

- transport throughput：100 successful request/s。
- SLO goodput：96 request/s。

- quality-adjusted goodput: 94 request/s。

相对输入的比例：

$$\frac{100}{120}=83.3\%$$

$$\frac{96}{120}=80.0\%$$

$$\frac{94}{120}=78.3\%$$

关键： 这里使用同一请求的 joint outcome。不能把“TTFT 90% 达标 × TPOT 95% 达标”相乘，除非证明独立；实际二者常因 queue / load 相关。

sanity check： $\text{quality goodput} \leq \text{SLO goodput} \leq \text{successful throughput} \leq \text{offered load}$ 。

面试追问： output length 不同如何用 good output tokens/s 补充？超时后仍完成的请求算什么成本？

7. Replica count、目标利用率与 N+1

题目

峰值 $\lambda_{\text{peak}}=350$ request/s。单 replica 在目标长度分布下，**满足全部 SLO** 的 sustainable goodput $\mu_{\text{good}}=55$ request/s。为吸收 burst，目标利用率 $\rho_{\text{target}}=0.75$ ；要求任一 replica 故障仍承载峰值，并在 rollout 时额外保留一个 canary replica。求总副本。

解答

steady replicas：

$$R_{\text{steady}}=\left\lceil\frac{350\text{ request/s}}{55\text{ request/s/replica}\times0.75}\right\rceil$$

$$=\lceil8.48\rceil=9\text{ replicas}$$

加一个 failure reserve 和一个 rollout reserve：

$$R_{\text{total}}=9+1+1=11\text{ replicas}$$

答案： 初始规划 11 replicas。

N+1 sanity check： 故障一台且 canary 不接生产流量时，9 个 steady replicas 仍在；峰值每台负载 $350/9=38.9$ request/s，低于 $55\times0.75=41.25$ request/s 的目标。

边界： 若 canary 也占用一个原本的 steady replica，或 AZ 故障一次损失多台，reserve 公式必须重算。

8. KV transfer 下界：判断 disaggregation 是否可能成立

题目

P/D 拆分需要为一个请求传 4 GiB KV。测得有效单向带宽 80 Gbit/s，先忽略 queue / setup / serialization。求传输时间下界。若聚合部署相对拆分只多 250 ms compute interference，这个方案是否有胜算？

解答

先统一单位：

$$4 \text{ GiB} \times 8 \text{ bit/byte} = 32 \text{ Gibit}$$

若按题目近似把 80 Gbit/s 当 $80 \times 10^9 \text{ bit/s}$ ，严格换算后约：

$$\frac{4 \times 2^{30} \text{ byte} \times 8 \text{ bit/byte}}{80 \times 10^9 \text{ bit/s}} \approx 0.429 \text{ s}$$

即理论下界约 429 ms，还未加 queue / setup。它已经大于只节省的 250 ms，因此按当前假设**没有端到端 latency 胜算**。

sanity check： 带宽翻倍，时间下界减半；传输数据翻倍，时间翻倍。

面试题追问： 为什么实际有效带宽低于链路标称？如果 P/D 的价值是独立扩缩而非单请求 latency，应怎样重新评估？

9. 证据优先的故障诊断八题

统一答题顺序：确认症状 → 校验观测 → 分层假设 → 只读取证 → 最小缓解 → 复测 → 回滚 / 根修。下列 signal 名是类别；实际 Prometheus 名必须从当前 SHA / metrics inventory 选择。

场景 A：TTFT p99 上升，TPOT 正常

首要证据： offered load 与 input length buckets、waiting queue / queue time、tokenizer / renderer CPU、prefill tokens / step、cold model / prefix hit、gateway time。

假设排序： 长 prompt 或排队 > frontend CPU > cache locality collapse > cold / compile > 网络。

最小动作： 对超长输入做 admission / 独立 pool；扩 frontend 只在 CPU 证据成立时做。

失败 / 回滚： 降 token budget 若让 queue 更长且 TTFT 更差，立即回滚；不能因 GPU util 低直接扩 GPU。

场景 B：TPOT p99 上升，TTFT 正常

首要证据： running / batch work、output length、KV usage / preemption、decode backend / graph fallback、collective、GPU memory bandwidth 与 power / thermal。

假设排序： decode batch / context 变大 > preemption / thrash > kernel fallback > NCCL / hardware。

最小动作： canary 降 active sequences 或 step token work，验证 TPOT 与 goodput 共同变化。

失败 / 回滚： TPOT 恢复但 goodput 跌破容量门禁，不是完成修复；应扩容或分池。

场景 C：启动或运行 OOM

先要证据： 每 rank 启动 profile、weights、non-KV peak、KV page count、graph capture、LoRA / encoder cache、OOM stack 与同节点其他进程。

区分： 启动 OOM、capture OOM、首个大请求 OOM、长期碎片 / 泄漏不是一类。

最小动作： 恢复已验证 memory / graph / concurrency config；先 admission，后优化。

失败 / 回滚： 禁止无界降低 gpu_memory_utilization 后反复重启；要保留原 profile 与可复现输入。

场景 D：Preemption 持续增长

先要证据： KV usage、running / waiting、context / output length、max sequences、prefix hit tokens、abort / retry。

假设排序： active working set 超 KV > 长度分布变化 > cache locality 降 > admission / retry 放大。

最小动作： 限 active sequences / output tokens，隔离长 context；观察重复 prefill 与 TTFT。

失败 / 回滚： 只扩 max_num_seqs 往往加剧 thrash；发现 preemption 增长即撤销。

场景 E：NCCL hang 或多 rank timeout

先要证据： 第一个异常 rank、所有 rank stack / logs、rank map、GPU / NIC health、topology、collective type / size、版本和最近变更。

假设排序： 单 rank 提前异常 > group / rank 配错 > link / NIC > collective mismatch > driver / library。

最小动作： drain 整个 model group / node failure domain，不把单 rank 重启后塞回旧 group。

失败 / 回滚： 禁止只延长 timeout 掩盖 collective 不一致；恢复必须做多 rank soak。

场景 F：Prefix-cache hit rate 突降

先要证据： hit **tokens** 与 query tokens、chat template / tokenizer / model revision、cache salt、LoRA / multimodal mix、router、prompt hash sample。

假设排序： workload / template 变化 > routing locality > capacity eviction > identity / salt 改变 > metric 口径。

最小动作： 对比升级前后 tokenized prefix 与路由，不先调大 cache。

失败 / 回滚： 若输出一致性异常，优先禁用 cache / 回滚版本，而不是追求 hit rate。

场景 G：GPU 低利用、frontend CPU 满

先要证据： tokenizer / renderer / JSON / media profile、API event-loop lag、engine queue、request size、API replicas 与 DP mapping。

假设排序： tokenization / chat rendering > media IO / processor > serialization / logprobs response > Python contention。

最小动作： 按 profile 扩 / 隔离 frontend，限制巨大请求；确保 API replica 与 engine topology 支持该扩展。

失败 / 回滚： 不能在未知瓶颈时开启更多 API processes；processor cache / runtime LoRA 与多 API server 有约束。

场景 H: Retry storm 与 queue 不收敛

先要证据： original vs retry attempts、429 / 5xx / timeout、retry delay / budget、waiting age、client / gateway layers、cancellation传播。

假设排序： 无界同步 retry > timeout/小于实际 service time > downstream partial outage > admission缺失。

最小动作： fail closed admission、retry budget、jitter、circuit breaker；停止重复工作比盲目扩容更快。

失败 / 回滚： queue下降但成功 goodput也归零说明 shedding过度；按优先级恢复并监测系统是否收敛。

10. 自己出题与评分

给自己换一组模型 config / dtype / 长度，重做题 1–8。每题 5 分：

维度	1 分标准
Assumption	所有未给输入显式写出
Formula	公式与对象匹配
Units	每步单位可约分
Intermediate	中间值可复核
Sanity	有趋势 / 物理上限检查

故障题每题再按“证据、假设排序、最小动作、复测、回滚”各 1 分。任何一步引用不存在的 metric 或把估算称作硬件实测，该题最高 2 分。

生产取舍： 容量表给的是决策边界，不是调度器的行为模拟。最终值必须由目标硬件 benchmark、failure test 与质量 gate闭环。

硬件验证状态： 未执行当前 SHA GPU run；本章无硬件实测结果。

小结

- 权重决定能否放置，KV 决定 context / active capacity，service demand 决定 SLO 下 goodput。
- TP×PP×DP 先映射物理 ranks；EP 是否增加维度要看 group 定义。
- replica 公式必须用 SLO goodput，并加入 failure / rollout reserve。
- 排障先建立 joint evidence，单变量缓解后同时复测 SLO、quality 与 goodput。

自检

- 为什么 $13B \times 2 \text{ bytes}$ 不是 13B 模型的部署显存？

2. 为什么不能把 TTFT 达标率与 TPOT 达标率直接相乘？
3. 一个 AZ 同时损失三副本时，题 7 的 N+1 要怎样改？
4. Retry storm 中“扩容”和“先停止重复工作”分别何时有效？

参考答案

1. $13B \times 2$ bytes 只是权重下界；部署还需要 KV cache、activation/workspace、CUDA/NCCL、CUDA Graph、allocator 碎片、LoRA/encoder cache 和安全余量，且 TP/PP 还会改变每 rank 分片与复制项。
2. TTFT 和 TPOT 达标率往往相关但不独立，简单相乘隐含独立性假设，可能重复计算同一请求或掩盖尾延迟分布。应直接按请求判断“TTFT、TPOT、错误、质量全部通过”，计算 goodput@SLO。
3. 若一个 AZ 可同时损失三副本，N+1 不够；至少按 failure-domain loss 重新计算，保留能在剩余 AZ 上承载峰值 goodput 的副本和 warm spare，并把跨 AZ 网络、容量和恢复时间纳入模型。
4. 扩容适合真实到达率持续超过健康容量、且重复请求已被抑制的情况；retry storm 中首先停止无界重试、启用幂等/指数退避和 admission control，否则加的实例只会接收更多重复工作。止血后再按剩余有效流量扩容。

下一步

- 04-mock-interview-and-rubric.md：把计算题放进完整五轮面试
- 08-production-capstone.md：生成真实 benchmark / incident evidence

Source trail

- vllm/config/{cache,scheduler,parallel}.py
- vllm/v1/kv_cache_interface.py、vllm/v1/core/kv_cache_manager.py
- vllm/v1/core/sched/scheduler.py
- vllm/distributed/parallel_state.py
- vllm/entrypoints/、vllm/v1/engine/

vLLM 五轮模拟面试与评分 Rubric

谁该读这一篇？ 准备 inference engine / serving / performance / platform 岗位的候选人，以及需要结构化面试题与评分锚点的面试官。

前置阅读： 01-common-questions.md、02-system-design.md、03-capacity-and-troubleshooting-drills.md

耗时： 90 分钟模拟 + 30 分钟复盘

难度： 高级

当前性说明： 本章按 vLLM b23bd73f540175f9e117eae5029cd7d8df63964 静态复核；评分奖励证据边界，不奖励背诵未经当前环境验证的性能数字。

学完能：

1. 完成概念、源码追踪、计算、系统设计、事故五轮模拟
2. 用统一 1-5 rubric 评价准确性、证据、取舍、验证 / 回滚与沟通
3. 把真实项目经历组织成可核验叙事，不虚构指标

0. 模拟规则

- 总时长 90 分钟：5 + 15 + 15 + 30 + 20，另留 5 分钟候选人提问。
- 候选人可以说“不知道”，但要说明下一步如何从源码 / 运行证据求证。
- 面试官不给隐藏模型 config；候选人必须主动问缺失输入。
- 所有数字必须标注“题设、推导、观测或估计”。把估计称作实测是严重扣分。
- 每轮都要求一个 failure boundary 与 rollback；只讲 happy path 不超过 3 分。

建议两人练习：面试官只按 prompt 和 follow-up 提问，不在过程中纠错；最后逐项给出证据。

1. 统一 1-5 Rubric

1.1 准确性 Accuracy

分数	锚点
1	核心概念错误，答案与题目无关，或编造不存在能力
2	能说名词但机制混乱；把 V0 / 第三方 / 当前 V1 混写
3	主干正确，有少量未限定实现细节；提醒后可修正
4	机制、状态与边界正确，主动区分契约 / 当前实现 / 假设
5	在 4 的基础上能处理反例、数值边界和跨组件影响

1.2 证据 Evidence

分数	锚点
1	只有口号或硬编码数字，无来源
2	报文件 / metric 名但无法说明其证明什么，或名称过时
3	能给一个有效源码入口或运行信号
4	给出相互印证的源码、配置、日志 / metric / experiment chain
5	说明证据缺口、口径、采样偏差与如何复现 / 归档

1.3 取舍 Tradeoffs

分数	锚点
1	所有 feature 都开，只有收益没有成本
2	泛泛说 “看情况”，没有翻转条件
3	能讲一个收益 / 代价对和适用 workload
4	用 SLO、quality、capacity、failure domain 比较 alternatives
5	给出敏感变量、拒绝条件与成本 / 运维后果

1.4 验证与回滚 Validation / Rollback

分数	锚点
1	直接全量变更，无 gate / rollback
2	只说“压测 / 监控”，没有指标与失败条件
3	有 canary、核心指标和人工回滚
4	有 quality + SLO + failure gate、drain、不可变回滚单位
5	还覆盖失败注入、证据归档、恢复验证与 fail-closed 边界

1.5 沟通 Communication

分数	锚点
1	术语堆砌，无法直接回答，单位混乱
2	有结论但结构散乱，关键假设藏在后面
3	结论先行、结构基本清晰，能回答追问
4	主动确认需求、画最小图 / 公式、清楚标未知
5	根据听众调整深度，压缩 / 展开自如，能总结决策与下一步

单轮 25 分。总分只用于复盘，不用于机械录用：任一轮 Accuracy=1 或 Validation=1 都应单独讨论风险。

Round 1：概念快问（5 分钟）

面试官 Prompt

用两分钟解释：Paged KV cache、continuous batching 与 prefix caching 分别解决什么问题？三者之间是什么关系？

期望证据

- Paged KV：逻辑 token → 非连续物理 block，按需分配；仍有尾块 / metadata 成本。
- Continuous batching：step 边界重调度，处理长度异质性；受 token budget 与 KV capacity 共同约束。
- Prefix caching：按完整 block 链式 hash 复用已计算 KV；extra keys 隔离 LoRA / multimodal / salt。
- 能指出 vllm/v1/core/{block_pool,kv_cache_manager,kv_cache_utils}.py 和 scheduler 作为核准入口。

Follow-ups

1. block size 越小是否越好？
2. prefix hit rate 高，为什么 TTFT 可能不降？

3. continuous batching 是否意味着 prefill / decode 一定同 step 混跑？

Strong signals

- 先分别定义，再画 “Scheduler 使用 KV manager；prefix cache 是 block 复用状态” 的关系。
- 不引用固定碎片百分比、默认 block size或性能倍数。
- 主动提到 tokenizer / template / LoRA identity 的正确性边界。

Weak signals

- 把 PagedAttention 说成 CPU swap；把 prefix cache 说成 response cache。
- 声称 GPU utilization 必然接近 100%。
- 只讲论文，不知道当前 V1 组件入口。

本轮评分重点

Accuracy 与 Communication 权重最高。满分回答在 2 分钟内完成主干，追问时才展开实现。

Round 2：源码追踪（15 分钟）

面试官 Prompt

一个带 top_p、seed、logprobs=5 的 streaming chat 请求进入 V1。请从 frontend 追到 sampler，再说 token 如何返回。指出哪些细节必须读当前源码，不能凭经验假设。

候选人应画出的最小链路

```
flowchart LR
  A["OpenAI frontend<br/>validate • render • tokenize"] --> B["EngineCore Request"]
  B --> C["Scheduler<br/>tokens • blocks"]
  C --> D["GPU ModelRunner<br/>persistent batch"]
  D --> E["Sampler<br/>processors • penalties • top-k/p"]
  E --> F["Engine update<br/>finish reason"]
  F --> G["stream response"]
```

期望证据

- frontend：vllm/entrypoints/openai/，配置 / renderer / tokenizer 参与请求语义。
- request / core：vllm/v1/request.py、vllm/v1/engine/core.py。
- scheduler：vllm/v1/core/sched/scheduler.py 产出本步 token / block metadata。
- runner / sampler：vllm/v1/worker/gpu_model_runner.py、vllm/v1/sample/sampler.py。
- sampling 顺序：raw logprob mode、FP32、allowed / bad words / processors / penalties、temperature、min-p、top-k/p、gather。
- seed 只控制 generator，不承诺跨 backend / batch / hardware bitwise deterministic。

- top-N logprobs 仍可能做 full-vocab log-softmax；返回体是另一个成本。

Follow-ups

1. 为什么 per-request generator 可能影响 top-k/top-p backend？
2. logprobs=-1 与 5 的成本差异是什么？
3. 客户端断开后，哪条证据证明 KV 最终释放？
4. 如果加 structured output，bitmask 在哪里接入？

Strong signals

- 用“读取当前 startup log / config / source”回答 backend，而不是背默认。
- 能区分 raw / processed logprobs 和 API serialization。
- 把 cancellation / cleanup 纳入完整 lifecycle。

Weak signals

- 声称请求可以上传任意 Python logits processor。
- 把 logprobs top-N 误写成不做 full-vocab softmax。
- 只列文件，不说明状态如何流动。

本轮评分重点

Evidence 必须 ≥ 3 才算通过。Accuracy=5 要能指出至少一个当前实现易漂移点，并说明如何核准。

Round 3：容量计算（15 分钟）

面试官 Prompt

题设：模型 40×10^9 params，BF16 weights；L=60、H_kv=8、D_head=128、BF16 KV。每请求 p95 是 3,000 input + 1,000 output tokens。四张 80 GiB GPU 做 TP=4。假设每 rank 除权重外固定 non-KV peak 为 12 GiB，并要求留 8 GiB 安全余量。先算理论每 rank 可给 KV 的容量与 p95 request KV，再说明为什么还不能给生产并发承诺。

参考推导

权重：

$$40 \times 10^9 \text{ param} \times 2 \text{ byte/param} = 80 \times 10^9 \text{ byte} \approx 74.51 \text{ GiB}$$

均匀 TP=4：

$$74.51 / 4 \approx 18.63 \text{ GiB/rank}$$

题设下每 rank KV 理论预算：

$\$80 - 18.63 - 12 - 8 = 41.37 \text{ GiB/rank}$

KV bytes/token (全模型逻辑):

$\$2 \times 60 \times 8 \times 128 \times 2 = 245,760 \text{ byte/token} = 240 \text{ KiB/token}$

p95 请求逻辑 KV:

$\$4,000 \text{ token} \times 240 \text{ KiB/token} = 960,000 \text{ KiB} = 937.5 \text{ MiB}$

若 KV heads 在 TP=4 均匀分片, 每 rank 逻辑部分近似:

$\$937.5 / 4 = 234.375 \text{ MiB/request/rank}$

粗糙 KV-only 上界:

$\$41.37 \text{ GiB} \times 1024 \text{ MiB/GiB} \div 234.375 \text{ MiB/request} \approx 180 \text{ requests}$

必须主动否定的承诺

180 不是生产并发: 题设忽略 replicated weights、quant / padding、page 离散、实际 cache spec、max batch activation、prefix / output length tail、scheduler token budget 与 SLO。正确下一步是从启动 profile / page count 校验, 再阶梯压测得 μ_{good} 。

Follow-ups

1. 若 $H_{\text{kv}}=2 < \text{TP}=4$, 分片假设会怎样?
2. 若 KV 改 FP8, 理论和质量 gate 各怎样变化?
3. 为什么 p95 request size 不能保护 p99 / max OOM?
4. 如何加入 N+1 和 rollout reserve?

Strong signals

- 单位从 byte 到 GiB 全程可约分。
- 把理论上界、启动可分配、SLO 并发明确分层。
- 最后给 sanity check: context / dtype 变化趋势正确。

Weak signals

- 直接说 40B BF16 是 80 GiB, 忽略十进制 / 二进制。
- 用 query heads 计算 KV; 把全模型 KV 再错误乘 layer。
- 把 180 写进 HPA capacity。

本轮评分重点

Accuracy 包括数学与对象选择; Communication 看是否先列假设再算。算术小误差但方法正确可得 4, 单位 / 分片概念错不超过 2。

Round 4：系统设计（30 分钟）

面试官 Prompt

设计一个多租户企业 Agent 平台：两个模型版本，每个版本有 20 个 LoRA；请求包含 2k–20k token、tool schema 与 structured output；峰值 120 request/s；TTFT p99 < 1.5 s、TPOT p99 < 80 ms；任何一个节点故障仍满足 SLO。GPU 和预算未给。请先问问题，再给最小方案、alternative 与 rollout / rollback。

期望澄清

- 模型架构 / 参数 / tokenizer / dtype / quality gate；LoRA rank / size / working set。
- input / output 联合分布、turn 数、tool schema 复用、arrival burst、cancel / retry。
- GPU / topology / regions、artifact storage、cold-start、change window。
- tenant auth / quota、LoRA / tool schema 谁可更新、data retention / audit。
- structured backend、reasoning parser、speculative / prefix policy 是否可变。

期望设计内容

1. **最小方案：** 每模型版本独立 deployment；单节点能放下则小 TP + replica DP；gateway 负责 auth / token / schema limits，router 以 model / LoRA 为 hard constraint，以 load / prefix / active adapter 为有界 soft score。
2. **容量：** 用真实模型算 weight / KV；用 benchmark 得 μ_{good} ； $\text{ceil}(120/(\mu_{\text{good}} \times p)) + \text{node-failure} + \text{rollout}$ 。
3. **隔离：** tenant quota、cache salt / identity、tool execution authorization、静态版本化 LoRA。普通请求不能提供任意 path。
4. **Structured output：** auto vs explicit backend 的 upgrade tradeoff；schema complexity / compile budget 与 semantic validation。
5. **Failure domains：** node / model group、router、artifact、adapter、retry；drain whole TP group。
6. **Observability：** TTFT / TPOT histograms、queue / lengths、KV / preemption、adapter state、grammar errors、quality / tool validation。
7. **Deployment：** immutable compatibility matrix、shadow、canary、expand、drain old、rollback；旧 cache 不跨不兼容版本复用。
8. **Alternative：** 长 context 独立 pool；单节点放不下才引入 PP / 跨节点；P/D 只有 transfer / queue 证明收益才选。
9. **Cost：** 每百万 good output tokens + warm reserve / rollout overlap。

Follow-ups

1. 一个 LoRA 热点占 70% 流量，router 怎么防热点又保 active-slot 命中？
2. auto backend 升级后从 xgrammar 换 guidance，如何发现？
3. 节点故障后所有请求 retry，怎样避免 queue collapse？
4. 20k 请求让短请求 TPOT 越线，先改什么？
5. 什么证据会让你改用 P/D disaggregation？

Strong signals

- 前 5 分钟只做澄清与假设排序，不抢答 GPU 数。
- 同时给 hard rejection criteria：quality、SLO、security、N+1。

- 复杂方案都有“何时才需要”的翻转条件。
- data plane 与 control plane 都画出。

Weak signals

- 直接假设 H100 / 70B / FP8并报卡数。
- 把 structured output 当工具授权，把 runtime LoRA API直接暴露给 tenant。
- HPA 只看 GPU util；没有 admission / retry budget。
- 没有旧版本 drain与 rollback。

本轮评分重点

Tradeoffs、Validation / Rollback 各需 ≥ 4 才是 senior-level。图漂亮但无容量、failure与quality gate不超过 3。

Round 5：事故响应（20 分钟）

面试官 Prompt

周一 10:05 发布新 vLLM image 到 10% canary。10:12 起全池 TPOT p99 从 55 ms 到 180 ms，TTFT p99 变化不大；waiting 增长、客户端 retry 翻三倍。Canary 日志出现 sampling backend fallback，但旧 Pod 也开始变慢。你是 incident commander，怎么做？

期望时间线

0-5 分钟：确认与止损

- 宣布 incident / owner / channel，冻结其他变更；确认 metric 口径与影响 tenant / model。
- 暂停 rollout；限制 retry budget / 启用 admission，防止重复工作放大。
- 若 canary 与变更高度相关且 rollback safe，停止新流量并 drain canary，不直接 kill streaming requests。

5-15 分钟：只读取证

- 对比 canary / old：image SHA、model / config、sampling backend、batch / output length、KV / preemption、graph / compile、GPU / collective。
- 解释旧 Pod 变慢可能是 retry / reroute放大，而非证明旧版本也有同一 kernel bug。
- 用 request attempts 区分 original load 与 retry load。

15-30 分钟：单变量验证

- rollout rollback + retry shedding 后，看 queue是否收敛、TPOT / goodput是否恢复。
- 在隔离环境复现 backend fallback；核准触发条件（seed、logprobs mode、platform、batch等），不在生产强制未知 backend。

恢复与事后

- 只有 SLO、error、queue、quality 与 retry 恢复并持续观察窗口后结束 incident。
- 保存 timeline、commands、dashboards、raw logs；根修增加兼容矩阵与 canary gate。

Follow-ups

1. 如果回滚 canary 后 TPOT 仍不降？
2. 如果没有当前 metric inventory，如何避免用错 PromQL？
3. 什么情况下应直接 shed low-priority traffic？
4. 如何证明没有请求 / KV / adapter state 泄漏？

Strong signals

- 先抑制 retry amplification，再讨论扩容。
- 区分 correlation 与 causation，用 old Pod 变慢解释 shared load effect。
- 每个 mutation 都有 approval、范围、预期、验证与 rollback。
- 恢复不是“图绿了”，还含 quality 与持续窗口。

Weak signals

- 立即把 max_num_seqs 调大或全池重启。
- 只看平均 GPU util；没有 original / retry 分离。
- 在证据不足时切全池 backend。
- 回滚后不 drain、不验证、不留证据。

本轮评分重点

Validation / Rollback=5 需要完整 incident lifecycle：detect、contain、diagnose、recover、verify、learn。

2. 总分解释与复盘动作

总分（125）	解释	下一步
105–125	能在证据边界内独立做设计 / 事故决策	补目标团队特定模型与平台
85–104	主干扎实，部分轮次缺 failure / quality / source depth	针对最低两维做一次重试
65–84	能讲概念，但容量、证据或回滚不稳定	回到 drills 与 production chapters
<65	术语记忆多于工程闭环	先做 capstone，留下真实证据包

不要用总分掩盖单项风险。把每轮最低的两个 rubric 维度写成下一次练习的具体动作，例如“从 /metrics inventory 选三条真实 signal”，而不是“多看源码”。

3. 项目经历叙事模板：不虚构结果

项目题不是要求一个“吞吐提升 3 倍”的英雄故事。使用下面模板，缺失数据明确标 未采集，不要补想象数字。

3.1 Context

- 业务 / 用户：[真实范围]
- 模型与版本：[model revision, tokenizer, vLLM SHA/image]
- workload：[input/output 分布、arrival、并发、feature]
- SLO / quality / security：[实际门禁；若当时没有，明确说没有]

3.2 Problem 与证据

- 现象：[TTFT/TPOT/error/quality/cost 的具体口径]
- 时间窗和对照组：[...]
- 证据链：[metrics/log/trace/profile/source/experiment]
- 排除项：[哪些假设被什么证据否定]
- 证据缺口：[当时未采集什么，怎样限制结论]

3.3 Decision

- Alternatives：[至少两个]
- 选择：[...]
- Tradeoff：[收益、代价、翻转条件]
- Rejection criteria：[何时不做 / 停止]
- 权限与协作：[谁批准、谁执行、谁 owner]

3.4 Validation / Rollout / Rollback

- 离线 / shadow / canary：[样本、时长、门禁]
- Failure injection：[...]
- Rollback unit：[immutable image/model/config]
- Drain / recovery evidence：[...]

3.5 Result

- 只写已记录结果：[raw artifact / dashboard / report]
- 同时报 quality、SLO、goodput 与 cost；不能只挑改善项。
- 若只能定性：写“在 [观察窗] 未再出现 [故障定义]”，不要杜撰百分比。
- Remaining risks：[...]

3.6 Learning

- 如果重做会先补什么 instrumentation / test？
- 哪个假设后来被证伪？
- 这次改变了哪些 release / runbook / ownership contract？

4. 自助模拟实验与失败证据

完整录制一次 90 分钟模拟，保存：题目版本、候选人假设表、白板 / 公式、评分表、争议证据链接与重试答案。第二次只重做最低两轮，比较的是 rubric 维度，不是语速。

失败证据：面试官不断提示才问 requirements；源码路径不存在；算式单位不闭合；系统设计无 alternative；incident 先 mutation 后取证；项目经历无法指出原始 artifact。每项都要转成下一次可观察动作。

生产取舍： 模拟面试追求可迁移的判断结构，不追求背完当前 SHA。对实现细节说“需核准”不是弱点，前提是能给出准确核准路径。

硬件验证状态： 未执行当前 SHA GPU run；本章不包含硬件实测性能结论。

小结

- 五轮分别测概念压缩、源码状态追踪、单位完整计算、需求优先设计和 fail-closed 事故响应。
- 统一 rubric 让“感觉不错”变成可复盘证据。
- senior 信号是能给 boundary、alternative、rejection criteria 和 rollback。
- 项目叙事宁可暴露证据缺口，也不要虚构漂亮数字。

自检

1. 任一轮得 5 分需要哪些比“答对”更多的证据？
2. 系统设计中哪三个问题如果未澄清，会让 GPU 数量计算无效？
3. 事故轮为什么要先限制 retry，再判断是否扩容？
4. 项目结果没有历史 dashboard 时，怎样诚实又有价值地回答？

参考答案

1. 5 分不仅要求结论正确，还要有机制、源码/组件入口、量化假设、验证命令、失败边界和回滚条件。能明确区分“已实测、静态复核、合理推断、尚未验证”，通常比背一个漂亮数字更重要。
2. 至少澄清模型与权重/KV 形状、真实 workload/到达率与长度分布、以及 SLO/故障余量。缺少任一项，GPU 数量、并行方式和 replica 公式都可能完全翻转。
3. retry 会放大负载并污染故障信号，先限制它能让真实到达率和资源状态稳定下来；随后再判断是否需要扩容。否则扩容可能只是给重复请求提供更多燃料。
4. 明确说没有历史 dashboard，不伪造实测；提供当前可复现的基线命令、指标 schema、实验矩阵和证据缺口。答案价值来自“下一步怎样测、什么结果会否决”，而不是虚构过去的曲线。

下一步

- 08-production-capstone.md：补一份真实、可复现的 project evidence package
- 07-incident-playbook.md：练 incident command 与恢复证据

- 12-upgrades-rollbacks-and-compatibility.md: 把 rollout 回答变成兼容矩阵

Source trail

- vllm/entrypoints/openai/、vllm/v1/engine/
- vllm/v1/core/sched/scheduler.py、vllm/v1/core/kv_cache_manager.py
- vllm/v1/worker/gpu_model_runner.py、vllm/v1/sample/
- vllm/v1/structured_output/、vllm/lora/
- vllm/distributed/parallel_state.py

01. 环境搭建与第一个 Demo

谁该读这一篇？ 第一次跑 vLLM、想把"读过的概念"变成"亲手验证"的学习者；准备给团队做演示的工程师。

前置阅读： 01-overview/00-prerequisites.md（环境与背景知识），01-overview/01-what-is-vllm.md（先知道 vLLM 是什么再装它）。

耗时： 约 15 分钟（不含模型下载与首次编译时间）。

学完能：

- 1. 用 uv 装好 vLLM 并跑通最小离线推理脚本
- 2. 启动 OpenAI 兼容的 vllm serve 并用 curl / openai SDK 发请求
- 3. 看流式输出并解释为什么 token 是一个个吐
- 4. 用自带 benchmark 跑出第一组吞吐 / 延迟数字
- 5. 遇到常见环境问题（OOM / NCCL / Compile 卡）能查表自救

当前源码复核（b23bd73）： 本地路线分 Linux NVIDIA GPU、当前受支持 CPU、源码 build、precompiled package 与 container；macOS 默认走远端 endpoint。所有 Python/安装命令都应在隔离的 uv 环境执行。先记录 vllm --version、平台/driver 与成功响应，再开始性能实验。

光读不练假把式。本节让你跑通最小例子并加日志观察内部行为。

1. 系统要求

路线	运行位置	成功证据	清理
NVIDIA GPU	受支持 Linux/CUDA 主机	version、driver、最小 generate/API	按 PID 停服务，保留实验记录
CPU	当前支持的 Linux CPU 平台	对应 wheel/build + 最小功能请求	删除专用 uv 环境/临时证据目录
源码开发	vLLM checkout 的 uv .venv	editable install + targeted tests	不混用系统 Python/pip
Container	NVIDIA container runtime/受支持镜像	image digest + health/models	按容器名 stop/remove
Remote endpoint	macOS/无本地受支持设备	health/models + 授权请求	删除临时 key/本地输出

- **硬件：** 至少一张 NVIDIA GPU（建议 ≥ 16GB 显存。无显卡可用 CPU 后端但很慢）
- **CUDA：** 12.1+（H100/H200 用 12.4+）
- **Python：** 3.10 / 3.11 / 3.12（vLLM 官方推荐 3.12）

如果只是看代码而不跑：可以跳过下面"安装"，直接 IDE 打开 vllm/ 目录读。

2. 推荐安装方式（用 uv）

vLLM 自己的 AGENTS.md 强制用 uv：

```
# 装 uv (如果还没装)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 在 vllm 仓库目录下
cd /Users/zjw/Documents/github-project/vllm
uv venv --python 3.12
source .venv/bin/activate

# 装预编译版 (不改 C++ 代码)
VLLM_USE_PRECOMPILED=1 uv pip install -e . --torch-backend=auto

# 或者从 PyPI 装最新 release (不需要 dev)
uv pip install vllm
```

如果只是读代码做笔记，不需要安装——但要跑实验、改代码、加日志，必须装。

3. 第一个 demo：离线推理

新建文件 hello_vllm.py：

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="facebook/opt-125m", # 最小模型，几秒下载
    gpu_memory_utilization=0.5, # 实验环境别吃满
    enforce_eager=True, # 关 CUDA Graph 加速启动
)

prompts = [
    "Hello, my name is",
    "The capital of France is",
    "Once upon a time",
]

params = SamplingParams(temperature=0.0, max_tokens=20)
outputs = llm.generate(prompts, params)

for o in outputs:
    print(f"PROMPT: {o.prompt!r}")
    print(f"OUTPUT: {o.outputs[0].text!r}\n")
```

运行：

```
.venv/bin/python hello_vllm.py
```

观察启动日志：你会看到这些关键行（理解每行对应代码里哪个动作）：

- Loading model weights took X GB → gpu_worker.py:load_model
- # GPU blocks: NNNN → profile run + KV 计算

- Capturing CUDA Graphs → 如果没有 enforce_eager，会 capture 多个 batch_size

4. 第二个 demo：在线服务

Terminal 1: 启动 server

```
vllm serve facebook/opt-125m --port 8000 --enforce-eager
```

Terminal 2: 发请求

```
curl -X POST http://localhost:8000/v1/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "facebook/opt-125m",  
  "prompt": "Hello, my name is",  
  "max_tokens": 20,  
  "temperature": 0  
}
```

5. 第三个 demo：流式

```
from openai import OpenAI
```

```
client = OpenAI(base_url="http://localhost:8000/v1", api_key="x")
```

```
stream = client.completions.create(  
    model="facebook/opt-125m",  
    prompt="Once upon a time",  
    max_tokens=50,  
    stream=True,  
)  
for chunk in stream:  
    print(chunk.choices[0].text, end="", flush=True)  
print()
```

观察 token 一个个吐出来——这就是 continuous batching 的用户视角。

6. Benchmark：自带的性能测试

当前 vLLM 用统一的 vllm bench 入口；先以锁定 commit 的 --help 核对参数：

```
# 吞吐 benchmark (离线)
vllm bench throughput \
  --model facebook/opt-125m \
  --num-prompts 100 \
  --enforce-eager

# 延迟 benchmark (单请求)
vllm bench latency \
  --model facebook/opt-125m \
  --batch-size 1 \
  --input-len 32 \
  --output-len 64

# 在线服务 benchmark
# 先启动 server, 再:
vllm bench serve \
  --model facebook/opt-125m \
  --num-prompts 100 \
  --request-rate 10
```

记下结果。后面对比开关 prefix caching、改 batch size 等的影响。

7. 常见环境问题

现象	原因	解决
ImportError: ... vllm_C ...	编译后端没装	VLLM_USE_PRECOMPILED=1 uv pip install -e .
OOM during profile run	gpu_memory_utilization 太高	降到 0.5
CUDA Graph capture error	显存不够	加 --enforce-eager
启动卡在 "Compiling..."	torch.compile 第一次很慢	等, 或 --compilation-config '{"level": 0}'
RuntimeError: NCCL ...	TP 通信失败	检查 NCCL_DEBUG=INFO、NVLink 状态

8. 推荐 IDE 设置

- VS Code / Cursor: 打开 vllm 仓库根目录
- 装 Python / Pylance 扩展
- 设置解释器为 .venv/bin/python
- 用 "Go to Definition" 跳转看类继承
- 用 "Find All References" 看一个类被哪些地方用

9. 小结

- 推荐用 uv + Python 3.12 装 vLLM; 只读代码不跑也可以, 但实验环节必须装。

- 离线推理用 LLM(...).generate(...), 在线服务用 vllm serve, 流式用 OpenAI SDK——三种入口都验证一遍最佳。
- 启动日志的 3 行 (权重大小、# GPU blocks、Capturing CUDA Graphs) 分别对应权重加载、profile run + KV 估算、CUDA Graph 录制。
- benchmarks/ 里 3 个脚本 (throughput / latency / serving) 覆盖几乎所有性能测试需求。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. "# GPU blocks: NNNN" 是哪个模块算的？受哪 3 个启动参数影响？

模块：vllm/v1/worker/gpu_worker.py 的 determine_available_memory() + vllm/v1/core/kv_cache_utils.py 的 get_num_blocks()。

算法：启动时跑一次 profile_run (用 max_num_batched_tokens 的 dummy batch 走一次 forward)，观察峰值显存占用，剩下的显存按"单 block 字节数"切成 num_blocks。

影响 N 的 3 个启动参数：

1. **--gpu-memory-utilization** (当前默认 0.92)：它是当前实例的 executor 预算，不是 nvidia-smi GPU-Util；可分配 block 还受权重、workspace、graph 和 profile 峰值影响
2. **--block-size** (默认 16)：单 block 字节数 = block_size × num_kv_heads × head_dim × layers × 2 (K+V) × dtype_bytes。
block_size 翻倍 → 单 block 翻倍 → num_blocks 减半
3. **--max-num-batched-tokens**：默认按 usage context 与硬件生成 (API server 常见 2048 或 8192)；值越大可能增加 profile/运行激活峰值，必须读取最终配置

其他间接影响：--dtype (FP16/BF16/FP8)、--kv-cache-dtype (fp8 KV 让单 block 减半)、--enforce-eager (不开 CUDA Graph 多省 1-2 GB)。

验证：去掉 enforce_eager=True 重跑，num_blocks 会因为 CUDA Graph capture 占用而**减少 ~5-10%**。

2. enforce_eager=True 去掉后启动时间增加多少？TPOT 下降？

启动时间：

- enforce_eager=True：跳过 CUDA Graph capture，启动快约 **30-60 秒**
- 去掉后：CUDA Graph 录制需要遍历 capture sizes (如 [1,2,4,8,16,32,...,256])，单个 capture 1-5 秒，总共增加 **30-120 秒**
- compile/CUDA Graph 最终模式由能力解析；首次编译/capture 可能显著增加启动时间，具体值必须实测

→ 生产部署要记录 cold/warm 启动分布，并通过 --compilation-config 的 cache 配置与持久卷复用产物；不要依赖当前源码中不存在的旧环境变量。

TPOT 是否下降：

- **下降**，典型 decode TPOT 从 30-50ms 降到 20-30ms (30-40% 提速)
- 原因：CUDA Graph 消 kernel launch overhead，每步省几 ms；torch.compile fusion 再省几 ms
- 但**首请求 TPOT 会变高** (先触发 compile)，第二个请求开始才稳定

实测对比 (Llama-3-8B + H100 + decode batch=8)：

- enforce_eager=True: TPOT median 38 ms
- 默认 (CG + compile): TPOT median 24 ms

→ 生产部署绝不要 enforce_eager, 调试时才用。

3. --request-rate 10 vs --request-rate 100 对比 TTFT 与 throughput, 解释拐点。

设系统单 GPU 极限吞吐约 50 req/s (看模型大小)。

--request-rate 10 (10 req/s):

- 远低于极限 → 请求几乎不排队
- TTFT p99 ~ 单请求 prefill 时长 = 50-200 ms
- throughput = 10 req/s (达到 offered load)
- KV cache utilization 低 (30-50%)

--request-rate 100 (100 req/s):

- 远超极限 → 请求大量堆积在 waiting queue
- TTFT p99 ~ "排队时长 + prefill 时长" = 几秒甚至几十秒
- throughput 被卡在 50 req/s (系统极限)
- KV utilization ~100%, 开始触发 preempt

拐点 (约 50 req/s 附近):

- 之下: TTFT 缓慢上升、throughput 线性跟随 rate
- 之上: TTFT 雪崩式增长 (排队论 M/M/1 公式: 等待时间 $\propto 1/(\mu - \lambda)$, λ 接近 μ 时趋无穷)、throughput 平台

生产意义:

- 用 benchmark_serving.py 找到拐点 → 单实例容量
- 设 HPA target = 拐点的 70% (留缓冲)
- 跨过拐点的告警条件: `vllm:num_requests_waiting > 阈值 + vllm:kv_cache_usage_perc > 0.9`

详见 08-production-deployment/04-autoscaling-and-capacity.md。

4. gpu_memory_utilization=0.95 触发 OOM, 错误栈定位到哪?

错误链路:

1. profile_run 时分配比 0.9 时更多的 activation buffer
2. capture CUDA Graph 时还要额外 1-2 GB
3. 总占用 $> 0.95 \times \text{HBM}$ → CUDA OOM

错误栈通常显示:

```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate ...  
File "vllm/v1/worker/gpu_model_runner.py", line XXXX, in execute_model  
    output = self.model.forward(...)  
File "vllm/model_executor/models/llama.py", line XXX, in forward  
    hidden_states = layer(positions, hidden_states, ...)  
File "vllm/model_executor/layers/attention.py", line XXX, in forward  
    attn_output = self.impl.forward(...)
```

真正的"分配出错点":

- 如果在 profile_run 时 OOM: vllm/v1/worker/gpu_worker.py::determine_available_memory()
- 如果在 CG capture 时 OOM: vllm/v1/worker/gpu_model_runner.py::capture_model()
- 如果在请求时 OOM (kvcache 容量不够): vllm/v1/core/block_pool.py::get_new_blocks() 抛 IndexError 而非 OOM——这是 vLLM 设计: kvcache 满了应该 preempt 而不是真 OOM

修法:

- 调回 --gpu-memory-utilization 0.9 或更小
- 看启动日志的 "available memory" 行确认 vLLM 估算
- 加 VLLM_LOGGING_LEVEL=DEBUG 看更详细的显存分配 trace

→ 调试技巧: 从 0.9 出发, 每次 +0.02 找到能稳跑的最大值。生产环境留 5-10% 安全边际。

下一步

- 下一节: 07-hands-on/02-trace-a-request.md (给一个请求加日志看完整内部行为)
- 想看源码: vllm/entrypoints/llm.py (LLM 类)、vllm/entrypoints/openai/api_server.py (serve 入口)、vllm/benchmarks/serve.py
- 想动手: 07-hands-on/03-mini-experiments.md (5 个独立小实验把直觉变数字)
- 想从生产视角理解: 08-production-deployment/01-deployment-architectures.md (从单机 demo 到生产部署)

02. Trace 一个请求：把内部跑明白

谁该读这一篇？ 想从"读过 Scheduler 源码"升级到"看着日志能解释每个 step 在干什么"的学习者；调试线上请求异常时需要 hook 进单步行为的工程师。

前置阅读： 07-hands-on/01-setup.md（环境已经能跑通最小 demo），03-code-walkthrough/02-scheduler.md（Scheduler 源码概览），03-code-walkthrough/03-kv-cache-manager.md（KV 分配函数定位）。

耗时： 约 20 分钟。

学完能：

1. 打开 `VLLM_LOGGING_LEVEL=DEBUG` 和 `stat logger` 实时看 step 状态
2. 在 Scheduler / KVCacheManager 里加 `print`，验证一个请求的每步决策
3. 用 `/metrics` 端点读 Prometheus 指标，把数字翻译成请求行为
4. 用 `torch.profiler` 看出 `prefill` / `decode` 各自的 kernel 时间分布

当前源码复核（b23bd73f540175f9e117eae5029cd7d8df63964）： 日志适合解释单次事件，`/metrics` 适合观察时间序列，`profiler` 用来归因 CPU/GPU 时间。当前关键指标是 `num_requests_running`、`num_requests_waiting`、`kv_cache_usage_perc`、`prefix cache hit/query counter`，以及 `TTFT`、`ITL`、`E2E histogram`；不要沿用旧文章里的 `num_running`、`num_waiting` 或预先算好的“命中率”指标。

让 vLLM 把它每一步在干什么打印出来。看一次比读 10 遍文档管用。

1. 准备：打开调试日志

vLLM 用 Python 标准 logging。最快开 debug：

```
VLLM_LOGGING_LEVEL=DEBUG .venv/bin/python hello_vllm.py 2>&1 | tee vllm.log
```

或代码内：

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

输出 5 万行不是夸张。先用最小例子（1 个 prompt、5 个 token）。

2. 打开 stat logger 看 step

vLLM 自带每 step 的统计输出。在创建 LLM 时加：


```
llm = LLM(  
    model="facebook/opt-125m",  
    enforce_eager=True,  
    disable_log_stats=False, # 默认是 False, 意为开启  
)
```

或 server: vllm serve ... --no-disable-log-stats

你会看到周期性的日志:

```
Engine: Avg prompt throughput: ... tokens/s  
Avg generation throughput: ... tokens/s  
Running: 3 reqs  Waiting: 0 reqs  
GPU KV cache usage: 12.3%  
Prefix cache hit rate: 0.0
```

每一项对应代码里某个 metric。

3. 手动加 print: 观察 Scheduler 决策

实验: 只在一次性实验分支中, 在 vllm/v1/core/sched/scheduler.py 的 schedule() 返回前临时加:

```
# scheduler_output 已在 schedule() 尾部构造; 插在  
# self._update_after_schedule(scheduler_output) 之前。  
print(  
    "[SCHED]",  
    f"running={len(self.running)}",  
    f"waiting={len(self.waiting)}",  
    f"total_tokens={scheduler_output.total_num_scheduled_tokens}",  
    f"scheduled={list(scheduler_output.num_scheduled_tokens)}",  
    f"preempted={sorted(scheduler_output.preempted_req_ids)}",  
)
```

跑:

```
llm = LLM("facebook/opt-125m", enforce_eager=True)  
llm.generate(["A"*100, "B"*200, "C"*50], SamplingParams(max_tokens=30))
```

观察 print, 你能看出:

- 第一个 step 是 prefill (total_tokens 等于所有 prompt 长度之和或 chunk)
- 后续 step 是 decode (total_tokens $\approx$ n_running)
- 短请求先 finish, 长请求继续

4. 观察 KV 分配

在 vllm/v1/core/kv_cache_manager.py 的 allocate_slots() 中给 coordinator.allocate_new_blocks(...) 之后设断点, 观察:

```
print(
    "[KV]",
    f"req={request.request_id}",
    f"new_tokens={num_new_tokens}",
    f"new_blocks={len(new_blocks)}",
    f"free_blocks={self.block_pool.get_num_free_blocks()}",
)
```

跑同样 demo，观察：

- 第一次 allocate 通常是大头（prompt 未命中的 token 需要 slot）
- 后续 decode 只有在已有 block 容量不足时才新增 block
- prefix cache、sliding window、KV connector 会改变分配量，不能只用 prompt 长度直接推断

实验结束后用 git diff 检查并只撤掉你自己的临时观测代码；不要把 print 带进生产构建。

5. 观察 Prefix Caching 命中

```
# 跑一遍：3 次完全相同的 prompt
prompts = ["System: be concise. User: hi"] * 3
```

如果 prefix caching 工作正常，从后续相同前缀请求开始：

- prefix_cache_hits_total 与 prefix_cache_queries_total 的增量可用于计算命中比例
- allocate_slots() 只为未命中部分和新生成 token 申请新 block

不要把 free_blocks “几乎不减” 当成唯一证据：并发请求释放、cache eviction 和尾部非整块 token 都会影响它。优先同时保存请求级日志与上述两个 counter 的前后快照。

6. 用 Prometheus 看 metric

启动 server 时 vLLM 默认在 /metrics 暴露：

```
vllm serve facebook/opt-125m --port 8000 --enforce-eager
# 另开窗口
curl http://localhost:8000/metrics | grep -E 'vllm:(num_|prefix_|kv_)' | head -20
```

关键指标（排查时可引用）：

- vllm:num_requests_waiting
- vllm:num_requests_running
- vllm:num_preemptions_total
- vllm:prefix_cache_hits_total / vllm:prefix_cache_queries_total
- vllm:kv_cache_usage_perc
- vllm:iteration_tokens_total (histogram)
- vllm:time_to_first_token_seconds (histogram)

- vllm:request_time_per_output_token_seconds (histogram)

7. 用 torch profiler 看 GPU

```
from torch.profiler import profile, ProfilerActivity

with profile(activities=[ProfilerActivity.CUDA, ProfilerActivity.CPU]) as prof:
    llm.generate(prompts, params)
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=20))
```

你会看到每个 CUDA kernel 的时间。预期看到：

- prefill 阶段：matmul（QKV、MLP）占大头
- decode 阶段：attention 和访存类 kernel 占比上升

8. 一份实验作业清单

跑完下面 5 个，你对 vLLM 有"肉身记忆"：

1. 空 vs 满 batch：单个 prompt vs 100 个并发 prompt，看 throughput / GPU util / KV usage 差距
2. 开关 prefix caching：相同 prompt 重复 10 次，对比 TTFT
3. 不同 max-num-batched-tokens：设 1024/4096/16384，看 TPOT 方差
4. enforce-eager 开关：开和关分别跑，看 startup time 和 decode latency
5. 加 fp8 KV：--kv-cache-dtype fp8，看 # GPU blocks 是否翻倍

每个实验记一条结论。这些记录后面做分享、排障复盘或方案评审时都能直接复用。

9. 小结

- 三档观察工具：VLLM_LOGGING_LEVEL=DEBUG（全量日志） / stat logger（周期摘要） / /metrics（Prometheus 指标），从粗到细。
- 在 Scheduler 的 schedule() 末尾和 KVCacheManager 的 allocate_slots 加 print，是最快验证调度 / KV 分配假设的方式。
- prefix caching 命中时 free_blocks 几乎不减，这是判断 cache 真在工作最直接的信号。
- torch.profiler 能区分 prefill（matmul-bound）和 decode（attention + 访存-bound）的 kernel 占比，是后续 profiling 章节的入门工具。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 同 prompt 跑 3 次（开 prefix caching），第 2 次 free_blocks 变化多少？为什么？

假设 prompt 占 N 个 block。

第 1 次：

- prefix 命中 0 个（首次，cache 为空）
- alloc N 个 block prefill + 后续 decode 累积
- free_blocks 减少 N + decode 增量

第 2 次（同 prompt）：

- prefix 命中 **N-1 个 block**（最后一个 block 因为只用了部分 token，下次匹配时 hash 不同——见 02-core-concepts/04-prefix-caching.md § 5.1）。注意：若是完美 block 对齐的 prompt，可能命中全部 N 个 block
- 命中的 block ref_cnt++ → **它们不从 free_queue 取出**（已被引用），所以 free_blocks 不减
- 只需要 alloc decode 累积的少量 block

→ **第 2 次的 free_blocks 比第 1 次少减约 N-1 个。**具体来说：

- 第 1 次结束 free_blocks = initial - (N + decode_blocks)
- 第 2 次 prefill 启动：free_blocks 减 1 或 2（最后一个非对齐 block + decode 第一个 block）

判断 cache 真在工作：第 2 次开始时如果 free_blocks 几乎不变，说明 prefix 命中了。如果还在减 N 个 block，要么命中率为 0（hash 没匹配），要么 cache 已被 evict。

2. 5k 长 prompt + 50 token 短 prompt 同时进，前 3 step 的 [SCHED] 输出 + chunked prefill 怎么救短请求？

假设 max_num_batched_tokens = 2048。

[SCHED] step 1:

num_scheduled_tokens = {req_short: 50, req_long: 1998}
total = 2048

解释: 短请求一次性 prefill 完 50 token；长请求 prefill 第 1 个 chunk (1998 token)

[SCHED] step 2:

num_scheduled_tokens = {req_short: 1, req_long: 2047}
total = 2048

解释: 短请求开始 decode 第 1 个新 token；长请求 prefill 第 2 个 chunk

[SCHED] step 3:

num_scheduled_tokens = {req_short: 1, req_long: 955}
total = 956

解释: 短请求 decode 第 2 个 token；长请求剩余 prefill (5000-1998-2047=955) 跑完
总 token < budget 因为长请求 prefill 已完

chunked prefill 救短请求的原理：

- 每步都给短请求 1 个 decode token slot（除了 step 1 它在 prefill）
- 长请求被切成 chunk（1998, 2047, 955），不是 5000 一次性
- 每步 GPU 时长稳定约 2048 token forward
- 短请求每步都能 decode → TPOT 平稳

反例（不开 chunked prefill）：

step 1: {req_short: 50, req_long: 5000} → total 5050, 单步几百 ms
这一步内 req_long 5000 token prefill 全跑
req_short 也"跟着算了 50 个 prefill token + 0 decode"
step 2: 全员 decode

短请求第一个 token 要等长请求 5000 token prefill 跑完——TTFT 飙到几百 ms。chunked prefill 把这压到 ~50 ms。

3. num_preemptions_total 与 kv_cache_usage_perc 因果关系。

因果链：

请求并发 ↑
→ KV 占用 ↑ (vllm:kv_cache_usage_perc 升高)
→ 接近 100% 时 alloc 失败
→ 触发 preempt (vllm:num_preemptions_total 累加)
→ 被踢请求 KV 释放
→ KV 占用回落

典型曲线：



preemptions 几乎只在 cache_usage > 0.9 时增长——存在强因果。如果 preemptions 涨而 cache_usage 不高，多半是 PRIORITY 模式下优先级反演（高优请求挤低优）。

告警规则：

```
# 容量预警
vllm:kv_cache_usage_perc > 0.9 for 5m → warning

# 抢占速率
rate(vllm:num_preemptions_total[5m]) > 0.5 → warning

# 两者关联表明真饿 KV
kv_cache_usage > 0.95 AND preempt_rate > 1 → critical (扩容)
```

详见 08-production-deployment/08-monitoring-cookbook.md。

4. torch.profiler prefill vs decode 排前 3 kernel + 差异。

Prefill 阶段排前 3（典型）：

- flash_attn_varlen_func 或 _flash_attn_v3_fwd_kernel — attention 主 kernel
- gemm_kernel（cuBLAS / cutlass）— Q/K/V/O 投影 + MLP 矩阵乘
- silu_and_mul_kernel（fused）— SwiGLU MLP 激活

特点：**compute-bound**——大 batch × 长 seq，算力打满。kernel 时长正比 token 数。

Decode 阶段排前 3（典型）：

- 1. paged_attention_v2_kernel 或 flash_attn_with_kvcache — 单 query attention，纯 memory-bound
- 2. gemm_kernel（GEMM）— Linear 投影，每步 batch_size × hidden_size 的矩阵乘
- 3. _rms_norm_kernel 或 reshape_and_cache_kernel — LayerNorm + KV 写入 cache

特点：**memory-bound**——每 token 1 个 query，算力闲着，主要读 weight 和 KV。kernel 时长几乎与 batch_size 无关（只要还在 memory-bound 区）。

关键差异：

维度	prefill	decode
attention kernel	flash_attn_varlen (大 Q × 长 K)	paged_attention (Q=1 × 长 K)
算力利用	80%+ (compute-bound)	30-50% (memory-bound)
HBM 带宽	中	接近峰值
GEMM 占比	50%+	30-40%
kernel 时长 / token	0.05 ms (Llama-7B)	0.2-0.5 ms

→ 这就是为什么 vLLM 设计**continuous batching + chunked prefill 把两者混跑**——decode 用算力空隙，prefill 用 GPU 满载，互补。

下一步

- 下一节：07-hands-on/03-mini-experiments.md（把"我看到了 X"升级成"我测量出 Y"）
- 想看源码：vllm/v1/core/sched/scheduler.py、vllm/v1/core/kv_cache_manager.py、vllm/v1/core/block_pool.py
- 想动手：本节末尾的"实验作业清单"5 条都做一遍
- 想从生产视角理解：08-production-deployment/05-slo-and-observability.md（同样的 metric 在 Grafana 里怎么搭面板）

03. 5 个 Mini 实验：把直觉变成数字

谁该读这一篇？ 想在技术分享里讲"我跑过 X，看到 Y"而不是只复述论文的工程师；带新人时希望给出可复现验证清单的 mentor。

前置阅读： 07-hands-on/01-setup.md（环境已装好），07-hands-on/02-trace-a-request.md（会用 stat logger 和 metric），02-core-concepts/04-prefix-caching.md + 02-core-concepts/05-chunked-prefill.md（理解实验背后概念）。

耗时： 约 30 分钟阅读 + 1-2 小时跑完 5 个实验。

学完能：

1. 复现 prefix caching 对 TTFT 的提升（量化数字）
2. 验证 max-num-batched-tokens 大小如何影响 TPOT 方差
3. 看到 FP8 KV cache 让 num_blocks 接近翻倍
4. 故意制造 KV 压力观察 Scheduler 的 preempt 行为
5. 测量 ngram 投机解码在不同 workload 下的吞吐收益

当前源码复核（b23bd73f540175f9e117eae5029cd7d8df63964）： 每次实验只改一个自变量；模型、commit、硬件、driver、输入/输出长度、并发和预热次数必须入报告。下面写的是待验证假设，不是通用性能承诺；所有数字都应来自你的原始结果。标有 NVIDIA/H100/A100 的实验不能拿无 GPU 的静态检查冒充实测。

读再多笔记不如自己测一次。下面 5 个实验都基于 facebook/opt-125m 或 Qwen2.5-0.5B（小模型省 GPU），但结论可以推广到大模型。跑完后，每个实验记一段 200 字以内的"我观察到 X，所以 Y"，后续做分享、复盘和方案评审都用得上。

实验 1：Prefix Caching 对 TTFT 的真实影响

目标

量化"重复 system prompt"在 prefix cache 开/关下的差距。

脚本

```
# experiment1_prefix_cache.py
import time
from vllm import LLM, SamplingParams

SYSTEM = "你是一个助手。" * 200    # 大约 1k tokens
USER_QUERIES = [f"用户问题 {i}" for i in range(10)]

def run(enable_prefix_caching: bool):
    llm = LLM(
        model="Qwen/Qwen2.5-0.5B-Instruct",
        enforce_eager=True,
        enable_prefix_caching=enable_prefix_caching,
        gpu_memory_utilization=0.5,
    )
    params = SamplingParams(max_tokens=50, temperature=0)
    prompts = [SYSTEM + q for q in USER_QUERIES]

    # 第一次跑：填 cache
    llm.generate(prompts[:1], params)

    # 计时：后续 9 次
    t0 = time.perf_counter()
    llm.generate(prompts[1:], params)
    dt = time.perf_counter() - t0
    print(f"prefix_caching={enable_prefix_caching}: 9 requests in {dt*1000:.0f}ms")

run(enable_prefix_caching=False)
run(enable_prefix_caching=True)
```

待验证假设

重复前缀跨越至少一个完整 cache block 时，开启 prefix caching 后续请求的 TTFT 应下降。下降幅度由前缀长度、block 对齐、并发和硬件决定；记录 p50/p95/p99 与 hit/query counter 增量，不预填百分比。

自测题

- 如果改成 temperature=0.7（每次输出不同），prefix cache 还能命中吗？
- 如果把 SYSTEM 改成 "你是一个助手。" * 200 + str(time.time())（每次微小不同），命中率会怎样？

参考答案

- **仍然可以命中。** Prefix cache 复用的是 prompt 对应的 KV，不是采样结果；temperature=0.7 只影响 prefill 完成后的 logits 采样和后续 decode，因此相同的完整前缀仍可复用。不同请求会从各自的 RNG 状态继续生成，输出不必相同。
- **命中率会显著下降，甚至接近 0。** 时间戳改变了前缀 token 序列；链式 block hash 从发生变化的 block 开始全部不同，后续 block 也无法命中。若时间戳位于第一个完整 block 内，整个 prompt 的可复用前缀都会丢失；只有它之前已经闭合的完整 block 仍可能命中。验证时应比较命中 token 数，而不只看请求级 hit/miss。

实验 2：max-num-batched-tokens 对 TPOT 抖动的影响

目标

观察 token budget 大小对单步延迟方差的影响。

脚本

```
# 把 max-num-batched-tokens 设小 (不易混入长 prefill)
```

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct \  
--enforce-eager \  
--gpu-memory-utilization 0.5 \  
--max-num-batched-tokens 1024 \  
--port 8001 &
```

```
# 设大 (一个长 prefill 可能占满整步)
```

```
vllm serve Qwen/Qwen2.5-0.5B-Instruct \  
--enforce-eager \  
--gpu-memory-utilization 0.5 \  
--max-num-batched-tokens 16384 \  
--port 8002 &
```

```
# experiment2_tpot.py: 同时发短请求 + 一个长请求
```

```
import asyncio
```

```
import time
```

```
import httpx
```

```
async def short_req(client, port):
```

```
    t0 = time.perf_counter()
```

```
    await client.post(f"http://localhost:{port}/v1/completions", json={
```

```
        "model": "Qwen/Qwen2.5-0.5B-Instruct",
```

```
        "prompt": "Hi, how are you?",
```

```
        "max_tokens": 30,
```

```
        "temperature": 0,
```

```
    })
```

```
    return time.perf_counter() - t0
```

```
async def long_req(client, port):
```

```
    await client.post(f"http://localhost:{port}/v1/completions", json={
```

```
        "model": "Qwen/Qwen2.5-0.5B-Instruct",
```

```
        "prompt": "重复内容 " * 4000, # ~8k tokens
```

```
        "max_tokens": 10,
```

```
        "temperature": 0,
```

```
    })
```

```
async def run(port):
```

```
    async with httpx.AsyncClient(timeout=60) as client:
```

```
        # 同时发 5 个短请求 + 1 个长请求
```

```
        tasks = [short_req(client, port) for _ in range(5)] + [long_req(client, port)]
```

```
        results = await asyncio.gather(*tasks)
```

```
        print(f"port {port}: short req latencies = {[f'{r*1000:.0f}ms' for r in results[:5]]}")
```

```
asyncio.run(run(8001)) # 小 budget
```

```
asyncio.run(run(8002)) # 大 budget
```

待验证假设

- 小 budget 会限制单步 prefill 工作量, 可能降低短请求尾延迟, 但会增加完成长 prompt 所需的 step 数。
- 大 budget 可能提高吞吐, 也可能让混合负载中的 decode 延迟更抖。用同一 workload 比较 TPOT p99, 不能用单次端到端耗时替代 TPOT。

自测题

- 哪种配置更适合 chatbot? 哪种更适合批量推理?

参考答案

- **Chatbot** 通常优先选择较小的 max-num-batched-tokens (例如从 2048/4096 起做压测), 让长 prompt 被切成更小的 chunk, 给正在 decode 的请求留下插入机会, 从而控制 TPOT/ITL p99。最终值要以固定到达率、长度分布和 SLO 的 A/B 结果决定。
- **批量推理** 更适合较大的 token budget (例如 8192 或更高), 减少调度轮数和 kernel launch 次数, 优先最大化 goodput。若离线任务完全不关心交互延迟, 可以继续增大, 但仍要监控显存峰值、OOM 和单步长尾。

实验 3: FP8 KV Cache 真的让 num_blocks 翻倍吗?

目标

验证启动日志里的 "# GPU blocks: NNNN" 是否符合预期。

脚本

```
# baseline
vllm serve Qwen/Qwen2.5-0.5B-Instruct \
  --enforce-eager \
  --gpu-memory-utilization 0.5 >baseline.log 2>&1 &
BASELINE_PID=$!

# 等待 health ready 后保存日志; 只终止刚刚记录的进程。
curl --retry 60 --retry-delay 2 --retry-connrefused -fsS \
  http://127.0.0.1:8000/health
grep -E "GPU blocks|KV cache" baseline.log
kill -TERM "$BASELINE_PID"
wait "$BASELINE_PID" || true

# fp8 KV
vllm serve Qwen/Qwen2.5-0.5B-Instruct \
  --enforce-eager \
  --gpu-memory-utilization 0.5 \
  --kv-cache-dtype fp8 2>&1 | grep -E "GPU blocks|KV cache"
```

待验证假设

在平台支持 FP8 KV 且其他参数相同的前提下, 单 block 的 KV 字节数下降, 可用 block 数通常增加, 但并不承诺恰好 2 倍。把两次启动日志、有效 KV 容量和平台支持信息一并保存。

自测题

- 显存利用率从 0.5 改成 0.9, num_blocks 是否线性放大?

参考答案

不保证严格线性。 可用 KV 显存大致随 `gpu_memory_utilization` 增长，但 `num_blocks` 还受模型权重、CUDA/NCCL 缓冲区、activation 峰值、CUDA Graph workspace、对齐和 profiling 保留量影响。把 0.5 调到 0.9 通常会增加 block 数，却不会承诺变成 1.8 倍；过高还会挤压运行态临时显存并提高 OOM 风险。应保存两次启动日志中的实际 KV bytes、`num_gpu_blocks` 和运行态峰值，按实测比值判断。

实验 4：观察 Scheduler 的 preempt 行为

目标

故意制造 KV 压力，观察 preempt 发生。

脚本

```
# experiment4_preempt.py
import asyncio
import httpx

# 启动 server 时强制小 KV:
# vllm serve Qwen/Qwen2.5-0.5B-Instruct \
# --enforce-eager --gpu-memory-utilization 0.2 \
# --max-num-seqs 256

async def long_req(client, i):
    return await client.post("http://localhost:8000/v1/completions", json={
        "model": "Qwen/Qwen2.5-0.5B-Instruct",
        "prompt": f"请求 {i} " + "上下文 " * 500,
        "max_tokens": 500,
        "temperature": 0.7,
    })

async def run():
    async with httpx.AsyncClient(timeout=120) as client:
        # 同时发 200 个长请求
        tasks = [long_req(client, i) for i in range(200)]
        await asyncio.gather(*tasks)

asyncio.run(run())
```

同时另开终端：

```
watch -n 0.5 'curl -s localhost:8000/metrics | grep -E "vllm:(num_preemptions_total|num_requests_running|num_requests_waiting)'"
```

待验证假设

- KV 压力足够大时，`vllm:num_preemptions_total` 增长。
- 同时保存 `num_requests_running`、`num_requests_waiting` 与 `kv_cache_usage_perc`；若没有抢占，先证明 workload 是否真的造成 KV 压力，不要把“没有复现”改写成成功。

自测题

- 如果改成 `--scheduling-policy priority` 并给一半请求高 `priority`，会有什么变化？

参考答案

高 `priority` 请求会在 `waiting` 队列排序和资源不足时的准入顺序上占优，但 **priority 不是带宽配额**。高优先级请求可能更早获得 KV block、较少等待，低优先级请求的 `queue time`、`preemption` 或 `starvation` 风险则会上升；如果高优先级流量持续不断，低优先级流量可能长期得不到服务。比较时要同时记录按 `priority` 分组的 TTFT、TPOT、完成率、`waiting` 时长和 `num_preemptions`，不能只看全局吞吐。生产上通常还要加租户配额、最大等待时间或多实例隔离。

实验 5：投机解码的接受率与吞吐

目标

量化投机解码在 `chat workload` 下的实际收益。

脚本

```
# baseline
vllm bench throughput \
  --model Qwen/Qwen2.5-0.5B-Instruct \
  --num-prompts 100 \
  --input-len 256 \
  --output-len 256 \
  --enforce-eager

# 用 ngram spec
vllm bench throughput \
  --model Qwen/Qwen2.5-0.5B-Instruct \
  --num-prompts 100 \
  --input-len 256 \
  --output-len 256 \
  --enforce-eager \
  --speculative-config '{"method": "ngram", "num_speculative_tokens": 3, "prompt_lookup_max": 4}'
```

待验证假设

- `ngram` 在可从 `prompt` 匹配后续 `token` 的 `workload` 中可能提高吞吐。
- 普通 `chat` 的接受率与收益取决于数据；必须同时报告 `acceptance`、`output throughput`、TTFT/TPOT 与基线，不能预填倍数。

自测题

- 如果你换成 `{"method": "eagle", ...}` + 一个 EAGLE 模型，效果怎样？
- 为什么 `batch_size` 越大，投机解码收益越小？

参考答案

- EAGLE 使用额外的 draft/预测头提出多个候选 token；如果目标模型、EAGLE 权重、tokenizer、dtype 和当前 vLLM 版本完全匹配，接受率可能高于 n-gram，长输出场景的 target forward 次数有机会下降。但它会增加 draft 计算、显存和启动配置复杂度，收益必须用 acceptance rate、accepted tokens/step、TTFT、TPOT、吞吐和拒绝路径开销一起验证；模型不匹配时可能无法启动或退回普通 decode。
- batch 越大时，target 模型越接近 compute-bound。投机一次多验证几个 token 会真实增加矩阵计算和 KV 写入量，draft、候选整理与校验开销也会按 batch 放大；小 batch 下 GPU 原本有空余，额外 token 近似“搭便车”，大 batch 下则可能抵消甚至超过减少的 decode step。应按并发分桶测收益，不能用 batch=1 的加速比外推生产峰值。

实验报告模板

每个实验跑完，把下面 3 句话填好放进笔记里：

实验 X：[一句话目标]
观察：[最关键的 1-2 个数字]
结论：[这告诉我们关于 vLLM 的什么]
踩坑：[过程中遇到的意外，怎么解决的]

下面只是**格式示例**，数字必须替换为你保存的原始结果，不能作为本章结论引用：

实验 1：Prefix Caching 对 TTFT 的影响 观察：基线 TTFT p50/p99 为 <实测>，开启后为 <实测>；hit/query 增量为 <实测>。结论：在本次 <模型、commit、硬件、workload> 下，结果支持/不支持“重复前缀降低 TTFT”的假设。踩坑：第一次没看到效果，发现是 enable_prefix_caching 写错；测试时还要排除 model load 时间。

这些数据会让理解变得非常扎实。**概念要讲，数字也要拿得出来。**

进阶实验（如果有大卡）

下面 4 个实验都需要更大的硬件（H100/A100 单卡或多卡），但每一个都能产出一篇**博客级技术内容**。

进阶实验 1：4 种量化方法的吞吐 / 延迟 / 精度三维对比

目标

用同一个模型、同一个 workload 跑 FP16 / FP8 / AWQ-INT4 / GPTQ-INT4 四组数据，给生产部署整理一份量化选型表。**特别要回答**：什么硬件该选什么量化、精度损失实际多少、是否走了 Marlin kernel。

硬件 / 软件要求

- **必须 H100 80GB**（H100 才有 FP8 硬件原生支持，A100 跑 FP8 是软件模拟，结论不可比）
- 准备 4 个 checkpoint：
- meta-llama/Llama-2-7b-hf （FP16 baseline）

- nm-testing/llama-2-7b-fp8 （FP8 dynamic, NeuralMagic 出品）
- TheBloke/Llama-2-7B-AWQ （AWQ INT4）
- TheBloke/Llama-2-7B-GPTQ （GPTQ INT4）
- 工具： benchmark_serving.py + lm-eval-harness （精度）

脚本

```
# 1. 性能 benchmark (4 组)
for fmt in fp16 fp8 awq gptq; do
  case $fmt in
    fp16) MODEL=meta-llama/Llama-2-7b-hf;    QUANT="" ;;
    fp8)  MODEL=nm-testing/llama-2-7b-fp8;    QUANT="--quantization fp8" ;;
    awq)  MODEL=TheBloke/Llama-2-7B-AWQ;     QUANT="--quantization awq_marlin" ;;
    gptq) MODEL=TheBloke/Llama-2-7B-GPTQ;    QUANT="--quantization gptq_marlin" ;;
  esac

  echo "=== $fmt ==="
  vllm serve $MODEL $QUANT \
    --gpu-memory-utilization 0.9 \
    --max-num-seqs 256 \
    --port 8000 &
  SERVER_PID=$!
  sleep 90 # 等 compile + warmup

  vllm bench serve \
    --model $MODEL \
    --dataset-name sharegpt \
    --dataset-path ShareGPT_V3_unfiltered_cleaned_split.json \
    --num-prompts 500 \
    --request-rate 10 \
    --result-filename results_${fmt}.json

  # 取关键 metric
  curl -s :8000/metrics | grep -E "vllm:(kv_cache_usage|num_requests_running)" \
    > runtime_${fmt}.txt

  kill $SERVER_PID && sleep 30
done

# 2. 精度 benchmark
for fmt in fp16 fp8 awq gptq; do
  lm_eval --model vllm \
    --model_args pretrained=$MODEL,quantization=$QUANT \
    --tasks wikitext,arc_easy,hellaswag \
    --batch_size auto \
    --output_path eval_${fmt}.json
done
```

结果记录（不得预填）

格式	显存	吞吐 tok/s	TTFT p99	TPOT p99	WikiText PPL	备注
FP16/BF16 baseline	<实测>	<实测>	<实测>	<实测>	<实测>	记录 dtype/backend
FP8	<实测>	<实测>	<实测>	<实测>	<实测>	记录平台原生支持
AWQ-INT4	<实测>	<实测>	<实测>	<实测>	<实测>	记录实际 kernel
GPTQ-INT4	<实测>	<实测>	<实测>	<实测>	<实测>	记录实际 kernel

必须核对

- 启动日志中的实际 quantization backend/kernel 是否与预期一致；fallback 必须记入结果。
- 权重 dtype 与 KV cache dtype 是两个自变量，不要在同一轮一起改。
- 吞吐变化可能来自每 token 计算、可容纳 batch、KV 容量或 kernel fallback；用 batch=1 和饱和负载拆因。
- PPL 不能替代业务质量评估；至少补一组与业务相似的 code/math/chat 任务。

自测题

1. 把 batch=1 跑一遍，量化的相对收益还在吗？为什么？
2. WikiText PPL 增长 2.7% 在你的业务能接受吗？怎么换算到 chatbot 用户体验？
3. AWQ 和 GPTQ 哪个 calibration 时间更长？checkpoint 文件大小差多少？
4. FP8 KV cache（--kv-cache-dtype fp8）和 FP8 权重 量化，能不能同时开？

参考答案

1. **batch=1 仍可能有收益，但来源不同。** 权重量化减少模型权重读取和显存占用，通常对 memory-bound decode 有帮助；但若 kernel fallback、反量化开销或 launch overhead 占主导，收益可能很小甚至变负。应固定模型、输入和采样参数，对比权重 dtype、实际量化 kernel、单 token 延迟和峰值显存。
2. PPL 上升 2.7% 是否可接受不能脱离业务定义。应在代表性 chat/code/math 集上比较拒答率、事实性、格式成功率、人工偏好或离线 reward，并建立“PPL 变化 → 业务指标变化”的回归，而不是把 2.7% 直接等同于用户体验下降 2.7%。若质量影响只出现在少数高价值请求，应采用路由或模型分层，而不是全量上线。
3. Calibration 时间取决于数据量、算法实现和硬件，不能只凭 AWQ/GPTQ 名称断言谁一定更慢；通常 GPTQ 的逐层 Hessian/误差优化更重，AWQ 的 activation-aware 搜索也可能成为瓶颈。checkpoint 大小主要由权重 bit 数、scale/zero-point、group size 和元数据决定，应对实际文件执行 du -h，并记录 safetensors 分片、索引和 tokenizer 是否计入。
4. **通常可以同时开启，但要分开验证。** FP8 权重量化影响权重存储/矩阵计算，--kv-cache-dtype fp8 影响运行态 KV；是否可组合取决于目标模型、量化 backend、硬件和当前版本的支持矩阵。先用 vllm serve --help、启动日志和最小正确性请求确认 backend，随后分别做权重-only、KV-only、两者同时的质量与性能 A/B。

可产出的博客角度

- "为什么 H100 用户应该默认上 FP8"——几乎零成本翻倍并发
- "AWQ vs GPTQ：4 步选型决策树"——配 ecosystem 兼容性矩阵
- "你以为开了量化就一定快？"——把 Marlin fallback 的坑讲透
- "量化的精度损失分布在哪？"——把 PPL 增长拆到不同任务（math、code、闲聊）

进阶实验 2：TP scaling 真实曲线 + 通信开销可视化

目标

量化 TP 的 **scaling efficiency**，看清"为什么 TP=8 不是 TP=1 的 8 倍吞吐"，并用 nsys 直观看到 AllReduce 占 forward 多少。

硬件 / 软件要求

- 4-8 卡 H100 / A100 **同机**（必须 NVLink，跨机 TP 是另一个故事）
- Model: **Llama-2-13B** (BF16 26GB，刚好单 H100 80G 能放，方便测 TP=1 作为 baseline)
- 工具: benchmark_serving.py + nsys profile

脚本

```
NUM_GPUS=$(nvidia-smi -L | wc -l)
for tp in 1 2 4 8; do
    [ $tp -gt $NUM_GPUS ] && continue

    vllm serve meta-llama/Llama-2-13b-hf \
        --tensor-parallel-size $tp \
        --max-num-seqs 256 \
        --port 8000 &
    SERVER_PID=$!
    sleep 120 # 等 NCCL init + compile

    # 多 QPS 曲线 (找到拐点)
    for qps in 5 10 20 50 100; do
        vllm bench serve \
            --model meta-llama/Llama-2-13b-hf \
            --dataset-name sharegpt \
            --num-prompts 300 \
            --request-rate $qps \
            --result-filename results_tp${tp}_qps${qps}.json
    done

    # nsys 看 NCCL 占比 (取 QPS=20 这个中等负载)
    nsys profile -t cuda,nvtx,osrt \
        --capture-range cudaProfilerApi --capture-range-end stop \
        -o profile_tp${tp} \
        .venv/bin/python -c "
from vllm import LLM, SamplingParams
import torch
llm = LLM('meta-llama/Llama-2-13b-hf', tensor_parallel_size=$tp, enforce_eager=True)
torch.cuda.cudart().cudaProfilerStart()
llm.generate(['Hello'] * 32, SamplingParams(max_tokens=100))
torch.cuda.cudart().cudaProfilerStop()
"

    kill $SERVER_PID && sleep 30
done

# 用 nsys-ui 或 ncu 打开 profile_tp*.nsys-rep, 看:
# - NCCL AllReduce kernel 占 timeline 的 % 比例
# - 不同 TP 下 forward 总时长
```

结果记录（不得预填）

TP	单卡吞吐 (tok/s)	总吞吐	scaling efficiency	AllReduce 占 forward
1	<实测>	<实测>	100%（基准）	<实测>
2	<实测>	<实测>	<计算>	<实测>
4	<实测>	<实测>	<计算>	<实测>
8	<实测>	<实测>	<计算>	<实测>

必须核对

- 先验证各 TP 使用相同模型、dtype、输入分布、并发和拓扑；记录 NVLink/PCIe 与 NCCL 版本。
- scaling efficiency 用 TP=n 总吞吐 / (n × TP=1 总吞吐) 计算，不凭经验填 “甜点”。
- 用 nsys 区分 GEMM、attention、NCCL 和空洞时间，再解释 TTFT/TPOT 变化。

自测题

- 同样 13B 模型，TP=4 和 4 实例 DP=4 哪个总吞吐高？什么场景下选哪个？
- 如果换 PCIe 替 NVLink，TP=8 还能跑吗？为什么效率会暴跌？算一下传输时长。
- TP scaling 曲线在 batch=1 和 batch=64 下形状有何不同？为什么 batch 越大效率反而越高？
- 给定 70B 模型 + 4 × A100 40G，能跑通吗？需要怎么配置？

参考答案

- 没有普适答案：TP=4 把一个副本切到 4 卡，适合单请求延迟、模型必须跨卡才能放下或需要共享大 KV 的场景；4 实例 DP=4 复制四份模型，若单卡能放下且请求足够独立，通常总吞吐和故障隔离更好。应在相同 QPS、长度分布和 SLO 下比较 goodput，而不是只比较某一个 batch 的 tok/s。
- 能运行但效率可能暴跌。** TP=8 需要频繁 collective；PCIe 的有效带宽和延迟远差于 NVLink。粗略估算传输时间应使用本机 nccl-tests 或 nvidia-smi topo -m 得到的实测带宽： $t \approx \text{bytes} / \text{effective_bandwidth} + \text{latency}$ ，不能直接套理论 PCIe 峰值；跨节点还要把 IB/RDMA、交换机拥塞和同步等待计入。
- batch=1 时通信延迟和 kernel launch 占比高，增加 TP 很快遇到收益递减；batch=64 时 GEMM 更饱和，计算量增大后 collective 可与计算重叠，曲线通常更平滑、scaling efficiency 更高。但 batch 过大也会受显存、KV 和通信带宽限制，因此仍需按 prefill/decode 分别测。
- 70B BF16 权重大约 140 GB，4 × 40 GB 只有 160 GB 原始容量，扣除 runtime、KV、通信和 workspace 后余量很紧；通常要使用 TP=4、低 gpu_memory_utilization 起步并启用权重量化（如 FP8/AWQ/GPTQ，取决于 backend），同时缩短 max_model_len、限制 max_num_seqs，必要时使用 CPU offload。能否稳定跑通必须以实际启动 profile、KV block 数和长上下文压测证明，不能只按容量相加。

可产出的博客角度

- "TP scaling 的 80% 法则"——每翻倍 TP 大约掉 15-20% 效率
- "用 nsys 找出 AllReduce 真实占比"——截图 + 逐段解读
- "什么时候停止增加 TP"——Pareto 前沿分析
- "TP vs DP：什么时候多实例反而更好"

进阶实验 3: AsyncScheduler 对 CPU overhead 的真实影响

目标

量化 V1 AsyncScheduler 在大 batch 下能省多少端到端时间，用 **flame graph** 直接看 **scheduler** 与 **forward** 是否真的 **overlap**。

硬件 / 软件要求

- 任意一卡 GPU（这个实验关注 CPU 行为，GPU 不是瓶颈）
- 工具：py-spy（采样 profiler）+ torch.profiler（kernel + Python 时间）

脚本

```
# 1. 启动服务
vllm serve Qwen/Qwen2.5-0.5B-Instruct \
  --max-num-seqs 256 \
  --max-num-batched-tokens 8192 \
  --port 8000 &
SERVER_PID=$!
sleep 60

# 2. 先查看进程树，人工确认属于 SERVER_PID 的 EngineCore 子进程；
# 不要用全机 pgrep 后取第一个结果。
ps -eo pid,ppid,cmd | grep -E "PID|$SERVER_PID|EngineCore"
export SCHEDULER_PID='<verified-engine-core-pid>'
ps -p "$SCHEDULER_PID" -o pid,ppid,cmd
sudo py-spy record \
  -o profile_async.svg \
  --pid "$SCHEDULER_PID" \
  --duration 30 \
  --rate 1000 \
  --threads &
PYSPY_PID=$!

# 3. 同时跑大 batch workload 制造 scheduler 压力
vllm bench serve \
  --model Qwen/Qwen2.5-0.5B-Instruct \
  --dataset-name sharegpt \
  --num-prompts 500 \
  --request-rate 50

wait "$PYSPY_PID"

# 4. 看 profile_async.svg:
# - 找 schedule() 调用栈，看它在 30s 里占多少
# - 看 update_from_output / preempt 等子函数分布
# - 看 scheduler thread 状态: busy% 多少
```

补一段 Python 端 torch.profiler 配合：

```
# experiment_async.py
import torch
from torch.profiler import profile, ProfilerActivity
from vllm import LLM, SamplingParams

llm = LLM(
    model="Qwen/Qwen2.5-0.5B-Instruct",
    enforce_eager=True, # 关掉 CUDA Graph, 让 CPU 时间清晰可见
    max_num_seqs=256,
    gpu_memory_utilization=0.9,
)

# 大 batch 制造调度压力
prompts = ["写一段 200 字介绍 vLLM 的文章。"] * 200
params = SamplingParams(max_tokens=200, temperature=0)

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True,
    with_stack=True,
) as prof:
    outs = llm.generate(prompts, params)

print(prof.key_averages().table(sort_by="cpu_time_total", row_limit=30))
prof.export_chrome_trace("trace_async.json")
# 用 chrome://tracing 或 https://ui.perfetto.dev 打开 trace_async.json
```

结果记录（不得预填）

在 Perfetto 时间线上标出 schedule 与 forward 是否重叠；下面只是读图示意，不代表实测比例：

线程 EngineCore-main  ← schedule() 间歇执行
线程 Worker-0 GPU  ← forward 间歇执行
^ schedule 与 forward 错开，几乎完全重叠

报告中填写 schedule CPU time、forward GPU time、重叠区间、吞吐与 TPOT p99。若没有建立关闭/开启 async scheduling 的可比基线，就只能描述 trace，不能声称收益来自 AsyncScheduler。

必须核对

- overlap 优化减少的是关键路径，不等于减少 schedule 的 CPU 总工作量。
- batch 大小、请求长度和采样功能都会改变调度成本；至少测低/中/高三个负载点。
- 关键源码入口是 vllm/v1/core/sched/async_scheduler.py；实验开关必须以当前 CLI/help 为准，不建议 patch V0 做对照。

自测题

1. 为什么 schedule 时间会随 batch 增大？给出 2 个主要原因（提示：preempt 候选 + KV alloc）
2. AsyncScheduler 如果 schedule 比 forward 还慢，发生什么？什么场景会出现？
3. 这套 producer-consumer overlap 在 OS 课里有什么对应概念？（double buffering / pipelining）
4. 当前 CLI 若提供受支持的 async scheduling 开关，应如何设计只改该开关的 A/B？若没有，为什么不能用 patch V0 冒充可比基线？

参考答案

1. schedule 时间会随 batch 增大，主要因为：一是要遍历更多 running/waiting 请求、更新 token 数并评估 preempt victim；二是要为更多序列计算 prefix/KV 命中、分配 block、更新持久 InputBatch 和 metadata。还可能叠加 grammar、LoRA、encoder 或 connector 的 per-request 工作。
2. 如果 schedule 比 forward 还慢，CPU 就会成为关键路径：GPU 完成一个 step 后等待下一轮 batch，吞吐下降、TPOT 抖动，队列也会堆积。长上下文、高并发、频繁 preemption、复杂 structured output/LoRA、CPU 过载或 Python profiling 往往会触发这种情况。应在 trace 中确认是 schedule 本身慢，还是 IPC、tokenizer 或日志造成的假象。
3. 这对应 OS/体系结构中的 **producer-consumer**、**double buffering** 和 **pipeline overlap**：producer 在 CPU 准备下一批调度结果，consumer 在 GPU 执行当前批；两块缓冲区交替使用，只有生产速度跟不上消费速度时才暴露气泡。
4. 若当前 CLI 提供受支持的 async 开关，A/B 只能改这一项，固定模型、硬件、环境变量、请求序列、warmup、采样与 benchmark 窗口，并同时记录 schedule CPU time、GPU step time、TTFT/TPOT 和吞吐。若没有该开关，不能把手工 patch V0 当基线：代码路径、调度语义、kernel capture 与 IPC 都变了，测到的是两个不同系统，无法把差异归因于 async scheduling。

可产出的博客角度

- "V1 AsyncScheduler 看似简单，实则 vLLM 性能跃迁的关键之一"
- "如何用 py-spy + flame graph 量化推理引擎的 CPU 开销"
- "AsyncScheduler 与 1F1B：推理与训练的调度差异"
- "schedule() 逐行注解 + profile 截图"——把 200 行 Python 讲透

进阶实验 4：Disaggregated prefill / decode 模拟

目标

在同一台机器跑两个 vLLM 实例（一个专 prefill、一个专 decode），用 **KV connector** 模拟 disaggregated 部署，量化 TTFT / TPOT 改善。

硬件 / 软件要求

- 2 个 GPU 同机（H100 或 A100 都行）
- NIXL 库（NVIDIA GPU-Direct）或回退到 shared memory connector
- 本教程锁定的源码 commit；KV connector 仍是实验性接口，升级后先重查示例与 schema

受控步骤

当前协议参考是 examples/disaggregated/disaggregated_prefill.sh，但该开发示例会安装依赖并广泛结束 Python 进程，**不要在共享机器原样运行**。先静态审计：

```
EXAMPLE=examples/disaggregated/disaggregated_prefill.sh
test -f "$EXAMPLE"
grep -nE 'kv_connector|kv_role|proxy|pip install|pgrep|pkill|kill -9' "$EXAMPLE"
```

把审计后的实验 launcher 放到独立目录，并满足以下验收条件后再跑：

1. producer、consumer、proxy、baseline 各自绑定独立端口与 GPU；模型和 connector schema 从**当前示例**复制，不凭旧版本记忆填写。
2. 依赖在实验前通过项目环境安装，launcher 内不执行 pip install。
3. 每个后台进程启动后立即保存 \$!；cleanup 只对这些 PID 先 TERM、等待、超时后再有选择地升级，不使用 pgrep python / pkill -f python。
4. 先分别验证 producer/consumer health，再通过 proxy 的公开端口做功能请求；不能把 producer 端口的响应当作完整 P/D 服务。
5. baseline 与 P/D 使用相同模型、dtype、总 GPU、workload 和 benchmark 参数；保存 proxy、两侧服务及 connector 日志。

完成 launcher 的安全审查后，用当前 CLI 对 proxy 和 baseline 分别运行 vllm bench serve。命令参数先用 vllm bench serve --help 在锁定 commit 的环境确认；下一章会给出完整 benchmark 方法。

结果记录（不得预填）

配置	TTFT p50	TTFT p99	TPOT p50	吞吐 tok/s
TP=2 单实例（baseline）	<实测>	<实测>	<实测>	<实测>
Disaggregated (P=1, D=1)	<实测>	<实测>	<实测>	<实测>

同时记录 KV transfer 成功/失败、传输字节、connector、链路、重试和两侧队列。若只请求 prefill 端口而没有完成当前 connector 示例要求的路由协议，该结果不是有效的 P/D 对照。

必须核对

- KV transfer 有固定和按字节增长的成本，必须在当前链路实测，不能套用通用 $\mu\text{s}/\text{GB}$ 。
- 分别测短/长 prompt、短/长输出与 prefix hit/miss，找出本环境的 break-even point。
- P/D 增加路由、connector 与局部故障面；性能结论必须和正确性、重试与容量结果一起交付。

自测题

1. KV transfer 跨 GPU 走的是哪条路径？PCIe vs NVLink vs RDMA？分别多快？
2. 如果 prefill 节点 OOM crash，请求会怎样？跟单实例的故障模式有何不同？
3. 多用户 chatbot 场景，prefix cache 命中率怎么影响 disaggregated 收益？
4. 算一下 100K token prompt 的 KV 大小（Llama-2-7B GQA）。NVLink 与 PCIe 跨卡分别传多久？

参考答案

1. 路径取决于拓扑和 connector：同卡是 device copy；同机 GPU 间通常走 NVLink，若拓扑不支持则回退 PCIe；跨节点通常走 RDMA/GPUDirect（具体由 NIXL、Mooncake 或其他 connector 决定）。延迟与带宽必须用当前机器的 nccl-tests、ib_write_bw/connector benchmark 和实际 payload 测量；不能把“NVLink/PCIe/RDMA 理论值”当端到端 KV transfer 时间，因为还包括 metadata、同步、注册和重试。
2. prefill 节点 OOM 时，它持有的请求和未完成 KV transfer 会失败；proxy/调度层应超时、取消或重试到健康的 prefill 节点，并防止 consumer 使用不完整 KV。与单实例相比，P/D 还有 transfer 状态、路由和幂等问题，可能出现“prefill 已算但 decode 未收到”的半完成状态，因此要有 request ID、generation/lease、超时和 fallback。

3. prefix hit 越高，prefill 计算量越少，P/D 分离可节省的 prefill FLOPs 也越少；固定的路由、连接和 KV transfer 开销可能超过收益。应按 hit/miss、prompt 长度和输出长度分别测 break-even：高命中短 prompt 往往适合单实例，长且低命中的 prompt 才更可能从 P/D 获益。
4. 以 Llama-2-7B 的 GQA 为例，需使用实际 num_kv_heads、head_dim、层数和 KV dtype 计算： $bytes = tokens \times layers \times 2(K/V) \times num_kv_heads \times head_dim \times dtype_bytes$ 。若按常见 32 层、32 个 KV head、128 head_dim、BF16 估算，100K token 约 $100000 \times 32 \times 2 \times 32 \times 128 \times 2 \approx 52.4$ GB（十进制）；若 checkpoint 使用更少 KV head，结果按比例下降。传输时间应按 $t \approx bytes /$ 实测有效带宽 + 固定延迟 计算，并分别用 NVLink、PCIe 的本机实测带宽代入；不能用单纯链路峰值替代 connector 的端到端测量。

可产出的博客角度

- "Disaggregated prefill：一次真实的部署成本/收益账"
- "vLLM KV connector 是什么——一个抽象的演化史"
- "什么 workload 适合 disaggregated，什么不适合"——决策矩阵
- "NIXL vs LMCache vs Mooncake：3 种 KV transfer 后端对比"

每个进阶实验都能产出一篇博客级别的内容。建议跑完后把"目标 / 数据 / 关键观察 / 反直觉的发现"4 段直接复制到 Notion 或博客里，再补 1-2 张 nsys / py-spy 截图就成稿。

小结

- 5 个实验分别验证了 prefix caching、token budget、FP8 KV、preempt、投机解码这 5 个 vLLM 核心机制。
- 实验脚本都用小模型（OPT-125m / Qwen-0.5B）就能跑，但结论对大模型一样适用。
- "目标 / 预期 / 自测题"三段式让每个实验都有"可复现 + 可推理"的双重价值。
- 实验报告模板（目标 / 观察 / 结论 / 踩坑）是技术分享和复盘里最有效的素材结构。

自检

不用照着原文复述，重点是把现象、机制、源码入口和取舍讲顺。

1. 实验 1 结论：prefix caching 在 chatbot 场景 TTFT 降低 X%。

典型结果（system prompt $\approx$ 500 token，用户 query $\approx$ 50 token，无 cache vs 有 cache）：

"在 system prompt = 500 token + 用户 query = 50 token 的 chatbot workload 下，开启 prefix caching 让**首次同 prompt 之后的请求 TTFT 从 ~180ms 降到 ~30ms，降低约 83%**。降幅与 (cached_tokens / total_prompt_tokens) 比值正相关——cached 占比越高，降幅越接近 95%。"

详细数字依赖：

- system prompt 越长 $\rightarrow$ 降幅越大（500-token prompt 降 83%，2000-token prompt 降 95%+）
- 命中率（连续请求间是否同 prefix）决定平均收益
- 第一次请求 TTFT 不变（cache 还没建立）

工程上可引申：这是 RAG / chatbot 场景的主要优化路径，比 quantization / spec decode 收益还大。

2. 实验 2 长请求让短请求多等多少 ms + max-num-batched-tokens 建议。

典型实验 2 设置：长请求 prompt=8192，短请求 prompt=50，同时进。

不开 chunked prefill（或 budget 极大）：

- 长请求一次 forward 跑 8192 token $\rightarrow$ $\sim 250\text{ms}$
- 短请求 TTFT = 排队 + 长请求 prefill 时长 $\approx$ **250 ms**

开 chunked prefill, max-num-batched-tokens=2048：

- 长请求被切成 4 chunk，每步 2048 token $\rightarrow$ $\sim 60\text{ms} / \text{step}$
- 短请求 step 1 就能并行 prefill（50 + 1998 = 2048 内） $\rightarrow$ TTFT $\approx$ **60 ms**

$\rightarrow$ 短请求多等的 ms = 250 - 60 = 190 ms（4 $\times$ 减少）。

max-num-batched-tokens 取值建议（参考表）：

- 4096-8192：通用 chatbot，平衡 TTFT 和 throughput
- 2048：TPOT 敏感（code completion），切更小 chunk
- 16384+：离线 batch，不在意 TPOT 抖动
- < 1024：极端 TPOT 要求（agent 多轮交互），但每 chunk 太小 schedule overhead 占比上升

调参逻辑：先按业务 SLO 选初值 $\rightarrow$ 跑 benchmark $\rightarrow$ 看 TPOT p99 是否达标 $\rightarrow$ 不达标减半 / 翻倍

3. 实验 3 baseline vs fp8 num_blocks 比值是否严格 2 $\times$ ？什么是固定项拉低比值？

答：不严格 2 $\times$ ，典型实测 1.6-1.8 $\times$ 。

理由：单 block 字节数严格减半（K/V 各 1 byte vs 2 byte），但可用 KV 显存不是全部显存：

```
total_hbm = 80 GB (H100)
minus 模型权重    (~16 GB Llama-3-8B BF16)
minus CUDA buffer  (~2 GB)
minus activation 预算 (~4-6 GB, 与 max_num_batched_tokens 相关)
minus CUDA Graph buffer (~1-2 GB)
minus profiling 留 5% (~4 GB)
= KV 可用          (~50 GB)

 $\rightarrow$  KV blocks (BF16) = 50 / single_block_bf16
 $\rightarrow$  KV blocks (FP8) = 50 / single_block_fp8 = 50 / (single_block_bf16 / 2) = 100 / single_block_bf16

ratio = 100 / 50 = 2.0  $\leftarrow$  理论值
```

但实际：FP8 KV 启用后 attention kernel 需要额外 k_scale, v_scale per layer $\rightarrow$ $\sim$ MB 级开销，影响微小。

真正拉低比值的因素：

- 模型权重始终占固定显存
- profile_run 用大 batch 测峰值，FP8 时激活仍 BF16/FP16 $\rightarrow$ 激活预算不变

- CUDA Graph capture sizes 与 dtype 无关

→ **比值近 2× 但不严格 2×**，这是预期行为，不是 bug。

4. 实验 4 看 num_preemptions_total 增长率，KV 再降一半会怎样？

实验 4 通常用 --gpu-memory-utilization 0.5 故意压缩 KV 空间观察 preempt。

当前 preempt 速率例：约 1-5 次/s（在并发饱和时）。

KV 再降一半：

- num_blocks 减半 → 同时能装的并发请求减半
- 同样并发流量下：
- kv_cache_usage_perc 持续 100%
- num_preemptions_total 增长率 **指数级上升**（不是 2×）
- 因为每次 preempt 释放出来的空间立刻被下一个排队请求占满，触发下一次 preempt
- **症状：**throughput 崩溃，TPOT 抖动剧烈（10×+），出现"刚 admit 又被踢"的振荡

临界条件：

- 单请求平均 KV 占用 = $K \text{ KB} / \text{请求}$
- 可用 KV = $M \text{ KB}$
- 稳定不 preempt 的并发上限 $\approx M / K$
- 一旦 offered concurrency > 这个上限，preempt 必然发生
- KV 减半 = 上限减半 = 提前 2× 触发 preempt 风暴

生产经验：留至少 30% KV 安全边际，宁可少几个并发也不要再在 KV 边缘走。

5. 实验 5 ngram spec decode 在什么 workload 收益最高？为什么大 batch 收益下降？

ngram 收益最高的 workload：

- **高重复性文本：**code completion（变量名、API、boilerplate 反复出现）
- **结构化输出：**JSON / XML / SQL（语法 token 高度可预测）
- **长输出 + 模板化：**写报告、邮件模板、文档生成
- **多轮对话同 system prompt：**前缀重复带来 ngram 库丰富

ngram 接受率典型范围：

- code/JSON：30-50%
- 通用对话：10-20%
- 创意写作（高随机性）：5-10%

为什么 batch_size 大时收益下降？

batch_size 与 GPU 状态的关系：

batch_size	GPU 状态	spec decode 收益
1-4	memory-bound（算力闲）	高 —— 多算 N 个 token 几乎免费
8-32	接近 compute-bound	中
64+	compute-bound（算力满）	低甚至负 —— 每个 token 都要算力

算力账：

- 小 batch：target 跑 1 token 的算力 = 跑 5 token 的算力（GPU 闲着，多算免费）
- 大 batch：target 跑 1 个新 token 的实际算力 $\approx$ batch_size 个 token 的工作量。多算 N 个 token 就是真实地多花 $N \times \text{batch_size}$ 的算力
- spec decode 加速比 $\approx 1 + \text{acceptance_rate} \times (N - 1) \times (1 - \text{overhead})$
- 大 batch 下 overhead 上升 + 算力成本上升 $\rightarrow$ 实际加速比可能 < 1

生产建议：

- 用 `vllm:num_requests_running` 当 batch_size 代理
- batch_size > 32 时自动关 spec decode（动态开关）
- 或换成 MTP（DeepSeek-V3 内置）—— 几乎零额外开销，大 batch 也能开

补充细节：EAGLE 在小 batch 下加速比 2-3 $\times$ ，大 batch 下可能反而拖慢 20%。决策权在监控数据，不在文档建议。

下一步

- 下一节：07-hands-on/04-profiling-and-debugging.md（从"我能跑出数字"升级成"我能定位 kernel-level 异常"）
- 想看源码：`vllm/benchmarks/throughput.py`、`vllm/benchmarks/serve.py`、`vllm/v1/core/sched/scheduler.py`
- 想动手：把每个实验改成"对比 2 个 vLLM 版本"——这能直接产出社区 PR 的回归测试材料
- 想从生产视角理解：08-production-deployment/05-slo-and-observability.md（同样的指标在生产怎么报警）

04. Profiling 与调试：怎么定位 vLLM 性能 / 正确性问题

谁该读这一篇？ 已经能跑 vLLM、需要在生产 / 压测中定位"为什么慢、为什么 hang、为什么结果错"的工程师；on-call 期间想 5 分钟内定位 incident 的 SRE。

前置阅读： 07-hands-on/02-trace-a-request.md（会用 stat logger 和 metric），07-hands-on/03-mini-experiments.md（能跑 benchmark 出基线数字），04-optimizations/03-cudagraph-and-compile.md（解释为何 enforce-eager 是 debug 第一步）。

耗时： 约 30 分钟。

学完能：

- 1. 写出 NCCL hang / 慢请求 / 结果异常三类问题的排查命令
- 2. 区分 torch.profiler 与 nsys 各自的最佳使用场景
- 3. 列出 6 个生产里最重要的调试环境变量
- 4. 用"宏观 metric → trace → profiler → kernel"的标准流程定位 TPOT p99 抖动

当前源码复核（b23bd73f540175f9e117eae5029cd7d8df63964）： 内建 profiling 由 ProfilerConfig / --profiler-config 配置，并通过 /start_profile、/stop_profile 控制。旧教程常见的 VLLM_TORCH_PROFILER_DIR、VLLM_NVTX_LOGGING、VLLM_USE_V1 不应再作为当前命令。nsys、py-spy、gdb 等工具还受平台、容器权限和安全策略约束。

知道概念 + 看过代码还不够。出问题怎么快速定位？本节列 6 套实战工具与场景：torch.profiler、Nsight Systems、py-spy、layerwise profiler、stat logger、benchmark 对比。

1. 工具速查表

场景	工具	入口
哪个 CUDA kernel 慢	Nsight Systems (nsys)	nsys profile vllm serve ...
哪一层 / 哪段代码慢	torch.profiler	with profile(...)
进程卡死	py-spy / gdb	py-spy dump --pid N
单层耗时分析	vllm/profiler/layerwise_profile.py	--otlp-traces-endpoint + custom hook
一次性快速看吞吐 / TTFT	benchmark_serving / benchmark_throughput	vllm bench serve / vllm bench throughput
整体调度时间线	vLLM stat logger	VLLM_LOG_STATS_INTERVAL=1
单步信息	OpenTelemetry trace	--otlp-traces-endpoint http://...

2. 关键调试开关

来自 vllm/envs.py:

```
VLLM_LOGGING_LEVEL=DEBUG      # 详细日志
VLLM_LOG_STATS_INTERVAL=1     # stat logger 每 1s 打一次
VLLM_LOG_BATCHSIZE_INTERVAL=1 # 单 step batch size 直方图
VLLM_TRACE_FUNCTION=1         # 打开 Python function trace (极慢, 仅 debug)
VLLM_LOG_MODEL_INSPECTION=true # 启动时打印模型结构

# CUDA / NCCL
NCCL_DEBUG=INFO
TORCH_NCCL_BLOCKING_WAIT=1     # 调试超时; 先核对当前 PyTorch/NCCL 文档
CUDA_LAUNCH_BLOCKING=1        # 同步执行, 错误堆栈准确 (极慢)
PYTORCH_NO_CUDA_MEMORY_CACHING=1 # 关 PyTorch caching alloc (debug only)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True # 缓解碎片
```

这些开关多数会改变时序或性能。一次只启用一个, 记录开启前后的现象。验证 attention backend 假设时使用当前 CLI 的 `--attention-backend / --attention-config`, 所选 backend 不受当前平台支持时应撤回而不是强行比较。

3. torch.profiler: 定位"哪个 op 慢"

最简单的代码层 profiler。

3.1 离线模式

```
from torch.profiler import profile, ProfilerActivity, record_function
from vllm import LLM, SamplingParams

llm = LLM("Qwen/Qwen2.5-0.5B-Instruct", enforce_eager=True, gpu_memory_utilization=0.5)
prompts = ["Hello"] * 8
params = SamplingParams(max_tokens=50, temperature=0)

# warmup
llm.generate(prompts, params)

# profile
with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True,
    with_stack=True,
) as prof:
    llm.generate(prompts, params)

# 看汇总
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=30))

# 导出 chrome trace 文件
prof.export_chrome_trace("vllm_trace.json")
# 用 chrome://tracing 或 Perfetto 打开
```

3.2 实时模式 (serve 内)